```
============================================================
  JH MANAGER - CODE SOURCE COMPLET
  Généré le 2026-10-06 04:34
============================================================

------------------------------------------------------------
FICHIER : ./AGENTS.md
------------------------------------------------------------
```

AGENTS.md

Project Context

This is a Base44 app repository. Treat it as user-owned application code, keep changes focused on the user's request, and preserve existing project conventions.

Start with `README.md` for local setup, environment variables, and publish workflow.

Base44 References

- CLI overview: https://docs.base44.com/developers/references/cli/get-started/overview.md
- Agent skills: https://docs.base44.com/developers/backend/overview/skills.md

If your agent supports Agent Skills, install or update Base44 skills before Base44-specific work:

```bash
npx skills add base44/skills
```

Key Files

- `src/`: frontend application source.
- `src/api/base44Client.js`: frontend Base44 SDK client.
- `vite.config.js`: Vite config and Base44 Vite plugin setup.
- `.env.local`: local-only environment values; never commit secrets.

Working Notes

- Use `base44 dev` as the default local development command when you need the local Base44 backend. It can run the backend and frontend together.
- When docs or code mention the frontend being started automatically, that usually means the Base44 project config includes `site.serveCommand`, for example `"serveCommand": "npm run dev"` in `base44/config.jsonc`.
- Use `npm run dev` only for frontend-only work against the hosted Base44 backend.
- Prefer the existing Base44 CLI workflow over adding new npm scripts for Base44-specific tasks.
- Reuse the existing SDK client and Vite plugin patterns before adding new Base44 integration paths.
- Run the relevant checks from `package.json` before finishing code changes.

```
------------------------------------------------------------
FICHIER : ./CLAUDE.md
------------------------------------------------------------
```

See AGENTS.md

Follow the instructions in `AGENTS.md`.

```
------------------------------------------------------------
FICHIER : ./README.md
------------------------------------------------------------
```

Base44 Project

Use this repository to run and edit the app locally, then publish changes back through Base44.

Any change pushed to the repo will also be reflected in the Base44 Builder.

Prerequisites

1. Clone the repository using the project's Git URL.
2. Navigate to the project directory.
3. Install dependencies: `npm install`.
4. Install the Base44 CLI: `npm install -g base44@latest`.
5. Install [Deno](https://docs.deno.com/runtime/getting_started/installation/) — the local Base44 backend runs on it.

Run `base44 --help` (or see the [CLI reference](https://docs.base44.com/developers/references/cli/commands/introduction)) for the full command surface.

Run Locally

Three commands, from the project root:

```bash
base44 login   # one-time per machine
base44 link    # one-time per clone
base44 dev     # local backend + frontend together
```

Open the frontend URL that `base44 dev` prints (typically `http://localhost:5173`).

Notes:

- **Every fresh clone needs `base44 link`.** It writes `base44/.app.jsonc` (the app-id pointer), which is deliberately gitignored. Your app id is in the Builder URL (`app.base44.com/apps/<id>/...`); `base44 link --help` shows the non-interactive flags.

- **`base44 dev` runs the frontend for you** (via `site.serveCommand` in this repo's `base44/config.jsonc`) — never run `npm run dev` yourself: alone it serves a UI with no backend behind it (`[base44] Proxy not enabled`, every `/api` call fails), and alongside `base44 dev` the second Vite silently takes the next port and you end up looking at the wrong one.
- **The app must be published at least once for the UI to load under `base44 dev`.** The frontend boots by fetching app settings from the hosted app; before the first publish that fails and every page redirects to login. The local API works regardless.
- Entities, functions, and auth run locally — entity data is **in-memory only**, wiped when `base44 dev` restarts. Everything else (Core integrations, OAuth login) is forwarded to your deployed app. Full breakdown: [Local development overview](https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview).

Frontend Only, Hosted Backend

To work on just the frontend against your app's live hosted backend:

```bash
base44 dev --remote
```

& þ I n t h i s m o d e w r i t e s g o t o y o u r a p p ' s * * p r o d u c t i o n d a t a * * — p l a i n

Publish Your Changes

After pushing your changes to git, open the Base44 dashboard and publish the app:

```bash
base44 dashboard open
```

This repo syncs to Base44 through git, so publish from the dashboard rather than `base44 deploy` — a CLI deploy ships your local tree directly, bypassing the sync, and the deployed state silently diverges from the repo.

Docs & Support

GitHub integration: https://docs.base44.com/developers/app-code/local-development/github

Local development: https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview

Support: https://app.base44.com/support

FICHIER : ./base44/agents/briefing.jsonc

```
{
  "description": "Briefing Quotidien JPM : synth\u00e8se de la journ\u00e9e.",
  "instructions": "Tu es le Briefing Quotidien de JPM. Tu synth\u00e9tises la journ\u00e9e de l'utilisateur : t\u00e2ches du jour, \u00e9ch\u00e9ances \u00e0 venir (selon la fen\u00eatre de rappel), d\u00e9penses r\u00e9centes et alertes budget. Tu es en lecture seule : tu lis Task, Expense et Preferences mais tu ne cr\u00e9es ni ne modifies rien. R\u00e9ponds dans la langue de l'utilisateur (fran\u00e7ais par d\u00e9faut, anglais si demand\u00e9). Sois concis, structur\u00e9 et motivant.",
  "tool_configs": [
    {
      "entity_name": "Task",
      "allowed_operations": [
        "read"
      ]
    },
    {
      "entity_name": "Expense",
      "allowed_operations": [
        "read"
      ]
    },
    {
      "entity_name": "Preferences",
      "allowed_operations": [
        "read"
      ]
    },
    {
      "function_name": "generateMedia",
      "description": "G\u00e9n\u00e8re un fichier multim\u00e9dia (image, audio ou vid\u00e9o) \u00e0 partir d'un texte descriptif. Param\u00e8tres: kind ('image'|'audio'|'video'), prompt (description), voice (optionnel: river/honey/sunny/storm/spark). Renvoie { file_url, kind }. Incluez l'URL dans la r\u00e9ponse : image en markdown ![](url), audio/vid\u00e9o sous forme de lien/lecteur. Utilisez 'video' avec parcimonie (co\u00fbte plus de cr\u00e9dits)."
    }
  ],
  "context_files": [],
  "app_user_connector_configs": [],
  "selected_skill_names": [],
  "selected_workspace_skill_ids": [],
  "model": "automatic",
  "whatsapp_greeting": null,
  "telegram_greeting": null,
  "line_greeting": null,
  "memory_config": {
    "enabled": true,
    "scope": "both",
    "include_other_conversation_context": false,
    "instructions": null
  },
  "voice_config": {
```

```
    "voice_id": null
  },
  "emails_settings": null,
  "allow_anonymous_access": false,
  "name": "briefing"
}

------------------------------------------------------------
FICHIER : ./base44/agents/finances.jsonc
------------------------------------------------------------
{
  "description": "Assistant Finances JPM : conseiller budget et gestion des d\u00e9penses.",
  "instructions": "Tu es l'Assistant Finances de JPM, une application de gestion personnelle (finances, t\u00e2ches, planning).
Tu aides l'utilisateur \u00e0 analyser ses d\u00e9penses, optimiser son budget et enregistrer ou ajuster des d\u00e9penses.
R\u00e9ponds dans la langue de l'utilisateur (fran\u00e7ais par d\u00e9faut, anglais si demand\u00e9). Sois concis, concret et
bienveillant. N'invente jamais de montants ni de donn\u00e9es : utilise uniquement les enregistrements r\u00e9els (Expense,
Preferences) de l'utilisateur. Donne des conseils chiffr\u00e9s et actionnables.",
  "tool_configs": [
    {
      "entity_name": "Expense",
      "allowed_operations": [
        "read",
        "create",
        "update"
      ]
    },
    {
      "entity_name": "Preferences",
      "allowed_operations": [
        "read",
        "update"
      ]
    },
    {
      "function_name": "generateMedia",
      "description": "G\u00e9n\u00e8re un fichier multim\u00e9dia (image, audio ou vid\u00e9o) \u00e0 partir d'un texte
descriptif. Param\u00e8tres: kind ('image'|'audio'|'video'), prompt (description), voice (optionnel:
river/honey/sunny/storm/spark). Renvoie { file_url, kind }. Incluez l'URL dans la r\u00e9ponse : image en markdown ![](url),
audio/vid\u00e9o sous forme de lien/lecteur. Utilisez 'video' avec parcimonie (co\u00fbte plus de cr\u00e9dits)."
    }
  ],
  "context_files": [],
  "app_user_connector_configs": [],
  "selected_skill_names": [],
  "selected_workspace_skill_ids": [],
  "model": "automatic",
  "whatsapp_greeting": null,
  "telegram_greeting": null,
  "line_greeting": null,
  "memory_config": {
    "enabled": true,
    "scope": "both",
    "include_other_conversation_context": false,
    "instructions": null
  },
  "voice_config": {
    "voice_id": null
  },
  "emails_settings": null,
  "allow_anonymous_access": false,
  "name": "finances"
}

------------------------------------------------------------
FICHIER : ./base44/agents/inventaire.jsonc
------------------------------------------------------------
{
  "description": "Assistant Inventaire : optimisation du stock et r\u00e9approvisionnement.",
  "instructions": "Tu es l'Assistant Inventaire de JM (JH Manager). Tu aides l'utilisateur \u00e0 optimiser son stock :
d\u00e9tecter les ruptures et les produits sous le seuil de r\u00e9approvisionnement, calculer la valeur du stock, identifier
les produits \u00e0 forte ou faible rotation, et proposer des r\u00e9approvisionnements chiffr\u00e9s. R\u00e9ponds dans la
langue de l'utilisateur. Sois concis et concret. N'invente jamais de donn\u00e9es : utilise uniquement les enregistrements
r\u00e9els de l'entit\u00e9 Product.",
  "tool_configs": [
    {
      "entity_name": "Product",
      "allowed_operations": [
        "read",
        "update"
      ]
    },
    {
      "function_name": "generateMedia",
      "description": "G\u00e9n\u00e8re un fichier multim\u00e9dia (image, audio ou vid\u00e9o) \u00e0 partir d'un texte
descriptif. Param\u00e8tres: kind ('image'|'audio'|'video'), prompt (description), voice (optionnel:
river/honey/sunny/storm/spark). Renvoie { file_url, kind }. Incluez l'URL dans la r\u00e9ponse : image en markdown ![](url),
audio/vid\u00e9o sous forme de lien/lecteur. Utilisez 'video' avec parcimonie (co\u00fbte plus de cr\u00e9dits)."
    }
  ],
  "context_files": [],
  "app_user_connector_configs": [],
  "selected_skill_names": [],
```

```
  "selected_workspace_skill_ids": [],
  "model": "automatic",
  "whatsapp_greeting": null,
  "telegram_greeting": null,
  "line_greeting": null,
  "memory_config": {
    "enabled": true,
    "scope": "both",
    "include_other_conversation_context": false,
    "instructions": null
  },
  "voice_config": {
    "voice_id": null
  },
  "emails_settings": null,
  "allow_anonymous_access": false,
  "name": "inventaire"
}

------------------------------------------------------------
FICHIER : ./base44/agents/jbm.jsonc
------------------------------------------------------------
{
  "description": "Assistant JBM : conseiller g\u00e9n\u00e9ral pour la gestion d'entreprise (ventes, stock, factures,
personnel).",
  "instructions": "Tu es l'Assistant JBM de JM (JH Manager), la partie gestion d'entreprise de l'application. Tu aides
l'utilisateur \u00e0 piloter son entreprise : analyser les ventes, surveiller le stock et les r\u00e9approvisionnements, suivre
les factures et leur paiement, et synth\u00e9tiser la masse salariale. R\u00e9ponds dans la langue de l'utilisateur
(fran\u00e7ais par d\u00e9faut, anglais si demand\u00e9). Sois concis, chiffr\u00e9 et actionnable. N'invente jamais de
donn\u00e9es : utilise uniquement les enregistrements r\u00e9els (Company, Product, Sale, Invoice, Employee) de l'utilisateur.
Signale les ruptures de stock (quantity <= reorder_level), les factures en retard, et les produits \u00e0 forte marge.",
  "tool_configs": [
    {
      "entity_name": "Company",
      "allowed_operations": [
        "read",
        "create",
        "update"
      ]
    },
    {
      "entity_name": "Product",
      "allowed_operations": [
        "read",
        "update"
      ]
    },
    {
      "entity_name": "Sale",
      "allowed_operations": [
        "read",
        "create"
      ]
    },
    {
      "entity_name": "Invoice",
      "allowed_operations": [
        "read",
        "update"
      ]
    },
    {
      "entity_name": "Employee",
      "allowed_operations": [
        "read"
      ]
    },
    {
      "function_name": "generateMedia",
      "description": "G\u00e9n\u00e8re un fichier multim\u00e9dia (image, audio ou vid\u00e9o) \u00e0 partir d'un texte
descriptif. Param\u00e8tres: kind ('image'|'audio'|'video'), prompt (description), voice (optionnel:
river/honey/sunny/storm/spark). Renvoie { file_url, kind }. Incluez l'URL dans la r\u00e9ponse : image en markdown ![](url),
audio/vid\u00e9o sous forme de lien/lecteur. Utilisez 'video' avec parcimonie (co\u00fbte plus de cr\u00e9dits)."
    }
  ],
  "context_files": [],
  "app_user_connector_configs": [],
  "selected_skill_names": [],
  "selected_workspace_skill_ids": [],
  "model": "automatic",
  "whatsapp_greeting": null,
  "telegram_greeting": null,
  "line_greeting": null,
  "memory_config": {
    "enabled": true,
    "scope": "both",
    "include_other_conversation_context": false,
    "instructions": null
  },
  "voice_config": {
    "voice_id": null
  },
```

```
  "emails_settings": null,
  "allow_anonymous_access": false,
  "name": "jbm"
}

------------------------------------------------------------
FICHIER : ./base44/agents/notifications.jsonc
------------------------------------------------------------
{
  "description": "Assistant Notifications : cr\u00e9e des notifications personnalis\u00e9es pour l'utilisateur.",
  "instructions": "Tu es l'assistant Notifications. L'utilisateur d\u00e9crit le type de notification qu'il souhaite recevoir
(rappel, conseil, alerte personnalis\u00e9e). Cr\u00e9e une notification via l'outil Notification avec un title clair, un
message concis, le type pertinent (rappel, conseil, personnalisee ou systeme) et, si utile, une due_date au format ISO.
R\u00e9ponds dans la langue de l'utilisateur (fran\u00e7ais par d\u00e9faut). Sois bref et confirme la cr\u00e9ation.",
  "tool_configs": [
    {
      "entity_name": "Notification",
      "allowed_operations": [
        "create",
        "read",
        "update"
      ]
    }
  ],
  "name": "notifications"
}

------------------------------------------------------------
FICHIER : ./base44/agents/rapports.jsonc
------------------------------------------------------------
{
  "description": "Assistant Rapports : g\u00e9n\u00e8re des fichiers de rapport (CSV) \u00e0 partir des donn\u00e9es de
l'utilisateur.",
  "instructions": "Tu es l'assistant Rapports. L'utilisateur demande un rapport (d\u00e9penses, stock, ventes ou t\u00e2ches).
Appelle la fonction generateReport avec le domaine ('finances', 'stock', 'ventes' ou 'taches'). Renvoie ensuite un lien
cliquable vers le fichier g\u00e9n\u00e9r\u00e9 (file_url) et indique son nom. R\u00e9ponds dans la langue de l'utilisateur
(fran\u00e7ais par d\u00e9faut). Sois bref.",
  "tool_configs": [
    {
      "function_name": "generateReport",
      "description": "G\u00e9n\u00e8re un rapport CSV (finances, stock, ventes, t\u00e2ches) et renvoie une URL de
t\u00e9l\u00e9chargement."
    },
    {
      "function_name": "generateMedia",
      "description": "G\u00e9n\u00e8re un fichier multim\u00e9dia (image, audio ou vid\u00e9o) \u00e0 partir d'un texte
descriptif. Param\u00e8tres: kind ('image'|'audio'|'video'), prompt (description), voice (optionnel:
river/honey/sunny/storm/spark). Renvoie { file_url, kind }. Incluez l'URL dans la r\u00e9ponse : image en markdown ![](url),
audio/vid\u00e9o sous forme de lien/lecteur. Utilisez 'video' avec parcimonie (co\u00fbte plus de cr\u00e9dits)."
    }
  ],
  "context_files": [],
  "app_user_connector_configs": [],
  "selected_skill_names": [],
  "selected_workspace_skill_ids": [],
  "model": "automatic",
  "whatsapp_greeting": null,
  "telegram_greeting": null,
  "line_greeting": null,
  "memory_config": {
    "enabled": true,
    "scope": "both",
    "include_other_conversation_context": false,
    "instructions": null
  },
  "voice_config": {
    "voice_id": null
  },
  "emails_settings": null,
  "allow_anonymous_access": false,
  "name": "rapports"
}

------------------------------------------------------------
FICHIER : ./base44/agents/rh.jsonc
------------------------------------------------------------
{
  "description": "Assistant RH : gestion du personnel et masse salariale.",
  "instructions": "Tu es l'Assistant RH de JM (JH Manager). Tu aides l'utilisateur \u00e0 g\u00e9rer son personnel :
synth\u00e9tiser la masse salariale, lister les employ\u00e9s actifs/inactifs, suivre les anniversaires d'embauche, et
sugg\u00e9rer des r\u00e9partitions de r\u00f4les. R\u00e9ponds dans la langue de l'utilisateur. Sois concis et concret.
N'invente jamais de donn\u00e9es : utilise uniquement les enregistrements r\u00e9els de l'entit\u00e9 Employee.",
  "tool_configs": [
    {
      "entity_name": "Employee",
      "allowed_operations": [
        "read",
        "update"
      ]
    },
    {
      "function_name": "generateMedia",
```

```
      "description": "G\u00e9n\u00e8re un fichier multim\u00e9dia (image, audio ou vid\u00e9o) \u00e0 partir d'un texte descriptif. Param\u00e8tres: kind ('image'|'audio'|'video'), prompt (description), voice (optionnel: river/honey/sunny/storm/spark). Renvoie { file_url, kind }. Incluez l'URL dans la r\u00e9ponse : image en markdown ![](url), audio/vid\u00e9o sous forme de lien/lecteur. Utilisez 'video' avec parcimonie (co\u00fbte plus de cr\u00e9dits)."
    }
  ],
  "context_files": [],
  "app_user_connector_configs": [],
  "selected_skill_names": [],
  "selected_workspace_skill_ids": [],
  "model": "automatic",
  "whatsapp_greeting": null,
  "telegram_greeting": null,
  "line_greeting": null,
  "memory_config": {
    "enabled": true,
    "scope": "both",
    "include_other_conversation_context": false,
    "instructions": null
  },
  "voice_config": {
    "voice_id": null
  },
  "emails_settings": null,
  "allow_anonymous_access": false,
  "name": "rh"
}

------------------------------------------------------------
FICHIER : ./base44/agents/taches.jsonc
------------------------------------------------------------
{
  "description": "Assistant T\u00e2ches JPM : planification et priorisation des t\u00e2ches.",
  "instructions": "Tu es l'Assistant T\u00e2ches de JPM, une application de gestion personnelle. Tu aides l'utilisateur \u00e0 planifier, prioriser, cr\u00e9er, modifier et organiser ses t\u00e2ches. D\u00e9tecte les conflits horaires entre t\u00e2ches et propose des cr\u00e9neaux. R\u00e9ponds dans la langue de l'utilisateur (fran\u00e7ais par d\u00e9faut, anglais si demand\u00e9). Sois concis et concret. N'invente jamais de donn\u00e9es : utilise uniquement les enregistrements r\u00e9els (Task, Preferences) de l'utilisateur.",
  "tool_configs": [
    {
      "entity_name": "Task",
      "allowed_operations": [
        "read",
        "create",
        "update",
        "delete"
      ]
    },
    {
      "entity_name": "Preferences",
      "allowed_operations": [
        "read"
      ]
    },
    {
      "function_name": "generateMedia",
      "description": "G\u00e9n\u00e8re un fichier multim\u00e9dia (image, audio ou vid\u00e9o) \u00e0 partir d'un texte descriptif. Param\u00e8tres: kind ('image'|'audio'|'video'), prompt (description), voice (optionnel: river/honey/sunny/storm/spark). Renvoie { file_url, kind }. Incluez l'URL dans la r\u00e9ponse : image en markdown ![](url), audio/vid\u00e9o sous forme de lien/lecteur. Utilisez 'video' avec parcimonie (co\u00fbte plus de cr\u00e9dits)."
    }
  ],
  "context_files": [],
  "app_user_connector_configs": [],
  "selected_skill_names": [],
  "selected_workspace_skill_ids": [],
  "model": "automatic",
  "whatsapp_greeting": null,
  "telegram_greeting": null,
  "line_greeting": null,
  "memory_config": {
    "enabled": true,
    "scope": "both",
    "include_other_conversation_context": false,
    "instructions": null
  },
  "voice_config": {
    "voice_id": null
  },
  "emails_settings": null,
  "allow_anonymous_access": false,
  "name": "taches"
}

------------------------------------------------------------
FICHIER : ./base44/config.jsonc
------------------------------------------------------------
{
  "name": "Untitled",
  "site": {
    "installCommand": "npm install",
    "buildCommand": "npm run build",
```

```
    "serveCommand": "npm run dev",
    "outputDirectory": "./dist"
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Affiliate.jsonc
------------------------------------------------------------
{
  "name": "Affiliate",
  "type": "object",
  "properties": {
    "referral_code": {
      "type": "string",
      "description": "Code unique d'affiliation"
    },
    "phone_number": {
      "type": "string",
      "description": "Num\u00e9ro Mobile Money pour le versement des commissions"
    },
    "provider": {
      "type": "string",
      "enum": [
        "airtel",
        "moov"
      ],
      "description": "Op\u00e9rateur Mobile Money"
    },
    "status": {
      "type": "string",
      "enum": [
        "actif",
        "inactif"
      ],
      "default": "actif"
    }
  },
  "required": [
    "referral_code",
    "phone_number",
    "provider"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/AttendanceRecord.jsonc
------------------------------------------------------------
{
  "name": "AttendanceRecord",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise"
    },
    "employee_id": {
      "type": "string",
      "description": "Employ\u00e9 concern\u00e9"
```

```
    },
    "employee_name": {
      "type": "string",
      "description": "Nom de l'employ\u00e9 (affichage)"
    },
    "type": {
      "type": "string",
      "enum": [
        "presence",
        "absence",
        "retard",
        "conge",
        "discipline"
      ],
      "default": "presence",
      "description": "Type d'enregistrement"
    },
    "date": {
      "type": "string",
      "format": "date",
      "description": "Date"
    },
    "note": {
      "type": "string",
      "description": "Comportement / motif / note libre"
    },
    "severity": {
      "type": "string",
      "enum": [
        "info",
        "avertissement",
        "sanction"
      ],
      "default": "info",
      "description": "Gravit\u00e9 (discipline)"
    }
  },
  "required": [
    "employee_id",
    "type",
    "date"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/CalcDefinition.jsonc
------------------------------------------------------------
{
  "name": "CalcDefinition",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Identifiant alg\u00e9brique (ex: CA, prix_unitaire)"
    },
    "label": {
      "type": "string",
      "description": "Libell\u00e9 affich\u00e9"
    },
    "kind": {
      "type": "string",
      "enum": [
        "grandeur",
        "fonction",
        "gain"
      ],
      "default": "grandeur",
      "description": "Grandeur (valeur), fonction (f(variable)) ou gain (formule de gains/facturation)"
    },
    "expression": {
      "type": "string",
      "description": "Formule d'une grandeur (r\u00e9f\u00e9rence d'autres grandeurs), expression f(variable) d'une fonction, ou
formule d'un gain"
    },
    "unit": {
      "type": "string",
      "description": "Unit\u00e9 (XAF, kg, \u2026)"
    },
```

```
    "value": {
      "type": "number",
      "description": "Valeur saisie pour une grandeur manuelle sans formule"
    },
    "source_type": {
      "type": "string",
      "enum": [
        "manual",
        "finance",
        "stock",
        "ventes",
        "factures"
      ],
      "default": "manual",
      "description": "Source de la grandeur"
    },
    "source_key": {
      "type": "string",
      "description": "Cl\u00e9 de l'indicateur li\u00e9 (si source non manuelle)"
    },
    "gain_recurring": {
      "type": "boolean",
      "default": false,
      "description": "Gain r\u00e9current (p\u00e9riodique) ou ponctuel"
    },
    "gain_period": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "trimestre",
        "semestre",
        "annee"
      ],
      "default": "mois",
      "description": "P\u00e9riodicit\u00e9 du gain r\u00e9current"
    },
    "plot_variable": {
      "type": "string",
      "default": "x",
      "description": "Variable de la fonction"
    },
    "plot_x_min": {
      "type": "number",
      "default": -10,
      "description": "Borne min du trac\u00e9"
    },
    "plot_x_max": {
      "type": "number",
      "default": 10,
      "description": "Borne max du trac\u00e9"
    },
    "plot_steps": {
      "type": "number",
      "default": 100,
      "description": "Nombre de points \u00e9chantillonn\u00e9s"
    },
    "plot_type": {
      "type": "string",
      "enum": [
        "courbe",
        "aire",
        "histogramme",
        "nuage"
      ],
      "default": "courbe",
      "description": "Type de repr\u00e9sentation graphique"
    }
  },
  "required": [
    "name",
    "kind"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Category.jsonc
```

```
------------------------------------------------------------
{
  "name": "Category",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Nom de la cat\u00e9gorie"
    },
    "scope": {
      "type": "string",
      "enum": [
        "jpm",
        "jbm"
      ],
      "description": "Espace concern\u00e9"
    },
    "domain": {
      "type": "string",
      "enum": [
        "depense",
        "produit",
        "vente",
        "tache",
        "evenement"
      ],
      "description": "Domaine cat\u00e9goris\u00e9"
    },
    "company_id": {
      "type": "string",
      "description": "Entreprise li\u00e9e (JBM). Vide = cat\u00e9gorie commune \u00e0 toutes les entreprises."
    }
  },
  "required": [
    "name",
    "scope",
    "domain"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Client.jsonc
------------------------------------------------------------
{
  "name": "Client",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise / \u00e9tablissement"
    },
    "name": {
      "type": "string",
      "description": "Nom du client"
    },
    "type": {
      "type": "string",
      "enum": [
        "particulier",
        "entreprise"
      ],
      "default": "particulier",
      "description": "Type de client"
    },
    "phone": {
      "type": "string",
      "description": "T\u00e9l\u00e9phone"
    },
    "email": {
      "type": "string",
      "description": "Email"
    },
    "address": {
      "type": "string",
      "description": "Adresse"
    },
    "notes": {
      "type": "string",
```

```
      "description": "Notes"
    }
  },
  "required": [
    "name"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Collaboration.jsonc
------------------------------------------------------------
{
  "name": "Collaboration",
  "type": "object",
  "properties": {
    "members": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "description": "Identifiants des deux collaborateurs [userA, userB]"
    },
    "partner_name": {
      "type": "string",
      "description": "Nom de l'autre collaborateur (affichage)"
    }
  },
  "required": [
    "members"
  ],
  "rls": {
    "read": {
      "data.members": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "data.members": {
        "$in": [
          "{{user.id}}"
        ]
      }
    },
    "delete": {
      "data.members": {
        "$in": [
          "{{user.id}}"
        ]
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/CollaborationRequest.jsonc
------------------------------------------------------------
{
  "name": "CollaborationRequest",
  "type": "object",
  "properties": {
    "to_user_id": {
      "type": "string",
      "description": "Utilisateur cibl\u00e9 par la demande"
    },
    "to_name": {
      "type": "string",
      "description": "Nom du destinataire (affichage)"
    },
    "status": {
      "type": "string",
      "enum": [
        "pending",
        "accepted",
        "rejected"
      ],
      "default": "pending",
      "description": "Statut de la demande"
```

```
    }
  },
  "required": [
    "to_user_id"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.to_user_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.to_user_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Collaborator.jsonc
------------------------------------------------------------
{
  "name": "Collaborator",
  "type": "object",
  "properties": {
    "display_name": {
      "type": "string",
      "description": "Nom affich\u00e9 et recherchable par les autres utilisateurs"
    }
  },
  "required": [
    "display_name"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "user_condition": {
            "role": "user"
          }
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
```

```
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Commission.jsonc
------------------------------------------------------------
{
  "name": "Commission",
  "type": "object",
  "properties": {
    "affiliate_id": {
      "type": "string",
      "description": "Identifiant de l'affili\u00e9 parrain"
    },
    "affiliate_owner_id": {
      "type": "string",
      "description": "User id du parrain (ancrage RLS)"
    },
    "referred_user_email": {
      "type": "string",
      "description": "Email du filleul ayant pay\u00e9"
    },
    "source_amount": {
      "type": "number",
      "description": "Montant de l'abonnement parrain\u00e9 (XAF)"
    },
    "amount": {
      "type": "number",
      "description": "Commission vers\u00e9e (20 % du montant source)"
    },
    "account_type": {
      "type": "string",
      "enum": [
        "particulier",
        "entreprise"
      ]
    },
    "paid_at": {
      "type": "string",
      "format": "date-time",
      "description": "Date de la commission"
    }
  },
  "required": [
    "affiliate_id",
    "amount",
    "source_amount"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "data.affiliate_owner_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "user_condition": {
        "role": "admin"
      }
    },
    "update": {
      "user_condition": {
        "role": "admin"
      }
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Company.jsonc
------------------------------------------------------------
{
  "name": "Company",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Nom de l'entreprise"
    },
    "business_type": {
```

```
      "type": "string",
      "description": "Type d'entreprise (commerce, restauration, services, sant\u00e9...)"
    },
    "location": {
      "type": "string",
      "description": "Lieu / ville de l'entreprise"
    },
    "sector": {
      "type": "string",
      "description": "Secteur d'activit\u00e9"
    },
    "address": {
      "type": "string",
      "description": "Adresse"
    },
    "phone": {
      "type": "string",
      "description": "T\u00e9l\u00e9phone"
    },
    "email": {
      "type": "string",
      "description": "Email"
    },
    "tax_id": {
      "type": "string",
      "description": "Num\u00e9ro fiscal / RC"
    }
  },
  "required": [
    "name",
    "business_type",
    "location"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "user_condition": {
            "role": "user"
          }
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/DeletedAccount.jsonc
------------------------------------------------------------
{
  "name": "DeletedAccount",
  "type": "object",
  "properties": {
    "email": {
      "type": "string",
      "description": "Email (minuscules) du compte supprim\u00e9 par un admin"
    },
    "full_name": {
      "type": "string",
      "description": "Nom affich\u00e9 au moment de la suppression"
    },
    "reason": {
      "type": "string",
      "description": "Motif de la suppression"
    },
    "deleted_by": {
      "type": "string",
      "description": "ID de l'admin ayant supprim\u00e9 le compte"
    }
  },
  "required": [
    "email"
  ],
  "rls": {
    "read": {
      "user_condition": {
        "role": "admin"
      }
```

```
    },
    "create": {
      "user_condition": {
        "role": "admin"
      }
    },
    "update": {
      "user_condition": {
        "role": "admin"
      }
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Employee.jsonc
------------------------------------------------------------
{
  "name": "Employee",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise"
    },
    "full_name": {
      "type": "string",
      "description": "Nom complet"
    },
    "role": {
      "type": "string",
      "description": "Poste / fonction"
    },
    "headcount": {
      "type": "number",
      "default": 1,
      "description": "Nombre de personnes \u00e0 ce poste"
    },
    "salary": {
      "type": "number",
      "description": "Salaire"
    },
    "salary_periodicity": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "trimestre",
        "semestre",
        "annee"
      ],
      "default": "mois",
      "description": "P\u00e9riodicit\u00e9 du salaire"
    },
    "bonus": {
      "type": "number",
      "default": 0,
      "description": "Prime (ponctuelle ou p\u00e9riodique)"
    },
    "bonus_type": {
      "type": "string",
      "enum": [
        "ponctuel",
        "periodique"
      ],
      "default": "ponctuel",
      "description": "Type de prime"
    },
    "bonus_periodicity": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "trimestre",
        "semestre",
        "annee"
      ],
      "description": "P\u00e9riodicit\u00e9 de la prime (si periodique)"
    },
    "phone": {
      "type": "string",
      "description": "T\u00e9l\u00e9phone"
    },
    "email": {
      "type": "string",
```

```
      "description": "Email"
    },
    "hire_date": {
      "type": "string",
      "format": "date"
    },
    "status": {
      "type": "string",
      "enum": [
        "actif",
        "inactif"
      ],
      "default": "actif"
    }
  },
  "required": [
    "full_name"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Event.jsonc
------------------------------------------------------------
{
  "name": "Event",
  "type": "object",
  "properties": {
    "title": {
      "type": "string",
      "description": "Nom de l'\u00e9v\u00e9nement"
    },
    "description": {
      "type": "string",
      "description": "Description / d\u00e9tails"
    },
    "company_id": {
      "type": "string",
      "description": "Entreprise / \u00e9tablissement (JBM / \u00c9cole). Vide en espace personnel."
    },
    "start": {
      "type": "string",
      "format": "date-time",
      "description": "Date et heure de d\u00e9but"
    },
    "end": {
      "type": "string",
      "format": "date-time",
      "description": "Date et heure de fin"
    },
    "location": {
      "type": "string",
      "description": "Lieu"
    },
    "category": {
      "type": "string",
      "enum": [
        "reunion",
        "rendez_vous",
        "formation",
        "ceremonie",
        "autre"
      ],
      "default": "autre",
      "description": "Cat\u00e9gorie d'\u00e9v\u00e9nement"
    },
    "priority": {
      "type": "string",
      "enum": [
        "basse",
        "normale",
        "haute"
      ],
      "default": "normale",
      "description": "Priorit\u00e9"
    },
    "shared_context_type": {
      "type": "string",
      "enum": [
```

```
        "collaboration",
        "group"
      ],
      "description": "Contexte partag\u00e9 (vide = \u00e9v\u00e9nement personnel)"
    },
    "shared_context_id": {
      "type": "string",
      "description": "Identifiant de la collaboration ou du groupe"
    },
    "participants": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "default": [],
      "description": "Membres pouvant voir/g\u00e9rer l'\u00e9v\u00e9nement partag\u00e9 (d\u00e9normalis\u00e9 pour RLS)"
    }
  },
  "required": [
    "title",
    "start",
    "end"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Expense.jsonc
------------------------------------------------------------
{
  "name": "Expense",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Nom de la d\u00e9pense"
    },
    "quantity": {
      "type": "number",
      "default": 1
    },
    "unit_cost": {
      "type": "number",
      "description": "Co\u00fbt unitaire"
    },
    "category": {
      "type": "string",
      "description": "Cat\u00e9gorie (alimentation, transport, etc.)"
    },
    "date": {
      "type": "string",
      "format": "date"
    },
    "shared_context_type": {
      "type": "string",
      "enum": [
        "collaboration",
        "group"
```

```
      ],
      "description": "Contexte partag\u00e9 (vide = d\u00e9pense personnelle)"
    },
    "shared_context_id": {
      "type": "string",
      "description": "Identifiant de la collaboration ou du groupe"
    },
    "participants": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "default": [],
      "description": "Membres pouvant voir/g\u00e9rer le budget partag\u00e9 (d\u00e9normalis\u00e9 pour RLS)"
    },
    "period": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "trimestre",
        "semestre",
        "annee",
        "personnalise"
      ],
      "description": "P\u00e9riode du budget partag\u00e9"
    },
    "custom_count": {
      "type": "number",
      "description": "Nombre d'unit\u00e9s (p\u00e9riode personnalis\u00e9e)"
    },
    "custom_unit": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "annee"
      ],
      "description": "Unit\u00e9 de la p\u00e9riode personnalis\u00e9e"
    },
    "recurring": {
      "type": "boolean",
      "default": false,
      "description": "Budget r\u00e9current"
    }
  },
  "required": [
    "name",
    "unit_cost",
    "date"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/FaqQuestion.jsonc
```

```
------------------------------------------------------------
{
  "name": "FaqQuestion",
  "type": "object",
  "properties": {
    "question": {
      "type": "string",
      "description": "Question pos\u00e9e par l'utilisateur"
    },
    "category": {
      "type": "string",
      "enum": [
        "general",
        "jpm",
        "jbm",
        "paiement",
        "affiliation",
        "compte",
        "autre"
      ],
      "default": "general",
      "description": "Cat\u00e9gorie estim\u00e9e"
    },
    "status": {
      "type": "string",
      "enum": [
        "nouveau",
        "en_cours",
        "traite"
      ],
      "default": "nouveau",
      "description": "Statut de traitement"
    },
    "answer": {
      "type": "string",
      "description": "R\u00e9ponse / synth\u00e8se (admin)"
    },
    "group_key": {
      "type": "string",
      "description": "Cl\u00e9 de regroupement IA (d\u00e9duplication des questions similaires)"
    },
    "group_label": {
      "type": "string",
      "description": "Libell\u00e9 du groupe th\u00e9matique attribu\u00e9 par l'IA"
    },
    "is_published": {
      "type": "boolean",
      "default": false,
      "description": "Afficher la question + r\u00e9ponse sur la page d'accueil"
    }
  },
  "required": [
    "question"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "user_condition": {
        "role": "admin"
      }
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Group.jsonc
------------------------------------------------------------
{
  "name": "Group",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
```

```
      "description": "Nom du groupe"
    },
    "members": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "description": "Identifiants des membres"
    },
    "admins": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "default": [],
      "description": "Membres administrateurs"
    },
    "write_restricted": {
      "type": "boolean",
      "default": false,
      "description": "Si vrai, seuls les admins et allowed_writers peuvent \u00e9crire"
    },
    "allowed_writers": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "default": [],
      "description": "Membres autoris\u00e9s \u00e0 \u00e9crire quand write_restricted est vrai"
    },
    "media_restricted": {
      "type": "boolean",
      "default": false,
      "description": "Si vrai, seuls les admins et allowed_media peuvent envoyer des m\u00e9dias"
    },
    "allowed_media": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "default": [],
      "description": "Membres autoris\u00e9s \u00e0 envoyer des m\u00e9dias quand media_restricted est vrai"
    }
  },
  "required": [
    "name",
    "members"
  ],
  "rls": {
    "read": {
      "data.members": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.admins": {
            "$in": [
              "{{user.id}}"
            ]
          }
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/GroupInvite.jsonc
------------------------------------------------------------
```

```
{
  "name": "GroupInvite",
  "type": "object",
  "properties": {
    "group_id": {
      "type": "string",
      "description": "Groupe concern\u00e9"
    },
    "group_name": {
      "type": "string",
      "description": "Nom du groupe (affichage)"
    },
    "to_user_id": {
      "type": "string",
      "description": "Utilisateur invit\u00e9"
    },
    "to_name": {
      "type": "string",
      "description": "Nom de l'invit\u00e9 (affichage)"
    },
    "status": {
      "type": "string",
      "enum": [
        "pending",
        "accepted",
        "rejected"
      ],
      "default": "pending",
      "description": "Statut de l'invitation"
    }
  },
  "required": [
    "group_id",
    "to_user_id"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.to_user_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.to_user_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Invoice.jsonc
------------------------------------------------------------
{
  "name": "Invoice",
  "type": "object",
  "properties": {
    "company_id": {
```

```
      "type": "string",
      "description": "Entreprise"
    },
    "number": {
      "type": "string",
      "description": "Num\u00e9ro de facture"
    },
    "customer_name": {
      "type": "string",
      "description": "Client"
    },
    "amount": {
      "type": "number",
      "description": "Montant"
    },
    "issue_date": {
      "type": "string",
      "format": "date",
      "description": "Date d'\u00e9mission"
    },
    "due_date": {
      "type": "string",
      "format": "date",
      "description": "Date d'\u00e9ch\u00e9ance"
    },
    "status": {
      "type": "string",
      "enum": [
        "brouillon",
        "envoyee",
        "payee",
        "en_retard"
      ],
      "default": "brouillon"
    }
  },
  "required": [
    "number",
    "customer_name",
    "amount",
    "issue_date"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Loan.jsonc
------------------------------------------------------------
{
  "name": "Loan",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Nom de l'emprunt / dette"
    },
    "principal": {
      "type": "number",
      "description": "Capital emprunt\u00e9 (montant initial)"
    },
    "interest_rate": {
      "type": "number",
      "description": "Taux d'int\u00e9r\u00eat par p\u00e9riode (%)"
    },
    "interest_type": {
      "type": "string",
      "enum": [
        "simple",
        "compose"
      ],
      "default": "compose",
      "description": "Int\u00e9r\u00eats simples (lin\u00e9aires) ou compos\u00e9s (annuit\u00e9 constante)"
    },
    "frequency": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
```

```
        "trimestre",
        "semestre",
        "annee",
        "personnalise"
      ],
      "default": "mois",
      "description": "P\u00e9riodicit\u00e9 des \u00e9ch\u00e9ances"
    },
    "custom_count": {
      "type": "number",
      "description": "Nombre d'unit\u00e9s (p\u00e9riodicit\u00e9 personnalis\u00e9e)"
    },
    "custom_unit": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "annee"
      ],
      "description": "Unit\u00e9 de la p\u00e9riodicit\u00e9 personnalis\u00e9e"
    },
    "term_periods": {
      "type": "number",
      "description": "Nombre d'\u00e9ch\u00e9ances (dur\u00e9e)"
    },
    "start_date": {
      "type": "string",
      "format": "date",
      "description": "Date de la premi\u00e8re \u00e9ch\u00e9ance"
    },
    "monthly_payment": {
      "type": "number",
      "description": "Mensualit\u00e9 / \u00e9ch\u00e9ance calcul\u00e9e"
    },
    "total_to_repay": {
      "type": "number",
      "description": "Total \u00e0 rembourser (capital + int\u00e9r\u00eats)"
    },
    "total_interest": {
      "type": "number",
      "description": "Int\u00e9r\u00eats totaux"
    },
    "payments_made": {
      "type": "number",
      "default": 0,
      "description": "Nombre d'\u00e9ch\u00e9ances d\u00e9j\u00e0 r\u00e9gl\u00e9es"
    },
    "remaining_balance": {
      "type": "number",
      "description": "Solde restant \u00e0 rembourser"
    },
    "next_payment_date": {
      "type": "string",
      "format": "date",
      "description": "Date de la prochaine \u00e9ch\u00e9ance"
    },
    "status": {
      "type": "string",
      "enum": [
        "en_cours",
        "rembourse",
        "archive"
      ],
      "default": "en_cours",
      "description": "Statut de l'emprunt"
    },
    "company_id": {
      "type": "string",
      "description": "Entreprise (JBM). Vide en espace personnel (JPM)."
    },
    "description": {
      "type": "string",
      "description": "Description / note"
    }
  },
  "required": [
    "name",
    "principal",
    "interest_rate",
    "interest_type",
    "frequency",
    "term_periods",
    "start_date"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
```

```
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/LoanPayment.jsonc
------------------------------------------------------------
{
  "name": "LoanPayment",
  "type": "object",
  "properties": {
    "loan_id": {
      "type": "string",
      "description": "Emprunt li\u00e9"
    },
    "loan_name": {
      "type": "string",
      "description": "Nom de l'emprunt (affichage)"
    },
    "payment_number": {
      "type": "number",
      "description": "Num\u00e9ro de l'\u00e9ch\u00e9ance"
    },
    "amount": {
      "type": "number",
      "description": "Montant vers\u00e9"
    },
    "interest_portion": {
      "type": "number",
      "description": "Part d'int\u00e9r\u00eats"
    },
    "principal_portion": {
      "type": "number",
      "description": "Part de capital rembours\u00e9"
    },
    "remaining_balance": {
      "type": "number",
      "description": "Solde restant apr\u00e8s ce versement"
    },
    "date": {
      "type": "string",
      "format": "date",
      "description": "Date du versement"
    }
  },
  "required": [
    "loan_id",
    "amount",
    "date"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Message.jsonc
------------------------------------------------------------
{
  "name": "Message",
  "type": "object",
  "properties": {
    "conversation_type": {
      "type": "string",
      "enum": [
        "direct",
        "group"
      ],
      "description": "Type de conversation"
    },
    "conversation_id": {
      "type": "string",
      "description": "Identifiant de la collaboration ou du groupe"
    },
    "participants": {
      "type": "array",
```

```
      "items": {
        "type": "string"
      },
      "description": "Membres pouvant lire le message (d\u00e9normalis\u00e9 pour RLS)"
    },
    "content": {
      "type": "string",
      "description": "Texte du message"
    },
    "attachment_url": {
      "type": "string",
      "description": "URL publique du fichier multim\u00e9dia (optionnel)"
    },
    "attachment_name": {
      "type": "string",
      "description": "Nom du fichier joint (optionnel)"
    }
  },
  "required": [
    "conversation_type",
    "conversation_id",
    "participants"
  ],
  "rls": {
    "read": {
      "data.participants": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Notification.jsonc
------------------------------------------------------------
{
  "name": "Notification",
  "type": "object",
  "properties": {
    "title": {
      "type": "string",
      "description": "Titre de la notification"
    },
    "message": {
      "type": "string",
      "description": "Contenu"
    },
    "type": {
      "type": "string",
      "enum": [
        "rappel",
        "conseil",
        "personnalisee",
        "systeme"
      ],
      "default": "personnalisee"
    },
    "read": {
      "type": "boolean",
      "default": false
    },
    "enabled": {
      "type": "boolean",
      "default": true
    },
    "due_date": {
      "type": "string",
      "format": "date-time"
    },
    "source": {
      "type": "string",
      "enum": [
        "auto",
        "agent",
        "manuel"
      ],
      "default": "agent"
    }
  },
  "required": [
    "title"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
```

```
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Partnership.jsonc
------------------------------------------------------------
{
  "name": "Partnership",
  "type": "object",
  "properties": {
    "scope": {
      "type": "string",
      "enum": [
        "entreprise",
        "collaboration",
        "group"
      ],
      "default": "entreprise",
      "description": "Espace concern\u00e9"
    },
    "company_id": {
      "type": "string",
      "description": "Entreprise"
    },
    "partner_name": {
      "type": "string",
      "description": "Nom du partenaire"
    },
    "partner_type": {
      "type": "string",
      "enum": [
        "individu",
        "personne_morale"
      ],
      "default": "individu",
      "description": "Type de partenaire"
    },
    "contact_name": {
      "type": "string",
      "description": "Personne de contact"
    },
    "email": {
      "type": "string",
      "description": "Email"
    },
    "phone": {
      "type": "string",
      "description": "T\u00e9l\u00e9phone"
    },
    "modalities": {
      "type": "string",
      "description": "Modalit\u00e9s du partenariat (champ libre)"
    },
    "status": {
      "type": "string",
      "enum": [
        "prospect",
        "actif",
        "inactif"
      ],
      "default": "prospect",
      "description": "Statut"
    },
    "start_date": {
      "type": "string",
      "format": "date",
      "description": "Date de d\u00e9but"
    },
    "shared_context_type": {
      "type": "string",
      "enum": [
        "collaboration",
        "group"
      ],
      "description": "Contexte partag\u00e9 (vide = partenariat entreprise)"
    },
    "shared_context_id": {
      "type": "string",
      "description": "Identifiant de la collaboration ou du groupe"
    },
    "participants": {
      "type": "array",
```

```
      "items": {
        "type": "string"
      },
      "default": [],
      "description": "Membres pouvant voir/g\u00e9rer le partenariat partag\u00e9 (d\u00e9normalis\u00e9 pour RLS)"
    }
  },
  "required": [
    "partner_name",
    "partner_type"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/PersonalItem.jsonc
------------------------------------------------------------
{
  "name": "PersonalItem",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Nom de l'objet"
    },
    "category": {
      "type": "string",
      "description": "Cat\u00e9gorie"
    },
    "location": {
      "type": "string",
      "description": "Emplacement"
    },
    "purchase_date": {
      "type": "string",
      "format": "date",
      "description": "Date d'acquisition"
    },
    "value": {
      "type": "number",
      "description": "Valeur estim\u00e9e (XAF)"
    },
    "usage_frequency": {
      "type": "string",
      "enum": [
        "quotidien",
        "hebdomadaire",
        "mensuel",
        "occasionnel",
        "rare"
      ],
      "default": "occasionnel",
      "description": "Fr\u00e9quence d'utilisation"
    },
    "last_used_date": {
      "type": "string",
      "format": "date",
      "description": "Derni\u00e8re utilisation"
```

```
    },
    "notes": {
      "type": "string",
      "description": "Notes"
    }
  },
  "required": [
    "name"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Preferences.jsonc
------------------------------------------------------------
{
  "name": "Preferences",
  "type": "object",
  "properties": {
    "currency": {
      "type": "string",
      "enum": [
        "XAF",
        "EUR",
        "USD"
      ],
      "default": "XAF"
    },
    "initial_budget": {
      "type": "number",
      "default": 0,
      "description": "Budget personnel initial"
    },
    "budget_period": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "trimestre",
        "semestre",
        "annee",
        "personnalise"
      ],
      "default": "mois",
      "description": "P\u00e9riode du budget"
    },
    "budget_custom_count": {
      "type": "number",
      "description": "Nombre d'unit\u00e9s (budget p\u00e9riode personnalis\u00e9e)"
    },
    "budget_custom_unit": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "annee"
      ],
      "description": "Unit\u00e9 de la p\u00e9riode personnalis\u00e9e du budget"
    },
    "reminder_days": {
      "type": "number",
      "default": 7,
      "description": "Fen\u00eatre des rappels \u00e0 venir, en jours"
    },
    "theme": {
      "type": "string",
      "enum": [
        "system",
        "light",
        "dark"
      ],
      "default": "system",
      "description": "Pr\u00e9f\u00e9rence de th\u00e8me d'affichage"
    },
    "language": {
      "type": "string",
      "enum": [
```

```
        "fr",
        "en"
      ],
      "default": "fr",
      "description": "Langue de l'interface"
    },
    "active_space": {
      "type": "string",
      "enum": [
        "jpm",
        "jbm"
      ],
      "default": "jpm",
      "description": "Espace actif : JPM ou JBM"
    },
    "active_company_id": {
      "type": "string",
      "description": "Identifiant de l'entreprise actuellement s\u00e9lectionn\u00e9e"
    },
    "notifications_enabled": {
      "type": "boolean",
      "default": true,
      "description": "Activer/d\u00e9sactiver toutes les notifications"
    }
  },
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  },
  "required": []
}

------------------------------------------------------------
FICHIER : ./base44/entities/Product.jsonc
------------------------------------------------------------
{
  "name": "Product",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise propri\u00e9taire"
    },
    "name": {
      "type": "string",
      "description": "Nom du produit"
    },
    "sku": {
      "type": "string",
      "description": "R\u00e9f\u00e9rence / code"
    },
    "category": {
      "type": "string",
      "description": "Cat\u00e9gorie"
    },
    "quantity": {
      "type": "number",
      "default": 0,
      "description": "Quantit\u00e9 en stock"
    },
    "unit_cost": {
      "type": "number",
      "description": "Co\u00fbt unitaire"
    },
    "sale_price": {
      "type": "number",
      "description": "Prix de vente"
    },
    "reorder_level": {
      "type": "number",
      "default": 0,
      "description": "Seuil de r\u00e9approvisionnement"
    }
  },
  "required": [
    "name"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
```

```
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Quote.jsonc
------------------------------------------------------------
{
  "name": "Quote",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise / \u00e9tablissement"
    },
    "number": {
      "type": "string",
      "description": "Num\u00e9ro du devis"
    },
    "client_name": {
      "type": "string",
      "description": "Client"
    },
    "amount": {
      "type": "number",
      "description": "Montant"
    },
    "issue_date": {
      "type": "string",
      "format": "date",
      "description": "Date d'\u00e9mission"
    },
    "validity_date": {
      "type": "string",
      "format": "date",
      "description": "Date de validit\u00e9"
    },
    "status": {
      "type": "string",
      "enum": [
        "brouillon",
        "envoye",
        "accepte",
        "refuse"
      ],
      "default": "brouillon",
      "description": "Statut du devis"
    },
    "notes": {
      "type": "string",
      "description": "Description / modalit\u00e9s"
    }
  },
  "required": [
    "number",
    "client_name",
    "amount",
    "issue_date"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/ReSubscriptionRequest.jsonc
------------------------------------------------------------
{
  "name": "ReSubscriptionRequest",
  "type": "object",
  "properties": {
    "email": {
      "type": "string",
      "description": "Email (minuscules) du demandeur"
```

```
    },
    "message": {
      "type": "string",
      "description": "Message / motif du demandeur"
    },
    "status": {
      "type": "string",
      "enum": [
        "pending",
        "approved",
        "rejected"
      ],
      "default": "pending",
      "description": "Statut de la demande"
    },
    "admin_note": {
      "type": "string",
      "description": "Note interne admin"
    }
  },
  "required": [
    "email"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "user_condition": {
        "role": "admin"
      }
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Report.jsonc
------------------------------------------------------------
{
  "name": "Report",
  "type": "object",
  "properties": {
    "target_type": {
      "type": "string",
      "enum": [
        "user",
        "group"
      ],
      "description": "Cible du signalement"
    },
    "target_id": {
      "type": "string",
      "description": "ID de l'utilisateur ou du groupe signal\u00e9"
    },
    "target_name": {
      "type": "string",
      "description": "Nom affich\u00e9 de la cible"
    },
    "reason": {
      "type": "string",
      "enum": [
        "spam",
        "harcelement",
        "contenu_inapproprie",
        "arnaque",
        "usurpation",
        "autre"
      ],
      "default": "autre",
      "description": "Raison du signalement"
    },
    "custom_reason": {
      "type": "string",
      "description": "Raison r\u00e9dig\u00e9e librement"
    },
```

```
    "status": {
      "type": "string",
      "enum": [
        "nouveau",
        "en_cours",
        "traite"
      ],
      "default": "nouveau",
      "description": "Statut de traitement"
    },
    "admin_note": {
      "type": "string",
      "description": "Note interne admin"
    }
  },
  "required": [
    "target_type",
    "target_id",
    "reason"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "user_condition": {
        "role": "admin"
      }
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Sale.jsonc
------------------------------------------------------------
{
  "name": "Sale",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise"
    },
    "product_id": {
      "type": "string",
      "description": "Produit li\u00e9 (optionnel)"
    },
    "customer_name": {
      "type": "string",
      "description": "Client"
    },
    "item_name": {
      "type": "string",
      "description": "Article / service vendu"
    },
    "category": {
      "type": "string",
      "description": "Cat\u00e9gorie de vente"
    },
    "quantity": {
      "type": "number",
      "default": 1
    },
    "unit_price": {
      "type": "number",
      "description": "Prix unitaire (saisi ou calcul\u00e9 par formule)"
    },
    "base_price": {
      "type": "number",
      "description": "Prix de base utilis\u00e9 par la formule de tarification"
    },
    "pricing_formula": {
      "type": "string",
      "description": "Formule de tarification : expression r\u00e9f\u00e9ren\u00e7ant qte (quantit\u00e9), base (prix de base)
et les grandeurs du calculateur. Ex: base + qte*100"
```

```
    },
    "scheduled": {
      "type": "boolean",
      "default": false,
      "description": "Vente programm\u00e9e \u00e0 l'avance (notification J-1 et jour J)"
    },
    "date": {
      "type": "string",
      "format": "date"
    },
    "status": {
      "type": "string",
      "enum": [
        "en_attente",
        "payee",
        "annulee"
      ],
      "default": "payee"
    }
  },
  "required": [
    "item_name",
    "date"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/SavingsPlan.jsonc
------------------------------------------------------------
{
  "name": "SavingsPlan",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Nom du plan d'\u00e9pargne"
    },
    "target_amount": {
      "type": "number",
      "description": "Somme \u00e0 \u00e9pargner sur la p\u00e9riode"
    },
    "period": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "trimestre",
        "semestre",
        "annee",
        "personnalise"
      ],
      "default": "mois",
      "description": "P\u00e9riode du plan d'\u00e9pargne"
    },
    "custom_count": {
      "type": "number",
      "description": "Nombre d'unit\u00e9s (p\u00e9riode personnalis\u00e9e)"
    },
    "custom_unit": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "annee"
      ],
      "description": "Unit\u00e9 de la p\u00e9riode personnalis\u00e9e"
    },
    "start_date": {
      "type": "string",
      "format": "date",
      "description": "Date de d\u00e9but du plan"
    },
    "saved_amount": {
      "type": "number",
      "default": 0,
      "description": "Montant d\u00e9j\u00e0 \u00e9pargn\u00e9 (suivi manuel optionnel)"
```

```
    },
    "status": {
      "type": "string",
      "enum": [
        "actif",
        "atteint",
        "archive"
      ],
      "default": "actif",
      "description": "Statut du plan"
    }
  },
  "required": [
    "name",
    "target_amount",
    "period"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Subscription.jsonc
------------------------------------------------------------
{
  "name": "Subscription",
  "type": "object",
  "properties": {
    "account_type": {
      "type": "string",
      "enum": [
        "particulier",
        "entreprise"
      ],
      "description": "Type d'abonnement"
    },
    "amount": {
      "type": "number",
      "description": "Montant pay\u00e9 (XAF)"
    },
    "companies_count": {
      "type": "number",
      "default": 1,
      "description": "Nombre d'entreprises couvertes (JH Business Manager). Le prix = 20 000 XAF multipli\u00e9 par ce nombre."
    },
    "account_name": {
      "type": "string",
      "description": "Nom figurant sur le compte Mobile Money du payeur (identification)"
    },
    "phone_number": {
      "type": "string",
      "description": "Num\u00e9ro Mobile Money du payeur (format international)"
    },
    "provider": {
      "type": "string",
      "enum": [
        "airtel",
        "moov"
      ],
      "description": "Op\u00e9rateur Mobile Money"
    },
    "status": {
      "type": "string",
      "enum": [
        "payee",
        "en_attente",
        "echouee",
        "resilie"
      ],
      "default": "payee"
    },
    "paid_at": {
      "type": "string",
      "format": "date-time",
      "description": "Date du paiement"
    },
    "reference": {
      "type": "string",
      "description": "R\u00e9f\u00e9rence de transaction renvoy\u00e9e par la passerelle"
    }
```

```
  },
  "required": [
    "account_type",
    "amount",
    "phone_number",
    "provider"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "user_condition": {
        "role": "admin"
      }
    },
    "update": {
      "user_condition": {
        "role": "admin"
      }
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Suggestion.jsonc
------------------------------------------------------------
{
  "name": "Suggestion",
  "type": "object",
  "properties": {
    "content": {
      "type": "string",
      "description": "Contenu de la suggestion"
    },
    "category": {
      "type": "string",
      "enum": [
        "amelioration",
        "correction",
        "autre"
      ],
      "default": "amelioration",
      "description": "Cat\u00e9gorie"
    },
    "status": {
      "type": "string",
      "enum": [
        "nouveau",
        "en_cours",
        "traite"
      ],
      "default": "nouveau",
      "description": "Statut de traitement"
    }
  },
  "required": [
    "content"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "user_condition": {
        "role": "admin"
```

```
      }
    },
    "delete": {
      "user_condition": {
        "role": "admin"
      }
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Supplier.jsonc
------------------------------------------------------------
{
  "name": "Supplier",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise / \u00e9tablissement"
    },
    "name": {
      "type": "string",
      "description": "Nom du prestataire"
    },
    "category": {
      "type": "string",
      "description": "Cat\u00e9gorie / sp\u00e9cialit\u00e9"
    },
    "contact_person": {
      "type": "string",
      "description": "Personne de contact"
    },
    "phone": {
      "type": "string",
      "description": "T\u00e9l\u00e9phone"
    },
    "email": {
      "type": "string",
      "description": "Email"
    },
    "address": {
      "type": "string",
      "description": "Adresse"
    },
    "notes": {
      "type": "string",
      "description": "Notes"
    }
  },
  "required": [
    "name"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Task.jsonc
------------------------------------------------------------
{
  "name": "Task",
  "type": "object",
  "properties": {
    "title": {
      "type": "string",
      "description": "Nom de la t\u00e2che"
    },
    "duration_minutes": {
      "type": "number",
      "description": "Dur\u00e9e estim\u00e9e en minutes"
    },
    "start": {
      "type": "string",
      "format": "date-time",
      "description": "Date et heure de d\u00e9but"
    },
    "end": {
      "type": "string",
      "format": "date-time",
```

```
      "description": "Date et heure de fin"
    },
    "status": {
      "type": "string",
      "enum": [
        "a_faire",
        "en_cours",
        "terminee"
      ],
      "default": "a_faire"
    },
    "priority": {
      "type": "string",
      "enum": [
        "basse",
        "normale",
        "haute"
      ],
      "default": "normale"
    },
    "category": {
      "type": "string",
      "description": "Cat\u00e9gorie de la t\u00e2che"
    },
    "shared_context_type": {
      "type": "string",
      "enum": [
        "collaboration",
        "group"
      ],
      "description": "Contexte partag\u00e9 (vide = t\u00e2che personnelle)"
    },
    "shared_context_id": {
      "type": "string",
      "description": "Identifiant de la collaboration ou du groupe"
    },
    "participants": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "default": [],
      "description": "Membres pouvant voir/g\u00e9rer la t\u00e2che partag\u00e9e (d\u00e9normalis\u00e9 pour RLS)"
    },
    "assignee_id": {
      "type": "string",
      "description": "Membre responsable de la t\u00e2che partag\u00e9e"
    }
  },
  "required": [
    "title",
    "start",
    "end"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "data.participants": "{{user.id}}"
        }
      ]
    }
  }
}
```

FICHIER : ./base44/entities/Tax.jsonc

```
{
  "name": "Tax",
  "type": "object",
  "properties": {
    "company_id": {
      "type": "string",
      "description": "Entreprise / \u00e9tablissement (JBM). Vide en espace personnel."
    },
    "scope": {
      "type": "string",
      "enum": [
        "jpm",
        "jbm"
      ],
      "default": "jpm",
      "description": "Espace concern\u00e9 : jpm (particulier) ou jbm (entreprise)"
    },
    "name": {
      "type": "string",
      "description": "Nom de la taxe / charge"
    },
    "type": {
      "type": "string",
      "enum": [
        "tva",
        "charge",
        "impot",
        "autre"
      ],
      "default": "charge",
      "description": "Type"
    },
    "value_mode": {
      "type": "string",
      "enum": [
        "pourcentage",
        "montant"
      ],
      "default": "montant",
      "description": "Mode de calcul : pourcentage (de la base) ou montant fixe"
    },
    "rate": {
      "type": "number",
      "description": "Taux (%) utilis\u00e9\u00e9 quand value_mode = pourcentage"
    },
    "base_amount": {
      "type": "number",
      "description": "Assiette / base de calcul (pour value_mode = pourcentage)"
    },
    "amount": {
      "type": "number",
      "description": "Montant calcul\u00e9 (pourcentage) ou valeur pr\u00e9cise (montant fixe)"
    },
    "frequency": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "trimestre",
        "semestre",
        "annee",
        "personnalise"
      ],
      "description": "P\u00e9riodicit\u00e9 de la taxe r\u00e9currente"
    },
    "custom_count": {
      "type": "number",
      "description": "Nombre d'unit\u00e9s (p\u00e9riodicit\u00e9 personnalis\u00e9e)"
    },
    "custom_unit": {
      "type": "string",
      "enum": [
        "jour",
        "semaine",
        "mois",
        "annee"
      ],
      "description": "Unit\u00e9 de la p\u00e9riodicit\u00e9 personnalis\u00e9e"
    },
    "recurring": {
      "type": "boolean",
      "default": false,
      "description": "Taxe r\u00e9currente (g\u00e9n\u00e8re plusieurs \u00e9ch\u00e9ances)"
    },
    "start_date": {
      "type": "string",
      "format": "date",
      "description": "Date de la premi\u00e8re \u00e9ch\u00e9ance"
    },
```

```
    "period": {
      "type": "string",
      "description": "Libell\u00e9 de p\u00e9riode (ex. 2026, T1 2026) \u2014 libre"
    },
    "description": {
      "type": "string",
      "description": "Description"
    }
  },
  "required": [
    "name",
    "scope",
    "value_mode"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/Threshold.jsonc
------------------------------------------------------------
{
  "name": "Threshold",
  "type": "object",
  "properties": {
    "domain": {
      "type": "string",
      "enum": [
        "budget",
        "stock",
        "invoices",
        "tasks"
      ],
      "description": "Domaine concern\u00e9"
    },
    "medium_value": {
      "type": "number",
      "description": "Seuil moyen"
    },
    "critical_value": {
      "type": "number",
      "description": "Seuil critique"
    }
  },
  "required": [
    "domain",
    "medium_value",
    "critical_value"
  ],
  "rls": {
    "read": {
      "created_by_id": "{{user.id}}"
    },
    "create": {
      "created_by_id": "{{user.id}}"
    },
    "update": {
      "created_by_id": "{{user.id}}"
    },
    "delete": {
      "created_by_id": "{{user.id}}"
    }
  }
}

------------------------------------------------------------
FICHIER : ./base44/entities/User.jsonc
------------------------------------------------------------
{
  "properties": {
    "role": {
      "type": "string",
      "description": "The role of the user in the app",
      "enum": [
        "admin",
        "user"
      ]
    },
    "account_type": {
      "type": "string",
```

```
      "enum": [
        "particulier",
        "entreprise"
      ],
      "description": "Type de compte : Particulier (JPM) ou Entreprise (JBM)"
    }
  },
  "required": [
    "role"
  ],
  "name": "User",
  "type": "object"
}

------------------------------------------------------------
FICHIER : ./base44/functions/analyzeSuggestions/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

// Analyse les suggestions utilisateur et regroupe les suggestions sémantiquement
// équivalentes (dédoublonnage par IA). Renvoie des "idées uniques" avec le nombre
// d'utilisateurs distincts les ayant émises et le pourcentage correspondant.
export default async function (req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });
    if (user.role !== 'admin') return Response.json({ error: 'Admin only' }, { status: 403 });

    const suggestions = await base44.asServiceRole.entities.Suggestion.list('-created_date', 1000);
    if (!suggestions || suggestions.length === 0) {
      return Response.json({ success: true, groups: [], totalUsers: 0 });
    }

    const items = suggestions.map((s, i) => ({
      index: i,
      content: String(s.content || '').slice(0, 280),
      category: s.category || 'autre',
      author: s.created_by_id || s.id,
    }));

    const prompt = `Tu es un assistant qui regroupe des suggestions d'utilisateurs.
Voici une liste de suggestions (index, contenu, catégorie). Regroupe en clusters les
suggestions qui expriment la même idée (même amélioration ou correction demandée),
même si formulées différemment. Pour chaque cluster, donne un titre court représentatif
et la liste des index concernés. Réponds UNIQUEMENT au format JSON attendu.

Suggestions:
${JSON.stringify(items.map((x) => ({ index: x.index, content: x.content, category: x.category })))}`;

    const llm = await base44.asServiceRole.integrations.Core.InvokeLLM({
      prompt,
      model: 'gpt_5_mini',
      response_json_schema: {
        type: 'object',
        properties: {
          groups: {
            type: 'array',
            items: {
              type: 'object',
              properties: {
                label: { type: 'string' },
                indices: { type: 'array', items: { type: 'number' } },
              },
              required: ['label', 'indices'],
            },
          },
        },
        required: ['groups'],
      },
    });

    const groupsRaw = (llm && llm.groups) || (Array.isArray(llm) ? llm : []);

    const totalUsers = new Set(items.map((x) => x.author)).size || 1;
    const groups = groupsRaw
      .map((grp) => {
        const idxs = Array.isArray(grp.indices) ? grp.indices : [];
        const members = idxs
          .map((i) => items.find((x) => x.index === i))
          .filter(Boolean);
        const distinctAuthors = new Set(members.map((m) => m.author)).size;
        return {
          label: String(grp.label || '').slice(0, 140),
          occurrences: members.length,
          users: distinctAuthors,
          percentage: Math.round((distinctAuthors / totalUsers) * 1000) / 10,
          category: members[0]?.category || 'autre',
        };
      })
      .filter((g) => g.users > 0)
      .sort((a, b) => b.users - a.users);
```

```
    return Response.json({ success: true, groups, totalUsers });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/breakCollaboration/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

// Rupture d'une collaboration 1-à-1.
// - supprime la collaboration
// - retire automatiquement l'ex-collaborateur des groupes dont l'autre est créateur
 / /       ( d a n s   l e s   d e u x   s e n s   :   s i   j e   s u i s   c r é a t e u r   d u   g r o u p e   !’   j e   r e t i r
 / /         s i   l ' a u t r e   e s t   c r é a t e u r   d u   g r o u p e   !’   j e   m ' e n   r e t i r e )
export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const body = await req.json().catch(() => ({}));
    const collaborationId = body.collaboration_id;
    if (!collaborationId) return Response.json({ error: 'collaboration_id requis' }, { status: 400 });

    const collab = await base44.entities.Collaboration.get(collaborationId);
    if (!collab) return Response.json({ error: 'Collaboration introuvable' }, { status: 404 });

    const members = collab.members || [];
    if (!members.includes(user.id)) {
      return Response.json({ error: 'Non autorisé' }, { status: 403 });
    }
    const otherId = members.find((m) => m !== user.id);

    // Suppression de la collaboration (l'utilisateur est membre, donc autorisé)
    await base44.entities.Collaboration.delete(collaborationId);

    // Retrait automatique dans les groupes communs (lecture limitée par RLS aux groupes dont je suis membre)
    if (otherId) {
      const groups = await base44.entities.Group.list();
      for (const g of groups) {
        const gm = g.members || [];
        const creator = g.created_by_id;
        const toRemove = creator === user.id ? otherId : (creator === otherId ? user.id : null);
        if (toRemove && gm.includes(toRemove)) {
          const patch = {
            members: gm.filter((m) => m !== toRemove),
            admins: (g.admins || []).filter((a) => a !== toRemove),
            allowed_writers: (g.allowed_writers || []).filter((a) => a !== toRemove),
            allowed_media: (g.allowed_media || []).filter((a) => a !== toRemove),
          };
          // Mise à jour via service role : garantit le retrait même si le créateur est l'autre
          await base44.asServiceRole.entities.Group.update(g.id, patch);
        }
      }
    }

    return Response.json({ ok: true });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/cancelSubscription/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    // Find the user's latest paid (active) subscription.
    const subs = await base44.asServiceRole.entities.Subscription.filter(
      { created_by_id: user.id, status: 'payee' },
      '-paid_at',
      1
    );
    const sub = subs?.[0];
    if (!sub) {
      return Response.json({ error: 'Aucun abonnement actif' }, { status: 404 });
    }

    await base44.asServiceRole.entities.Subscription.update(sub.id, { status: 'resilie' });
    return Response.json({ success: true, subscriptionId: sub.id });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
```

```
}

------------------------------------------------------------
FICHIER : ./base44/functions/checkAlerts/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const [thresholds, prefs, exp, prod, inv, tsk, notifs] = await Promise.all([
      base44.entities.Threshold.list("-created_date", 50),
      base44.entities.Preferences.list(),
      base44.entities.Expense.list("-date", 1000),
      base44.entities.Product.list("-created_date", 1000),
      base44.entities.Invoice.list("-issue_date", 1000),
      base44.entities.Task.list("-start", 1000),
      base44.entities.Notification.list("-created_date", 200),
    ]);

    const budget = (prefs?.[0]?.initial_budget || 0) - (exp || []).reduce((s, e) => s + (Number(e.quantity) || 0) *
(Number(e.unit_cost) || 0), 0);
    const stock = (prod || []).reduce((s, p) => s + (Number(p.quantity) || 0) * (Number(p.unit_cost) || 0), 0);
    const invoices = (inv || []).filter((i) => i.status !== "payee").reduce((s, i) => s + (Number(i.amount) || 0), 0);
    const now = new Date();
    const tasks = (tsk || []).filter((x) => x.status !== "terminee" && x.end && new Date(x.end) < now).length;

    const today = new Date().toISOString().slice(0, 10);
    const defs = [
      { domain: "budget", value: budget, direction: "low", label: "Budget restant", money: true },
      { domain: "stock", value: stock, direction: "low", label: "Valeur du stock", money: true },
      { domain: "invoices", value: invoices, direction: "high", label: "Factures impayées", money: true },
      { domain: "tasks", value: tasks, direction: "high", label: "Tâches en retard", money: false },
    ];

    const toCreate = [];
    for (const d of defs) {
      const th = (thresholds || []).find((x) => x.domain === d.domain);
      if (!th) continue;
      const v = Number(d.value) || 0;
      const med = Number(th.medium_value) || 0;
      const crit = Number(th.critical_value) || 0;
      let level = null;
      if (d.direction === "low") {
        if (v <= crit) level = "critique";
        else if (v <= med) level = "moyen";
      } else {
        if (v >= crit) level = "critique";
        else if (v >= med) level = "moyen";
      }
      if (!level) continue;
      const prefix = `Alerte ${level} : ${d.label}`;
      const exists = (notifs || []).some((n) => (n.title || "").startsWith(prefix) && (n.created_date || "").slice(0, 10) ===
today);
      if (exists) continue;
      const disp = d.money ? `${Math.round(v).toLocaleString("fr-FR")} XAF` : `${v}`;
      toCreate.push({
        title: prefix,
        message: `${d.label} : ${disp} — seuil ${level} atteint. Consultez la section concernée pour agir.`,
        type: "rappel",
        source: "auto",
        read: false,
        enabled: true,
      });
    }

    let created = [];
    if (toCreate.length) created = await base44.entities.Notification.bulkCreate(toCreate);
    return Response.json({ created: created || [], count: (created || []).length });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/dailyQuote/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const body = await req.json().catch(() => ({}));
    const space = body?.space || "jpm";

    const domains = {
```

```
      jpm: "la gestion personnelle (budget, dépenses, tâches, planning, calendrier)",
      jbm: "la gestion d'entreprise (ventes, stock, personnel, facturation, clients, partenariats)",
      ecole: "la gestion d'un établissement scolaire (classes, étudiants, matières, notes, pédagogie)"
    };
    const domaine = domains[space] || domains.jpm;

    const prompt = `Tu es un coach bienveillant pour un abonné d'une app de gestion au Gabon. Donne UN conseil pratique du jour, en français, lié à ${domaine}. Le conseil doit aider à la fois dans l'usage de l'application ET dans sa vie active/professionnelle, en restant sur ces sujets. Une seule phrase, max 28 mots, directe et actionnable. Réponds uniquement par le conseil, sans guillemets ni explication.`;
    const res = await base44.asServiceRole.integrations.Core.InvokeLLM({ prompt, model: "gpt_5_mini" });
    const quote = typeof res === "string" ? res : (res?.text || res?.content || "");
    return Response.json({ quote });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/deleteAccount/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.44';

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Non autorisé' }, { status: 401 });

    const userId = user.id;

    // Delete all of the user's own data (service role bypasses RLS for a clean wipe).
    await base44.asServiceRole.entities.Task.deleteMany({ created_by_id: userId });
    await base44.asServiceRole.entities.Expense.deleteMany({ created_by_id: userId });
    await base44.asServiceRole.entities.Preferences.deleteMany({ created_by_id: userId });

    // Verify the data is actually gone.
    const remainingTasks = await base44.asServiceRole.entities.Task.filter({ created_by_id: userId });
    const remainingExpenses = await base44.asServiceRole.entities.Expense.filter({ created_by_id: userId });
    const remainingPrefs = await base44.asServiceRole.entities.Preferences.filter({ created_by_id: userId });

    if (remainingTasks.length || remainingExpenses.length || remainingPrefs.length) {
      return Response.json({
        error: "Certaines données n'ont pas pu être supprimées.",
        remaining: {
          tasks: remainingTasks.length,
          expenses: remainingExpenses.length,
          preferences: remainingPrefs.length,
        },
      }, { status: 500 });
    }

    // Finally, delete the user account itself.
    try {
      await base44.asServiceRole.entities.User.delete(userId);
    } catch (e) {
      // Some deployments restrict user deletion; report clearly.
      return Response.json({
        warning: 'Données supprimées mais le compte utilisateur ne peut pas être supprimé automatiquement.',
        dataDeleted: true,
        userDeleted: false,
      }, { status: 200 });
    }

    return Response.json({ success: true, dataDeleted: true, userDeleted: true });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/deleteSubscriber/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

// Entités appartenant à l'utilisateur dont on efface les enregistrements.
const USER_ENTITIES = [
  'Task', 'Expense', 'Preferences', 'Tax', 'Loan', 'LoanPayment', 'SavingsPlan',
  'Category', 'Company', 'Collaborator', 'CollaborationRequest', 'Collaboration',
  'Group', 'GroupInvite', 'Message', 'Event', 'Partnership', 'Product', 'Sale',
  'Invoice', 'Client', 'Quote', 'Supplier', 'Employee', 'Notification',
  'Subscription', 'PersonalItem', 'Threshold', 'FaqQuestion', 'Suggestion',
  'Report', 'ReSubscriptionRequest',
];

export default async function (req) {
  try {
    const base44 = createClientFromRequest(req);
    const caller = await base44.auth.me();
    if (!caller) return Response.json({ error: 'Non autorisé' }, { status: 401 });
    if (caller.role !== 'admin') return Response.json({ error: 'Réservé aux administrateurs' }, { status: 403 });
```

```
    const body = await req.json().catch(() => ({}));
    const targetId = body?.user_id;
    if (!targetId) return Response.json({ error: 'user_id requis' }, { status: 400 });
    if (targetId === caller.id) return Response.json({ error: 'Vous ne pouvez pas supprimer votre propre compte' }, { status:
400 });

    const users = await base44.asServiceRole.entities.User.list();
    const target = (users || []).find((u) => u.id === targetId);
    if (!target) return Response.json({ error: 'Utilisateur introuvable' }, { status: 404 });
    if (target.role === 'admin') return Response.json({ error: 'Impossible de supprimer un administrateur' }, { status: 400 });

    const email = (target.email || '').toLowerCase().trim();
    const reason = (body?.reason || '').toString().slice(0, 500);

    // Efface toutes les données créées par l'abonné (service role = contournement RLS).
    let dataDeleted = 0;
    for (const name of USER_ENTITIES) {
      try {
        const res = await base44.asServiceRole.entities[name].deleteMany({ created_by_id: targetId });
        dataDeleted += res?.deletedCount || 0;
      } catch { /* entité inexistante ou non disponible */ }
    }

    // Tente de supprimer le compte auth lui-même.
    let userDeleted = false;
    try {
      await base44.asServiceRole.entities.User.delete(targetId);
      userDeleted = true;
    } catch { /* certaines plateformes refusent la suppression du User */ }

    // Marque l'email comme supprimé : toute réinscription devra passer par une demande de réabonnement.
    try {
      await base44.asServiceRole.entities.DeletedAccount.deleteMany({ email });
      await base44.asServiceRole.entities.DeletedAccount.create({
        email,
        full_name: target.full_name || '',
        reason,
        deleted_by: caller.id,
      });
    } catch { /* échec non bloquant */ }

    return Response.json({ success: true, dataDeleted, userDeleted, email });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/downloadSource/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.52';

// URI de l'archive source (snapshot) uploadée en stockage privé.
const SOURCE_FILE_URI = 'mp/private/6aa7cc96de4ea0a27fc06e6d/c0a7c26dd_jhmanager-src.zip';

export default async function(req: Request): Promise<Response> {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Non autorisé' }, { status: 401 });

    const { signed_url } = await base44.asServiceRole.integrations.Core.CreateFileSignedUrl({
      file_uri: SOURCE_FILE_URI,
      expires_in: 3600,
    });
    return Response.json({ signed_url });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/emailAlerts/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

const URGENT_HORIZON_DAYS = 3;
const BIG_EXPENSE_THRESHOLD = 50000;

function fmtDate(iso) {
  try {
    return new Date(iso).toLocaleDateString('fr-FR', { day: '2-digit', month: 'short', year: 'numeric' });
  } catch {
    return iso || '';
  }
}

export default async function (req) {
  try {
    const base44 = createClientFromRequest(req);
    const now = new Date();
    const horizon = new Date(now.getTime() + URGENT_HORIZON_DAYS * 24 * 60 * 60 * 1000);
```

```
    const since = new Date(now.getTime() - 24 * 60 * 60 * 1000);

    const tasks = await base44.asServiceRole.entities.Task.filter({});
    const urgent = (tasks || []).filter((t) => {
      if (!t.start) return false;
      if (t.status === 'termine' || t.status === 'done') return false;
      const d = new Date(t.start);
      return d <= horizon && (t.priority === 'haute' || d <= now);
    });

    const expenses = await base44.asServiceRole.entities.Expense.filter({});
    const big = (expenses || []).filter((e) => {
      const created = e.created_date ? new Date(e.created_date) : null;
      const amount = Number(e.amount) || 0;
      return created && created >= since && amount >= BIG_EXPENSE_THRESHOLD;
    });

    if (urgent.length === 0 && big.length === 0) {
      return Response.json({ sent: false, reason: 'no_alerts', urgent: 0, big: 0 });
    }

    const users = await base44.asServiceRole.entities.User.list();
    const admin = (users || []).find((u) => u.role === 'admin') || (users || [])[0];
    if (!admin || !admin.email) {
      return Response.json({ sent: false, reason: 'no_recipient' });
    }

    let html = `<div style="font-family:Inter,Arial,sans-serif;color:#1a2e2a">
      <h2 style="color:#2f6b54">JH Manager — Alertes du ${fmtDate(now.toISOString())}</h2>`;
    if (urgent.length) {
            h t m l   + =   ` < h 3   s t y l e = " c o l o r : # b 4 5 3 0 9 " >#ð   $ { u r g e n t . l e n g t h }   t â c h e ( s
      urgent.slice(0, 20).forEach((t) => {
        html += `<li><strong>${t.title || 'Sans titre'}</strong> — échéance ${fmtDate(t.start)}${t.priority === 'haute' ? ' ·
priorité haute' : ''}</li>`;
      });
      html += `</ul>`;
    }
    if (big.length) {
            h t m l   + =   ` < h 3   s t y l e = " c o l o r : # b 9 1 c 1 c " >Ø=Ü¸   $ { b i g . l e n g t h }   d é p e n s e ( s
      big.slice(0, 20).forEach((e) => {
        html += `<li><strong>${(Number(e.amount) || 0).toLocaleString('fr-FR')} XAF</strong> — ${e.category ||
'dépense'}${e.date ? ` · ${fmtDate(e.date)}` : ''}</li>`;
      });
      html += `</ul>`;
    }
    html += `<p style="color:#6b7280;font-size:12px;margin-top:24px">Alerte automatique JH Manager.</p></div>`;

    await base44.asServiceRole.integrations.Core.SendEmail({
      to: admin.email,
      subject: `JH Manager — Alertes du ${fmtDate(now.toISOString())}`,
      html,
    });

    return Response.json({ sent: true, to: admin.email, urgent: urgent.length, big: big.length });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/generateMedia/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

// Génère un fichier multimédia (image, audio ou vidéo) via les intégrations Core.
// Exposé comme outil aux agents IA pour qu'ils produisent et partagent des médias.
export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const body = await req.json().catch(() => ({}));
    const kind = String(body?.kind || 'image').toLowerCase();
    const prompt = String(body?.prompt || '').slice(0, 1200);
    if (!prompt) return Response.json({ error: 'Prompt requis' }, { status: 400 });

    let res;
    if (kind === 'image') {
      res = await base44.asServiceRole.integrations.Core.GenerateImage({ prompt });
      return Response.json({ kind: 'image', file_url: res?.url, prompt });
    }
    if (kind === 'audio') {
      const text = String(body?.text || prompt).slice(0, 5000);
      const voice = body?.voice || 'river';
      res = await base44.asServiceRole.integrations.Core.GenerateSpeech({ text, voice });
      return Response.json({ kind: 'audio', file_url: res?.url, prompt });
    }
    if (kind === 'video') {
      const aspect_ratio = body?.aspect_ratio === '9:16' ? '9:16' : '16:9';
      res = await base44.asServiceRole.integrations.Core.GenerateVideo({ prompt, aspect_ratio, generate_audio: true });
      return Response.json({ kind: 'video', file_url: res?.url, prompt });
```

```
    }
    return Response.json({ error: "kind inconnu (image|audio|video)" }, { status: 400 });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/generateReport/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const body = await req.json().catch(() => ({}));
    const domain = body?.domain || "finances";

    let columns = [], rows = [], filename = "rapport";
    const cell = (c) => { const s = String(c ?? ""); return /[;"\n]/.test(s) ? `"${s.replace(/"/g, '""')}"` : s; };

    if (domain === "finances") {
      const exp = await base44.entities.Expense.list("-date", 1000);
      columns = ["name", "category", "quantity", "unit_cost", "date"];
      rows = (exp || []).map((e) => [e.name || "", e.category || "", Number(e.quantity) || 0, Number(e.unit_cost) || 0, e.date ?
e.date.slice(0, 10) : ""]);
      filename = "depenses";
    } else if (domain === "stock") {
      const p = await base44.entities.Product.list("-created_date", 1000);
      columns = ["name", "sku", "category", "quantity", "unit_cost", "sale_price"];
      rows = (p || []).map((x) => [x.name || "", x.sku || "", x.category || "", Number(x.quantity) || 0, Number(x.unit_cost) ||
0, Number(x.sale_price) || 0]);
      filename = "stock";
    } else if (domain === "ventes") {
      const s = await base44.entities.Sale.list("-date", 1000);
      columns = ["item_name", "customer_name", "quantity", "unit_price", "date", "status"];
      rows = (s || []).map((x) => [x.item_name || "", x.customer_name || "", Number(x.quantity) || 0, Number(x.unit_price) || 0,
x.date ? x.date.slice(0, 10) : "", x.status || ""]);
      filename = "ventes";
    } else if (domain === "taches") {
      const t = await base44.entities.Task.list("-start", 1000);
      columns = ["title", "status", "priority", "start", "end"];
      rows = (t || []).map((x) => [x.title || "", x.status || "", x.priority || "", x.start ? x.start.slice(0, 10) : "", x.end ?
x.end.slice(0, 10) : ""]);
      filename = "taches";
    } else {
      return Response.json({ error: "Domaine inconnu" }, { status: 400 });
    }

    const csv = "\uFEFF" + columns.join(";") + "\n" + rows.map((r) => r.map(cell).join(";")).join("\n");
    const stamp = new Date().toISOString().slice(0, 10);
    const fname = `${filename}_${stamp}.csv`;
    const file = new File([csv], fname, { type: "text/csv;charset=utf-8;" });
    const res = await base44.asServiceRole.integrations.Core.UploadPublicFile({ file });
    return Response.json({ file_url: res?.file_url, filename: fname });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/manageGroup/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

// Gestion des membres d'un groupe (actions nécessitant un contournement de RLS).
// - accept_invite : l'invité s'ajoute au groupe et marque l'invitation acceptée
// - remove_member : un admin retire un membre, OU un membre se retire lui-même
// - add_member : un admin ajoute directement un membre
export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const body = await req.json().catch(() => ({}));
    const action = body.action;
    const Group = base44.asServiceRole.entities.Group;

    if (action === 'accept_invite') {
      const invite = await base44.entities.GroupInvite.get(body.invite_id);
      if (!invite) return Response.json({ error: 'Invitation introuvable' }, { status: 404 });
      if (invite.to_user_id !== user.id) return Response.json({ error: 'Non autorisé' }, { status: 403 });
      if (invite.status !== 'pending') return Response.json({ error: 'Invitation déjà traitée' }, { status: 400 });
      const group = await Group.get(invite.group_id);
      if (group) {
        const members = group.members || [];
        if (!members.includes(user.id)) {
          await Group.update(group.id, { members: [...members, user.id] });
```

```
        }
      }
      await base44.entities.GroupInvite.update(invite.id, { status: 'accepted' });
      return Response.json({ ok: true });
    }

    if (action === 'remove_member') {
      const group = await base44.entities.Group.get(body.group_id);
      if (!group) return Response.json({ error: 'Groupe introuvable' }, { status: 404 });
      const target = body.target_id;
      if (!target) return Response.json({ error: 'target_id requis' }, { status: 400 });
      const isSelf = target === user.id;
      const isAdmin = group.created_by_id === user.id || (group.admins || []).includes(user.id);
      if (!isSelf && !isAdmin) return Response.json({ error: 'Non autorisé' }, { status: 403 });
      await Group.update(body.group_id, {
        members: (group.members || []).filter((m) => m !== target),
        admins: (group.admins || []).filter((a) => a !== target),
        allowed_writers: (group.allowed_writers || []).filter((a) => a !== target),
        allowed_media: (group.allowed_media || []).filter((a) => a !== target),
      });
      return Response.json({ ok: true });
    }

    if (action === 'add_member') {
      const group = await base44.entities.Group.get(body.group_id);
      if (!group) return Response.json({ error: 'Groupe introuvable' }, { status: 404 });
      const isAdmin = group.created_by_id === user.id || (group.admins || []).includes(user.id);
      if (!isAdmin) return Response.json({ error: 'Non autorisé' }, { status: 403 });
      const members = group.members || [];
      if (!members.includes(body.user_id)) {
        await Group.update(body.group_id, { members: [...members, body.user_id] });
      }
      return Response.json({ ok: true });
    }

    return Response.json({ error: 'Action inconnue' }, { status: 400 });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/morningBriefing/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.52';

export default async function (req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const [prefs, tasks, events, expenses, savings, loans, taxes] = await Promise.all([
      base44.entities.Preferences.list().catch(() => []),
      base44.entities.Task.list("-start", 200).catch(() => []),
      base44.entities.Event.list("-start", 200).catch(() => []),
      base44.entities.Expense.list("-date", 200).catch(() => []),
      base44.entities.SavingsPlan.list("-created_date", 100).catch(() => []),
      base44.entities.Loan.list("-created_date", 100).catch(() => []),
      base44.entities.Tax.list("-created_date", 200).catch(() => []),
    ]);

    const pref = prefs?.[0] || {};
    const currency = pref.currency || 'XAF';
    const reminderDays = Number(pref.reminder_days) || 7;
    const budget = Number(pref.initial_budget) || 0;

    const now = new Date();
    const todayStr = now.toISOString().slice(0, 10);
    const horizon = new Date(now.getTime() + reminderDays * 86400000);

    const expenseAmt = (e) => (Number(e.quantity) || 1) * (Number(e.unit_cost) || 0);
    const totalExpense = (expenses || []).reduce((s, e) => s + expenseAmt(e), 0);
    const depensesJour = (expenses || []).filter((e) => e.date === todayStr).reduce((s, e) => s + expenseAmt(e), 0);

    const tasksToday = (tasks || []).filter((t) => (t.start || "").slice(0, 10) === todayStr && t.status !== "terminee");
    const tasksUpcoming = (tasks || []).filter((t) => {
      const d = (t.start || "").slice(0, 10);
      return d > todayStr && new Date(t.start) <= horizon && t.status !== "terminee";
    });
    const eventsToday = (events || []).filter((ev) => (ev.start || "").slice(0, 10) === todayStr);
    const eventsUpcoming = (events || []).filter((ev) => {
      const d = (ev.start || "").slice(0, 10);
      return d > todayStr && new Date(ev.start) <= horizon;
    });

    const loanBurden = (loans || []).filter((l) => l.status === "en_cours").reduce((s, l) => s + (Number(l.monthly_payment) ||
0), 0);
    const taxBurden = (taxes || []).filter((t) => t.scope === "jpm").reduce((s, t) => s + (Number(t.amount) || 0), 0);
    const reste = budget - totalExpense - loanBurden - taxBurden;
    const activePlans = (savings || []).filter((p) => p.status !== "archive").length;
```

```
    const kpis = {
      budget, totalExpense, reste, depensesJour, loanBurden, taxBurden, currency,
      tasksToday: tasksToday.length, tasksUpcoming: tasksUpcoming.length,
      eventsToday: eventsToday.length, eventsUpcoming: eventsUpcoming.length,
      activePlans,
    };

    let message = '';
    try {
      const prompt = `Tu es le Briefing Quotidien de JH Manager. Voici le résumé du jour pour un utilisateur. Rédige une
synthèse courte, structurée et motivante en français (max 180 mots), avec des puces.\nDonnées:\n- Tâches du jour:
${tasksToday.length}\n- Tâches à venir (${reminderDays}j): ${tasksUpcoming.length}\n- Événements du jour:
${eventsToday.length}\n- Dépenses du jour: ${depensesJour} ${currency}\n- Total dépenses: ${totalExpense} ${currency}\n- Budget:
${budget} ${currency}\n- Reste: ${reste} ${currency}`;
      const res = await base44.asServiceRole.integrations.Core.InvokeLLM({ prompt });
      message = typeof res === 'string' ? res : (res?.text || res?.response || JSON.stringify(res));
    } catch { /* ignore */ }

    return Response.json({
      date: todayStr,
      greeting: `Bonjour ${user.full_name || ''}`.trim(),
      kpis,
      message,
      tasksToday: tasksToday.map((t) => ({ title: t.title, priority: t.priority, start: t.start })),
      tasksUpcoming: tasksUpcoming.slice(0, 10).map((t) => ({ title: t.title, priority: t.priority, start: t.start })),
      eventsToday: eventsToday.map((ev) => ({ title: ev.title, start: ev.start, location: ev.location })),
      eventsUpcoming: eventsUpcoming.slice(0, 10).map((ev) => ({ title: ev.title, start: ev.start })),
      recentExpenses: (expenses || []).slice(0, 6).map((e) => ({ name: e.name, amount: expenseAmt(e), date: e.date })),
    });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/notifyEvent/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

const APP_URL = 'https://jhmanager.base44.app';

const REASON_LABELS = {
  spam: 'Spam',
  harcelement: 'Harcèlement',
  contenu_inapproprie: 'Contenu inapproprié',
  arnaque: 'Arnaque',
  usurpation: 'Usurpation',
  autre: 'Autre',
};

function esc(s) {
  return (s == null ? '' : String(s)).replace(/[&<>"]/g, (c) => ({ '&': '&', '<': '<', '>': '>', '"': '"' }[c]));
}

function link(label, path) {
  return `<a href="${APP_URL}${path}" style="background:#2f6b54;color:#fff;padding:10px
16px;border-radius:8px;text-decoration:none;display:inline-block;margin-top:12px">${label}</a>`;
}

function wrap(title, inner) {
  return `<div style="font-family:Inter,Arial,sans-serif;color:#1a2e2a;max-width:560px;margin:auto"><h2
style="color:#2f6b54">${title}</h2>${inner}<p style="color:#6b7280;font-size:12px;margin-top:24px">Notification automatique JH
Manager.</p></div>`;
}

export default async function (req) {
  try {
    const base44 = createClientFromRequest(req);
    const body = await req.json().catch(() => ({}));
    const type = body?.event_type;
    const recordId = body?.record_id;
    if (!type || !recordId) return Response.json({ sent: false, reason: 'missing_args' });

    const get = async (name) => {
      try { return await base44.asServiceRole.entities[name].get(recordId); }
      catch { return null; }
    };
    const allUsers = async () => {
      try { return await base44.asServiceRole.entities.User.list(); }
      catch { return []; }
    };
    const prefsEnabled = async (userId) => {
      try {
        const p = await base44.asServiceRole.entities.Preferences.filter({ created_by_id: userId });
        return p?.[0]?.notifications_enabled !== false;
      } catch { return true; }
    };
    const send = async (to, subject, html) => {
      if (!to) return false;
      try { await base44.asServiceRole.integrations.Core.SendEmail({ to, subject, html }); return true; }
      catch { return false; }
    };
```

```
    const displayName = async (userId) => {
      try {
        const c = await base44.asServiceRole.entities.Collaborator.filter({ created_by_id: userId });
        return c?.[0]?.display_name || '';
      } catch { return ''; }
    };

    // 1) Demande de collaboration reçue
    if (type === 'collab_request') {
      const r = await get('CollaborationRequest');
      if (!r) return Response.json({ sent: false });
      const users = await allUsers();
      const toUser = users.find((u) => u.id === r.to_user_id);
      if (!toUser || !(await prefsEnabled(toUser.id))) return Response.json({ sent: false, reason: 'disabled' });
      const fromName = (await displayName(r.created_by_id)) || 'Un utilisateur';
      const ok = await send(toUser.email, 'JH Manager — Nouvelle demande de collaboration',
        wrap('Nouvelle demande de collaboration',
          `<p><strong>${esc(fromName)}</strong> souhaite collaborer avec vous.</p>${link('Voir la demande',
'/collaborateurs')}`));
      return Response.json({ sent: ok, to: toUser.email });
    }

    // 2) Message reçu
    if (type === 'message') {
      const m = await get('Message');
      if (!m) return Response.json({ sent: false });
      const users = await allUsers();
      const senderName = (await displayName(m.created_by_id)) || users.find((u) => u.id === m.created_by_id)?.full_name || 'Un
utilisateur';
      const recipients = (m.participants || []).filter((id) => id !== m.created_by_id);
      let count = 0;
      for (const rid of recipients) {
        const u = users.find((x) => x.id === rid);
        if (!u || !(await prefsEnabled(u.id))) continue;
        const preview = m.content ? esc(m.content.slice(0, 300)) : (m.attachment_name ? `Fichier joint :
${esc(m.attachment_name)}` : 'Nouveau message');
        const ok = await send(u.email, 'JH Manager — Nouveau message',
          wrap(`Nouveau message de ${esc(senderName)}`, `<p>${preview}</p>${link('Ouvrir la conversation',
'/collaborateurs')}`));
        if (ok) count++;
      }
      return Response.json({ sent: count > 0, to: count });
    }

    // 3) Alertes admin : question FAQ, suggestion, signalement
    if (type === 'faq' || type === 'suggestion' || type === 'report') {
      const map = { faq: 'FaqQuestion', suggestion: 'Suggestion', report: 'Report' };
      const rec = await get(map[type]);
      if (!rec) return Response.json({ sent: false });
      const users = await allUsers();
      const admins = (users || []).filter((u) => u.role === 'admin');

      let subject, inner;
      if (type === 'faq') {
        subject = 'JH Manager — Nouvelle question FAQ';
        inner = `<p><strong>${esc(rec.question)}</strong></p><p style="color:#6b7280">Catégorie : ${esc(rec.category ||
'-')}</p>`;
      } else if (type === 'suggestion') {
        subject = 'JH Manager — Nouvelle suggestion';
        inner = `<p><strong>${esc(rec.content)}</strong></p><p style="color:#6b7280">Catégorie : ${esc(rec.category ||
'-')}</p>`;
      } else {
        subject = 'JH Manager — Nouveau signalement';
        const kind = rec.target_type === 'group' ? 'Groupe' : 'Utilisateur';
        inner = `<p><strong>${esc(rec.target_name || 'Cible inconnue')}</strong> (${kind})</p><p>Raison :
${esc(REASON_LABELS[rec.reason] || rec.reason || '-')}</p>${rec.custom_reason ? `<p>Détails : ${esc(rec.custom_reason)}</p>` :
''}`;
      }
      const html = wrap(subject, inner + link('Ouvrir le tableau de bord', '/admin'));

      let count = 0;
      for (const a of admins) {
        if (!(await prefsEnabled(a.id))) continue;
        if (await send(a.email, subject, html)) count++;
      }
      return Response.json({ sent: count > 0, to: count });
    }

    return Response.json({ sent: false, reason: 'unknown_type' });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/processNotes/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.52';

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
```

```
const user = await base44.auth.me();
if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

const body = await req.json().catch(() => ({}));
const note = String(body.note || '').trim();
if (!note) return Response.json({ error: 'Note vide' }, { status: 400 });
const scope = body.scope === 'jbm' ? 'jbm' : 'jpm';
const companyId = body.company_id ? String(body.company_id) : null;

const llm = await base44.asServiceRole.integrations.Core.InvokeLLM({
  prompt:
    'Analyse la note suivante rédigée par un utilisateur au sujet d\'un partenaire ou prestataire. ' +
    'Extrais les éléments actionnables : dépenses (finances), tâches (tâches), activités ou rendez-vous (événements). ' +
    'Pour chaque élément remplis les champs. Les dates doivent être au format ISO (YYYY-MM-DD ou YYYY-MM-DDTHH:mm:ss). ' +
    'Si une information est inconnue, mets null (jamais de texte inventé). Réponds uniquement via le schéma JSON. ' +
    'Note : """' + note + '"""',
  response_json_schema: {
    type: 'object',
    properties: {
      expenses: {
        type: 'array',
        items: {
          type: 'object',
          properties: {
            name: { type: 'string' },
            unit_cost: { type: 'number' },
            quantity: { type: 'number' },
            date: { type: 'string' }
          },
          required: ['name']
        }
      },
      tasks: {
        type: 'array',
        items: {
          type: 'object',
          properties: {
            title: { type: 'string' },
            start: { type: 'string' },
            end: { type: 'string' },
            duration_minutes: { type: 'number' }
          },
          required: ['title']
        }
      },
      events: {
        type: 'array',
        items: {
          type: 'object',
          properties: {
            title: { type: 'string' },
            start: { type: 'string' },
            end: { type: 'string' },
            location: { type: 'string' }
          },
          required: ['title']
        }
      },
      summary: { type: 'string' }
    },
    required: ['expenses', 'tasks', 'events', 'summary']
  }
});

const created = { expenses: 0, tasks: 0, events: 0 };

for (const e of (llm.expenses || [])) {
  try {
    await base44.entities.Expense.create({
      name: e.name,
      unit_cost: e.unit_cost || 0,
      quantity: e.quantity || 1,
      date: e.date || new Date().toISOString().slice(0, 10)
    });
    created.expenses++;
  } catch (err) {
    console.error('Expense create failed:', err.message);
  }
}
for (const tk of (llm.tasks || [])) {
  try {
    const start = tk.start || new Date().toISOString();
    const end = tk.end || new Date(new Date(start).getTime() + ((tk.duration_minutes || 60) * 60000)).toISOString();
    await base44.entities.Task.create({ title: tk.title, start, end });
    created.tasks++;
  } catch (err) {
    console.error('Task create failed:', err.message);
  }
}
for (const ev of (llm.events || [])) {
  try {
    const start = ev.start || new Date().toISOString();
```

```
        const end = ev.end || new Date(new Date(start).getTime() + 3600000).toISOString();
        await base44.entities.Event.create({
          title: ev.title,
          start,
          end,
          location: ev.location || null,
          company_id: scope === 'jbm' ? companyId : null
        });
        created.events++;
      } catch (err) {
        console.error('Event create failed:', err.message);
      }
    }

    return Response.json({ success: true, summary: llm.summary, created });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/processPayment/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

const PAYMENT_ENDPOINT = 'https://backend-paiement-gabon.onrender.com/api/payments/pay';
const VALID_ACCOUNT_TYPES = ['particulier', 'entreprise'];
const VALID_PROVIDERS = ['airtel', 'moov'];
const TEST_EMAILS = ['jhmendome@gmail.com', 'obiangmbarosly4@gmail.com'];

// --- Vérification du statut de la transaction (défensive) ---
// Le contrat exact du backend externe n'étant pas connu, on inspecte la réponse
// de /pay puis on tente, si une référence est disponible, d'interroger des routes
// de vérification candidates. On ne bloque QUE les échecs explicites ; en cas
// d'ambiguïté (route absente ou statut pending non résolu), on conserve le
// comportement actuel (activation) pour ne pas pénaliser les paiements valides.
const SUCCESS_KW = ['success', 'succeeded', 'successful', 'completed', 'complete', 'paid', 'confirmed', 'ok', 'settled'];
const PENDING_KW = ['pending', 'processing', 'in_progress', 'waiting', 'initiated', 'submitted', 'ongoing', 'en_attente'];
const FAILED_KW = ['failed', 'error', 'declined', 'rejected', 'cancelled', 'canceled', 'insufficient', 'timeout', 'expired',
'abandoned', 'echoue'];

const STATUS_ROUTE_FNS = [
  (r) => `/api/payments/status?reference=${encodeURIComponent(r)}`,
  (r) => `/api/payments/verify?reference=${encodeURIComponent(r)}`,
  (r) => `/api/payments/check?reference=${encodeURIComponent(r)}`,
  (r) => `/api/payments/transaction/${encodeURIComponent(r)}`,
  (r) => `/api/payments/${encodeURIComponent(r)}`,
];

function extractStatusText(obj) {
  if (!obj || typeof obj !== 'object') return '';
  const vals = [];
  for (const k of ['status', 'state', 'transaction_status', 'payment_status', 'result', 'code', 'message']) {
    if (obj[k] != null) vals.push(String(obj[k]));
  }
  for (const k of ['data', 'transaction', 'payment', 'response', 'result']) {
    if (obj[k] && typeof obj[k] === 'object') {
      for (const sk of ['status', 'state', 'payment_status', 'transaction_status']) {
        if (obj[k][sk] != null) vals.push(String(obj[k][sk]));
      }
    }
  }
  return vals.join(' ').toLowerCase();
}

function classifyStatus(text) {
  if (!text) return 'unknown';
  if (FAILED_KW.some((k) => text.includes(k))) return 'failed';
  if (PENDING_KW.some((k) => text.includes(k))) return 'pending';
  if (SUCCESS_KW.some((k) => text.includes(k))) return 'confirmed';
  return 'unknown';
}

async function verifyPaymentStatus(reference, initialData) {
  const initial = classifyStatus(extractStatusText(initialData));
  if (initial === 'confirmed') return { confirmed: true, failed: false, source: 'pay_response', note: 'Succès détecté dans la
réponse /pay' };
  if (initial === 'failed') return { confirmed: false, failed: true, source: 'pay_response', note: 'Échec détecté dans la
réponse /pay' };
  if (!reference) return { confirmed: false, failed: false, source: 'none', note: 'Aucune référence pour vérifier le statut' };

  const base = PAYMENT_ENDPOINT.replace(/\/api\/payments\/pay$/, '');
  for (let attempt = 0; attempt < 4; attempt++) {
    if (attempt > 0) await new Promise((res) => setTimeout(res, 2500));
    let anyRouteResponded = false; // une route qui n'est pas un 404/erreur
    for (const routeFn of STATUS_ROUTE_FNS) {
      try {
        const r = await fetch(base + routeFn(reference), { method: 'GET', headers: { Accept: 'application/json' } });
        if (r.status === 404) continue; // route inexistante : on essaie la suivante
        if (!r.ok) { anyRouteResponded = true; continue; }
        anyRouteResponded = true;
        const j = await r.json().catch(() => null);
```

```
        const cls = classifyStatus(extractStatusText(j));
        if (cls === 'confirmed') return { confirmed: true, failed: false, source: 'status_route', note: 'Succès confirmé par la
route de vérification' };
        if (cls === 'failed') return { confirmed: false, failed: true, source: 'status_route', note: 'Échec confirmé par la
route de vérification' };
        // pending ou unknown : on continue à sonder les autres routes
      } catch {
        // route indisponible ou réponse illisible : on essaie la suivante
      }
    }
    // Si aucune route de statut n'existe (tout en 404/erreur), inutile de
    // re-sonder : on s'arrête immédiatement pour ne pas ralentir le paiement.
    if (!anyRouteResponded) break;
  }
  return { confirmed: false, failed: false, source: 'timeout', note: 'Route de statut indisponible ou statut ambigu — activation
conservée par défaut' };
}

function flattenGatewayError(data) {
  if (!data) return 'réponse illisible';
  const cand = [];
  const push = (v) => { if (v != null && v !== '') cand.push(String(v)); };
  push(data.message);
  push(data.error_description);
  push(data.detail);
  if (typeof data.error === 'string') push(data.error);
  else if (data.error && typeof data.error === 'object') {
    push(data.error.error_description);
    push(data.error.message);
    push(data.error.error);
  }
  const txt = cand.filter(Boolean).join(' — ').trim();
  return txt || 'erreur inconnue';
}

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });

    const body = await req.json().catch(() => ({}));
    const accountName = String(body.account_name || '').trim();
    const phoneNumber = String(body.phoneNumber || '').trim();
    const provider = String(body.provider || '').trim();
    const accountType = String(body.account_type || '').trim();

    // Pricing is computed server-side to prevent client tampering.
    // Particulier (JPM): flat 5 000 XAF.
    // Entreprise (JBM): 20 000 XAF per managed company (1 = 20 000, 2 = 40 000, 3 = 60 000...).
    const companiesCount =
      accountType === 'entreprise' ? Math.max(1, Math.floor(Number(body.companies_count) || 1)) : 1;
    // Lors d'un ajout d'entreprise (mise à niveau), on ne facture que les entreprises
    // nouvellement couvertes : celles déjà payées (already_covered) ne sont pas re-débitées.
    // Le total de l'abonnement reste 20 000 × nombre d'entreprises couvertes.
    const alreadyCovered =
      accountType === 'entreprise' ? Math.max(0, Math.floor(Number(body.already_covered) || 0)) : 0;
    const billableUnits =
      accountType === 'entreprise' ? Math.max(0, companiesCount - alreadyCovered) : 1;
    let normalAmount;
    if (accountType === 'entreprise') {
      normalAmount = 20000 * billableUnits;
    } else {
      normalAmount = 5000;
    }

    // Affiliation : détermine si l'utilisateur est parrainé et s'il bénéficie de
    // la réduction de 20 % valable les 6 premiers mois.
    const refCode = String(body.referral_code || '').trim();
    let affiliate = null;
    let isReferred = false;
    if (refCode) {
      const affs = await base44.asServiceRole.entities.Affiliate.filter(
        { referral_code: refCode }, null, 1
      );
      affiliate = affs?.[0];
      if (affiliate && affiliate.created_by_id !== user.id) isReferred = true;
    }
    // Même si le code n'est plus en localStorage, on détecte un parrainage antérieur.
    if (!isReferred) {
      const prior = await base44.asServiceRole.entities.Commission.filter(
        { referred_user_email: user.email || '' }, null, 1
      );
      if (prior && prior.length) isReferred = true;
    }

    let discount = false;
    if (isReferred) {
      const priorSubs = await base44.asServiceRole.entities.Subscription.filter(
        { created_by_id: user.id }, 'paid_at', 500
      );
      let firstTs = Date.now();
```

```
      for (const s of (priorSubs || [])) {
        const t = new Date(s.paid_at || s.created_date).getTime();
        if (t && t < firstTs) firstTs = t;
      }
      const sixMonthsMs = 6 * 30 * 24 * 60 * 60 * 1000;
      if (!priorSubs?.length || Date.now() - firstTs < sixMonthsMs) discount = true;
    }
    const amount = discount ? Math.round(normalAmount * 0.8) : normalAmount;

    const isTest = body.test === true || String(body.test || '') === 'true';

    if (!VALID_ACCOUNT_TYPES.includes(accountType)) {
      return Response.json({ error: 'Type de compte invalide' }, { status: 400 });
    }

    // Mode test : réservé aux adresses autorisées. Aucun débit Mobile Money réel.
    let data = null;
    if (isTest) {
      const email = (user.email || '').trim().toLowerCase();
      if (!TEST_EMAILS.includes(email)) {
        return Response.json({ error: 'Mode test non autorisé pour ce compte' }, { status: 403 });
      }
      data = { test: true, reference: 'TEST-' + Date.now() };
    } else {
      if (!phoneNumber) {
        return Response.json({ error: 'Numéro de téléphone requis' }, { status: 400 });
      }
      if (!VALID_PROVIDERS.includes(provider)) {
        return Response.json({ error: 'Opérateur invalide' }, { status: 400 });
      }
      // Credentials Airtel transmis à la passerelle (OAuth2 client_credentials).
      // Renseignés via les secrets de l'app : PAYMENT_CLIENT_ID / PAYMENT_CLIENT_SECRET.
      const client_id = process.env.PAYMENT_CLIENT_ID;
      const client_secret = process.env.PAYMENT_CLIENT_SECRET;
      if (!client_id || !client_secret) {
        return Response.json(
          {
            success: false,
            error: 'Service de paiement non configuré : credentials de la passerelle manquants (PAYMENT_CLIENT_ID /
PAYMENT_CLIENT_SECRET).',
          },
          { status: 200 }
        );
      }
      const apiRes = await fetch(PAYMENT_ENDPOINT, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ amount, phoneNumber, provider, client_id, client_secret }),
      });
      data = await apiRes.json().catch(() => null);
      if (!apiRes.ok) {
        // On renvoie 200 + success:false pour que le frontend affiche la cause
        // réelle (au lieu d'un "Request failed with status code 500" générique).
        return Response.json(
          {
            success: false,
            error: 'Paiement refusé par la passerelle (' + flattenGatewayError(data) + '). Le service Mobile Money n\'est pas
correctement configuré.',
            gateway_status: apiRes.status,
            details: data,
          },
          { status: 200 }
        );
      }
    }

    // --- Vérification du statut de la transaction ---
    // En mode test, aucun débit réel : on considère le paiement comme confirmé.
    // En mode réel, on inspecte la réponse et, si possible, une route de statut.
    let verification = { confirmed: true, failed: false, source: 'test', note: 'Mode test — aucun débit réel' };
    if (!isTest) {
      const ref = (data && (data.reference || data.transactionId || data.id || data.paymentId)) || phoneNumber;
      verification = await verifyPaymentStatus(ref, data);
      if (verification.failed) {
        return Response.json(
          { success: false, error: 'Paiement non confirmé par la passerelle', verification, details: data },
          { status: 200 }
        );
      }
    }

    // Record the subscription server-side (service role bypasses RLS) so revenue is trustworthy.
    let subscription = null;
    try {
      subscription = await base44.asServiceRole.entities.Subscription.create({
        account_type: accountType,
        amount,
        companies_count: accountType === 'entreprise' ? companiesCount : 1,
        account_name: accountName || null,
        phone_number: phoneNumber || 'TEST',
        provider: isTest ? (VALID_PROVIDERS.includes(provider) ? provider : 'airtel') : provider,
        status: 'payee',
```

```
        paid_at: new Date().toISOString(),
        // La référence passerelle est conservée si présente ; à défaut, on ancre
        // l'identification sur le numéro Mobile Money (unique par compte), qui
        // devient l'identifiant de l'abonnement — les renouvellements restent
        // distincts via l'horodatage (paid_at / created_date).
        reference:
          (data && (data.reference || data.transactionId || data.id || data.paymentId)) || phoneNumber || null,
      });
    } catch (recErr) {
      // Payment succeeded; recording failure must not fail the whole request.
      console.error('Subscription record failed:', recErr.message);
    }

    // Affiliation : 20 % de commission calculés sur le prix NORMAL (non réduit).
    // Le parrain touche toujours 1 000 XAF/mois (particulier) ou 4 000 XAF/mois
    // (entreprise, × nombre d'entreprises), quelle que soit la réduction du filleul.
    try {
      if (affiliate && affiliate.created_by_id !== user.id) {
        const commission = Math.round(normalAmount * 0.2);
        await base44.asServiceRole.entities.Commission.create({
          affiliate_id: affiliate.id,
          affiliate_owner_id: affiliate.created_by_id,
          referred_user_email: user.email || null,
          source_amount: normalAmount,
          amount: commission,
          account_type: accountType,
          paid_at: new Date().toISOString(),
        });
      }
    } catch (affErr) {
      console.error('Affiliation commission failed:', affErr.message);
    }

    return Response.json({ success: true, data, subscription, verification });
  } catch (error) {
    // Erreur interne : on renvoie 200 + success:false pour que le frontend
    // affiche un message lisible au lieu d'un 500 générique masqué par axios.
    return Response.json({ success: false, error: 'Erreur technique lors du paiement : ' + (error && error.message || 'erreur
inconnue') }, { status: 200 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/resubscriptionStatus/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

export default async function (req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Non autorisé' }, { status: 401 });

    const email = (user.email || '').toLowerCase().trim();
    if (!email) return Response.json({ blocked: false, status: 'active' });

    const deleted = await base44.asServiceRole.entities.DeletedAccount.filter({ email });
    if (!deleted || deleted.length === 0) {
      return Response.json({ blocked: false, status: 'active' });
    }

    const reqs = await base44.asServiceRole.entities.ReSubscriptionRequest.filter({ email }, '-created_date', 1);
    const latest = (reqs && reqs[0]) || null;
    const status = latest?.status || 'none';
    const blocked = status !== 'approved';

    return Response.json({
      blocked,
      status,
      requestId: latest?.id || null,
      message: latest?.message || '',
    });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/scheduledSalesAlert/entry.ts
------------------------------------------------------------
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.52';

const APP_URL = 'https://jhmanager.base44.app';

function esc(s) {
  return (s == null ? '' : String(s)).replace(/[&<>"]/g, (c) => ({ '&': '&', '<': '<', '>': '>', '"': '"' }[c]));
}

function fmtDate(d) {
  return new Date(d).toLocaleDateString('fr-FR');
}
```

```
function wrap(title, inner) {
  return `<div style="font-family:Inter,Arial,sans-serif;color:#1a2e2a;max-width:560px;margin:auto"><h2
style="color:#2f6b54">${title}</h2>${inner}<p style="color:#6b7280;font-size:12px;margin-top:24px">Rappel automatique JH Manager
— ventes programmées.</p></div>`;
}

// Rappel J-1 et jour J pour les ventes programmées (status en_attente + scheduled).
export default async function (req) {
  try {
    const base44 = createClientFromRequest(req);
    const sales = await base44.asServiceRole.entities.Sale.list("-date", 2000);
    const users = await base44.asServiceRole.entities.User.list();

    const todayStr = new Date().toISOString().slice(0, 10);
    const tomorrow = new Date(Date.now() + 86400000);
    const tomorrowStr = tomorrow.toISOString().slice(0, 10);

    const upcoming = (sales || []).filter((s) =>
      s.scheduled && s.status === "en_attente" && (s.date === todayStr || s.date === tomorrowStr)
    );

    // Group by owner
    const byOwner = {};
    for (const s of upcoming) {
      const k = s.created_by_id;
      if (!k) continue;
      if (!byOwner[k]) byOwner[k] = [];
      byOwner[k].push(s);
    }

    let sent = 0;
    for (const [ownerId, items] of Object.entries(byOwner)) {
      const u = users.find((x) => x.id === ownerId);
      if (!u || !u.email) continue;
      try {
        const prefs = await base44.asServiceRole.entities.Preferences.filter({ created_by_id: ownerId });
        if (prefs?.[0]?.notifications_enabled === false) continue;
      } catch { /* ignore */ }

      const rows = items.map((s) => {
        const when = s.date === todayStr ? "Aujourd'hui" : "Demain";
        const total = (Number(s.quantity) || 0) * (Number(s.unit_price) || 0);
        return `<tr><td style="padding:6px 8px">${esc(when)}</td><td style="padding:6px 8px">${esc(s.item_name)}</td><td
style="padding:6px 8px">${esc(s.customer_name || '-')}</td><td style="padding:6px 8px;text-align:right">${Number(s.quantity) ||
0} × ${Number(s.unit_price) || 0} = ${total}</td></tr>`;
      }).join("");
      const html = wrap("Rappel : ventes programmées",
        `<p>Voici les ventes programmées à venir :</p>` +
        `<table style="border-collapse:collapse;width:100%;font-size:13px"><thead><tr style="background:#eef5f1"><th
style="padding:6px 8px;text-align:left">Quand</th><th style="padding:6px 8px;text-align:left">Article</th><th style="padding:6px
8px;text-align:left">Client</th><th style="padding:6px
8px;text-align:right">Montant</th></tr></thead><tbody>${rows}</tbody></table>` +
        `<p style="margin-top:12px"><a href="${APP_URL}/jbm/ventes" style="background:#2f6b54;color:#fff;padding:10px
16px;border-radius:8px;text-decoration:none;display:inline-block">Voir les ventes</a></p>`
      );
      try {
        await base44.asServiceRole.integrations.Core.SendEmail({
          to: u.email,
          subject: "JH Manager — Rappel de ventes programmées",
          html,
        });
        sent++;
      } catch { /* ignore per-user failure */ }
    }

    return Response.json({ sent, checked: upcoming.length });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/functions/triageFaq/entry.ts
------------------------------------------------------------
import { triageFaqs } from "../../shared/faqTriage.ts";

import { createClientFromRequest } from 'npm:@base44/sdk@0.8.49';

export default async function(req) {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401 });
    if (user.role !== 'admin') return Response.json({ error: 'Forbidden' }, { status: 403 });

    // Charge toutes les questions non triées (sans group_label).
    const all = await base44.asServiceRole.entities.FaqQuestion.list('-created_date', 500);
    const fresh = (all || []).filter((q) => !q.group_label);

    if (!fresh.length) {
      return Response.json({ message: 'Rien à trier', grouped: 0 });
    }
```

```
    // Demande à l'IA de regrouper les questions similaires.
    const res = await base44.asServiceRole.integrations.Core.InvokeLLM({
      prompt: `Voici des questions posées par les utilisateurs sur une app SaaS (JM : JPM gestion perso, JBM gestion
entreprise).
Pour CHAQUE question, attribue un "group_key" court (slug anglais, ex: "price_jbm", "mobile_money_payment", "affiliate_payout")
et un "group_label" clair en français (ex: "Tarif abonnement entreprise", "Paiement Mobile Money").
Regroupe sous le MÊME group_key/group_label les questions qui traitent du même sujet, pour éviter les répétitions.
Réponds en JSON strict conforme au schéma.`,
      response_json_schema: {
        type: 'object',
        properties: {
          items: {
            type: 'array',
            items: {
              type: 'object',
              properties: {
                question: { type: 'string' },
                group_key: { type: 'string' },
                group_label: { type: 'string' },
              },
              required: ['question', 'group_key', 'group_label'],
            },
          },
        },
        required: ['items'],
      },
      model: 'gemini_3_8_flash',
    });

    const mapping = triageFaqs(res, fresh);

    let grouped = 0;
    for (const m of mapping) {
      try {
        await base44.asServiceRole.entities.FaqQuestion.update(m.id, {
          group_key: m.group_key,
          group_label: m.group_label,
          status: 'en_cours',
        });
        grouped++;
      } catch (e) { /* ignore single failure */ }
    }

    return Response.json({ message: 'Tri terminé', grouped });
  } catch (error) {
    return Response.json({ error: error.message }, { status: 500 });
  }
}

------------------------------------------------------------
FICHIER : ./base44/shared/faqTriage.ts
------------------------------------------------------------
// Logique partagée de tri/regroupement des questions FAQ par l'IA.
// Garde les réponses robustes : si l'IA renvoie n'importe quoi, on retombe
// sur un regroupement par mot-clé simple pour ne jamais perdre les questions.

export function triageFaqs(llmResult, fresh) {
  const items = Array.isArray(llmResult?.items) ? llmResult.items : [];
  // Index par question (texte) pour faire correspondre la sortie de l'IA.
  const byText = {};
  items.forEach((it) => {
    if (it && it.question) {
      byText[normalize(it.question)] = it;
    }
  });

  return fresh.map((q) => {
    const match = byText[normalize(q.question)];
    if (match && match.group_key && match.group_label) {
      return {
        id: q.id,
        group_key: slug(match.group_key),
        group_label: String(match.group_label).slice(0, 80),
      };
    }
    // Fallback : regroupement par mot-clé simple.
    return { id: q.id, group_key: keywordGroup(q.question), group_label: keywordLabel(q.question) };
  });
}

function normalize(s) {
  return String(s || '').toLowerCase().trim().replace(/ /g, ' ');
}

function slug(s) {
  return String(s || '').toLowerCase().trim().replace(/[^a-z0-9]+/g, '_').replace(/^_+|_+$/g, '').slice(0, 40) || 'divers';
}

const KEYWORDS = [
  { keys: ['prix', 'tarif', 'combien', 'coût', 'cout', 'abonnement'], key: 'tarif', label: 'Tarif & abonnement' },
  { keys: ['paiement', 'payer', 'airtel', 'moov', 'mobile money', 'transaction'], key: 'paiement', label: 'Paiement Mobile
```

```
Money' },
  { keys: ['affili', 'parrain', 'commission', 'filleul', 'lien'], key: 'affiliation', label: 'Affiliation & commissions' },
  { keys: ['entreprise', 'jbm', 'stock', 'vente', 'facture'], key: 'jbm', label: 'Espace entreprise (JBM)' },
  { keys: ['personnel', 'jpm', 'budget', 'tache', 'planning'], key: 'jpm', label: 'Espace personnel (JPM)' },
  { keys: ['compte', 'connexion', 'inscript', 'mot de passe', 'email'], key: 'compte', label: 'Compte & connexion' },
];

function keywordGroup(q) {
  const s = String(q || '').toLowerCase();
  for (const k of KEYWORDS) if (k.keys.some((w) => s.includes(w))) return k.key;
  return 'divers';
}
function keywordLabel(q) {
  const s = String(q || '').toLowerCase();
  for (const k of KEYWORDS) if (k.keys.some((w) => s.includes(w))) return k.label;
  return 'Autres questions';
}

------------------------------------------------------------
FICHIER : ./base44/workflows/Alerte Collaboration.jsonc
------------------------------------------------------------
{
  "trigger": {
    "condition": null,
    "config": {
      "trigger_type": "entity",
      "events": [
        "create"
      ],
      "entity_name": "CollaborationRequest"
    }
  },
  "name": "Alerte Collaboration",
  "definition": {
    "document": {
      "dsl": "1.0.0",
      "name": "alerte_collaboration",
      "version": "1.0",
      "namespace": "base44"
    },
    "do": [
      {
        "notifier": {
          "call": "invoke_backend_function",
          "with": {
            "function_name": "notifyEvent",
            "args": {
              "event_type": "collab_request",
              "record_id": "${ .trigger.entity_id }"
            }
          },
          "then": "end",
          "x-base44": {
            "title": "Notifier la demande",
            "description": "Email au destinataire"
          }
        }
      }
    ]
  },
  "description": "Envoie un email au destinataire lors d'une nouvelle demande de collaboration."
}

------------------------------------------------------------
FICHIER : ./base44/workflows/Alerte FAQ.jsonc
------------------------------------------------------------
{
  "trigger": {
    "condition": null,
    "config": {
      "trigger_type": "entity",
      "events": [
        "create"
      ],
      "entity_name": "FaqQuestion"
    }
  },
  "name": "Alerte FAQ",
  "definition": {
    "document": {
      "dsl": "1.0.0",
      "name": "alerte_faq",
      "version": "1.0",
      "namespace": "base44"
    },
    "do": [
      {
        "notifier": {
          "call": "invoke_backend_function",
          "with": {
            "function_name": "notifyEvent",
            "args": {
```

```
              "event_type": "faq",
              "record_id": "${ .trigger.entity_id }"
            }
          },
          "then": "end",
          "x-base44": {
            "title": "Notifier la question",
            "description": "Email aux admins"
          }
        }
      }
    ]
  },
  "description": "Notifie les administrateurs par email lors d'une nouvelle question FAQ."
}

------------------------------------------------------------
FICHIER : ./base44/workflows/Alerte Message.jsonc
------------------------------------------------------------
{
  "trigger": {
    "condition": null,
    "config": {
      "trigger_type": "entity",
      "events": [
        "create"
      ],
      "entity_name": "Message"
    }
  },
  "name": "Alerte Message",
  "definition": {
    "document": {
      "dsl": "1.0.0",
      "name": "alerte_message",
      "version": "1.0",
      "namespace": "base44"
    },
    "do": [
      {
        "notifier": {
          "call": "invoke_backend_function",
          "with": {
            "function_name": "notifyEvent",
            "args": {
              "event_type": "message",
              "record_id": "${ .trigger.entity_id }"
            }
          },
          "then": "end",
          "x-base44": {
            "title": "Notifier le message",
            "description": "Email aux participants"
          }
        }
      }
    ]
  },
  "description": "Envoie un email aux participants (hors exp\u00e9diteur) lors d'un nouveau message."
}

------------------------------------------------------------
FICHIER : ./base44/workflows/Alerte Signalement.jsonc
------------------------------------------------------------
{
  "trigger": {
    "condition": null,
    "config": {
      "trigger_type": "entity",
      "events": [
        "create"
      ],
      "entity_name": "Report"
    }
  },
  "name": "Alerte Signalement",
  "definition": {
    "document": {
      "dsl": "1.0.0",
      "name": "alerte_signalement",
      "version": "1.0",
      "namespace": "base44"
    },
    "do": [
      {
        "notifier": {
          "call": "invoke_backend_function",
          "with": {
            "function_name": "notifyEvent",
            "args": {
              "event_type": "report",
              "record_id": "${ .trigger.entity_id }"
```

```
            }
          },
          "then": "end",
          "x-base44": {
            "title": "Notifier le signalement",
            "description": "Email aux admins"
          }
        }
      }
    ]
  },
  "description": "Notifie les administrateurs par email lors d'un nouveau signalement (abonn\u00e9 ou groupe)."
}

------------------------------------------------------------
FICHIER : ./base44/workflows/Alerte Suggestion.jsonc
------------------------------------------------------------
{
  "trigger": {
    "condition": null,
    "config": {
      "trigger_type": "entity",
      "events": [
        "create"
      ],
      "entity_name": "Suggestion"
    }
  },
  "name": "Alerte Suggestion",
  "definition": {
    "document": {
      "dsl": "1.0.0",
      "name": "alerte_suggestion",
      "version": "1.0",
      "namespace": "base44"
    },
    "do": [
      {
        "notifier": {
          "call": "invoke_backend_function",
          "with": {
            "function_name": "notifyEvent",
            "args": {
              "event_type": "suggestion",
              "record_id": "${ .trigger.entity_id }"
            }
          },
          "then": "end",
          "x-base44": {
            "title": "Notifier la suggestion",
            "description": "Email aux admins"
          }
        }
      }
    ]
  },
  "description": "Notifie les administrateurs par email lors d'une nouvelle suggestion."
}

------------------------------------------------------------
FICHIER : ./base44/workflows/Alertes Email.jsonc
------------------------------------------------------------
{
  "description": "Envoie deux fois par jour un r\u00e9sum\u00e9 par email des t\u00e2ches urgentes (\u00e0 \u00e9ch\u00e9ance)
et des d\u00e9penses importantes.",
  "definition": {
    "document": {
      "dsl": "1.0.0",
      "name": "alertes_email",
      "version": "1.0",
      "namespace": "base44"
    },
    "do": [
      {
        "send_alerts": {
          "call": "invoke_backend_function",
          "with": {
            "function_name": "emailAlerts"
          },
          "then": "end",
          "x-base44": {
            "title": "Envoyer les alertes",
            "description": "T\u00e2ches urgentes & d\u00e9penses importantes"
          }
        }
      }
    ]
  },
  "trigger": {
    "condition": null,
    "config": {
      "trigger_type": "scheduled",
```

```
      "events": [],
      "schedule_mode": "recurring",
      "cron_expression": "0 8,18 * * *",
      "one_time_date": null,
      "timezone": "UTC",
      "interval_value": null,
      "interval_unit": null,
      "interval_anchor": null,
      "ends_type": "never",
      "ends_on_date": null,
      "ends_after_count": null
    }
  },
  "name": "Alertes Email"
}

------------------------------------------------------------
FICHIER : ./base44/workflows/Rappel Ventes Programmées.jsonc
------------------------------------------------------------
{
  "name": "Rappel Ventes Programm\u00e9es",
  "definition": {
    "document": {
      "dsl": "1.0.0",
      "name": "rappel_ventes_programmees",
      "version": "1.0",
      "namespace": "base44"
    },
    "do": [
      {
        "rappeler": {
          "call": "invoke_backend_function",
          "with": {
            "function_name": "scheduledSalesAlert"
          },
          "then": "end",
          "x-base44": {
            "title": "Rappel ventes programm\u00e9es",
            "description": "Envoie les rappels J-1 et J"
          }
        }
      }
    ]
  },
  "trigger": {
    "condition": null,
    "config": {
      "trigger_type": "scheduled",
      "events": [],
      "schedule_mode": "recurring",
      "cron_expression": "0 7 * * *",
      "one_time_date": null,
      "timezone": "Africa/Libreville",
      "interval_value": null,
      "interval_unit": null,
      "interval_anchor": null,
      "ends_type": "never",
      "ends_on_date": null,
      "ends_after_count": null
    }
  },
  "description": "Rappelle chaque matin (J-1 et jour J) les ventes programm\u00e9es en attente."
}

------------------------------------------------------------
FICHIER : ./components.json
------------------------------------------------------------
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": false,
  "tsx": false,
  "tailwind": {
    "config": "tailwind.config.js",
    "css": "src/index.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "iconLibrary": "lucide"
}

------------------------------------------------------------
FICHIER : ./eslint.config.js
------------------------------------------------------------
```

```
import globals from "globals";
import pluginJs from "@eslint/js";
import pluginReact from "eslint-plugin-react";
import pluginReactHooks from "eslint-plugin-react-hooks";
import pluginUnusedImports from "eslint-plugin-unused-imports";

export default [
  {
    files: [
      "src/components/**/*.{js,mjs,cjs,jsx}",
      "src/pages/**/*.{js,mjs,cjs,jsx}",
      "src/Layout.jsx",
    ],
    ignores: ["src/lib/**/*", "src/components/ui/**/*"],
    ...pluginJs.configs.recommended,
    ...pluginReact.configs.flat.recommended,
    languageOptions: {
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 2022,
        sourceType: "module",
        ecmaFeatures: {
          jsx: true,
        },
      },
    },
    settings: {
      react: {
        version: "detect",
      },
    },
    plugins: {
      react: pluginReact,
      "react-hooks": pluginReactHooks,
      "unused-imports": pluginUnusedImports,
    },
    rules: {
      "no-unused-vars": "off",
      "react/jsx-uses-vars": "error",
      "react/jsx-uses-react": "error",
      "unused-imports/no-unused-imports": "error",
      "unused-imports/no-unused-vars": [
        "warn",
        {
          vars: "all",
          varsIgnorePattern: "^_",
          args: "after-used",
          argsIgnorePattern: "^_",
        },
      ],
      "react/prop-types": "off",
      "react/react-in-jsx-scope": "off",
      "react/no-unknown-property": [
        "error",
        { ignore: ["cmdk-input-wrapper", "toast-close"] },
      ],
      "react-hooks/rules-of-hooks": "error",
    },
  },
];


------------------------------------------------------------
FICHIER : ./index.html
------------------------------------------------------------
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="https://base44.com/logo_v2.svg" />
    <link rel="apple-touch-icon" href="https://base44.com/logo_v2.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover" />
    <meta name="theme-color" content="#1f3a2e" />
    <meta name="description" content="JH Manager — gestion tout-en-un pour particuliers (JH Personal Manager : finances, tâches,
planning) et entreprises (JH Business Manager : ventes, stock, personnel, devis)." />
    <meta name="apple-mobile-web-app-capable" content="yes" />
    <meta name="mobile-web-app-capable" content="yes" />
    <meta name="apple-mobile-web-app-status-bar-style" content="default" />
    <meta name="apple-mobile-web-app-title" content="JH Manager" />
    <meta property="og:title" content="JH Manager" />
    <meta property="og:description" content="Gestion personnelle et d'entreprise tout-en-un : JH Personal Manager + JH Business
Manager." />
    <meta property="og:type" content="website" />
    <link rel="manifest" href="/manifest.json" />
    <title>JH Manager — Gestion tout-en-un</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

```
------------------------------------------------------------
FICHIER : ./jsconfig.json
------------------------------------------------------------
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    },
    "jsx": "react-jsx",
    "module": "esnext",
    "moduleResolution": "bundler",
    "lib": ["esnext", "dom"],
    "target": "esnext",
    "checkJs": true,
    "skipLibCheck": true,
    "allowSyntheticDefaultImports": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "types": []
  },
  "include": ["src/components/**/*.js", "src/pages/**/*.jsx", "src/Layout.jsx"],
  "exclude": ["node_modules", "dist", "src/vite-plugins", "src/components/ui", "src/api", "src/lib"]
}

------------------------------------------------------------
FICHIER : ./package.json
------------------------------------------------------------
{
  "name": "base44-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint . --quiet",
    "lint:fix": "eslint . --fix",
    "typecheck": "tsc -p ./jsconfig.json",
    "preview": "vite preview"
  },
  "dependencies": {
    "@base44/sdk": "^0.8.52",
    "@base44/vite-plugin": "^1.0.44",
    "@hello-pangea/dnd": "^17.0.0",
    "@hookform/resolvers": "^4.1.2",
    "@radix-ui/react-accordion": "^1.2.3",
    "@radix-ui/react-alert-dialog": "^1.1.6",
    "@radix-ui/react-aspect-ratio": "^1.1.2",
    "@radix-ui/react-avatar": "^1.1.3",
    "@radix-ui/react-checkbox": "^1.1.4",
    "@radix-ui/react-collapsible": "^1.1.3",
    "@radix-ui/react-context-menu": "^2.2.6",
    "@radix-ui/react-dialog": "^1.1.6",
    "@radix-ui/react-dropdown-menu": "^2.1.6",
    "@radix-ui/react-hover-card": "^1.1.6",
    "@radix-ui/react-label": "^2.1.2",
    "@radix-ui/react-menubar": "^1.1.6",
    "@radix-ui/react-navigation-menu": "^1.2.5",
    "@radix-ui/react-popover": "^1.1.6",
    "@radix-ui/react-progress": "^1.1.2",
    "@radix-ui/react-radio-group": "^1.2.3",
    "@radix-ui/react-scroll-area": "^1.2.3",
    "@radix-ui/react-select": "^2.1.6",
    "@radix-ui/react-separator": "^1.1.2",
    "@radix-ui/react-slider": "^1.2.3",
    "@radix-ui/react-slot": "^1.1.2",
    "@radix-ui/react-switch": "^1.1.3",
    "@radix-ui/react-tabs": "^1.1.3",
    "@radix-ui/react-toast": "^1.2.2",
    "@radix-ui/react-toggle": "^1.1.2",
    "@radix-ui/react-toggle-group": "^1.1.2",
    "@radix-ui/react-tooltip": "^1.1.8",
    "@stripe/react-stripe-js": "^3.0.0",
    "@stripe/stripe-js": "^5.2.0",
    "@tanstack/react-query": "^5.84.1",
    "canvas-confetti": "^1.9.4",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.0.0",
    "date-fns": "^3.6.0",
    "embla-carousel-react": "^8.5.2",
    "framer-motion": "^11.16.4",
    "html2canvas": "^1.4.1",
    "input-otp": "^1.4.2",
    "jspdf": "^4.2.1",
    "lodash": "^4.17.21",
    "lucide-react": "^0.475.0",
    "moment": "^2.30.1",
    "next-themes": "^0.4.4",
    "react": "^18.2.0",
```

```
    "react-day-picker": "^8.10.1",
    "react-dom": "^18.2.0",
    "react-hook-form": "^7.54.2",
    "react-hot-toast": "^2.6.0",
    "react-leaflet": "^4.2.1",
    "react-markdown": "^9.0.1",
    "react-quill-new": "^3.8.3",
    "react-resizable-panels": "^2.1.7",
    "react-router-dom": "^6.26.0",
    "recharts": "^2.15.4",
    "sonner": "^2.0.1",
    "tailwind-merge": "^3.0.2",
    "tailwindcss-animate": "^1.0.7",
    "three": "^0.171.0",
    "vaul": "^1.1.2",
    "zod": "^3.24.2"
  },
  "devDependencies": {
    "@eslint/js": "^9.19.0",
    "@types/node": "^22.13.5",
    "@types/react": "^18.2.66",
    "@types/react-dom": "^18.2.22",
    "@vitejs/plugin-react": "^6.0.5",
    "autoprefixer": "^10.4.20",
    "baseline-browser-mapping": "^2.8.32",
    "eslint": "^9.19.0",
    "eslint-plugin-react": "^7.37.4",
    "eslint-plugin-react-hooks": "^5.0.0",
    "eslint-plugin-react-refresh": "^0.4.18",
    "eslint-plugin-unused-imports": "^4.3.0",
    "globals": "^15.14.0",
    "nitro": "^3.0.260610-beta",
    "postcss": "^8.5.3",
    "tailwindcss": "^3.4.17",
    "typescript": "^5.8.2",
    "vite": "^8.2.0"
  }
}


------------------------------------------------------------
FICHIER : ./postcss.config.js
------------------------------------------------------------
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}


------------------------------------------------------------
FICHIER : ./public/manifest.json
------------------------------------------------------------
{
  "name": "JH Manager",
  "short_name": "JH Manager",
  "description": "Gestion tout-en-un pour particuliers (JH Personal Manager) et entreprises (JH Business Manager) : finances,
tâches, planning, ventes, stock, personnel et devis.",
  "start_url": "/",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#f7f5f0",
  "theme_color": "#1f3a2e",
  "icons": [
    {
      "src": "https://base44.com/logo_v2.svg",
      "sizes": "192x192",
      "type": "image/svg+xml",
      "purpose": "any"
    },
    {
      "src": "https://base44.com/logo_v2.svg",
      "sizes": "512x512",
      "type": "image/svg+xml",
      "purpose": "any maskable"
    }
  ]
}


------------------------------------------------------------
FICHIER : ./src/App.jsx
------------------------------------------------------------
import React, { useState, useEffect } from "react";
import { Toaster } from "@/components/ui/toaster"
import { QueryClientProvider } from '@tanstack/react-query'
import { queryClientInstance } from '@/lib/query-client'
import { BrowserRouter as Router, Route, Routes, Navigate } from 'react-router-dom';
import PageNotFound from './lib/PageNotFound';
import { AuthProvider, useAuth } from '@/lib/AuthContext';
import { base44 } from '@/api/base44Client';
```

```
import { ThemeProvider } from '@/lib/ThemeProvider';
import UserNotRegisteredError from '@/components/UserNotRegisteredError';
import ScrollToTop from './components/ScrollToTop';
import ProtectedRoute from '@/components/ProtectedRoute';
import Login from '@/pages/Login';
import Register from '@/pages/Register';
import ForgotPassword from '@/pages/ForgotPassword';
import ResetPassword from '@/pages/ResetPassword';
import Privacy from '@/pages/Privacy';
import Terms from '@/pages/Terms';
import Layout from '@/components/Layout';
import { I18nProvider } from '@/lib/i18n';
import OAuthConsent from '@/pages/OAuthConsent';
import Onboarding from '@/pages/Onboarding';
import Home from '@/pages/Home';
import Finances from '@/pages/Finances';
import Tasks from '@/pages/Tasks';
import Planning from '@/pages/Planning';
import CalendarPage from '@/pages/Calendar';
import Admin from '@/pages/Admin';
import Affiliation from '@/pages/Affiliation';
import NotificationsPage from '@/pages/NotificationsPage';
import Settings from '@/pages/Settings';
import Agents from '@/pages/Agents';
import BusinessDashboard from '@/pages/BusinessDashboard';
import Inventory from '@/pages/Inventory';
import Sales from '@/pages/Sales';
import Personnel from '@/pages/Personnel';
import Companies from '@/pages/Companies';
import Calculators from '@/pages/Calculators';
import Thresholds from '@/pages/Thresholds';
import Clients from '@/pages/Clients';
import Quotes from '@/pages/Quotes';
import Suppliers from '@/pages/Suppliers';
import Taxes from '@/pages/Taxes';
import Loans from '@/pages/Loans';
import Partnerships from '@/pages/Partnerships';
import SuggestionsPage from '@/pages/SuggestionsPage';
import Reporting from '@/pages/Reporting';
import Export from '@/pages/Export';
import Events from '@/pages/Events';
import Categories from '@/pages/Categories';
import Collaborateurs from '@/pages/Collaborateurs';
import Faq from '@/pages/Faq';
import MorningBriefing from '@/pages/MorningBriefing';
import ResubscriptionGate from '@/components/ResubscriptionGate';
import Demo from '@/pages/Demo';
import CodeSource from '@/pages/CodeSource';

const AuthenticatedApp = () => {
  const { user, isLoadingAuth, isLoadingPublicSettings, authError, navigateToLogin } = useAuth();
  const [resub, setResub] = useState(null);
  const checkResub = async () => {
    if (!user || user.role === 'admin') { setResub({ blocked: false }); return; }
    try { setResub(await base44.functions.invoke('resubscriptionStatus', {})); }
    catch { setResub({ blocked: false }); }
  };
  useEffect(() => { checkResub(); /* eslint-disable-next-line */ }, [user?.id]);

  // Show loading spinner while checking app public settings or auth
  if (isLoadingPublicSettings || isLoadingAuth) {
    return (
      <div className="fixed inset-0 flex items-center justify-center">
        <div className="w-8 h-8 border-4 border-slate-200 border-t-slate-800 rounded-full animate-spin"></div>
      </div>
    );
  }

  // Handle authentication errors
  if (authError) {
    if (authError.type === 'user_not_registered') {
      return <UserNotRegisteredError />;
    } else if (authError.type === 'auth_required') {
      // Redirect to login automatically
      navigateToLogin();
      return null;
    }
  }

  // Compte supprimé par un admin : réabonnement requis avant tout accès.
  if (user && user.role !== 'admin' && !resub) {
    return (
      <div className="fixed inset-0 flex items-center justify-center">
        <div className="w-8 h-8 border-4 border-slate-200 border-t-slate-800 rounded-full animate-spin"></div>
      </div>
    );
  }
  if (resub?.blocked) {
    return <ResubscriptionGate status={resub.status} onSubmitted={checkResub} />;
  }

  // Authenticated user with no account type (and not admin) must pick a subscription / account type first.
```

```
  const accountType = user?.data?.account_type || user?.account_type;
  const needsOnboarding = !!user && user.role !== "admin" && !accountType;

  // Render the main app
  return (
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route path="/forgot-password" element={<ForgotPassword />} />
      <Route path="/reset-password" element={<ResetPassword />} />
      <Route path="/confidentialite" element={<Privacy />} />
      <Route path="/conditions" element={<Terms />} />
      <Route path="/oauth-consent" element={<OAuthConsent />} />
      <Route path="/" element={<Home />} />
      <Route path="/demo" element={<Demo />} />
      <Route element={<ProtectedRoute unauthenticatedElement={<Navigate to="/login" replace />} />}>
        {needsOnboarding ? (
          <Route path="/*" element={<Onboarding />} />
        ) : (
          <Route element={<Layout />}>
            <Route path="/finances" element={<Finances />} />
            <Route path="/taches" element={<Tasks />} />
            <Route path="/planning" element={<Planning />} />
            <Route path="/calendrier" element={<CalendarPage />} />
            <Route path="/admin" element={<Admin />} />
            <Route path="/affiliation" element={<Affiliation />} />
            <Route path="/notifications" element={<NotificationsPage />} />
            <Route path="/parametres" element={<Settings />} />
            <Route path="/assistants" element={<Agents />} />
            <Route path="/jbm" element={<BusinessDashboard />} />
            <Route path="/jbm/stock" element={<Inventory />} />
            <Route path="/jbm/ventes" element={<Sales />} />
            <Route path="/jbm/personnel" element={<Personnel />} />
            <Route path="/jbm/entreprises" element={<Companies />} />
            <Route path="/calculateurs" element={<Calculators />} />
            <Route path="/seuils" element={<Thresholds />} />
            <Route path="/suggestions" element={<SuggestionsPage />} />
            <Route path="/reporting" element={<Reporting />} />
            <Route path="/exportations" element={<Export />} />
            <Route path="/evenements" element={<Events />} />
            <Route path="/categories" element={<Categories />} />
            <Route path="/collaborateurs" element={<Collaborateurs />} />
            <Route path="/faq" element={<Faq />} />
            <Route path="/resume-matinal" element={<MorningBriefing />} />
            <Route path="/code-source" element={<CodeSource />} />
            <Route path="/taxes" element={<Taxes scope="jpm" />} />
            <Route path="/emprunts" element={<Loans />} />

            <Route path="/jbm/clients" element={<Clients />} />
            <Route path="/jbm/devis" element={<Quotes />} />
            <Route path="/jbm/prestataires" element={<Suppliers />} />
            <Route path="/jbm/taxes" element={<Taxes scope="jbm" />} />
            <Route path="/jbm/partenariats" element={<Partnerships />} />
          </Route>
        )}
      </Route>
      <Route path="*" element={<PageNotFound />} />
    </Routes>
  );
};


function App() {

  return (
    <AuthProvider>
      <ThemeProvider>
        <I18nProvider>
        <QueryClientProvider client={queryClientInstance}>
          <Router>
            <ScrollToTop />
            <AuthenticatedApp />
          </Router>
          <Toaster />
        </QueryClientProvider>
      </I18nProvider>
      </ThemeProvider>
    </AuthProvider>
  )
}

export default App

------------------------------------------------------------
FICHIER : ./src/api/base44Client.js
------------------------------------------------------------
import { createClient } from '@base44/sdk';
import { appParams } from '@/lib/app-params';

const { appId, token, functionsVersion, appBaseUrl } = appParams;

export const base44 = createClient({
```

```
  appId,
  token,
  functionsVersion,
  serverUrl: '',
  appBaseUrl
});


------------------------------------------------------------
FICHIER : ./src/components/AuthLayout.jsx
------------------------------------------------------------
import React from "react";

export default function AuthLayout({ icon: Icon, title, subtitle, footer, children }) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-background px-4">
      <div className="w-full max-w-md">
        <div className="text-center mb-10">
          <div className="inline-flex items-center justify-center w-14 h-14 rounded-2xl bg-primary mb-4">
            <Icon className="w-7 h-7 text-primary-foreground" aria-hidden="true" />
          </div>
          <h1 className="text-3xl font-bold tracking-tight text-foreground">{title}</h1>
          {subtitle && <p className="text-muted-foreground mt-2">{subtitle}</p>}
        </div>
        <div className="bg-card rounded-2xl shadow-sm border border-border p-8">
          {children}
        </div>
        {footer && (
          <p className="text-center text-sm text-muted-foreground mt-6">{footer}</p>
        )}
      </div>
    </div>
  );
}


------------------------------------------------------------
FICHIER : ./src/components/ExportMenu.jsx
------------------------------------------------------------
import React from "react";
import { Download, FileText, Sheet } from "lucide-react";
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger } from "@/components/ui/dropdown-menu";
import { Button } from "@/components/ui/button";
import { useI18n } from "@/lib/i18n";
import { exportToPdf, exportToXls } from "@/lib/exportReport";

// columns: [{ key, label, type? }]; rows, currency, summary: [{label,value}]
export default function ExportMenu({ title, subtitle, columns, rows, currency, summary }) {
  const { t } = useI18n();
  const disabled = !rows || rows.length === 0;

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="outline" disabled={disabled}>
          <Download className="w-4 h-4 mr-2" /> {t("export.title")}
        </Button>
      </DropdownMenuTrigger>
      <DropdownMenuContent align="end">
        <DropdownMenuItem onClick={() => exportToPdf(title, subtitle, columns, rows, currency, summary)}>
          <FileText className="w-4 h-4 mr-2" /> {t("export.pdf")}
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => exportToXls(title, subtitle, columns, rows, currency, summary)}>
          <Sheet className="w-4 h-4 mr-2" /> {t("export.excel")}
        </DropdownMenuItem>
      </DropdownMenuContent>
    </DropdownMenu>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/GoogleIcon.jsx
------------------------------------------------------------
import React from "react";

export default function GoogleIcon({ className = "w-5 h-5" }) {
  return (
    <svg className={className} viewBox="0 0 24 24" aria-hidden="true">
      <path d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92a5.06 5.06 0 01-2.2 3.32v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.1z"
fill="#4285F4" />
      <path d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99
20.53 7.7 23 12 23z" fill="#34A853" />
      <path d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18
4.93l2.85-2.22.81-.62z" fill="#FBBC05" />
      <path d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6
3.3-4.53 6.16-4.53z" fill="#EA4335" />
    </svg>
  );
}


------------------------------------------------------------
```

FICHIER : ./src/components/Layout.jsx

```
import React, { useEffect, useState } from "react";
import { useLocation, useOutlet, Navigate } from "react-router-dom";
import { AnimatePresence, motion } from "framer-motion";
import { base44 } from "@/api/base44Client";
import SidebarNav from "@/components/layout/SidebarNav";
import BottomNav from "@/components/layout/BottomNav";
import PullToRefresh from "@/components/PullToRefresh";
import AccountBar from "@/components/layout/AccountBar";
import { RefreshProvider, useRefresh } from "@/lib/RefreshContext";
import { SpaceProvider, useSpace } from "@/lib/SpaceContext";
import { useAuth } from "@/lib/AuthContext";
import { pageVariants, pageTransition } from "@/lib/pageTransitions";

// Routes strictement liées à un espace. Un utilisateur entreprise n'accède pas
// aux outils JPM, et inverse versa. L'admin garde accès à tout.
const JPM_ONLY = ["/finances", "/taches", "/planning", "/calendrier", "/seuils", "/reporting", "/taxes", "/emprunts"];

function SpaceGuard({ children }) {
  const location = useLocation();
  const { user } = useAuth();
  const { availableSpaces } = useSpace();
  if (!user || user.role === "admin") return children;
  const path = location.pathname;
  // Accès autorisé selon les espaces possédés (un utilisateur peut avoir les deux).
  if (!availableSpaces.includes("jpm") && JPM_ONLY.includes(path)) {
    return <Navigate to="/jbm" replace />;
  }
  if (!availableSpaces.includes("jbm") && path.startsWith("/jbm")) {
    return <Navigate to="/" replace />;
  }
  return children;
}

function AnimatedContent({ children }) {
  const location = useLocation();
  const outlet = useOutlet();
  const { run } = useRefresh();
  const content = outlet ?? children;

  return (
    <PullToRefresh onRefresh={run}>
      <AnimatePresence mode="wait">
        <motion.div
          key={location.pathname}
          variants={pageVariants}
          initial="initial"
          animate="animate"
          exit="exit"
          transition={pageTransition}
        >
          {content}
        </motion.div>
      </AnimatePresence>
    </PullToRefresh>
  );
}

export default function Layout({ children }) {
  const [email, setEmail] = useState("");

  useEffect(() => {
    base44.auth.me().then((u) => setEmail(u?.email || "")).catch(() => {});
  }, []);

  return (
    <div className="min-h-screen bg-background font-body">
      <SidebarNav userEmail={email} />
      <main className="md:pl-64 pb-24 md:pb-10 min-h-screen safe-top">
        <SpaceProvider>
          <RefreshProvider>
            <SpaceGuard>
              <div className="max-w-5xl mx-auto px-4 sm:px-6 py-6 sm:py-10">
                <div className="flex justify-end mb-4">
                  <AccountBar />
                </div>
                <AnimatedContent children={children} />
              </div>
            </SpaceGuard>
          </RefreshProvider>
        </SpaceProvider>
      </main>
      <BottomNav />
    </div>
  );
}
```

FICHIER : ./src/components/ProtectedRoute.jsx

```
import { useEffect } from 'react';
```

```
import { Outlet } from 'react-router-dom';
import { useAuth } from '@/lib/AuthContext';
import UserNotRegisteredError from '@/components/UserNotRegisteredError';

const DefaultFallback = () => (
  <div className="fixed inset-0 flex items-center justify-center">
    <div className="w-8 h-8 border-4 border-slate-200 border-t-slate-800 rounded-full animate-spin"></div>
  </div>
);

export default function ProtectedRoute({ fallback = <DefaultFallback />, unauthenticatedElement }) {
  const { isAuthenticated, isLoadingAuth, authChecked, authError, checkUserAuth } = useAuth();

  useEffect(() => {
    if (!authChecked && !isLoadingAuth) {
      checkUserAuth();
    }
  }, [authChecked, isLoadingAuth, checkUserAuth]);

  if (isLoadingAuth || !authChecked) {
    return fallback;
  }

  if (authError) {
    if (authError.type === 'user_not_registered') {
      return <UserNotRegisteredError />;
    }
    return unauthenticatedElement;
  }

  if (!isAuthenticated) {
    return unauthenticatedElement;
  }

  return <Outlet />;
}


------------------------------------------------------------
FICHIER : ./src/components/PullToRefresh.jsx
------------------------------------------------------------
import React, { useRef, useState, useCallback, useEffect } from "react";
import { Loader2, RefreshCw } from "lucide-react";

const THRESHOLD = 70;
const MAX = 100;

/**
 * Native-style pull-to-refresh that works with the window/page scroll
 * (no nested scroll container), so desktop window scrolling stays intact.
 * Engages only when the page is scrolled to the very top.
 */
export default function PullToRefresh({ onRefresh, children }) {
  const [pull, setPull] = useState(0);
  const [refreshing, setRefreshing] = useState(false);
  const startY = useRef(null);
  const pulling = useRef(false);

  const atTop = () => {
    const el = document.scrollingElement || document.documentElement;
    return (el ? el.scrollTop : window.scrollY) <= 0;
  };

  const onTouchStart = useCallback((e) => {
    if (refreshing || !atTop()) {
      startY.current = null;
      pulling.current = false;
      return;
    }
    startY.current = e.touches[0].clientY;
    pulling.current = false;
  }, [refreshing]);

  const onTouchMove = useCallback((e) => {
    if (startY.current === null || refreshing) return;
    if (!atTop()) {
      if (pull > 0) setPull(0);
      pulling.current = false;
      return;
    }
    const dy = e.touches[0].clientY - startY.current;
    if (dy <= 0) {
      if (pull > 0) setPull(0);
      pulling.current = false;
      return;
    }
    pulling.current = true;
    setPull(Math.min(MAX, dy * 0.5));
  }, [refreshing, pull]);

  const onTouchEnd = useCallback(async () => {
    if (!pulling.current || refreshing) {
```

```
      setPull(0);
      startY.current = null;
      return;
    }
    startY.current = null;
    pulling.current = false;
    if (pull >= THRESHOLD) {
      setRefreshing(true);
      setPull(THRESHOLD);
      try {
        await Promise.resolve(onRefresh && onRefresh());
      } catch (e) {
        /* errors handled by caller toasts */
      } finally {
        setRefreshing(false);
        setPull(0);
      }
    } else {
      setPull(0);
    }
  }, [pull, refreshing, onRefresh]);

  useEffect(() => {
    document.addEventListener("touchstart", onTouchStart, { passive: true });
    document.addEventListener("touchmove", onTouchMove, { passive: true });
    document.addEventListener("touchend", onTouchEnd, { passive: true });
    return () => {
      document.removeEventListener("touchstart", onTouchStart);
      document.removeEventListener("touchmove", onTouchMove);
      document.removeEventListener("touchend", onTouchEnd);
    };
  }, [onTouchStart, onTouchMove, onTouchEnd]);

  const progress = Math.min(1, pull / THRESHOLD);

  return (
    <>
      <div
        className="fixed left-0 right-0 top-0 flex items-end justify-center pointer-events-none z-30"
        style={{
          height: pull,
          opacity: progress,
          transition: pull === 0 || refreshing ? "height 0.2s ease, opacity 0.2s ease" : "none",
        }}
        aria-hidden="true"
      >
        {refreshing ? (
          <Loader2 className="w-6 h-6 text-primary animate-spin mb-2" />
        ) : (
          <RefreshCw
            className="w-5 h-5 text-primary mb-2"
            style={{ transform: `rotate(${progress * 360}deg)` }}
          />
        )}
      </div>
      <div
        style={{
          transform: `translateY(${pull}px)`,
          transition: pull === 0 || refreshing ? "transform 0.2s ease" : "none",
        }}
      >
        {children}
      </div>
    </>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/ResubscriptionGate.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { base44 } from "@/api/base44Client";
import { useAuth } from "@/lib/AuthContext";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";
import { ShieldAlert, Clock, XCircle, Send, Loader2 } from "lucide-react";
import { useToast } from "@/components/ui/use-toast";

export default function ResubscriptionGate({ status, onSubmitted }) {
  const { user } = useAuth();
  const { toast } = useToast();
  const [message, setMessage] = useState("");
  const [busy, setBusy] = useState(false);

  const submit = async () => {
    setBusy(true);
    try {
      await base44.entities.ReSubscriptionRequest.create({
        email: (user?.email || "").toLowerCase().trim(),
        message: message.trim(),
      });
      toast({ title: "Demande envoyée", description: "Les administrateurs ont été notifiés." });
```

```
      setMessage("");
      onSubmitted?.();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally {
      setBusy(false);
    }
  };

  const canSubmit = status !== "pending";

  return (
    <div className="min-h-screen bg-background flex items-center justify-center p-6">
      <div className="max-w-md w-full bg-card border border-border rounded-3xl p-6 space-y-4 text-center">
        <ShieldAlert className="w-10 h-10 text-accent mx-auto" />
        <h1 className="font-display text-xl">Compte désactivé</h1>

        {status === "pending" ? (
          <>
            <Clock className="w-6 h-6 text-muted-foreground mx-auto" />
            <p className="text-sm text-muted-foreground">
              Votre demande de réabonnement est en cours de traitement. Un administrateur vous répondra bientôt.
            </p>
          </>
        ) : (
          <>
            {status === "rejected" && (
              <>
                <XCircle className="w-6 h-6 text-destructive mx-auto" />
                <p className="text-sm text-muted-foreground">
                  Votre précédente demande a été refusée. Vous pouvez en soumettre une nouvelle.
                </p>
              </>
            )}
            {status === "none" && (
              <p className="text-sm text-muted-foreground">
                Votre compte a été supprimé par un administrateur. Pour recréer un compte avec cette adresse,
                vous devez d'abord demander un réabonnement.
              </p>
            )}
            <div className="text-left space-y-2">
              <Textarea
                value={message}
                onChange={(e) => setMessage(e.target.value)}
                placeholder="Expliquez pourquoi vous souhaitez réactiver votre compte…"
                rows={4}
              />
            </div>
            <Button onClick={submit} disabled={busy || !canSubmit} className="w-full">
              {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : <><Send className="w-4 h-4 mr-2" /> Demander un
réabonnement</>}
            </Button>
          </>
        )}
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/ScrollToTop.jsx
------------------------------------------------------------
import { useEffect } from "react";
import { useLocation, useNavigationType } from "react-router-dom";

const getHashId = (hash) => {
  const rawId = hash.slice(1);

  try {
    return decodeURIComponent(rawId);
  } catch {
    return rawId;
  }
};

export default function ScrollToTop() {
  const { pathname, hash } = useLocation();
  const navigationType = useNavigationType();

  useEffect(() => {
    if (navigationType === "POP") return;

    if (hash) {
      const id = getHashId(hash);
      const timer = window.setTimeout(() => {
        document.getElementById(id)?.scrollIntoView({ behavior: "smooth" });
      }, 50);
      return () => window.clearTimeout(timer);
    }

    window.scrollTo({ top: 0, left: 0, behavior: "instant" });
  }, [pathname, hash, navigationType]);
```

```
  return null;
}


------------------------------------------------------------
FICHIER : ./src/components/UserNotRegisteredError.jsx
------------------------------------------------------------
import React from 'react';

const UserNotRegisteredError = () => {
  return (
    <div className="flex flex-col items-center justify-center min-h-screen bg-gradient-to-b from-white to-slate-50">
      <div className="max-w-md w-full p-8 bg-white rounded-lg shadow-lg border border-slate-100">
        <div className="text-center">
          <div className="inline-flex items-center justify-center w-16 h-16 mb-6 rounded-full bg-orange-100">
            <svg className="w-8 h-8 text-orange-600" fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0
2.502-1.667 1.732-3L13.732 4c-.77-1.333-2.694-1.333-3.464 0L3.34 16c-.77 1.333.192 3 1.732 3z" />
            </svg>
          </div>
          <h1 className="text-3xl font-bold text-slate-900 mb-4">Accès restreint</h1>
          <p className="text-slate-600 mb-8">
            Vous n'êtes pas inscrit pour utiliser cette application. Merci de contacter l'administrateur pour demander l'accès.
          </p>
          <div className="p-4 bg-slate-50 rounded-md text-sm text-slate-600">
            <p>Si vous pensez qu'il s'agit d'une erreur, vous pouvez :</p>
            <ul className="list-disc list-inside mt-2 space-y-1">
              <li>Vérifier que vous êtes connecté avec le bon compte</li>
              <li>Contacter l'administrateur pour obtenir l'accès</li>
              <li>Vous déconnecter puis vous reconnecter</li>
            </ul>
          </div>
        </div>
      </div>
    </div>
  );
};

export default UserNotRegisteredError;

------------------------------------------------------------
FICHIER : ./src/components/account/AddAccountButton.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Plus } from "lucide-react";
import { Button } from "@/components/ui/button";
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter,
} from "@/components/ui/dialog";
import { Label } from "@/components/ui/label";
import { RadioGroup, RadioGroupItem } from "@/components/ui/radio-group";
import SubscribeDialog from "@/components/onboarding/SubscribeDialog";

const PLANS = {
  particulier: { type: "particulier", label: "Compte Particulier", baseAmount: 5000 },
  entreprise: { type: "entreprise", label: "Compte Entreprise", unitPrice: 20000 },
};

export default function AddAccountButton({ label = "Ajouter un compte" }) {
  const [open, setOpen] = useState(false);
  const [type, setType] = useState("particulier");
  const [plan, setPlan] = useState(null);

  return (
    <>
      <Button variant="outline" onClick={() => setOpen(true)} className="no-select-tap">
        <Plus className="w-4 h-4 mr-2" /> {label}
      </Button>

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader>
            <DialogTitle className="font-display">Ajouter un compte</DialogTitle>
          </DialogHeader>
          <div className="space-y-3">
            <Label>Type de compte (nouvel abonnement)</Label>
            <RadioGroup value={type} onValueChange={setType}>
              <div className="flex items-center gap-2 border border-border rounded-lg p-3">
                <RadioGroupItem value="particulier" id="add-p" />
                <label htmlFor="add-p" className="text-sm">Particulier (JPM) — 5 000 XAF</label>
              </div>
              <div className="flex items-center gap-2 border border-border rounded-lg p-3">
                <RadioGroupItem value="entreprise" id="add-e" />
                <label htmlFor="add-e" className="text-sm">Entreprise (JBM) — 20 000 XAF / entreprise</label>
              </div>
            </RadioGroup>
          </div>
          <DialogFooter>
            <Button variant="outline" onClick={() => setOpen(false)}>Annuler</Button>
            <Button onClick={() => { setPlan(PLANS[type]); setOpen(false); }}>Continuer vers le paiement</Button>
          </DialogFooter>
```

```
        </DialogContent>
      </Dialog>

      <SubscribeDialog
        plan={plan}
        open={!!plan}
        onOpenChange={(o) => !o && setPlan(null)}
        onSuccess={() => { setPlan(null); window.location.reload(); }}
      />
    </>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/admin/AdminFaq.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { motion } from "framer-motion";
import { Sparkles, Loader2, CheckCircle2, Tag, Users } from "lucide-react";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";
import { useToast } from "@/components/ui/use-toast";
import { formatCurrency } from "@/lib/format";

export default function AdminFaq() {
  const { toast } = useToast();
  const [questions, setQuestions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [triaging, setTriaging] = useState(false);
  const [answering, setAnswering] = useState(null);

  const load = async () => {
    try {
      const list = await base44.entities.FaqQuestion.list("-created_date", 200);
      setQuestions(list || []);
    } catch { setQuestions([]); }
    finally { setLoading(false); }
  };

  useEffect(() => { load(); }, []);

  const triage = async () => {
    setTriaging(true);
    try {
      const res = await base44.functions.invoke('triageFaq', {});
      toast({ title: res?.message || "Tri IA terminé", description: `${res?.grouped || 0} question(s) regroupée(s).` });
      load();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally { setTriaging(false); }
  };

  const save = async (q) => {
    setAnswering(q.id);
    try {
      await base44.entities.FaqQuestion.update(q.id, {
        answer: q.answer || "",
        is_published: !!q.is_published,
        status: q.answer ? "traite" : "en_cours",
      });
      toast({ title: "Réponse enregistrée" });
      load();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally { setAnswering(null); }
  };

  const update = (id, patch) => setQuestions((prev) => prev.map((q) => (q.id === id ? { ...q, ...patch } : q)));

  // Regrouper par group_label
  const groups = {};
  questions.forEach((q) => {
    const key = q.group_label || "Non triées";
    if (!groups[key]) groups[key] = [];
    groups[key].push(q);
  });

  if (loading) {
    return (
      <div className="bg-card border border-border rounded-2xl p-5 flex items-center justify-center h-40">
        <Loader2 className="w-5 h-5 animate-spin text-muted-foreground" />
      </div>
    );
  }

  return (
    <div className="bg-card border border-border rounded-2xl p-5 space-y-4">
      <div className="flex items-center justify-between flex-wrap gap-2">
        <div className="flex items-center gap-2">
          <Users className="w-5 h-5 text-accent" />
          <h2 className="font-display text-lg">FAQ reçues ({questions.length})</h2>
```

```
        </div>
        <Button onClick={triage} disabled={triaging} variant="outline" size="sm">
          {triaging ? <Loader2 className="w-4 h-4 animate-spin" /> : <Sparkles className="w-4 h-4" />}
          Trier avec l'IA
        </Button>
      </div>

      {questions.length === 0 ? (
        <p className="text-sm text-muted-foreground text-center py-6">Aucune question pour l'instant.</p>
      ) : (
        <div className="space-y-4 max-h-[60vh] overflow-y-auto pr-1">
          {Object.entries(groups).map(([label, items]) => (
            <div key={label} className="rounded-xl border border-border p-3">
              <div className="flex items-center gap-2 mb-2">
                <Tag className="w-3.5 h-3.5 text-accent" />
                <span className="text-sm font-medium">{label}</span>
                <span className="text-xs text-muted-foreground">· {items.length}</span>
              </div>
              <div className="space-y-3">
                {items.map((q) => (
                  <div key={q.id} className="rounded-lg bg-muted/40 p-3">
                    <p className="text-sm font-medium">{q.question}</p>
                    <p className="text-xs text-muted-foreground mt-0.5">
                      {q.category} · {new Date(q.created_date).toLocaleDateString("fr-FR")}
                      {q.status === "traite" && <CheckCircle2 className="inline w-3 h-3 ml-1 text-primary" />}
                    </p>
                    <Textarea
                      value={q.answer || ""}
                      onChange={(e) => update(q.id, { answer: e.target.value })}
                      placeholder="Réponse / synthèse…"
                      rows={2}
                      className="mt-2 bg-background text-sm"
                    />
                    <div className="flex items-center justify-between mt-2">
                      <label className="flex items-center gap-1.5 text-xs text-muted-foreground">
                        <input
                          type="checkbox"
                          checked={!!q.is_published}
                          onChange={(e) => update(q.id, { is_published: e.target.checked })}
                        />
                        Publier sur l'accueil
                      </label>
                      <Button size="sm" onClick={() => save(q)} disabled={answering === q.id}>
                        {answering === q.id ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : "Enregistrer"}
                      </Button>
                    </div>
                  </div>
                ))}
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/admin/AdminReports.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Flag, CheckCircle2, Loader2 } from "lucide-react";
import { useToast } from "@/components/ui/use-toast";
import { formatDateFr } from "@/lib/format";

const REASONS = {
  spam: "Spam",
  harcelement: "Harcèlement",
  contenu_inapproprie: "Contenu inapproprié",
  arnaque: "Arnaque",
  usurpation: "Usurpation",
  autre: "Autre",
};

export default function AdminReports() {
  const { toast } = useToast();
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [busy, setBusy] = useState(null);

  const load = async () => {
    try {
      const list = await base44.entities.Report.list("-created_date", 200);
      setItems(list || []);
    } catch { setItems([]); }
    finally { setLoading(false); }
  };

  useEffect(() => { load(); }, []);
```

```
  const markDone = async (r) => {
    setBusy(r.id);
    try {
      await base44.entities.Report.update(r.id, { status: "traite" });
      toast({ title: "Signalement traité" });
      load();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally {
      setBusy(null);
    }
  };

  return (
    <div className="bg-card border border-border rounded-2xl p-5 space-y-4">
      <div className="flex items-center gap-2">
        <Flag className="w-5 h-5 text-accent" />
        <div>
          <h2 className="font-display text-lg">Signalements ({items.length})</h2>
          <p className="text-xs text-muted-foreground">Utilisateurs et groupes signalés dans la section collaboration.</p>
        </div>
      </div>

      {loading ? (
        <div className="flex justify-center py-6"><Loader2 className="w-5 h-5 animate-spin text-muted-foreground" /></div>
      ) : items.length === 0 ? (
        <p className="text-sm text-muted-foreground text-center py-4">Aucun signalement pour l'instant.</p>
      ) : (
        <div className="space-y-2 max-h-[60vh] overflow-y-auto pr-1">
          {items.map((r) => (
            <div key={r.id} className="border border-border rounded-xl p-3 space-y-1">
              <div className="flex items-center justify-between gap-2">
                <p className="text-sm font-medium truncate">
                  {r.target_name || "Cible inconnue"}
                  <span className="text-xs text-muted-foreground ml-2">({r.target_type === "group" ? "Groupe" :
"Utilisateur"})</span>
                </p>
                <span className={`text-[11px] px-2 py-0.5 rounded-full ${r.status === "traite" ? "bg-primary/15 text-primary" :
"bg-accent/15 text-accent-foreground"}`}>
                  {r.status === "traite" ? "Traité" : "Nouveau"}
                </span>
              </div>
              <p className="text-xs text-muted-foreground">
                Raison : {REASONS[r.reason] || r.reason || "—"}
                {r.custom_reason && <span className="block mt-0.5">{r.custom_reason}</span>}
              </p>
              <p className="text-[11px] text-muted-foreground">{r.created_date ? formatDateFr(r.created_date, true) : "—"}</p>
              {r.status !== "traite" && (
                <Button size="sm" variant="outline" onClick={() => markDone(r)} disabled={busy === r.id} className="flex
items-center gap-1">
                  {busy === r.id ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <CheckCircle2 className="w-3.5 h-3.5" />}
Marquer traité
                </Button>
              )}
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/admin/AdminResubscription.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { UserCheck, Check, X, Loader2, Clock, CheckCircle2, XCircle } from "lucide-react";
import { useToast } from "@/components/ui/use-toast";
import { formatDateFr } from "@/lib/format";

const STATUS = {
  pending: { label: "En attente", icon: Clock, cls: "bg-accent/15 text-accent-foreground" },
  approved: { label: "Approuvée", icon: CheckCircle2, cls: "bg-primary/15 text-primary" },
  rejected: { label: "Refusée", icon: XCircle, cls: "bg-destructive/15 text-destructive" },
};

export default function AdminResubscription() {
  const { toast } = useToast();
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [busy, setBusy] = useState(null);

  const load = async () => {
    try {
      const list = await base44.entities.ReSubscriptionRequest.list("-created_date", 200);
      setItems(list || []);
    } catch { setItems([]); }
    finally { setLoading(false); }
  };
```

```
  useEffect(() => { load(); }, []);

  const decide = async (r, decision) => {
    setBusy(r.id);
    try {
      await base44.entities.ReSubscriptionRequest.update(r.id, { status: decision });
      if (decision === "approved") {
        try { await base44.entities.DeletedAccount.deleteMany({ email: r.email }); } catch { /* ignore */ }
      }
      toast({ title: decision === "approved" ? "Demande approuvée" : "Demande refusée" });
      load();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally {
      setBusy(null);
    }
  };

  return (
    <div className="bg-card border border-border rounded-2xl p-5 space-y-4">
      <div className="flex items-center gap-2">
        <UserCheck className="w-5 h-5 text-accent" />
        <div>
          <h2 className="font-display text-lg">Demandes de réabonnement ({items.length})</h2>
          <p className="text-xs text-muted-foreground">Abonnés supprimés souhaitant recréer un compte.</p>
        </div>
      </div>

      {loading ? (
        <div className="flex justify-center py-6"><Loader2 className="w-5 h-5 animate-spin text-muted-foreground" /></div>
      ) : items.length === 0 ? (
        <p className="text-sm text-muted-foreground text-center py-4">Aucune demande pour l'instant.</p>
      ) : (
        <div className="space-y-2 max-h-[60vh] overflow-y-auto pr-1">
          {items.map((r) => {
            const st = STATUS[r.status] || STATUS.pending;
            return (
              <div key={r.id} className="border border-border rounded-xl p-3 space-y-2">
                <div className="flex items-center justify-between gap-2">
                  <div className="min-w-0">
                    <p className="text-sm font-medium truncate">{r.email}</p>
                    <p className="text-xs text-muted-foreground">{r.created_date ? formatDateFr(r.created_date, true) : "—"}</p>
                  </div>
                  <span className={`text-[11px] px-2 py-0.5 rounded-full flex items-center gap-1 ${st.cls}`}>
                    <st.icon className="w-3 h-3" /> {st.label}
                  </span>
                </div>
                {r.message && <p className="text-sm text-muted-foreground bg-muted/40 rounded-lg p-2">{r.message}</p>}
                {r.status !== "approved" && (
                  <div className="flex gap-2">
                    <Button size="sm" onClick={() => decide(r, "approved")} disabled={busy === r.id} className="flex
items-center gap-1">
                      {busy === r.id ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Check className="w-3.5 h-3.5" />}
Approuver
                    </Button>
                    {r.status !== "rejected" && (
                      <Button size="sm" variant="outline" onClick={() => decide(r, "rejected")} disabled={busy === r.id}
className="flex items-center gap-1">
                        <X className="w-3.5 h-3.5" /> Refuser
                      </Button>
                    )}
                  </div>
                )}
              </div>
            );
          })}
        </div>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/admin/AdminSuggestions.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { base44 } from "@/api/base44Client";
import { Lightbulb, Loader2, Sparkles } from "lucide-react";
import { Button } from "@/components/ui/button";

const CAT = { amelioration: "Amélioration", correction: "Correction", autre: "Autre" };

export default function AdminSuggestions() {
  const [loading, setLoading] = useState(false);
  const [groups, setGroups] = useState(null);
  const [totalUsers, setTotalUsers] = useState(0);
  const [error, setError] = useState("");

  const run = async () => {
    setLoading(true); setError("");
    try {
      const res = await base44.functions.invoke('analyzeSuggestions', {});
```

```
      if (res?.success) { setGroups(res.groups || []); setTotalUsers(res.totalUsers || 0); }
      else setError(res?.error || "Échec de l'analyse");
    } catch (e) { setError(e?.message || "Erreur"); }
    setLoading(false);
  };

  return (
    <div className="bg-card border border-border rounded-2xl p-5 space-y-4">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div className="flex items-center gap-2">
          <Sparkles className="w-5 h-5 text-accent" />
          <div>
            <h2 className="font-display text-lg">Suggestions (synthèse IA)</h2>
            <p className="text-xs text-muted-foreground">L'IA regroupe les suggestions similaires en idées uniques.</p>
          </div>
        </div>
        <Button onClick={run} disabled={loading}>
          {loading ? <><Loader2 className="w-4 h-4 mr-2 animate-spin" /> Analyse…</> : <><Lightbulb className="w-4 h-4 mr-2" />
Analyser</>}
        </Button>
      </div>

      {error && <p className="text-sm text-destructive">{error}</p>}

      {groups && (
        <div className="space-y-2">
          {groups.length === 0 ? (
            <p className="text-sm text-muted-foreground">Aucune suggestion pour l'instant.</p>
          ) : (
            <>
              <p className="text-xs text-muted-foreground">{groups.length} idée(s) unique(s) · {totalUsers} utilisateur(s)
distinct(s)</p>
              {groups.map((g, i) => (
                <div key={i} className="border border-border rounded-xl p-3">
                  <div className="flex items-start justify-between gap-3">
                    <p className="text-sm font-medium">{g.label}</p>
                    <div className="flex items-center gap-2 shrink-0 text-xs">
                      <span className="px-2 py-0.5 rounded-full bg-secondary">{CAT[g.category] || g.category}</span>
                      <span className="text-muted-foreground">{g.users} utilisateur(s)</span>
                      <span className="font-medium">{g.percentage}%</span>
                    </div>
                  </div>
                  {g.occurrences !== g.users && <p className="text-xs text-muted-foreground mt-1">{g.occurrences} occurrence(s)
au total</p>}
                </div>
              ))}
            </>
          )}
        </div>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/admin/SubscriberDetail.jsx
------------------------------------------------------------
import React, { useState } from "react";
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogDescription,
} from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { base44 } from "@/api/base44Client";
import { Trash2, Loader2 } from "lucide-react";
import { formatCurrency, formatDateFr } from "@/lib/format";

function Info({ label, value }) {
  return (
    <div>
      <p className="text-xs text-muted-foreground">{label}</p>
      <p className="text-sm font-medium">{value}</p>
    </div>
  );
}

const accountTypeLabel = (u) =>
  u.accountType === "entreprise" ? "Entreprise (JBM)" : u.accountType === "particulier" ? "Particulier (JPM)" : "Non défini";

export default function SubscriberDetail({ user, subscriptions, open, onOpenChange }) {
  const [deleting, setDeleting] = useState(false);
  const { toast } = useToast();
  if (!user) return null;

  const removeSubscriber = async () => {
    if (!confirm(`Supprimer définitivement l'abonné ${user.email} ? Ses données seront effacées et il devra demander un
réabonnement pour recréer un compte.`)) return;
    setDeleting(true);
    try {
      const res = await base44.functions.invoke('deleteSubscriber', { user_id: user.id });
      toast({ title: "Abonné supprimé", description: res?.userDeleted ? "Compte utilisateur supprimé." : "Données effacées
(compte auth conservé)." });
```

```
      onOpenChange?.(false);
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally {
      setDeleting(false);
    }
  };
  const mySubs = subscriptions.filter((s) => s.created_by_id === user.id);
  const totalPaid = mySubs.reduce((sum, s) => sum + (Number(s.amount) || 0), 0);
  const companies = mySubs.length ? Math.max(...mySubs.map((s) => Number(s.companies_count) || 1)) : 0;

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-lg">
        <DialogHeader>
          <DialogTitle className="font-display">{user.full_name || user.email}</DialogTitle>
          <DialogDescription>{user.email}</DialogDescription>
        </DialogHeader>
        <div className="space-y-4 text-sm">
          <div className="grid grid-cols-2 gap-3">
            <Info label="Rôle" value={user.role} />
            <Info label="Type de compte" value={accountTypeLabel(user)} />
            <Info label="Entreprises" value={companies || "—"} />
            <Info label="Total payé" value={formatCurrency(totalPaid, "XAF")} />
            <Info label="Inscrit le" value={user.created_date ? formatDateFr(user.created_date) : "—"} />
          </div>

          <div>
            <p className="font-medium mb-2">Abonnements ({mySubs.length})</p>
            {mySubs.length === 0 ? (
              <p className="text-muted-foreground">Aucun abonnement enregistré.</p>
            ) : (
              <div className="space-y-2 max-h-60 overflow-y-auto">
                {mySubs.map((s) => (
                  <div key={s.id} className="border border-border rounded-lg p-2 text-xs">
                    <div className="flex justify-between">
                      <span>{s.account_type === "entreprise" ? "Entreprise" : "Particulier"}</span>
                      <span className="font-medium">{formatCurrency(s.amount, "XAF")}</span>
                    </div>
                    <div className="flex justify-between text-muted-foreground">
                      <span>{s.provider} · {s.status}</span>
                      <span>{s.paid_at ? formatDateFr(s.paid_at, true) : "—"}</span>
                    </div>
                    {Number(s.companies_count) > 0 && (
                      <p className="text-muted-foreground">{s.companies_count} entreprise(s)</p>
                    )}
                  </div>
                ))}
              </div>
            )}
          </div>

          <p className="text-xs text-muted-foreground">
            Note : la consultation des conversations avec les agents IA nécessite une API admin
            côté plateforme (à venir). Le SDK actuel limite les conversations à l'utilisateur connecté.
          </p>

          <div className="border-t border-border pt-3">
            <Button variant="destructive" size="sm" onClick={removeSubscriber} disabled={deleting} className="flex items-center
gap-1">
              {deleting ? <Loader2 className="w-4 h-4 animate-spin" /> : <Trash2 className="w-4 h-4" />}
              Supprimer cet abonné
            </Button>
          </div>
        </div>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/admin/SubscribersByCompanyChart.jsx
------------------------------------------------------------
import React, { useMemo } from "react";
import { BarChart, Bar, XAxis, YAxis, ResponsiveContainer, Tooltip, CartesianGrid } from "recharts";

export default function SubscribersByCompanyChart({ data = [] }) {
  if (!data.length) {
    return (
      <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
        <h2 className="font-display text-lg mb-3">Abonnés JBM par nombre d'entreprises</h2>
        <p className="text-sm text-muted-foreground">Aucun abonné entreprise pour l'instant.</p>
      </div>
    );
  }
  return (
    <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
      <h2 className="font-display text-lg mb-4">Abonnés JBM par nombre d'entreprises</h2>
      <div className="h-64">
        <ResponsiveContainer width="100%" height="100%">
          <BarChart data={data}>
            <CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" />
```

```
            <XAxis dataKey="companies" tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" />
            <YAxis allowDecimals={false} tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" />
            <Tooltip />
            <Bar dataKey="count" name="Abonnés" fill="hsl(var(--chart-1))" radius={[6, 6, 0, 0]} />
          </BarChart>
        </ResponsiveContainer>
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/agents/AgentChat.jsx
------------------------------------------------------------
import React, { useEffect, useRef, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Send, Loader2, Plus, Paperclip, X } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Image } from "@/components/ui/image";
import { useToast } from "@/components/ui/use-toast";
import { useI18n } from "@/lib/i18n";
import MessageBubble from "./MessageBubble";

const IMAGE_EXT = /\.(png|jpe?g|gif|webp)$/i;

export default function AgentChat({ agentName }) {
  const { t } = useI18n();
  const { toast } = useToast();
  const [conversation, setConversation] = useState(null);
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");
  const [attachment, setAttachment] = useState(null); // { url, name }
  const [uploading, setUploading] = useState(false);
  const [loading, setLoading] = useState(true);
  const [sending, setSending] = useState(false);
  const fileRef = useRef(null);
  const bottomRef = useRef(null);

  useEffect(() => {
    let unsub = () => {};
    setLoading(true);
    setMessages([]);
    setConversation(null);
    (async () => {
      try {
        const convs = await base44.agents.listConversations({ agent_name: agentName });
        let conv = convs?.[0];
        if (!conv) {
          conv = await base44.agents.createConversation({ agent_name: agentName, metadata: { name: agentName } });
        }
        setConversation(conv);
        setMessages(conv.messages || []);
        setLoading(false);
        unsub = base44.agents.subscribeToConversation(conv.id, (data) => {
          setMessages(data.messages || []);
        });
      } catch (e) {
        setLoading(false);
      }
    })();
    return () => { unsub(); };
  }, [agentName]);

  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages]);

  useEffect(() => {
    if (sending && messages.length && messages[messages.length - 1].role === "assistant") {
      setSending(false);
    }
  }, [messages, sending]);

  const newConversation = async () => {
    try {
      const conv = await base44.agents.createConversation({ agent_name: agentName, metadata: { name: agentName } });
      setConversation(conv);
      setMessages([]);
      setSending(false);
    } catch (e) {
      /* ignore */
    }
  };

  const onFile = async (e) => {
    const file = e.target.files?.[0];
    e.target.value = "";
    if (!file || sending || uploading) return;
    setUploading(true);
    try {
      const { file_url } = await base44.integrations.Core.UploadPublicFile({ file });
```

```
    setAttachment({ url: file_url, name: file.name });
  } catch (err) {
    toast({ title: "Erreur", description: err?.message, variant: "destructive" });
  } finally {
    setUploading(false);
  }
};

const send = async () => {
  const content = input.trim();
  if ((!content && !attachment) || !conversation || sending) return;
  const file_urls = attachment ? [attachment.url] : undefined;
  setInput("");
  setAttachment(null);
  setSending(true);
  setMessages((prev) => [...prev, { role: "user", content, file_urls: file_urls || [] }]);
  try {
    const conv = await base44.agents.addMessage(conversation, { role: "user", content, file_urls });
    setConversation(conv);
  } catch (e) {
    setSending(false);
  }
};

if (loading) {
  return (
    <div className="flex items-center justify-center h-48">
      <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
    </div>
  );
}

return (
  <div className="flex flex-col h-[65vh]">
    <div className="flex justify-end mb-2">
      <Button variant="outline" size="sm" onClick={newConversation} className="no-select-tap">
        <Plus className="w-4 h-4 mr-1" /> {t("agents.newConversation")}
      </Button>
    </div>
    <div className="flex-1 overflow-y-auto space-y-4 pb-4">
      {messages.length === 0 && (
        <p className="text-sm text-muted-foreground text-center mt-10">{t("agents.empty")}</p>
      )}
      {messages.map((m, i) => (
        <MessageBubble key={i} message={m} />
      ))}
      {sending && (
        <div className="flex items-center gap-2 text-sm text-muted-foreground">
          <Loader2 className="w-4 h-4 animate-spin" /> {t("agents.thinking")}
        </div>
      )}
      <div ref={bottomRef} />
    </div>
    <div className="pt-3 border-t border-border">
      {attachment && (
        <div className="flex items-center gap-2 mb-2 text-xs bg-muted rounded-lg px-2 py-1.5">
          {IMAGE_EXT.test(attachment.url) ? (
            <Image src={attachment.url} alt={attachment.name} className="w-8 h-8 rounded object-cover shrink-0" />
          ) : (
            <Paperclip className="w-4 h-4 shrink-0" />
          )}
          <span className="truncate flex-1">{attachment.name}</span>
          <button type="button" onClick={() => setAttachment(null)} className="no-select-tap p-1 rounded hover:bg-background">
            <X className="w-3.5 h-3.5" />
          </button>
        </div>
      )}
      <div className="flex gap-2">
        <input
          ref={fileRef}
          type="file"
          className="hidden"
          onChange={onFile}
          accept="image/*,audio/*,video/*,application/pdf"
        />
        <Button
          type="button"
          variant="ghost"
          size="icon"
          onClick={() => fileRef.current?.click()}
          disabled={sending || uploading}
          title="Joindre un fichier multimédia"
          className="no-select-tap shrink-0"
        >
          {uploading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Paperclip className="w-4 h-4" />}
        </Button>
        <Input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter" && !e.shiftKey) {
              e.preventDefault();
```

```
              send();
            }
          }}
          placeholder={t("agents.placeholder")}
          className="flex-1"
        />
        <Button onClick={send} disabled={(!input.trim() && !attachment) || sending} className="no-select-tap shrink-0">
          <Send className="w-4 h-4" />
        </Button>
      </div>
    </div>
  </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/agents/MessageBubble.jsx
------------------------------------------------------------
import React, { useState } from "react";
import ReactMarkdown from "react-markdown";
import { ChevronRight, CheckCircle2, Loader2, AlertCircle, Wrench } from "lucide-react";
import { useI18n } from "@/lib/i18n";
import { Image } from "@/components/ui/image";

const IMAGE_EXT = /\.(png|jpe?g|gif|webp)$/i;
const MEDIA_RE = /(https?:\/\/[^\s)]+\.(mp3|wav|m4a|ogg|mp4|webm|mov))(?=[\s)]|$)/gi;

function FunctionDisplay({ toolCall }) {
  const { t } = useI18n();
  const [expanded, setExpanded] = useState(false);
  const proj = toolCall.display_projection || {};
  const hideDetails = proj.hide_details && proj.details_redacted;
  const status = toolCall.status || "pending";
  const failed = status === "failed" || status === "error";
  const running = ["pending", "running", "in_progress"].includes(status);

  let parsedArgs = toolCall.arguments_string;
  if (typeof parsedArgs === "string") {
    try { parsedArgs = JSON.parse(parsedArgs); } catch (e) { /* keep raw */ }
  }
  let parsedResults = toolCall.results;
  if (typeof parsedResults === "string") {
    try { parsedResults = JSON.parse(parsedResults); } catch (e) { /* keep raw */ }
  }

  const label = hideDetails
    ? (failed ? (proj.error_label || status) : running ? (proj.active_label || t("agents.tool.running")) : (proj.label ||
status))
    : (toolCall.name || t("agents.tool.running"));

  return (
    <div className="mt-2 text-xs rounded-lg border border-border bg-secondary/40 p-2">
      <button
        type="button"
        onClick={() => !hideDetails && setExpanded((v) => !v)}
        className="flex items-center gap-2 w-full text-left"
      >
        {failed
          ? <AlertCircle className="w-3.5 h-3.5 text-destructive shrink-0" />
          : running
            ? <Loader2 className="w-3.5 h-3.5 animate-spin shrink-0" />
            : <CheckCircle2 className="w-3.5 h-3.5 text-primary shrink-0" />}
        <Wrench className="w-3 h-3 text-muted-foreground shrink-0" />
        <span className="font-medium truncate">{label}</span>
        {!hideDetails && (
          <ChevronRight className={`w-3 h-3 ml-auto shrink-0 transition-transform ${expanded ? "rotate-90" : ""}`} />
        )}
      </button>
      {expanded && !hideDetails && (
        <div className="mt-2 space-y-1">
          <p className="text-muted-foreground">{t("agents.tool.params")}</p>
          <pre className="whitespace-pre-wrap break-words rounded bg-background/60 p-1.5">
            {typeof parsedArgs === "string" ? parsedArgs : JSON.stringify(parsedArgs, null, 2)}
          </pre>
          {parsedResults !== undefined && (
            <>
              <p className="text-muted-foreground mt-1">{t("agents.tool.result")}</p>
              <pre className="whitespace-pre-wrap break-words rounded bg-background/60 p-1.5">
                {typeof parsedResults === "string" ? parsedResults : JSON.stringify(parsedResults, null, 2)}
              </pre>
            </>
          )}
        </div>
      )}
    </div>
  );
}

export default function MessageBubble({ message }) {
  const isUser = message.role === "user";
  return (
    <div className={isUser ? "flex justify-end" : "flex justify-start"}>
```

```
      <div
        className={`max-w-[85%] rounded-2xl px-4 py-2.5 ${
          isUser ? "bg-primary text-primary-foreground" : "bg-card border border-border"
        }`}
      >
        {message.content &&
          (isUser ? (
            <p className="text-sm whitespace-pre-wrap">{message.content}</p>
          ) : (
            <div className="text-sm space-y-2 [&_p]:m-0 [&_ul]:list-disc [&_ul]:pl-4 [&_ol]:list-decimal [&_ol]:pl-4
[&_code]:rounded [&_code]:bg-secondary [&_code]:px-1">
              <ReactMarkdown
                components={{ img: ({ src, alt }) => (
                  <Image src={src} alt={alt || ""} className="rounded-lg max-h-56 w-auto max-w-full object-cover my-1" />
                ) }}
              >
                {message.content.replace(MEDIA_RE, "").trim()}
              </ReactMarkdown>
              {(message.content.match(MEDIA_RE) || []).map((url, i) =>
                /\.(mp3|wav|m4a|ogg)$/i.test(url) ? (
                  <audio key={`media${i}`} controls src={url} className="w-full mt-1" />
                ) : (
                  <video key={`media${i}`} controls src={url} className="w-full mt-1 max-h-60 rounded-lg" />
                )
              )}
            </div>
          ))}
        {message.file_urls?.length > 0 && (
          <div className="mt-2 flex flex-col gap-1">
            {message.file_urls.map((url, i) =>
              IMAGE_EXT.test(url) ? (
                <Image key={i} src={url} alt="pièce jointe" className="rounded-lg max-h-48 w-auto max-w-full object-cover" />
              ) : (
                <a key={i} href={url} target="_blank" rel="noreferrer" className="underline text-xs break-all">
                  {url.split("/").pop() || "Fichier"}
                </a>
              )
            )}
          </div>
        )}
        {message.tool_calls?.map((tc, i) => (
          <FunctionDisplay key={i} toolCall={tc} />
        ))}
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/business/AttendancePanel.jsx
------------------------------------------------------------
import React, { useState, useEffect } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Trash2, CalendarDays } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { useToast } from "@/components/ui/use-toast";
import { formatDateFr } from "@/lib/format";

const TYPE_LABEL = {
  presence: "Présence", absence: "Absence", retard: "Retard", conge: "Congé", discipline: "Discipline",
};
const SEV_LABEL = { info: "Info", avertissement: "Avertissement", sanction: "Sanction" };
const SEV_STYLE = {
  info: "bg-secondary text-muted-foreground",
  avertissement: "bg-accent/15 text-accent-foreground",
  sanction: "bg-destructive/10 text-destructive",
};

const empty = { employee_id: "", type: "presence", date: "", note: "", severity: "info" };

export default function AttendancePanel({ companyId, employees }) {
  const [records, setRecords] = useState([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [form, setForm] = useState(empty);
  const { toast } = useToast();

  const load = async () => {
    try {
      const list = await base44.entities.AttendanceRecord.list("-date", 500);
      setRecords(list || []);
    } catch { setRecords([]); }
    setLoading(false);
  };
  useEffect(() => { load(); }, []);

  const mine = companyId ? records.filter((r) => r.company_id === companyId) : records;
```

```
  const empName = (id) => employees.find((e) => e.id === id)?.full_name || "—";

  const submit = async (e) => {
    e.preventDefault();
    if (!form.employee_id || !form.date) { toast({ title: "Employé et date requis", variant: "destructive" }); return; }
    const payload = {
      ...form,
      company_id: companyId,
      employee_name: empName(form.employee_id),
    };
    try {
      await base44.entities.AttendanceRecord.create(payload);
      toast({ title: "Enregistrement ajouté" });
      setOpen(false);
      setForm(empty);
      load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (r) => {
    try { await base44.entities.AttendanceRecord.delete(r.id); setRecords((c) => c.filter((x) => x.id !== r.id)); }
    catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  if (loading) return <div className="flex justify-center py-8"><div className="w-6 h-6 border-2 border-border border-t-primary
rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between gap-3">
        <p className="text-sm text-muted-foreground">Présences, absences, retards et comportements des employés.</p>
        <Button onClick={() => { setForm({ ...empty, date: new Date().toISOString().slice(0, 10) }); setOpen(true); }}>
          <Plus className="w-4 h-4 mr-2" /> Enregistrer
        </Button>
      </div>

      {mine.length === 0 ? (
        <div className="text-center py-10 text-muted-foreground">
          <CalendarDays className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p className="text-sm">Aucun enregistrement.</p>
        </div>
      ) : (
        <div className="space-y-2">
          {mine.map((r) => (
            <div key={r.id} className="bg-card border border-border rounded-xl p-3 flex items-center justify-between gap-3">
              <div className="min-w-0">
                <p className="text-sm font-medium truncate">{r.employee_name || empName(r.employee_id)}</p>
                <p className="text-xs text-muted-foreground mt-0.5">
                  {TYPE_LABEL[r.type] || r.type} · {formatDateFr(r.date)}
                  {r.note && ` · ${r.note}`}
                </p>
              </div>
              <div className="flex items-center gap-2 shrink-0">
                {r.type === "discipline" && (
                  <span className={`text-xs px-2 py-0.5 rounded-full ${SEV_STYLE[r.severity] || SEV_STYLE.info}`}>
                    {SEV_LABEL[r.severity] || r.severity}
                  </span>
                )}
                <button onClick={() => remove(r)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive">
                  <Trash2 className="w-4 h-4" />
                </button>
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader><DialogTitle className="font-display">Absence & discipline</DialogTitle></DialogHeader>
          <form onSubmit={submit} className="space-y-3">
            <div className="space-y-2">
              <Label>Employé</Label>
              <Select value={form.employee_id} onValueChange={(v) => setForm({ ...form, employee_id: v })}>
                <SelectTrigger><SelectValue placeholder="Sélectionner" /></SelectTrigger>
                <SelectContent>
                  {employees.map((e) => <SelectItem key={e.id} value={e.id}>{e.full_name}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
            <div className="grid grid-cols-2 gap-3">
              <div className="space-y-2">
                <Label>Type</Label>
                <Select value={form.type} onValueChange={(v) => setForm({ ...form, type: v })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent>
                    {Object.entries(TYPE_LABEL).map(([v, l]) => <SelectItem key={v} value={v}>{l}</SelectItem>)}
                  </SelectContent>
                </Select>
              </div>
```

```
            <div className="space-y-2">
              <Label>Date</Label>
              <Input type="date" value={form.date} onChange={(e) => setForm({ ...form, date: e.target.value })} />
            </div>
          </div>
          {form.type === "discipline" && (
            <div className="space-y-2">
              <Label>Gravité</Label>
              <Select value={form.severity} onValueChange={(v) => setForm({ ...form, severity: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  {Object.entries(SEV_LABEL).map(([v, l]) => <SelectItem key={v} value={v}>{l}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          <div className="space-y-2">
            <Label>Note / comportement</Label>
            <Textarea rows={2} value={form.note} onChange={(e) => setForm({ ...form, note: e.target.value })} />
          </div>
          <DialogFooter><Button type="submit" className="w-full">Enregistrer</Button></DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  </div>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/business/EmployeeForm.jsx
------------------------------------------------------------
import React, { useState, useEffect } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useI18n } from "@/lib/i18n";

const PERIODS = [
  { value: "jour", label: "Journalier" },
  { value: "semaine", label: "Hebdomadaire" },
  { value: "mois", label: "Mensuel" },
  { value: "trimestre", label: "Trimestriel" },
  { value: "semestre", label: "Semestriel" },
  { value: "annee", label: "Annuel" },
];

const empty = {
  full_name: "", role: "", headcount: "1", salary: "0", salary_periodicity: "mois",
  bonus: "0", bonus_type: "ponctuel", bonus_periodicity: "mois",
  phone: "", email: "", hire_date: "", status: "actif",
};

export default function EmployeeForm({ open, onOpenChange, employee, onSubmit }) {
  const { t } = useI18n();
  const [form, setForm] = useState(empty);
  const [error, setError] = useState("");

  useEffect(() => {
    if (employee) {
      setForm({
        full_name: employee.full_name || "",
        role: employee.role || "",
        headcount: String(employee.headcount ?? 1),
        salary: String(employee.salary ?? 0),
        salary_periodicity: employee.salary_periodicity || "mois",
        bonus: String(employee.bonus ?? 0),
        bonus_type: employee.bonus_type || "ponctuel",
        bonus_periodicity: employee.bonus_periodicity || "mois",
        phone: employee.phone || "",
        email: employee.email || "",
        hire_date: employee.hire_date || "",
        status: employee.status || "actif",
      });
    } else {
      setForm({ ...empty, hire_date: new Date().toISOString().slice(0, 10) });
    }
    setError("");
  }, [employee, open]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.full_name.trim()) {
      setError(t("jbm.err.name"));
      return;
    }
    onSubmit({
      full_name: form.full_name.trim(),
      role: form.role.trim(),
      headcount: parseInt(form.headcount, 10) || 1,
      salary: parseFloat(form.salary) || 0,
```

```
      salary_periodicity: form.salary_periodicity,
      bonus: parseFloat(form.bonus) || 0,
      bonus_type: form.bonus_type,
      bonus_periodicity: form.bonus_type === "periodique" ? form.bonus_periodicity : null,
      phone: form.phone.trim(),
      email: form.email.trim(),
      hire_date: form.hire_date,
      status: form.status,
    });
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display">{employee ? t("jbm.employee.edit") : t("jbm.employee.new")}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="space-y-2">
            <Label>{t("jbm.employee.name")}</Label>
            <Input value={form.full_name} onChange={(e) => setForm({ ...form, full_name: e.target.value })} />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.employee.role")}</Label>
              <Input value={form.role} onChange={(e) => setForm({ ...form, role: e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>Nombre à ce poste</Label>
              <Input type="number" min="1" value={form.headcount} onChange={(e) => setForm({ ...form, headcount: e.target.value
})} />
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.employee.salary")}</Label>
              <Input type="number" min="0" step="any" value={form.salary} onChange={(e) => setForm({ ...form, salary:
e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>Périodicité du salaire</Label>
              <Select value={form.salary_periodicity} onValueChange={(v) => setForm({ ...form, salary_periodicity: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  {PERIODS.map((p) => <SelectItem key={p.value} value={p.value}>{p.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>Prime</Label>
              <Input type="number" min="0" step="any" value={form.bonus} onChange={(e) => setForm({ ...form, bonus:
e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>Type de prime</Label>
              <Select value={form.bonus_type} onValueChange={(v) => setForm({ ...form, bonus_type: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  <SelectItem value="ponctuel">Ponctuelle</SelectItem>
                  <SelectItem value="periodique">Périodique</SelectItem>
                </SelectContent>
              </Select>
            </div>
          </div>
          {form.bonus_type === "periodique" && (
            <div className="space-y-2">
              <Label>Périodicité de la prime</Label>
              <Select value={form.bonus_periodicity} onValueChange={(v) => setForm({ ...form, bonus_periodicity: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  {PERIODS.map((p) => <SelectItem key={p.value} value={p.value}>{p.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.employee.hireDate")}</Label>
              <Input type="date" value={form.hire_date} onChange={(e) => setForm({ ...form, hire_date: e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.employee.status")}</Label>
              <Select value={form.status} onValueChange={(v) => setForm({ ...form, status: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  <SelectItem value="actif">{t("jbm.employee.statusActive")}</SelectItem>
                  <SelectItem value="inactif">{t("jbm.employee.statusInactive")}</SelectItem>
                </SelectContent>
              </Select>
            </div>
```

```
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.employee.phone")}</Label>
              <Input value={form.phone} onChange={(e) => setForm({ ...form, phone: e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.employee.email")}</Label>
              <Input type="email" value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} />
            </div>
          </div>
          {error && <p className="text-sm text-destructive">{error}</p>}
          <DialogFooter>
            <Button type="submit" className="w-full">{t("common.save")}</Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/business/EmployeeItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Pencil, Trash2 } from "lucide-react";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
  AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

const PERIOD_LABEL = {
  jour: "jour", semaine: "sem.", mois: "mois", trimestre: "trim.", semestre: "semest.", annee: "an",
};

export default function EmployeeItem({ employee, currency, onEdit, onDelete }) {
  const { t } = useI18n();
  const [open, setOpen] = useState(false);
  const per = PERIOD_LABEL[employee.salary_periodicity] || "mois";
  const headcount = Number(employee.headcount) || 1;

  return (
    <div className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
      <div className="min-w-0">
        <p className="text-sm font-medium truncate">{employee.full_name}</p>
        <p className="text-xs text-muted-foreground mt-0.5">
          {employee.role || "—"} · {headcount > 1 ? `${headcount} pers. · ` : ""}{formatCurrency(employee.salary,
currency)}/{per}
        </p>
        {Number(employee.bonus) > 0 && (
          <p className="text-xs text-primary mt-0.5">
            Prime : {formatCurrency(employee.bonus, currency)}{employee.bonus_type === "periodique" ?
`/${PERIOD_LABEL[employee.bonus_periodicity] || "mois"}` : " (ponctuelle)"}
          </p>
        )}
        {employee.hire_date && (
          <p className="text-xs text-muted-foreground">{t("jbm.employee.hireDate")}: {formatDateFr(employee.hire_date)}</p>
        )}
      </div>
      <div className="flex items-center gap-2 shrink-0">
        <span className={`text-xs px-2 py-0.5 rounded-full ${employee.status === "actif" ? "bg-primary/10 text-primary" :
"bg-secondary text-muted-foreground"}`}>
          {employee.status === "actif" ? t("jbm.employee.statusActive") : t("jbm.employee.statusInactive")}
        </span>
        <button onClick={() => onEdit(employee)} aria-label={t("taskitem.editAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-secondary">
          <Pencil className="w-4 h-4" />
        </button>
        <button onClick={() => setOpen(true)} aria-label={t("taskitem.deleteAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-destructive/10 text-destructive">
          <Trash2 className="w-4 h-4" />
        </button>
      </div>
      <AlertDialog open={open} onOpenChange={setOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>{t("jbm.employee.deleteTitle")}</AlertDialogTitle>
            <AlertDialogDescription>{t("jbm.employee.deleteDesc", { name: employee.full_name })}</AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>{t("common.cancel")}</AlertDialogCancel>
            <AlertDialogAction className="bg-destructive text-destructive-foreground hover:bg-destructive/90" onClick={(e) => {
e.preventDefault(); onDelete(employee); setOpen(false); }}>
              {t("common.delete")}
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
```

```
}

------------------------------------------------------------
FICHIER : ./src/components/business/InvoiceForm.jsx
------------------------------------------------------------
import React, { useState, useEffect } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useI18n } from "@/lib/i18n";

const empty = { number: "", customer_name: "", amount: "0", issue_date: "", due_date: "", status: "brouillon" };

export default function InvoiceForm({ open, onOpenChange, invoice, onSubmit, nextNumber }) {
  const { t } = useI18n();
  const [form, setForm] = useState(empty);
  const [error, setError] = useState("");

  useEffect(() => {
    if (invoice) {
      setForm({
        number: invoice.number || "",
        customer_name: invoice.customer_name || "",
        amount: String(invoice.amount ?? 0),
        issue_date: invoice.issue_date || "",
        due_date: invoice.due_date || "",
        status: invoice.status || "brouillon",
      });
    } else {
      setForm({
        ...empty,
        number: nextNumber || "",
        issue_date: new Date().toISOString().slice(0, 10),
      });
    }
    setError("");
  }, [invoice, open, nextNumber]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.number.trim() || !form.customer_name.trim() || !form.issue_date) {
      setError(t("jbm.err.invoiceFields"));
      return;
    }
    onSubmit({
      number: form.number.trim(),
      customer_name: form.customer_name.trim(),
      amount: parseFloat(form.amount) || 0,
      issue_date: form.issue_date,
      due_date: form.due_date,
      status: form.status,
    });
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display">{invoice ? t("jbm.invoice.edit") : t("jbm.invoice.new")}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.invoice.number")}</Label>
              <Input value={form.number} onChange={(e) => setForm({ ...form, number: e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.invoice.amount")}</Label>
              <Input type="number" min="0" step="any" value={form.amount} onChange={(e) => setForm({ ...form, amount:
e.target.value })} />
            </div>
          </div>
          <div className="space-y-2">
            <Label>{t("jbm.invoice.customer")}</Label>
            <Input value={form.customer_name} onChange={(e) => setForm({ ...form, customer_name: e.target.value })} />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.invoice.issueDate")}</Label>
              <Input type="date" value={form.issue_date} onChange={(e) => setForm({ ...form, issue_date: e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.invoice.dueDate")}</Label>
              <Input type="date" value={form.due_date} onChange={(e) => setForm({ ...form, due_date: e.target.value })} />
            </div>
          </div>
          <div className="space-y-2">
            <Label>{t("jbm.invoice.status")}</Label>
            <Select value={form.status} onValueChange={(v) => setForm({ ...form, status: v })}>
              <SelectTrigger><SelectValue /></SelectTrigger>
```

```
              <SelectContent>
                <SelectItem value="brouillon">{t("jbm.invoice.statusDraft")}</SelectItem>
                <SelectItem value="envoyee">{t("jbm.invoice.statusSent")}</SelectItem>
                <SelectItem value="payee">{t("jbm.invoice.statusPaid")}</SelectItem>
                <SelectItem value="en_retard">{t("jbm.invoice.statusLate")}</SelectItem>
              </SelectContent>
            </Select>
          </div>
          {error && <p className="text-sm text-destructive">{error}</p>}
          <DialogFooter>
            <Button type="submit" className="w-full">{t("common.save")}</Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/business/InvoiceItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Pencil, Trash2 } from "lucide-react";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
  AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

const STATUS_KEY = {
  brouillon: "jbm.invoice.statusDraft",
  envoyee: "jbm.invoice.statusSent",
  payee: "jbm.invoice.statusPaid",
  en_retard: "jbm.invoice.statusLate",
};

export default function InvoiceItem({ invoice, currency, onEdit, onDelete }) {
  const { t } = useI18n();
  const [open, setOpen] = useState(false);
  const tone =
    invoice.status === "payee" ? "bg-primary/10 text-primary"
      : invoice.status === "en_retard" ? "bg-destructive/10 text-destructive"
      : "bg-secondary text-secondary-foreground";

  return (
    <div className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
      <div className="min-w-0">
        <p className="text-sm font-medium truncate">{invoice.number} · {invoice.customer_name}</p>
        <p className="text-xs text-muted-foreground mt-0.5">
                    { f o r m a t D a t e F r ( i n v o i c e . i s s u e _ d a t e ) } { i n v o i c e . d u e _ d a t e  ?  `  !'
        </p>
      </div>
      <div className="flex items-center gap-2 shrink-0">
        <div className="text-right mr-1">
          <p className="text-sm font-medium">{formatCurrency(invoice.amount, currency)}</p>
          <span className={`text-xs px-2 py-0.5 rounded-full ${tone}`}>{t(STATUS_KEY[invoice.status] || "common.other")}</span>
        </div>
        <button onClick={() => onEdit(invoice)} aria-label={t("taskitem.editAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-secondary">
          <Pencil className="w-4 h-4" />
        </button>
        <button onClick={() => setOpen(true)} aria-label={t("taskitem.deleteAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-destructive/10 text-destructive">
          <Trash2 className="w-4 h-4" />
        </button>
      </div>
      <AlertDialog open={open} onOpenChange={setOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>{t("jbm.invoice.deleteTitle")}</AlertDialogTitle>
            <AlertDialogDescription>{t("jbm.invoice.deleteDesc", { number: invoice.number })}</AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>{t("common.cancel")}</AlertDialogCancel>
            <AlertDialogAction className="bg-destructive text-destructive-foreground hover:bg-destructive/90" onClick={(e) => {
e.preventDefault(); onDelete(invoice); setOpen(false); }}>
              {t("common.delete")}
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/business/ProductForm.jsx
------------------------------------------------------------
import React, { useState, useEffect } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
```

```
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import CategorySelect from "@/components/categories/CategorySelect";
import { useSpace } from "@/lib/SpaceContext";
import { useI18n } from "@/lib/i18n";

const empty = { name: "", sku: "", category: "", quantity: "0", unit_cost: "0", sale_price: "0", reorder_level: "0" };

export default function ProductForm({ open, onOpenChange, product, onSubmit }) {
  const { t } = useI18n();
  const { activeCompanyId } = useSpace();
  const [form, setForm] = useState(empty);
  const [error, setError] = useState("");

  useEffect(() => {
    if (product) {
      setForm({
        name: product.name || "",
        sku: product.sku || "",
        category: product.category || "",
        quantity: String(product.quantity ?? 0),
        unit_cost: String(product.unit_cost ?? 0),
        sale_price: String(product.sale_price ?? 0),
        reorder_level: String(product.reorder_level ?? 0),
      });
    } else {
      setForm(empty);
    }
    setError("");
  }, [product, open]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.name.trim()) {
      setError(t("jbm.err.name"));
      return;
    }
    onSubmit({
      name: form.name.trim(),
      sku: form.sku.trim(),
      category: form.category.trim(),
      quantity: parseFloat(form.quantity) || 0,
      unit_cost: parseFloat(form.unit_cost) || 0,
      sale_price: parseFloat(form.sale_price) || 0,
      reorder_level: parseFloat(form.reorder_level) || 0,
    });
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display">{product ? t("jbm.product.edit") : t("jbm.product.new")}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="space-y-2">
            <Label>{t("jbm.product.name")}</Label>
            <Input value={form.name} onChange={(e) => setForm({ ...form, name: e.target.value })} />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.product.sku")}</Label>
              <Input value={form.sku} onChange={(e) => setForm({ ...form, sku: e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.product.category")}</Label>
              <CategorySelect scope="jbm" domain="produit" companyId={activeCompanyId} value={form.category} onChange={(v) =>
setForm({ ...form, category: v })} />
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.product.quantity")}</Label>
              <Input type="number" min="0" value={form.quantity} onChange={(e) => setForm({ ...form, quantity: e.target.value
})} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.product.reorder")}</Label>
              <Input type="number" min="0" value={form.reorder_level} onChange={(e) => setForm({ ...form, reorder_level:
e.target.value })} />
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.product.unitCost")}</Label>
              <Input type="number" min="0" step="any" value={form.unit_cost} onChange={(e) => setForm({ ...form, unit_cost:
e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.product.salePrice")}</Label>
              <Input type="number" min="0" step="any" value={form.sale_price} onChange={(e) => setForm({ ...form, sale_price:
e.target.value })} />
```

```
          </div>
        </div>
        {error && <p className="text-sm text-destructive">{error}</p>}
        <DialogFooter>
          <Button type="submit" className="w-full">{t("common.save")}</Button>
        </DialogFooter>
      </form>
    </DialogContent>
  </Dialog>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/business/ProductItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Pencil, Trash2, AlertTriangle } from "lucide-react";
import { formatCurrency } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import {
 AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
 AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

export default function ProductItem({ product, currency, onEdit, onDelete }) {
 const { t } = useI18n();
 const [open, setOpen] = useState(false);
 const low = (Number(product.quantity) || 0) <= (Number(product.reorder_level) || 0);

 return (
  <div className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
   <div className="min-w-0">
    <div className="flex items-center gap-2">
     <p className="text-sm font-medium truncate">{product.name}</p>
     {low && Number(product.reorder_level) > 0 && (
      <span className="text-xs px-2 py-0.5 rounded-full bg-destructive/10 text-destructive flex items-center gap-1
shrink-0">
       <AlertTriangle className="w-3 h-3" /> {t("jbm.product.low")}
      </span>
     )}
    </div>
    <p className="text-xs text-muted-foreground mt-0.5">
     {product.sku ? `${product.sku} · ` : ""}{product.category || t("common.other")} · {t("jbm.product.stock")}:
{product.quantity}
    </p>
   </div>
   <div className="flex items-center gap-2 shrink-0">
    <div className="text-right mr-1">
     <p className="text-sm font-medium">{formatCurrency(product.sale_price, currency)}</p>
     <p className="text-xs text-muted-foreground">{formatCurrency(product.unit_cost, currency)}</p>
    </div>
    <button onClick={() => onEdit(product)} aria-label={t("taskitem.editAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-secondary">
     <Pencil className="w-4 h-4" />
    </button>
    <button onClick={() => setOpen(true)} aria-label={t("taskitem.deleteAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-destructive/10 text-destructive">
     <Trash2 className="w-4 h-4" />
    </button>
   </div>
   <AlertDialog open={open} onOpenChange={setOpen}>
    <AlertDialogContent>
     <AlertDialogHeader>
      <AlertDialogTitle>{t("jbm.product.deleteTitle")}</AlertDialogTitle>
      <AlertDialogDescription>{t("jbm.product.deleteDesc", { name: product.name })}</AlertDialogDescription>
     </AlertDialogHeader>
     <AlertDialogFooter>
      <AlertDialogCancel>{t("common.cancel")}</AlertDialogCancel>
      <AlertDialogAction className="bg-destructive text-destructive-foreground hover:bg-destructive/90" onClick={(e) => {
e.preventDefault(); onDelete(product); setOpen(false); }}>
       {t("common.delete")}
      </AlertDialogAction>
     </AlertDialogFooter>
    </AlertDialogContent>
   </AlertDialog>
  </div>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/business/SaleForm.jsx
------------------------------------------------------------
import React, { useState, useEffect } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Checkbox } from "@/components/ui/checkbox";
import { useI18n } from "@/lib/i18n";
import { base44 } from "@/api/base44Client";
import { evalExpr, computeGrandeurs, formatNumber } from "@/lib/mathEval";
```

```
import { loadMetrics } from "@/lib/calcSources";
import CategorySelect from "@/components/categories/CategorySelect";
import { useSpace } from "@/lib/SpaceContext";

const empty = { customer_name: "", item_name: "", quantity: "1", unit_price: "0", date: "", status: "payee", category: "",
use_formula: false, base_price: "0", pricing_formula: "", scheduled: false };

export default function SaleForm({ open, onOpenChange, sale, onSubmit }) {
  const { t } = useI18n();
  const { activeCompanyId } = useSpace();
  const [form, setForm] = useState(empty);
  const [repeat, setRepeat] = useState(false);
  const [frequency, setFrequency] = useState("mois");
  const [repeatCount, setRepeatCount] = useState("1");
  const [error, setError] = useState("");
  const [grandeurs, setGrandeurs] = useState([]);
  const [metrics, setMetrics] = useState({});

  useEffect(() => {
    if (sale) {
      setForm({
        customer_name: sale.customer_name || "",
        item_name: sale.item_name || "",
        quantity: String(sale.quantity ?? 1),
        unit_price: String(sale.unit_price ?? 0),
        date: sale.date || "",
        status: sale.status || "payee",
        category: sale.category || "",
        use_formula: !!sale.pricing_formula,
        base_price: String(sale.base_price ?? 0),
        pricing_formula: sale.pricing_formula || "",
        scheduled: sale.scheduled || sale.status === "en_attente",
      });
    } else {
      setForm({ ...empty, date: new Date().toISOString().slice(0, 10) });
    }
    setRepeat(false);
    setFrequency("mois");
    setRepeatCount("1");
    setError("");
  }, [sale, open]);

  useEffect(() => {
    if (!open) return;
    (async () => {
      try {
        const list = await base44.entities.CalcDefinition.list("-created_date", 200);
        setGrandeurs((list || []).filter((i) => i.kind === "grandeur"));
      } catch { /* ignore */ }
      try { setMetrics(await loadMetrics(activeCompanyId)); } catch { /* ignore */ }
    })();
  }, [open, activeCompanyId]);

  const scope = computeGrandeurs(grandeurs, metrics);
  const previewPrice = (() => {
    if (!form.use_formula || !form.pricing_formula.trim()) return null;
    try {
      const v = evalExpr(form.pricing_formula, { ...scope, qte: parseFloat(form.quantity) || 0, base:
parseFloat(form.base_price) || 0 });
      return isFinite(v) ? v : null;
    } catch { return null; }
  })();

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.item_name.trim() || !form.date) {
      setError(t("jbm.err.saleFields"));
      return;
    }
    const qte = parseFloat(form.quantity) || 1;
    let unit_price = parseFloat(form.unit_price) || 0;
    let pricing_formula = null;
    let base_price = null;
    if (form.use_formula && form.pricing_formula.trim()) {
      try {
        const v = evalExpr(form.pricing_formula, { ...scope, qte, base: parseFloat(form.base_price) || 0 });
        if (!isFinite(v)) { setError("Formule invalide"); return; }
        unit_price = v;
        pricing_formula = form.pricing_formula.trim();
        base_price = parseFloat(form.base_price) || 0;
      } catch (err) {
        setError("Formule invalide : " + err.message);
        return;
      }
    }
    const scheduled = form.scheduled;
    onSubmit({
      customer_name: form.customer_name.trim(),
      item_name: form.item_name.trim(),
      quantity: qte,
      unit_price,
      base_price,
```

```
      pricing_formula,
      scheduled,
      date: form.date,
      status: scheduled ? "en_attente" : form.status,
      category: form.category.trim(),
    }, repeat && !sale && parseInt(repeatCount, 10) > 1 ? { frequency, count: parseInt(repeatCount, 10) } : null);
  };

  const grandeurNames = grandeurs.map((g) => g.name).join(", ");

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md max-h-[90vh] overflow-y-auto">
        <DialogHeader>
          <DialogTitle className="font-display">{sale ? t("jbm.sale.edit") : t("jbm.sale.new")}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="space-y-2">
            <Label>{t("jbm.sale.item")}</Label>
            <Input value={form.item_name} onChange={(e) => setForm({ ...form, item_name: e.target.value })} />
          </div>
          <div className="space-y-2">
            <Label>{t("jbm.sale.customer")}</Label>
            <Input value={form.customer_name} onChange={(e) => setForm({ ...form, customer_name: e.target.value })} />
          </div>
          <div className="space-y-2">
            <Label>Catégorie</Label>
            <CategorySelect scope="jbm" domain="vente" companyId={activeCompanyId} value={form.category} onChange={(v) =>
setForm({ ...form, category: v })} placeholder="Catégorie" />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.sale.quantity")}</Label>
              <Input type="number" min="1" value={form.quantity} onChange={(e) => setForm({ ...form, quantity: e.target.value
})} />
            </div>
            <div className="space-y-2">
              <Label>{t("jbm.sale.status")}</Label>
              <Select value={form.status} onValueChange={(v) => setForm({ ...form, status: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  <SelectItem value="payee">{t("jbm.sale.statusPayee")}</SelectItem>
                  <SelectItem value="en_attente">{t("jbm.sale.statusPending")}</SelectItem>
                  <SelectItem value="annulee">{t("jbm.sale.statusCancelled")}</SelectItem>
                </SelectContent>
              </Select>
            </div>
          </div>

          <div className="rounded-xl border border-border p-3 space-y-3">
            <div className="flex items-center gap-2">
              <Checkbox checked={form.use_formula} onCheckedChange={(v) => setForm({ ...form, use_formula: !!v })}
id="sale-formula" />
              <Label htmlFor="sale-formula">Tarification par formule</Label>
            </div>
            {form.use_formula && (
              <div className="space-y-3">
                <p className="text-xs text-muted-foreground">
                  Variables : <span className="font-mono">qte</span> (quantité), <span className="font-mono">base</span> (prix
de base)
                  {grandeurs.length > 0 && <span>, et vos grandeurs : {grandeurNames}</span>}.
                </p>
                <div className="space-y-2">
                  <Label>Prix de base (base)</Label>
                  <Input type="number" min="0" step="any" value={form.base_price} onChange={(e) => setForm({ ...form,
base_price: e.target.value })} />
                </div>
                <div className="space-y-2">
                  <Label>Formule du prix unitaire</Label>
                  <Input value={form.pricing_formula} onChange={(e) => setForm({ ...form, pricing_formula: e.target.value })}
placeholder="base + qte * 100" />
                </div>
                <div className="bg-secondary rounded-lg p-2 text-sm">
                  Prix unitaire calculé : <span className="font-medium">{previewPrice != null ? formatNumber(previewPrice) :
"—"}</span>
                </div>
              </div>
            )}
          </div>

          {!form.use_formula && (
            <div className="space-y-2">
              <Label>{t("jbm.sale.unitPrice")}</Label>
              <Input type="number" min="0" step="any" value={form.unit_price} onChange={(e) => setForm({ ...form, unit_price:
e.target.value })} />
            </div>
          )}

          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("jbm.sale.date")}</Label>
              <Input type="date" value={form.date} onChange={(e) => setForm({ ...form, date: e.target.value })} />
```

```
            </div>
            <div className="space-y-2">
              <Label>Programmer</Label>
              <div className="flex items-center gap-2 h-9">
                <Checkbox checked={form.scheduled} onCheckedChange={(v) => setForm({ ...form, scheduled: !!v })} id="sale-sched"
/>
                <Label htmlFor="sale-sched" className="text-sm font-normal">Vente programmée (rappel J-1 et J)</Label>
              </div>
            </div>
          </div>

          {!sale && (
            <div className="space-y-3 rounded-xl border border-border p-3">
              <div className="flex items-center gap-2">
                <Checkbox checked={repeat} onCheckedChange={setRepeat} id="sale-repeat" />
                <Label htmlFor="sale-repeat">Répéter périodiquement</Label>
              </div>
              {repeat && (
                <div className="grid grid-cols-2 gap-3">
                  <div className="space-y-1.5">
                    <Label htmlFor="sale-freq" className="text-xs">Fréquence</Label>
                    <Select value={frequency} onValueChange={setFrequency}>
                      <SelectTrigger id="sale-freq"><SelectValue /></SelectTrigger>
                      <SelectContent>
                        <SelectItem value="jour">Jour</SelectItem>
                        <SelectItem value="semaine">Semaine</SelectItem>
                        <SelectItem value="trimestre">Trimestre</SelectItem>
                        <SelectItem value="mois">Mois</SelectItem>
                        <SelectItem value="annee">Année</SelectItem>
                      </SelectContent>
                    </Select>
                  </div>
                  <div className="space-y-1.5">
                    <Label htmlFor="sale-count" className="text-xs">Nombre d'échéances</Label>
                    <Input id="sale-count" type="number" min="1" max="120" value={repeatCount} onChange={(e) =>
setRepeatCount(e.target.value)} />
                  </div>
                </div>
              )}
            </div>
          )}
          {error && <p className="text-sm text-destructive">{error}</p>}
          <DialogFooter>
            <Button type="submit" className="w-full">{t("common.save")}</Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/business/SaleItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Pencil, Trash2 } from "lucide-react";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
  AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

const STATUS_KEY = { payee: "jbm.sale.statusPayee", en_attente: "jbm.sale.statusPending", annulee: "jbm.sale.statusCancelled" };

export default function SaleItem({ sale, currency, onEdit, onDelete }) {
  const { t } = useI18n();
  const [open, setOpen] = useState(false);
  const total = (Number(sale.quantity) || 0) * (Number(sale.unit_price) || 0);

  return (
    <div className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
      <div className="min-w-0">
        <p className="text-sm font-medium truncate">{sale.item_name}</p>
        <p className="text-xs text-muted-foreground mt-0.5">
          {sale.customer_name || "—"} · {formatDateFr(sale.date)}
          {sale.scheduled && <span className="ml-1.5 text-accent">· Programmée</span>}
        </p>
      </div>
      <div className="flex items-center gap-2 shrink-0">
        <div className="text-right mr-1">
          <p className="text-sm font-medium">{formatCurrency(total, currency)}</p>
          <span className="text-xs px-2 py-0.5 rounded-full bg-secondary text-secondary-foreground">{t(STATUS_KEY[sale.status]
|| "common.other")}</span>
        </div>
        <button onClick={() => onEdit(sale)} aria-label={t("taskitem.editAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-secondary">
          <Pencil className="w-4 h-4" />
        </button>
        <button onClick={() => setOpen(true)} aria-label={t("taskitem.deleteAria")} className="no-select-tap min-touch p-2
rounded-lg hover:bg-destructive/10 text-destructive">
          <Trash2 className="w-4 h-4" />
```

```
        </button>
      </div>
      <AlertDialog open={open} onOpenChange={setOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>{t("jbm.sale.deleteTitle")}</AlertDialogTitle>
            <AlertDialogDescription>{t("jbm.sale.deleteDesc", { name: sale.item_name })}</AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>{t("common.cancel")}</AlertDialogCancel>
            <AlertDialogAction className="bg-destructive text-destructive-foreground hover:bg-destructive/90" onClick={(e) => {
e.preventDefault(); onDelete(sale); setOpen(false); }}>
              {t("common.delete")}
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/calculator/FonctionsPanel.jsx
------------------------------------------------------------
import React, { useMemo, useState } from "react";
import { Plus, Trash2, Pencil, FunctionSquare, ChevronDown, ChevronRight } from "lucide-react";
import { LineChart, Line, AreaChart, Area, BarChart, Bar, ScatterChart, Scatter, XAxis, YAxis, CartesianGrid, Tooltip,
ResponsiveContainer, ReferenceLine } from "recharts";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { useToast } from "@/components/ui/use-toast";
import { sanitizeName, sampleFunction, studyFunction, formatNumber, computeGrandeurs } from "@/lib/mathEval";

const PLOT_TYPES = [
  { value: "courbe", label: "Courbe" },
  { value: "aire", label: "Aire" },
  { value: "histogramme", label: "Histogramme" },
  { value: "nuage", label: "Nuage de points" },
];
const empty = { name: "", label: "", expression: "", unit: "", plot_variable: "x", plot_x_min: "-10", plot_x_max: "10",
plot_steps: "100", plot_type: "courbe" };

const COLORS = {
  primary: "hsl(var(--primary))",
  accent: "hsl(var(--accent))",
  border: "hsl(var(--border))",
  muted: "hsl(var(--muted-foreground))",
};

function Chart({ f, scope }) {
  const variable = f.plot_variable || "x";
  const xMin = Number(f.plot_x_min) || 0;
  const xMax = Number(f.plot_x_max) || 10;
  const steps = f.plot_type === "histogramme" ? Math.min(24, f.plot_steps || 24) : Math.min(300, f.plot_steps || 100);
  const pts = useMemo(
    () => sampleFunction(f.expression, variable, xMin, xMax, steps, scope),
    [f.expression, variable, xMin, xMax, steps, scope]
  );
  const valid = pts.filter((p) => p.y != null);
  const tip = ({ active, payload }) =>
    active && payload?.length ? (
      <div className="bg-card border border-border rounded-lg px-2 py-1 text-xs">
        {variable}: {payload[0].payload.x} · f: {payload[0].payload.y}
      </div>
    ) : null;
  const axisProps = { domain: [xMin, xMax], type: "number", allowDecimals: true };
  const tick = { fontSize: 11, fill: COLORS.muted };

  if (valid.length === 0)
    return <p className="text-sm text-muted-foreground py-6 text-center">Aucune valeur finie sur cet intervalle.</p>;

  let chart;
  if (f.plot_type === "aire") {
    chart = (
      <AreaChart data={pts}>
        <CartesianGrid strokeDasharray="3 3" stroke={COLORS.border} />
        <XAxis dataKey="x" {...axisProps} tick={tick} />
        <YAxis tick={tick} />
        <Tooltip content={tip} />
        <ReferenceLine y={0} stroke={COLORS.muted} />
        <Area type="monotone" dataKey="y" stroke={COLORS.primary} fill={COLORS.primary} fillOpacity={0.15} dot={false}
connectNulls />
      </AreaChart>
    );
  } else if (f.plot_type === "histogramme") {
    chart = (
      <BarChart data={pts}>
        <CartesianGrid strokeDasharray="3 3" stroke={COLORS.border} />
```

```
        <XAxis dataKey="x" {...axisProps} tick={tick} />
        <YAxis tick={tick} />
        <Tooltip content={tip} />
        <ReferenceLine y={0} stroke={COLORS.muted} />
        <Bar dataKey="y" fill={COLORS.accent} />
      </BarChart>
    );
  } else if (f.plot_type === "nuage") {
    chart = (
      <ScatterChart>
        <CartesianGrid strokeDasharray="3 3" stroke={COLORS.border} />
        <XAxis dataKey="x" {...axisProps} tick={tick} />
        <YAxis dataKey="y" tick={tick} />
        <Tooltip content={tip} />
        <Scatter data={pts} fill={COLORS.primary} />
      </ScatterChart>
    );
  } else {
    chart = (
      <LineChart data={pts}>
        <CartesianGrid strokeDasharray="3 3" stroke={COLORS.border} />
        <XAxis dataKey="x" {...axisProps} tick={tick} />
        <YAxis tick={tick} />
        <Tooltip content={tip} />
        <ReferenceLine y={0} stroke={COLORS.muted} />
        <Line type="monotone" dataKey="y" stroke={COLORS.primary} dot={false} connectNulls strokeWidth={2} />
      </LineChart>
    );
  }

  return (
    <div className="h-64 w-full">
      <ResponsiveContainer width="100%" height="100%">{chart}</ResponsiveContainer>
    </div>
  );
}

export default function FonctionsPanel({ items, metrics, reload }) {
  const { toast } = useToast();
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);
  const [showTable, setShowTable] = useState({});

  const grandeurs = items.filter((i) => i.kind === "grandeur");
  const fonctions = items.filter((i) => i.kind === "fonction");
  const scope = useMemo(() => computeGrandeurs(grandeurs, metrics || {}), [grandeurs, metrics]);

  const openNew = () => { setEditing(null); setForm(empty); setOpen(true); };
  const openEdit = (f) => {
    setEditing(f);
    setForm({
      name: f.name, label: f.label, expression: f.expression || "", unit: f.unit || "",
      plot_variable: f.plot_variable || "x",
      plot_x_min: String(f.plot_x_min ?? -10),
      plot_x_max: String(f.plot_x_max ?? 10),
      plot_steps: String(f.plot_steps ?? 100),
      plot_type: f.plot_type || "courbe",
    });
    setOpen(true);
  };

  const save = async () => {
    const name = sanitizeName(form.name);
    if (!name || !form.expression) { toast({ title: "Nom et formule requis", variant: "destructive" }); return; }
    const payload = {
      name, label: form.label || name, kind: "fonction", expression: form.expression, unit: form.unit,
      plot_variable: form.plot_variable || "x",
      plot_x_min: Number(form.plot_x_min) || 0,
      plot_x_max: Number(form.plot_x_max) || 10,
      plot_steps: Math.max(2, Math.min(500, Number(form.plot_steps) || 100)),
      plot_type: form.plot_type,
    };
    if (payload.plot_x_max <= payload.plot_x_min) { toast({ title: "x_max doit être > x_min", variant: "destructive" }); return;
}
    try {
      if (editing) await base44.entities.CalcDefinition.update(editing.id, payload);
      else await base44.entities.CalcDefinition.create(payload);
      toast({ title: "Fonction enregistrée" });
      setOpen(false); reload();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (f) => {
    try { await base44.entities.CalcDefinition.delete(f.id); reload(); }
    catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between gap-2">
        <h2 className="font-display text-lg flex items-center gap-2"><FunctionSquare className="w-4 h-4" /> Fonctions</h2>
```

```
        <Button onClick={openNew} size="sm"><Plus className="w-4 h-4 mr-1" /> Nouvelle</Button>
      </div>

      {grandeurs.length > 0 && (
        <p className="text-xs text-muted-foreground">Astuce : vos fonctions peuvent référencer vos grandeurs (ex. <span
className="font-mono">CA * x</span>).</p>
      )}

      {fonctions.length === 0 ? (
        <div className="text-center py-10 text-muted-foreground text-sm">Aucune fonction. Créez-en une pour tracer et étudier
une courbe.</div>
      ) : (
        <div className="space-y-4">
          {fonctions.map((f) => {
            const study = studyFunction(f.expression, f.plot_variable || "x", Number(f.plot_x_min) || 0, Number(f.plot_x_max) ||
10, 200, scope);
            const openTbl = showTable[f.id];
            return (
              <div key={f.id} className="bg-card border border-border rounded-2xl p-4 space-y-3">
                <div className="flex items-start justify-between gap-3">
                  <div className="min-w-0">
                    <p className="text-sm font-medium truncate">{f.label} <span className="text-xs text-muted-foreground
font-mono">({f.name})</span></p>
                    <p className="text-sm font-mono text-accent truncate">f({f.plot_variable || "x"}) = {f.expression}{f.unit ?
` [${f.unit}]` : ""}</p>
                    <p className="text-xs text-muted-foreground">[{f.plot_x_min} ; {f.plot_x_max}] · {PLOT_TYPES.find((p) =>
p.value === f.plot_type)?.label}</p>
                  </div>
                  <div className="flex items-center gap-2 shrink-0">
                    <button onClick={() => openEdit(f)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                    <button onClick={() => remove(f)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive"><Trash2 className="w-4 h-4" /></button>
                  </div>
                </div>

                <Chart f={f} scope={scope} />

                <div className="grid grid-cols-3 gap-2 text-xs">
                  <div className="bg-secondary rounded-lg p-2"><p className="text-muted-foreground">Minimum</p><p
className="font-mono font-medium">{formatNumber(study.min)} {study.min != null && `@ ${formatNumber(study.minX)}`}</p></div>
                  <div className="bg-secondary rounded-lg p-2"><p className="text-muted-foreground">Maximum</p><p
className="font-mono font-medium">{formatNumber(study.max)} {study.max != null && `@ ${formatNumber(study.maxX)}`}</p></div>
                  <div className="bg-secondary rounded-lg p-2"><p className="text-muted-foreground">Dérivée (milieu)</p><p
className="font-mono font-medium">{formatNumber(study.derivMid)}</p></div>
                </div>

                <div className="text-xs">
                  <p className="text-muted-foreground">Zéros détectés : {study.roots.length ? study.roots.map((r) =>
formatNumber(r)).join(", ") : "aucun"}</p>
                </div>

                <button onClick={() => setShowTable((s) => ({ ...s, [f.id]: !s[f.id] }))} className="no-select-tap flex
items-center gap-1 text-xs text-primary">
                  {openTbl ? <ChevronDown className="w-3 h-3" /> : <ChevronRight className="w-3 h-3" />} Tableau de valeurs
                </button>
                {openTbl && (
                  <div className="border border-border rounded-lg overflow-hidden text-xs">
                    <div className="grid grid-cols-11 bg-secondary font-mono">
                      <div className="px-2 py-1">{f.plot_variable || "x"}</div>
                      {study.table.map((p, i) => <div key={i} className="px-1 py-1 text-right">{formatNumber(p.x)}</div>)}
                    </div>
                    <div className="grid grid-cols-11 border-t border-border font-mono">
                      <div className="px-2 py-1">f</div>
                      {study.table.map((p, i) => <div key={i} className="px-1 py-1 text-right">{formatNumber(p.y)}</div>)}
                    </div>
                  </div>
                )}
              </div>
            );
          })}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent>
          <DialogHeader><DialogTitle>{editing ? "Modifier la fonction" : "Nouvelle fonction"}</DialogTitle></DialogHeader>
          <div className="space-y-3">
            <div className="grid grid-cols-2 gap-3">
              <div><Label>Nom (identifiant)</Label><Input value={form.name} onChange={(e) => setForm({
...form, name: e.target.value })} placeholder="f, demande…" /></div>
              <div><Label>Libellé</Label><Input value={form.label} onChange={(e) => setForm({ ...form,
label: e.target.value })} placeholder="Coût total" /></div>
            </div>
            <div><Label>Formule f({form.plot_variable || "x"}) =</Label><Input value={form.expression}
onChange={(e) => setForm({ ...form, expression: e.target.value })} placeholder="x^2 + 3*x - 5" /></div>
            <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
              <div><Label>Variable</Label><Input value={form.plot_variable} onChange={(e) => setForm({
...form, plot_variable: e.target.value })} placeholder="x" /></div>
              <div><Label>x min</Label><Input type="number" value={form.plot_x_min} onChange={(e) =>
setForm({ ...form, plot_x_min: e.target.value })} /></div>
              <div><Label>x max</Label><Input type="number" value={form.plot_x_max} onChange={(e) =>
```

```
setForm({ ...form, plot_x_max: e.target.value })} /></div>
              <div className="space-y-1"><Label>Pas</Label><Input type="number" min="2" value={form.plot_steps} onChange={(e) =>
setForm({ ...form, plot_steps: e.target.value })} /></div>
            </div>
            <div className="grid grid-cols-2 gap-3">
              <div className="space-y-1"><Label>Représentation</Label>
                <Select value={form.plot_type} onValueChange={(v) => setForm({ ...form, plot_type: v })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent>{PLOT_TYPES.map((p) => <SelectItem key={p.value}
value={p.value}>{p.label}</SelectItem>)}</SelectContent>
                </Select>
              </div>
              <div className="space-y-1"><Label>Unité</Label><Input value={form.unit} onChange={(e) => setForm({ ...form, unit:
e.target.value })} placeholder="XAF, kg…" /></div>
            </div>
          </div>
          <DialogFooter><Button onClick={save}>Enregistrer</Button></DialogFooter>
        </DialogContent>
      </Dialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/calculator/GrandeursPanel.jsx
------------------------------------------------------------
import React, { useMemo, useState } from "react";
import { Plus, Trash2, Pencil, Link2, Sigma } from "lucide-react";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { useToast } from "@/components/ui/use-toast";
import { sanitizeName, formatNumber, computeGrandeurs } from "@/lib/mathEval";
import { METRIC_GROUPS, METRIC_BY_KEY } from "@/lib/calcSources";
import { formatCurrency } from "@/lib/format";

const empty = { name: "", label: "", expression: "", unit: "", value: "", source_type: "manual", source_key: "" };

export default function GrandeursPanel({ items, metrics, reload }) {
  const { toast } = useToast();
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);

  const grandeurs = items.filter((i) => i.kind === "grandeur");
  const scope = useMemo(() => computeGrandeurs(grandeurs, metrics), [grandeurs, metrics]);

  const openNew = () => { setEditing(null); setForm(empty); setOpen(true); };
  const openEdit = (g) => {
    setEditing(g);
    setForm({ name: g.name, label: g.label, expression: g.expression || "", unit: g.unit || "", value: g.value ?? "",
source_type: g.source_type || "manual", source_key: g.source_key || "" });
    setOpen(true);
  };

  const save = async () => {
    const name = sanitizeName(form.name);
    if (!name) { toast({ title: "Nom requis", variant: "destructive" }); return; }
    const payload = {
      name, label: form.label || name, kind: "grandeur",
      expression: form.expression, unit: form.unit,
      value: form.source_type === "manual" && !form.expression ? (parseFloat(form.value) || 0) : 0,
      source_type: form.source_type, source_key: form.source_type === "manual" ? "" : form.source_key,
    };
    if (form.source_type !== "manual" && !form.source_key) { toast({ title: "Choisir un indicateur", variant: "destructive" });
return; }
    try {
      if (editing) await base44.entities.CalcDefinition.update(editing.id, payload);
      else await base44.entities.CalcDefinition.create(payload);
      toast({ title: "Grandeur enregistrée" });
      setOpen(false); reload();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (g) => {
    try { await base44.entities.CalcDefinition.delete(g.id); reload(); }
    catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const importMetric = async (m) => {
    const name = sanitizeName(m.key);
    try {
      await base44.entities.CalcDefinition.create({ name, label: m.label, kind: "grandeur", source_type: m.source, source_key:
m.key, unit: m.unit });
      toast({ title: `${m.label} importée` });
      reload();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };
```

```
  const displayVal = (g) => {
    const v = scope[g.name];
    if (v == null || !isFinite(v)) return "—";
    return g.unit === "XAF" ? formatCurrency(v, "XAF") : formatNumber(v);
  };

  const metricValue = (m) => {
    const v = metrics?.[m.key];
    if (v == null) return "—";
    return m.unit === "XAF" ? formatCurrency(v, "XAF") : String(v);
  };

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between gap-2">
        <h2 className="font-display text-lg flex items-center gap-2"><Sigma className="w-4 h-4" /> Grandeurs</h2>
        <Button onClick={openNew} size="sm"><Plus className="w-4 h-4 mr-1" /> Nouvelle</Button>
      </div>

      <div className="bg-card border border-border rounded-2xl p-4 space-y-3">
        <p className="text-sm font-medium flex items-center gap-2"><Link2 className="w-4 h-4 text-accent" /> Importer depuis vos
sections</p>
        {METRIC_GROUPS.map((grp) => (
          <div key={grp.source} className="space-y-1.5">
            <p className="text-xs text-muted-foreground">{grp.label}</p>
            <div className="flex flex-wrap gap-2">
              {grp.metrics.map((m) => (
                <button
                  key={m.key}
                  onClick={() => importMetric(m)}
                  className="no-select-tap text-xs px-3 py-1.5 rounded-lg border border-border bg-secondary
hover:bg-secondary/70 flex items-center gap-1.5"
                >
                  <Plus className="w-3 h-3" /> {m.label}
                  <span className="text-muted-foreground">: {metricValue(m)}</span>
                </button>
              ))}
            </div>
          </div>
        ))}
      </div>

      {grandeurs.length === 0 ? (
        <div className="text-center py-10 text-muted-foreground text-sm">Aucune grandeur. Créez-en une ou importez un indicateur
ci-dessus.</div>
      ) : (
        <div className="space-y-2">
          {grandeurs.map((g) => (
            <div key={g.id} className="bg-card border border-border rounded-xl p-3 flex items-center justify-between gap-3">
              <div className="min-w-0">
                <p className="text-sm font-medium truncate">{g.label} <span className="text-xs text-muted-foreground
font-mono">({g.name})</span></p>
                <p className="text-xs text-muted-foreground truncate">
                  {g.source_type === "manual" ? (g.expression ? `= ${g.expression}` : "valeur saisie") : `lié ·
${METRIC_BY_KEY[g.source_key]?.label || g.source_key}`}
                  {g.unit ? ` · ${g.unit}` : ""}
                </p>
              </div>
              <div className="flex items-center gap-2 shrink-0">
                <span className="text-sm font-mono font-medium">{displayVal(g)}</span>
                <button onClick={() => openEdit(g)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                <button onClick={() => remove(g)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive"><Trash2 className="w-4 h-4" /></button>
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent>
          <DialogHeader><DialogTitle>{editing ? "Modifier la grandeur" : "Nouvelle grandeur"}</DialogTitle></DialogHeader>
          <div className="space-y-3">
            <div className="grid grid-cols-2 gap-3">
              <div className="space-y-1"><Label>Nom (identifiant)</Label><Input value={form.name} onChange={(e) => setForm({
...form, name: e.target.value })} placeholder="CA, prix_unitaire…" /></div>
              <div className="space-y-1"><Label>Libellé</Label><Input value={form.label} onChange={(e) => setForm({ ...form,
label: e.target.value })} placeholder="Chiffre d'affaires" /></div>
            </div>
            <div className="space-y-1"><Label>Source</Label>
              <Select value={form.source_type} onValueChange={(v) => setForm({ ...form, source_type: v, source_key: "",
expression: "" })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  <SelectItem value="manual">Manuelle (valeur ou formule)</SelectItem>
                  {METRIC_GROUPS.map((g) => <SelectItem key={g.source} value={g.source}>{g.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
            {form.source_type !== "manual" ? (
              <div className="space-y-1"><Label>Indicateur</Label>
```

```
              <Select value={form.source_key} onValueChange={(v) => setForm({ ...form, source_key: v })}>
                <SelectTrigger><SelectValue placeholder="Choisir…" /></SelectTrigger>
                <SelectContent>
                  {(METRIC_GROUPS.find((g) => g.source === form.source_type)?.metrics || []).map((m) => (
                    <SelectItem key={m.key} value={m.key}>{m.label}</SelectItem>
                  ))}
                </SelectContent>
              </Select>
            </div>
          ) : (
            <>
              <div className="space-y-1"><Label>Formule (optionnel — utilise les autres grandeurs)</Label><Input
value={form.expression} onChange={(e) => setForm({ ...form, expression: e.target.value })} placeholder="CA - couts" /></div>
              {!form.expression && <div className="space-y-1"><Label>Valeur</Label><Input type="number" value={form.value}
onChange={(e) => setForm({ ...form, value: e.target.value })} /></div>}
            </>
          )}
          <div className="space-y-1"><Label>Unité</Label><Input value={form.unit} onChange={(e) => setForm({ ...form, unit:
e.target.value })} placeholder="XAF, kg, …" /></div>
        </div>
        <DialogFooter><Button onClick={save}>Enregistrer</Button></DialogFooter>
      </DialogContent>
    </Dialog>
  </div>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/calculator/QuickTools.jsx
------------------------------------------------------------
import React, { useState, useMemo } from "react";
import { TrendingUp, Percent, Coins } from "lucide-react";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { formatCurrency } from "@/lib/format";

export default function QuickTools() {
  const [cost, setCost] = useState("");
  const [price, setPrice] = useState("");
  const [ht, setHt] = useState("");
  const [rate, setRate] = useState("18");
  const [loan, setLoan] = useState("");
  const [apr, setApr] = useState("");
  const [months, setMonths] = useState("12");

  const margin = useMemo(() => {
    const c = parseFloat(cost) || 0, p = parseFloat(price) || 0;
    const m = p - c;
    return { m, pct: p > 0 ? (m / p) * 100 : 0 };
  }, [cost, price]);

  const tva = useMemo(() => {
    const h = parseFloat(ht) || 0, r = parseFloat(rate) || 0;
    const tax = h * (r / 100);
    return { tax, ttc: h + tax };
  }, [ht, rate]);

  const loanCalc = useMemo(() => {
    const P = parseFloat(loan) || 0;
    const r = (parseFloat(apr) || 0) / 100 / 12;
    const n = parseInt(months, 10) || 0;
    if (P <= 0 || n <= 0) return { monthly: 0, total: 0, interest: 0 };
    const monthly = r === 0 ? P / n : (P * r) / (1 - Math.pow(1 + r, -n));
    const total = monthly * n;
    return { monthly, total, interest: total - P };
  }, [loan, apr, months]);

  return (
    <div className="space-y-4 max-w-2xl">
      <section className="bg-card border border-border rounded-2xl p-5 space-y-4">
        <h2 className="font-display text-lg flex items-center gap-2"><TrendingUp className="w-4 h-4" /> Marge / Rentabilité</h2>
        <div className="grid grid-cols-2 gap-3">
          <div><Label>Coût unitaire (XAF)</Label><Input type="number" min="0" value={cost} onChange={(e)
=> setCost(e.target.value)} /></div>
          <div><Label>Prix de vente (XAF)</Label><Input type="number" min="0" value={price} onChange={(e)
=> setPrice(e.target.value)} /></div>
        </div>
        <div className="text-sm space-y-1">
          <p>Marge : <span className="font-medium">{formatCurrency(margin.m, "XAF")}</span></p>
          <p>Taux de marge : <span className="font-medium">{margin.pct.toFixed(1)} %</span></p>
        </div>
      </section>

      <section className="bg-card border border-border rounded-2xl p-5 space-y-4">
        <h2 className="font-display text-lg flex items-center gap-2"><Percent className="w-4 h-4" /> TVA</h2>
        <div className="grid grid-cols-2 gap-3">
          <div><Label>Montant HT (XAF)</Label><Input type="number" min="0" value={ht} onChange={(e) =>
setHt(e.target.value)} /></div>
          <div><Label>Taux TVA (%)</Label><Input type="number" min="0" value={rate} onChange={(e) =>
setRate(e.target.value)} /></div>
        </div>
        <div className="text-sm space-y-1">
```

```
          <p>TVA : <span className="font-medium">{formatCurrency(tva.tax, "XAF")}</span></p>
          <p>Montant TTC : <span className="font-medium">{formatCurrency(tva.ttc, "XAF")}</span></p>
        </div>
      </section>

      <section className="bg-card border border-border rounded-2xl p-5 space-y-4">
        <h2 className="font-display text-lg flex items-center gap-2"><Coins className="w-4 h-4" /> Prêt / Remboursement</h2>
        <div className="grid grid-cols-1 sm:grid-cols-3 gap-3">
          <div className="space-y-1"><Label>Montant (XAF)</Label><Input type="number" min="0" value={loan} onChange={(e) =>
setLoan(e.target.value)} /></div>
          <div className="space-y-1"><Label>Taux annuel (%)</Label><Input type="number" min="0" value={apr} onChange={(e) =>
setApr(e.target.value)} /></div>
          <div className="space-y-1"><Label>Durée (mois)</Label><Input type="number" min="1" value={months} onChange={(e) =>
setMonths(e.target.value)} /></div>
        </div>
        <div className="text-sm space-y-1">
          <p>Mensualité : <span className="font-medium">{formatCurrency(loanCalc.monthly, "XAF")}</span></p>
          <p>Coût total : <span className="font-medium">{formatCurrency(loanCalc.total, "XAF")}</span></p>
          <p>Intérêts : <span className="font-medium">{formatCurrency(loanCalc.interest, "XAF")}</span></p>
        </div>
      </section>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/calculator/ScientificCalculator.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Delete } from "lucide-react";
import { evalExpr, formatNumber } from "@/lib/mathEval";

const KEYS = [
  ["7", "8", "9", "/", "C"],
  ["4", "5", "6", "*", "("],
  ["1", "2", "3", "-", ")"],
  ["0", ".", "^", "+", "="],
  ["sin(", "cos(", "tan(", "sqrt(", "pi"],
  ["ln(", "log(", "abs(", "exp(", "e"],
];

export default function ScientificCalculator({ scope = {} }) {
  const [expr, setExpr] = useState("");
  const [result, setResult] = useState("");
  const [history, setHistory] = useState([]);

  const press = (k) => {
    if (k === "C") { setExpr(""); setResult(""); return; }
    if (k === "=") { compute(); return; }
    setExpr((e) => e + k);
  };

  const compute = () => {
    if (!expr.trim()) return;
    try {
      const r = evalExpr(expr, scope);
      if (!isFinite(r)) { setResult("Erreur"); return; }
      const out = formatNumber(r);
      setResult(out);
      setHistory((h) => [{ expr, result: out, id: Date.now() }, ...h].slice(0, 8));
    } catch { setResult("Erreur"); }
  };

  const backspace = () => setExpr((e) => e.slice(0, -1));

  return (
    <div className="bg-card border border-border rounded-2xl p-5 space-y-4 max-w-2xl">
      <div className="space-y-2">
        <div className="min-h-[2.5rem] text-right text-2xl font-mono break-all">{expr || "0"}</div>
        <div className="min-h-[2rem] text-right text-xl font-mono text-accent">{result && "= " + result}</div>
      </div>
      <div className="grid grid-cols-5 gap-2">
        {KEYS.flat().map((k) => (
          <button
            key={k}
            onClick={() => press(k)}
            className={`no-select-tap min-touch rounded-xl text-sm font-mono transition-colors ${
              k === "=" ? "bg-primary text-primary-foreground" : "bg-secondary hover:bg-secondary/70"
            }`}
          >
            {k}
          </button>
        ))}
      </div>
      <div className="flex gap-2">
        <button onClick={backspace} className="no-select-tap min-touch flex-1 rounded-xl bg-secondary hover:bg-secondary/70 flex
items-center justify-center gap-1 text-sm">
          <Delete className="w-4 h-4" /> Effacer
        </button>
      </div>
      {history.length > 0 && (
        <div className="space-y-1 pt-2 border-t border-border">
```

```
          <p className="text-xs text-muted-foreground">Historique</p>
          {history.map((h) => (
            <button
              key={h.id}
              onClick={() => { setExpr(h.expr); setResult(h.result); }}
              className="block w-full text-left text-sm font-mono py-1 hover:bg-secondary rounded px-2"
            >
              {h.expr} = <span className="text-accent">{h.result}</span>
            </button>
          ))}
        </div>
      )}
      <p className="text-xs text-muted-foreground">
        Fonctions : sin, cos, tan, sqrt, ln, log, exp, abs, ^, pi, e. Les grandeurs que vous créez dans l'onglet « Grandeurs »
sont aussi utilisables ici via leur nom.
      </p>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/categories/CategorySelect.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Plus } from "lucide-react";

const NEW_VALUE = "__new__";

// Sélecteur de catégorie réutilisable : liste les catégories existantes pour
// (scope, domain, company_id) et permet d'en créer une nouvelle à la volée.
export default function CategorySelect({ scope, domain, companyId = null, value, onChange, placeholder = "Catégorie" }) {
  const [options, setOptions] = useState([]);
  const [creating, setCreating] = useState(false);
  const [newName, setNewName] = useState("");
  const [loading, setLoading] = useState(false);

  const load = async () => {
    setLoading(true);
    try {
      const list = await base44.entities.Category.list("-created_date", 500);
      const filtered = (list || []).filter(
        (c) => c.scope === scope && c.domain === domain && (!c.company_id || c.company_id === companyId)
      );
      setOptions(filtered);
    } catch {
      setOptions([]);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { load(); /* recharge au changement de contexte */ }, [scope, domain, companyId]);

  const handleValueChange = (v) => {
    if (v === NEW_VALUE) {
      setCreating(true);
      return;
    }
    const found = options.find((o) => o.id === v);
    onChange(found ? found.name : v);
  };

  const handleCreate = async () => {
    const name = newName.trim();
    if (!name) { setCreating(false); return; }
    try {
      const created = await base44.entities.Category.create({ name, scope, domain, company_id: companyId || undefined });
      setOptions((prev) => [created, ...prev]);
      onChange(name);
    } catch {
      onChange(name); // fallback : on conserve le texte même si la persistance échoue
    } finally {
      setNewName("");
      setCreating(false);
    }
  };

  const selectedId = options.find((o) => o.name === value)?.id || "";

  return (
    <div className="space-y-2">
      {creating ? (
        <div className="flex items-center gap-2">
          <Input
            autoFocus
            value={newName}
            onChange={(e) => setNewName(e.target.value)}
            placeholder="Nom de la nouvelle catégorie"
```

```
            onKeyDown={(e) => { if (e.key === "Enter") { e.preventDefault(); handleCreate(); } }}
          />
          <Button type="button" size="sm" onClick={handleCreate}>Ajouter</Button>
          <Button type="button" size="sm" variant="ghost" onClick={() => setCreating(false)}>Annuler</Button>
        </div>
      ) : (
        <Select value={selectedId || (value && !options.find((o) => o.name === value) ? value : "")}
onValueChange={handleValueChange} disabled={loading}>
          <SelectTrigger>
            <SelectValue placeholder={placeholder} />
          </SelectTrigger>
          <SelectContent>
            {options.map((o) => (
              <SelectItem key={o.id} value={o.id}>{o.name}</SelectItem>
            ))}
            {value && !options.find((o) => o.name === value) && (
              <SelectItem value={value}>{value} (hors liste)</SelectItem>
            )}
            <SelectItem value={NEW_VALUE}>
              <span className="flex items-center gap-1 text-accent"><Plus className="w-3.5 h-3.5" /> Nouvelle catégorie</span>
            </SelectItem>
          </SelectContent>
        </Select>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/collab/ChatView.jsx
------------------------------------------------------------
import React, { useEffect, useRef, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Send, Paperclip, Loader2 } from "lucide-react";
import { Image } from "@/components/ui/image";
import { canWriteInGroup, canSendMediaInGroup } from "@/lib/collabUtils";
import { useToast } from "@/components/ui/use-toast";

const IMAGE_EXT = /\.(png|jpe?g|gif|webp)$/i;

export default function ChatView({ conversationType, conversationId, participants, group, currentUserId }) {
  const [messages, setMessages] = useState([]);
  const [text, setText] = useState("");
  const [loading, setLoading] = useState(true);
  const [sending, setSending] = useState(false);
  const fileRef = useRef(null);
  const scrollRef = useRef(null);
  const { toast } = useToast();

  const canWrite = conversationType === "direct" ? true : canWriteInGroup(currentUserId, group);
  const canMedia = conversationType === "direct" ? true : canSendMediaInGroup(currentUserId, group);

  const load = async () => {
    setLoading(true);
    try {
      const list = await base44.entities.Message.filter({ conversation_id: conversationId }, "-created_date", 200);
      setMessages((list || []).reverse());
    } catch {
      setMessages([]);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    load();
    const unsubscribe = base44.entities.Message.subscribe((event) => {
      if (event.data?.conversation_id !== conversationId) return;
      setMessages((prev) => {
        if (event.type === "create" && !prev.find((m) => m.id === event.data.id)) return [...prev, event.data];
        if (event.type === "delete") return prev.filter((m) => m.id !== event.data.id);
        return prev;
      });
    });
    return unsubscribe;
  }, [conversationId]);

  useEffect(() => { scrollRef.current?.scrollTo({ top: scrollRef.current?.scrollHeight, behavior: "smooth" }); }, [messages]);

  const send = async (e) => {
    e?.preventDefault();
    const content = text.trim();
    if (!content || sending) return;
    setSending(true);
    try {
      await base44.entities.Message.create({
        conversation_type: conversationType,
        conversation_id: conversationId,
        participants,
        content,
```

```
      });
      setText("");
    } catch (err) {
      toast({ title: "Erreur", description: err?.message, variant: "destructive" });
    } finally {
      setSending(false);
    }
  };

  const onFile = async (e) => {
    const file = e.target.files?.[0];
    e.target.value = "";
    if (!file || !canMedia || sending) return;
    setSending(true);
    try {
      const { file_url } = await base44.integrations.Core.UploadPublicFile({ file });
      await base44.entities.Message.create({
        conversation_type: conversationType,
        conversation_id: conversationId,
        participants,
        content: "",
        attachment_url: file_url,
        attachment_name: file.name,
      });
    } catch (err) {
      toast({ title: "Erreur", description: err?.message, variant: "destructive" });
    } finally {
      setSending(false);
    }
  };

  return (
    <div className="flex flex-col h-[60vh] border border-border rounded-2xl bg-card">
      <div ref={scrollRef} className="flex-1 overflow-y-auto p-3 space-y-2">
        {loading ? (
          <div className="flex justify-center py-8"><Loader2 className="w-5 h-5 animate-spin text-muted-foreground" /></div>
        ) : messages.length === 0 ? (
          <p className="text-center text-sm text-muted-foreground py-8">Aucun message. Démarrez la conversation.</p>
        ) : messages.map((m) => {
          const mine = m.created_by_id === currentUserId;
          const isImg = m.attachment_url && IMAGE_EXT.test(m.attachment_url);
          return (
            <div key={m.id} className={`flex ${mine ? "justify-end" : "justify-start"}`}>
              <div className={`max-w-[78%] rounded-2xl px-3 py-2 text-sm ${mine ? "bg-primary text-primary-foreground" :
"bg-muted text-foreground"}`}>
                <p className="text-[10px] opacity-70 mb-0.5">{m.created_by || ""}</p>
                {m.content && <p className="whitespace-pre-wrap break-words">{m.content}</p>}
                {m.attachment_url && (isImg ? (
                  <Image src={m.attachment_url} alt={m.attachment_name || ""} className="mt-1 rounded-lg max-h-48 w-auto
max-w-full object-cover" />
                ) : (
                  <a href={m.attachment_url} target="_blank" rel="noreferrer" className="block mt-1 underline
text-xs">{m.attachment_name || "Fichier"}</a>
                ))}
              </div>
            </div>
          );
        })}
      </div>
      {!canWrite ? (
        <div className="p-3 text-center text-xs text-muted-foreground border-t border-border">
          L'écriture est réservée aux administrateurs / membres autorisés.
        </div>
      ) : (
        <form onSubmit={send} className="flex items-center gap-2 p-2 border-t border-border">
          <input ref={fileRef} type="file" className="hidden" onChange={onFile} accept="image/*,application/pdf,.doc,.docx,.txt"
/>
          <Button type="button" size="icon" variant="ghost" onClick={() => fileRef.current?.click()} disabled={!canMedia ||
sending} title={canMedia ? "Joindre un fichier" : "Médias restreints"}>
            <Paperclip className="w-4 h-4" />
          </Button>
          <Input value={text} onChange={(e) => setText(e.target.value)} placeholder="Écrire un message…" />
          <Button type="submit" size="icon" disabled={sending || !text.trim()}>
            {sending ? <Loader2 className="w-4 h-4 animate-spin" /> : <Send className="w-4 h-4" />}
          </Button>
        </form>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/collab/CompanySearch.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { UserPlus, Loader2, Building2, MapPin } from "lucide-react";
import { useToast } from "@/components/ui/use-toast";
import { BUSINESS_TYPES } from "@/lib/businessTypes";
```

```
export default function CompanySearch({ currentUserId, excludeIds = [], onSent }) {
  const [typeTerm, setTypeTerm] = useState("");
  const [locTerm, setLocTerm] = useState("");
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const [sending, setSending] = useState(null);
  const { toast } = useToast();

  useEffect(() => {
    const term = typeTerm.trim().toLowerCase();
    const loc = locTerm.trim().toLowerCase();
    const hasTerm = term.length >= 2 || loc.length >= 2;
    if (!hasTerm) { setResults([]); return; }
    let cancelled = false;
    setLoading(true);
    const timer = setTimeout(async () => {
      try {
        const list = await base44.entities.Company.list("-created_date", 500);
        if (cancelled) return;
        const matched = (list || []).filter((c) => {
          const bt = (c.business_type || "").toLowerCase();
          const lo = (c.location || "").toLowerCase();
          const typeOk = term.length >= 2 ? bt.includes(term) : true;
          const locOk = loc.length >= 2 ? lo.includes(loc) : true;
          return typeOk && locOk;
        });
        setResults(matched);
      } catch {
        if (!cancelled) setResults([]);
      } finally {
        if (!cancelled) setLoading(false);
      }
    }, 300);
    return () => { cancelled = true; clearTimeout(timer); };
  }, [typeTerm, locTerm]);

  const sendRequest = async (c) => {
    setSending(c.id);
    try {
      await base44.entities.CollaborationRequest.create({
        to_user_id: c.created_by_id,
        to_name: c.name,
        status: "pending",
      });
      toast({ title: "Demande envoyée", description: c.name });
      onSent?.();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message || "Échec de l'envoi", variant: "destructive" });
    } finally {
      setSending(null);
    }
  };

  const hasSearched = typeTerm.trim().length >= 2 || locTerm.trim().length >= 2;

  return (
    <div className="space-y-3">
      <div className="grid grid-cols-1 sm:grid-cols-2 gap-2">
        <Input
          list="business-types-list"
          value={typeTerm}
          onChange={(e) => setTypeTerm(e.target.value)}
          placeholder="Type d'entreprise (ex: Restauration)"
        />
        <Input
          value={locTerm}
          onChange={(e) => setLocTerm(e.target.value)}
          placeholder="Lieu (optionnel)"
        />
        <datalist id="business-types-list">
          {BUSINESS_TYPES.map((bt) => <option key={bt} value={bt} />)}
        </datalist>
      </div>

      {loading && (
        <div className="flex items-center gap-2 text-xs text-muted-foreground">
          <Loader2 className="w-3.5 h-3.5 animate-spin" /> Recherche…
        </div>
      )}

      {!loading && hasSearched && results.length === 0 && (
        <p className="text-xs text-muted-foreground text-center py-2">Aucune entreprise trouvée.</p>
      )}

      {results.length > 0 && (
        <div className="space-y-2">
          {results.map((c) => {
            const isSelf = c.created_by_id === currentUserId;
            const excluded = excludeIds.includes(c.created_by_id);
            return (
              <div key={c.id} className="flex items-center justify-between gap-2 bg-card border border-border rounded-xl p-2.5">
                <div className="min-w-0">
```

```
                <p className="text-sm font-medium truncate flex items-center gap-1.5">
                  <Building2 className="w-3.5 h-3.5 text-accent shrink-0" /> {c.name}
                </p>
                <p className="text-xs text-muted-foreground flex items-center gap-2 truncate">
                  <span>{c.business_type || "—"}</span>
                  {c.location && (
                    <span className="flex items-center gap-0.5">
                      <MapPin className="w-3 h-3" /> {c.location}
                    </span>
                  )}
                </p>
              </div>
              <Button
                type="button"
                size="sm"
                variant="outline"
                disabled={isSelf || excluded || sending === c.id}
                onClick={() => sendRequest(c)}
                className="flex items-center gap-1 shrink-0"
              >
                {sending === c.id ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <UserPlus className="w-3.5 h-3.5" />}
                {isSelf ? "Vous" : excluded ? "Collaborateur" : "Collaboration"}
              </Button>
            </div>
          );
        })}
      </div>
    )}
  </div>
);
}

------------------------------------------------------------
FICHIER : ./src/components/collab/GroupAdmin.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Search, X, Crown, ShieldCheck, ShieldOff, Pencil, ImageIcon, Trash2, LogOut, UserPlus } from "lucide-react";
import { isGroupAdmin, isGroupCreator } from "@/lib/collabUtils";
import { useToast } from "@/components/ui/use-toast";

export default function GroupAdmin({ group, currentUserId, nameMap, onChanged }) {
  const [q, setQ] = useState("");
  const [results, setResults] = useState([]);
  const { toast } = useToast();
  const admin = isGroupAdmin(currentUserId, group);
  const creator = isGroupCreator(currentUserId, group);

  const update = async (patch) => {
    try {
      await base44.entities.Group.update(group.id, patch);
      onChanged?.();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    }
  };

  const removeMember = async (uid) => {
    try {
      await base44.functions.invoke('manageGroup', { action: "remove_member", group_id: group.id, target_id: uid });
      onChanged?.();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    }
  };

  const toggleAdmin = (uid) => {
    if (uid === group.created_by_id) return;
    const admins = group.admins || [];
    update({ admins: admins.includes(uid) ? admins.filter((a) => a !== uid) : [...admins, uid] });
  };

  const toggleAllowed = (uid, field) => {
    const arr = group[field] || [];
    update({ [field]: arr.includes(uid) ? arr.filter((a) => a !== uid) : [...arr, uid] });
  };

  const search = async () => {
    const term = q.trim();
    if (term.length < 2) { setResults([]); return; }
    const list = await base44.entities.Collaborator.list("-created_date", 200);
    const t = term.toLowerCase();
    setResults((list || []).filter(
      (c) => (c.display_name || "").toLowerCase().includes(t)
        && !(group.members || []).includes(c.created_by_id)
        && c.created_by_id !== currentUserId
    ));
  };

  const invite = async (c) => {
```

```
    try {
      await base44.entities.GroupInvite.create({
        group_id: group.id, group_name: group.name,
        to_user_id: c.created_by_id, to_name: c.display_name, status: "pending",
      });
      toast({ title: "Invitation envoyée", description: c.display_name });
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    }
  };

  const leave = async () => {
    await removeMember(currentUserId);
    onChanged?.();
  };

  const del = async () => {
    try { await base44.entities.Group.delete(group.id); onChanged?.(); }
    catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
  };

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="font-display text-lg flex items-center gap-1.5">{group.name}</h3>
          <p className="text-xs text-muted-foreground">Créé par {nameMap[group.created_by_id] || group.created_by}</p>
        </div>
        {creator ? (
          <Button size="sm" variant="destructive" onClick={del} className="flex items-center gap-1"><Trash2 className="w-4 h-4"
/> Supprimer</Button>
        ) : (
          <Button size="sm" variant="outline" onClick={leave} className="flex items-center gap-1"><LogOut className="w-4 h-4" />
Quitter</Button>
        )}
      </div>

      {admin && (
        <div className="flex flex-wrap gap-2">
          <Button size="sm" variant={group.write_restricted ? "default" : "outline"} onClick={() => update({ write_restricted:
!group.write_restricted })} className="flex items-center gap-1">
            <Pencil className="w-4 h-4" /> Écriture restreinte
          </Button>
          <Button size="sm" variant={group.media_restricted ? "default" : "outline"} onClick={() => update({ media_restricted:
!group.media_restricted })} className="flex items-center gap-1">
            <ImageIcon className="w-4 h-4" /> Médias restreints
          </Button>
        </div>
      )}

      <div className="space-y-2">
        <p className="text-xs font-medium text-muted-foreground">Membres ({(group.members || []).length})</p>
        {(group.members || []).map((uid) => {
          const isCreator = uid === group.created_by_id;
          const isAdmin = (group.admins || []).includes(uid) || isCreator;
          return (
            <div key={uid} className="flex items-center justify-between gap-2 bg-card border border-border rounded-xl p-2.5">
              <div className="flex items-center gap-2 min-w-0">
                <span className="text-sm font-medium truncate">{nameMap[uid] || uid}</span>
                {isCreator && <Crown className="w-3.5 h-3.5 text-accent shrink-0" />}
                {!isCreator && isAdmin && <ShieldCheck className="w-3.5 h-3.5 text-primary shrink-0" />}
              </div>
              {admin && !isCreator && (
                <div className="flex items-center gap-1">
                  {creator && (
                    <Button size="icon" variant="ghost" title={isAdmin ? "Retirer admin" : "Promouvoir admin"} onClick={() =>
toggleAdmin(uid)}>
                      {isAdmin ? <ShieldOff className="w-4 h-4" /> : <ShieldCheck className="w-4 h-4" />}
                    </Button>
                  )}
                  {group.write_restricted && (
                    <Button size="icon" variant={(group.allowed_writers||[]).includes(uid) ? "default" : "ghost"}
title="Autorisé à écrire" onClick={() => toggleAllowed(uid, "allowed_writers")}>
                      <Pencil className="w-4 h-4" />
                    </Button>
                  )}
                  {group.media_restricted && (
                    <Button size="icon" variant={(group.allowed_media||[]).includes(uid) ? "default" : "ghost"} title="Autorisé
médias" onClick={() => toggleAllowed(uid, "allowed_media")}>
                      <ImageIcon className="w-4 h-4" />
                    </Button>
                  )}
                  <Button size="icon" variant="ghost" onClick={() => removeMember(uid)}><X className="w-4 h-4 text-destructive"
/></Button>
                </div>
              )}
            </div>
          );
        })}
      </div>

      {admin && (
```

```
        <div className="space-y-2">
          <p className="text-xs font-medium text-muted-foreground flex items-center gap-1"><UserPlus className="w-3.5 h-3.5" />
Inviter un membre</p>
          <div className="flex items-center gap-2">
            <Input value={q} onChange={(e) => setQ(e.target.value)} onKeyDown={(e) => { if (e.key === "Enter") {
e.preventDefault(); search(); } }} placeholder="Rechercher un utilisateur…" />
            <Button size="icon" variant="outline" onClick={search}><Search className="w-4 h-4" /></Button>
          </div>
          {results.map((c) => (
            <div key={c.id} className="flex items-center justify-between bg-card border border-border rounded-xl p-2.5">
              <span className="text-sm font-medium truncate">{c.display_name}</span>
              <Button size="sm" variant="outline" onClick={() => invite(c)}>Inviter</Button>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/collab/ReportDialog.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogDescription, DialogFooter,
} from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Select, SelectTrigger, SelectValue, SelectContent, SelectItem } from "@/components/ui/select";
import { useToast } from "@/components/ui/use-toast";
import { Loader2, Flag } from "lucide-react";

const REASONS = [
  ["spam", "Spam ou pub non sollicitée"],
  ["harcelement", "Harcèlement"],
  ["contenu_inapproprie", "Contenu inapproprié"],
  ["arnaque", "Arnaque / fraude"],
  ["usurpation", "Usurpation d'identité"],
  ["autre", "Autre"],
];

export default function ReportDialog({ open, onOpenChange, target }) {
  const { toast } = useToast();
  const [reason, setReason] = useState("spam");
  const [custom, setCustom] = useState("");
  const [busy, setBusy] = useState(false);

  useEffect(() => {
    if (open) { setReason("spam"); setCustom(""); }
  }, [open]);

  const submit = async () => {
    if (!target) return;
    setBusy(true);
    try {
      await base44.entities.Report.create({
        target_type: target.type,
        target_id: target.id,
        target_name: target.name || "",
        reason,
        custom_reason: custom.trim(),
      });
      toast({ title: "Signalement envoyé", description: "Les administrateurs ont été notifiés." });
      onOpenChange?.(false);
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally {
      setBusy(false);
    }
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="flex items-center gap-2"><Flag className="w-4 h-4 text-accent" /> Signaler</DialogTitle>
          <DialogDescription>
            {target?.type === "group" ? "Signaler le groupe" : "Signaler l'abonné"} « {target?.name || ""} ». Les
administrateurs examineront votre signalement.
          </DialogDescription>
        </DialogHeader>

        <div className="space-y-3">
          <div className="space-y-1.5">
            <Label className="text-xs">Raison</Label>
            <Select value={reason} onValueChange={setReason}>
              <SelectTrigger className="h-9"><SelectValue /></SelectTrigger>
              <SelectContent>
                {REASONS.map(([v, l]) => <SelectItem key={v} value={v}>{l}</SelectItem>)}
```

```
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-1.5">
          <Label className="text-xs">Précisions (optionnel)</Label>
          <Textarea
            value={custom}
            onChange={(e) => setCustom(e.target.value)}
            placeholder="Décrivez ce qui s'est passé…"
            rows={3}
          />
        </div>
      </div>

      <DialogFooter>
        <Button variant="outline" onClick={() => onOpenChange?.(false)}>Annuler</Button>
        <Button onClick={submit} disabled={busy}>
          {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : "Envoyer le signalement"}
        </Button>
      </DialogFooter>
    </DialogContent>
  </Dialog>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/collab/SharedPanel.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectTrigger, SelectValue, SelectContent, SelectItem } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { ListChecks, Wallet, CalendarDays, Handshake, Plus, Trash2, Loader2, UsersRound } from "lucide-react";
import { useToast } from "@/components/ui/use-toast";
import { canWriteInGroup } from "@/lib/collabUtils";

const PERIODS = [
  { value: "jour", label: "Jour" },
  { value: "semaine", label: "Semaine" },
  { value: "trimestre", label: "Trimestre" },
  { value: "mois", label: "Mois" },
  { value: "annee", label: "Année" },
];

function fmtMoney(n) {
  if (n == null || isNaN(n)) return "-";
  return new Intl.NumberFormat("fr-FR").format(Number(n)) + " XAF";
}

function toLocalInput(d) {
  const dt = new Date(d);
  const pad = (x) => String(x).padStart(2, "0");
  return `${dt.getFullYear()}-${pad(dt.getMonth() + 1)}-${pad(dt.getDate())}T${pad(dt.getHours())}:${pad(dt.getMinutes())}`;
}

function memberName(id, nameMap, fallback) {
  return nameMap?.[id] || (id === fallback ? "Vous" : (id || "").slice(-4));
}

export default function SharedPanel({ conversation, currentUserId, nameMap }) {
  const { toast } = useToast();
  const [domain, setDomain] = useState("taches");
  const [items, setItems] = useState({ taches: [], budget: [], evenements: [], partenariats: [] });
  const [loading, setLoading] = useState(false);
  const [showAdd, setShowAdd] = useState(false);

  const canManage =
    conversation?.type === "group"
      ? canWriteInGroup(currentUserId, conversation.group)
      : true;

  const participants = conversation?.participants || [];

  const loadDomain = async (d) => {
    const map = { taches: "Task", budget: "Expense", evenements: "Event", partenariats: "Partnership" };
    try {
      const list = await base44.entities[map[d]].filter({ shared_context_id: conversation.id });
      setItems((prev) => ({ ...prev, [d]: list || [] }));
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    }
  };

  useEffect(() => {
    if (!conversation) return;
    setLoading(true);
    Promise.all(["taches", "budget", "evenements", "partenariats"].map(loadDomain)).finally(() => setLoading(false));
    // eslint-disable-next-line
  }, [conversation?.id]);
```

```
  const remove = async (d, id) => {
    const map = { taches: "Task", budget: "Expense", evenements: "Event", partenariats: "Partnership" };
    if (!confirm("Supprimer cet élément partagé ?")) return;
    try {
      await base44.entities[map[d]].delete(id);
      setItems((prev) => ({ ...prev, [d]: prev[d].filter((x) => x.id !== id) }));
    } catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
  };

  const tabs = [
    { id: "taches", label: "Tâches", icon: ListChecks, count: items.taches.length },
    { id: "budget", label: "Budget", icon: Wallet, count: items.budget.length },
    { id: "evenements", label: "Événements", icon: CalendarDays, count: items.evenements.length },
    { id: "partenariats", label: "Partenariats", icon: Handshake, count: items.partenariats.length },
  ];

  return (
    <div className="bg-card border border-border rounded-2xl p-3 space-y-3">
      <div className="flex items-center gap-1.5 text-sm font-medium">
        <UsersRound className="w-4 h-4 text-accent" /> Espace partagé
        {!canManage && <span className="text-xs text-muted-foreground ml-auto">Lecture seule</span>}
      </div>

      <div className="flex gap-1 overflow-x-auto no-scrollbar">
        {tabs.map((t) => (
          <button key={t.id} onClick={() => setDomain(t.id)}
            className={`no-select-tap flex items-center gap-1 px-2.5 py-1.5 rounded-lg text-xs whitespace-nowrap ${domain ===
t.id ? "bg-primary text-primary-foreground" : "bg-muted/50 text-muted-foreground"}`}>
            <t.icon className="w-3.5 h-3.5" /> {t.label}
            <span className="opacity-70">{t.count}</span>
          </button>
        ))}
      </div>

      <div className="space-y-2">
        {loading ? (
          <div className="flex justify-center py-4"><Loader2 className="w-4 h-4 animate-spin" /></div>
        ) : domain === "taches" ? (
          <TaskList items={items.taches} nameMap={nameMap} currentUserId={currentUserId} canManage={canManage} onRemove={(id) =>
remove("taches", id)} />
        ) : domain === "budget" ? (
          <BudgetList items={items.budget} canManage={canManage} onRemove={(id) => remove("budget", id)} />
        ) : domain === "evenements" ? (
          <EventList items={items.evenements} canManage={canManage} onRemove={(id) => remove("evenements", id)} />
        ) : (
          <PartnerList items={items.partenariats} canManage={canManage} onRemove={(id) => remove("partenariats", id)} />
        )}
      </div>

      {canManage && (
        <Button size="sm" variant="outline" className="w-full flex items-center gap-1" onClick={() => setShowAdd(true)}>
          <Plus className="w-4 h-4" /> Ajouter
        </Button>
      )}

      {showAdd && (
        <AddDialog
          domain={domain}
          conversation={conversation}
          participants={participants}
          nameMap={nameMap}
          currentUserId={currentUserId}
          onClose={() => setShowAdd(false)}
          onCreated={(d) => { setShowAdd(false); loadDomain(d); }}
        />
      )}
    </div>
  );
}

function Row({ title, sub, meta, canManage, onRemove }) {
  return (
    <div className="flex items-start justify-between gap-2 bg-muted/40 rounded-xl p-2.5">
      <div className="min-w-0">
        <p className="text-sm font-medium truncate">{title}</p>
        {sub && <p className="text-xs text-muted-foreground truncate">{sub}</p>}
        {meta && <p className="text-[11px] text-muted-foreground mt-0.5">{meta}</p>}
      </div>
      {canManage && (
        <Button size="icon" variant="ghost" className="h-7 w-7 text-destructive" onClick={onRemove}><Trash2 className="w-3.5
h-3.5" /></Button>
      )}
    </div>
  );
}

function TaskList({ items, nameMap, currentUserId, canManage, onRemove }) {
  if (!items.length) return <p className="text-xs text-muted-foreground text-center py-3">Aucune tâche partagée.</p>;
  return items.map((t) => (
    <Row key={t.id}
      title={t.title}
```

```
      sub={t.assignee_id ? `Responsable : ${memberName(t.assignee_id, nameMap, currentUserId)}` : "Non assignée"}
      meta={`${t.priority || ""} · ${t.status || ""}`}
      canManage={canManage} onRemove={() => onRemove(t.id)} />
  ));
}

function BudgetList({ items, canManage, onRemove }) {
  if (!items.length) return <p className="text-xs text-muted-foreground text-center py-3">Aucun budget partagé.</p>;
  return items.map((e) => (
    <Row key={e.id}
      title={e.name}
      sub={fmtMoney((e.unit_cost || 0) * (e.quantity || 1))}
      meta={`${e.date || ""}${e.recurring ? " · récurrent" : ""}${e.period ? " · " + e.period : ""}`}
      canManage={canManage} onRemove={() => onRemove(e.id)} />
  ));
}

function EventList({ items, canManage, onRemove }) {
  if (!items.length) return <p className="text-xs text-muted-foreground text-center py-3">Aucun événement partagé.</p>;
  return items.map((ev) => (
    <Row key={ev.id}
      title={ev.title}
      sub={ev.location}
      meta={`${new Date(ev.start).toLocaleString("fr-FR")} · ${ev.category || ""}`}
      canManage={canManage} onRemove={() => onRemove(ev.id)} />
  ));
}

function PartnerList({ items, canManage, onRemove }) {
  if (!items.length) return <p className="text-xs text-muted-foreground text-center py-3">Aucun partenariat partagé.</p>;
  return items.map((p) => (
    <Row key={p.id}
      title={p.partner_name}
      sub={p.contact_name || p.email || p.phone}
      meta={`${p.partner_type || ""} · ${p.status || ""}`}
      canManage={canManage} onRemove={() => onRemove(p.id)} />
  ));
}

function AddDialog({ domain, conversation, participants, nameMap, currentUserId, onClose, onCreated }) {
  const ctxType = conversation.type === "group" ? "group" : "collaboration";
  const ctxId = conversation.id;
  const base = { shared_context_type: ctxType, shared_context_id: ctxId, participants };

  const [form, setForm] = useState(() => initForm(domain));
  const [busy, setBusy] = useState(false);
  const { toast } = useToast();

  const set = (k, v) => setForm((f) => ({ ...f, [k]: v }));

  const submit = async () => {
    const map = { taches: "Task", budget: "Expense", evenements: "Event", partenariats: "Partnership" };
    setBusy(true);
    try {
      const payload = buildPayload(domain, form, base);
      await base44.entities[map[domain]].create(payload);
      onCreated(domain);
    } catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  return (
    <Dialog open onOpenChange={onClose}>
      <DialogContent>
        <DialogHeader><DialogTitle>Nouvel élément partagé</DialogTitle></DialogHeader>
        <div className="space-y-3 max-h-[60vh] overflow-y-auto">
          {domain === "taches" && (
            <>
              <Field label="Titre"><Input value={form.title} onChange={(e) => set("title", e.target.value)} /></Field>
              <div className="grid grid-cols-2 gap-2">
                <Field label="Priorité">
                  <Sel value={form.priority} onChange={(v) => set("priority", v)} opts={[["basse", "Basse"], ["normale",
"Normale"], ["haute", "Haute"]]} />
                </Field>
                <Field label="Statut">
                  <Sel value={form.status} onChange={(v) => set("status", v)} opts={[["a_faire", "À faire"], ["en_cours", "En
cours"], ["terminee", "Terminée"]]} />
                </Field>
              </div>
              <div className="grid grid-cols-2 gap-2">
                <Field label="Début"><Input type="datetime-local" value={form.start} onChange={(e) => set("start",
e.target.value)} /></Field>
                <Field label="Fin"><Input type="datetime-local" value={form.end} onChange={(e) => set("end", e.target.value)}
/></Field>
              </div>
              <Field label="Responsable">
                <Sel value={form.assignee_id} onChange={(v) => set("assignee_id", v)}
                  opts={participants.map((id) => [id, memberName(id, nameMap, currentUserId)])} placeholder="Non assignée" />
              </Field>
            </>
          )}
          {domain === "budget" && (
```

```
            <>
              <Field label="Nom"><Input value={form.name} onChange={(e) => set("name", e.target.value)} /></Field>
              <div className="grid grid-cols-2 gap-2">
                <Field label="Coût unitaire"><Input type="number" value={form.unit_cost} onChange={(e) => set("unit_cost",
e.target.value)} /></Field>
                <Field label="Quantité"><Input type="number" value={form.quantity} onChange={(e) => set("quantity",
e.target.value)} /></Field>
              </div>
              <Field label="Date"><Input type="date" value={form.date} onChange={(e) => set("date", e.target.value)} /></Field>
              <div className="grid grid-cols-2 gap-2">
                <Field label="Période"><Sel value={form.period} onChange={(v) => set("period", v)} opts={PERIODS.map((p) =>
[p.value, p.label])} placeholder="—" /></Field>
                <Field label="Récurrent">
                  <Sel value={form.recurring ? "oui" : "non"} onChange={(v) => set("recurring", v === "oui")} opts={[["non",
"Non"], ["oui", "Oui"]]} />
                </Field>
              </div>
            </>
          )}
          {domain === "evenements" && (
            <>
              <Field label="Titre"><Input value={form.title} onChange={(e) => set("title", e.target.value)} /></Field>
              <div className="grid grid-cols-2 gap-2">
                <Field label="Début"><Input type="datetime-local" value={form.start} onChange={(e) => set("start",
e.target.value)} /></Field>
                <Field label="Fin"><Input type="datetime-local" value={form.end} onChange={(e) => set("end", e.target.value)}
/></Field>
              </div>
              <Field label="Lieu"><Input value={form.location} onChange={(e) => set("location", e.target.value)} /></Field>
              <div className="grid grid-cols-2 gap-2">
                <Field label="Catégorie"><Sel value={form.category} onChange={(v) => set("category", v)} opts={[["reunion",
"Réunion"], ["rendez_vous", "RDV"], ["formation", "Formation"], ["ceremonie", "Cérémonie"], ["autre", "Autre"]]} /></Field>
                <Field label="Priorité"><Sel value={form.priority} onChange={(v) => set("priority", v)} opts={[["basse",
"Basse"], ["normale", "Normale"], ["haute", "Haute"]]} /></Field>
              </div>
            </>
          )}
          {domain === "partenariats" && (
            <>
              <Field label="Nom du partenaire"><Input value={form.partner_name} onChange={(e) => set("partner_name",
e.target.value)} /></Field>
              <div className="grid grid-cols-2 gap-2">
                <Field label="Type"><Sel value={form.partner_type} onChange={(v) => set("partner_type", v)} opts={[["individu",
"Individu"], ["personne_morale", "Personne morale"]]} /></Field>
                <Field label="Statut"><Sel value={form.status} onChange={(v) => set("status", v)} opts={[["prospect",
"Prospect"], ["actif", "Actif"], ["inactif", "Inactif"]]} /></Field>
              </div>
              <Field label="Contact"><Input value={form.contact_name} onChange={(e) => set("contact_name", e.target.value)}
/></Field>
              <Field label="Date de début"><Input type="date" value={form.start_date} onChange={(e) => set("start_date",
e.target.value)} /></Field>
              <Field label="Modalités"><Input value={form.modalities} onChange={(e) => set("modalities", e.target.value)}
/></Field>
            </>
          )}
        </div>
        <DialogFooter>
          <Button variant="outline" onClick={onClose}>Annuler</Button>
          <Button onClick={submit} disabled={busy}>{busy ? <Loader2 className="w-4 h-4 animate-spin" /> : "Ajouter"}</Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  );
}

function Field({ label, children }) {
  return (
    <div className="space-y-1">
      <Label className="text-xs">{label}</Label>
      {children}
    </div>
  );
}

function Sel({ value, onChange, opts, placeholder }) {
  return (
    <Select value={value || ""} onValueChange={onChange}>
      <SelectTrigger className="h-9"><SelectValue placeholder={placeholder || "Sélectionner"} /></SelectTrigger>
      <SelectContent>
        {opts.map(([v, l]) => <SelectItem key={v} value={v}>{l}</SelectItem>)}
      </SelectContent>
    </Select>
  );
}

function initForm(domain) {
  const now = new Date();
  const later = new Date(now.getTime() + 60 * 60 * 1000);
  if (domain === "taches") return { title: "", priority: "normale", status: "a_faire", start: toLocalInput(now), end:
toLocalInput(later), assignee_id: "" };
  if (domain === "budget") return { name: "", unit_cost: "", quantity: 1, date: now.toISOString().slice(0, 10), period: "mois",
recurring: false };
```

```
  if (domain === "evenements") return { title: "", start: toLocalInput(now), end: toLocalInput(later), location: "", category:
"reunion", priority: "normale" };
  return { partner_name: "", partner_type: "individu", status: "prospect", contact_name: "", start_date:
now.toISOString().slice(0, 10), modalities: "" };
}

function buildPayload(domain, form, base) {
  if (domain === "taches") {
    return { ...base, title: form.title, priority: form.priority, status: form.status, start: new
Date(form.start).toISOString(), end: new Date(form.end).toISOString(), assignee_id: form.assignee_id || undefined };
  }
  if (domain === "budget") {
    return { ...base, name: form.name, unit_cost: Number(form.unit_cost) || 0, quantity: Number(form.quantity) || 1, date:
form.date, period: form.period, recurring: !!form.recurring };
  }
  if (domain === "evenements") {
    return { ...base, title: form.title, start: new Date(form.start).toISOString(), end: new Date(form.end).toISOString(),
location: form.location, category: form.category, priority: form.priority };
  }
  return { ...base, scope: base.shared_context_type, partner_name: form.partner_name, partner_type: form.partner_type, status:
form.status, contact_name: form.contact_name, start_date: form.start_date, modalities: form.modalities };
}

------------------------------------------------------------
FICHIER : ./src/components/collab/UserSearch.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { base44 } from "@/api/base44Client";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Search, UserPlus, Loader2 } from "lucide-react";
import { useToast } from "@/components/ui/use-toast";

export default function UserSearch({ currentUserId, excludeIds = [], onSent }) {
  const [q, setQ] = useState("");
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const [sending, setSending] = useState(null);
  const { toast } = useToast();

  const runSearch = async () => {
    const term = q.trim();
    if (term.length < 2) { setResults([]); return; }
    setLoading(true);
    try {
      const list = await base44.entities.Collaborator.list("-created_date", 200);
      const t = term.toLowerCase();
      setResults((list || []).filter((c) => (c.display_name || "").toLowerCase().includes(t)));
    } catch {
      setResults([]);
    } finally {
      setLoading(false);
    }
  };

  const sendRequest = async (c) => {
    setSending(c.id);
    try {
      await base44.entities.CollaborationRequest.create({
        to_user_id: c.created_by_id,
        to_name: c.display_name,
        status: "pending",
      });
      toast({ title: "Demande envoyée", description: c.display_name });
      onSent?.();
    } catch (e) {
      toast({ title: "Erreur", description: e?.message || "Échec de l'envoi", variant: "destructive" });
    } finally {
      setSending(null);
    }
  };

  return (
    <div className="space-y-3">
      <div className="flex items-center gap-2">
        <Input
          value={q}
          onChange={(e) => setQ(e.target.value)}
          onKeyDown={(e) => { if (e.key === "Enter") { e.preventDefault(); runSearch(); } }}
          placeholder="Rechercher un utilisateur par nom…"
        />
        <Button type="button" size="icon" onClick={runSearch} disabled={loading}>
          {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Search className="w-4 h-4" />}
        </Button>
      </div>
      {results.length > 0 && (
        <div className="space-y-2">
          {results.map((c) => {
            const isSelf = c.created_by_id === currentUserId;
            const excluded = excludeIds.includes(c.created_by_id);
            return (
              <div key={c.id} className="flex items-center justify-between gap-2 bg-card border border-border rounded-xl p-2.5">
```

```
                <div className="min-w-0">
                  <p className="text-sm font-medium truncate">{c.display_name}</p>
                </div>
                <Button
                  type="button"
                  size="sm"
                  variant="outline"
                  disabled={isSelf || excluded || sending === c.id}
                  onClick={() => sendRequest(c)}
                  className="flex items-center gap-1"
                >
                  {sending === c.id ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <UserPlus className="w-3.5 h-3.5" />}
                  {isSelf ? "Vous" : excluded ? "Collaborateur" : "Collaboration"}
                </Button>
              </div>
            );
          })}
        </div>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/common/BulkActionBar.jsx
------------------------------------------------------------
import React from "react";
import { Checkbox } from "@/components/ui/checkbox";
import { X } from "lucide-react";
import { Button } from "@/components/ui/button";

export default function BulkActionBar({ count, total, allSelected, onToggleAll, onClear, busy, children }) {
  return (
    <div className="bg-card border border-border rounded-2xl p-3 flex items-center gap-3 flex-wrap shadow-sm sticky top-2 z-10">
      <div className="flex items-center gap-2">
        <Checkbox
          checked={total > 0 && allSelected}
          onCheckedChange={onToggleAll}
          disabled={total === 0}
          aria-label="Tout sélectionner"
        />
        <span className="text-sm font-medium whitespace-nowrap">
          {count > 0 ? `${count} sélectionné${count > 1 ? "s" : ""}` : "Sélectionner"}
          {total > 0 && <span className="text-muted-foreground"> / {total}</span>}
        </span>
      </div>
      <div className="flex items-center gap-2 flex-wrap flex-1">{children}</div>
      <Button variant="ghost" size="sm" onClick={onClear} disabled={busy}>
        <X className="w-4 h-4 mr-1" /> Annuler
      </Button>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/common/PeriodPicker.jsx
------------------------------------------------------------
import React from "react";
import { Label } from "@/components/ui/label";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { PERIOD_OPTIONS, CUSTOM_UNITS } from "@/lib/periods";

// Sélecteur de période unifié. Renvoie { period, customCount, customUnit } via onChange.
export default function PeriodPicker({ period, customCount = "", customUnit = "jour", onChange, label = "Périodicité", id }) {
  return (
    <div className="space-y-2">
      <Label htmlFor={id}>{label}</Label>
      <Select value={period} onValueChange={(v) => onChange({ period: v, customCount, customUnit })}>
        <SelectTrigger id={id}><SelectValue /></SelectTrigger>
        <SelectContent>
          {PERIOD_OPTIONS.map((o) => <SelectItem key={o.value} value={o.value}>{o.label}</SelectItem>)}
        </SelectContent>
      </Select>
      {period === "personnalise" && (
        <div className="grid grid-cols-2 gap-2">
          <Input
            type="number"
            min="1"
            placeholder="Nombre"
            value={customCount}
            onChange={(e) => onChange({ period, customCount: e.target.value, customUnit })}
          />
          <Select value={customUnit} onValueChange={(v) => onChange({ period, customCount, customUnit: v })}>
            <SelectTrigger><SelectValue /></SelectTrigger>
            <SelectContent>
              {CUSTOM_UNITS.map((u) => <SelectItem key={u.value} value={u.value}>{u.label}</SelectItem>)}
            </SelectContent>
          </Select>
        </div>
      )}
    </div>
```

```
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/DashboardWindow.jsx
------------------------------------------------------------
import React from "react";
import { Link } from "react-router-dom";
import { ChevronRight } from "lucide-react";

export default function DashboardWindow({ to, icon: Icon, title, badge, children, hint }) {
  return (
    <div className="bg-card border border-border rounded-2xl p-5 flex flex-col">
      <div className="flex items-center gap-2 mb-2">
        <Link to={to} className="font-display text-lg hover:underline flex items-center gap-2">
          {Icon && <Icon className="w-4 h-4" />} {title}
        </Link>
        {badge != null && (
          <span className="text-xs px-2 py-0.5 rounded-full bg-primary/10 text-primary">{badge}</span>
        )}
      </div>
      {children}
      {hint && <p className="text-sm text-muted-foreground mt-2">{hint}</p>}
      <Link to={to} className="text-sm text-primary hover:underline mt-3 inline-flex items-center gap-1 self-start">
        Voir tout <ChevronRight className="w-3 h-3" />
      </Link>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/EventsWindow.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { CalendarClock } from "lucide-react";
import DashboardWindow from "./DashboardWindow";
import { formatDateFr } from "@/lib/format";
import { useSpace } from "@/lib/SpaceContext";

export default function EventsWindow() {
  const { space, activeCompanyId } = useSpace();
  const [events, setEvents] = useState([]);
  useEffect(() => {
    (async () => {
      try {
        const list = await base44.entities.Event.list("start", 200);
        const scoped = space === "jpm"
          ? (list || [])
          : (list || []).filter((e) => !activeCompanyId || e.company_id === activeCompanyId);
        const now = new Date();
        setEvents(scoped.filter((e) => new Date(e.start) >= now).slice(0, 4));
      } catch { /* ignore */ }
    })();
  }, [space, activeCompanyId]);

  return (
    <DashboardWindow to="/evenements" icon={CalendarClock} title="Événements" badge={events.length || undefined}>
      {events.length === 0 ? (
        <p className="text-sm text-muted-foreground">Aucun événement à venir.</p>
      ) : (
        <ul className="space-y-2">
          {events.map((e) => (
            <li key={e.id} className="flex items-center gap-2 text-sm">
              <CalendarClock className="w-3 h-3 text-accent shrink-0" />
              <span className="flex-1 truncate">{e.title}</span>
              <span className="text-xs text-muted-foreground whitespace-nowrap">{formatDateFr(e.start)}</span>
            </li>
          ))}
        </ul>
      )}
    </DashboardWindow>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/ExpenseChart.jsx
------------------------------------------------------------
import React, { useMemo } from "react";
import { PieChart, Pie, Cell, ResponsiveContainer, Tooltip, Legend } from "recharts";
import { useI18n } from "@/lib/i18n";
import { formatCurrency } from "@/lib/format";

const COLORS = [
  "hsl(var(--chart-1))",
  "hsl(var(--chart-2))",
  "hsl(var(--chart-3))",
  "hsl(var(--chart-4))",
  "hsl(var(--chart-5))",
];

export default function ExpenseChart({ expenses = [], currency = "XAF" }) {
```

```
  const { t } = useI18n();

  const data = useMemo(() => {
    const map = {};
    expenses.forEach((e) => {
      const cat = (e.category || "").trim() || "Autre";
      const val = (Number(e.quantity) || 0) * (Number(e.unit_cost) || 0);
      map[cat] = (map[cat] || 0) + val;
    });
    return Object.entries(map).map(([name, value]) => ({ name, value }));
  }, [expenses]);

  if (!data.length) {
    return (
      <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
        <h2 className="font-display text-lg mb-3">{t("dashboard.expenseChart")}</h2>
        <p className="text-sm text-muted-foreground">{t("dashboard.expenseChartEmpty")}</p>
      </div>
    );
  }

  return (
    <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
      <h2 className="font-display text-lg mb-4">{t("dashboard.expenseChart")}</h2>
      <div className="h-64">
        <ResponsiveContainer width="100%" height="100%">
          <PieChart>
            <Pie
              data={data}
              dataKey="value"
              nameKey="name"
              cx="50%"
              cy="50%"
              outerRadius={80}
              innerRadius={40}
              paddingAngle={2}
            >
              {data.map((_, i) => (
                <Cell key={i} fill={COLORS[i % COLORS.length]} />
              ))}
            </Pie>
            <Tooltip formatter={(v) => formatCurrency(v, currency)} />
            <Legend />
          </PieChart>
        </ResponsiveContainer>
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/KpiCard.jsx
------------------------------------------------------------
import React from "react";

export default function KpiCard({ icon: Icon, label, value, hint, tone = "default" }) {
  const toneClasses = {
    default: "bg-primary/10 text-primary",
    accent: "bg-accent/15 text-accent",
    warn: "bg-destructive/10 text-destructive",
  };

  return (
    <div className="bg-card border border-border rounded-2xl p-5 flex items-start gap-4 shadow-sm">
      <div className={`w-10 h-10 rounded-xl flex items-center justify-center shrink-0 ${toneClasses[tone]}`}>
        <Icon className="w-5 h-5" />
      </div>
      <div className="min-w-0">
        <p className="text-sm text-muted-foreground">{label}</p>
        <p className="font-display text-xl mt-0.5 truncate">{value}</p>
        {hint && <p className="text-sm text-muted-foreground mt-1">{hint}</p>}
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/LifeTipCard.jsx
------------------------------------------------------------
import React, { useMemo } from "react";
import { Heart } from "lucide-react";
import { getDailyLifeAdvice } from "@/lib/lifeAdvice";

export default function LifeTipCard({ space = "jpm" }) {
  const advice = useMemo(() => getDailyLifeAdvice(space), [space]);
  return (
    <div className="bg-card border border-border rounded-2xl p-5 sm:p-6 flex items-start gap-3">
      <Heart className="w-5 h-5 text-rose-500 shrink-0 mt-0.5" />
      <div>
        <p className="text-sm font-medium">Conseil de vie</p>
        <p className="text-sm text-muted-foreground mt-1">{advice}</p>
      </div>
```

```
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/ModeSwitcher.jsx
------------------------------------------------------------
import React from "react";
import { User, Building2 } from "lucide-react";
import { useSpace } from "@/lib/SpaceContext";
import { useAuth } from "@/lib/AuthContext";
import { cn } from "@/lib/utils";

// Admin-only preview toggle between JPM (Particulier) and JBM (Entreprise).
// Normal users keep the space fixed by their onboarding account type.
export default function ModeSwitcher() {
  const { user } = useAuth();
  const { space, setSpace } = useSpace();

  if (user?.role !== "admin") return null;

  const Btn = ({ value, icon: Icon, label, activeClass }) => (
    <button
      onClick={() => setSpace(value)}
      aria-pressed={space === value}
      className={cn(
        "no-select-tap min-touch flex items-center gap-2 px-3 py-1.5 rounded-lg text-sm font-medium transition-colors",
        space === value
          ? activeClass
          : "text-muted-foreground hover:text-foreground"
      )}
    >
      <Icon className="w-4 h-4" /> {label}
    </button>
  );

  return (
    <div className="inline-flex p-1 rounded-xl bg-secondary border border-border">
      <Btn value="jpm" icon={User} label="JPM" activeClass="bg-primary text-primary-foreground shadow" />
      <Btn value="jbm" icon={Building2} label="JBM" activeClass="bg-accent text-accent-foreground shadow" />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/QuickLinks.jsx
------------------------------------------------------------
import React from "react";
import { Link } from "react-router-dom";

export default function QuickLinks({ items }) {
  return (
    <div className="grid grid-cols-2 sm:grid-cols-3 gap-3">
      {items.map(({ to, label, icon: Icon, desc }) => (
        <Link
          key={to}
          to={to}
          className="bg-card border border-border rounded-2xl p-4 flex flex-col gap-2 hover:border-primary/40 transition-colors"
        >
          <Icon className="w-5 h-5 text-primary" />
          <span className="text-sm font-medium">{label}</span>
          {desc && <span className="text-xs text-muted-foreground">{desc}</span>}
        </Link>
      ))}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/RevenueCard.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Banknote } from "lucide-react";
import { base44 } from "@/api/base44Client";
import { formatCurrency } from "@/lib/format";

const PERIODS = [
  { key: "7d", label: "7 jours", days: 7 },
  { key: "30d", label: "30 jours", days: 30 },
  { key: "12m", label: "12 mois", days: 365 },
  { key: "all", label: "Tout", days: null },
];

export default function RevenueCard() {
  const [subs, setSubs] = useState([]);
  const [period, setPeriod] = useState("30d");
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    (async () => {
      try {
        setSubs(await base44.entities.Subscription.list("-paid_at", 1000));
```

```
      } catch {
        setSubs([]);
      }
      setLoading(false);
    })();
  }, []);

  const cfg = PERIODS.find((p) => p.key === period);
  const now = new Date();
  const since = cfg.days ? new Date(now.getTime() - cfg.days * 86400000) : null;
  const filtered = subs.filter(
    (s) => s.status === "payee" && (!since || new Date(s.paid_at || s.created_date) >= since)
  );
  const total = filtered.reduce((sum, s) => sum + (Number(s.amount) || 0), 0);
  const particulier = filtered
    .filter((s) => s.account_type === "particulier")
    .reduce((a, s) => a + (Number(s.amount) || 0), 0);
  const entreprise = filtered
    .filter((s) => s.account_type === "entreprise")
    .reduce((a, s) => a + (Number(s.amount) || 0), 0);

  return (
    <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
      <div className="flex items-center justify-between gap-3 mb-4 flex-wrap">
        <div className="flex items-center gap-2">
          <Banknote className="w-5 h-5 text-primary" />
          <h2 className="font-display text-lg">Chiffre d'affaires</h2>
          <span className="text-xs px-2 py-0.5 rounded-full bg-primary/10 text-primary">Admin</span>
        </div>
        <div className="flex gap-1 bg-muted rounded-lg p-1">
          {PERIODS.map((p) => (
            <button
              key={p.key}
              onClick={() => setPeriod(p.key)}
              className={`no-select-tap px-2.5 py-1 text-xs rounded-md transition-colors ${
                period === p.key ? "bg-card text-foreground shadow-sm" : "text-muted-foreground"
              }`}
            >
              {p.label}
            </button>
          ))}
        </div>
      </div>

      {loading ? (
        <div className="h-8 w-40 bg-muted animate-pulse rounded" />
      ) : (
        <>
          <p className="font-display text-3xl">{formatCurrency(total, "XAF")}</p>
          <p className="text-xs text-muted-foreground mt-1">
            {filtered.length} abonnement(s) sur la période.
          </p>
          <div className="grid grid-cols-2 gap-3 mt-4">
            <div className="rounded-xl bg-primary/5 p-3">
              <p className="text-xs text-muted-foreground">Particulier (JPM)</p>
              <p className="font-display text-lg">{formatCurrency(particulier, "XAF")}</p>
            </div>
            <div className="rounded-xl bg-accent/10 p-3">
              <p className="text-xs text-muted-foreground">Entreprise (JBM)</p>
              <p className="font-display text-lg">{formatCurrency(entreprise, "XAF")}</p>
            </div>
          </div>
        </>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/SalesChart.jsx
------------------------------------------------------------
import React, { useMemo } from "react";
import { BarChart, Bar, XAxis, YAxis, ResponsiveContainer, Tooltip, CartesianGrid } from "recharts";
import { format, parseISO } from "date-fns";
import { useI18n } from "@/lib/i18n";
import { formatCurrency } from "@/lib/format";

export default function SalesChart({ sales = [], currency = "XAF" }) {
  const { t } = useI18n();

  const data = useMemo(() => {
    const map = {};
    const now = new Date();
    for (let i = 5; i >= 0; i--) {
      const d = new Date(now.getFullYear(), now.getMonth() - i, 1);
      map[format(d, "yyyy-MM")] = 0;
    }
    sales.forEach((s) => {
      if (s.status === "annulee") return;
      const d = s.date ? parseISO(s.date) : null;
      if (!d) return;
      const key = format(d, "yyyy-MM");
```

```
      if (key in map) {
        map[key] += (Number(s.quantity) || 0) * (Number(s.unit_price) || 0);
      }
    });
    return Object.entries(map).map(([key, value]) => ({
      month: format(new Date(key + "-01"), "MMM"),
      value,
    }));
  }, [sales]);

  const hasData = data.some((d) => d.value > 0);

  if (!hasData) {
    return (
      <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
        <h2 className="font-display text-lg mb-3">{t("jbm.dashboard.salesChart")}</h2>
        <p className="text-sm text-muted-foreground">{t("jbm.dashboard.salesChartEmpty")}</p>
      </div>
    );
  }

  return (
    <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
      <h2 className="font-display text-lg mb-4">{t("jbm.dashboard.salesChart")}</h2>
      <div className="h-64">
        <ResponsiveContainer width="100%" height="100%">
          <BarChart data={data}>
            <CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" />
            <XAxis dataKey="month" tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" />
            <YAxis tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" />
            <Tooltip formatter={(v) => formatCurrency(v, currency)} />
            <Bar dataKey="value" fill="hsl(var(--chart-2))" radius={[6, 6, 0, 0]} />
          </BarChart>
        </ResponsiveContainer>
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/SpaceNav.jsx
------------------------------------------------------------
import React from "react";
import { useNavigate } from "react-router-dom";
import { User, Building2 } from "lucide-react";
import { useAuth } from "@/lib/AuthContext";
import { useSpace } from "@/lib/SpaceContext";
import { useLocation } from "react-router-dom";
import { cn } from "@/lib/utils";

// Bascule fluide JPM/JBM : un clic navigue directement vers l'espace choisi
// (et positionne l'espace actif). Réservé à l'admin, qui voit les deux.
export default function SpaceNav() {
  const { user } = useAuth();
  const { space, setSpace } = useSpace();
  const navigate = useNavigate();
  const location = useLocation();
  if (user?.role !== "admin") return null;

  const go = (next) => {
    setSpace(next);
    navigate(next === "jbm" ? "/jbm" : "/");
  };
  const here = location.pathname;

  const Btn = ({ value, icon: Icon, label }) => {
    const active = space === value;
    return (
      <button
        onClick={() => go(value)}
        aria-pressed={active}
        className={cn(
          "no-select-tap min-touch flex items-center gap-2 px-3.5 py-2 rounded-xl text-sm font-medium transition-colors",
          active ? "bg-primary text-primary-foreground shadow" : "bg-secondary text-muted-foreground hover:text-foreground"
        )}
      >
        <Icon className="w-4 h-4" /> {label}
      </button>
    );
  };

  return (
    <div className="inline-flex p-1 rounded-2xl bg-secondary/60 border border-border gap-1">
      <Btn value="jpm" icon={User} label="Espace personnel" />
      <Btn value="jbm" icon={Building2} label="Espace entreprise" />
      {here !== "/" && here !== "/jbm" && (
        <span className="hidden sm:inline text-xs text-muted-foreground self-center pl-1">Aller à l'espace souhaité</span>
      )}
    </div>
  );
}
```

```
------------------------------------------------------------
FICHIER : ./src/components/dashboard/TasksWindow.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { ListChecks } from "lucide-react";
import DashboardWindow from "./DashboardWindow";
import { formatDateFr } from "@/lib/format";

export default function TasksWindow() {
  const [tasks, setTasks] = useState([]);
  useEffect(() => {
    (async () => {
      try {
        const list = await base44.entities.Task.list("start", 200);
        const now = new Date();
        setTasks(
          (list || [])
            .filter((t) => t.status !== "terminee" && t.start && new Date(t.start) >= now)
            .slice(0, 4)
        );
      } catch { /* ignore */ }
    })();
  }, []);

  return (
    <DashboardWindow to="/taches" icon={ListChecks} title="Tâches" badge={tasks.length || undefined}>
      {tasks.length === 0 ? (
        <p className="text-sm text-muted-foreground">Aucune tâche à venir.</p>
      ) : (
        <ul className="space-y-2">
          {tasks.map((t) => (
            <li key={t.id} className="flex items-center gap-2 text-sm">
              <span className={`w-2 h-2 rounded-full shrink-0 ${t.priority === "haute" ? "bg-destructive" : t.priority ===
"normale" ? "bg-accent" : "bg-muted-foreground/40"}`} />
              <span className="flex-1 truncate">{t.title}</span>
              <span className="text-xs text-muted-foreground whitespace-nowrap">{formatDateFr(t.start)}</span>
            </li>
          ))}
        </ul>
      )}
    </DashboardWindow>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/dashboard/TipCard.jsx
------------------------------------------------------------
import React from "react";
import { Sparkles } from "lucide-react";

export default function TipCard({ title, children }) {
  return (
    <div className="bg-card border border-border rounded-2xl p-5 sm:p-6 flex items-start gap-3">
      <Sparkles className="w-5 h-5 text-accent shrink-0 mt-0.5" />
      <div>
        <p className="text-sm font-medium">{title}</p>
        <p className="text-sm text-muted-foreground mt-1">{children}</p>
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/demo/DashboardPreview.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { motion } from "framer-motion";
import {
  Wallet, TrendingDown, CheckCircle2, AlertTriangle, ListChecks, CalendarRange,
  CalendarDays, Bot, Calculator, Bell, Tag, UsersRound, Gift, Boxes,
  ShoppingCart, FileText, Handshake, BarChart3, Building2, User, ArrowRight,
} from "lucide-react";
import KpiCard from "@/components/dashboard/KpiCard";
import QuickLinks from "@/components/dashboard/QuickLinks";

const JPM_PREVIEW = {
  badge: "JPM",
  badgeIcon: User,
  tone: "primary",
  title: "Espace personnel (JPM)",
     s u b t i t l e :   " B i e n v e n u e  Ø=ÜK   V o i c i   u n   a p e r ç u   d e   v o t r e   t a b l e a u   d e   b o r d
  kpis: [
    { icon: Wallet, label: "Budget restant", value: "285 000 XAF", tone: "default" },
    { icon: TrendingDown, label: "Dépenses du mois", value: "215 000 XAF", tone: "accent" },
    { icon: CheckCircle2, label: "Tâches du jour", value: 3, tone: "default" },
    { icon: AlertTriangle, label: "En retard", value: 1, tone: "warn" },
  ],
  upcoming: [
    { title: "Réunion banque", when: "Demain · 09:00", priority: "Haute" },
    { title: "Course marché", when: "Sam. · 08:00", priority: "Normale" },
```

```
    { title: "Rendez-vous médecin", when: "Lun. · 14:30", priority: "Normale" },
  ],
  links: [
    { to: "#", label: "Finances", icon: Wallet, desc: "Budget, dépenses, taxes & emprunts" },
    { to: "#", label: "Tâches", icon: ListChecks, desc: "Vos tâches" },
    { to: "#", label: "Planning", icon: CalendarRange, desc: "Calendrier" },
    { to: "#", label: "Événements", icon: CalendarDays, desc: "RDV & événements" },
    { to: "#", label: "Calculatrice", icon: Calculator, desc: "Calculatrice & formules" },
    { to: "#", label: "Assistant IA JPM", icon: Bot, desc: "Aide intelligente" },
    { to: "#", label: "Catégories", icon: Tag, desc: "Organisez vos catégories" },
    { to: "#", label: "Collaborateurs", icon: UsersRound, desc: "Échangez ensemble" },
    { to: "#", label: "Suggestions", icon: Gift, desc: "Proposez des idées" },
    { to: "#", label: "Affiliation", icon: Gift, desc: "Parrainez et gagnez" },
  ],
};

const JBM_PREVIEW = {
  badge: "JBM",
  badgeIcon: Building2,
  tone: "accent",
  title: "Espace entreprise (JBM)",
  subtitle: "Pilotez votre activité comme un pro — multi-entreprises.",
  kpis: [
    { icon: TrendingDown, label: "Ventes du mois", value: "1 250 000 XAF", tone: "default" },
    { icon: Wallet, label: "Stock valorisé", value: "640 000 XAF", tone: "accent" },
    { icon: FileText, label: "Factures impayées", value: 4, tone: "warn" },
    { icon: AlertTriangle, label: "Alertes stock bas", value: 2, tone: "warn" },
  ],
  upcoming: [
    { title: "Livraison Fournisseur X", when: "Auj. · 16:00", priority: "Haute" },
    { title: "Réunion équipe ventes", when: "Jeu. · 10:00", priority: "Normale" },
    { title: "Échéance TVA", when: "15 oct.", priority: "Haute" },
  ],
  links: [
    { to: "#", label: "Stock", icon: Boxes, desc: "Inventaire & produits" },
    { to: "#", label: "Ventes", icon: ShoppingCart, desc: "Ventes & clients" },
    { to: "#", label: "Factures", icon: FileText, desc: "Factures & devis" },
    { to: "#", label: "Partenariats", icon: Handshake, desc: "Vos partenaires" },
    { to: "#", label: "Reporting", icon: BarChart3, desc: "Rapports & graphiques" },
    { to: "#", label: "Assistant IA JBM", icon: Bot, desc: "Aide métier" },
    { to: "#", label: "Personnel", icon: UsersRound, desc: "Équipe & salaires" },
    { to: "#", label: "Taxes", icon: TrendingDown, desc: "Taxes & charges" },
  ],
};

export default function DashboardPreview() {
  const [space, setSpace] = useState("jpm");
  const data = space === "jbm" ? JBM_PREVIEW : JPM_PREVIEW;
  const BadgeIcon = data.badgeIcon;

  return (
    <div className="rounded-3xl border border-border bg-card p-5 sm:p-7">
      {/* Bascule JPM / JBM */}
      <div className="flex items-center justify-between gap-3 flex-wrap mb-5">
        <div className="inline-flex items-center gap-2">
          <span className={`inline-flex items-center gap-1.5 px-2.5 py-1 rounded-full text-xs font-medium ${data.tone ===
"accent" ? "bg-accent/15 text-accent-foreground" : "bg-primary/10 text-primary"}`}>
            <BadgeIcon className="w-3.5 h-3.5" /> {data.badge}
          </span>
          <span className="text-sm text-muted-foreground">Abonnement simulé · validé automatiquement</span>
        </div>
        <div className="inline-flex rounded-xl border border-border overflow-hidden">
          <button
            onClick={() => setSpace("jpm")}
            className={`no-select-tap min-touch px-4 py-2 text-sm font-medium transition-colors ${space === "jpm" ? "bg-primary
text-primary-foreground" : "bg-transparent hover:bg-accent/10"}`}
          >
            Particulier
          </button>
          <button
            onClick={() => setSpace("jbm")}
            className={`no-select-tap min-touch px-4 py-2 text-sm font-medium transition-colors ${space === "jbm" ? "bg-accent
text-accent-foreground" : "bg-transparent hover:bg-accent/10"}`}
          >
            Entreprise
          </button>
        </div>
      </div>

      <div className="mb-5">
        <h3 className="font-display text-xl sm:text-2xl">{data.title}</h3>
        <p className="text-muted-foreground mt-1 text-sm">{data.subtitle}</p>
      </div>

      {/* Bandeau abonnement actif */}
      <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }}
        className="mb-5 rounded-2xl bg-gradient-to-r from-primary/10 to-accent/10 border border-primary/20 p-4 flex items-center
gap-3">
        <div className="w-10 h-10 rounded-xl bg-primary text-primary-foreground flex items-center justify-center shrink-0">
          <CheckCircle2 className="w-5 h-5" />
        </div>
        <div className="flex-1">
```

```
                  < p   c l a s s N a m e = " f o n t - m e d i u m   t e x t - s m " > A b o n n e m e n t   { s p a c e   = = =
          <p className="text-xs text-muted-foreground">Mode démo · aucune transaction réelle · valide par défaut</p>
        </div>
      </motion.div>

      <div className="grid grid-cols-2 md:grid-cols-4 gap-4 mb-6">
        {data.kpis.map((k) => (
          <KpiCard key={k.label} icon={k.icon} label={k.label} value={k.value} tone={k.tone} />
        ))}
      </div>

      <div className="mb-6">
        <h4 className="font-display text-lg mb-3">Accès rapide</h4>
        <QuickLinks items={data.links} />
      </div>

      <div className="bg-background border border-border rounded-2xl p-5">
        <div className="flex items-center justify-between mb-3">
          <h4 className="font-display text-lg">À venir</h4>
          <span className="text-sm text-muted-foreground inline-flex items-center gap-1">
            Voir le planning <ArrowRight className="w-3.5 h-3.5" />
          </span>
        </div>
        <div className="space-y-3">
          {data.upcoming.map((t) => (
            <div key={t.title} className="flex items-center justify-between gap-3">
              <div className="min-w-0">
                <p className="text-sm font-medium truncate">{t.title}</p>
                <p className="text-xs text-muted-foreground">{t.when}</p>
              </div>
              <span className="text-xs px-2 py-1 rounded-full bg-secondary text-secondary-foreground
shrink-0">{t.priority}</span>
            </div>
          ))}
        </div>
      </div>

      <p className="mt-5 text-xs text-muted-foreground text-center">
        Aperçu statique illustratif. Les liens ci-dessus ne naviguent pas (mode démo).
      </p>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/events/EventForm.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Checkbox } from "@/components/ui/checkbox";

function toLocalInput(value) {
  if (!value) return "";
  const d = new Date(value);
  if (isNaN(d.getTime())) return "";
  const pad = (n) => String(n).padStart(2, "0");
  return `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}T${pad(d.getHours())}:${pad(d.getMinutes())}`;
}

const CATEGORIES = [
  { value: "reunion", label: "Réunion" },
  { value: "rendez_vous", label: "Rendez-vous" },
  { value: "formation", label: "Formation" },
  { value: "ceremonie", label: "Cérémonie" },
  { value: "autre", label: "Autre" },
];

const PRIORITIES = [
  { value: "basse", label: "Basse" },
  { value: "normale", label: "Normale" },
  { value: "haute", label: "Haute" },
];

export default function EventForm({ open, onOpenChange, event, onSubmit }) {
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");
  const [start, setStart] = useState("");
  const [end, setEnd] = useState("");
  const [location, setLocation] = useState("");
  const [category, setCategory] = useState("autre");
  const [priority, setPriority] = useState("normale");
  const [repeat, setRepeat] = useState(false);
  const [frequency, setFrequency] = useState("mois");
  const [repeatCount, setRepeatCount] = useState("1");
  const [error, setError] = useState("");

  useEffect(() => {
```

```
    if (open) {
      setTitle(event?.title || "");
      setDescription(event?.description || "");
      setStart(toLocalInput(event?.start) || toLocalInput(new Date().toISOString()));
      setEnd(toLocalInput(event?.end) || toLocalInput(new Date(Date.now() + 60 * 60 * 1000).toISOString()));
      setLocation(event?.location || "");
      setCategory(event?.category || "autre");
      setPriority(event?.priority || "normale");
      setRepeat(false);
      setFrequency("mois");
      setRepeatCount("1");
      setError("");
    }
  }, [open, event]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title.trim()) {
      setError("Le nom de l'événement est obligatoire.");
      return;
    }
    const startDate = new Date(start);
    const endDate = new Date(end);
    if (isNaN(startDate.getTime()) || isNaN(endDate.getTime())) {
      setError("Dates invalides.");
      return;
    }
    if (endDate < startDate) {
      setError("La date de fin doit être après la date de début.");
      return;
    }
    onSubmit({
      title: title.trim(),
      description: description.trim(),
      start: startDate.toISOString(),
      end: endDate.toISOString(),
      location: location.trim(),
      category,
      priority,
    }, repeat && !event && parseInt(repeatCount, 10) > 1 ? { frequency, count: parseInt(repeatCount, 10) } : null);
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display">{event ? "Modifier l'événement" : "Nouvel événement"}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          {error && <p className="text-sm text-destructive">{error}</p>}
          <div className="space-y-2">
            <Label htmlFor="ev-title">Nom de l'événement</Label>
            <Input id="ev-title" value={title} onChange={(e) => setTitle(e.target.value)} placeholder="Réunion équipe,
rendez-vous client…" autoFocus />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label htmlFor="ev-start">Début</Label>
              <Input id="ev-start" type="datetime-local" value={start} onChange={(e) => setStart(e.target.value)} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="ev-end">Fin</Label>
              <Input id="ev-end" type="datetime-local" value={end} onChange={(e) => setEnd(e.target.value)} />
            </div>
          </div>
          <div className="space-y-2">
            <Label htmlFor="ev-location">Lieu</Label>
            <Input id="ev-location" value={location} onChange={(e) => setLocation(e.target.value)} placeholder="Bureau, salle,
adresse…" />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>Catégorie</Label>
              <Select value={category} onValueChange={setCategory}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  {CATEGORIES.map((c) => <SelectItem key={c.value} value={c.value}>{c.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
            <div className="space-y-2">
              <Label>Priorité</Label>
              <Select value={priority} onValueChange={setPriority}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  {PRIORITIES.map((p) => <SelectItem key={p.value} value={p.value}>{p.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          </div>
          <div className="space-y-2">
            <Label htmlFor="ev-desc">Description</Label>
```

```
            <Textarea id="ev-desc" value={description} onChange={(e) => setDescription(e.target.value)} rows={3}
placeholder="Détails, ordre du jour…" />
          </div>
          {!event && (
            <div className="space-y-3 rounded-xl border border-border p-3">
              <div className="flex items-center gap-2">
                <Checkbox checked={repeat} onCheckedChange={setRepeat} id="ev-repeat" />
                <Label htmlFor="ev-repeat">Répéter périodiquement</Label>
              </div>
              {repeat && (
                <div className="grid grid-cols-2 gap-3">
                  <div className="space-y-1.5">
                    <Label htmlFor="ev-freq" className="text-xs">Fréquence</Label>
                    <Select value={frequency} onValueChange={setFrequency}>
                      <SelectTrigger id="ev-freq"><SelectValue /></SelectTrigger>
                      <SelectContent>
                        <SelectItem value="jour">Jour</SelectItem>
                        <SelectItem value="semaine">Semaine</SelectItem>
                        <SelectItem value="trimestre">Trimestre</SelectItem>
                        <SelectItem value="mois">Mois</SelectItem>
                        <SelectItem value="annee">Année</SelectItem>
                      </SelectContent>
                    </Select>
                  </div>
                  <div className="space-y-1.5">
                    <Label htmlFor="ev-count" className="text-xs">Nombre d'occurrences</Label>
                    <Input id="ev-count" type="number" min="1" max="120" value={repeatCount} onChange={(e) =>
setRepeatCount(e.target.value)} />
                  </div>
                </div>
              )}
            </div>
          )}
          <DialogFooter>
            <Button type="submit" className="w-full">{event ? "Enregistrer" : "Ajouter l'événement"}</Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/events/EventItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Pencil, Trash2, MapPin } from "lucide-react";
import { formatDateFr } from "@/lib/format";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
  AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

const CATEGORY_BADGE = {
  reunion: "bg-primary/10 text-primary",
  rendez_vous: "bg-accent/20 text-accent-foreground",
  formation: "bg-chart-3/15 text-chart-3",
  ceremonie: "bg-sidebar-primary/20 text-sidebar-primary",
  autre: "bg-secondary text-secondary-foreground",
};

const CATEGORY_LABEL = {
  reunion: "Réunion",
  rendez_vous: "Rendez-vous",
  formation: "Formation",
  ceremonie: "Cérémonie",
  autre: "Autre",
};

const PRIORITY_DOT = {
  basse: "bg-muted-foreground/40",
  normale: "bg-accent",
  haute: "bg-destructive",
};

export default function EventItem({ event, onEdit, onDelete, conflict }) {
  const [confirmOpen, setConfirmOpen] = useState(false);
  return (
    <>
      <div className="flex items-center gap-3 p-4 rounded-2xl border border-border bg-card">
        <span className={`w-1.5 h-10 rounded-full shrink-0 ${PRIORITY_DOT[event.priority] || PRIORITY_DOT.normale}`}
title={event.priority || "normale"} />
        <div className="min-w-0 flex-1">
          <div className="flex items-center gap-2 flex-wrap">
            <p className="text-sm font-medium truncate">{event.title}</p>
            <span className={`text-[11px] px-2 py-0.5 rounded-full ${CATEGORY_BADGE[event.category] || CATEGORY_BADGE.autre}`}>
              {CATEGORY_LABEL[event.category] || "Autre"}
            </span>
          </div>
          <p className="text-sm text-muted-foreground mt-0.5">
                        {formatDateFr(event.start, true)} !' {formatDateFr(event
```

```
                    {conflict && <span className="text-destructive ml-2">&
          </p>
          {event.location && (
            <p className="text-xs text-muted-foreground mt-0.5 flex items-center gap-1 truncate">
              <MapPin className="w-3 h-3 shrink-0" /> {event.location}
            </p>
          )}
        </div>
        <div className="flex items-center gap-1 shrink-0">
          <button
            onClick={() => onEdit(event)}
            aria-label="Modifier"
            className="no-select-tap min-touch p-2.5 rounded-lg hover:bg-secondary text-muted-foreground hover:text-foreground
transition-colors"
          >
            <Pencil className="w-4 h-4" />
          </button>
          <button
            onClick={() => setConfirmOpen(true)}
            aria-label="Supprimer"
            className="no-select-tap min-touch p-2.5 rounded-lg hover:bg-secondary text-muted-foreground hover:text-destructive
transition-colors"
          >
            <Trash2 className="w-4 h-4" />
          </button>
        </div>
      </div>

      <AlertDialog open={confirmOpen} onOpenChange={setConfirmOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Supprimer cet événement ?</AlertDialogTitle>
            <AlertDialogDescription>
              Cette action est définitive. L'événement « {event.title} » sera supprimé pour toujours.
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Annuler</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
              onClick={() => onDelete(event)}
            >
              Supprimer
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/finances/ExpenseForm.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Checkbox } from "@/components/ui/checkbox";
import CategorySelect from "@/components/categories/CategorySelect";
import PeriodPicker from "@/components/common/PeriodPicker";
import { useSpace } from "@/lib/SpaceContext";
import { useI18n } from "@/lib/i18n";

function todayInput() {
  return new Date().toISOString().slice(0, 10);
}

export default function ExpenseForm({ open, onOpenChange, expense, onSubmit }) {
  const { t } = useI18n();
  const { space } = useSpace();
  const [name, setName] = useState("");
  const [quantity, setQuantity] = useState("1");
  const [unitCost, setUnitCost] = useState("");
  const [category, setCategory] = useState("");
  const [date, setDate] = useState(todayInput());
  const [repeat, setRepeat] = useState(false);
  const [frequency, setFrequency] = useState("mois");
  const [repeatCount, setRepeatCount] = useState("1");
  const [customCount, setCustomCount] = useState("");
  const [customUnit, setCustomUnit] = useState("jour");
  const [error, setError] = useState("");

  useEffect(() => {
    if (open) {
      setName(expense?.name || "");
      setQuantity(String(expense?.quantity ?? 1));
      setUnitCost(String(expense?.unit_cost ?? ""));
      setCategory(expense?.category || "");
      setDate(expense?.date || todayInput());
      setRepeat(false);
```

```
      setFrequency("mois");
      setRepeatCount("1");
      setCustomCount("");
      setCustomUnit("jour");
      setError("");
    }
  }, [open, expense]);

  const handleSubmit = (e) => {
    e.preventDefault();
    const qty = parseFloat(quantity);
    const cost = parseFloat(unitCost);
    if (!name.trim()) {
      setError(t("expform.err.name"));
      return;
    }
    if (isNaN(qty) || qty <= 0) {
      setError(t("expform.err.qty"));
      return;
    }
    if (isNaN(cost) || cost < 0) {
      setError(t("expform.err.cost"));
      return;
    }
    onSubmit({
      name: name.trim(),
      quantity: qty,
      unit_cost: cost,
      category: category.trim(),
      date,
    }, repeat && !expense && parseInt(repeatCount, 10) > 1 ? { frequency, count: parseInt(repeatCount, 10), custom: frequency
=== "personnalise" ? { count: parseInt(customCount, 10) || 0, unit: customUnit } : null } : null);
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display">{expense ? t("expform.edit") : t("expform.new")}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          {error && <p className="text-sm text-destructive">{error}</p>}
          <div className="space-y-2">
            <Label htmlFor="name">{t("expform.name")}</Label>
            <Input id="name" value={name} onChange={(e) => setName(e.target.value)} placeholder={t("expform.namePlaceholder")}
autoFocus />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label htmlFor="quantity">{t("expform.quantity")}</Label>
              <Input id="quantity" type="number" min="0" step="any" value={quantity} onChange={(e) =>
setQuantity(e.target.value)} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="unitCost">{t("expform.unitCost")}</Label>
              <Input id="unitCost" type="number" min="0" step="any" value={unitCost} onChange={(e) =>
setUnitCost(e.target.value)} />
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label htmlFor="category">{t("expform.category")}</Label>
              <CategorySelect scope={space} domain="depense" value={category} onChange={setCategory}
placeholder={t("expform.categoryPlaceholder")} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="date">{t("expform.date")}</Label>
              <Input id="date" type="date" value={date} onChange={(e) => setDate(e.target.value)} />
            </div>
          </div>
          {!expense && (
            <div className="space-y-3 rounded-xl border border-border p-3">
              <div className="flex items-center gap-2">
                <Checkbox checked={repeat} onCheckedChange={setRepeat} id="exp-repeat" />
                <Label htmlFor="exp-repeat">Répéter périodiquement</Label>
              </div>
              {repeat && (
                <div className="space-y-3">
                  <PeriodPicker
                    period={frequency}
                    customCount={customCount}
                    customUnit={customUnit}
                    onChange={({ period, customCount: cc, customUnit: cu }) => { setFrequency(period); setCustomCount(cc);
setCustomUnit(cu); }}
                    label="Fréquence"
                    id="exp-freq"
                  />
                  <div className="space-y-1.5">
                    <Label htmlFor="exp-count" className="text-xs">Nombre d'échéances</Label>
                    <Input id="exp-count" type="number" min="1" max="120" value={repeatCount} onChange={(e) =>
setRepeatCount(e.target.value)} />
                  </div>
```

```
              </div>
            )}
          </div>
        )}
        <DialogFooter>
          <Button type="submit" className="w-full">{expense ? t("common.save") : t("expform.add")}</Button>
        </DialogFooter>
      </form>
    </DialogContent>
  </Dialog>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/finances/ExpenseItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Pencil, Trash2 } from "lucide-react";
import { Checkbox } from "@/components/ui/checkbox";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import {
 AlertDialog, AlertDialogAction, AlertDialogCancel,
 AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
 AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

export default function ExpenseItem({ expense, currency, onEdit, onDelete, selectable, selected, onToggleSelect }) {
 const { t } = useI18n();
 const total = (Number(expense.quantity) || 0) * (Number(expense.unit_cost) || 0);
 const [confirmOpen, setConfirmOpen] = useState(false);

 return (
   <>
     <div className="flex items-center gap-3 p-4 rounded-2xl border border-border bg-card">
       {selectable && (
         <Checkbox
           checked={!!selected}
           onCheckedChange={() => onToggleSelect(expense.id)}
           aria-label="Sélectionner"
           className="shrink-0"
         />
       )}
       <div className="min-w-0 flex-1">
         <p className="text-sm font-medium truncate">{expense.name}</p>
         <p className="text-sm text-muted-foreground mt-0.5">
           {expense.category || t("common.other")} · {formatDateFr(expense.date)} · {expense.quantity} ×
{formatCurrency(expense.unit_cost, currency)}
         </p>
       </div>
       <p className="font-display text-sm shrink-0">{formatCurrency(total, currency)}</p>
       <div className="flex items-center gap-1 shrink-0">
         <button
           onClick={() => onEdit(expense)}
           aria-label={t("expitem.editAria")}
           className="no-select-tap min-touch p-2.5 rounded-lg hover:bg-secondary text-muted-foreground hover:text-foreground
transition-colors"
         >
           <Pencil className="w-4 h-4" />
         </button>
         <button
           onClick={() => setConfirmOpen(true)}
           aria-label={t("expitem.deleteAria")}
           className="no-select-tap min-touch p-2.5 rounded-lg hover:bg-secondary text-muted-foreground hover:text-destructive
transition-colors"
         >
           <Trash2 className="w-4 h-4" />
         </button>
       </div>
     </div>

     <AlertDialog open={confirmOpen} onOpenChange={setConfirmOpen}>
       <AlertDialogContent>
         <AlertDialogHeader>
           <AlertDialogTitle>{t("expitem.deleteTitle")}</AlertDialogTitle>
           <AlertDialogDescription>{t("expitem.deleteDesc", { name: expense.name })}</AlertDialogDescription>
         </AlertDialogHeader>
         <AlertDialogFooter>
           <AlertDialogCancel>{t("common.cancel")}</AlertDialogCancel>
           <AlertDialogAction
             className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
             onClick={() => onDelete(expense)}
           >
             {t("common.delete")}
           </AlertDialogAction>
         </AlertDialogFooter>
       </AlertDialogContent>
     </AlertDialog>
   </>
 );
}
```

```
------------------------------------------------------------
FICHIER : ./src/components/finances/GainsPanel.jsx
------------------------------------------------------------
import React, { useEffect, useMemo, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Trash2, Pencil, TrendingUp } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { useToast } from "@/components/ui/use-toast";
import { evalExpr, computeGrandeurs, sanitizeName, formatNumber } from "@/lib/mathEval";
import { loadMetrics } from "@/lib/calcSources";
import { useSpace } from "@/lib/SpaceContext";
import { formatCurrency } from "@/lib/format";

const PERIODS = [
  { value: "jour", label: "Journalier" },
  { value: "semaine", label: "Hebdomadaire" },
  { value: "mois", label: "Mensuel" },
  { value: "trimestre", label: "Trimestriel" },
  { value: "semestre", label: "Semestriel" },
  { value: "annee", label: "Annuel" },
];
const MONTH_FACTOR = { jour: 30, semaine: 4.345, mois: 1, trimestre: 1 / 3, semestre: 1 / 6, annee: 1 / 12 };
const toMonthly = (amount, period) => (Number(amount) || 0) * (MONTH_FACTOR[period] || 0);

const empty = { name: "", label: "", expression: "", unit: "XAF", gain_recurring: false, gain_period: "mois" };

export default function GainsPanel({ currency }) {
  const { activeCompanyId } = useSpace();
  const { toast } = useToast();
  const [items, setItems] = useState([]);
  const [metrics, setMetrics] = useState({});
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);

  const reload = async () => {
    const list = await base44.entities.CalcDefinition.list("-created_date", 500);
    setItems((list || []).filter((i) => i.kind === "gain"));
  };
  useEffect(() => {
    (async () => {
      await reload();
      try { setMetrics(await loadMetrics(activeCompanyId)); } catch { /* ignore */ }
      setLoading(false);
    })();
  }, [activeCompanyId]);

  const grandeurs = items; // gains can also reference other grandeurs
  const allDefs = useMemo(() => items, [items]);
  const scope = computeGrandeurs(allDefs, metrics);

  const evalGain = (g) => {
    try {
      const v = evalExpr(g.expression, { ...scope, ...metrics });
      return isFinite(v) ? v : null;
    } catch { return null; }
  };

  const totalMonthly = items.reduce((s, g) => {
    const v = evalGain(g);
    if (v == null) return s;
    return s + (g.gain_recurring ? toMonthly(v, g.gain_period) : 0);
  }, 0);
  const totalOneShot = items.reduce((s, g) => {
    const v = evalGain(g);
    if (v == null) return s;
    return s + (g.gain_recurring ? 0 : v);
  }, 0);

  const openNew = () => { setEditing(null); setForm(empty); setOpen(true); };
  const openEdit = (g) => {
    setEditing(g);
    setForm({
      name: g.name || "",
      label: g.label || "",
      expression: g.expression || "",
      unit: g.unit || "XAF",
      gain_recurring: !!g.gain_recurring,
      gain_period: g.gain_period || "mois",
    });
    setOpen(true);
  };

  const save = async () => {
    const name = sanitizeName(form.name);
    if (!name || !form.expression.trim()) { toast({ title: "Nom et formule requis", variant: "destructive" }); return; }
    const payload = { name, label: form.label || name, kind: "gain", expression: form.expression, unit: form.unit,
```

```
gain_recurring: form.gain_recurring, gain_period: form.gain_period };
    try {
      if (editing) await base44.entities.CalcDefinition.update(editing.id, payload);
      else await base44.entities.CalcDefinition.create(payload);
      toast({ title: "Gain enregistré" });
      setOpen(false); reload();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (g) => {
    try { await base44.entities.CalcDefinition.delete(g.id); reload(); }
    catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  if (loading) return <div className="flex justify-center py-8"><div className="w-6 h-6 border-2 border-border border-t-primary
rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between gap-2">
        <div className="grid grid-cols-2 gap-3 w-full">
          <div className="bg-card border border-border rounded-2xl p-4">
            <p className="text-sm text-muted-foreground">Gains récurrents (mois)</p>
            <p className="font-display text-xl mt-1">{formatCurrency(totalMonthly, currency)}</p>
          </div>
          <div className="bg-card border border-border rounded-2xl p-4">
            <p className="text-sm text-muted-foreground">Gains ponctuels</p>
            <p className="font-display text-xl mt-1">{formatCurrency(totalOneShot, currency)}</p>
          </div>
        </div>
        <Button onClick={openNew} size="sm"><Plus className="w-4 h-4 mr-1" /> Nouveau</Button>
      </div>

      {items.length === 0 ? (
        <div className="text-center py-10 text-muted-foreground">
          <TrendingUp className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p className="text-sm">Aucune formule de gain. Créez-en une (ex. <span className="font-mono">budget_initial *
0.2</span>).</p>
        </div>
      ) : (
        <div className="space-y-2">
          {items.map((g) => {
            const v = evalGain(g);
            const monthly = g.gain_recurring && v != null ? toMonthly(v, g.gain_period) : 0;
            return (
              <div key={g.id} className="bg-card border border-border rounded-xl p-4">
                <div className="flex items-start justify-between gap-3">
                  <div className="min-w-0">
                    <p className="text-sm font-medium truncate">{g.label} <span className="text-xs text-muted-foreground
font-mono">({g.name})</span></p>
                    <p className="text-sm font-mono text-accent truncate">{g.expression}{g.unit ? ` [${g.unit}]` : ""}</p>
                    <p className="text-xs text-muted-foreground mt-0.5">
                      {g.gain_recurring ? `Récurrent · ${PERIODS.find((p) => p.value === g.gain_period)?.label}` : "Ponctuel"} ·
                      {" "}Valeur : {v != null ? formatCurrency(v, currency) : "—"}
                                    { g . g a i n _ r e c u r r i n g   & &   v   ! =   n u l l   & &   `   ·  "H   $ { f o r
                    </p>
                  </div>
                  <div className="flex items-center gap-2 shrink-0">
                    <button onClick={() => openEdit(g)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                    <button onClick={() => remove(g)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive"><Trash2 className="w-4 h-4" /></button>
                  </div>
                </div>
              </div>
            );
          })}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent>
          <DialogHeader><DialogTitle>{editing ? "Modifier le gain" : "Nouvelle formule de gain"}</DialogTitle></DialogHeader>
          <div className="space-y-3">
            <div className="grid grid-cols-2 gap-3">
              <div className="space-y-1"><Label>Nom (identifiant)</Label><Input value={form.name} onChange={(e) => setForm({
...form, name: e.target.value })} placeholder="benefice_net" /></div>
              <div className="space-y-1"><Label>Libellé</Label><Input value={form.label} onChange={(e) => setForm({ ...form,
label: e.target.value })} placeholder="Bénéfice net" /></div>
            </div>
            <div className="space-y-1"><Label>Formule</Label><Input value={form.expression} onChange={(e) => setForm({ ...form,
expression: e.target.value })} placeholder="budget_initial * 0.2 - total_depenses" />
              <p className="text-xs text-muted-foreground">Référencez vos indicateurs : budget_initial, total_depenses,
ca_ventes, etc.</p>
            </div>
            <div className="grid grid-cols-2 gap-3">
              <div className="space-y-1"><Label>Type</Label>
                <Select value={form.gain_recurring ? "rec" : "ponc"} onValueChange={(v) => setForm({ ...form, gain_recurring: v
=== "rec" })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent><SelectItem value="ponc">Ponctuel</SelectItem><SelectItem
value="rec">Récurrent</SelectItem></SelectContent>
```

```
              </Select>
            </div>
            {form.gain_recurring && (
              <div className="space-y-1"><Label>Périodicité</Label>
                <Select value={form.gain_period} onValueChange={(v) => setForm({ ...form, gain_period: v })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent>{PERIODS.map((p) => <SelectItem key={p.value}
value={p.value}>{p.label}</SelectItem>)}</SelectContent>
                </Select>
              </div>
            )}
          </div>
          <div className="space-y-1"><Label>Unité</Label><Input value={form.unit} onChange={(e) => setForm({ ...form, unit:
e.target.value })} placeholder="XAF" /></div>
        </div>
        <DialogFooter><Button onClick={save}>Enregistrer</Button></DialogFooter>
      </DialogContent>
    </Dialog>
  </div>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/finances/SavingsPlanForm.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import PeriodPicker from "@/components/common/PeriodPicker";
import { daysInPeriod, periodLabel } from "@/lib/periods";
import { formatCurrency } from "@/lib/format";

const empty = { name: "", target_amount: "", period: "mois", customCount: "", customUnit: "jour", start_date: new
Date().toISOString().slice(0, 10) };

export default function SavingsPlanForm({ open, onOpenChange, onSubmit, currency = "XAF" }) {
  const [form, setForm] = useState(empty);
  const [error, setError] = useState("");

  useEffect(() => { if (open) { setForm(empty); setError(""); } }, [open]);

  const amount = parseFloat(form.target_amount) || 0;
  const days = daysInPeriod(form.period, new Date(form.start_date || Date.now()), { count: form.customCount, unit:
form.customUnit });
  const daily = days > 0 ? amount / days : 0;

  const submit = (e) => {
    e.preventDefault();
    if (!form.name.trim()) { setError("Nom requis"); return; }
    if (isNaN(amount) || amount <= 0) { setError("Montant invalide"); return; }
    onSubmit({
      name: form.name.trim(),
      target_amount: amount,
      period: form.period,
      custom_count: form.period === "personnalise" ? (parseInt(form.customCount, 10) || 0) : null,
      custom_unit: form.period === "personnalise" ? form.customUnit : null,
      start_date: form.start_date,
      status: "actif",
      saved_amount: 0,
    });
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader><DialogTitle className="font-display">Nouveau plan d'épargne</DialogTitle></DialogHeader>
        <form onSubmit={submit} className="space-y-3">
          {error && <p className="text-sm text-destructive">{error}</p>}
          <div className="space-y-2"><Label>Nom</Label><Input value={form.name} onChange={(e) => setForm({ ...form, name:
e.target.value })} placeholder="Ex. Vacances, fonds d'urgence…" required /></div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2"><Label>Montant à épargner ({currency})</Label><Input type="number" min="0" step="any"
value={form.target_amount} onChange={(e) => setForm({ ...form, target_amount: e.target.value })} required /></div>
            <div className="space-y-2"><Label>Date de début</Label><Input type="date" value={form.start_date} onChange={(e) =>
setForm({ ...form, start_date: e.target.value })} /></div>
          </div>
          <PeriodPicker
            period={form.period}
            customCount={form.customCount}
            customUnit={form.customUnit}
            onChange={({ period, customCount, customUnit }) => setForm({ ...form, period, customCount, customUnit })}
            label="Période d'épargne"
          />
          {amount > 0 && (
            <div className="rounded-xl bg-secondary p-3 text-sm space-y-1">
              <p>Soit <span className="font-medium">{formatCurrency(daily, currency)}</span> / jour</p>
              <p className="text-muted-foreground">Sur {days} jour(s) · {periodLabel(form.period, { count: form.customCount,
unit: form.customUnit })}</p>
            </div>
          )}
```

```
          <DialogFooter><Button type="submit" className="w-full">Créer le plan</Button></DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/finances/SavingsPlanList.jsx
------------------------------------------------------------
import React from "react";
import { PiggyBank, Trash2, CheckCircle2 } from "lucide-react";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { daysInPeriod, periodLabel, customOf } from "@/lib/periods";

export default function SavingsPlanList({ plans = [], currency = "XAF", onDelete, onToggleReached }) {
  if (!plans.length) {
    return (
      <div className="text-center py-12 text-muted-foreground">
        <PiggyBank className="w-8 h-8 mx-auto mb-3 opacity-50" />
        <p className="text-sm">Aucun plan d'épargne. Créez-en un pour mettre de côté automatiquement.</p>
      </div>
    );
  }
  return (
    <div className="space-y-2">
      {plans.map((p) => {
        const days = daysInPeriod(p.period, new Date(p.start_date || Date.now()), customOf(p));
        const daily = days > 0 ? (Number(p.target_amount) || 0) / days : 0;
        const reached = p.status === "atteint";
        return (
          <div key={p.id} className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
            <div className="min-w-0">
              <div className="flex items-center gap-2">
                <p className="text-sm font-medium truncate">{p.name}</p>
                {reached && <CheckCircle2 className="w-4 h-4 text-primary shrink-0" />}
              </div>
              <p className="text-xs text-muted-foreground">
                {formatCurrency(p.target_amount, currency)} sur {periodLabel(p.period, customOf(p))}
                {p.start_date ? ` · dès ${formatDateFr(p.start_date)}` : ""}
                {` · ${formatCurrency(daily, currency)}/jour`}
              </p>
            </div>
            <div className="flex items-center gap-1 shrink-0">
              <button onClick={() => onToggleReached(p)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-secondary"
title="Marquer atteint">
                <CheckCircle2 className={`w-4 h-4 ${reached ? "text-primary" : "text-muted-foreground"}`} />
              </button>
              <button onClick={() => onDelete(p)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive" title="Supprimer">
                <Trash2 className="w-4 h-4" />
              </button>
            </div>
          </div>
        );
      })}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/landing/FaqSection.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { motion } from "framer-motion";
import { ChevronDown, HelpCircle, Loader2, Send, MessageCircleQuestion } from "lucide-react";
import { base44 } from "@/api/base44Client";
import { useAuth } from "@/lib/AuthContext";
import { useToast } from "@/components/ui/use-toast";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";
import { Link } from "react-router-dom";

// Questions fréquentes statiques (présentation claire de la plateforme).
const SHORT = "JH Personal Manager";
const BIZ = "JH Business Manager";
const BRAND = "JH Manager";

const STATIC_FAQ = [
  {
    q: "Qu'est-ce que JH Manager ?",
    a: `${BRAND} est une plateforme tout-en-un : ${SHORT} pour gérer votre vie personnelle (finances, tâches, planning) et
${BIZ} pour gérer votre entreprise (stock, ventes, devis, partenariats). Un seul abonnement, un seul compte.`,
  },
  {
    q: `Quelle est la différence entre ${SHORT} et ${BIZ} ?`,
    a: `${SHORT} (Particulier, 5 000 XAF/mois) couvre la gestion personnelle. ${BIZ} (Entreprise, 20 000 XAF/mois par entreprise
gérée) couvre la gestion d'activité. Vous choisissez l'espace qui correspond à votre besoin.`,
  },
  {
    q: "Comment fonctionne l'affiliation ?",
    a: "Devenez affilié, partagez votre lien. Toute personne qui s'abonne via votre lien vous rapporte 20 % du prix normal (1
```

```
000 XAF/mois par particulier, 4 000 XAF/mois par entreprise) — à vie. Le filleul bénéficie de 20 % de réduction pendant 6
mois.",
  },
  {
    q: "Comment payer l'abonnement ?",
    a: "Le paiement se fait directement via Airtel Money ou Moov Money (Gabon). Aucune carte bancaire requise.",
  },
  {
    q: `Puis-je avoir plusieurs entreprises en ${BIZ} ?`,
    a: `Oui. Le tarif ${BIZ} est de 20 000 XAF/mois par entreprise gérée. Vous ajoutez autant d'entreprises que nécessaire.`,
  },
];

export default function FaqSection() {
  const { user } = useAuth();
  const { toast } = useToast();
  const [open, setOpen] = useState(0);
  const [published, setPublished] = useState([]);
  const [question, setQuestion] = useState("");
  const [category, setCategory] = useState("general");
  const [busy, setBusy] = useState(false);
  const [mine, setMine] = useState([]);

  const load = async () => {
    if (!user) return;
    try {
      const list = await base44.entities.FaqQuestion.list("-created_date", 50);
      setMine((list || []).filter((q) => q.is_published || q.created_by_id === user.id || user.role === "admin"));
      setPublished((list || []).filter((q) => q.is_published && q.answer));
    } catch { /* ignore */ }
  };

  useEffect(() => { load(); /* eslint-disable-line */ }, [user]);

  const submit = async (e) => {
    e.preventDefault();
    if (!question.trim()) return;
    setBusy(true);
    try {
      await base44.entities.FaqQuestion.create({ question: question.trim(), category });
      toast({ title: "Question envoyée", description: "Notre équipe vous répondra bientôt." });
      setQuestion("");
      setCategory("general");
      load();
    } catch (err) {
      toast({ title: "Erreur", description: err?.message || "Échec de l'envoi", variant: "destructive" });
    } finally { setBusy(false); }
  };

  const allFaq = [...STATIC_FAQ, ...published.map((p) => ({ q: p.question, a: p.answer }))];

  return (
    <section className="py-14 px-4">
      <div className="max-w-3xl mx-auto">
        <motion.div
          initial={{ opacity: 0, y: 20 }} whileInView={{ opacity: 1, y: 0 }} viewport={{ once: true }}
          className="text-center mb-8"
        >
          <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-accent/15 text-accent-foreground text-xs
font-medium mb-3">
            <MessageCircleQuestion className="w-4 h-4" /> FAQ
          </div>
          <h2 className="font-display text-3xl sm:text-4xl">Questions fréquentes</h2>
          <p className="text-muted-foreground mt-2">Tout ce que vous devez savoir avant de commencer.</p>
        </motion.div>

        <div className="space-y-2">
          {allFaq.map((f, i) => (
            <div key={i} className="rounded-2xl border border-border bg-card overflow-hidden">
              <button
                onClick={() => setOpen(open === i ? -1 : i)}
                className="no-select-tap w-full flex items-center justify-between gap-3 p-4 text-left"
              >
                <span className="font-medium text-sm sm:text-base">{f.q}</span>
                <ChevronDown className={`w-4 h-4 shrink-0 transition-transform ${open === i ? "rotate-180" : ""}`} />
              </button>
              <motion.div
                initial={false}
                animate={{ height: open === i ? "auto" : 0, opacity: open === i ? 1 : 0 }}
                className="overflow-hidden"
              >
                <p className="px-4 pb-4 text-sm text-muted-foreground">{f.a}</p>
              </motion.div>
            </div>
          ))}
        </div>

        <div className="mt-8 rounded-3xl border border-accent/30 bg-gradient-to-br from-accent/10 to-primary/5 p-6">
          <div className="flex items-center gap-2 mb-3">
            <HelpCircle className="w-5 h-5 text-accent" />
            <h3 className="font-display text-lg">Posez votre question</h3>
          </div>
```

```
          {user ? (
            <form onSubmit={submit} className="space-y-3">
              <Textarea
                value={question}
                onChange={(e) => setQuestion(e.target.value)}
                placeholder="Décrivez votre question…"
                rows={3}
                className="bg-background"
              />
              <div className="flex flex-col sm:flex-row gap-2">
                <select
                  value={category}
                  onChange={(e) => setCategory(e.target.value)}
                  className="h-9 rounded-md border border-input bg-transparent px-3 text-sm"
                >
                  <option value="general">Général</option>
                  <option value="jpm">JH Personal Manager</option>
                  <option value="jbm">JH Business Manager</option>
                  <option value="paiement">Paiement</option>
                  <option value="affiliation">Affiliation</option>
                  <option value="compte">Compte</option>
                </select>
                <Button type="submit" disabled={busy} className="flex-1">
                  {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : <Send className="w-4 h-4" />}
                  Envoyer
                </Button>
              </div>
            </form>
          ) : (
            <div className="text-center py-2">
              <p className="text-sm text-muted-foreground mb-3">Connectez-vous pour poser une question à notre équipe.</p>
              <Button asChild variant="outline">
                <Link to="/register?returnTo=/">Se connecter pour poser une question</Link>
              </Button>
            </div>
          )}
        </div>
      </div>
    </section>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/layout/AccountBar.jsx
------------------------------------------------------------
import React from "react";
import SpaceSwitcher from "@/components/layout/SpaceSwitcher";
import AddAccountButton from "@/components/account/AddAccountButton";
import { useSpace } from "@/lib/SpaceContext";
import { useAuth } from "@/lib/AuthContext";

// Barre de compte affichée en haut du contenu :
// - deux espaces possédés ! sélecteur JPM / JBM ;
// - un seul espace (non-admin) ! bouton « Ajouter un compte ».
export default function AccountBar() {
  const { availableSpaces, loading } = useSpace();
  const { user } = useAuth();
  if (loading || !user) return null;
  const isAdmin = user.role === "admin";
  if (isAdmin || availableSpaces.length >= 2) return <SpaceSwitcher />;
  return <AddAccountButton />;
}

------------------------------------------------------------
FICHIER : ./src/components/layout/BottomNav.jsx
------------------------------------------------------------
import React from "react";
import { NavLink } from "react-router-dom";
import { LayoutDashboard, Wallet, ListChecks, CalendarRange, CalendarDays, Bell, Calculator, Gauge, Settings, Bot, Boxes,
ShoppingCart, Users, FileText, ScrollText, Truck, Percent, Link, CalendarClock, Lightbulb, UsersRound, HelpCircle } from
"lucide-react";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";

export default function BottomNav() {
  const { t } = useI18n();
  const { space } = useSpace();

  const JPM_ITEMS = [
    { to: "/", label: t("nav.home"), icon: LayoutDashboard },
    { to: "/finances", label: t("nav.finances"), icon: Wallet },
    { to: "/taches", label: t("nav.tasks"), icon: ListChecks },
    { to: "/planning", label: t("nav.planning"), icon: CalendarRange },
    { to: "/evenements", label: t("nav.events"), icon: CalendarClock },
    { to: "/calendrier", label: t("nav.calendar"), icon: CalendarDays },
    { to: "/notifications", label: t("nav.notifications"), icon: Bell },
    { to: "/seuils", label: t("nav.thresholds"), icon: Gauge },
    { to: "/calculateurs", label: t("nav.calculators"), icon: Calculator },
    { to: "/collaborateurs", label: "Collaborateurs", icon: UsersRound },
    { to: "/assistants", label: t("nav.agents"), icon: Bot },
    { to: "/suggestions", label: t("nav.suggestions"), icon: Lightbulb },
    { to: "/faq", label: "FAQ", icon: HelpCircle },
```

```
    { to: "/parametres", label: t("nav.settings"), icon: Settings },
  ];

  const JBM_ITEMS = [
    { to: "/jbm", label: t("jbm.nav.dashboard"), icon: LayoutDashboard },
    { to: "/finances", label: t("nav.finances"), icon: Wallet },
    { to: "/taches", label: t("nav.tasks"), icon: ListChecks },
    { to: "/planning", label: t("nav.planning"), icon: CalendarRange },
    { to: "/evenements", label: t("nav.events"), icon: CalendarClock },
    { to: "/calendrier", label: t("nav.calendar"), icon: CalendarDays },
    { to: "/jbm/stock", label: t("jbm.nav.stock"), icon: Boxes },
    { to: "/jbm/ventes", label: t("jbm.nav.sales"), icon: ShoppingCart },
    { to: "/jbm/factures", label: t("jbm.nav.invoices"), icon: FileText },
    { to: "/jbm/clients", label: t("nav.clients"), icon: Users },
    { to: "/jbm/devis", label: t("nav.quotes"), icon: ScrollText },
    { to: "/jbm/prestataires", label: t("nav.suppliers"), icon: Truck },
    { to: "/jbm/taxes", label: t("nav.taxes"), icon: Percent },
    { to: "/jbm/partenariats", label: t("nav.partnerships"), icon: Link },
    { to: "/collaborateurs", label: "Collaborateurs", icon: UsersRound },
    { to: "/assistants", label: t("nav.agents"), icon: Bot },
    { to: "/faq", label: "FAQ", icon: HelpCircle },
    { to: "/parametres", label: t("nav.settings"), icon: Settings },
  ];

  const NAV_ITEMS = space === "jbm" ? JBM_ITEMS : JPM_ITEMS;

  return (
    <nav className="no-select-tap md:hidden fixed bottom-0 inset-x-0 bg-card/95 backdrop-blur border-t border-border z-20">
      <div className="flex items-stretch overflow-x-auto no-scrollbar px-1 pb-[env(safe-area-inset-bottom)]">
        {NAV_ITEMS.map(({ to, label, icon: Icon }) => (
          <NavLink
            key={to}
            to={to}
            end={to === "/" || to === "/jbm"}
            className={({ isActive }) =>
              `no-select-tap min-touch shrink-0 flex flex-col items-center justify-center gap-1 px-2.5 py-2 text-[11px]
transition-colors ${
                isActive ? "text-primary" : "text-muted-foreground"
              }`
            }
          >
            <Icon className="w-5 h-5" />
            <span className="max-w-[60px] truncate">{label}</span>
          </NavLink>
        ))}
      </div>
    </nav>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/layout/SidebarNav.jsx
------------------------------------------------------------
import React from "react";
import { NavLink } from "react-router-dom";
import { LayoutDashboard, Wallet, ListChecks, CalendarRange, CalendarDays, Bell, Calculator, Gauge, Settings, LogOut, Bot,
Boxes, ShoppingCart, Users, FileText, Building2, ScrollText, Truck, Percent, Link, Lightbulb, CalendarClock, BarChart3,
UsersRound, HelpCircle, FileDown, Sunrise } from "lucide-react";
import { base44 } from "@/api/base44Client";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";

const ACCOUNT_BADGE = {
  jpm: { label: "Particulier", className: "bg-primary/15 text-primary" },
  jbm: { label: "Entreprise", className: "bg-accent/20 text-accent" },
};

export default function SidebarNav({ userEmail }) {
  const { t } = useI18n();
  const { space, companies, activeCompanyId } = useSpace();
  const badge = ACCOUNT_BADGE[space] || ACCOUNT_BADGE.jpm;

  const JPM_ITEMS = [
    { to: "/", label: t("nav.home"), icon: LayoutDashboard },
    { to: "/finances", label: t("nav.finances"), icon: Wallet },
    { to: "/taches", label: t("nav.tasks"), icon: ListChecks },
    { to: "/planning", label: t("nav.planning"), icon: CalendarRange },
    { to: "/evenements", label: t("nav.events"), icon: CalendarClock },
    { to: "/calendrier", label: t("nav.calendar"), icon: CalendarDays },
    { to: "/notifications", label: t("nav.notifications"), icon: Bell },
    { to: "/seuils", label: t("nav.thresholds"), icon: Gauge },
    { to: "/calculateurs", label: t("nav.calculators"), icon: Calculator },
    { to: "/reporting", label: "Reporting", icon: BarChart3 },
    { to: "/exportations", label: "Exportations", icon: FileDown },
    { to: "/assistants", label: t("nav.agents"), icon: Bot },
    { to: "/collaborateurs", label: "Collaborateurs", icon: UsersRound },
    { to: "/suggestions", label: t("nav.suggestions"), icon: Lightbulb },
    { to: "/faq", label: "FAQ", icon: HelpCircle },
    { to: "/resume-matinal", label: "Résumé matinal", icon: Sunrise },
    { to: "/parametres", label: t("nav.settings"), icon: Settings },
  ];
```

```
  const JBM_ITEMS = [
    { to: "/jbm", label: t("jbm.nav.dashboard"), icon: LayoutDashboard },
    { to: "/finances", label: t("nav.finances"), icon: Wallet },
    { to: "/taches", label: t("nav.tasks"), icon: ListChecks },
    { to: "/planning", label: t("nav.planning"), icon: CalendarRange },
    { to: "/evenements", label: t("nav.events"), icon: CalendarClock },
    { to: "/calendrier", label: t("nav.calendar"), icon: CalendarDays },
    { to: "/jbm/stock", label: t("jbm.nav.stock"), icon: Boxes },
    { to: "/jbm/ventes", label: t("jbm.nav.sales"), icon: ShoppingCart },
    { to: "/jbm/clients", label: t("nav.clients"), icon: Users },
    { to: "/jbm/devis", label: t("nav.quotes"), icon: ScrollText },
    { to: "/jbm/personnel", label: t("jbm.nav.personnel"), icon: Users },
    { to: "/jbm/prestataires", label: t("nav.suppliers"), icon: Truck },
    { to: "/jbm/taxes", label: t("nav.taxes"), icon: Percent },
    { to: "/jbm/partenariats", label: t("nav.partnerships"), icon: Link },
    { to: "/jbm/entreprises", label: t("jbm.nav.companies"), icon: Building2 },
    { to: "/calculateurs", label: t("nav.calculators"), icon: Calculator },
    { to: "/reporting", label: "Reporting", icon: BarChart3 },
    { to: "/exportations", label: "Exportations", icon: FileDown },
    { to: "/collaborateurs", label: "Collaborateurs", icon: UsersRound },
    { to: "/assistants", label: t("nav.agents"), icon: Bot },
    { to: "/suggestions", label: t("nav.suggestions"), icon: Lightbulb },
    { to: "/faq", label: "FAQ", icon: HelpCircle },
    { to: "/parametres", label: t("nav.settings"), icon: Settings },
  ];

  const ITEMS = space === "jbm" ? JBM_ITEMS : JPM_ITEMS;
  const activeCompany = companies.find((c) => c.id === activeCompanyId);

  return (
    <aside className="hidden md:flex md:flex-col md:fixed md:inset-y-0 md:left-0 md:w-64 bg-sidebar text-sidebar-foreground px-5
py-8 overflow-y-auto">
      <div className="mb-6 px-2">
        <p className="font-display text-2xl tracking-tight">JH Manager</p>
      </div>
      <div className="mb-6 px-1">
        <span className={`inline-flex items-center px-2.5 py-1 rounded-full text-xs font-medium ${badge.className}`}>
          {badge.label}
        </span>
      </div>
      {space === "jbm" && (
        <p className="text-xs text-sidebar-foreground/50 px-2 mb-4 truncate">
          {activeCompany?.name || t("jbm.companies.empty")}
        </p>
      )}
      <nav className="flex-1 space-y-1">
        {ITEMS.map(({ to, label, icon: Icon }) => (
          <NavLink
            key={to}
            to={to}
            end={to === "/" || to === "/jbm"}
            className={({ isActive }) =>
              `flex items-center gap-3 px-3 py-2.5 rounded-xl text-sm transition-colors ${
                isActive
                  ? "bg-sidebar-accent text-sidebar-accent-foreground"
                  : "text-sidebar-foreground/70 hover:bg-sidebar-accent/60 hover:text-sidebar-accent-foreground"
              }`
            }
          >
            <Icon className="w-4 h-4" />
            {label}
          </NavLink>
        ))}
      </nav>
      <div className="pt-4 border-t border-sidebar-border/60">
        <p className="text-xs text-sidebar-foreground/50 px-3 truncate mb-2">{userEmail}</p>
        <button
          onClick={() => base44.auth.logout()}
          className="no-select-tap flex items-center gap-3 px-3 py-2.5 rounded-xl text-sm text-sidebar-foreground/70
hover:bg-sidebar-accent/60 hover:text-sidebar-accent-foreground w-full transition-colors"
        >
          <LogOut className="w-4 h-4" />
          {t("nav.logout")}
        </button>
      </div>
    </aside>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/layout/SpaceSwitcher.jsx
------------------------------------------------------------
import React from "react";
import { useNavigate } from "react-router-dom";
import { User, Building2 } from "lucide-react";
import { useSpace } from "@/lib/SpaceContext";
import { useI18n } from "@/lib/i18n";

export default function SpaceSwitcher() {
  const { space, setSpace } = useSpace();
  const { t } = useI18n();
  const navigate = useNavigate();
```

```
  const switchTo = (next) => {
    if (next === space) return;
    setSpace(next);
    navigate(next === "jbm" ? "/jbm" : "/");
  };

  return (
    <div className="no-select-tap inline-flex rounded-full bg-secondary p-1 text-xs font-medium">
      <button
        type="button"
        onClick={() => switchTo("jpm")}
        className={`flex items-center gap-1.5 px-3 py-1.5 rounded-full transition-colors ${
          space === "jpm" ? "bg-primary text-primary-foreground" : "text-muted-foreground"
        }`}
      >
        <User className="w-3.5 h-3.5" />
        {t("space.jpm")}
      </button>
      <button
        type="button"
        onClick={() => switchTo("jbm")}
        className={`flex items-center gap-1.5 px-3 py-1.5 rounded-full transition-colors ${
          space === "jbm" ? "bg-primary text-primary-foreground" : "text-muted-foreground"
        }`}
      >
        <Building2 className="w-3.5 h-3.5" />
        {t("space.jbm")}
      </button>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/loans/LoanForm.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter, DialogDescription } from "@/components/ui/dialog";
import { LOAN_INTEREST_TYPES, computePayment, totalToRepay, totalInterest } from "@/lib/loanMath";
import PeriodPicker from "@/components/common/PeriodPicker";
import { formatCurrency } from "@/lib/format";

const empty = { name: "", principal: "", interest_rate: "", interest_type: "compose", frequency: "mois", customCount: "",
customUnit: "jour", term_periods: "", start_date: new Date().toISOString().slice(0, 10), description: "" };

export default function LoanForm({ open, onOpenChange, onSubmit, loading }) {
  const [form, setForm] = useState(empty);

  useEffect(() => { if (open) setForm(empty); }, [open]);

  const P = parseFloat(form.principal) || 0;
  const r = parseFloat(form.interest_rate) || 0;
  const n = parseInt(form.term_periods, 10) || 0;
  const payment = computePayment(P, r, n, form.interest_type);
  const total = totalToRepay(P, r, n, form.interest_type);
  const interest = totalInterest(P, r, n, form.interest_type);

  const submit = (e) => {
    e.preventDefault();
    if (!form.name.trim() || P <= 0 || n <= 0 || !form.start_date) return;
    onSubmit({
      name: form.name.trim(),
      principal: P,
      interest_rate: r,
      interest_type: form.interest_type,
      frequency: form.frequency,
      custom_count: form.frequency === "personnalise" ? (parseInt(form.customCount, 10) || 0) : null,
      custom_unit: form.frequency === "personnalise" ? form.customUnit : null,
      term_periods: n,
      start_date: form.start_date,
      monthly_payment: payment,
      total_to_repay: total,
      total_interest: interest,
      remaining_balance: P,
      payments_made: 0,
      next_payment_date: form.start_date,
      status: "en_cours",
      description: form.description.trim(),
    });
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display">Nouvel emprunt</DialogTitle>
```

```
          <DialogDescription>Définissez le capital, le taux d'intérêt et la période. Le suivi s'actualise à chaque
versement.</DialogDescription>
        </DialogHeader>
        <form onSubmit={submit} className="space-y-3">
          <div className="space-y-2"><Label>Libellé</Label><Input value={form.name} onChange={(e) => setForm({ ...form, name:
e.target.value })} placeholder="Ex. Prêt moto, crédit étudiant…" required /></div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2"><Label>Capital (XAF)</Label><Input type="number" min="0" value={form.principal}
onChange={(e) => setForm({ ...form, principal: e.target.value })} required /></div>
            <div className="space-y-2"><Label>Taux / période (%)</Label><Input type="number" min="0" step="0.01"
value={form.interest_rate} onChange={(e) => setForm({ ...form, interest_rate: e.target.value })} required /></div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2"><Label>Type d'intérêt</Label>
              <Select value={form.interest_type} onValueChange={(v) => setForm({ ...form, interest_type: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>{LOAN_INTEREST_TYPES.map((o) => <SelectItem key={o.value}
value={o.value}>{o.label}</SelectItem>)}</SelectContent>
              </Select>
            </div>
            <div className="space-y-2">
              <PeriodPicker
                period={form.frequency}
                customCount={form.customCount}
                customUnit={form.customUnit}
                onChange={({ period, customCount, customUnit }) => setForm({ ...form, frequency: period, customCount, customUnit
})}
                label="Périodicité"
              />
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2"><Label>Nb d'échéances</Label><Input type="number" min="1" value={form.term_periods}
onChange={(e) => setForm({ ...form, term_periods: e.target.value })} required /></div>
                        <div   c l a s s N a m e = " s p a c e - y - 2 " > < L a b e l > 1 ³ I   é c h é a n c e < / L a b e l > <
setForm({ ...form, start_date: e.target.value })} required /></div>
          </div>
          {payment > 0 && (
            <div className="rounded-xl bg-secondary p-3 text-sm space-y-1">
              <p>Échéance : <span className="font-medium">{formatCurrency(payment, "XAF")}</span></p>
              <p>Total à rembourser : <span className="font-medium">{formatCurrency(total, "XAF")}</span></p>
              <p className="text-muted-foreground">Dont intérêts : {formatCurrency(interest, "XAF")}</p>
            </div>
          )}
          <div className="space-y-2"><Label>Description</Label><Textarea rows={2} value={form.description} onChange={(e) =>
setForm({ ...form, description: e.target.value })} /></div>
          <DialogFooter><Button type="submit" className="w-full" disabled={loading}>{loading && <Loader2 className="w-4 h-4 mr-2
animate-spin" />}Créer l'emprunt</Button></DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/loans/LoanItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { ChevronDown, ChevronUp, CheckCircle2, Trash2, Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
AlertDialogHeader, AlertDialogTitle } from "@/components/ui/alert-dialog";
import { buildSchedule } from "@/lib/loanMath";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { periodLabel, customOf } from "@/lib/periods";

export default function LoanItem({ loan, payments, onPay, onDelete, payingId, delId, setDelId }) {
  const [open, setOpen] = useState(false);
  const schedule = buildSchedule(loan.principal, loan.interest_rate, loan.term_periods, loan.interest_type);
  const made = loan.payments_made || 0;
  const progress = loan.term_periods > 0 ? Math.min(100, Math.round((made / loan.term_periods) * 100)) : 0;
  const isDone = loan.status === "rembourse" || made >= loan.term_periods;
  const next = loan.payments_made < schedule.length ? schedule[loan.payments_made] : null;

  return (
    <div className="bg-card border border-border rounded-2xl p-4">
      <div className="flex items-start justify-between gap-3">
        <div className="min-w-0">
          <div className="flex items-center gap-2">
            <p className="text-sm font-medium truncate">{loan.name}</p>
            {isDone && <CheckCircle2 className="w-4 h-4 text-primary shrink-0" />}
          </div>
          <p className="text-xs text-muted-foreground">
            {formatCurrency(loan.principal, "XAF")} · {loan.interest_rate}% / {periodLabel(loan.frequency, customOf(loan))} ·
{loan.interest_type === "simple" ? "intérêts simples" : "composés"} · {loan.term_periods} échéances
          </p>
        </div>
        <button onClick={() => setDelId(loan)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive shrink-0"><Trash2 className="w-4 h-4" /></button>
      </div>

      <div className="grid grid-cols-3 gap-2 mt-3 text-center">
```

```
        <div className="rounded-lg bg-secondary p-2"><p className="text-[11px] text-muted-foreground">Échéance</p><p
className="text-sm font-medium">{formatCurrency(loan.monthly_payment, "XAF")}</p></div>
        <div className="rounded-lg bg-secondary p-2"><p className="text-[11px] text-muted-foreground">Solde restant</p><p
className="text-sm font-medium">{formatCurrency(loan.remaining_balance, "XAF")}</p></div>
        <div className="rounded-lg bg-secondary p-2"><p className="text-[11px] text-muted-foreground">Prochaine</p><p
className="text-sm font-medium">{isDone ? "—" : formatDateFr(loan.next_payment_date)}</p></div>
      </div>

      <div className="mt-3">
        <div className="flex justify-between text-xs text-muted-foreground mb-1"><span>{made}/{loan.term_periods}
réglées</span><span>{progress}%</span></div>
        <div className="h-2 rounded-full bg-secondary overflow-hidden"><div className="h-full bg-primary transition-all"
style={{ width: `${progress}%` }} /></div>
      </div>

      <div className="flex items-center gap-2 mt-3">
        {!isDone ? (
          <Button size="sm" className="flex-1" onClick={() => onPay(loan)} disabled={payingId === loan.id}>
            {payingId === loan.id && <Loader2 className="w-4 h-4 mr-2 animate-spin" />} Enregistrer l'échéance n°{made + 1}
          </Button>
        ) : (
          <p className="text-sm text-primary flex items-center gap-1"><CheckCircle2 className="w-4 h-4" /> Emprunt remboursé</p>
        )}
        <Button size="sm" variant="outline" onClick={() => setOpen((o) => !o)}>{open ? <ChevronUp className="w-4 h-4" /> :
<ChevronDown className="w-4 h-4" />}Détails</Button>
      </div>

      {open && (
        <div className="mt-3 overflow-x-auto">
          <table className="w-full text-xs">
            <thead><tr className="text-muted-foreground text-left">
              <th className="py-1 pr-2">#</th><th className="py-1 pr-2">Échéance</th><th className="py-1 pr-2">Intérêts</th><th
className="py-1 pr-2">Capital</th><th className="py-1">Solde</th>
            </tr></thead>
            <tbody>
              {schedule.map((row) => {
                const paid = row.n <= made;
                return (
                  <tr key={row.n} className={paid ? "text-muted-foreground line-through" : ""}>
                    <td className="py-1 pr-2">{row.n}</td>
                    <td className="py-1 pr-2">{formatCurrency(row.payment, "XAF")}</td>
                    <td className="py-1 pr-2">{formatCurrency(row.interest, "XAF")}</td>
                    <td className="py-1 pr-2">{formatCurrency(row.principal, "XAF")}</td>
                    <td className="py-1">{formatCurrency(row.balance, "XAF")}</td>
                  </tr>
                );
              })}
            </tbody>
          </table>
        </div>
      )}

      <AlertDialog open={!!delId} onOpenChange={(o) => !o && setDelId(null)}>
        <AlertDialogContent>
          <AlertDialogHeader><AlertDialogTitle>Supprimer cet emprunt ?</AlertDialogTitle><AlertDialogDescription>« {delId?.name}
» et son historique seront supprimés.</AlertDialogDescription></AlertDialogHeader>
          <AlertDialogFooter><AlertDialogCancel>Annuler</AlertDialogCancel><AlertDialogAction className="bg-destructive
text-destructive-foreground hover:bg-destructive/90" onClick={(e) => { e.preventDefault(); onDelete(delId);
}}>Supprimer</AlertDialogAction></AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/onboarding/SubscribeDialog.jsx
------------------------------------------------------------
import React, { useState, useEffect } from "react";
import { Loader2, Minus, Plus, Check } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import {
  Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader, DialogTitle,
} from "@/components/ui/dialog";
import { useToast } from "@/components/ui/use-toast";
import { base44 } from "@/api/base44Client";

const UNIT_PRICE_ENTREPRISE = 20000;
const PRICE_PARTICULIER = 5000;

export default function SubscribeDialog({ plan, open, onOpenChange, onSuccess, defaultCount = 1, alreadyCovered = 0 }) {
  const [accountName, setAccountName] = useState("");
  const [phone, setPhone] = useState("");
  const [provider, setProvider] = useState("airtel");
  const [companiesCount, setCompaniesCount] = useState(defaultCount);
  const [submitting, setSubmitting] = useState(false);
  const { toast } = useToast();

  useEffect(() => {
```

```
    if (open) {
      setAccountName("");
      setPhone("");
      setProvider("airtel");
      setCompaniesCount(defaultCount);
    }
  }, [open, defaultCount]);

  if (!plan) return null;
  const isEnt = plan.type === "entreprise";
  const unitPrice = plan.unitPrice || UNIT_PRICE_ENTREPRISE;
  const isUpgrade = alreadyCovered > 0;
  // En mise à niveau (ajout d'entreprise), le compteur est figé au nouveau total
  // et l'on ne facture que l'entreprise supplémentaire (les autres sont déjà payées).
  const newTotal = isUpgrade ? defaultCount : companiesCount;
  const billableUnits = isEnt ? Math.max(0, newTotal - alreadyCovered) : 1;
  const fullAmount = isEnt
    ? unitPrice * billableUnits
    : plan.baseAmount || PRICE_PARTICULIER;
  // Réduction parrainage : 20 % les 6 premiers mois (validée côté serveur).
  const hasRef = typeof localStorage !== "undefined" && !!localStorage.getItem("jpm_ref");
  const discount = hasRef ? Math.round(fullAmount * 0.8) : fullAmount;
  const amount = discount;

  const setCount = (v) => setCompaniesCount(Math.max(1, Math.min(20, v)));

  const handlePay = async (e) => {
    e.preventDefault();
    if (!accountName.trim()) {
      toast({ title: "Nom requis", description: "Indiquez le nom figurant sur votre compte Mobile Money.", variant:
"destructive" });
      return;
    }
    const digits = phone.replace(/\D/g, "");
    if (digits.length < 6) {
      toast({ title: "Numéro invalide", description: "Entrez un numéro d'au moins 6 chiffres (ex. 60367591).", variant:
"destructive" });
      return;
    }
    const fullPhone = "+241" + digits;
    setSubmitting(true);
    try {
      const res = await base44.functions.invoke('processPayment', {
        account_name: accountName.trim(),
        phoneNumber: fullPhone,
        provider,
        account_type: plan.type,
        companies_count: isEnt ? newTotal : 1,
        already_covered: alreadyCovered,
        referral_code: (typeof localStorage !== "undefined" ? localStorage.getItem("jpm_ref") : "") || "",
      });
      if (res?.success) {
        try {
          const patch = { account_type: plan.type };
          await base44.auth.updateMe(patch);
        } catch { /* non-fatal */ }
        toast({ title: "Paiement validé", description: "Votre abonnement est actif. Bienvenue !" });
        onSuccess?.();
      } else {
        toast({ title: "Échec du paiement", description: res?.error || "Erreur inconnue", variant: "destructive" });
      }
    } catch (err) {
      const body = err && err.response && err.response.data;
      const msg = (body && body.error) || (err && err.message) || "Erreur réseau";
      toast({
        title: "Échec du paiement",
        description: /invoke|undefined|Request failed/i.test(msg)
          ? "Erreur technique : la fonction de paiement est injoignable. Réessayez plus tard."
          : msg,
        variant: "destructive",
      });
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent>
        <DialogHeader>
          <DialogTitle>Abonnement {plan.label}</DialogTitle>
          <DialogDescription>
            {isEnt
              ? `${unitPrice.toLocaleString("fr-FR")} XAF par entreprise gérée.`
              : `Montant : ${amount.toLocaleString("fr-FR")} XAF — prélevés sur votre Mobile Money.`}
          </DialogDescription>
        </DialogHeader>
        <form onSubmit={handlePay} className="space-y-4">
          {isEnt && !isUpgrade && (
            <div className="space-y-2">
              <Label>Nombre d'entreprises à gérer</Label>
              <div className="flex items-center gap-3">
```

```
                <button type="button" onClick={() => setCount(companiesCount - 1)} className="no-select-tap min-touch p-2
rounded-lg border border-input"><Minus className="w-4 h-4" /></button>
                <span className="font-display text-xl w-10 text-center">{companiesCount}</span>
                <button type="button" onClick={() => setCount(companiesCount + 1)} className="no-select-tap min-touch p-2
rounded-lg border border-input"><Plus className="w-4 h-4" /></button>
                <span className="ml-auto text-sm text-muted-foreground">1 = {unitPrice.toLocaleString("fr-FR")} XAF</span>
              </div>
            </div>
          )}
          {isEnt && isUpgrade && (
            <div className="rounded-lg border border-input bg-secondary/40 px-3 py-2 text-sm space-y-1">
              <div className="font-medium">Ajout d'une entreprise</div>
              <div className="text-muted-foreground">Total couvert après paiement : {newTotal} entreprise(s) · coût de la
nouvelle entreprise : {unitPrice.toLocaleString("fr-FR")} XAF.</div>
            </div>
          )}
          <div className="space-y-1">
            <div className="text-sm font-medium">{`Total : ${amount.toLocaleString("fr-FR")} XAF`}</div>
            {hasRef && (
              <p className="text-xs text-primary flex items-center gap-1">
                <Check className="w-3.5 h-3.5" /> Parrainage : 20 % de réduction les 6 premiers mois.
                {!isEnt && null}
              </p>
            )}
          </div>
          <div className="space-y-2">
            <Label>Pays</Label>
            <Select value="GA" disabled>
              <SelectTrigger><SelectValue /></SelectTrigger>
              <SelectContent><SelectItem value="GA">Gabon (+241)</SelectItem></SelectContent>
            </Select>
          </div>
          <div className="space-y-2">
            <Label htmlFor="sub-name">Nom sur le compte Mobile Money</Label>
            <Input id="sub-name" value={accountName} onChange={(e) => setAccountName(e.target.value)} placeholder="Ex. Jean
Dupont" />
          </div>
          <div className="space-y-2">
            <Label htmlFor="sub-phone">Numéro Mobile Money</Label>
            <div className="flex items-center gap-2">
              <span className="px-3 h-9 inline-flex items-center rounded-md border border-input bg-muted text-sm
shrink-0">+241</span>
              <Input id="sub-phone" type="tel" inputMode="numeric" value={phone} onChange={(e) => setPhone(e.target.value)}
placeholder="Ex. 074 12 34 56" />
            </div>
          </div>
          <div className="space-y-2">
            <Label>Opérateur</Label>
            <Select value={provider} onValueChange={setProvider}>
              <SelectTrigger><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="airtel">Airtel Money</SelectItem>
                <SelectItem value="moov">Moov Money</SelectItem>
              </SelectContent>
            </Select>
          </div>
          <DialogFooter>
            <Button type="button" variant="outline" onClick={() => onOpenChange(false)} disabled={submitting}>Annuler</Button>
            <Button type="submit" disabled={submitting} className={isEnt ? "bg-accent text-accent-foreground hover:bg-accent/90"
: ""}>
              {submitting ? <><Loader2 className="w-4 h-4 animate-spin" /> Traitement…</> : `Payer
${amount.toLocaleString("fr-FR")} XAF`}
            </Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/search/GlobalSearch.jsx
------------------------------------------------------------
import React, { useState, useEffect, useMemo, useCallback } from "react";
import { Search, Loader2 } from "lucide-react";
import { Dialog, DialogContent } from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { base44 } from "@/api/base44Client";
import { useSpace } from "@/lib/SpaceContext";
import { useNavigate } from "react-router-dom";

// Recherche globale : charge les principales entités puis filre côté client.
const CONFIG = [
  { key: "Task", label: "Tâche", fields: ["title"], link: "/taches", primary: "title" },
  { key: "Expense", label: "Dépense", fields: ["name", "category"], link: "/finances", primary: "name" },
  { key: "Event", label: "Événement", fields: ["title", "location"], link: "/evenements", primary: "title" },
  { key: "Notification", label: "Notification", fields: ["title", "message"], link: "/notifications", primary: "title" },
  { key: "Suggestion", label: "Suggestion", fields: ["content"], link: "/suggestions", primary: "content" },
  { key: "Product", label: "Produit", fields: ["name"], link: "/jbm/stock", primary: "name", scoped: true },
  { key: "Sale", label: "Vente", fields: ["item_name", "customer_name"], link: "/jbm/ventes", primary: "item_name", scoped: true
},
  { key: "Invoice", label: "Facture", fields: ["number", "customer_name"], link: "/jbm/factures", primary: "number", scoped:
```

```
true },
  { key: "Client", label: "Client", fields: ["name"], link: "/jbm/clients", primary: "name", scoped: true },
  { key: "Quote", label: "Devis", fields: ["number", "client_name"], link: "/jbm/devis", primary: "number", scoped: true },
  { key: "Supplier", label: "Prestataire", fields: ["name"], link: "/jbm/prestataires", primary: "name", scoped: true },
  { key: "Tax", label: "Taxe", fields: ["name"], link: "/jbm/taxes", primary: "name", scoped: true },
  { key: "Partnership", label: "Partenariat", fields: ["partner_name"], link: "/jbm/partenariats", primary: "partner_name",
scoped: true },
  { key: "Employee", label: "Employé", fields: ["full_name", "role"], link: "/jbm/personnel", primary: "full_name", scoped: true
},
  { key: "Company", label: "Entreprise", fields: ["name"], link: "/jbm/entreprises", primary: "name" },
];

export default function GlobalSearch() {
  const [open, setOpen] = useState(false);
  const [q, setQ] = useState("");
  const [loading, setLoading] = useState(false);
  const [data, setData] = useState({});
  const { activeCompanyId } = useSpace();
  const navigate = useNavigate();

  const loadAll = useCallback(async () => {
    setLoading(true);
    const entries = await Promise.all(
      CONFIG.map(async (c) => {
        try {
          const list = await base44.entities[c.key].list("-created_date", 200);
          let recs = list || [];
          if (c.scoped && activeCompanyId) recs = recs.filter((r) => r.company_id === activeCompanyId);
          return [c.key, recs];
        } catch {
          return [c.key, []];
        }
      })
    );
    setData(Object.fromEntries(entries));
    setLoading(false);
  }, [activeCompanyId]);

  // Recharge quand on ouvre ou quand l'entreprise active change.
  useEffect(() => {
    if (open) loadAll();
  }, [open, loadAll]);

  // Raccourci Cmd/Ctrl+K.
  useEffect(() => {
    const onKey = (e) => {
      if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === "k") {
        e.preventDefault();
        setOpen((o) => !o);
      }
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
  }, []);

  const results = useMemo(() => {
    const query = q.trim().toLowerCase();
    if (!query) return [];
    const out = [];
    for (const c of CONFIG) {
      const recs = data[c.key] || [];
      const matches = [];
      for (const r of recs) {
        const hit = c.fields.some((f) => {
          const v = r[f];
          return v != null && String(v).toLowerCase().includes(query);
        });
        if (hit) {
          matches.push(r);
          if (matches.length >= 6) break;
        }
      }
      if (matches.length) out.push({ config: c, matches });
    }
    return out;
  }, [q, data]);

  const go = (link) => {
    setOpen(false);
    setQ("");
    navigate(link);
  };

  return (
    <>
      <button
        type="button"
        onClick={() => setOpen(true)}
        className="no-select-tap min-touch flex items-center gap-2 w-full sm:w-64 h-9 px-3 rounded-md border border-input
bg-muted/50 text-sm text-muted-foreground hover:bg-muted transition-colors"
      >
        <Search className="w-4 h-4" />
```

```
        <span className="flex-1 text-left truncate">Rechercher…</span>
                < k b d   c l a s s N a m e = " h i d d e n   s m : i n l i n e   t e x t - [ 1 0 p x ]   p x - 1 . 5   p y - 0 . 5
      </button>

      <Dialog open={open} onOpenChange={(o) => { setOpen(o); if (!o) setQ(""); }}>
        <DialogContent className="p-0 max-w-xl gap-0 overflow-hidden">
          <div className="flex items-center gap-2 px-4 border-b border-border">
            <Search className="w-4 h-4 text-muted-foreground shrink-0" />
            <Input
              autoFocus
              value={q}
              onChange={(e) => setQ(e.target.value)}
              placeholder="Tâche, client, facture, produit…"
              className="border-0 shadow-none focus-visible:ring-0 h-12 px-0"
            />
            {loading && <Loader2 className="w-4 h-4 animate-spin text-muted-foreground shrink-0" />}
          </div>
          <div className="max-h-[60vh] overflow-y-auto p-2">
            {!q.trim() ? (
              <p className="text-sm text-muted-foreground px-3 py-8 text-center">
                Saisissez quelques mots pour rechercher dans toutes vos données.
              </p>
            ) : results.length === 0 ? (
              <p className="text-sm text-muted-foreground px-3 py-8 text-center">
                Aucun résultat pour « {q.trim()} ».
              </p>
            ) : (
              results.map((g) => (
                <div key={g.config.key} className="mb-1">
                  <p className="text-[11px] uppercase tracking-wide text-muted-foreground px-3 pt-2 pb-1">
                    {g.config.label}
                  </p>
                  {g.matches.map((r) => (
                    <button
                      key={r.id}
                      type="button"
                      onClick={() => go(g.config.link)}
                      className="no-select-tap w-full text-left px-3 py-2 rounded-lg hover:bg-accent/10 transition-colors"
                    >
                      <span className="text-sm font-medium truncate block">
                        {r[g.config.primary] || "(sans titre)"}
                      </span>
                    </button>
                  ))}
                </div>
              ))
            )}
          </div>
        </DialogContent>
      </Dialog>
    </>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/settings/LegalLinks.jsx
------------------------------------------------------------
import React from "react";
import { Link } from "react-router-dom";
import { Shield, FileText, Info } from "lucide-react";
import { useI18n } from "@/lib/i18n";

export default function LegalLinks() {
  const { t } = useI18n();
  return (
    <div className="bg-card border border-border rounded-2xl p-6 space-y-3">
      <div>
        <h2 className="font-display text-lg flex items-center gap-2"><Shield className="w-4 h-4" /> {t("settings.legal")}</h2>
        <p className="text-sm text-muted-foreground mt-1">JM (JH Manager)</p>
      </div>
      <div className="flex flex-col gap-2 text-sm">
        <Link to="/confidentialite" className="text-primary hover:underline flex items-center gap-2"><FileText className="w-4
h-4" /> {t("settings.privacy")}</Link>
        <Link to="/conditions" className="text-primary hover:underline flex items-center gap-2"><FileText className="w-4 h-4" />
{t("settings.terms")}</Link>
        <Link to="/confidentialite" className="text-muted-foreground hover:underline flex items-center gap-2"><Info
className="w-4 h-4" /> {t("settings.about")}</Link>
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/settings/PaymentCard.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { CreditCard, Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import {
```

```
  Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader, DialogTitle,
} from "@/components/ui/dialog";
import { useToast } from "@/components/ui/use-toast";
import { base44 } from "@/api/base44Client";

export default function PaymentCard() {
  const [open, setOpen] = useState(false);
  const [amount, setAmount] = useState("");
  const [phoneNumber, setPhoneNumber] = useState("");
  const [provider, setProvider] = useState("airtel");
  const [submitting, setSubmitting] = useState(false);
  const { toast } = useToast();

  const handlePay = async (e) => {
    e.preventDefault();
    const amt = parseFloat(amount);
    if (!amt || amt <= 0) {
      toast({ title: "Montant invalide", variant: "destructive" });
      return;
    }
    if (!phoneNumber.trim()) {
      toast({ title: "Numéro de téléphone requis", variant: "destructive" });
      return;
    }
    setSubmitting(true);
    try {
      const res = await base44.functions.invoke('processPayment', {
        amount: amt,
        phoneNumber: phoneNumber.trim(),
        provider,
      });
      if (res?.success || res?.data) {
        toast({
          title: "Paiement initié",
          description: res?.data?.message || "Votre demande de paiement a été envoyée.",
        });
        setOpen(false);
        setAmount("");
        setPhoneNumber("");
      } else {
        toast({
          title: "Échec du paiement",
          description: res?.error || "Erreur inconnue",
          variant: "destructive",
        });
      }
    } catch (err) {
      const body = err && err.response && err.response.data;
      toast({
        title: "Échec du paiement",
        description: (body && body.error) || err?.message || "Erreur réseau",
        variant: "destructive",
      });
    }
    setSubmitting(false);
  };

  return (
    <div className="bg-card border border-border rounded-2xl p-6 space-y-3">
      <div className="flex items-start gap-3">
        <div className="w-10 h-10 rounded-xl bg-accent/15 text-accent flex items-center justify-center shrink-0">
          <CreditCard className="w-5 h-5" />
        </div>
        <div>
          <h2 className="font-display text-lg">Paiement Mobile Money</h2>
          <p className="text-sm text-muted-foreground mt-0.5">
            Rechargez votre compte via Airtel Money ou Moov Money.
          </p>
        </div>
      </div>
      <Button className="w-full" onClick={() => setOpen(true)}>
        <CreditCard className="w-4 h-4" /> Payer
      </Button>

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>Paiement Mobile Money</DialogTitle>
            <DialogDescription>
              Renseignez les détails pour initier le paiement.
            </DialogDescription>
          </DialogHeader>
          <form onSubmit={handlePay} className="space-y-4">
            <div className="space-y-2">
              <Label htmlFor="pay-amount">Montant (XAF)</Label>
              <Input
                id="pay-amount"
                type="number"
                min="1"
                step="1"
                value={amount}
                onChange={(e) => setAmount(e.target.value)}
```

```
              placeholder="1000"
            />
          </div>
          <div className="space-y-2">
            <Label htmlFor="pay-phone">Numéro de téléphone</Label>
            <Input
              id="pay-phone"
              type="tel"
              value={phoneNumber}
              onChange={(e) => setPhoneNumber(e.target.value)}
              placeholder="06 00 00 00 00"
            />
          </div>
          <div className="space-y-2">
            <Label>Opérateur</Label>
            <Select value={provider} onValueChange={setProvider}>
              <SelectTrigger><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="airtel">Airtel Money</SelectItem>
                <SelectItem value="moov">Moov Money</SelectItem>
              </SelectContent>
            </Select>
          </div>
          <DialogFooter>
            <Button type="button" variant="outline" onClick={() => setOpen(false)} disabled={submitting}>
              Annuler
            </Button>
            <Button type="submit" disabled={submitting}>
              {submitting ? (
                <>
                  <Loader2 className="w-4 h-4 animate-spin" /> Traitement…
                </>
              ) : (
                "Payer"
              )}
            </Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  </div>
 );
}

------------------------------------------------------------
FICHIER : ./src/components/tasks/TaskForm.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Checkbox } from "@/components/ui/checkbox";
import { useI18n } from "@/lib/i18n";
import CategorySelect from "@/components/categories/CategorySelect";
import { useSpace } from "@/lib/SpaceContext";

function toLocalInput(value) {
  if (!value) return "";
  const d = new Date(value);
  if (isNaN(d.getTime())) return "";
  const pad = (n) => String(n).padStart(2, "0");
  return `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}T${pad(d.getHours())}:${pad(d.getMinutes())}`;
}

export default function TaskForm({ open, onOpenChange, task, onSubmit }) {
  const { t } = useI18n();
  const { space } = useSpace();
  const [title, setTitle] = useState("");
  const [category, setCategory] = useState("");
  const [start, setStart] = useState("");
  const [end, setEnd] = useState("");
  const [priority, setPriority] = useState("normale");
  const [status, setStatus] = useState("a_faire");
  const [repeat, setRepeat] = useState(false);
  const [frequency, setFrequency] = useState("mois");
  const [repeatCount, setRepeatCount] = useState("1");
  const [error, setError] = useState("");

  useEffect(() => {
    if (open) {
      setTitle(task?.title || "");
      setStart(toLocalInput(task?.start) || toLocalInput(new Date().toISOString()));
      setEnd(toLocalInput(task?.end) || toLocalInput(new Date(Date.now() + 60 * 60 * 1000).toISOString()));
      setPriority(task?.priority || "normale");
      setStatus(task?.status || "a_faire");
      setCategory(task?.category || "");
      setRepeat(false);
      setFrequency("mois");
      setRepeatCount("1");
      setError("");
```

```
    }
  }, [open, task]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title.trim()) {
      setError(t("taskform.err.title"));
      return;
    }
    const startDate = new Date(start);
    const endDate = new Date(end);
    if (endDate.isNaN || isNaN(endDate.getTime()) || isNaN(startDate.getTime())) {
      setError(t("taskform.err.dates"));
      return;
    }
    if (endDate < startDate) {
      setError(t("taskform.err.endBeforeStart"));
      return;
    }
    const duration_minutes = Math.round((endDate.getTime() - startDate.getTime()) / 60000);
    onSubmit({
      title: title.trim(),
      start: startDate.toISOString(),
      end: endDate.toISOString(),
      duration_minutes,
      priority,
      status,
      category: category.trim(),
    }, repeat && !task && parseInt(repeatCount, 10) > 1 ? { frequency, count: parseInt(repeatCount, 10) } : null);
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display">{task ? t("taskform.edit") : t("taskform.new")}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          {error && <p className="text-sm text-destructive">{error}</p>}
          <div className="space-y-2">
            <Label htmlFor="title">{t("taskform.name")}</Label>
            <Input id="title" value={title} onChange={(e) => setTitle(e.target.value)}
placeholder={t("taskform.namePlaceholder")} autoFocus />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label htmlFor="start">{t("taskform.start")}</Label>
              <Input id="start" type="datetime-local" value={start} onChange={(e) => setStart(e.target.value)} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="end">{t("taskform.end")}</Label>
              <Input id="end" type="datetime-local" value={end} onChange={(e) => setEnd(e.target.value)} />
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label>{t("taskform.priority")}</Label>
              <Select value={priority} onValueChange={setPriority}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  <SelectItem value="basse">{t("priority.basse")}</SelectItem>
                  <SelectItem value="normale">{t("priority.normale")}</SelectItem>
                  <SelectItem value="haute">{t("priority.haute")}</SelectItem>
                </SelectContent>
              </Select>
            </div>
            <div className="space-y-2">
              <Label>{t("taskform.status")}</Label>
              <Select value={status} onValueChange={setStatus}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>
                  <SelectItem value="a_faire">{t("tasks.filter.todo")}</SelectItem>
                  <SelectItem value="en_cours">{t("tasks.filter.progress")}</SelectItem>
                  <SelectItem value="terminee">{t("tasks.filter.done")}</SelectItem>
                </SelectContent>
              </Select>
            </div>
          </div>
          <div className="space-y-2">
            <Label>Catégorie</Label>
            <CategorySelect scope={space} domain="tache" value={category} onChange={setCategory} placeholder="Catégorie" />
          </div>
          {!task && (
            <div className="space-y-3 rounded-xl border border-border p-3">
              <div className="flex items-center gap-2">
                <Checkbox checked={repeat} onCheckedChange={setRepeat} id="task-repeat" />
                <Label htmlFor="task-repeat">Répéter périodiquement</Label>
              </div>
              {repeat && (
                <div className="grid grid-cols-2 gap-3">
                  <div className="space-y-1.5">
                    <Label htmlFor="task-freq" className="text-xs">Fréquence</Label>
```

```
                    <Select value={frequency} onValueChange={setFrequency}>
                      <SelectTrigger id="task-freq"><SelectValue /></SelectTrigger>
                      <SelectContent>
                        <SelectItem value="jour">Jour</SelectItem>
                        <SelectItem value="semaine">Semaine</SelectItem>
                        <SelectItem value="trimestre">Trimestre</SelectItem>
                        <SelectItem value="mois">Mois</SelectItem>
                        <SelectItem value="annee">Année</SelectItem>
                      </SelectContent>
                    </Select>
                  </div>
                  <div className="space-y-1.5">
                    <Label htmlFor="task-count" className="text-xs">Nombre d'occurrences</Label>
                    <Input id="task-count" type="number" min="1" max="120" value={repeatCount} onChange={(e) =>
setRepeatCount(e.target.value)} />
                  </div>
                </div>
              )}
            </div>
          )}
          <DialogFooter>
            <Button type="submit" className="w-full">{task ? t("common.save") : t("taskform.add")}</Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/tasks/TaskItem.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Pencil, Trash2, CheckCircle2, Circle } from "lucide-react";
import { Checkbox } from "@/components/ui/checkbox";
import { formatDateFr } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
  AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

const PRIORITY_DOT = {
  basse: "bg-muted-foreground/40",
  normale: "bg-accent",
  haute: "bg-destructive",
};

export default function TaskItem({ task, onEdit, onDelete, onToggleDone, conflict, selectable, selected, onToggleSelect }) {
  const { t } = useI18n();
  const done = task.status === "terminee";
  const [confirmOpen, setConfirmOpen] = useState(false);

  return (
    <>
      <div className={`flex items-center gap-3 p-4 rounded-2xl border border-border bg-card ${done ? "opacity-60" : ""}`}>
        {selectable && (
          <Checkbox
            checked={!!selected}
            onCheckedChange={() => onToggleSelect(task.id)}
            aria-label="Sélectionner"
            className="shrink-0"
          />
        )}
        <button
          onClick={() => onToggleDone(task)}
          aria-label={t("taskitem.toggleAria")}
          className="no-select-tap min-touch shrink-0 flex items-center justify-center rounded-full text-muted-foreground
hover:text-primary transition-colors"
        >
          {done ? <CheckCircle2 className="w-5 h-5 text-primary" /> : <Circle className="w-5 h-5" />}
        </button>
        <div className="min-w-0 flex-1">
          <div className="flex items-center gap-2">
            <p className={`text-sm font-medium truncate ${done ? "line-through" : ""}`}>{task.title}</p>
            <span className={`w-1.5 h-1.5 rounded-full shrink-0 ${PRIORITY_DOT[task.priority] || PRIORITY_DOT.normale}`}
title={t("priority." + (task.priority || "normale"))} />
          </div>
          <p className="text-sm text-muted-foreground mt-0.5">
                        {formatDateFr(task.start, true)} !' {formatDateFr(task.e
            {conflict && <span className="text-destructive ml-2">{t("taskitem.conflict")}</span>}
          </p>
        </div>
        <div className="flex items-center gap-1 shrink-0">
          <button
            onClick={() => onEdit(task)}
            aria-label={t("taskitem.editAria")}
            className="no-select-tap min-touch p-2.5 rounded-lg hover:bg-secondary text-muted-foreground hover:text-foreground
transition-colors"
          >
            <Pencil className="w-4 h-4" />
```

```
          </button>
          <button
            onClick={() => setConfirmOpen(true)}
            aria-label={t("taskitem.deleteAria")}
            className="no-select-tap min-touch p-2.5 rounded-lg hover:bg-secondary text-muted-foreground hover:text-destructive
transition-colors"
          >
            <Trash2 className="w-4 h-4" />
          </button>
        </div>
      </div>

      <AlertDialog open={confirmOpen} onOpenChange={setConfirmOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>{t("taskitem.deleteTitle")}</AlertDialogTitle>
            <AlertDialogDescription>{t("taskitem.deleteDesc", { title: task.title })}</AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>{t("common.cancel")}</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
              onClick={() => onDelete(task)}
            >
              {t("common.delete")}
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/ui/accordion.jsx
------------------------------------------------------------
import * as React from "react"
import * as AccordionPrimitive from "@radix-ui/react-accordion"
import { ChevronDown } from "lucide-react"

import { cn } from "@/lib/utils"

const Accordion = AccordionPrimitive.Root

const AccordionItem = React.forwardRef(({ className, ...props }, ref) => (
  <AccordionPrimitive.Item ref={ref} className={cn("border-b", className)} {...props} />
))
AccordionItem.displayName = "AccordionItem"

const AccordionTrigger = React.forwardRef(({ className, children, ...props }, ref) => (
  <AccordionPrimitive.Header className="flex">
    <AccordionPrimitive.Trigger
      ref={ref}
      className={cn(
        "flex flex-1 items-center justify-between py-4 text-sm font-medium transition-all hover:underline text-left
[&[data-state=open]>svg]:rotate-180",
        className
      )}
      {...props}>
      {children}
      <ChevronDown
        className="h-4 w-4 shrink-0 text-muted-foreground transition-transform duration-200" />
    </AccordionPrimitive.Trigger>
  </AccordionPrimitive.Header>
))
AccordionTrigger.displayName = AccordionPrimitive.Trigger.displayName

const AccordionContent = React.forwardRef(({ className, children, ...props }, ref) => (
  <AccordionPrimitive.Content
    ref={ref}
    className="overflow-hidden text-sm data-[state=closed]:animate-accordion-up data-[state=open]:animate-accordion-down"
    {...props}>
    <div className={cn("pb-4 pt-0", className)}>{children}</div>
  </AccordionPrimitive.Content>
))
AccordionContent.displayName = AccordionPrimitive.Content.displayName

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent }


------------------------------------------------------------
FICHIER : ./src/components/ui/alert-dialog.jsx
------------------------------------------------------------
import * as React from "react"
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog"

import { cn } from "@/lib/utils"
import { buttonVariants } from "@/components/ui/button"

const AlertDialog = AlertDialogPrimitive.Root

const AlertDialogTrigger = AlertDialogPrimitive.Trigger
```

```
const AlertDialogPortal = AlertDialogPrimitive.Portal

const AlertDialogOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
      className
    )}
    {...props}
    ref={ref} />
))
AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName

const AlertDialogContent = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPortal>
    <AlertDialogOverlay />
    <AlertDialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-background
p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2
data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className
      )}
      {...props} />
  </AlertDialogPortal>
))
AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName

const AlertDialogHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col space-y-2 text-center sm:text-left", className)}
    {...props} />
)
AlertDialogHeader.displayName = "AlertDialogHeader"

const AlertDialogFooter = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
    {...props} />
)
AlertDialogFooter.displayName = "AlertDialogFooter"

const AlertDialogTitle = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Title ref={ref} className={cn("text-lg font-semibold", className)} {...props} />
))
AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName

const AlertDialogDescription = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
AlertDialogDescription.displayName =
  AlertDialogPrimitive.Description.displayName

const AlertDialogAction = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Action ref={ref} className={cn(buttonVariants(), className)} {...props} />
))
AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName

const AlertDialogCancel = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Cancel
    ref={ref}
    className={cn(buttonVariants({ variant: "outline" }), "mt-2 sm:mt-0", className)}
    {...props} />
))
AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,
  AlertDialogCancel,
```

```
}


------------------------------------------------------------
FICHIER : ./src/components/ui/alert.jsx
------------------------------------------------------------
import * as React from "react"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const alertVariants = cva(
  "relative w-full rounded-lg border px-4 py-3 text-sm [&>svg+div]:translate-y-[-3px] [&>svg]:absolute [&>svg]:left-4
[&>svg]:top-4 [&>svg]:text-foreground [&>svg~*]:pl-7",
  {
    variants: {
      variant: {
        default: "bg-background text-foreground",
        destructive:
          "border-destructive/50 text-destructive dark:border-destructive [&>svg]:text-destructive",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

const Alert = React.forwardRef(({ className, variant, ...props }, ref) => (
  <div
    ref={ref}
    role="alert"
    className={cn(alertVariants({ variant }), className)}
    {...props} />
))
Alert.displayName = "Alert"

const AlertTitle = React.forwardRef(({ className, ...props }, ref) => (
  <h5
    ref={ref}
    className={cn("mb-1 font-medium leading-none tracking-tight", className)}
    {...props} />
))
AlertTitle.displayName = "AlertTitle"

const AlertDescription = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("text-sm [&_p]:leading-relaxed", className)}
    {...props} />
))
AlertDescription.displayName = "AlertDescription"

export { Alert, AlertTitle, AlertDescription }


------------------------------------------------------------
FICHIER : ./src/components/ui/aspect-ratio.jsx
------------------------------------------------------------
import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio"

const AspectRatio = AspectRatioPrimitive.Root

export { AspectRatio }


------------------------------------------------------------
FICHIER : ./src/components/ui/avatar.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import * as AvatarPrimitive from "@radix-ui/react-avatar"

import { cn } from "@/lib/utils"

const Avatar = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Root
    ref={ref}
    className={cn("relative flex h-10 w-10 shrink-0 overflow-hidden rounded-full", className)}
    {...props} />
))
Avatar.displayName = AvatarPrimitive.Root.displayName

const AvatarImage = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Image
    ref={ref}
    className={cn("aspect-square h-full w-full", className)}
    {...props} />
))
AvatarImage.displayName = AvatarPrimitive.Image.displayName
```

```
const AvatarFallback = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Fallback
    ref={ref}
    className={cn(
      "flex h-full w-full items-center justify-center rounded-full bg-muted",
      className
    )}
    {...props} />
))
AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName

export { Avatar, AvatarImage, AvatarFallback }


------------------------------------------------------------
FICHIER : ./src/components/ui/badge.jsx
------------------------------------------------------------
import * as React from "react"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const badgeVariants = cva(
  "inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs font-semibold transition-colors focus:outline-none
focus:ring-2 focus:ring-ring focus:ring-offset-2",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground shadow hover:bg-primary/80",
        secondary:
          "border-transparent bg-secondary text-secondary-foreground hover:bg-secondary/80",
        destructive:
          "border-transparent bg-destructive text-destructive-foreground shadow hover:bg-destructive/80",
        outline: "text-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

function Badge({
  className,
  variant,
  ...props
}) {
  return (<div className={cn(badgeVariants({ variant }), className)} {...props} />);
}

export { Badge, badgeVariants }


------------------------------------------------------------
FICHIER : ./src/components/ui/breadcrumb.jsx
------------------------------------------------------------
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { ChevronRight, MoreHorizontal } from "lucide-react"

import { cn } from "@/lib/utils"

const Breadcrumb = React.forwardRef(
  ({ ...props }, ref) => <nav ref={ref} aria-label="breadcrumb" {...props} />
)
Breadcrumb.displayName = "Breadcrumb"

const BreadcrumbList = React.forwardRef(({ className, ...props }, ref) => (
  <ol
    ref={ref}
    className={cn(
      "flex flex-wrap items-center gap-1.5 break-words text-sm text-muted-foreground sm:gap-2.5",
      className
    )}
    {...props} />
))
BreadcrumbList.displayName = "BreadcrumbList"

const BreadcrumbItem = React.forwardRef(({ className, ...props }, ref) => (
  <li
    ref={ref}
    className={cn("inline-flex items-center gap-1.5", className)}
    {...props} />
))
BreadcrumbItem.displayName = "BreadcrumbItem"

const BreadcrumbLink = React.forwardRef(({ asChild, className, ...props }, ref) => {
  const Comp = asChild ? Slot : "a"

  return (
    (<Comp
```

```
      ref={ref}
      className={cn("transition-colors hover:text-foreground", className)}
      {...props} />)
  );
})
BreadcrumbLink.displayName = "BreadcrumbLink"

const BreadcrumbPage = React.forwardRef(({ className, ...props }, ref) => (
  <span
    ref={ref}
    role="link"
    aria-disabled="true"
    aria-current="page"
    className={cn("font-normal text-foreground", className)}
    {...props} />
))
BreadcrumbPage.displayName = "BreadcrumbPage"

const BreadcrumbSeparator = ({
  children,
  className,
  ...props
}) => (
  <li
    role="presentation"
    aria-hidden="true"
    className={cn("[&>svg]:w-3.5 [&>svg]:h-3.5", className)}
    {...props}>
    {children ?? <ChevronRight />}
  </li>
)
BreadcrumbSeparator.displayName = "BreadcrumbSeparator"

const BreadcrumbEllipsis = ({
  className,
  ...props
}) => (
  <span
    role="presentation"
    aria-hidden="true"
    className={cn("flex h-9 w-9 items-center justify-center", className)}
    {...props}>
    <MoreHorizontal className="h-4 w-4" />
    <span className="sr-only">More</span>
  </span>
)
BreadcrumbEllipsis.displayName = "BreadcrumbElipssis"

export {
  Breadcrumb,
  BreadcrumbList,
  BreadcrumbItem,
  BreadcrumbLink,
  BreadcrumbPage,
  BreadcrumbSeparator,
  BreadcrumbEllipsis,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/button.jsx
------------------------------------------------------------
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const buttonVariants = cva(
  "no-select-tap inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium
transition-colors focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none
disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default:
          "bg-primary text-primary-foreground shadow hover:bg-primary/90",
        destructive:
          "bg-destructive text-destructive-foreground shadow-sm hover:bg-destructive/90",
        outline:
          "border border-input bg-transparent shadow-sm hover:bg-accent hover:text-accent-foreground",
        secondary:
          "bg-secondary text-secondary-foreground shadow-sm hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-9 px-4 py-2",
        sm: "h-8 rounded-md px-3 text-xs",
        lg: "h-10 rounded-md px-8",
        icon: "h-9 w-9",
      },
```

```
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

const Button = React.forwardRef(({ className, variant, size, asChild = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "button"
  return (
    (<Comp
      className={cn(buttonVariants({ variant, size, className }))}
      ref={ref}
      {...props} />)
  );
})
Button.displayName = "Button"

export { Button, buttonVariants }

------------------------------------------------------------
FICHIER : ./src/components/ui/calendar.jsx
------------------------------------------------------------
import * as React from "react"
import { ChevronLeft, ChevronRight } from "lucide-react"
import { DayPicker } from "react-day-picker"

import { cn } from "@/lib/utils"
import { buttonVariants } from "@/components/ui/button"

function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  ...props
}) {
  return (
    (<DayPicker
      showOutsideDays={showOutsideDays}
      className={cn("p-3", className)}
      classNames={{
        months: "flex flex-col sm:flex-row space-y-4 sm:space-x-4 sm:space-y-0",
        month: "space-y-4",
        caption: "flex justify-center pt-1 relative items-center",
        caption_label: "text-sm font-medium",
        nav: "space-x-1 flex items-center",
        nav_button: cn(
          buttonVariants({ variant: "outline" }),
          "h-7 w-7 bg-transparent p-0 opacity-50 hover:opacity-100"
        ),
        nav_button_previous: "absolute left-1",
        nav_button_next: "absolute right-1",
        table: "w-full border-collapse space-y-1",
        head_row: "flex",
        head_cell:
          "text-muted-foreground rounded-md w-8 font-normal text-[0.8rem]",
        row: "flex w-full mt-2",
        cell: cn(
          "relative p-0 text-center text-sm focus-within:relative focus-within:z-20 [&:has([aria-selected])]:bg-accent
[&:has([aria-selected].day-outside)]:bg-accent/50 [&:has([aria-selected].day-range-end)]:rounded-r-md",
          props.mode === "range"
            ? "[&:has(>.day-range-end)]:rounded-r-md [&:has(>.day-range-start)]:rounded-l-md
first:[&:has([aria-selected])]:rounded-l-md last:[&:has([aria-selected])]:rounded-r-md"
            : "[&:has([aria-selected])]:rounded-md"
        ),
        day: cn(
          buttonVariants({ variant: "ghost" }),
          "h-8 w-8 p-0 font-normal aria-selected:opacity-100"
        ),
        day_range_start: "day-range-start",
        day_range_end: "day-range-end",
        day_selected:
          "bg-primary text-primary-foreground hover:bg-primary hover:text-primary-foreground focus:bg-primary
focus:text-primary-foreground",
        day_today: "bg-accent text-accent-foreground",
        day_outside:
          "day-outside text-muted-foreground aria-selected:bg-accent/50 aria-selected:text-muted-foreground",
        day_disabled: "text-muted-foreground opacity-50",
        day_range_middle:
          "aria-selected:bg-accent aria-selected:text-accent-foreground",
        day_hidden: "invisible",
        ...classNames,
      }}
      components={{
        IconLeft: ({ className, ...props }) => (
          <ChevronLeft className={cn("h-4 w-4", className)} {...props} />
        ),
        IconRight: ({ className, ...props }) => (
          <ChevronRight className={cn("h-4 w-4", className)} {...props} />
        ),
      }}
```

```
      {...props} />)
  );
}
Calendar.displayName = "Calendar"

export { Calendar }


------------------------------------------------------------
FICHIER : ./src/components/ui/card.jsx
------------------------------------------------------------
import * as React from "react"

import { cn } from "@/lib/utils"

const Card = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("rounded-xl border bg-card text-card-foreground shadow", className)}
    {...props} />
))
Card.displayName = "Card"

const CardHeader = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("flex flex-col space-y-1.5 p-6", className)}
    {...props} />
))
CardHeader.displayName = "CardHeader"

const CardTitle = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("font-semibold leading-none tracking-tight", className)}
    {...props} />
))
CardTitle.displayName = "CardTitle"

const CardDescription = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
CardDescription.displayName = "CardDescription"

const CardContent = React.forwardRef(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
))
CardContent.displayName = "CardContent"

const CardFooter = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("flex items-center p-6 pt-0", className)}
    {...props} />
))
CardFooter.displayName = "CardFooter"

export { Card, CardHeader, CardFooter, CardTitle, CardDescription, CardContent }


------------------------------------------------------------
FICHIER : ./src/components/ui/carousel.jsx
------------------------------------------------------------
import * as React from "react"
import useEmblaCarousel from "embla-carousel-react";
import { ArrowLeft, ArrowRight } from "lucide-react"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"

const CarouselContext = React.createContext(null)

function useCarousel() {
  const context = React.useContext(CarouselContext)

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />")
  }

  return context
}

const Carousel = React.forwardRef((
  {
    orientation = "horizontal",
    opts,
    setApi,
    plugins,
    className,
```

```
    children,
    ...props
  },
  ref
) => {
  const [carouselRef, api] = useEmblaCarousel({
    ...opts,
    axis: orientation === "horizontal" ? "x" : "y",
  }, plugins)
  const [canScrollPrev, setCanScrollPrev] = React.useState(false)
  const [canScrollNext, setCanScrollNext] = React.useState(false)

  const onSelect = React.useCallback((api) => {
    if (!api) {
      return
    }

    setCanScrollPrev(api.canScrollPrev())
    setCanScrollNext(api.canScrollNext())
  }, [])

  const scrollPrev = React.useCallback(() => {
    api?.scrollPrev()
  }, [api])

  const scrollNext = React.useCallback(() => {
    api?.scrollNext()
  }, [api])

  const handleKeyDown = React.useCallback((event) => {
    if (event.key === "ArrowLeft") {
      event.preventDefault()
      scrollPrev()
    } else if (event.key === "ArrowRight") {
      event.preventDefault()
      scrollNext()
    }
  }, [scrollPrev, scrollNext])

  React.useEffect(() => {
    if (!api || !setApi) {
      return
    }

    setApi(api)
  }, [api, setApi])

  React.useEffect(() => {
    if (!api) {
      return
    }

    onSelect(api)
    api.on("reInit", onSelect)
    api.on("select", onSelect)

    return () => {
      api?.off("select", onSelect)
    };
  }, [api, onSelect])

  return (
    (<CarouselContext.Provider
      value={{
        carouselRef,
        api: api,
        opts,
        orientation:
          orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
        scrollPrev,
        scrollNext,
        canScrollPrev,
        canScrollNext,
      }}>
      <div
        ref={ref}
        onKeyDownCapture={handleKeyDown}
        className={cn("relative", className)}
        role="region"
        aria-roledescription="carousel"
        {...props}>
        {children}
      </div>
    </CarouselContext.Provider>)
  );
})
Carousel.displayName = "Carousel"

const CarouselContent = React.forwardRef(({ className, ...props }, ref) => {
  const { carouselRef, orientation } = useCarousel()

  return (
```

```
    (<div ref={carouselRef} className="overflow-hidden">
      <div
        ref={ref}
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className
        )}
        {...props} />
    </div>)
  );
})
CarouselContent.displayName = "CarouselContent"

const CarouselItem = React.forwardRef(({ className, ...props }, ref) => {
  const { orientation } = useCarousel()

  return (
    (<div
      ref={ref}
      role="group"
      aria-roledescription="slide"
      className={cn(
        "min-w-0 shrink-0 grow-0 basis-full",
        orientation === "horizontal" ? "pl-4" : "pt-4",
        className
      )}
      {...props} />)
  );
})
CarouselItem.displayName = "CarouselItem"

const CarouselPrevious = React.forwardRef(({ className, variant = "outline", size = "icon", ...props }, ref) => {
  const { orientation, scrollPrev, canScrollPrev } = useCarousel()

  return (
    (<Button
      ref={ref}
      variant={variant}
      size={size}
      className={cn("absolute  h-8 w-8 rounded-full", orientation === "horizontal"
        ? "-left-12 top-1/2 -translate-y-1/2"
        : "-top-12 left-1/2 -translate-x-1/2 rotate-90", className)}
      disabled={!canScrollPrev}
      onClick={scrollPrev}
      {...props}>
      <ArrowLeft className="h-4 w-4" />
      <span className="sr-only">Previous slide</span>
    </Button>)
  );
})
CarouselPrevious.displayName = "CarouselPrevious"

const CarouselNext = React.forwardRef(({ className, variant = "outline", size = "icon", ...props }, ref) => {
  const { orientation, scrollNext, canScrollNext } = useCarousel()

  return (
    (<Button
      ref={ref}
      variant={variant}
      size={size}
      className={cn("absolute h-8 w-8 rounded-full", orientation === "horizontal"
        ? "-right-12 top-1/2 -translate-y-1/2"
        : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90", className)}
      disabled={!canScrollNext}
      onClick={scrollNext}
      {...props}>
      <ArrowRight className="h-4 w-4" />
      <span className="sr-only">Next slide</span>
    </Button>)
  );
})
CarouselNext.displayName = "CarouselNext"

export { Carousel, CarouselContent, CarouselItem, CarouselPrevious, CarouselNext };


------------------------------------------------------------
FICHIER : ./src/components/ui/chart.jsx
------------------------------------------------------------
"use client";
import * as React from "react"
import * as RechartsPrimitive from "recharts"

import { cn } from "@/lib/utils"

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = {
  light: "",
  dark: ".dark"
}
```

```
const ChartContext = React.createContext(null)

function useChart() {
  const context = React.useContext(ChartContext)

  if (!context) {
    throw new Error("useChart must be used within a <ChartContainer />")
  }

  return context
}

const ChartContainer = React.forwardRef(({ id, className, children, config, ...props }, ref) => {
  const uniqueId = React.useId()
  const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`

  return (
    (<ChartContext.Provider value={{ config }}>
      <div
        data-chart={chartId}
        ref={ref}
        className={cn(
          "flex aspect-video justify-center text-xs [&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground
[&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50 [&_.recharts-curve.recharts-tooltip-cursor]:stroke-border
[&_.recharts-dot[stroke='#fff']]:stroke-transparent [&_.recharts-layer]:outline-none
[&_.recharts-polar-grid_[stroke='#ccc']]:stroke-border [&_.recharts-radial-bar-background-sector]:fill-muted
[&_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted [&_.recharts-reference-line_[stroke='#ccc']]:stroke-border
[&_.recharts-sector[stroke='#fff']]:stroke-transparent [&_.recharts-sector]:outline-none [&_.recharts-surface]:outline-none",
          className
        )}
        {...props}>
        <ChartStyle id={chartId} config={config} />
        <RechartsPrimitive.ResponsiveContainer>
          {children}
        </RechartsPrimitive.ResponsiveContainer>
      </div>
    </ChartContext.Provider>)
  );
})
ChartContainer.displayName = "Chart"

const ChartStyle = ({
  id,
  config
}) => {
  const colorConfig = Object.entries(config).filter(([, config]) => config.theme || config.color)

  if (!colorConfig.length) {
    return null
  }

  return (
    (<style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig
.map(([key, itemConfig]) => {
const color =
  itemConfig.theme?.[theme] ||
  itemConfig.color
return color ? `  --color-${key}: ${color};` : null
})
.join("\n")}
}
`)
          .join("\n"),
      }} />)
  );
}

const ChartTooltip = RechartsPrimitive.Tooltip

const ChartTooltipContent = React.forwardRef((
  {
    active,
    payload,
    className,
    indicator = "dot",
    hideLabel = false,
    hideIndicator = false,
    label,
    labelFormatter,
    labelClassName,
    formatter,
    color,
    nameKey,
    labelKey,
  },
  ref
) => {
```

```
  const { config } = useChart()

  const tooltipLabel = React.useMemo(() => {
    if (hideLabel || !payload?.length) {
      return null
    }

    const [item] = payload
    const key = `${labelKey || item.dataKey || item.name || "value"}`
    const itemConfig = getPayloadConfigFromPayload(config, item, key)
    const value =
      !labelKey && typeof label === "string"
        ? config[label]?.label || label
        : itemConfig?.label

    if (labelFormatter) {
      return (
        (<div className={cn("font-medium", labelClassName)}>
          {labelFormatter(value, payload)}
        </div>)
      );
    }

    if (!value) {
      return null
    }

    return <div className={cn("font-medium", labelClassName)}>{value}</div>;
  }, [
    label,
    labelFormatter,
    payload,
    hideLabel,
    labelClassName,
    config,
    labelKey,
  ])

  if (!active || !payload?.length) {
    return null
  }

  const nestLabel = payload.length === 1 && indicator !== "dot"

  return (
    (<div
      ref={ref}
      className={cn(
        "grid min-w-[8rem] items-start gap-1.5 rounded-lg border border-border/50 bg-background px-2.5 py-1.5 text-xs
shadow-xl",
        className
      )}>
      {!nestLabel ? tooltipLabel : null}
      <div className="grid gap-1.5">
        {payload.map((item, index) => {
          const key = `${nameKey || item.name || item.dataKey || "value"}`
          const itemConfig = getPayloadConfigFromPayload(config, item, key)
          const indicatorColor = color || item.payload.fill || item.color

          return (
            (<div
              key={item.dataKey}
              className={cn(
                "flex w-full flex-wrap items-stretch gap-2 [&>svg]:h-2.5 [&>svg]:w-2.5 [&>svg]:text-muted-foreground",
                indicator === "dot" && "items-center"
              )}>
              {formatter && item?.value !== undefined && item.name ? (
                formatter(item.value, item.name, item, index, item.payload)
              ) : (
                <>
                  {itemConfig?.icon ? (
                    <itemConfig.icon />
                  ) : (
                    !hideIndicator && (
                      <div
                        className={cn("shrink-0 rounded-[2px] border-[--color-border] bg-[--color-bg]", {
                          "h-2.5 w-2.5": indicator === "dot",
                          "w-1": indicator === "line",
                          "w-0 border-[1.5px] border-dashed bg-transparent":
                            indicator === "dashed",
                          "my-0.5": nestLabel && indicator === "dashed",
                        })}
                        style={
                          {
                            "--color-bg": indicatorColor,
                            "--color-border": indicatorColor
                          }
                        } />
                    )
                  )}
                  <div
                    className={cn(
```

```
                    "flex flex-1 justify-between leading-none",
                    nestLabel ? "items-end" : "items-center"
                  )}>
                  <div className="grid gap-1.5">
                    {nestLabel ? tooltipLabel : null}
                    <span className="text-muted-foreground">
                      {itemConfig?.label || item.name}
                    </span>
                  </div>
                  {item.value && (
                    <span className="font-mono font-medium tabular-nums text-foreground">
                      {item.value.toLocaleString()}
                    </span>
                  )}
                </div>
              </>
            )}
          </div>)
        );
      })}
    </div>
  </div>)
);
})
ChartTooltipContent.displayName = "ChartTooltip"

const ChartLegend = RechartsPrimitive.Legend

const ChartLegendContent = React.forwardRef((
  { className, hideIcon = false, payload, verticalAlign = "bottom", nameKey },
  ref
) => {
  const { config } = useChart()

  if (!payload?.length) {
    return null
  }

  return (
    (<div
      ref={ref}
      className={cn(
        "flex items-center justify-center gap-4",
        verticalAlign === "top" ? "pb-3" : "pt-3",
        className
      )}>
      {payload.map((item) => {
        const key = `${nameKey || item.dataKey || "value"}`
        const itemConfig = getPayloadConfigFromPayload(config, item, key)

        return (
          (<div
            key={item.value}
            className={cn(
              "flex items-center gap-1.5 [&>svg]:h-3 [&>svg]:w-3 [&>svg]:text-muted-foreground"
            )}>
            {itemConfig?.icon && !hideIcon ? (
              <itemConfig.icon />
            ) : (
              <div
                className="h-2 w-2 shrink-0 rounded-[2px]"
                style={{
                  backgroundColor: item.color,
                }} />
            )}
            {itemConfig?.label}
          </div>)
        );
      })}
    </div>)
  );
})
ChartLegendContent.displayName = "ChartLegend"

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(
  config,
  payload,
  key
) {
  if (typeof payload !== "object" || payload === null) {
    return undefined
  }

  const payloadPayload =
    "payload" in payload &&
    typeof payload.payload === "object" &&
    payload.payload !== null
      ? payload.payload
      : undefined

  let configLabelKey = key
```

```
  if (
    key in payload &&
    typeof payload[key] === "string"
  ) {
    configLabelKey = payload[key]
  } else if (
    payloadPayload &&
    key in payloadPayload &&
    typeof payloadPayload[key] === "string"
  ) {
    configLabelKey = payloadPayload[key]
  }

  return configLabelKey in config
    ? config[configLabelKey]
    : config[key];
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/checkbox.jsx
------------------------------------------------------------
import * as React from "react"
import * as CheckboxPrimitive from "@radix-ui/react-checkbox"
import { Check } from "lucide-react"

import { cn } from "@/lib/utils"

const Checkbox = React.forwardRef(({ className, ...props }, ref) => (
  <CheckboxPrimitive.Root
    ref={ref}
    className={cn(
      "peer h-4 w-4 shrink-0 rounded-sm border border-primary shadow focus-visible:outline-none focus-visible:ring-1
focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50 data-[state=checked]:bg-primary
data-[state=checked]:text-primary-foreground",
      className
    )}
    {...props}>
    <CheckboxPrimitive.Indicator className={cn("flex items-center justify-center text-current")}>
      <Check className="h-4 w-4" />
    </CheckboxPrimitive.Indicator>
  </CheckboxPrimitive.Root>
))
Checkbox.displayName = CheckboxPrimitive.Root.displayName

export { Checkbox }


------------------------------------------------------------
FICHIER : ./src/components/ui/collapsible.jsx
------------------------------------------------------------
"use client"

import * as CollapsiblePrimitive from "@radix-ui/react-collapsible"

const Collapsible = CollapsiblePrimitive.Root

const CollapsibleTrigger = CollapsiblePrimitive.CollapsibleTrigger

const CollapsibleContent = CollapsiblePrimitive.CollapsibleContent

export { Collapsible, CollapsibleTrigger, CollapsibleContent }


------------------------------------------------------------
FICHIER : ./src/components/ui/command.jsx
------------------------------------------------------------
import * as React from "react"
import { Command as CommandPrimitive } from "cmdk"
import { Search } from "lucide-react"

import { cn } from "@/lib/utils"
import { Dialog, DialogContent } from "@/components/ui/dialog"

const Command = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive
    ref={ref}
    className={cn(
      "flex h-full w-full flex-col overflow-hidden rounded-md bg-popover text-popover-foreground",
      className
    )}
    {...props} />
```

```
))
Command.displayName = CommandPrimitive.displayName

const CommandDialog = ({
  children,
  ...props
}) => {
  return (
    (<Dialog {...props}>
      <DialogContent className="overflow-hidden p-0">
        <Command
          className="[&_[cmdk-group-heading]]:px-2 [&_[cmdk-group-heading]]:font-medium
[&_[cmdk-group-heading]]:text-muted-foreground [&_[cmdk-group]:not([hidden])_~[cmdk-group]]:pt-0 [&_[cmdk-group]]:px-2
[&_[cmdk-input-wrapper]_svg]:h-5 [&_[cmdk-input-wrapper]_svg]:w-5 [&_[cmdk-input]]:h-12 [&_[cmdk-item]]:px-2
[&_[cmdk-item]]:py-3 [&_[cmdk-item]_svg]:h-5 [&_[cmdk-item]_svg]:w-5">
          {children}
        </Command>
      </DialogContent>
    </Dialog>)
  );
}

const CommandInput = React.forwardRef(({ className, ...props }, ref) => (
  <div className="flex items-center border-b px-3" cmdk-input-wrapper="">
    <Search className="mr-2 h-4 w-4 shrink-0 opacity-50" />
    <CommandPrimitive.Input
      ref={ref}
      className={cn(
        "flex h-10 w-full rounded-md bg-transparent py-3 text-sm outline-none placeholder:text-muted-foreground
disabled:cursor-not-allowed disabled:opacity-50",
        className
      )}
      {...props} />
  </div>
))

CommandInput.displayName = CommandPrimitive.Input.displayName

const CommandList = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.List
    ref={ref}
    className={cn("max-h-[300px] overflow-y-auto overflow-x-hidden", className)}
    {...props} />
))

CommandList.displayName = CommandPrimitive.List.displayName

const CommandEmpty = React.forwardRef((props, ref) => (
  <CommandPrimitive.Empty ref={ref} className="py-6 text-center text-sm" {...props} />
))

CommandEmpty.displayName = CommandPrimitive.Empty.displayName

const CommandGroup = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.Group
    ref={ref}
    className={cn(
      "overflow-hidden p-1 text-foreground [&_[cmdk-group-heading]]:px-2 [&_[cmdk-group-heading]]:py-1.5
[&_[cmdk-group-heading]]:text-xs [&_[cmdk-group-heading]]:font-medium [&_[cmdk-group-heading]]:text-muted-foreground",
      className
    )}
    {...props} />
))

CommandGroup.displayName = CommandPrimitive.Group.displayName

const CommandSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.Separator ref={ref} className={cn("-mx-1 h-px bg-border", className)} {...props} />
))
CommandSeparator.displayName = CommandPrimitive.Separator.displayName

const CommandItem = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default gap-2 select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none
data-[disabled=true]:pointer-events-none data-[selected=true]:bg-accent data-[selected=true]:text-accent-foreground
data-[disabled=true]:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
      className
    )}
    {...props} />
))

CommandItem.displayName = CommandPrimitive.Item.displayName

const CommandShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
```

```
      {...props} />)
  );
}
CommandShortcut.displayName = "CommandShortcut"

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/context-menu.jsx
------------------------------------------------------------
import * as React from "react"
import * as ContextMenuPrimitive from "@radix-ui/react-context-menu"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const ContextMenu = ContextMenuPrimitive.Root

const ContextMenuTrigger = ContextMenuPrimitive.Trigger

const ContextMenuGroup = ContextMenuPrimitive.Group

const ContextMenuPortal = ContextMenuPrimitive.Portal

const ContextMenuSub = ContextMenuPrimitive.Sub

const ContextMenuRadioGroup = ContextMenuPrimitive.RadioGroup

const ContextMenuSubTrigger = React.forwardRef(({ className, inset, children, ...props }, ref) => (
  <ContextMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
      inset && "pl-8",
      className
    )}
    {...props}>
    {children}
    <ChevronRight className="ml-auto h-4 w-4" />
  </ContextMenuPrimitive.SubTrigger>
))
ContextMenuSubTrigger.displayName = ContextMenuPrimitive.SubTrigger.displayName

const ContextMenuSubContent = React.forwardRef(({ className, ...props }, ref) => (
  <ContextMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg
data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props} />
))
ContextMenuSubContent.displayName = ContextMenuPrimitive.SubContent.displayName

const ContextMenuContent = React.forwardRef(({ className, ...props }, ref) => (
  <ContextMenuPrimitive.Portal>
    <ContextMenuPrimitive.Content
      ref={ref}
      className={cn(
        "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md
data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </ContextMenuPrimitive.Portal>
))
ContextMenuContent.displayName = ContextMenuPrimitive.Content.displayName

const ContextMenuItem = React.forwardRef(({ className, inset, ...props }, ref) => (
  <ContextMenuPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      inset && "pl-8",
```

```
      className
    )}
    {...props} />
))
ContextMenuItem.displayName = ContextMenuPrimitive.Item.displayName

const ContextMenuCheckboxItem = React.forwardRef(({ className, children, checked, ...props }, ref) => (
  <ContextMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    checked={checked}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <ContextMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </ContextMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </ContextMenuPrimitive.CheckboxItem>
))
ContextMenuCheckboxItem.displayName =
  ContextMenuPrimitive.CheckboxItem.displayName

const ContextMenuRadioItem = React.forwardRef(({ className, children, ...props }, ref) => (
  <ContextMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <ContextMenuPrimitive.ItemIndicator>
        <Circle className="h-4 w-4 fill-current" />
      </ContextMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </ContextMenuPrimitive.RadioItem>
))
ContextMenuRadioItem.displayName = ContextMenuPrimitive.RadioItem.displayName

const ContextMenuLabel = React.forwardRef(({ className, inset, ...props }, ref) => (
  <ContextMenuPrimitive.Label
    ref={ref}
    className={cn(
      "px-2 py-1.5 text-sm font-semibold text-foreground",
      inset && "pl-8",
      className
    )}
    {...props} />
))
ContextMenuLabel.displayName = ContextMenuPrimitive.Label.displayName

const ContextMenuSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <ContextMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-border", className)}
    {...props} />
))
ContextMenuSeparator.displayName = ContextMenuPrimitive.Separator.displayName

const ContextMenuShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
      {...props} />)
  );
}
ContextMenuShortcut.displayName = "ContextMenuShortcut"

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
```

```
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/dialog.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import * as DialogPrimitive from "@radix-ui/react-dialog"
import { X } from "lucide-react"

import { cn } from "@/lib/utils"

const Dialog = DialogPrimitive.Root

const DialogTrigger = DialogPrimitive.Trigger

const DialogPortal = DialogPrimitive.Portal

const DialogClose = DialogPrimitive.Close

const DialogOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      "fixed inset-0 z-50 bg-black/80  data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
      className
    )}
    {...props} />
))
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName

const DialogContent = React.forwardRef(({ className, children, ...props }, ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-background
p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2
data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className
      )}
      {...props}>
      {children}
      <DialogPrimitive.Close
        className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background transition-opacity hover:opacity-100
focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none data-[state=open]:bg-accent
data-[state=open]:text-muted-foreground">
        <X className="h-4 w-4" />
        <span className="sr-only">Close</span>
      </DialogPrimitive.Close>
    </DialogPrimitive.Content>
  </DialogPortal>
))
DialogContent.displayName = DialogPrimitive.Content.displayName

const DialogHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col space-y-1.5 text-center sm:text-left", className)}
    {...props} />
)
DialogHeader.displayName = "DialogHeader"

const DialogFooter = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
    {...props} />
)
DialogFooter.displayName = "DialogFooter"

const DialogTitle = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold leading-none tracking-tight", className)}
    {...props} />
))
DialogTitle.displayName = DialogPrimitive.Title.displayName

const DialogDescription = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
DialogDescription.displayName = DialogPrimitive.Description.displayName

export {
  Dialog,
  DialogPortal,
  DialogOverlay,
  DialogTrigger,
  DialogClose,
  DialogContent,
  DialogHeader,
  DialogFooter,
  DialogTitle,
  DialogDescription,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/drawer.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import { Drawer as DrawerPrimitive } from "vaul"

import { cn } from "@/lib/utils"

const Drawer = ({
  shouldScaleBackground = true,
  ...props
}) => (
  <DrawerPrimitive.Root shouldScaleBackground={shouldScaleBackground} {...props} />
)
Drawer.displayName = "Drawer"

const DrawerTrigger = DrawerPrimitive.Trigger

const DrawerPortal = DrawerPrimitive.Portal

const DrawerClose = DrawerPrimitive.Close

const DrawerOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <DrawerPrimitive.Overlay
    ref={ref}
    className={cn("fixed inset-0 z-50 bg-black/80", className)}
    {...props} />
))
DrawerOverlay.displayName = DrawerPrimitive.Overlay.displayName

const DrawerContent = React.forwardRef(({ className, children, ...props }, ref) => (
  <DrawerPortal>
    <DrawerOverlay />
    <DrawerPrimitive.Content
      ref={ref}
      className={cn(
        "fixed inset-x-0 bottom-0 z-50 mt-24 flex h-auto flex-col rounded-t-[10px] border bg-background",
        className
      )}
      {...props}>
      <div className="mx-auto mt-4 h-2 w-[100px] rounded-full bg-muted" />
      {children}
    </DrawerPrimitive.Content>
  </DrawerPortal>
))
DrawerContent.displayName = "DrawerContent"

const DrawerHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("grid gap-1.5 p-4 text-center sm:text-left", className)}
    {...props} />
)
DrawerHeader.displayName = "DrawerHeader"

const DrawerFooter = ({
  className,
  ...props
}) => (
  <div className={cn("mt-auto flex flex-col gap-2 p-4", className)} {...props} />
)
DrawerFooter.displayName = "DrawerFooter"

const DrawerTitle = React.forwardRef(({ className, ...props }, ref) => (
  <DrawerPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold leading-none tracking-tight", className)}
```

```
    {...props} />
))
DrawerTitle.displayName = DrawerPrimitive.Title.displayName

const DrawerDescription = React.forwardRef(({ className, ...props }, ref) => (
  <DrawerPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
DrawerDescription.displayName = DrawerPrimitive.Description.displayName

export {
  Drawer,
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/dropdown-menu.jsx
------------------------------------------------------------
import * as React from "react"
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const DropdownMenu = DropdownMenuPrimitive.Root

const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger

const DropdownMenuGroup = DropdownMenuPrimitive.Group

const DropdownMenuPortal = DropdownMenuPrimitive.Portal

const DropdownMenuSub = DropdownMenuPrimitive.Sub

const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup

const DropdownMenuSubTrigger = React.forwardRef(({ className, inset, children, ...props }, ref) => (
  <DropdownMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default gap-2 select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent
data-[state=open]:bg-accent [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
      inset && "pl-8",
      className
    )}
    {...props}>
    {children}
    <ChevronRight className="ml-auto" />
  </DropdownMenuPrimitive.SubTrigger>
))
DropdownMenuSubTrigger.displayName =
  DropdownMenuPrimitive.SubTrigger.displayName

const DropdownMenuSubContent = React.forwardRef(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg
data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props} />
))
DropdownMenuSubContent.displayName =
  DropdownMenuPrimitive.SubContent.displayName

const DropdownMenuContent = React.forwardRef(({ className, sideOffset = 4, ...props }, ref) => (
  <DropdownMenuPrimitive.Portal>
    <DropdownMenuPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md",
        "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
```

```
  </DropdownMenuPrimitive.Portal>
))
DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName

const DropdownMenuItem = React.forwardRef(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none transition-colors
focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50 [&>svg]:size-4
[&>svg]:shrink-0",
      inset && "pl-8",
      className
    )}
    {...props} />
))
DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName

const DropdownMenuCheckboxItem = React.forwardRef(({ className, children, checked, ...props }, ref) => (
  <DropdownMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transition-colors
focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    checked={checked}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
))
DropdownMenuCheckboxItem.displayName =
  DropdownMenuPrimitive.CheckboxItem.displayName

const DropdownMenuRadioItem = React.forwardRef(({ className, children, ...props }, ref) => (
  <DropdownMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transition-colors
focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-current" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.RadioItem>
))
DropdownMenuRadioItem.displayName = DropdownMenuPrimitive.RadioItem.displayName

const DropdownMenuLabel = React.forwardRef(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8", className)}
    {...props} />
))
DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName

const DropdownMenuSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props} />
))
DropdownMenuSeparator.displayName = DropdownMenuPrimitive.Separator.displayName

const DropdownMenuShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest opacity-60", className)}
      {...props} />)
  );
}
DropdownMenuShortcut.displayName = "DropdownMenuShortcut"

export {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
```

```
  DropdownMenuRadioItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuGroup,
  DropdownMenuPortal,
  DropdownMenuSub,
  DropdownMenuSubContent,
  DropdownMenuSubTrigger,
  DropdownMenuRadioGroup,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/form.jsx
------------------------------------------------------------
"use client";
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { Controller, FormProvider, useFormContext } from "react-hook-form";

import { cn } from "@/lib/utils"
import { Label } from "@/components/ui/label"

const Form = FormProvider

const FormFieldContext = React.createContext({})

const FormField = (
  {
    ...props
  }
) => {
  return (
    (<FormFieldContext.Provider value={{ name: props.name }}>
      <Controller {...props} />
    </FormFieldContext.Provider>)
  );
}

const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext)
  const itemContext = React.useContext(FormItemContext)
  const { getFieldState, formState } = useFormContext()

  const fieldState = getFieldState(fieldContext.name, formState)

  if (!fieldContext) {
    throw new Error("useFormField should be used within <FormField>")
  }

  const { id } = itemContext

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  }
}

const FormItemContext = React.createContext({})

const FormItem = React.forwardRef(({ className, ...props }, ref) => {
  const id = React.useId()

  return (
    (<FormItemContext.Provider value={{ id }}>
      <div ref={ref} className={cn("space-y-2", className)} {...props} />
    </FormItemContext.Provider>)
  );
})
FormItem.displayName = "FormItem"

const FormLabel = React.forwardRef(({ className, ...props }, ref) => {
  const { error, formItemId } = useFormField()

  return (
    (<Label
      ref={ref}
      className={cn(error && "text-destructive", className)}
      htmlFor={formItemId}
      {...props} />)
  );
})
FormLabel.displayName = "FormLabel"

const FormControl = React.forwardRef(({ ...props }, ref) => {
  const { error, formItemId, formDescriptionId, formMessageId } = useFormField()
```

```
  return (
    (<Slot
      ref={ref}
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props} />)
  );
})
FormControl.displayName = "FormControl"

const FormDescription = React.forwardRef(({ className, ...props }, ref) => {
  const { formDescriptionId } = useFormField()

  return (
    (<p
      ref={ref}
      id={formDescriptionId}
      className={cn("text-[0.8rem] text-muted-foreground", className)}
      {...props} />)
  );
})
FormDescription.displayName = "FormDescription"

const FormMessage = React.forwardRef(({ className, children, ...props }, ref) => {
  const { error, formMessageId } = useFormField()
  const body = error ? String(error?.message) : children

  if (!body) {
    return null
  }

  return (
    (<p
      ref={ref}
      id={formMessageId}
      className={cn("text-[0.8rem] font-medium text-destructive", className)}
      {...props}>
      {body}
    </p>)
  );
})
FormMessage.displayName = "FormMessage"

export {
  useFormField,
  Form,
  FormItem,
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/hover-card.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import * as HoverCardPrimitive from "@radix-ui/react-hover-card"

import { cn } from "@/lib/utils"

const HoverCard = HoverCardPrimitive.Root

const HoverCardTrigger = HoverCardPrimitive.Trigger

const HoverCardContent = React.forwardRef(({ className, align = "center", sideOffset = 4, ...props }, ref) => (
  <HoverCardPrimitive.Content
    ref={ref}
    align={align}
    sideOffset={sideOffset}
    className={cn(
      "z-50 w-64 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props} />
))
HoverCardContent.displayName = HoverCardPrimitive.Content.displayName

export { HoverCard, HoverCardTrigger, HoverCardContent }
```

```
------------------------------------------------------------
FICHIER : ./src/components/ui/image-helpers.js
------------------------------------------------------------
const WIX_MEDIA_HOSTS = {
  "media.base44.com": "/images/public/",
  "static.wixstatic.com": "/media/",
}

export const DEFAULT_TRANSFORM_WIDTH = 1024
export const IMAGE_LOAD_MODE = {
  OPTIMIZED: "optimized",
  ORIGINAL: "original",
  FALLBACK: "fallback",
}

const DEVICE_PIXEL_RATIOS = [1, 2, 3]
const MAX_DIMENSION = 6000

export function splitImageProps(props) {
  const wrapperProps = {}
  const imageProps = {}
  for (const [key, value] of Object.entries(props)) {
    if (key.startsWith("data-")) wrapperProps[key] = value
    else imageProps[key] = value
  }
  return { wrapperProps, imageProps }
}

export function getImagePreviewClassName(className, currentClassName, baselineClassName) {
  const sourceClasses = new Set((className || "").split(/ /))
  const baselineClasses = new Set(baselineClassName.split(/ /))
  return currentClassName.split(/ /).filter((token) =>
    !["inline-block", "relative"].includes(token) || !baselineClasses.has(token) || sourceClasses.has(token)
  ).join(" ")
}

/** Returns transform metadata only for canonical public Wix image URLs. */
export function parseWixMediaUrl(src) {
  try {
    const url = new URL(src)
    if (
      url.protocol !== "https:" ||
      url.username ||
      url.password ||
      (url.port && url.port !== "443")
    ) {
      return null
    }

    const pathPrefix = WIX_MEDIA_HOSTS[url.hostname]
    if (!pathPrefix) return null

    const transformed = url.pathname.match(/^(.*)\/v1\/(?:fill|fit)\/[^/]+\/[^/]+$/i)
    const basePath = transformed ? transformed[1] : url.pathname
    const filename = basePath.split("/").pop()
    if (
      !basePath.startsWith(pathPrefix) ||
      !filename ||
      !/\.[a-z0-9]+$/i.test(filename) ||
      /\.svg$/i.test(filename)
    ) {
      return null
    }

    return { baseUrl: `${url.origin}${basePath}`, filename }
  } catch {
    return null
  }
}

const clampDim = (n) => Math.min(Math.max(Math.round(n), 1), MAX_DIMENSION)
const clamp01 = (n) => Math.min(1, Math.max(0, n))

export function buildTransformUrl(
  { baseUrl, filename },
  { width, height, crop, focalPoint, quality }
) {
  const params = [`w_${clampDim(width)}`, `h_${clampDim(height || width)}`]
  if (crop) {
    params.push(
      focalPoint
        ? `fp_${clamp01(focalPoint.x).toFixed(2)}_${clamp01(focalPoint.y).toFixed(2)}`
        : "al_c"
    )
  }
  params.push(`q_${quality}`, "usm_0.66_1.00_0.01", "enc_webp", "quality_auto")
  const outputName = /\.gif$/i.test(filename)
    ? filename
    : filename.replace(/\.[a-z0-9]+$/i, "") + ".webp"
  return `${baseUrl}/v1/${crop ? "fill" : "fit"}/${params.join(",")}/${outputName}`
}
```

```
export function buildSrcSet(parsed, options) {
  return DEVICE_PIXEL_RATIOS.map(
    (dpr) =>
      `${buildTransformUrl(parsed, {
        ...options,
        width: options.width * dpr,
        height: options.height ? options.height * dpr : undefined,
      })} ${dpr}x`
  ).join(", ")
}

export function getOriginalImageUrl(src, parsed) {
  return parsed?.baseUrl || src
}

export function nextImageLoadMode(mode) {
  return mode === IMAGE_LOAD_MODE.OPTIMIZED
    ? IMAGE_LOAD_MODE.ORIGINAL
    : IMAGE_LOAD_MODE.FALLBACK
}


------------------------------------------------------------
FICHIER : ./src/components/ui/image.jsx
------------------------------------------------------------
import * as React from "react"
import { ResponsiveImage } from "./responsive-image"
import {
  getOriginalImageUrl,
  IMAGE_LOAD_MODE,
  nextImageLoadMode,
  parseWixMediaUrl,
} from "./image-helpers"

const FALLBACK_IMAGE_URL =
  "https://static.wixstatic.com/media/12d367_4f26ccd17f8f4e3a8958306ea08c2332~mv2.png"

/**
 * Image with built-in Wix Media Platform support: canonical public images on
 * media.base44.com and static.wixstatic.com/media are resized to the rendered
 * container per device pixel ratio and re-encoded to WebP; `fittingType="fill"`
 * crops server-side, optionally anchored at a focal point. Other URLs render
 * as a plain <img>. Failed transforms retry the original URL; only a broken
 * original swaps to the generic fallback image.
 */
const Image = React.forwardRef(
  (
    {
      src: source,
      fittingType = "fill",
      originWidth,
      originHeight,
      focalPointX,
      focalPointY,
      quality = 90,
      onError,
      ...props
    },
    ref
  ) => {
    const [previewSource, setPreviewSource] = React.useState(null)
    const preview = previewSource?.source === source ? previewSource : null
    const src = preview ? preview.value : source
    const replaceSource = (value, className) => setPreviewSource({
      source, value, className, sourceClassName: props.className,
    })
    React.useEffect(() => setPreviewSource(null), [source])
    const parsedSource = src && src !== FALLBACK_IMAGE_URL ? parseWixMediaUrl(src) : null
    const initialMode = parsedSource ? IMAGE_LOAD_MODE.OPTIMIZED : IMAGE_LOAD_MODE.ORIGINAL
    const [loadState, setLoadState] = React.useState({ src, mode: initialMode })
    const mode = loadState.src === src ? loadState.mode : initialMode

    React.useEffect(() => {
      setLoadState({ src, mode: initialMode })
    }, [src, initialMode])

    const handleError = (event) => {
      if (mode === IMAGE_LOAD_MODE.FALLBACK) return
      const nextMode = nextImageLoadMode(mode)
      setLoadState({ src, mode: nextMode })
      if (nextMode === IMAGE_LOAD_MODE.FALLBACK) onError?.(event)
    }

    const imageProps = {
      ...props,
      className: preview && preview.sourceClassName === props.className ? preview.className : props.className,
      onError: handleError,
    }

    if (!src) {
      // Renders as a real <img> (not a <div>) — the visual editor's
```

```
      // click-to-edit toolbar keys its "Replace Image" action off the DOM
      // tag being `img`, so a placeholder div would be unrecoverable in the
      // editor. FALLBACK_IMAGE_URL doubles as the "no image chosen" graphic.
      return <img ref={ref} src={FALLBACK_IMAGE_URL} {...imageProps} data-empty-image />
    }

    // A failed transform retries the underlying original as a plain image.
    // Only a failure of that original advances to the generic fallback.
    const parsed = mode === IMAGE_LOAD_MODE.OPTIMIZED ? parsedSource : null

    if (!parsed) {
      const isErrorMode = mode === IMAGE_LOAD_MODE.FALLBACK
      const imageSrc = isErrorMode ? FALLBACK_IMAGE_URL : getOriginalImageUrl(src, parsedSource)
      return (
        <img ref={ref} src={imageSrc} {...imageProps} data-error-image={isErrorMode || undefined} />
      )
    }

    const focalPoint =
      typeof focalPointX === "number" && typeof focalPointY === "number"
        ? { x: focalPointX, y: focalPointY }
        : undefined
    // Origin dimensions are optional — when known they stabilize layout via
    // the wrapper's aspect-ratio before the image loads.
    const aspectRatio =
      originWidth && originHeight ? `${originWidth} / ${originHeight}` : undefined

    return (
      <ResponsiveImage
        ref={ref}
        src={src}
        parsed={parsed}
        onSourceChange={replaceSource}
        fittingType={fittingType}
        focalPoint={focalPoint}
        quality={quality}
        aspectRatio={aspectRatio}
        {...imageProps}
      />
    )
  }
)
Image.displayName = "Image"

export { Image }


------------------------------------------------------------
FICHIER : ./src/components/ui/input-otp.jsx
------------------------------------------------------------
import * as React from "react"
import { OTPInput, OTPInputContext } from "input-otp"
import { Minus } from "lucide-react"

import { cn } from "@/lib/utils"

const InputOTP = React.forwardRef(({ className, containerClassName, ...props }, ref) => (
  <OTPInput
    ref={ref}
    containerClassName={cn("flex items-center gap-2 has-[:disabled]:opacity-50", containerClassName)}
    className={cn("disabled:cursor-not-allowed", className)}
    {...props} />
))
InputOTP.displayName = "InputOTP"

const InputOTPGroup = React.forwardRef(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("flex items-center", className)} {...props} />
))
InputOTPGroup.displayName = "InputOTPGroup"

const InputOTPSlot = React.forwardRef(({ index, className, ...props }, ref) => {
  const inputOTPContext = React.useContext(OTPInputContext)
  const { char, hasFakeCaret, isActive } = inputOTPContext.slots[index]

  return (
    (<div
      ref={ref}
      className={cn(
        "relative flex h-9 w-9 items-center justify-center border-y border-r border-input text-sm shadow-sm transition-all
first:rounded-l-md first:border-l last:rounded-r-md",
        isActive && "z-10 ring-1 ring-ring",
        className
      )}
      {...props}>
      {char}
      {hasFakeCaret && (
        <div
          className="pointer-events-none absolute inset-0 flex items-center justify-center">
          <div className="h-4 w-px animate-caret-blink bg-foreground duration-1000" />
        </div>
      )}
    </div>)
```

```
  );
})
InputOTPSlot.displayName = "InputOTPSlot"

const InputOTPSeparator = React.forwardRef(({ ...props }, ref) => (
  <div ref={ref} role="separator" {...props}>
    <Minus />
  </div>
))
InputOTPSeparator.displayName = "InputOTPSeparator"

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator }


------------------------------------------------------------
FICHIER : ./src/components/ui/input.jsx
------------------------------------------------------------
import * as React from "react"

import { cn } from "@/lib/utils"

const Input = React.forwardRef(({ className, type, ...props }, ref) => {
  return (
    (<input
      type={type}
      className={cn(
        "flex h-9 w-full rounded-md border border-input bg-transparent px-3 py-1 text-base shadow-sm transition-colors
file:border-0 file:bg-transparent file:text-sm file:font-medium file:text-foreground placeholder:text-muted-foreground
focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50
md:text-sm",
        className
      )}
      ref={ref}
      {...props} />)
  );
})
Input.displayName = "Input"

export { Input }


------------------------------------------------------------
FICHIER : ./src/components/ui/label.jsx
------------------------------------------------------------
import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const labelVariants = cva(
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
)

const Label = React.forwardRef(({ className, ...props }, ref) => (
  <LabelPrimitive.Root ref={ref} className={cn(labelVariants(), className)} {...props} />
))
Label.displayName = LabelPrimitive.Root.displayName

export { Label }


------------------------------------------------------------
FICHIER : ./src/components/ui/menubar.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import * as MenubarPrimitive from "@radix-ui/react-menubar"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utils"

function MenubarMenu({
  ...props
}) {
  return <MenubarPrimitive.Menu {...props} />;
}

function MenubarGroup({
  ...props
}) {
  return <MenubarPrimitive.Group {...props} />;
}

function MenubarPortal({
  ...props
}) {
  return <MenubarPrimitive.Portal {...props} />;
}

function MenubarRadioGroup({
```

```
  ...props
}) {
  return <MenubarPrimitive.RadioGroup {...props} />;
}

function MenubarSub({
  ...props
}) {
  return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />;
}

const Menubar = React.forwardRef(({ className, ...props }, ref) => (
  <MenubarPrimitive.Root
    ref={ref}
    className={cn(
      "flex h-9 items-center space-x-1 rounded-md border bg-background p-1 shadow-sm",
      className
    )}
    {...props} />
))
Menubar.displayName = MenubarPrimitive.Root.displayName

const MenubarTrigger = React.forwardRef(({ className, ...props }, ref) => (
  <MenubarPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-3 py-1 text-sm font-medium outline-none focus:bg-accent
focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
      className
    )}
    {...props} />
))
MenubarTrigger.displayName = MenubarPrimitive.Trigger.displayName

const MenubarSubTrigger = React.forwardRef(({ className, inset, children, ...props }, ref) => (
  <MenubarPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
      inset && "pl-8",
      className
    )}
    {...props}>
    {children}
    <ChevronRight className="ml-auto h-4 w-4" />
  </MenubarPrimitive.SubTrigger>
))
MenubarSubTrigger.displayName = MenubarPrimitive.SubTrigger.displayName

const MenubarSubContent = React.forwardRef(({ className, ...props }, ref) => (
  <MenubarPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg
data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props} />
))
MenubarSubContent.displayName = MenubarPrimitive.SubContent.displayName

const MenubarContent = React.forwardRef((
  { className, align = "start", alignOffset = -4, sideOffset = 8, ...props },
  ref
) => (
  <MenubarPrimitive.Portal>
    <MenubarPrimitive.Content
      ref={ref}
      align={align}
      alignOffset={alignOffset}
      sideOffset={sideOffset}
      className={cn(
        "z-50 min-w-[12rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md
data-[state=open]:animate-in data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </MenubarPrimitive.Portal>
))
MenubarContent.displayName = MenubarPrimitive.Content.displayName

const MenubarItem = React.forwardRef(({ className, inset, ...props }, ref) => (
  <MenubarPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
```

```
      inset && "pl-8",
      className
    )}
    {...props} />
))
MenubarItem.displayName = MenubarPrimitive.Item.displayName

const MenubarCheckboxItem = React.forwardRef(({ className, children, checked, ...props }, ref) => (
  <MenubarPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    checked={checked}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <MenubarPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </MenubarPrimitive.ItemIndicator>
    </span>
    {children}
  </MenubarPrimitive.CheckboxItem>
))
MenubarCheckboxItem.displayName = MenubarPrimitive.CheckboxItem.displayName

const MenubarRadioItem = React.forwardRef(({ className, children, ...props }, ref) => (
  <MenubarPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <MenubarPrimitive.ItemIndicator>
        <Circle className="h-4 w-4 fill-current" />
      </MenubarPrimitive.ItemIndicator>
    </span>
    {children}
  </MenubarPrimitive.RadioItem>
))
MenubarRadioItem.displayName = MenubarPrimitive.RadioItem.displayName

const MenubarLabel = React.forwardRef(({ className, inset, ...props }, ref) => (
  <MenubarPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8", className)}
    {...props} />
))
MenubarLabel.displayName = MenubarPrimitive.Label.displayName

const MenubarSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <MenubarPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props} />
))
MenubarSeparator.displayName = MenubarPrimitive.Separator.displayName

const MenubarShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
      {...props} />)
  );
}
MenubarShortcut.displayname = "MenubarShortcut"

export {
  Menubar,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarItem,
  MenubarSeparator,
  MenubarLabel,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarPortal,
  MenubarSubContent,
  MenubarSubTrigger,
  MenubarGroup,
  MenubarSub,
  MenubarShortcut,
}
```

```
------------------------------------------------------------
FICHIER : ./src/components/ui/navigation-menu.jsx
------------------------------------------------------------
import * as React from "react"
import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu"
import { cva } from "class-variance-authority"
import { ChevronDown } from "lucide-react"

import { cn } from "@/lib/utils"

const NavigationMenu = React.forwardRef(({ className, children, ...props }, ref) => (
  <NavigationMenuPrimitive.Root
    ref={ref}
    className={cn(
      "relative z-10 flex max-w-max flex-1 items-center justify-center",
      className
    )}
    {...props}>
    {children}
    <NavigationMenuViewport />
  </NavigationMenuPrimitive.Root>
))
NavigationMenu.displayName = NavigationMenuPrimitive.Root.displayName

const NavigationMenuList = React.forwardRef(({ className, ...props }, ref) => (
  <NavigationMenuPrimitive.List
    ref={ref}
    className={cn(
      "group flex flex-1 list-none items-center justify-center space-x-1",
      className
    )}
    {...props} />
))
NavigationMenuList.displayName = NavigationMenuPrimitive.List.displayName

const NavigationMenuItem = NavigationMenuPrimitive.Item

const navigationMenuTriggerStyle = cva(
  "group inline-flex h-9 w-max items-center justify-center rounded-md bg-background px-4 py-2 text-sm font-medium
transition-colors hover:bg-accent hover:text-accent-foreground focus:bg-accent focus:text-accent-foreground focus:outline-none
disabled:pointer-events-none disabled:opacity-50 data-[active]:bg-accent/50 data-[state=open]:bg-accent/50"
)

const NavigationMenuTrigger = React.forwardRef(({ className, children, ...props }, ref) => (
  <NavigationMenuPrimitive.Trigger
    ref={ref}
    className={cn(navigationMenuTriggerStyle(), "group", className)}
    {...props}>
    {children}{" "}
    <ChevronDown
      className="relative top-[1px] ml-1 h-3 w-3 transition duration-300 group-data-[state=open]:rotate-180"
      aria-hidden="true" />
  </NavigationMenuPrimitive.Trigger>
))
NavigationMenuTrigger.displayName = NavigationMenuPrimitive.Trigger.displayName

const NavigationMenuContent = React.forwardRef(({ className, ...props }, ref) => (
  <NavigationMenuPrimitive.Content
    ref={ref}
    className={cn(
      "left-0 top-0 w-full data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out data-[motion^=from-]:fade-in
data-[motion^=to-]:fade-out data-[motion=from-end]:slide-in-from-right-52 data-[motion=from-start]:slide-in-from-left-52
data-[motion=to-end]:slide-out-to-right-52 data-[motion=to-start]:slide-out-to-left-52 md:absolute md:w-auto ",
      className
    )}
    {...props} />
))
NavigationMenuContent.displayName = NavigationMenuPrimitive.Content.displayName

const NavigationMenuLink = NavigationMenuPrimitive.Link

const NavigationMenuViewport = React.forwardRef(({ className, ...props }, ref) => (
  <div className={cn("absolute left-0 top-full flex justify-center")}>
    <NavigationMenuPrimitive.Viewport
      className={cn(
        "origin-top-center relative mt-1.5 h-[var(--radix-navigation-menu-viewport-height)] w-full overflow-hidden rounded-md
border bg-popover text-popover-foreground shadow data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90 md:w-[var(--radix-navigation-menu-viewport-width)]",
        className
      )}
      ref={ref}
      {...props} />
  </div>
))
NavigationMenuViewport.displayName =
  NavigationMenuPrimitive.Viewport.displayName

const NavigationMenuIndicator = React.forwardRef(({ className, ...props }, ref) => (
  <NavigationMenuPrimitive.Indicator
    ref={ref}
```

```
    className={cn(
      "top-full z-[1] flex h-1.5 items-end justify-center overflow-hidden data-[state=visible]:animate-in
data-[state=hidden]:animate-out data-[state=hidden]:fade-out data-[state=visible]:fade-in",
      className
    )}
    {...props}>
    <div
      className="relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm bg-border shadow-md" />
  </NavigationMenuPrimitive.Indicator>
))
NavigationMenuIndicator.displayName =
  NavigationMenuPrimitive.Indicator.displayName

export {
  navigationMenuTriggerStyle,
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/pagination.jsx
------------------------------------------------------------
import * as React from "react"
import { ChevronLeft, ChevronRight, MoreHorizontal } from "lucide-react"

import { cn } from "@/lib/utils"
import { buttonVariants } from "@/components/ui/button";

const Pagination = ({
  className,
  ...props
}) => (
  <nav
    role="navigation"
    aria-label="pagination"
    className={cn("mx-auto flex w-full justify-center", className)}
    {...props} />
)
Pagination.displayName = "Pagination"

const PaginationContent = React.forwardRef(({ className, ...props }, ref) => (
  <ul
    ref={ref}
    className={cn("flex flex-row items-center gap-1", className)}
    {...props} />
))
PaginationContent.displayName = "PaginationContent"

const PaginationItem = React.forwardRef(({ className, ...props }, ref) => (
  <li ref={ref} className={cn("", className)} {...props} />
))
PaginationItem.displayName = "PaginationItem"

const PaginationLink = ({
  className,
  isActive,
  size = "icon",
  ...props
}) => (
  <a
    aria-current={isActive ? "page" : undefined}
    className={cn(buttonVariants({
      variant: isActive ? "outline" : "ghost",
      size,
    }), className)}
    {...props} />
)
PaginationLink.displayName = "PaginationLink"

const PaginationPrevious = ({
  className,
  ...props
}) => (
  <PaginationLink
    aria-label="Go to previous page"
    size="default"
    className={cn("gap-1 pl-2.5", className)}
    {...props}>
    <ChevronLeft className="h-4 w-4" />
    <span>Previous</span>
  </PaginationLink>
)
PaginationPrevious.displayName = "PaginationPrevious"

const PaginationNext = ({
```

```
  className,
  ...props
}) => (
  <PaginationLink
    aria-label="Go to next page"
    size="default"
    className={cn("gap-1 pr-2.5", className)}
    {...props}>
    <span>Next</span>
    <ChevronRight className="h-4 w-4" />
  </PaginationLink>
)
PaginationNext.displayName = "PaginationNext"

const PaginationEllipsis = ({
  className,
  ...props
}) => (
  <span
    aria-hidden
    className={cn("flex h-9 w-9 items-center justify-center", className)}
    {...props}>
    <MoreHorizontal className="h-4 w-4" />
    <span className="sr-only">More pages</span>
  </span>
)
PaginationEllipsis.displayName = "PaginationEllipsis"

export {
  Pagination,
  PaginationContent,
  PaginationLink,
  PaginationItem,
  PaginationPrevious,
  PaginationNext,
  PaginationEllipsis,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/popover.jsx
------------------------------------------------------------
import * as React from "react"
import * as PopoverPrimitive from "@radix-ui/react-popover"

import { cn } from "@/lib/utils"

const Popover = PopoverPrimitive.Root

const PopoverTrigger = PopoverPrimitive.Trigger

const PopoverAnchor = PopoverPrimitive.Anchor

const PopoverContent = React.forwardRef(({ className, align = "center", sideOffset = 4, ...props }, ref) => (
  <PopoverPrimitive.Portal>
    <PopoverPrimitive.Content
      ref={ref}
      align={align}
      sideOffset={sideOffset}
      className={cn(
        "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </PopoverPrimitive.Portal>
))
PopoverContent.displayName = PopoverPrimitive.Content.displayName

export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor }


------------------------------------------------------------
FICHIER : ./src/components/ui/progress.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import * as ProgressPrimitive from "@radix-ui/react-progress"

import { cn } from "@/lib/utils"

const Progress = React.forwardRef(({ className, value, ...props }, ref) => (
  <ProgressPrimitive.Root
    ref={ref}
    className={cn(
      "relative h-2 w-full overflow-hidden rounded-full bg-primary/20",
      className
    )}
    {...props}>
```

```
    <ProgressPrimitive.Indicator
      className="h-full w-full flex-1 bg-primary transition-all"
      style={{ transform: `translateX(-${100 - (value || 0)}%)` }} />
  </ProgressPrimitive.Root>
))
Progress.displayName = ProgressPrimitive.Root.displayName

export { Progress }


------------------------------------------------------------
FICHIER : ./src/components/ui/radio-group.jsx
------------------------------------------------------------
import * as React from "react"
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group"
import { Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const RadioGroup = React.forwardRef(({ className, ...props }, ref) => {
  return (<RadioGroupPrimitive.Root className={cn("grid gap-2", className)} {...props} ref={ref} />);
})
RadioGroup.displayName = RadioGroupPrimitive.Root.displayName

const RadioGroupItem = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<RadioGroupPrimitive.Item
      ref={ref}
      className={cn(
        "aspect-square h-4 w-4 rounded-full border border-primary text-primary shadow focus:outline-none focus-visible:ring-1
focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50",
        className
      )}
      {...props}>
      <RadioGroupPrimitive.Indicator className="flex items-center justify-center">
        <Circle className="h-3.5 w-3.5 fill-primary" />
      </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>)
  );
})
RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName

export { RadioGroup, RadioGroupItem }


------------------------------------------------------------
FICHIER : ./src/components/ui/resizable.jsx
------------------------------------------------------------
"use client"

import { GripVertical } from "lucide-react"
import * as ResizablePrimitive from "react-resizable-panels"

import { cn } from "@/lib/utils"

const ResizablePanelGroup = ({
  className,
  ...props
}) => (
  <ResizablePrimitive.PanelGroup
    className={cn(
      "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
      className
    )}
    {...props} />
)

const ResizablePanel = ResizablePrimitive.Panel

const ResizableHandle = ({
  withHandle,
  className,
  ...props
}) => (
  <ResizablePrimitive.PanelResizeHandle
    className={cn(
      "relative flex w-px items-center justify-center bg-border after:absolute after:inset-y-0 after:left-1/2 after:w-1
after:-translate-x-1/2 focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring focus-visible:ring-offset-1
data-[panel-group-direction=vertical]:h-px data-[panel-group-direction=vertical]:w-full
data-[panel-group-direction=vertical]:after:left-0 data-[panel-group-direction=vertical]:after:h-1
data-[panel-group-direction=vertical]:after:w-full data-[panel-group-direction=vertical]:after:-translate-y-1/2
data-[panel-group-direction=vertical]:after:translate-x-0 [&[data-panel-group-direction=vertical]>div]:rotate-90",
      className
    )}
    {...props}>
    {withHandle && (
      <div
        className="z-10 flex h-4 w-3 items-center justify-center rounded-sm border bg-border">
        <GripVertical className="h-2.5 w-2.5" />
      </div>
    )}
  </ResizablePrimitive.PanelResizeHandle>
```

```
)

export { ResizablePanelGroup, ResizablePanel, ResizableHandle }


------------------------------------------------------------
FICHIER : ./src/components/ui/responsive-image.jsx
------------------------------------------------------------
import * as React from "react"
import { cn } from "../../lib/utils"
import { buildSrcSet, buildTransformUrl, splitImageProps } from "./image-helpers"
import { useResponsiveImage } from "./use-responsive-image"

export const ResponsiveImage = React.forwardRef(
  ({ src, parsed, fittingType, focalPoint, quality, className, style, aspectRatio, onLoad, onSourceChange, ...props }, ref) => {
    const { wrapperRef, imgRef, loaded, options, handleLoad } = useResponsiveImage(
      { parsed, fittingType, focalPoint, quality, className, onLoad, onSourceChange }, ref
    )
    const { wrapperProps, imageProps } = splitImageProps(props)

    return (
      <span
        ref={wrapperRef}
        className={cn("inline-block relative", className)}
        style={{ aspectRatio, ...style }}
        {...wrapperProps}
        data-base44-image=""
        data-base44-image-src={src}
      >
        {/* Contain both image layers inside the padded content box without adding an edit target. */}
        <span data-source-location={undefined} className="block relative w-full h-full overflow-hidden">
          {options && !loaded && (
            <img
              data-source-location={undefined}
              src={buildTransformUrl(parsed, {
                ...options,
                width: 20,
                height: options.height
                  ? Math.max(1, Math.round((20 * options.height) / options.width))
                  : undefined,
                quality: 20,
              })}
              alt=""
              aria-hidden="true"
              className="w-full h-full inset-0 absolute"
              style={{
                objectFit: fittingType === "fit" ? "contain" : "cover",
                filter: "blur(10px)",
                transform: "scale(1.1)",
              }}
            />
          )}
          {options && (
            <img
              data-source-location={undefined}
              ref={imgRef}
              src={buildTransformUrl(parsed, options)}
              srcSet={buildSrcSet(parsed, options)}
              loading="lazy"
              className={cn(
                "w-full h-full inset-0 absolute",
                fittingType === "fit" ? "object-contain" : "object-cover"
              )}
              onLoad={handleLoad}
              {...imageProps}
            />
          )}
        </span>
      </span>
    )
  }
)
ResponsiveImage.displayName = "ResponsiveImage"


------------------------------------------------------------
FICHIER : ./src/components/ui/scroll-area.jsx
------------------------------------------------------------
import * as React from "react"
import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area"

import { cn } from "@/lib/utils"

const ScrollArea = React.forwardRef(({ className, children, ...props }, ref) => (
  <ScrollAreaPrimitive.Root
    ref={ref}
    className={cn("relative overflow-hidden", className)}
    {...props}>
    <ScrollAreaPrimitive.Viewport className="h-full w-full rounded-[inherit]">
      {children}
    </ScrollAreaPrimitive.Viewport>
    <ScrollBar />
```

```
    <ScrollAreaPrimitive.Corner />
  </ScrollAreaPrimitive.Root>
))
ScrollArea.displayName = ScrollAreaPrimitive.Root.displayName

const ScrollBar = React.forwardRef(({ className, orientation = "vertical", ...props }, ref) => (
  <ScrollAreaPrimitive.ScrollAreaScrollbar
    ref={ref}
    orientation={orientation}
    className={cn(
      "flex touch-none select-none transition-colors",
      orientation === "vertical" &&
        "h-full w-2.5 border-l border-l-transparent p-[1px]",
      orientation === "horizontal" &&
        "h-2.5 flex-col border-t border-t-transparent p-[1px]",
      className
    )}
    {...props}>
    <ScrollAreaPrimitive.ScrollAreaThumb className="relative flex-1 rounded-full bg-border" />
  </ScrollAreaPrimitive.ScrollAreaScrollbar>
))
ScrollBar.displayName = ScrollAreaPrimitive.ScrollAreaScrollbar.displayName

export { ScrollArea, ScrollBar }


------------------------------------------------------------
FICHIER : ./src/components/ui/select.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import * as SelectPrimitive from "@radix-ui/react-select"
import { Check, ChevronDown, ChevronUp } from "lucide-react"

import { cn } from "@/lib/utils"

const Select = SelectPrimitive.Root

const SelectGroup = SelectPrimitive.Group

const SelectValue = SelectPrimitive.Value

const SelectTrigger = React.forwardRef(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex h-9 w-full items-center justify-between whitespace-nowrap rounded-md border border-input bg-transparent px-3 py-2
text-sm shadow-sm ring-offset-background data-[placeholder]:text-muted-foreground focus:outline-none focus:ring-1
focus:ring-ring disabled:cursor-not-allowed disabled:opacity-50 [&>span]:line-clamp-1",
      className
    )}
    {...props}>
    {children}
    <SelectPrimitive.Icon asChild>
      <ChevronDown className="h-4 w-4 opacity-50" />
    </SelectPrimitive.Icon>
  </SelectPrimitive.Trigger>
))
SelectTrigger.displayName = SelectPrimitive.Trigger.displayName

const SelectScrollUpButton = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollUpButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1", className)}
    {...props}>
    <ChevronUp className="h-4 w-4" />
  </SelectPrimitive.ScrollUpButton>
))
SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName

const SelectScrollDownButton = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollDownButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1", className)}
    {...props}>
    <ChevronDown className="h-4 w-4" />
  </SelectPrimitive.ScrollDownButton>
))
SelectScrollDownButton.displayName =
  SelectPrimitive.ScrollDownButton.displayName

const SelectContent = React.forwardRef(({ className, children, position = "popper", ...props }, ref) => (
  <SelectPrimitive.Portal>
    <SelectPrimitive.Content
      ref={ref}
      className={cn(
        "relative z-50 max-h-96 min-w-[8rem] overflow-hidden rounded-md border bg-popover text-popover-foreground shadow-md
data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        position === "popper" &&
```

```
          "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1
data-[side=top]:-translate-y-1",
        className
      )}
      position={position}
      {...props}>
      <SelectScrollUpButton />
      <SelectPrimitive.Viewport
        className={cn("p-1", position === "popper" &&
          "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)]")}>
        {children}
      </SelectPrimitive.Viewport>
      <SelectScrollDownButton />
    </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
))
SelectContent.displayName = SelectPrimitive.Content.displayName

const SelectLabel = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", className)}
    {...props} />
))
SelectLabel.displayName = SelectPrimitive.Label.displayName

const SelectItem = React.forwardRef(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5 pl-2 pr-8 text-sm outline-none
focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute right-2 flex h-3.5 w-3.5 items-center justify-center">
      <SelectPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </SelectPrimitive.ItemIndicator>
    </span>
    <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
  </SelectPrimitive.Item>
))
SelectItem.displayName = SelectPrimitive.Item.displayName

const SelectSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props} />
))
SelectSeparator.displayName = SelectPrimitive.Separator.displayName

export {
  Select,
  SelectGroup,
  SelectValue,
  SelectTrigger,
  SelectContent,
  SelectLabel,
  SelectItem,
  SelectSeparator,
  SelectScrollUpButton,
  SelectScrollDownButton,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/separator.jsx
------------------------------------------------------------
import * as React from "react"
import * as SeparatorPrimitive from "@radix-ui/react-separator"

import { cn } from "@/lib/utils"

const Separator = React.forwardRef((
  { className, orientation = "horizontal", decorative = true, ...props },
  ref
) => (
  <SeparatorPrimitive.Root
    ref={ref}
    decorative={decorative}
    orientation={orientation}
    className={cn(
      "shrink-0 bg-border",
      orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",
      className
    )}
    {...props} />
))
Separator.displayName = SeparatorPrimitive.Root.displayName
```

```
export { Separator }


------------------------------------------------------------
FICHIER : ./src/components/ui/sheet.jsx
------------------------------------------------------------
"use client";
import * as React from "react"
import * as SheetPrimitive from "@radix-ui/react-dialog"
import { cva } from "class-variance-authority";
import { X } from "lucide-react"

import { cn } from "@/lib/utils"

const Sheet = SheetPrimitive.Root

const SheetTrigger = SheetPrimitive.Trigger

const SheetClose = SheetPrimitive.Close

const SheetPortal = SheetPrimitive.Portal

const SheetOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <SheetPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-black/80  data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
      className
    )}
    {...props}
    ref={ref} />
))
SheetOverlay.displayName = SheetPrimitive.Overlay.displayName

const sheetVariants = cva(
  "fixed z-50 gap-4 bg-background p-6 shadow-lg transition ease-in-out data-[state=closed]:duration-300
data-[state=open]:duration-500 data-[state=open]:animate-in data-[state=closed]:animate-out",
  {
    variants: {
      side: {
        top: "inset-x-0 top-0 border-b data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top",
        bottom:
          "inset-x-0 bottom-0 border-t data-[state=closed]:slide-out-to-bottom data-[state=open]:slide-in-from-bottom",
        left: "inset-y-0 left-0 h-full w-3/4 border-r data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left
sm:max-w-sm",
        right:
          "inset-y-0 right-0 h-full w-3/4 border-l data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right
sm:max-w-sm",
      },
    },
    defaultVariants: {
      side: "right",
    },
  }
)

const SheetContent = React.forwardRef(({ side = "right", className, children, ...props }, ref) => (
  <SheetPortal>
    <SheetOverlay />
    <SheetPrimitive.Content ref={ref} className={cn(sheetVariants({ side }), className)} {...props}>
      <SheetPrimitive.Close
        className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background transition-opacity hover:opacity-100
focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none
data-[state=open]:bg-secondary">
        <X className="h-4 w-4" />
        <span className="sr-only">Close</span>
      </SheetPrimitive.Close>
      {children}
    </SheetPrimitive.Content>
  </SheetPortal>
))
SheetContent.displayName = SheetPrimitive.Content.displayName

const SheetHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col space-y-2 text-center sm:text-left", className)}
    {...props} />
)
SheetHeader.displayName = "SheetHeader"

const SheetFooter = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
    {...props} />
)
SheetFooter.displayName = "SheetFooter"
```

```
const SheetTitle = React.forwardRef(({ className, ...props }, ref) => (
  <SheetPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold text-foreground", className)}
    {...props} />
))
SheetTitle.displayName = SheetPrimitive.Title.displayName

const SheetDescription = React.forwardRef(({ className, ...props }, ref) => (
  <SheetPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
SheetDescription.displayName = SheetPrimitive.Description.displayName

export {
  Sheet,
  SheetPortal,
  SheetOverlay,
  SheetTrigger,
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/sidebar.jsx
------------------------------------------------------------
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva } from "class-variance-authority";
import { PanelLeft } from "lucide-react"

import { useIsMobile } from "@/hooks/use-mobile"
import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Separator } from "@/components/ui/separator"
import { Sheet, SheetContent } from "@/components/ui/sheet"
import { Skeleton } from "@/components/ui/skeleton"
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from "@/components/ui/tooltip"

const SIDEBAR_COOKIE_NAME = "sidebar_state"
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7
const SIDEBAR_WIDTH = "16rem"
const SIDEBAR_WIDTH_MOBILE = "18rem"
const SIDEBAR_WIDTH_ICON = "3rem"
const SIDEBAR_KEYBOARD_SHORTCUT = "b"

const SidebarContext = React.createContext(null)

function useSidebar() {
  const context = React.useContext(SidebarContext)
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.")
  }

  return context
}

const SidebarProvider = React.forwardRef((
  {
    defaultOpen = true,
    open: openProp,
    onOpenChange: setOpenProp,
    className,
    style,
    children,
    ...props
  },
  ref
) => {
  const isMobile = useIsMobile()
  const [openMobile, setOpenMobile] = React.useState(false)

  // This is the internal state of the sidebar.
  // We use openProp and setOpenProp for control from outside the component.
  const [_open, _setOpen] = React.useState(defaultOpen)
  const open = openProp ?? _open
  const setOpen = React.useCallback((value) => {
    const openState = typeof value === "function" ? value(open) : value
```

```
    if (setOpenProp) {
      setOpenProp(openState)
    } else {
      _setOpen(openState)
    }

    // This sets the cookie to keep the sidebar state.
    document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`
  }, [setOpenProp, open])

  // Helper to toggle the sidebar.
  const toggleSidebar = React.useCallback(() => {
    return isMobile
      ? setOpenMobile((open) => !open)
      : setOpen((open) => !open);
  }, [isMobile, setOpen, setOpenMobile])

  // Adds a keyboard shortcut to toggle the sidebar.
  React.useEffect(() => {
    const handleKeyDown = (event) => {
      if (
        event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
        (event.metaKey || event.ctrlKey)
      ) {
        event.preventDefault()
        toggleSidebar()
      }
    }

    window.addEventListener("keydown", handleKeyDown)
    return () => window.removeEventListener("keydown", handleKeyDown);
  }, [toggleSidebar])

  // We add a state so that we can do data-state="expanded" or "collapsed".
  // This makes it easier to style the sidebar with Tailwind classes.
  const state = open ? "expanded" : "collapsed"

  const contextValue = React.useMemo(() => ({
    state,
    open,
    setOpen,
    isMobile,
    openMobile,
    setOpenMobile,
    toggleSidebar,
  }), [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar])

  return (
    (<SidebarContext.Provider value={contextValue}>
      <TooltipProvider delayDuration={0}>
        <div
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH,
              "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
              ...style
            }
          }
          className={cn(
            "group/sidebar-wrapper flex min-h-svh w-full has-[[data-variant=inset]]:bg-sidebar",
            className
          )}
          ref={ref}
          {...props}>
          {children}
        </div>
      </TooltipProvider>
    </SidebarContext.Provider>)
  );
})
SidebarProvider.displayName = "SidebarProvider"

const Sidebar = React.forwardRef((
  {
    side = "left",
    variant = "sidebar",
    collapsible = "offcanvas",
    className,
    children,
    ...props
  },
  ref
) => {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar()

  if (collapsible === "none") {
    return (
      (<div
        className={cn(
          "flex h-full w-[--sidebar-width] flex-col bg-sidebar text-sidebar-foreground",
          className
        )}
```

```
        ref={ref}
        {...props}>
        {children}
      </div>)
    );
  }

  if (isMobile) {
    return (
      (<Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
        <SheetContent
          data-sidebar="sidebar"
          data-mobile="true"
          className="w-[--sidebar-width] bg-sidebar p-0 text-sidebar-foreground [&>button]:hidden"
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE
            }
          }
          side={side}>
          <div className="flex h-full w-full flex-col">{children}</div>
        </SheetContent>
      </Sheet>)
    );
  }

  return (
    (<div
      ref={ref}
      className="group peer hidden text-sidebar-foreground md:block"
      data-state={state}
      data-collapsible={state === "collapsed" ? collapsible : ""}
      data-variant={variant}
      data-side={side}>
      {/* This is what handles the sidebar gap on desktop */}
      <div
        className={cn(
          "relative h-svh w-[--sidebar-width] bg-transparent transition-[width] duration-200 ease-linear",
          "group-data-[collapsible=offcanvas]:w-0",
          "group-data-[side=right]:rotate-180",
          variant === "floating" || variant === "inset"
            ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spacing.4))]"
            : "group-data-[collapsible=icon]:w-[--sidebar-width-icon]"
        )} />
      <div
        className={cn(
          "fixed inset-y-0 z-10 hidden h-svh w-[--sidebar-width] transition-[left,right,width] duration-200 ease-linear
md:flex",
          side === "left"
            ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"
            : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",
          // Adjust the padding for floating and inset variants.
          variant === "floating" || variant === "inset"
            ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spacing.4)_+2px)]"
            : "group-data-[collapsible=icon]:w-[--sidebar-width-icon] group-data-[side=left]:border-r
group-data-[side=right]:border-l",
          className
        )}
        {...props}>
        <div
          data-sidebar="sidebar"
          className="flex h-full w-full flex-col bg-sidebar group-data-[variant=floating]:rounded-lg
group-data-[variant=floating]:border group-data-[variant=floating]:border-sidebar-border group-data-[variant=floating]:shadow">
          {children}
        </div>
      </div>
    </div>)
  );
})
Sidebar.displayName = "Sidebar"

const SidebarTrigger = React.forwardRef(({ className, onClick, asChild = false, ...props }, ref) => {
  const { toggleSidebar } = useSidebar()

  return (
    (<Button
      ref={ref}
      data-sidebar="trigger"
      variant="ghost"
      size="icon"
      className={cn("h-7 w-7", className)}
      onClick={(event) => {
        onClick?.(event)
        toggleSidebar()
      }}
      asChild={asChild}
      {...props}>
      {asChild ? (
        <PanelLeft />
      ) : (
        <>
          <PanelLeft />
```

```
          <span className="sr-only">Toggle Sidebar</span>
        </>
      )}
    </Button>)
  );
})
SidebarTrigger.displayName = "SidebarTrigger"

const SidebarRail = React.forwardRef(({ className, ...props }, ref) => {
  const { toggleSidebar } = useSidebar()

  return (
    (<button
      ref={ref}
      data-sidebar="rail"
      aria-label="Toggle Sidebar"
      tabIndex={-1}
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        "absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all ease-linear after:absolute after:inset-y-0
after:left-1/2 after:w-[2px] hover:after:bg-sidebar-border group-data-[side=left]:-right-4 group-data-[side=right]:left-0
sm:flex",
        "[[data-side=left]_&]:cursor-w-resize [[data-side=right]_&]:cursor-e-resize",
        "[[data-side=left][data-state=collapsed]_&]:cursor-e-resize
[[data-side=right][data-state=collapsed]_&]:cursor-w-resize",
        "group-data-[collapsible=offcanvas]:translate-x-0 group-data-[collapsible=offcanvas]:after:left-full
group-data-[collapsible=offcanvas]:hover:bg-sidebar",
        "[[data-side=left][data-collapsible=offcanvas]_&]:-right-2",
        "[[data-side=right][data-collapsible=offcanvas]_&]:-left-2",
        className
      )}
      {...props} />)
  );
})
SidebarRail.displayName = "SidebarRail"

const SidebarInset = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<main
      ref={ref}
      className={cn(
        "relative flex min-h-svh flex-1 flex-col bg-background",
        "peer-data-[variant=inset]:min-h-[calc(100svh-theme(spacing.4))] md:peer-data-[variant=inset]:m-2
md:peer-data-[state=collapsed]:peer-data-[variant=inset]:ml-2 md:peer-data-[variant=inset]:ml-0
md:peer-data-[variant=inset]:rounded-xl md:peer-data-[variant=inset]:shadow",
        className
      )}
      {...props} />)
  );
})
SidebarInset.displayName = "SidebarInset"

const SidebarInput = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<Input
      ref={ref}
      data-sidebar="input"
      className={cn(
        "h-8 w-full bg-background shadow-none focus-visible:ring-2 focus-visible:ring-sidebar-ring",
        className
      )}
      {...props} />)
  );
})
SidebarInput.displayName = "SidebarInput"

const SidebarHeader = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="header"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props} />)
  );
})
SidebarHeader.displayName = "SidebarHeader"

const SidebarFooter = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="footer"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props} />)
  );
})
SidebarFooter.displayName = "SidebarFooter"

const SidebarSeparator = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<Separator
```

```
      ref={ref}
      data-sidebar="separator"
      className={cn("mx-2 w-auto bg-sidebar-border", className)}
      {...props} />)
  );
})
SidebarSeparator.displayName = "SidebarSeparator"

const SidebarContent = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="content"
      className={cn(
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto group-data-[collapsible=icon]:overflow-hidden",
        className
      )}
      {...props} />)
  );
})
SidebarContent.displayName = "SidebarContent"

const SidebarGroup = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props} />)
  );
})
SidebarGroup.displayName = "SidebarGroup"

const SidebarGroupLabel = React.forwardRef(({ className, asChild = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "div"

  return (
    (<Comp
      ref={ref}
      data-sidebar="group-label"
      className={cn(
        "flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium text-sidebar-foreground/70 outline-none
ring-sidebar-ring transition-[margin,opacity] duration-200 ease-linear focus-visible:ring-2 [&>svg]:size-4 [&>svg]:shrink-0",
        "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",
        className
      )}
      {...props} />)
  );
})
SidebarGroupLabel.displayName = "SidebarGroupLabel"

const SidebarGroupAction = React.forwardRef(({ className, asChild = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "button"

  return (
    (<Comp
      ref={ref}
      data-sidebar="group-action"
      className={cn(
        "absolute right-3 top-3.5 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreground
outline-none ring-sidebar-ring transition-transform hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
focus-visible:ring-2 [&>svg]:size-4 [&>svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 after:md:hidden",
        "group-data-[collapsible=icon]:hidden",
        className
      )}
      {...props} />)
  );
})
SidebarGroupAction.displayName = "SidebarGroupAction"

const SidebarGroupContent = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    data-sidebar="group-content"
    className={cn("w-full text-sm", className)}
    {...props} />
))
SidebarGroupContent.displayName = "SidebarGroupContent"

const SidebarMenu = React.forwardRef(({ className, ...props }, ref) => (
  <ul
    ref={ref}
    data-sidebar="menu"
    className={cn("flex w-full min-w-0 flex-col gap-1", className)}
    {...props} />
))
SidebarMenu.displayName = "SidebarMenu"

const SidebarMenuItem = React.forwardRef(({ className, ...props }, ref) => (
  <li
```

```
    ref={ref}
    data-sidebar="menu-item"
    className={cn("group/menu-item relative", className)}
    {...props} />
))
SidebarMenuItem.displayName = "SidebarMenuItem"

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm outline-none
ring-sidebar-ring transition-[width,height,padding] hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
focus-visible:ring-2 active:bg-sidebar-accent active:text-sidebar-accent-foreground disabled:pointer-events-none
disabled:opacity-50 group-has-[[data-sidebar=menu-action]]/menu-item:pr-8 aria-disabled:pointer-events-none
aria-disabled:opacity-50 data-[active=true]:bg-sidebar-accent data-[active=true]:font-medium
data-[active=true]:text-sidebar-accent-foreground data-[state=open]:hover:bg-sidebar-accent
data-[state=open]:hover:text-sidebar-accent-foreground group-data-[collapsible=icon]:!size-8 group-data-[collapsible=icon]:!p-2
[&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_0_1px_hsl(var(--sidebar-border))] hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground hover:shadow-[0_0_0_1px_hsl(var(--sidebar-accent))]",
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:!p-0",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

const SidebarMenuButton = React.forwardRef((
  {
    asChild = false,
    isActive = false,
    variant = "default",
    size = "default",
    tooltip,
    className,
    ...props
  },
  ref
) => {
  const Comp = asChild ? Slot : "button"
  const { isMobile, state } = useSidebar()

  const button = (
    <Comp
      ref={ref}
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props} />
  )

  if (!tooltip) {
    return button
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    }
  }

  return (
    (<Tooltip>
      <TooltipTrigger asChild>{button}</TooltipTrigger>
      <TooltipContent
        side="right"
        align="center"
        hidden={state !== "collapsed" || isMobile}
        {...tooltip} />
    </Tooltip>)
  );
})
SidebarMenuButton.displayName = "SidebarMenuButton"

const SidebarMenuAction = React.forwardRef(({ className, asChild = false, showOnHover = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "button"

  return (
    (<Comp
      ref={ref}
      data-sidebar="menu-action"
```

```
      className={cn(
        "absolute right-1 top-1.5 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreground
outline-none ring-sidebar-ring transition-transform hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
focus-visible:ring-2 peer-hover/menu-button:text-sidebar-accent-foreground [&>svg]:size-4 [&>svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 after:md:hidden",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        showOnHover &&
        "group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100 data-[state=open]:opacity-100
peer-data-[active=true]/menu-button:text-sidebar-accent-foreground md:opacity-0",
        className
      )}
      {...props} />)
  );
})
SidebarMenuAction.displayName = "SidebarMenuAction"

const SidebarMenuBadge = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    data-sidebar="menu-badge"
    className={cn(
      "pointer-events-none absolute right-1 flex h-5 min-w-5 select-none items-center justify-center rounded-md px-1 text-xs
font-medium tabular-nums text-sidebar-foreground",
      "peer-hover/menu-button:text-sidebar-accent-foreground
peer-data-[active=true]/menu-button:text-sidebar-accent-foreground",
      "peer-data-[size=sm]/menu-button:top-1",
      "peer-data-[size=default]/menu-button:top-1.5",
      "peer-data-[size=lg]/menu-button:top-2.5",
      "group-data-[collapsible=icon]:hidden",
      className
    )}
    {...props} />
))
SidebarMenuBadge.displayName = "SidebarMenuBadge"

const SidebarMenuSkeleton = React.forwardRef(({ className, showIcon = false, ...props }, ref) => {
  // Random width between 50 to 90%.
  const width = React.useMemo(() => {
    return `${Math.floor(Math.random() * 40) + 50}%`;
  }, [])

  return (
    (<div
      ref={ref}
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
      {...props}>
      {showIcon && (
        <Skeleton className="size-4 rounded-md" data-sidebar="menu-skeleton-icon" />
      )}
      <Skeleton
        className="h-4 max-w-[--skeleton-width] flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            "--skeleton-width": width
          }
        } />
    </div>)
  );
})
SidebarMenuSkeleton.displayName = "SidebarMenuSkeleton"

const SidebarMenuSub = React.forwardRef(({ className, ...props }, ref) => (
  <ul
    ref={ref}
    data-sidebar="menu-sub"
    className={cn(
      "mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l border-sidebar-border px-2.5 py-0.5",
      "group-data-[collapsible=icon]:hidden",
      className
    )}
    {...props} />
))
SidebarMenuSub.displayName = "SidebarMenuSub"

const SidebarMenuSubItem = React.forwardRef(({ ...props }, ref) => <li ref={ref} {...props} />)
SidebarMenuSubItem.displayName = "SidebarMenuSubItem"

const SidebarMenuSubButton = React.forwardRef(
  ({ asChild = false, size = "md", isActive, className, ...props }, ref) => {
    const Comp = asChild ? Slot : "a"

    return (
      (<Comp
        ref={ref}
        data-sidebar="menu-sub-button"
        data-size={size}
```

```
        data-active={isActive}
        className={cn(
          "flex h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden rounded-md px-2 text-sidebar-foreground
outline-none ring-sidebar-ring hover:bg-sidebar-accent hover:text-sidebar-accent-foreground focus-visible:ring-2
active:bg-sidebar-accent active:text-sidebar-accent-foreground disabled:pointer-events-none disabled:opacity-50
aria-disabled:pointer-events-none aria-disabled:opacity-50 [&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0
[&>svg]:text-sidebar-accent-foreground",
          "data-[active=true]:bg-sidebar-accent data-[active=true]:text-sidebar-accent-foreground",
          size === "sm" && "text-xs",
          size === "md" && "text-sm",
          "group-data-[collapsible=icon]:hidden",
          className
        )}
        {...props} />)
    );
  }
)
SidebarMenuSubButton.displayName = "SidebarMenuSubButton"

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
  SidebarTrigger,
  useSidebar,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/skeleton.jsx
------------------------------------------------------------
import { cn } from "@/lib/utils"

function Skeleton({
  className,
  ...props
}) {
  return (
    (<div
      className={cn("animate-pulse rounded-md bg-primary/10", className)}
      {...props} />)
  );
}

export { Skeleton }


------------------------------------------------------------
FICHIER : ./src/components/ui/slider.jsx
------------------------------------------------------------
import * as React from "react"
import * as SliderPrimitive from "@radix-ui/react-slider"

import { cn } from "@/lib/utils"

const Slider = React.forwardRef(({ className, ...props }, ref) => (
  <SliderPrimitive.Root
    ref={ref}
    className={cn("relative flex w-full touch-none select-none items-center", className)}
    {...props}>
    <SliderPrimitive.Track
      className="relative h-1.5 w-full grow overflow-hidden rounded-full bg-primary/20">
      <SliderPrimitive.Range className="absolute h-full bg-primary" />
    </SliderPrimitive.Track>
    <SliderPrimitive.Thumb
      className="block h-4 w-4 rounded-full border border-primary/50 bg-background shadow transition-colors
focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none disabled:opacity-50" />
  </SliderPrimitive.Root>
))
Slider.displayName = SliderPrimitive.Root.displayName

export { Slider }
```

```
------------------------------------------------------------
FICHIER : ./src/components/ui/sonner.jsx
------------------------------------------------------------
"use client";
import { useTheme } from "next-themes"
import { Toaster as Sonner } from "sonner"

const Toaster = ({
  ...props
}) => {
  const { theme = "system" } = useTheme()

  return (
    (<Sonner
      theme={theme}
      className="toaster group"
      toastOptions={{
        classNames: {
          toast:
            "group toast group-[.toaster]:bg-background group-[.toaster]:text-foreground group-[.toaster]:border-border
group-[.toaster]:shadow-lg",
          description: "group-[.toast]:text-muted-foreground",
          actionButton:
            "group-[.toast]:bg-primary group-[.toast]:text-primary-foreground",
          cancelButton:
            "group-[.toast]:bg-muted group-[.toast]:text-muted-foreground",
        },
      }}
      {...props} />)
  );
}

export { Toaster }


------------------------------------------------------------
FICHIER : ./src/components/ui/switch.jsx
------------------------------------------------------------
import * as React from "react"
import * as SwitchPrimitives from "@radix-ui/react-switch"

import { cn } from "@/lib/utils"

const Switch = React.forwardRef(({ className, ...props }, ref) => (
  <SwitchPrimitives.Root
    className={cn(
      "peer inline-flex h-5 w-9 shrink-0 cursor-pointer items-center rounded-full border-2 border-transparent shadow-sm
transition-colors focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2
focus-visible:ring-offset-background disabled:cursor-not-allowed disabled:opacity-50 data-[state=checked]:bg-primary
data-[state=unchecked]:bg-input",
      className
    )}
    {...props}
    ref={ref}>
    <SwitchPrimitives.Thumb
      className={cn(
        "pointer-events-none block h-4 w-4 rounded-full bg-background shadow-lg ring-0 transition-transform
data-[state=checked]:translate-x-4 data-[state=unchecked]:translate-x-0"
      )} />
  </SwitchPrimitives.Root>
))
Switch.displayName = SwitchPrimitives.Root.displayName

export { Switch }


------------------------------------------------------------
FICHIER : ./src/components/ui/table.jsx
------------------------------------------------------------
import * as React from "react"

import { cn } from "@/lib/utils"

const Table = React.forwardRef(({ className, ...props }, ref) => (
  <div className="relative w-full overflow-auto">
    <table
      ref={ref}
      className={cn("w-full caption-bottom text-sm", className)}
      {...props} />
  </div>
))
Table.displayName = "Table"

const TableHeader = React.forwardRef(({ className, ...props }, ref) => (
  <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
))
TableHeader.displayName = "TableHeader"

const TableBody = React.forwardRef(({ className, ...props }, ref) => (
  <tbody
    ref={ref}
```

```
    className={cn("[&_tr:last-child]:border-0", className)}
    {...props} />
))
TableBody.displayName = "TableBody"

const TableFooter = React.forwardRef(({ className, ...props }, ref) => (
  <tfoot
    ref={ref}
    className={cn("border-t bg-muted/50 font-medium [&>tr]:last:border-b-0", className)}
    {...props} />
))
TableFooter.displayName = "TableFooter"

const TableRow = React.forwardRef(({ className, ...props }, ref) => (
  <tr
    ref={ref}
    className={cn(
      "border-b transition-colors hover:bg-muted/50 data-[state=selected]:bg-muted",
      className
    )}
    {...props} />
))
TableRow.displayName = "TableRow"

const TableHead = React.forwardRef(({ className, ...props }, ref) => (
  <th
    ref={ref}
    className={cn(
      "h-10 px-2 text-left align-middle font-medium text-muted-foreground [&:has([role=checkbox])]:pr-0
[&>[role=checkbox]]:translate-y-[2px]",
      className
    )}
    {...props} />
))
TableHead.displayName = "TableHead"

const TableCell = React.forwardRef(({ className, ...props }, ref) => (
  <td
    ref={ref}
    className={cn(
      "p-2 align-middle [&:has([role=checkbox])]:pr-0 [&>[role=checkbox]]:translate-y-[2px]",
      className
    )}
    {...props} />
))
TableCell.displayName = "TableCell"

const TableCaption = React.forwardRef(({ className, ...props }, ref) => (
  <caption
    ref={ref}
    className={cn("mt-4 text-sm text-muted-foreground", className)}
    {...props} />
))
TableCaption.displayName = "TableCaption"

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}


------------------------------------------------------------
FICHIER : ./src/components/ui/tabs.jsx
------------------------------------------------------------
import * as React from "react"
import * as TabsPrimitive from "@radix-ui/react-tabs"

import { cn } from "@/lib/utils"

const Tabs = TabsPrimitive.Root

const TabsList = React.forwardRef(({ className, ...props }, ref) => (
  <TabsPrimitive.List
    ref={ref}
    className={cn(
      "inline-flex h-9 items-center justify-center rounded-lg bg-muted p-1 text-muted-foreground",
      className
    )}
    {...props} />
))
TabsList.displayName = TabsPrimitive.List.displayName

const TabsTrigger = React.forwardRef(({ className, ...props }, ref) => (
  <TabsPrimitive.Trigger
    ref={ref}
    className={cn(
```

```
      "inline-flex items-center justify-center whitespace-nowrap rounded-md px-3 py-1 text-sm font-medium ring-offset-background
transition-all focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2
disabled:pointer-events-none disabled:opacity-50 data-[state=active]:bg-background data-[state=active]:text-foreground
data-[state=active]:shadow",
      className
    )}
    {...props} />
))
TabsTrigger.displayName = TabsPrimitive.Trigger.displayName

const TabsContent = React.forwardRef(({ className, ...props }, ref) => (
  <TabsPrimitive.Content
    ref={ref}
    className={cn(
      "mt-2 ring-offset-background focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
      className
    )}
    {...props} />
))
TabsContent.displayName = TabsPrimitive.Content.displayName

export { Tabs, TabsList, TabsTrigger, TabsContent }


------------------------------------------------------------
FICHIER : ./src/components/ui/textarea.jsx
------------------------------------------------------------
import * as React from "react"

import { cn } from "@/lib/utils"

const Textarea = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<textarea
      className={cn(
        "flex min-h-[60px] w-full rounded-md border border-input bg-transparent px-3 py-2 text-base shadow-sm
placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring
disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
        className
      )}
      ref={ref}
      {...props} />)
  );
})
Textarea.displayName = "Textarea"

export { Textarea }


------------------------------------------------------------
FICHIER : ./src/components/ui/toast.jsx
------------------------------------------------------------
import * as React from "react";
import * as ToastPrimitives from "@radix-ui/react-toast";
import { cva } from "class-variance-authority";
import { X } from "lucide-react";
import { cn } from "@/lib/utils";

const ToastProvider = ToastPrimitives.Provider;

const ToastViewport = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Viewport
    ref={ref}
    className={cn(
      "fixed top-0 z-[100] flex max-h-screen w-full flex-col-reverse p-4 sm:bottom-0 sm:right-0 sm:top-auto sm:flex-col
md:max-w-[420px]",
      className
    )}
    {...props}
  />
));
ToastViewport.displayName = ToastPrimitives.Viewport.displayName;

const toastVariants = cva(
  "group pointer-events-auto relative flex w-full items-center justify-between space-x-4 overflow-hidden rounded-md border p-6
pr-8 shadow-lg transition-all data-[swipe=cancel]:translate-x-0 data-[swipe=end]:translate-x-[var(--radix-toast-swipe-end-x)]
data-[swipe=move]:translate-x-[var(--radix-toast-swipe-move-x)] data-[swipe=move]:transition-none data-[state=open]:animate-in
data-[state=closed]:animate-out data-[swipe=end]:animate-out data-[state=closed]:fade-out-80
data-[state=closed]:slide-out-to-right-full data-[state=open]:slide-in-from-top-full
data-[state=open]:sm:slide-in-from-bottom-full",
  {
    variants: {
      variant: {
        default: "border bg-background text-foreground",
        destructive:
          "destructive group border-destructive bg-destructive text-destructive-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
```

```
  }
);

const Toast = React.forwardRef(({ className, variant, ...props }, ref) => {
  return (
    <ToastPrimitives.Root
      ref={ref}
      className={cn(toastVariants({ variant }), className)}
      {...props}
    />
  );
});
Toast.displayName = ToastPrimitives.Root.displayName;

const ToastAction = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Action
    ref={ref}
    className={cn(
      "inline-flex h-8 shrink-0 items-center justify-center rounded-md border bg-transparent px-3 text-sm font-medium
ring-offset-background transition-colors hover:bg-secondary focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2
disabled:pointer-events-none disabled:opacity-50 group-[.destructive]:border-muted/40
group-[.destructive]:hover:border-destructive/30 group-[.destructive]:hover:bg-destructive
group-[.destructive]:hover:text-destructive-foreground group-[.destructive]:focus:ring-destructive",
      className
    )}
    {...props}
  />
));
ToastAction.displayName = ToastPrimitives.Action.displayName;

const ToastClose = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Close
    ref={ref}
    className={cn(
      "absolute right-2 top-2 rounded-md p-1 text-foreground/50 opacity-0 transition-opacity hover:text-foreground
focus:opacity-100 focus:outline-none focus:ring-2 group-hover:opacity-100 group-[.destructive]:text-red-300
group-[.destructive]:hover:text-red-50 group-[.destructive]:focus:ring-red-400 group-[.destructive]:focus:ring-offset-red-600",
      className
    )}
    toast-close=""
    {...props}
  >
    <X className="h-4 w-4" />
  </ToastPrimitives.Close>
));
ToastClose.displayName = ToastPrimitives.Close.displayName;

const ToastTitle = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Title
    ref={ref}
    className={cn("text-sm font-semibold", className)}
    {...props}
  />
));
ToastTitle.displayName = ToastPrimitives.Title.displayName;

const ToastDescription = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Description
    ref={ref}
    className={cn("text-sm opacity-90", className)}
    {...props}
  />
));
ToastDescription.displayName = ToastPrimitives.Description.displayName;

export {
  ToastProvider,
  ToastViewport,
  Toast,
  ToastTitle,
  ToastDescription,
  ToastClose,
  ToastAction,
};


------------------------------------------------------------
FICHIER : ./src/components/ui/toaster.jsx
------------------------------------------------------------
import { useToast } from "@/components/ui/use-toast";
import {
  Toast,
  ToastClose,
  ToastDescription,
  ToastProvider,
  ToastTitle,
  ToastViewport,
} from "@/components/ui/toast";

export function Toaster() {
  const { toasts } = useToast();
```

```
  return (
    <ToastProvider>
      {toasts.map(function ({ id, title, description, action, ...props }) {
        return (
          <Toast key={id} {...props}>
            <div className="grid gap-1">
              {title && <ToastTitle>{title}</ToastTitle>}
              {description && (
                <ToastDescription>{description}</ToastDescription>
              )}
            </div>
            {action}
            <ToastClose />
          </Toast>
        );
      })}
      <ToastViewport />
    </ToastProvider>
  );
}

------------------------------------------------------------
FICHIER : ./src/components/ui/toggle-group.jsx
------------------------------------------------------------
"use client";
import * as React from "react"
import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group"

import { cn } from "@/lib/utils"
import { toggleVariants } from "@/components/ui/toggle"

const ToggleGroupContext = React.createContext({
  size: "default",
  variant: "default",
})

const ToggleGroup = React.forwardRef(({ className, variant, size, children, ...props }, ref) => (
  <ToggleGroupPrimitive.Root
    ref={ref}
    className={cn("flex items-center justify-center gap-1", className)}
    {...props}>
    <ToggleGroupContext.Provider value={{ variant, size }}>
      {children}
    </ToggleGroupContext.Provider>
  </ToggleGroupPrimitive.Root>
))

ToggleGroup.displayName = ToggleGroupPrimitive.Root.displayName

const ToggleGroupItem = React.forwardRef(({ className, children, variant, size, ...props }, ref) => {
  const context = React.useContext(ToggleGroupContext)

  return (
    (<ToggleGroupPrimitive.Item
      ref={ref}
      className={cn(toggleVariants({
        variant: context.variant || variant,
        size: context.size || size,
      }), className)}
      {...props}>
      {children}
    </ToggleGroupPrimitive.Item>)
  );
})

ToggleGroupItem.displayName = ToggleGroupPrimitive.Item.displayName

export { ToggleGroup, ToggleGroupItem }


------------------------------------------------------------
FICHIER : ./src/components/ui/toggle.jsx
------------------------------------------------------------
import * as React from "react"
import * as TogglePrimitive from "@radix-ui/react-toggle"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const toggleVariants = cva(
  "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium transition-colors hover:bg-muted
hover:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none
disabled:opacity-50 data-[state=on]:bg-accent data-[state=on]:text-accent-foreground [&_svg]:pointer-events-none [&_svg]:size-4
[&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent shadow-sm hover:bg-accent hover:text-accent-foreground",
      },
      size: {
```

```
        default: "h-9 px-2 min-w-9",
        sm: "h-8 px-1.5 min-w-8",
        lg: "h-10 px-2.5 min-w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

const Toggle = React.forwardRef(({ className, variant, size, ...props }, ref) => (
  <TogglePrimitive.Root
    ref={ref}
    className={cn(toggleVariants({ variant, size, className }))}
    {...props} />
))

Toggle.displayName = TogglePrimitive.Root.displayName

export { Toggle, toggleVariants }


------------------------------------------------------------
FICHIER : ./src/components/ui/tooltip.jsx
------------------------------------------------------------
"use client"

import * as React from "react"
import * as TooltipPrimitive from "@radix-ui/react-tooltip"

import { cn } from "@/lib/utils"

const TooltipProvider = TooltipPrimitive.Provider

const Tooltip = TooltipPrimitive.Root

const TooltipTrigger = TooltipPrimitive.Trigger

const TooltipContent = React.forwardRef(({ className, sideOffset = 4, ...props }, ref) => (
  <TooltipPrimitive.Portal>
    <TooltipPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        "z-50 overflow-hidden rounded-md bg-primary px-3 py-1.5 text-xs text-primary-foreground animate-in fade-in-0 zoom-in-95
data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=closed]:zoom-out-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </TooltipPrimitive.Portal>
))
TooltipContent.displayName = TooltipPrimitive.Content.displayName

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider }


------------------------------------------------------------
FICHIER : ./src/components/ui/use-responsive-image.jsx
------------------------------------------------------------
import * as React from "react"
import { useSize } from "../../hooks/use-size"
import { cn } from "../../lib/utils"
import { DEFAULT_TRANSFORM_WIDTH, getImagePreviewClassName } from "./image-helpers"

export function useResponsiveImage({ parsed, fittingType, focalPoint, quality, className, onLoad, onSourceChange }, parentRef) {
  const wrapperRef = React.useRef(null)
  const imgRef = React.useRef(null)
  const size = useSize(wrapperRef)
  const [loaded, setLoaded] = React.useState(false)

  React.useImperativeHandle(parentRef, () => imgRef.current)
  React.useEffect(() => setLoaded(false), [parsed.baseUrl])
  React.useEffect(() => {
    const wrapper = wrapperRef.current
    const replace = (event) => onSourceChange(
      event.detail.src, getImagePreviewClassName(className, wrapper.className, cn("inline-block relative", className))
    )
    wrapper.addEventListener("base44:image-replace", replace)
    return () => wrapper.removeEventListener("base44:image-replace", replace)
  }, [className, onSourceChange])

  const crop = fittingType !== "fit"
  // Wait for useSize's pre-paint measurement before requesting a transform.
  const options = size && {
    width: size.width || DEFAULT_TRANSFORM_WIDTH,
    height: size.height || undefined,
    crop,
    focalPoint: crop ? focalPoint : undefined,
```

```
    quality,
  }

  return {
    wrapperRef, imgRef, loaded, options,
    handleLoad: (event) => {
      setLoaded(true)
      onLoad?.(event)
    },
  }
}


------------------------------------------------------------
FICHIER : ./src/components/ui/use-toast.jsx
------------------------------------------------------------
// Inspired by react-hot-toast library
import { useState, useEffect } from "react";

const TOAST_LIMIT = 20;
const TOAST_REMOVE_DELAY = 1000000;

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
};

let count = 0;

function genId() {
  count = (count + 1) % Number.MAX_VALUE;
  return count.toString();
}

const toastTimeouts = new Map();

const addToRemoveQueue = (toastId) => {
  if (toastTimeouts.has(toastId)) {
    return;
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId);
    dispatch({
      type: actionTypes.REMOVE_TOAST,
      toastId,
    });
  }, TOAST_REMOVE_DELAY);

  toastTimeouts.set(toastId, timeout);
};

const _clearFromRemoveQueue = (toastId) => {
  const timeout = toastTimeouts.get(toastId);
  if (timeout) {
    clearTimeout(timeout);
    toastTimeouts.delete(toastId);
  }
};

export const reducer = (state, action) => {
  switch (action.type) {
    case actionTypes.ADD_TOAST:
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      };

    case actionTypes.UPDATE_TOAST:
      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === action.toast.id ? { ...t, ...action.toast } : t
        ),
      };

    case actionTypes.DISMISS_TOAST: {
      const { toastId } = action;

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId);
      } else {
        state.toasts.forEach((toast) => {
          addToRemoveQueue(toast.id);
        });
      }

      return {
```

```
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t
        ),
      };
    }
    case actionTypes.REMOVE_TOAST:
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        };
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) => t.id !== action.toastId),
      };
  }
};

const listeners = [];

let memoryState = { toasts: [] };

function dispatch(action) {
  memoryState = reducer(memoryState, action);
  listeners.forEach((listener) => {
    listener(memoryState);
  });
}

function toast({ ...props }) {
  const id = genId();

  const update = (props) =>
    dispatch({
      type: actionTypes.UPDATE_TOAST,
      toast: { ...props, id },
    });

  const dismiss = () =>
    dispatch({ type: actionTypes.DISMISS_TOAST, toastId: id });

  dispatch({
    type: actionTypes.ADD_TOAST,
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss();
      },
    },
  });

  return {
    id,
    dismiss,
    update,
  };
}

function useToast() {
  const [state, setState] = useState(memoryState);

  useEffect(() => {
    listeners.push(setState);
    return () => {
      const index = listeners.indexOf(setState);
      if (index > -1) {
        listeners.splice(index, 1);
      }
    };
  }, [state]);

  return {
    ...state,
    toast,
    dismiss: (toastId) => dispatch({ type: actionTypes.DISMISS_TOAST, toastId }),
  };
}

export { useToast, toast };

------------------------------------------------------------
FICHIER : ./src/hooks/use-mobile.jsx
------------------------------------------------------------
```

```
import * as React from "react"

const MOBILE_BREAKPOINT = 768

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState(undefined)

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    }
    mql.addEventListener("change", onChange)
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    return () => mql.removeEventListener("change", onChange);
  }, [])

  return !!isMobile
}


------------------------------------------------------------
FICHIER : ./src/hooks/use-size.jsx
------------------------------------------------------------
import * as React from "react"

export function useSize(ref) {
  const [size, setSize] = React.useState(null)

  // useLayoutEffect (not useEffect): the initial measurement must land before
  // the browser paints, so consumers can render their real content on the
  // very first painted frame instead of a guess. A ResizeObserver's first
  // callback arrives too late for that — by then an <img> src guess has
  // already been dispatched to the network.
  React.useLayoutEffect(() => {
    const element = ref.current
    if (!element) return

    const rect = element.getBoundingClientRect()
    setSize({ width: rect.width, height: rect.height })

    const observer = new ResizeObserver(([entry]) => {
      const { width, height } = entry.contentRect
      setSize({ width, height })
    })

    observer.observe(element)
    return () => observer.disconnect()
  }, [ref])

  return size
}


------------------------------------------------------------
FICHIER : ./src/index.css
------------------------------------------------------------
@import url('https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,400;9..144,500;9..144,600;9..144,700&family=Inte
r:wght@400;500;600;700&display=swap');

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --background: 40 38% 97%;
    --foreground: 150 18% 12%;
    --card: 0 0% 100%;
    --card-foreground: 150 18% 12%;
    --popover: 0 0% 100%;
    --popover-foreground: 150 18% 12%;
    --primary: 152 32% 20%;
    --primary-foreground: 42 40% 97%;
    --secondary: 40 24% 92%;
    --secondary-foreground: 150 18% 15%;
    --muted: 40 22% 93%;
    --muted-foreground: 150 8% 42%;
    --accent: 36 62% 54%;
    --accent-foreground: 150 25% 12%;
    --destructive: 6 65% 48%;
    --destructive-foreground: 40 40% 97%;
    --border: 40 18% 87%;
    --input: 40 18% 87%;
    --ring: 152 32% 20%;
    --chart-1: 152 32% 32%;
    --chart-2: 36 62% 54%;
    --chart-3: 200 30% 40%;
    --chart-4: 12 55% 55%;
    --chart-5: 270 20% 50%;
    --radius: 0.85rem;
    --font-heading: 'Fraunces', ui-serif, Georgia, serif;
```

```
    --font-body: 'Inter', ui-sans-serif, system-ui, sans-serif;
    --font-display: 'Fraunces', ui-serif, Georgia, serif;
    --font-mono: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace;
    --sidebar-background: 152 30% 14%;
    --sidebar-foreground: 42 30% 92%;
    --sidebar-primary: 36 62% 54%;
    --sidebar-primary-foreground: 152 30% 14%;
    --sidebar-accent: 152 26% 20%;
    --sidebar-accent-foreground: 42 30% 92%;
    --sidebar-border: 152 24% 22%;
    --sidebar-ring: 36 62% 54%;
  }

  .dark {
    --background: 152 20% 8%;
    --foreground: 40 30% 94%;
    --card: 152 18% 11%;
    --card-foreground: 40 30% 94%;
    --popover: 152 18% 11%;
    --popover-foreground: 40 30% 94%;
    --primary: 40 40% 92%;
    --primary-foreground: 152 30% 14%;
    --secondary: 152 16% 16%;
    --secondary-foreground: 40 30% 94%;
    --muted: 152 16% 16%;
    --muted-foreground: 40 12% 65%;
    --accent: 36 62% 54%;
    --accent-foreground: 152 25% 12%;
    --destructive: 6 60% 42%;
    --destructive-foreground: 40 30% 94%;
    --border: 152 16% 18%;
    --input: 152 16% 18%;
    --ring: 36 62% 54%;
    --chart-1: 40 40% 80%;
    --chart-2: 36 62% 54%;
    --chart-3: 200 30% 60%;
    --chart-4: 12 55% 65%;
    --chart-5: 270 20% 65%;
    --sidebar-background: 152 22% 9%;
    --sidebar-foreground: 40 30% 92%;
    --sidebar-primary: 36 62% 54%;
    --sidebar-primary-foreground: 152 30% 14%;
    --sidebar-accent: 152 18% 15%;
    --sidebar-accent-foreground: 40 30% 92%;
    --sidebar-border: 152 18% 16%;
    --sidebar-ring: 36 62% 54%;
  }
}



@layer base {
  * {
    @apply border-border outline-ring/50;
  }

  html {
    -webkit-tap-highlight-color: transparent;
  }

  body {
    @apply bg-background text-foreground font-body;
    overscroll-behavior-y: none;
    -webkit-tap-highlight-color: transparent;
  }

  /* Native-style safe area padding for the main scroll container top */
  .safe-top {
    padding-top: env(safe-area-inset-top);
  }
  .safe-bottom {
    padding-bottom: env(safe-area-inset-bottom);
  }
}

@layer utilities {
  /* Touch controls: no accidental text selection, no tap flash, 44px min target */
  .no-select-tap {
    -webkit-user-select: none;
    user-select: none;
    -webkit-tap-highlight-color: transparent;
    -webkit-touch-callout: none;
  }
  .min-touch {
    min-height: 2.75rem;
    min-width: 2.75rem;
  }
  .no-scrollbar::-webkit-scrollbar {
    display: none;
  }
  .no-scrollbar {
    -ms-overflow-style: none;
```

```
    scrollbar-width: none;
  }
}

------------------------------------------------------------
FICHIER : ./src/lib/AuthContext.jsx
------------------------------------------------------------
import React, { createContext, useState, useContext, useEffect } from 'react';
import { base44 } from '@/api/base44Client';
import { appParams } from '@/lib/app-params';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [isLoadingAuth, setIsLoadingAuth] = useState(true);
  const [isLoadingPublicSettings, setIsLoadingPublicSettings] = useState(true);
  const [authError, setAuthError] = useState(null);
  const [authChecked, setAuthChecked] = useState(false);
  const [appPublicSettings, setAppPublicSettings] = useState(null); // Contains only { id, public_settings }

  useEffect(() => {
    checkAppState();
  }, []);

  const checkAppState = async () => {
    try {
      setIsLoadingPublicSettings(true);
      setAuthError(null);

      try {
        const publicSettings = await base44.app.getPublicSettings();
        setAppPublicSettings(publicSettings);

        // If we got the app public settings successfully, check if user is authenticated
        if (appParams.token) {
          await checkUserAuth();
        } else {
          setIsLoadingAuth(false);
          setIsAuthenticated(false);
          setAuthChecked(true);
        }
        setIsLoadingPublicSettings(false);
      } catch (appError) {
        console.error('App state check failed:', appError);

        // Handle app-level errors
        if (appError.status === 403 && appError.data?.extra_data?.reason) {
          const reason = appError.data.extra_data.reason;
          if (reason === 'auth_required') {
            setAuthError({
              type: 'auth_required',
              message: 'Authentication required'
            });
          } else if (reason === 'user_not_registered') {
            setAuthError({
              type: 'user_not_registered',
              message: 'User not registered for this app'
            });
          } else {
            setAuthError({
              type: reason,
              message: appError.message
            });
          }
        } else {
          setAuthError({
            type: 'unknown',
            message: appError.message || 'Failed to load app'
          });
        }
        setIsLoadingPublicSettings(false);
        setIsLoadingAuth(false);
      }
    } catch (error) {
      console.error('Unexpected error:', error);
      setAuthError({
        type: 'unknown',
        message: error.message || 'An unexpected error occurred'
      });
      setIsLoadingPublicSettings(false);
      setIsLoadingAuth(false);
    }
  };

  const checkUserAuth = async () => {
    try {
      // Now check if the user is authenticated
      setIsLoadingAuth(true);
      const currentUser = await base44.auth.me();
      setUser(currentUser);
```

```
      setIsAuthenticated(true);
      setIsLoadingAuth(false);
      setAuthChecked(true);
    } catch (error) {
      console.error('User auth check failed:', error);
      setIsLoadingAuth(false);
      setIsAuthenticated(false);
      setAuthChecked(true);

      // If user auth fails, it might be an expired token
      if (error.status === 401 || error.status === 403) {
        setAuthError({
          type: 'auth_required',
          message: 'Authentication required'
        });
      }
    }
  };

  const logout = (shouldRedirect = true) => {
    setUser(null);
    setIsAuthenticated(false);

    if (shouldRedirect) {
      // Use the SDK's logout method which handles token cleanup and redirect
      base44.auth.logout(window.location.href);
    } else {
      // Just remove the token without redirect
      base44.auth.logout();
    }
  };

  const navigateToLogin = () => {
    // Use the SDK's redirectToLogin method
    base44.auth.redirectToLogin(window.location.href);
  };

  return (
    <AuthContext.Provider value={{
      user,
      isAuthenticated,
      isLoadingAuth,
      isLoadingPublicSettings,
      authError,
      appPublicSettings,
      authChecked,
      logout,
      navigateToLogin,
      checkUserAuth,
      checkAppState
    }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};


------------------------------------------------------------
FICHIER : ./src/lib/PageNotFound.jsx
------------------------------------------------------------
import { useLocation } from 'react-router-dom';
import { base44 } from '@/api/base44Client';
import { useQuery } from '@tanstack/react-query';


export default function PageNotFound({}) {
    const location = useLocation();
    const pageName = location.pathname.substring(1);

    const { data: authData, isFetched } = useQuery({
        queryKey: ['user'],
        queryFn: async () => {
            try {
                const user = await base44.auth.me();
                return { user, isAuthenticated: true };
            } catch (error) {
                return { user: null, isAuthenticated: false };
            }
        }
    });

    return (
        <div className="min-h-screen flex items-center justify-center p-6 bg-slate-50">
            <div className="max-w-md w-full">
```

```
                <div className="text-center space-y-6">
                    {/* 404 Error Code */}
                    <div className="space-y-2">
                        <h1 className="text-7xl font-light text-slate-300">404</h1>
                        <div className="h-0.5 w-16 bg-slate-200 mx-auto"></div>
                    </div>

                    {/* Main Message */}
                    <div className="space-y-3">
                        <h2 className="text-2xl font-medium text-slate-800">
                            Page Not Found
                        </h2>
                        <p className="text-slate-600 leading-relaxed">
                            The page <span className="font-medium text-slate-700">"{pageName}"</span> could not be found in this
application.
                        </p>
                    </div>

                    {/* Admin Note */}
                    {isFetched && authData.isAuthenticated && authData.user?.role === 'admin' && (
                        <div className="mt-8 p-4 bg-slate-100 rounded-lg border border-slate-200">
                            <div className="flex items-start space-x-3">
                                <div className="flex-shrink-0 w-5 h-5 rounded-full bg-orange-100 flex items-center
justify-center mt-0.5">
                                    <div className="w-2 h-2 rounded-full bg-orange-400"></div>
                                </div>
                                <div className="text-left space-y-1">
                                    <p className="text-sm font-medium text-slate-700">Admin Note</p>
                                    <p className="text-sm text-slate-600 leading-relaxed">
                                        This could mean that the AI hasn't implemented this page yet. Ask it to implement it in
the chat.
                                    </p>
                                </div>
                            </div>
                        </div>
                    )}

                    {/* Action Button */}
                    <div className="pt-6">
                        <button
                            onClick={() => window.location.href = '/'}
                            className="inline-flex items-center px-4 py-2 text-sm font-medium text-slate-700 bg-white border
border-slate-200 rounded-lg hover:bg-slate-50 hover:border-slate-300 transition-colors duration-200 focus:outline-none
focus:ring-2 focus:ring-offset-2 focus:ring-slate-500"
                        >
                            <svg className="w-4 h-4 mr-2" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M3 12l2-2m0 0l7-7 7 7M5
10v10a1 1 0 001 1h3m10-11l2 2m-2-2v10a1 1 0 01-1 1h-3m-6 0a1 1 0 001-1v-4a1 1 0 011-1h2a1 1 0 011 1v4a1 1 0 001 1m-6 0h6" />
                            </svg>
                            Go Home
                        </button>
                    </div>
                </div>
            </div>
        </div>
    )
}

------------------------------------------------------------
FICHIER : ./src/lib/RefreshContext.jsx
------------------------------------------------------------
import React, { createContext, useContext, useRef, useCallback } from "react";

const RefreshContext = createContext({ register: () => {}, run: async () => {} });

export function RefreshProvider({ children }) {
  const ref = useRef(null);
  const register = useCallback((fn) => {
    ref.current = fn;
  }, []);
  const run = useCallback(async () => {
    if (ref.current) await Promise.resolve(ref.current());
  }, []);
  return <RefreshContext.Provider value={{ register, run }}>{children}</RefreshContext.Provider>;
}

export function useRefresh() {
  return useContext(RefreshContext);
}

------------------------------------------------------------
FICHIER : ./src/lib/SpaceContext.jsx
------------------------------------------------------------
import { createContext, useContext, useEffect, useState, useCallback } from "react";
import { base44 } from "@/api/base44Client";

const SpaceContext = createContext({
  space: "jpm",
  setSpace: () => {},
  availableSpaces: ["jpm"],
  activeCompanyId: null,
  setActiveCompanyId: () => {},
```

```
  companies: [],
  refreshCompanies: () => {},
  loading: true,
});

const SPACES = ["jpm", "jbm"];

const accountTypeToSpace = (t) => (t === "entreprise" ? "jbm" : "jpm");

export function SpaceProvider({ children }) {
  const [space, setSpaceState] = useState("jpm");
  const [availableSpaces, setAvailableSpaces] = useState(["jpm"]);
  const [isAdmin, setIsAdmin] = useState(false);
  const [activeCompanyId, setActiveCompanyIdState] = useState(null);
  const [companies, setCompanies] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);

  const load = useCallback(async () => {
    try {
      const [prefList, compList, me, subs] = await Promise.all([
        base44.entities.Preferences.list(),
        base44.entities.Company.list("-created_date", 100),
        base44.auth.me().catch(() => null),
        base44.entities.Subscription.filter({ status: "payee" }, { limit: 100 }).catch(() => ({})),
      ]);
      const pref = prefList?.[0] || null;
      setPreferences(pref);
      const accountType = me?.data?.account_type || me?.account_type;
      const admin = me?.role === "admin";
      setIsAdmin(admin);

      // Espaces auxquels l'utilisateur a droit : son type de compte courant +
      // tous les types d'abonnement payés (un utilisateur peut posséder les deux).
      const entitled = new Set();
      if (accountType) entitled.add(accountTypeToSpace(accountType));
      const subItems = subs?.items || subs || [];
      for (const s of subItems) {
        if (s && s.account_type) entitled.add(accountTypeToSpace(s.account_type));
      }
      const spaces = SPACES.filter((s) => entitled.has(s));
      const avail = spaces.length ? spaces : ["jpm"];
      setAvailableSpaces(admin ? SPACES : avail);

      // Admin : aperçu local via localStorage. Sinon : active_space persisté
      // s'il est autorisé, sinon le premier espace disponible.
      const preview =
        admin && typeof localStorage !== "undefined"
          ? localStorage.getItem("jm_preview_space")
          : null;
      let derived;
      if (admin) {
        derived = SPACES.includes(preview) ? preview : "jpm";
      } else {
        const saved = pref?.active_space;
        derived = avail.includes(saved) ? saved : avail[0];
      }
      setSpaceState(derived);
      setActiveCompanyIdState(pref?.active_company_id || compList?.[0]?.id || null);
      setCompanies(compList || []);
    } catch (e) {
      /* ignore */
    }
    setLoading(false);
  }, []);

  useEffect(() => { load(); }, [load]);

  const persist = async (patch) => {
    if (!preferences) {
      const created = await base44.entities.Preferences.create(patch);
      setPreferences(created);
      return created;
    }
    const updated = await base44.entities.Preferences.update(preferences.id, patch);
    setPreferences(updated);
    return updated;
  };

  const setSpace = useCallback((next) => {
    const value = SPACES.includes(next) ? next : "jpm";
    // Admin : aperçu libre (persisté en localStorage).
    if (isAdmin) {
      setSpaceState(value);
      try { localStorage.setItem("jm_preview_space", value); } catch (e) {}
      return;
    }
    // Utilisateur : bascule uniquement entre ses espaces autorisés.
    if (!availableSpaces.includes(value)) return;
    setSpaceState(value);
    persist({ active_space: value }).catch(() => {});
  }, [isAdmin, availableSpaces]);
```

```
  const setActiveCompanyId = useCallback((id) => {
    setActiveCompanyIdState(id);
    persist({ active_company_id: id }).catch(() => {});
  }, [preferences]);

  const refreshCompanies = useCallback(async () => {
    const compList = await base44.entities.Company.list("-created_date", 100);
    setCompanies(compList || []);
    if (!activeCompanyId && compList?.length) setActiveCompanyId(compList[0].id);
  }, [activeCompanyId]);

  return (
    <SpaceContext.Provider
      value={{
        space,
        setSpace,
        availableSpaces,
        activeCompanyId,
        setActiveCompanyId,
        companies,
        refreshCompanies,
        loading,
      }}
    >
      {children}
    </SpaceContext.Provider>
  );
}

export function useSpace() {
  return useContext(SpaceContext);
}

------------------------------------------------------------
FICHIER : ./src/lib/ThemeProvider.jsx
------------------------------------------------------------
import React, { createContext, useContext, useEffect, useState, useCallback } from "react";

const ThemeContext = createContext({ theme: "system", setTheme: () => {}, resolved: "light" });

const STORAGE_KEY = "jpm-theme";

function getSystemTheme() {
  if (typeof window === "undefined" || !window.matchMedia) return "light";
  return window.matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light";
}

function applyTheme(resolved) {
  const root = document.documentElement;
  if (resolved === "dark") {
    root.classList.add("dark");
  } else {
    root.classList.remove("dark");
  }
  root.style.colorScheme = resolved;
}

export function ThemeProvider({ children }) {
  const [theme, setThemeState] = useState(() => {
    if (typeof window === "undefined") return "system";
    return window.localStorage.getItem(STORAGE_KEY) || "system";
  });
  const [resolved, setResolved] = useState(() => (theme === "system" ? getSystemTheme() : theme));

  // Apply + keep in sync with system changes when in "system" mode.
  useEffect(() => {
    const r = theme === "system" ? getSystemTheme() : theme;
    setResolved(r);
    applyTheme(r);

    if (theme !== "system" || !window.matchMedia) return undefined;
    const mq = window.matchMedia("(prefers-color-scheme: dark)");
    const onChange = () => {
      const nr = getSystemTheme();
      setResolved(nr);
      applyTheme(nr);
    };
    mq.addEventListener("change", onChange);
    return () => mq.removeEventListener("change", onChange);
  }, [theme]);

  const setTheme = useCallback((next) => {
    setThemeState(next);
    try {
      window.localStorage.setItem(STORAGE_KEY, next);
    } catch (e) {
      /* ignore */
    }
  }, []);

  return (
```

```
    <ThemeContext.Provider value={{ theme, setTheme, resolved }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  return useContext(ThemeContext);
}

------------------------------------------------------------
FICHIER : ./src/lib/app-params.js
------------------------------------------------------------
import { getAccessToken } from '@base44/sdk';

const isNode = typeof window === 'undefined';

const isClearAccessTokenRequested = () =>
        !isNode && new URLSearchParams(window.location.search).get("clear_access_token") === 'true';

const clearStoredAccessToken = () => {
        window.localStorage.removeItem('base44_access_token');
        window.localStorage.removeItem('token');
}

const getAppParams = () => {
        if (isClearAccessTokenRequested()) {
                clearStoredAccessToken();
        }
        return {
                appId: import.meta.env.VITE_BASE44_APP_ID,
                token: getAccessToken(),
                functionsVersion: import.meta.env.VITE_BASE44_FUNCTIONS_VERSION,
                appBaseUrl: import.meta.env.VITE_BASE44_APP_BASE_URL,
        }
}


export const appParams = {
        ...getAppParams()
}


------------------------------------------------------------
FICHIER : ./src/lib/authReturnTo.js
------------------------------------------------------------
// Shared by the auth pages (Login, Register, and any page that resumes a flow
// after sign-in, e.g. the MCP OAuth consent page). Keep the redirect
// validation in one place — it is security-sensitive and easy to drift.

// Resolve ?returnTo= to a safe same-origin path, else "/".
//
// The same-origin check alone is not enough: a value like /.//evil.com or
// /\evil.com parses same-origin but normalizes to a protocol-relative
// //evil.com when assigned to location.href — an open redirect. So require the
// resolved path to be exactly one leading slash (no "//" prefix, no backslash).
export function safeReturnTo() {
  const raw = new URLSearchParams(window.location.search).get("returnTo");
  if (!raw) return "/";
  try {
    const url = new URL(raw, window.location.origin);
    if (url.origin !== window.location.origin) return "/";
    // Only access_token/clear_access_token are still URL-read by app-params.js, but the
    // whole bootstrap set stays stripped: one going back to a URL read must not silently
    // become injectable again. Normal app-flow params (e.g. the OAuth consent ctx) are kept.
    for (const p of ["access_token", "clear_access_token", "app_id", "app_base_url", "functions_version", "from_url"]) {
      url.searchParams.delete(p);
    }
    const path = url.pathname + url.search;
    if (!path.startsWith("/") || path.startsWith("//") || path.includes("\\")) return "/";
    return path;
  } catch {
    return "/";
  }
}


------------------------------------------------------------
FICHIER : ./src/lib/businessTypes.js
------------------------------------------------------------
export const BUSINESS_TYPES = [
  "Commerce",
  "Restauration",
  "Services",
  "Agriculture",
  "Industrie",
  "Artisanat",
  "Santé",
  "Transport",
  "BTP / Construction",
  "Technologie",
  "Éducation",
```

```
  "Tourisme / Hôtellerie",
  "Beauté / Bien-être",
  "Alimentation",
  "Autre",
];

------------------------------------------------------------
FICHIER : ./src/lib/calcSources.js
------------------------------------------------------------
import { base44 } from "@/api/base44Client";

// Predefined metrics the user can import as grandeurs, computed live from
// existing app data (Finances, Stock, Ventes, Factures).
export const METRIC_GROUPS = [
  {
    source: "finance",
    label: "Finances (JPM)",
    metrics: [
      { key: "budget_initial", label: "Budget initial", unit: "XAF" },
      { key: "total_depenses", label: "Total des dépenses", unit: "XAF" },
      { key: "depenses_mois", label: "Dépenses du mois", unit: "XAF" },
      { key: "emprunts_mensualite", label: "Mensualités d'emprunts", unit: "XAF" },
      { key: "emprunts_solde_restant", label: "Solde restant des emprunts", unit: "XAF" },
      { key: "emprunts_total_interets", label: "Intérêts totaux des emprunts", unit: "XAF" },
    ],
  },
  {
    source: "stock",
    label: "Stock (JBM)",
    metrics: [
      { key: "nb_produits", label: "Nombre de produits", unit: "" },
      { key: "valeur_stock", label: "Valeur du stock", unit: "XAF" },
      { key: "stock_total_qty", label: "Quantité totale en stock", unit: "" },
    ],
  },
  {
    source: "ventes",
    label: "Ventes (JBM)",
    metrics: [
      { key: "ca_ventes", label: "Chiffre d'affaires (ventes)", unit: "XAF" },
      { key: "nb_ventes", label: "Nombre de ventes", unit: "" },
    ],
  },
  {
    source: "factures",
    label: "Factures (JBM)",
    metrics: [
      { key: "total_facture", label: "Total facturé", unit: "XAF" },
      { key: "total_impaye", label: "Total impayé", unit: "XAF" },
      { key: "nb_factures_retard", label: "Factures en retard", unit: "" },
    ],
  },
  {
    source: "partage",
    label: "Éléments partagés",
    metrics: [
      { key: "budget_partage_total", label: "Budget partagé (total)", unit: "XAF" },
      { key: "budget_partage_mensuel", label: "Budget partagé récurrent (mensuel)", unit: "XAF" },
      { key: "nb_taches_partagees", label: "Tâches partagées", unit: "" },
      { key: "nb_taches_partagees_ouvertes", label: "Tâches partagées ouvertes", unit: "" },
      { key: "nb_evenements_partages", label: "Événements partagés", unit: "" },
      { key: "nb_partenariats_partages", label: "Partenariats partagés", unit: "" },
    ],
  },
];

export const ALL_METRICS = METRIC_GROUPS.flatMap((g) =>
  g.metrics.map((m) => ({ ...m, source: g.source }))
);
export const METRIC_BY_KEY = Object.fromEntries(
  ALL_METRICS.map((m) => [m.key, m])
);

export async function loadMetrics(activeCompanyId) {
  const [prefs, expenses, products, sales, invoices, tasks, events, partnerships, loans] = await Promise.all([
    base44.entities.Preferences.list().catch(() => []),
    base44.entities.Expense.list("-date", 500).catch(() => []),
    base44.entities.Product.list("-created_date", 500).catch(() => []),
    base44.entities.Sale.list("-created_date", 500).catch(() => []),
    base44.entities.Invoice.list("-created_date", 500).catch(() => []),
    base44.entities.Task.list("-created_date", 500).catch(() => []),
    base44.entities.Event.list("-created_date", 500).catch(() => []),
    base44.entities.Partnership.list("-created_date", 500).catch(() => []),
    base44.entities.Loan.list("-created_date", 500).catch(() => []),
  ]);
  const pref = prefs?.[0] || {};
  const comp = (list) => (activeCompanyId ? list.filter((x) => x.company_id === activeCompanyId) : list);
  const compProducts = comp(products);
  const compSales = comp(sales);
  const compInvoices = comp(invoices);

  const now = new Date();
```

```
  const mY = now.getFullYear();
  const mM = now.getMonth();
  const depMois = expenses.filter((e) => {
    const d = new Date(e.date);
    return d.getFullYear() === mY && d.getMonth() === mM;
  });

  const amt = (x) => Number(x.amount) || 0;
  const expenseAmt = (e) => (Number(e.quantity) || 1) * (Number(e.unit_cost) || 0);
  const saleAmt = (s) => (Number(s.quantity) || 1) * (Number(s.unit_price) || 0);
  const productVal = (p) => (Number(p.sale_price) || 0) * (Number(p.quantity) || 0);

  const todayStr = now.toISOString().slice(0, 10);
  const nbRetard = compInvoices.filter(
    (i) => i.status === "en_retard" || (i.due_date && i.due_date < todayStr && i.status !== "payee")
  ).length;

  // Éléments partagés (collaborations / groupes)
  const sharedExpenses = expenses.filter((e) => e.shared_context_id);
  const PERIOD_MONTH = { jour: 30, semaine: 4.33, trimestre: 1 / 3, mois: 1, annee: 1 / 12 };
  const monthlyEquiv = (e) => {
    const a = expenseAmt(e);
    return e.recurring && e.period ? a * (PERIOD_MONTH[e.period] || 0) : 0;
  };
  const sharedTasks = (tasks || []).filter((t) => t.shared_context_id);
  const sharedEvents = (events || []).filter((ev) => ev.shared_context_id);
  const sharedPartnerships = (partnerships || []).filter((p) => p.shared_context_id);

  return {
    budget_partage_total: sharedExpenses.reduce((s, e) => s + expenseAmt(e), 0),
    budget_partage_mensuel: sharedExpenses.reduce((s, e) => s + monthlyEquiv(e), 0),
    nb_taches_partagees: sharedTasks.length,
    nb_taches_partagees_ouvertes: sharedTasks.filter((t) => t.status !== "terminee").length,
    nb_evenements_partages: sharedEvents.length,
    nb_partenariats_partages: sharedPartnerships.length,
    budget_initial: Number(pref.initial_budget) || 0,
    total_depenses: expenses.reduce((s, e) => s + expenseAmt(e), 0),
    depenses_mois: depMois.reduce((s, e) => s + expenseAmt(e), 0),
    nb_produits: compProducts.length,
    valeur_stock: compProducts.reduce((s, p) => s + productVal(p), 0),
    stock_total_qty: compProducts.reduce((s, p) => s + (Number(p.quantity) || 0), 0),
    ca_ventes: compSales.reduce((s, x) => s + saleAmt(x), 0),
    nb_ventes: compSales.length,
    total_facture: compInvoices.reduce((s, i) => s + amt(i), 0),
    total_impaye: compInvoices.filter((i) => i.status !== "payee").reduce((s, i) => s + amt(i), 0),
    nb_factures_retard: nbRetard,
    emprunts_mensualite: (loans || []).filter((l) => l.status === "en_cours").reduce((s, l) => s + (Number(l.monthly_payment) ||
0), 0),
    emprunts_solde_restant: (loans || []).filter((l) => l.status === "en_cours").reduce((s, l) => s +
(Number(l.remaining_balance) || 0), 0),
    emprunts_total_interets: (loans || []).reduce((s, l) => s + (Number(l.total_interest) || 0), 0),
  };
}

------------------------------------------------------------
FICHIER : ./src/lib/collabUtils.js
------------------------------------------------------------
// Helpers de permissions pour les groupes de collaboration.

export function isGroupAdmin(userId, group) {
  if (!group) return false;
  return group.created_by_id === userId || (group.admins || []).includes(userId);
}

export function isGroupCreator(userId, group) {
  return group && group.created_by_id === userId;
}

export function canWriteInGroup(userId, group) {
  if (!group) return true;
  if (!group.write_restricted) return true;
  if (isGroupAdmin(userId, group)) return true;
  return (group.allowed_writers || []).includes(userId);
}

export function canSendMediaInGroup(userId, group) {
  if (!group) return true;
  if (!group.media_restricted) return true;
  if (isGroupAdmin(userId, group)) return true;
  return (group.allowed_media || []).includes(userId);
}

------------------------------------------------------------
FICHIER : ./src/lib/exportReport.js
------------------------------------------------------------
import { jsPDF } from "jspdf";
import { formatCurrency, formatDateFr } from "@/lib/format";

// rows: array of objects; columns: [{ key, label, type? }] type: "currency"|"date"|"text"
function cellValue(col, row, currency) {
  const v = row[col.key];
  if (col.type === "currency") return formatCurrency(Number(v) || 0, currency);
```

```
  if (col.type === "date") return v ? formatDateFr(v) : "";
  return v == null ? "" : String(v);
}

function stamp() {
  return new Date().toISOString().slice(0, 10);
}

export function exportToCsv(filename, columns, rows, currency) {
  const header = columns.map((c) => c.label).join(";");
  const lines = rows.map((r) =>
    columns
      .map((c) => {
        let val = cellValue(c, r, currency);
        // escape for CSV: wrap in quotes if contains ; or " or newline
        if (/[;"\n]/.test(val)) val = `"${val.replace(/"/g, '""')}"`;
        return val;
      })
      .join(";")
  );
  const csv = "\uFEFF" + [header, ...lines].join("\n");
  const blob = new Blob([csv], { type: "text/csv;charset=utf-8;" });
  triggerDownload(blob, `${filename}_${stamp()}.csv`);
}

export function exportToXls(filename, subtitle, columns, rows, currency, summary) {
  const esc = (s) => String(s == null ? "" : s).replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
  let html =
    '<html xmlns:o="urn:schemas-microsoft-com:office:office" xmlns:x="urn:schemas-microsoft-com:office:excel"
xmlns="http://www.w3.org/TR/REC-html40">' +
    '<head><meta charset="utf-8"><!--[if gte mso 9]><xml><x:ExcelWorkbook><x:ExcelWorksheets><x:ExcelWorksheet>' +
    `<x:Name>${esc(filename)}</x:Name><x:WorksheetOptions><x:DisplayGridlines/></x:WorksheetOptions>` +
    '</x:ExcelWorksheet></x:ExcelWorksheets></x:ExcelWorkbook></xml><![endif]--></head><body><table border="1">';
  html += `<tr><td colspan="${columns.length}"><b>${esc(filename)}</b></td></tr>`;
  if (subtitle) html += `<tr><td colspan="${columns.length}">${esc(subtitle)}</td></tr>`;
  if (summary && summary.length) {
    summary.forEach((s) => {
      html += `<tr><td colspan="${columns.length}"><b>${esc(s.label)}</b> : ${esc(s.value)}</td></tr>`;
    });
    html += `<tr><td colspan="${columns.length}"></td></tr>`;
  }
  html += "<tr>" + columns.map((c) => `<th style="background:#1f3a2e;color:#ffffff">${esc(c.label)}</th>`).join("") + "</tr>";
  rows.forEach((r) => {
    html += "<tr>" + columns.map((c) => `<td>${esc(cellValue(c, r, currency))}</td>`).join("") + "</tr>";
  });
  html += "</table></body></html>";
  const blob = new Blob(["\uFEFF", html], { type: "application/vnd.ms-excel;charset=utf-8;" });
  triggerDownload(blob, `${filename}_${stamp()}.xls`);
}

export function exportToPdf(title, subtitle, columns, rows, currency, summary) {
  const doc = new jsPDF({ orientation: "landscape", unit: "pt", format: "a4" });
  const pageW = doc.internal.pageSize.getWidth();
  const margin = 40;
  let y = margin;

  doc.setFont("helvetica", "bold");
  doc.setFontSize(16);
  doc.text(title, margin, y);
  y += 18;
  doc.setFont("helvetica", "normal");
  doc.setFontSize(10);
  doc.setTextColor(110);
  doc.text(subtitle, margin, y);
  y += 14;

  if (summary && summary.length) {
    summary.forEach((s) => {
      doc.text(`${s.label}: ${s.value}`, margin, y);
      y += 14;
    });
    y += 6;
  }
  doc.setTextColor(20);

  // Table
  const colW = Math.max(80, (pageW - margin * 2) / columns.length);
  const rowH = 18;

  // header
  doc.setFillColor(31, 58, 46);
  doc.setTextColor(255, 255, 255);
  doc.setFont("helvetica", "bold");
  doc.setFontSize(9);
  columns.forEach((c, i) => {
    doc.rect(margin + i * colW, y, colW, rowH, "F");
    doc.text(String(c.label).slice(0, 30), margin + i * colW + 6, y + 12);
  });
  y += rowH;

  doc.setFont("helvetica", "normal");
  doc.setTextColor(30);
```

```
  rows.forEach((r, ri) => {
    if (y > doc.internal.pageSize.getHeight() - margin) {
      doc.addPage();
      y = margin;
    }
    if (ri % 2 === 0) {
      doc.setFillColor(245, 243, 238);
      doc.rect(margin, y, columns.length * colW, rowH, "F");
    }
    columns.forEach((c, i) => {
      const val = cellValue(c, r, currency).slice(0, 40);
      doc.text(val, margin + i * colW + 6, y + 12, { maxWidth: colW - 8 });
    });
    y += rowH;
  });

  const pageCount = doc.internal.getNumberOfPages();
  for (let i = 1; i <= pageCount; i++) {
    doc.setPage(i);
    doc.setFontSize(8);
    doc.setTextColor(140);
    doc.text(`Document généré par JM — ${stamp()}`, margin, doc.internal.pageSize.getHeight() - 14);
    doc.text(`${i}/${pageCount}`, pageW - margin, doc.internal.pageSize.getHeight() - 14, { align: "right" });
  }

  doc.save(`${title.replace(/ /g, "_")}_${stamp()}.pdf`);
}

function triggerDownload(blob, filename) {
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  setTimeout(() => URL.revokeObjectURL(url), 1000);
}

------------------------------------------------------------
FICHIER : ./src/lib/format.js
------------------------------------------------------------
let _locale = "fr-FR";

export function setFormatLocale(lang) {
  _locale = lang === "en" ? "en-US" : "fr-FR";
}

export function formatCurrency(amount, currency = "XAF") {
  const value = Number(amount) || 0;
  try {
    return new Intl.NumberFormat(_locale, {
      style: "currency",
      currency,
      maximumFractionDigits: currency === "XAF" ? 0 : 2,
    }).format(value);
  } catch {
    return `${value.toLocaleString(_locale)} ${currency}`;
  }
}

export function formatDateFr(dateStr, withTime = false) {
  if (!dateStr) return "";
  const d = new Date(dateStr);
  if (isNaN(d.getTime())) return "";
  return d.toLocaleString(_locale, withTime ? { dateStyle: "medium", timeStyle: "short" } : { dateStyle: "medium" });
}

export const STATUS_LABELS = {
  a_faire: "À faire",
  en_cours: "En cours",
  terminee: "Terminée",
};

export const PRIORITY_LABELS = {
  basse: "Basse",
  normale: "Normale",
  haute: "Haute",
};

------------------------------------------------------------
FICHIER : ./src/lib/holidays.js
------------------------------------------------------------
import { format } from "date-fns";

// Calcul du dimanche de Pâques (algorithme de Meeus/Jones/Butcher)
function easter(year) {
  const a = year % 19;
  const b = Math.floor(year / 100);
  const c = year % 100;
  const d = Math.floor(b / 4);
  const e = b % 4;
```

```
  const f = Math.floor((b + 8) / 25);
  const g = Math.floor((b - f + 1) / 3);
  const h = (19 * a + b - d - g + 15) % 30;
  const i = Math.floor(c / 4);
  const k = c % 4;
  const l = (32 + 2 * e + 2 * i - h - k) % 7;
  const m = Math.floor((a + 11 * h + 22 * l) / 451);
  const month = Math.floor((h + l - 7 * m + 114) / 31);
  const day = ((h + l - 7 * m + 114) % 31) + 1;
  return new Date(year, month - 1, day);
}

const FIXED = [
  ["01-01", "Jour de l'An"],
  ["03-08", "Droits de la femme"],
  ["05-01", "Fête du Travail"],
  ["08-15", "Assomption"],
  ["08-17", "Fête nationale"],
  ["11-01", "Toussaint"],
  ["12-25", "Noël"],
];

// Retourne une map { "yyyy-MM-dd": "Nom du jour férié" } pour l'année donnée.
export function getHolidays(year) {
  const map = {};
  FIXED.forEach(([md, name]) => { map[`${year}-${md}`] = name; });
  const e = easter(year);
  const add = (offset, name) => {
    const d = new Date(e.getFullYear(), e.getMonth(), e.getDate() + offset);
    map[format(d, "yyyy-MM-dd")] = name;
  };
  add(1, "Lundi de Pâques");
  add(39, "Ascension");
  add(50, "Lundi de Pentecôte");
  return map;
}

------------------------------------------------------------
FICHIER : ./src/lib/i18n.jsx
------------------------------------------------------------
import React, { createContext, useContext, useState, useCallback, useEffect } from "react";
import { setFormatLocale } from "@/lib/format";

const STORAGE_KEY = "jpm-lang";

const translations = {
  fr: {
    "nav.home": "Accueil",
    "nav.finances": "Finances",
    "nav.tasks": "Tâches",
    "nav.planning": "Planning",
    "nav.calendar": "Calendrier",
    "nav.notifications": "Notifications",
    "nav.calculators": "Calculateurs",
    "nav.personalItems": "Effets personnels",
    "nav.thresholds": "Seuils",
    "nav.settings": "Paramètres",
    "nav.logout": "Déconnexion",

        "dashboard.greeting": "Bonjour Ø=ÜK",
    "dashboard.subtitle": "Voici la synthèse de votre journée.",
    "kpi.remaining": "Solde restant",
    "kpi.totalExpenses": "Dépenses totales",
    "kpi.todayTasks": "Tâches aujourd'hui",
    "kpi.overdue": "En retard",
    "dashboard.upcoming": "Prochaines échéances",
    "dashboard.viewPlanning": "Voir le planning",
    "dashboard.noUpcoming": "Aucune échéance à venir dans les {days} prochains jours.",
    "dashboard.expenseChart": "Dépenses par catégorie",
    "dashboard.expenseChartEmpty": "Aucune dépense à analyser pour l'instant.",
    "calendar.title": "Calendrier",
    "calendar.subtitle": "Vue mensuelle de vos tâches.",
    "calendar.prev": "Mois précédent",
    "calendar.next": "Mois suivant",
    "calendar.today": "Aujourd'hui",
    "calendar.add": "Ajouter une tâche",

    "tasks.title": "Tâches",
    "tasks.subtitle": "Gérez vos tâches et leurs échéances.",
    "tasks.new": "Nouvelle tâche",
    "tasks.filter.all": "Toutes",
    "tasks.filter.todo": "À faire",
    "tasks.filter.progress": "En cours",
    "tasks.filter.done": "Terminées",
    "tasks.empty": "Aucune tâche ici pour le moment.",
    "tasks.loadMore": "Charger plus ({n} restantes)",

    "planning.title": "Planning",
    "planning.subtitle": "Vue chronologique de vos tâches non terminées.",
        "planning.conflictWarn": "&{n} tâche(s) en conflit horaire.",
    "planning.empty": "Aucune tâche à planifier.",
    "planning.loadMore": "Charger plus ({n} restantes)",
```

```
    "finances.title": "Finances",
    "finances.subtitle": "Budget et dépenses personnelles.",
    "finances.add": "Ajouter",
    "finances.budget": "Budget initial",
    "finances.editBudget": "Modifier le budget",
    "finances.totalExpenses": "Dépenses totales",
    "finances.remaining": "Restant",
    "finances.history": "Historique des dépenses",
    "finances.empty": "Aucune dépense enregistrée pour le moment.",
    "finances.loadMore": "Charger plus ({n} restantes)",
    "finances.setBudget": "Définir le budget",
    "finances.budgetLabel": "Budget initial ({currency})",

    "settings.title": "Paramètres",
    "settings.subtitle": "Personnalisez votre expérience JPM.",
    "settings.currency": "Devise",
    "settings.language": "Langue",
    "settings.theme": "Thème d'affichage",
    "settings.budget": "Budget personnel initial",
    "settings.reminder": "Fenêtre des rappels (jours)",
    "settings.save": "Enregistrer",
    "settings.saving": "Enregistrement...",
    "settings.dangerZone": "Zone sensible",
    "settings.dangerDesc": "Supprime définitivement votre compte et toutes vos données (tâches, dépenses, préférences).",
    "settings.deleteAccount": "Supprimer mon compte",
    "settings.deleteTitle": "Supprimer définitivement votre compte ?",
    "settings.deleteDesc": "Cette action est irréversible. Toutes vos tâches, dépenses et préférences seront effacées pour
toujours.",
    "settings.confirmDelete": "Supprimer définitivement",
    "settings.deleting": "Suppression...",

    "theme.system": "Système",
    "theme.light": "Clair",
    "theme.dark": "Sombre",
    "lang.fr": "Français",
    "lang.en": "English",
    "currency.XAF": "XAF — Franc CFA",
    "currency.EUR": "EUR — Euro",
    "currency.USD": "USD — Dollar",

    "common.cancel": "Annuler",
    "common.delete": "Supprimer",
    "common.other": "Autre",
    "common.save": "Enregistrer",

    "taskform.edit": "Modifier la tâche",
    "taskform.new": "Nouvelle tâche",
    "taskform.name": "Nom de la tâche",
    "taskform.namePlaceholder": "Préparer rapport",
    "taskform.start": "Début",
    "taskform.end": "Fin",
    "taskform.priority": "Priorité",
    "taskform.status": "Statut",
    "taskform.add": "Ajouter la tâche",
    "taskform.err.title": "Le nom de la tâche est obligatoire.",
    "taskform.err.dates": "Dates invalides.",
    "taskform.err.endBeforeStart": "La date de fin doit être après la date de début.",

    "taskitem.toggleAria": "Basculer le statut terminé",
    "taskitem.editAria": "Modifier",
    "taskitem.deleteAria": "Supprimer",
        " t a s k i t e m . c o n f l i c t " :  "&   C o n f l i t  d é t e c t é " ,
    "taskitem.deleteTitle": "Supprimer cette tâche ?",
    "taskitem.deleteDesc": "Cette action est définitive. La tâche « {title} » sera supprimée pour toujours.",

    "expform.edit": "Modifier la dépense",
    "expform.new": "Ajouter une dépense",
    "expform.name": "Nom",
    "expform.namePlaceholder": "Courses",
    "expform.quantity": "Quantité",
    "expform.unitCost": "Coût unitaire",
    "expform.category": "Catégorie",
    "expform.categoryPlaceholder": "Alimentation",
    "expform.date": "Date",
    "expform.add": "Ajouter",
    "expform.err.name": "Le nom de la dépense est obligatoire.",
    "expform.err.qty": "La quantité doit être un nombre positif.",
    "expform.err.cost": "Le coût unitaire doit être positif.",

    "expitem.editAria": "Modifier",
    "expitem.deleteAria": "Supprimer",
    "expitem.deleteTitle": "Supprimer cette dépense ?",
    "expitem.deleteDesc": "Cette action est définitive. La dépense « {name} » sera supprimée pour toujours.",

    "toast.taskUpdated": "Tâche mise à jour",
    "toast.taskCreated": "Tâche créée",
    "toast.taskDeleted": "Tâche supprimée",
    "toast.taskFinished": "Tâche terminée",
    "toast.expenseUpdated": "Dépense mise à jour",
    "toast.expenseAdded": "Dépense ajoutée",
    "toast.expenseDeleted": "Dépense supprimée",
```

```
"toast.budgetUpdated": "Budget mis à jour",
"toast.prefsSaved": "Préférences enregistrées",
"toast.accountDeleted": "Compte supprimé",
"toast.accountDeletedDesc": "Vos données ont été effacées.",
"toast.partial": "Partiel",
"toast.partialDesc": "Données effacées, compte à supprimer manuellement.",
"toast.error": "Erreur",

"err.update": "Mise à jour impossible.",
"err.create": "Création impossible.",
"err.delete": "Suppression impossible.",
"err.modify": "Modification impossible.",
"err.budget": "Budget non enregistré.",
"err.save": "Enregistrement impossible.",
"err.deleteAccount": "Suppression impossible.",

"priority.basse": "Basse",
"priority.normale": "Normale",
"priority.haute": "Haute",

"nav.agents": "Assistants",
"agents.title": "Assistants",
"agents.subtitle": "Vos assistants intelligents JPM.",
"agents.newConversation": "Nouvelle conversation",
"agents.placeholder": "Écrivez votre message…",
"agents.thinking": "L'assistant réfléchit…",
"agents.empty": "Démarrez la conversation avec votre assistant.",
"agents.tool.params": "Paramètres",
"agents.tool.result": "Résultat",
"agents.tool.running": "En cours…",
"agents.finances.title": "Assistant Finances",
"agents.finances.desc": "Conseils budget et analyse des dépenses.",
"agents.tasks.title": "Assistant Tâches",
"agents.tasks.desc": "Planifiez et priorisez vos tâches.",
"agents.briefing.title": "Briefing Quotidien",
"agents.briefing.desc": "Synthèse de votre journée.",
"agents.notifications.title": "Assistant Notifications",
"agents.notifications.desc": "Crée des notifications personnalisées.",
"agents.rapports.title": "Assistant Rapports",
"agents.rapports.desc": "Génère des fichiers de rapport.",
"agents.rapports.title": "Assistant Rapports",
"agents.rapports.desc": "Génère des fichiers de rapport.",

"space.jpm": "Particulier",
"space.jbm": "Entreprise",

"jbm.dashboard": "Entreprise",
"jbm.dashboardSubtitle": "Pilotage de votre activité",
"jbm.nav.dashboard": "Accueil",
"jbm.nav.stock": "Stock",
"jbm.nav.sales": "Ventes",
"jbm.nav.personnel": "Personnel",
"jbm.nav.invoices": "Factures",
"jbm.nav.companies": "Entreprises",
"jbm.kpi.sales": "Ventes",
"jbm.kpi.stock": "Valeur stock",
"jbm.kpi.payroll": "Masse salariale",
"jbm.kpi.unpaid": "Factures impayées",
"jbm.dashboard.salesChart": "Ventes par mois",
"jbm.dashboard.salesChartEmpty": "Aucune vente à analyser pour l'instant.",
"jbm.dashboard.stockHint": "{n} produit(s) en stock",
"jbm.dashboard.invoiceHint": "{n} facture(s) émise(s)",
"jbm.dashboard.tips": "Points d'attention",
"jbm.dashboard.tipLowStock": "{n} produit(s) sous le seuil de réapprovisionnement.",
"jbm.dashboard.tipNoLowStock": "Aucune rupture de stock détectée.",
"jbm.dashboard.tipUnpaid": "{amount} de factures impayées à relancer.",
"jbm.dashboard.tipNoUnpaid": "Toutes les factures sont payées.",
"jbm.dashboard.tipAssistant": "Demandez conseil à l'assistant JBM.",
"jbm.add": "Ajouter",
"jbm.err.name": "Le nom est obligatoire.",
"jbm.err.saleFields": "L'article et la date sont obligatoires.",
"jbm.err.invoiceFields": "Le numéro, le client et la date sont obligatoires.",
"jbm.product.new": "Nouveau produit",
"jbm.product.edit": "Modifier le produit",
"jbm.product.name": "Nom",
"jbm.product.sku": "Référence",
"jbm.product.category": "Catégorie",
"jbm.product.quantity": "Quantité",
"jbm.product.reorder": "Seuil réappro.",
"jbm.product.unitCost": "Coût unitaire",
"jbm.product.salePrice": "Prix de vente",
"jbm.product.stock": "Stock",
"jbm.product.low": "Bas",
"jbm.product.deleteTitle": "Supprimer ce produit ?",
"jbm.product.deleteDesc": "Le produit « {name} » sera supprimé définitivement.",
"jbm.stock.title": "Stock",
"jbm.stock.subtitle": "Gérez votre inventaire.",
"jbm.stock.empty": "Aucun produit enregistré.",
"jbm.sale.new": "Nouvelle vente",
"jbm.sale.edit": "Modifier la vente",
"jbm.sale.item": "Article",
"jbm.sale.customer": "Client",
```

```
    "jbm.sale.quantity": "Quantité",
    "jbm.sale.unitPrice": "Prix unitaire",
    "jbm.sale.date": "Date",
    "jbm.sale.status": "Statut",
    "jbm.sale.statusPayee": "Payée",
    "jbm.sale.statusPending": "En attente",
    "jbm.sale.statusCancelled": "Annulée",
    "jbm.sale.deleteTitle": "Supprimer cette vente ?",
    "jbm.sale.deleteDesc": "La vente « {name} » sera supprimée définitivement.",
    "jbm.sales.title": "Ventes",
    "jbm.sales.subtitle": "Suivez vos ventes et encaissements.",
    "jbm.sales.empty": "Aucune vente enregistrée.",
    "jbm.employee.new": "Nouvel employé",
    "jbm.employee.edit": "Modifier l'employé",
    "jbm.employee.name": "Nom complet",
    "jbm.employee.role": "Poste",
    "jbm.employee.salary": "Salaire",
    "jbm.employee.hireDate": "Date d'embauche",
    "jbm.employee.phone": "Téléphone",
    "jbm.employee.email": "Email",
    "jbm.employee.status": "Statut",
    "jbm.employee.statusActive": "Actif",
    "jbm.employee.statusInactive": "Inactif",
    "jbm.employee.deleteTitle": "Supprimer cet employé ?",
    "jbm.employee.deleteDesc": "L'employé « {name} » sera supprimé définitivement.",
    "jbm.personnel.title": "Personnel",
    "jbm.personnel.subtitle": "Gérez vos collaborateurs.",
    "jbm.personnel.empty": "Aucun employé enregistré.",
    "jbm.invoice.new": "Nouvelle facture",
    "jbm.invoice.edit": "Modifier la facture",
    "jbm.invoice.number": "Numéro",
    "jbm.invoice.amount": "Montant",
    "jbm.invoice.customer": "Client",
    "jbm.invoice.issueDate": "Date d'émission",
    "jbm.invoice.dueDate": "Échéance",
    "jbm.invoice.status": "Statut",
    "jbm.invoice.statusDraft": "Brouillon",
    "jbm.invoice.statusSent": "Envoyée",
    "jbm.invoice.statusPaid": "Payée",
    "jbm.invoice.statusLate": "En retard",
    "jbm.invoice.deleteTitle": "Supprimer cette facture ?",
    "jbm.invoice.deleteDesc": "La facture « {number} » sera supprimée définitivement.",
    "jbm.invoices.title": "Factures",
    "jbm.invoices.subtitle": "Suivez vos factures et paiements.",
    "jbm.invoices.empty": "Aucune facture émise.",
    "jbm.companies.title": "Entreprises",
    "jbm.companies.subtitle": "Gérez vos entreprises et sélectionnez l'entreprise active.",
    "jbm.companies.new": "Nouvelle entreprise",
    "jbm.companies.edit": "Modifier l'entreprise",
    "jbm.companies.name": "Nom",
    "jbm.companies.sector": "Secteur",
    "jbm.companies.address": "Adresse",
    "jbm.companies.phone": "Téléphone",
    "jbm.companies.email": "Email",
    "jbm.companies.taxId": "Numéro fiscal / RC",
    "jbm.companies.empty": "Aucune entreprise. Créez-en une pour commencer.",
    "jbm.companies.deleteTitle": "Supprimer cette entreprise ?",
    "jbm.companies.deleteDesc": "L'entreprise « {name} » sera supprimée définitivement.",
    "agents.jbm.title": "Assistant Entreprise",
    "agents.jbm.desc": "Conseils ventes, stock, factures, personnel.",
    "agents.inventaire.title": "Assistant Inventaire",
    "agents.inventaire.desc": "Optimisez votre stock et réapprovisionnements.",
    "agents.rh.title": "Assistant RH",
    "agents.rh.desc": "Masse salariale et gestion du personnel.",

    "settings.legal": "Mentions légales",
    "settings.privacy": "Confidentialité",
    "settings.terms": "Conditions d'utilisation",
    "settings.about": "À propos de JM",

    "legal.privacyTitle": "Politique de confidentialité",
    "legal.privacyIntro": "JM (JH Manager) protège vos données personnelles et professionnelles. Cette page décrit ce que nous
collectons et comment nous l'utilisons.",
    "legal.privacyCollectTitle": "Données collectées",
    "legal.privacyCollect": "Nous collectons votre adresse e-mail, vos tâches, dépenses, préférences, et pour l'espace
entreprise : entreprises, produits, ventes, factures et personnel. Ces données appartiennent uniquement à votre compte.",
    "legal.privacyUseTitle": "Utilisation",
    "legal.privacyUse": "Vos données servent uniquement à afficher et gérer vos informations au sein de l'application. Elles ne
sont jamais vendues ni partagées avec des tiers à des fins commerciales.",
    "legal.privacyStorageTitle": "Stockage et sécurité",
    "legal.privacyStorage": "Les données sont stockées de manière sécurisée et isolées par compte (chaque utilisateur n'accède
qu'à ses propres enregistrements). Vous pouvez à tout moment supprimer votre compte et toutes vos données depuis les
Paramètres.",
    "legal.privacyRightsTitle": "Vos droits",
    "legal.privacyRights": "Vous pouvez consulter, modifier ou supprimer vos données à tout moment. Pour toute demande,
contactez le support.",
    "legal.termsTitle": "Conditions d'utilisation",
    "legal.termsIntro": "En utilisant JM (JH Manager), vous acceptez les conditions suivantes.",
    "legal.termsUseTitle": "Utilisation du service",
    "legal.termsUse": "L'application est fournie à titre d'outil de gestion. Vous êtes responsable de l'exactitude des données
que vous saisissez.",
    "legal.termsAccountsTitle": "Comptes",
```

```
    "legal.termsAccounts": "Vous devez fournir des informations exactes lors de la création de votre compte et en garantir la
confidentialité.",
    "legal.termsLiabilityTitle": "Responsabilité",
    "legal.termsLiability": "L'application est fournie « telle quelle ». Nous ne saurions être tenus responsables des pertes de
données ou décisions prises sur la base des informations affichées.",
    "legal.termsContactTitle": "Contact",
    "legal.termsContact": "Pour toute question relative à ces conditions, contactez le support.",
    "legal.back": "Retour à la connexion",

    "export.title": "Exporter",
    "export.pdf": "PDF",
    "export.excel": "Excel (CSV)",
    "export.empty": "Aucune donnée à exporter.",
    "report.sales": "Rapport des ventes",
    "report.salesSub": "Liste des ventes enregistrées",
    "report.stock": "Rapport de stock",
    "report.stockSub": "Inventaire des produits",
    "report.invoices": "Rapport des factures",
    "report.invoicesSub": "Factures émises",
    "report.personnel": "Rapport du personnel",
    "report.personnelSub": "Liste des employés",
    "report.expenses": "Rapport des dépenses",
    "report.expensesSub": "Dépenses personnelles enregistrées",

    "space.ecole": "École",
    "nav.clients": "Clients",
    "nav.quotes": "Devis",
    "nav.suppliers": "Prestataires",
    "nav.taxes": "Taxes",
    "nav.partnerships": "Partenariats",
    "nav.students": "Étudiants",
    "nav.classes": "Classes",
    "nav.subjects": "Matières",
    "nav.grades": "Notes",
    "nav.suggestions": "Suggestions",
    "nav.events": "Événements",
    "ecole.nav.dashboard": "Accueil",
    "ecole.nav.establishments": "Établissements"
  },
  en: {
    "nav.home": "Home",
    "nav.finances": "Finances",
    "nav.tasks": "Tasks",
    "nav.planning": "Planning",
    "nav.calendar": "Calendar",
    "nav.notifications": "Notifications",
    "nav.calculators": "Calculators",
    "nav.personalItems": "Personal items",
    "nav.thresholds": "Thresholds",
    "nav.settings": "Settings",
    "nav.logout": "Sign out",

        " d a s h b o a r d . g r e e t i n g " :   " H e l l o  Ø=ÜK " ,
    "dashboard.subtitle": "Here's your day at a glance.",
    "kpi.remaining": "Remaining balance",
    "kpi.totalExpenses": "Total expenses",
    "kpi.todayTasks": "Today's tasks",
    "kpi.overdue": "Overdue",
    "dashboard.upcoming": "Upcoming deadlines",
    "dashboard.viewPlanning": "View planning",
    "dashboard.noUpcoming": "No deadlines in the next {days} days.",
    "dashboard.expenseChart": "Expenses by category",
    "dashboard.expenseChartEmpty": "No expenses to analyze yet.",
    "calendar.title": "Calendar",
    "calendar.subtitle": "Monthly view of your tasks.",
    "calendar.prev": "Previous month",
    "calendar.next": "Next month",
    "calendar.today": "Today",
    "calendar.add": "Add a task",

    "tasks.title": "Tasks",
    "tasks.subtitle": "Manage your tasks and deadlines.",
    "tasks.new": "New task",
    "tasks.filter.all": "All",
    "tasks.filter.todo": "To do",
    "tasks.filter.progress": "In progress",
    "tasks.filter.done": "Done",
    "tasks.empty": "No tasks here yet.",
    "tasks.loadMore": "Load more ({n} left)",

    "planning.title": "Planning",
    "planning.subtitle": "Chronological view of your unfinished tasks.",
        " p l a n n i n g . c o n f l i c t W a r n " :   "&    { n }   t a s k ( s )   w i t h   s c h e d u l i n g   c o n f l i c
    "planning.empty": "No tasks to schedule.",
    "planning.loadMore": "Load more ({n} left)",

    "finances.title": "Finances",
    "finances.subtitle": "Personal budget and expenses.",
    "finances.add": "Add",
    "finances.budget": "Initial budget",
    "finances.editBudget": "Edit budget",
    "finances.totalExpenses": "Total expenses",
```

```
"finances.remaining": "Remaining",
"finances.history": "Expense history",
"finances.empty": "No expenses recorded yet.",
"finances.loadMore": "Load more ({n} left)",
"finances.setBudget": "Set budget",
"finances.budgetLabel": "Initial budget ({currency})",

"settings.title": "Settings",
"settings.subtitle": "Customize your JPM experience.",
"settings.currency": "Currency",
"settings.language": "Language",
"settings.theme": "Display theme",
"settings.budget": "Initial personal budget",
"settings.reminder": "Reminder window (days)",
"settings.save": "Save",
"settings.saving": "Saving...",
"settings.dangerZone": "Sensitive zone",
"settings.dangerDesc": "Permanently deletes your account and all your data (tasks, expenses, preferences).",
"settings.deleteAccount": "Delete my account",
"settings.deleteTitle": "Permanently delete your account?",
"settings.deleteDesc": "This action is irreversible. All your tasks, expenses and preferences will be erased forever.",
"settings.confirmDelete": "Delete permanently",
"settings.deleting": "Deleting...",

"theme.system": "System",
"theme.light": "Light",
"theme.dark": "Dark",
"lang.fr": "Français",
"lang.en": "English",
"currency.XAF": "XAF — CFA Franc",
"currency.EUR": "EUR — Euro",
"currency.USD": "USD — Dollar",

"common.cancel": "Cancel",
"common.delete": "Delete",
"common.other": "Other",
"common.save": "Save",

"taskform.edit": "Edit task",
"taskform.new": "New task",
"taskform.name": "Task name",
"taskform.namePlaceholder": "Prepare report",
"taskform.start": "Start",
"taskform.end": "End",
"taskform.priority": "Priority",
"taskform.status": "Status",
"taskform.add": "Add task",
"taskform.err.title": "Task name is required.",
"taskform.err.dates": "Invalid dates.",
"taskform.err.endBeforeStart": "End date must be after start date.",

"taskitem.toggleAria": "Toggle completed status",
"taskitem.editAria": "Edit",
"taskitem.deleteAria": "Delete",
    " t a s k i t e m . c o n f l i c t " :  "&   C o n f l i c t  d e t e c t e d " ,
"taskitem.deleteTitle": "Delete this task?",
"taskitem.deleteDesc": "This action is permanent. The task \"{title}\" will be deleted forever.",

"expform.edit": "Edit expense",
"expform.new": "Add expense",
"expform.name": "Name",
"expform.namePlaceholder": "Groceries",
"expform.quantity": "Quantity",
"expform.unitCost": "Unit cost",
"expform.category": "Category",
"expform.categoryPlaceholder": "Food",
"expform.date": "Date",
"expform.add": "Add",
"expform.err.name": "Expense name is required.",
"expform.err.qty": "Quantity must be a positive number.",
"expform.err.cost": "Unit cost must be positive.",

"expitem.editAria": "Edit",
"expitem.deleteAria": "Delete",
"expitem.deleteTitle": "Delete this expense?",
"expitem.deleteDesc": "This action is permanent. The expense \"{name}\" will be deleted forever.",

"toast.taskUpdated": "Task updated",
"toast.taskCreated": "Task created",
"toast.taskDeleted": "Task deleted",
"toast.taskFinished": "Task completed",
"toast.expenseUpdated": "Expense updated",
"toast.expenseAdded": "Expense added",
"toast.expenseDeleted": "Expense deleted",
"toast.budgetUpdated": "Budget updated",
"toast.prefsSaved": "Preferences saved",
"toast.accountDeleted": "Account deleted",
"toast.accountDeletedDesc": "Your data has been erased.",
"toast.partial": "Partial",
"toast.partialDesc": "Data erased, account must be deleted manually.",
"toast.error": "Error",
```

```
"err.update": "Could not update.",
"err.create": "Could not create.",
"err.delete": "Could not delete.",
"err.modify": "Could not modify.",
"err.budget": "Budget not saved.",
"err.save": "Could not save.",
"err.deleteAccount": "Could not delete.",

"priority.basse": "Low",
"priority.normale": "Normal",
"priority.haute": "High",

"nav.agents": "Assistants",
"agents.title": "Assistants",
"agents.subtitle": "Your smart JPM assistants.",
"agents.newConversation": "New conversation",
"agents.placeholder": "Type your message…",
"agents.thinking": "The assistant is thinking…",
"agents.empty": "Start a conversation with your assistant.",
"agents.tool.params": "Parameters",
"agents.tool.result": "Result",
"agents.tool.running": "Running…",
"agents.finances.title": "Finance Assistant",
"agents.finances.desc": "Budget advice and spending analysis.",
"agents.tasks.title": "Task Assistant",
"agents.tasks.desc": "Plan and prioritize your tasks.",
"agents.briefing.title": "Daily Briefing",
"agents.briefing.desc": "Your day, summarized.",
"agents.notifications.title": "Notifications Assistant",
"agents.notifications.desc": "Creates custom notifications.",

"space.jpm": "Personal",
"space.jbm": "Business",

"jbm.dashboard": "Business",
"jbm.dashboardSubtitle": "Run your activity",
"jbm.nav.dashboard": "Home",
"jbm.nav.stock": "Inventory",
"jbm.nav.sales": "Sales",
"jbm.nav.personnel": "Staff",
"jbm.nav.invoices": "Invoices",
"jbm.nav.companies": "Companies",
"jbm.kpi.sales": "Sales",
"jbm.kpi.stock": "Stock value",
"jbm.kpi.payroll": "Payroll",
"jbm.kpi.unpaid": "Unpaid invoices",
"jbm.dashboard.salesChart": "Sales by month",
"jbm.dashboard.salesChartEmpty": "No sales to analyze yet.",
"jbm.dashboard.stockHint": "{n} product(s) in stock",
"jbm.dashboard.invoiceHint": "{n} invoice(s) issued",
"jbm.dashboard.tips": "Key points",
"jbm.dashboard.tipLowStock": "{n} product(s) below reorder level.",
"jbm.dashboard.tipNoLowStock": "No stock shortages detected.",
"jbm.dashboard.tipUnpaid": "{amount} of unpaid invoices to chase.",
"jbm.dashboard.tipNoUnpaid": "All invoices are paid.",
"jbm.dashboard.tipAssistant": "Ask the JBM assistant for advice.",
"jbm.add": "Add",
"jbm.err.name": "Name is required.",
"jbm.err.saleFields": "Item and date are required.",
"jbm.err.invoiceFields": "Number, customer and date are required.",
"jbm.product.new": "New product",
"jbm.product.edit": "Edit product",
"jbm.product.name": "Name",
"jbm.product.sku": "SKU",
"jbm.product.category": "Category",
"jbm.product.quantity": "Quantity",
"jbm.product.reorder": "Reorder level",
"jbm.product.unitCost": "Unit cost",
"jbm.product.salePrice": "Sale price",
"jbm.product.stock": "Stock",
"jbm.product.low": "Low",
"jbm.product.deleteTitle": "Delete this product?",
"jbm.product.deleteDesc": "The product \"{name}\" will be permanently deleted.",
"jbm.stock.title": "Inventory",
"jbm.stock.subtitle": "Manage your stock.",
"jbm.stock.empty": "No products recorded.",
"jbm.sale.new": "New sale",
"jbm.sale.edit": "Edit sale",
"jbm.sale.item": "Item",
"jbm.sale.customer": "Customer",
"jbm.sale.quantity": "Quantity",
"jbm.sale.unitPrice": "Unit price",
"jbm.sale.date": "Date",
"jbm.sale.status": "Status",
"jbm.sale.statusPayee": "Paid",
"jbm.sale.statusPending": "Pending",
"jbm.sale.statusCancelled": "Cancelled",
"jbm.sale.deleteTitle": "Delete this sale?",
"jbm.sale.deleteDesc": "The sale \"{name}\" will be permanently deleted.",
"jbm.sales.title": "Sales",
"jbm.sales.subtitle": "Track your sales and receipts.",
"jbm.sales.empty": "No sales recorded.",
```

```
    "jbm.employee.new": "New employee",
    "jbm.employee.edit": "Edit employee",
    "jbm.employee.name": "Full name",
    "jbm.employee.role": "Role",
    "jbm.employee.salary": "Salary",
    "jbm.employee.hireDate": "Hire date",
    "jbm.employee.phone": "Phone",
    "jbm.employee.email": "Email",
    "jbm.employee.status": "Status",
    "jbm.employee.statusActive": "Active",
    "jbm.employee.statusInactive": "Inactive",
    "jbm.employee.deleteTitle": "Delete this employee?",
    "jbm.employee.deleteDesc": "The employee \"{name}\" will be permanently deleted.",
    "jbm.personnel.title": "Staff",
    "jbm.personnel.subtitle": "Manage your team.",
    "jbm.personnel.empty": "No employees recorded.",
    "jbm.invoice.new": "New invoice",
    "jbm.invoice.edit": "Edit invoice",
    "jbm.invoice.number": "Number",
    "jbm.invoice.amount": "Amount",
    "jbm.invoice.customer": "Customer",
    "jbm.invoice.issueDate": "Issue date",
    "jbm.invoice.dueDate": "Due date",
    "jbm.invoice.status": "Status",
    "jbm.invoice.statusDraft": "Draft",
    "jbm.invoice.statusSent": "Sent",
    "jbm.invoice.statusPaid": "Paid",
    "jbm.invoice.statusLate": "Late",
    "jbm.invoice.deleteTitle": "Delete this invoice?",
    "jbm.invoice.deleteDesc": "The invoice \"{number}\" will be permanently deleted.",
    "jbm.invoices.title": "Invoices",
    "jbm.invoices.subtitle": "Track your invoices and payments.",
    "jbm.invoices.empty": "No invoices issued.",
    "jbm.companies.title": "Companies",
    "jbm.companies.subtitle": "Manage your companies and select the active one.",
    "jbm.companies.new": "New company",
    "jbm.companies.edit": "Edit company",
    "jbm.companies.name": "Name",
    "jbm.companies.sector": "Sector",
    "jbm.companies.address": "Address",
    "jbm.companies.phone": "Phone",
    "jbm.companies.email": "Email",
    "jbm.companies.taxId": "Tax ID / RC",
    "jbm.companies.empty": "No company yet. Create one to start.",
    "jbm.companies.deleteTitle": "Delete this company?",
    "jbm.companies.deleteDesc": "The company \"{name}\" will be permanently deleted.",
    "agents.jbm.title": "Business Assistant",
    "agents.jbm.desc": "Advice on sales, stock, invoices, staff.",
    "agents.inventaire.title": "Inventory Assistant",
    "agents.inventaire.desc": "Optimize stock and reorders.",
    "agents.rh.title": "HR Assistant",
    "agents.rh.desc": "Payroll and staff management.",

    "settings.legal": "Legal",
    "settings.privacy": "Privacy",
    "settings.terms": "Terms of use",
    "settings.about": "About JM",

    "legal.privacyTitle": "Privacy Policy",
    "legal.privacyIntro": "JM (JH Manager) protects your personal and business data. This page describes what we collect and how we use it.",
    "legal.privacyCollectTitle": "Data collected",
    "legal.privacyCollect": "We collect your email address, tasks, expenses, preferences, and for the business space: companies, products, sales, invoices and staff. This data belongs only to your account.",
    "legal.privacyUseTitle": "Use",
    "legal.privacyUse": "Your data is used solely to display and manage your information within the app. It is never sold or shared with third parties for commercial purposes.",
    "legal.privacyStorageTitle": "Storage and security",
    "legal.privacyStorage": "Data is stored securely and isolated per account (each user only accesses their own records). You can delete your account and all your data at any time from Settings.",
    "legal.privacyRightsTitle": "Your rights",
    "legal.privacyRights": "You can view, modify or delete your data at any time. For any request, contact support.",
    "legal.termsTitle": "Terms of Use",
    "legal.termsIntro": "By using JM (JH Manager), you accept the following terms.",
    "legal.termsUseTitle": "Use of the service",
    "legal.termsUse": "The app is provided as a management tool. You are responsible for the accuracy of the data you enter.",
    "legal.termsAccountsTitle": "Accounts",
    "legal.termsAccounts": "You must provide accurate information when creating your account and keep it confidential.",
    "legal.termsLiabilityTitle": "Liability",
    "legal.termsLiability": "The app is provided \"as is\". We are not liable for data loss or decisions made based on the information displayed.",
    "legal.termsContactTitle": "Contact",
    "legal.termsContact": "For any question about these terms, contact support.",
    "legal.back": "Back to login",

    "export.title": "Export",
    "export.pdf": "PDF",
    "export.excel": "Excel (CSV)",
    "export.empty": "No data to export.",
    "report.sales": "Sales report",
    "report.salesSub": "Recorded sales",
    "report.stock": "Inventory report",
```

```
    "report.stockSub": "Product inventory",
    "report.invoices": "Invoices report",
    "report.invoicesSub": "Issued invoices",
    "report.personnel": "Staff report",
    "report.personnelSub": "Employee list",
    "report.expenses": "Expenses report",
    "report.expensesSub": "Recorded personal expenses",

    "space.ecole": "School",
    "nav.clients": "Clients",
    "nav.quotes": "Quotes",
    "nav.suppliers": "Suppliers",
    "nav.taxes": "Taxes",
    "nav.partnerships": "Partnerships",
    "nav.students": "Students",
    "nav.classes": "Classes",
    "nav.subjects": "Subjects",
    "nav.grades": "Grades",
    "nav.suggestions": "Suggestions",
    "nav.events": "Events",
    "ecole.nav.dashboard": "Home",
    "ecole.nav.establishments": "Establishments"
  }
};

function interpolate(str, vars) {
  if (!vars) return str;
  return str.replace(/\{(\w+)\}/g, (_, k) => (vars[k] !== undefined ? String(vars[k]) : `{${k}}`));
}

const I18nContext = createContext({ lang: "fr", setLang: () => {}, t: (k) => k });

export function I18nProvider({ children }) {
  const [lang, setLangState] = useState(() => {
    if (typeof window === "undefined") return "fr";
    return window.localStorage.getItem(STORAGE_KEY) || "fr";
  });

  useEffect(() => {
    const resolved = translations[lang] ? lang : "fr";
    try {
      window.localStorage.setItem(STORAGE_KEY, resolved);
    } catch (e) {
      /* ignore */
    }
    document.documentElement.lang = resolved;
    setFormatLocale(resolved);
  }, [lang]);

  const setLang = useCallback((next) => setLangState(next), []);
  const t = useCallback(
    (key, vars) => {
      const dict = translations[lang] || translations.fr;
      return interpolate(dict[key] ?? translations.fr[key] ?? key, vars);
    },
    [lang]
  );

  return (
    <I18nContext.Provider value={{ lang, setLang, t }}>
      {children}
    </I18nContext.Provider>
  );
}

export function useI18n() {
  return useContext(I18nContext);
}

------------------------------------------------------------
FICHIER : ./src/lib/lifeAdvice.js
------------------------------------------------------------
// Conseils de vie quotidiens sur les thèmes de l'application.
// Rotation par jour (un conseil par jour), sans appel IA -> économise les crédits d'intégration.

const JPM_ADVICE = [
  "Commencez votre journée par la tâche la plus difficile : le reste vous semblera facile.",
  "Fixez un budget hebdomadaire avant d'acheter : l'anticipation évite les dépenses impulsives.",
  "Une liste de tâches courte et terminée vaut mieux qu'une longue liste abandonnée.",
  "Mettez 10 % de chaque revenu de côté dès la réception : le futur vous remerciera.",
  "Planifiez vos repas de la semaine : vous mangerez mieux, dépenserez moins et gagnerez du temps.",
  "Distinguez l'urgent de l'important : l'urgent se gère, l'important se construit.",
  "Coupez les notifications une heure le matin pour conserver votre concentration sur l'essentiel.",
  "Marchez 20 minutes par jour : le corps et l'esprit vous le rendront au travail.",
  "Notez chaque soir trois choses accomplies : la motivation se nourrit de progrès visibles.",
  "Déléguer n'est pas céder : confier une tâche libère du temps pour décider mieux.",
  "Un agenda tenu vaut dix bons souvenirs : écrivez vos rendez-vous, ne vous fiez pas à la mémoire.",
  "Éteignez les écrans 30 minutes avant de dormir : le sommeil est votre meilleur investissement.",
  "Apprenez à dire non poliment : chaque « oui » de trop coûte du temps et de l'énergie.",
  "Revoyez vos abonnements chaque trimestre : les petits montants répétés vident un budget.",
];
```

```
const JBM_ADVICE = [
  "Un client satisfait revient : la qualité du service paie mieux que la publicité.",
  "Connaissez votre coût de revient avant de fixer un prix : vendre à perte ruine l'entreprise.",
  "Payez vos fournisseurs à temps : la confiance s'achète par la régularité, pas par les discours.",
  "Un stock qui ne tourne pas est de l'argent dormant : surveillez vos invendus.",
  "Recrutez sur l'attitude, formez sur les compétences : les bons caractères se conservent.",
  "Relancez vos factures impayées sans attendre : plus le temps passe, plus le recouvrement durcit.",
  "Tenez une trésorerie de 3 mois de charges : elle absorbe les coups durs sans panique.",
  "Un tableau de bord simple lu chaque semaine vaut un rapport complexe ignoré.",
  "Vos meilleurs commerciaux sont vos clients actuels : demandez-leur des recommandations.",
  "Séparez le compte pro du compte personnel : la clarté financière protège les deux.",
  "Un partenariat se construit sur l'intérêt mutuel, pas sur la dépendance.",
  "Formez un remplaçant pour chaque poste clé : l'entreprise doit survivre à chaque absence.",
  "Mesurez ce qui compte : ce qui n'est pas chiffré ne s'améliore pas.",
  "La réputation se construit en années et se perd en un jour : protégez-la sur chaque commande.",
];

export function getDailyLifeAdvice(space = "jpm") {
  const list = space === "jbm" ? JBM_ADVICE : JPM_ADVICE;
  const dayIndex = Math.floor(Date.now() / 86400000); // jours écoulés depuis 1970-01-01
  return list[dayIndex % list.length];
}

------------------------------------------------------------
FICHIER : ./src/lib/loanMath.js
------------------------------------------------------------
// Calculs d'amortissement d'emprunt (intérêts simples ou composés).

export const LOAN_FREQUENCIES = [
  { value: "mois", label: "Mensuel" },
  { value: "trimestre", label: "Trimestriel" },
  { value: "annee", label: "Annuel" },
];

export const LOAN_INTEREST_TYPES = [
  { value: "compose", label: "Composé (annuité constante)" },
  { value: "simple", label: "Simple (linéaire)" },
];

const rate = (r) => (Number(r) || 0) / 100;

// Mensualité / échéance constante.
export function computePayment(principal, ratePercent, term, interestType = "compose") {
  const P = Number(principal) || 0;
  const n = Number(term) || 0;
  if (P <= 0 || n <= 0) return 0;
  const r = rate(ratePercent);
  if (interestType === "simple" || r === 0) {
    // Linéaire : total = P + P*r*n ; échéance = total / n
    return (P + P * r * n) / n;
  }
  // Annuité constante : P * r * (1+r)^n / ((1+r)^n - 1)
  const f = Math.pow(1 + r, n);
  return (P * r * f) / (f - 1);
}

// Tableau d'amortissement complet.
export function buildSchedule(principal, ratePercent, term, interestType = "compose") {
  const P = Number(principal) || 0;
  const n = Math.max(0, Math.floor(Number(term) || 0));
  if (n <= 0) return [];
  const r = rate(ratePercent);
  const payment = computePayment(P, ratePercent, n, interestType);
  const schedule = [];
  let balance = P;
  for (let i = 1; i <= n; i++) {
    let interest, principalPart;
    if (interestType === "simple" || r === 0) {
      interest = P * r;
      principalPart = payment - interest;
    } else {
      interest = balance * r;
      principalPart = payment - interest;
    }
    balance = Math.max(0, balance - principalPart);
    schedule.push({
      n: i,
      payment,
      interest,
      principal: principalPart,
      balance,
    });
  }
  // Corrige le solde final à 0 (arrondis) sur la dernière ligne.
  if (schedule.length) schedule[schedule.length - 1].balance = 0;
  return schedule;
}

export function totalToRepay(principal, ratePercent, term, interestType) {
  return computePayment(principal, ratePercent, term, interestType) * (Number(term) || 0);
}
```

```
export function totalInterest(principal, ratePercent, term, interestType) {
  return totalToRepay(principal, ratePercent, term, interestType) - (Number(principal) || 0);
}

// Solde restant après `paymentsMade` échéances (d'après le tableau).
export function remainingAfter(schedule, paymentsMade) {
  if (!schedule || !schedule.length) return null;
  const idx = Math.min(Math.max(0, paymentsMade), schedule.length);
  if (idx === 0) return schedule[0].balance + schedule[0].principal; // capital initial
  return schedule[idx - 1].balance;
}

------------------------------------------------------------
FICHIER : ./src/lib/mathEval.js
------------------------------------------------------------
// Safe-ish math expression evaluator for the calculator.
// Supports user-defined variables and a whitelist of Math functions/constants.

const FUNCS = {
  sin: Math.sin, cos: Math.cos, tan: Math.tan,
  asin: Math.asin, acos: Math.acos, atan: Math.atan, atan2: Math.atan2,
  sinh: Math.sinh, cosh: Math.cosh, tanh: Math.tanh,
  sqrt: Math.sqrt, cbrt: Math.cbrt, abs: Math.abs,
  exp: Math.exp, ln: Math.log, log: Math.log10, log10: Math.log10, log2: Math.log2,
  pow: Math.pow, floor: Math.floor, ceil: Math.ceil, round: Math.round,
  min: Math.min, max: Math.max, sign: Math.sign,
};
const CONSTS = { pi: Math.PI, e: Math.E };

export function sanitizeName(n) {
  let s = (n || "").trim().replace(/ /g, "_").replace(/[^A-Za-z0-9_]/g, "");
  if (/^[0-9]/.test(s)) s = "_" + s;
  return s;
}

export function evalExpr(expr, scope = {}) {
  if (expr == null || String(expr).trim() === "") return NaN;
  const s = String(expr).replace(/\^/g, "**");
  const merged = { ...FUNCS, ...CONSTS, ...scope };
  const allowed = new Set(Object.keys(merged));
  const ids = s.match(/[A-Za-z_][A-Za-z0-9_]*/g) || [];
  for (const id of ids) {
    if (!allowed.has(id)) throw new Error(`Identifiant inconnu : « ${id} »`);
  }
  const keys = Object.keys(merged);
  const vals = keys.map((k) => merged[k]);
  const fn = new Function(...keys, `"use strict"; return (${s});`);
  const r = fn(...vals);
  return typeof r === "number" ? r : Number(r);
}

export function formatNumber(v, digits = 4) {
  if (v == null || !isFinite(v)) return "—";
  const n = Number(v);
  if (Number.isInteger(n)) return String(n);
  return Number(n.toFixed(digits)).toString();
}

export function sampleFunction(expr, variable, xMin, xMax, steps = 100, baseScope = {}) {
  const pts = [];
  const n = Math.max(2, Math.min(2000, steps));
  for (let i = 0; i <= n; i++) {
    const x = xMin + ((xMax - xMin) * i) / n;
    let y;
    try { y = evalExpr(expr, { ...baseScope, [variable]: x }); } catch { y = NaN; }
    pts.push({ x: +x.toFixed(6), y: isFinite(y) ? +y.toFixed(6) : null });
  }
  return pts;
}

export function studyFunction(expr, variable, xMin, xMax, steps = 200, baseScope = {}) {
  const pts = sampleFunction(expr, variable, xMin, xMax, steps, baseScope);
  const finite = pts.filter((p) => p.y != null);
  let min = Infinity, max = -Infinity, minX = 0, maxX = 0;
  for (const p of finite) {
    if (p.y < min) { min = p.y; minX = p.x; }
    if (p.y > max) { max = p.y; maxX = p.x; }
  }
  const roots = [];
  for (let i = 0; i < pts.length - 1; i++) {
    const a = pts[i], b = pts[i + 1];
    if (a.y == null || b.y == null) continue;
    if (a.y === 0) roots.push(a.x);
    else if ((a.y < 0 && b.y > 0) || (a.y > 0 && b.y < 0)) {
      roots.push(+(a.x - (a.y * (b.x - a.x)) / (b.y - a.y)).toFixed(4));
    }
  }
  const mid = xMin + (xMax - xMin) / 2;
  const h = Math.abs((xMax - xMin) / 1000) || 1e-3;
  let derivMid;
  try {
    const f1 = evalExpr(expr, { ...baseScope, [variable]: mid + h });
```

```
    const f2 = evalExpr(expr, { ...baseScope, [variable]: mid - h });
    derivMid = (f1 - f2) / (2 * h);
  } catch { derivMid = NaN; }
  const table = [];
  for (let i = 0; i <= 10; i++) {
    const x = xMin + ((xMax - xMin) * i) / 10;
    let y;
    try { y = evalExpr(expr, { ...baseScope, [variable]: x }); } catch { y = NaN; }
    table.push({ x: +x.toFixed(4), y: isFinite(y) ? +y.toFixed(4) : null });
  }
  return {
    min: isFinite(min) ? min : null, minX,
    max: isFinite(max) ? max : null, maxX,
    roots, derivMid, table,
  };
}

// Compute numeric values for a set of grandeurs, resolving manual values,
// linked indicators (metrics) and inter-grandeur formula dependencies.
export function computeGrandeurs(grandeurs, metrics = {}) {
  const scope = {};
  for (const g of grandeurs) {
    if (g.kind !== "grandeur") continue;
    if (g.source_type && g.source_type !== "manual" && g.source_key) {
      scope[g.name] = metrics[g.source_key] ?? 0;
    } else if (g.expression && String(g.expression).trim()) {
      scope[g.name] = undefined;
    } else {
      scope[g.name] = Number(g.value) || 0;
    }
  }
  for (let pass = 0; pass < 15; pass++) {
    let resolved = false;
    for (const g of grandeurs) {
      if (g.kind !== "grandeur") continue;
      if (g.source_type && g.source_type !== "manual" && g.source_key) continue;
      if (!g.expression || !String(g.expression).trim()) continue;
      if (scope[g.name] !== undefined) continue;
      const ids = String(g.expression).match(/[A-Za-z_][A-Za-z0-9_]*/g) || [];
      const local = {};
      let ready = true;
      for (const id of ids) {
        if (FUNCS[id] !== undefined || CONSTS[id] !== undefined) continue;
        if (id in scope) {
          if (scope[id] === undefined) { ready = false; break; }
          local[id] = scope[id];
        }
      }
      if (!ready) continue;
      try {
        const r = evalExpr(g.expression, local);
        if (isFinite(r)) { scope[g.name] = r; resolved = true; }
      } catch { /* expression invalid */ }
    }
    if (!resolved) break;
  }
  for (const k in scope) if (scope[k] === undefined) scope[k] = NaN;
  return scope;
}

------------------------------------------------------------
FICHIER : ./src/lib/pageTransitions.js
------------------------------------------------------------
// Lightweight fade/slide page transition variants for framer-motion.
export const pageVariants = {
  initial: { opacity: 0, y: 8 },
  animate: { opacity: 1, y: 0 },
  exit: { opacity: 0, y: -8 },
};

export const pageTransition = {
  duration: 0.22,
  ease: "easeOut",
};

------------------------------------------------------------
FICHIER : ./src/lib/periods.js
------------------------------------------------------------
// Système unifié de périodicité : jour, semaine, mois, trimestre, semestre, année, personnalisée.

export const PERIOD_OPTIONS = [
  { value: "jour", label: "Jour" },
  { value: "semaine", label: "Semaine" },
  { value: "mois", label: "Mois" },
  { value: "trimestre", label: "Trimestre" },
  { value: "semestre", label: "Semestre" },
  { value: "annee", label: "Année" },
  { value: "personnalise", label: "Personnalisée" },
];

export const PERIOD_LABELS = {
  jour: "jour",
```

```
  semaine: "semaine",
  mois: "mois",
  trimestre: "trimestre",
  semestre: "semestre",
  annee: "an",
  personnalise: "perso.",
};

export const CUSTOM_UNITS = [
  { value: "jour", label: "jours" },
  { value: "semaine", label: "semaines" },
  { value: "mois", label: "mois" },
  { value: "annee", label: "années" },
];

// Nombre de jours couverts par une période démarrant à `refDate`.
// `custom` = { count, unit } utilisé seulement quand period === "personnalise".
export function daysInPeriod(period, refDate = new Date(), custom = null) {
  const start = new Date(refDate);
  const s = new Date(start.getFullYear(), start.getMonth(), start.getDate());
  const end = new Date(s);
  switch (period) {
    case "jour": end.setDate(end.getDate() + 1); break;
    case "semaine": end.setDate(end.getDate() + 7); break;
    case "mois": end.setMonth(end.getMonth() + 1); break;
    case "trimestre": end.setMonth(end.getMonth() + 3); break;
    case "semestre": end.setMonth(end.getMonth() + 6); break;
    case "annee": end.setFullYear(end.getFullYear() + 1); break;
    case "personnalise": {
      const n = Math.max(1, Number(custom?.count) || 0);
      const unit = custom?.unit || "jour";
      if (unit === "jour") end.setDate(end.getDate() + n);
      else if (unit === "semaine") end.setDate(end.getDate() + n * 7);
      else if (unit === "mois") end.setMonth(end.getMonth() + n);
      else if (unit === "annee") end.setFullYear(end.getFullYear() + n);
      break;
    }
    default: end.setMonth(end.getMonth() + 1);
  }
  return Math.max(1, Math.round((end - s) / 86400000));
}

// Libellé humain d'une période (gère le personnalisé).
export function periodLabel(period, custom = null) {
  if (period === "personnalise" && custom?.count) {
    const unit = CUSTOM_UNITS.find((u) => u.value === custom.unit);
    return `${custom.count} ${unit?.label || ""}`;
  }
  return PERIOD_LABELS[period] || period || "mois";
}

// Normalise un enregistrement (qui peut stocker custom_count/custom_unit) en objet custom.
export function customOf(record = {}) {
  return {
    count: Number(record?.custom_count) || 0,
    unit: record?.custom_unit || "jour",
  };
}

------------------------------------------------------------
FICHIER : ./src/lib/query-client.js
------------------------------------------------------------
import { QueryClient } from '@tanstack/react-query';


export const queryClientInstance = new QueryClient({
        defaultOptions: {
                queries: {
                        refetchOnWindowFocus: false,
                        retry: 1,
                },
        },
});

------------------------------------------------------------
FICHIER : ./src/lib/recurrence.js
------------------------------------------------------------
// Helpers pour générer des échéances récurrentes sur tout type de période.
// Périodes supportées : jour, semaine, mois, trimestre, semestre, année, personnalise.

import { PERIOD_OPTIONS, PERIOD_LABELS, CUSTOM_UNITS } from "@/lib/periods";

export { PERIOD_OPTIONS, PERIOD_LABELS, CUSTOM_UNITS };

// Compatibilité : liste plate des fréquences (sans "personnalise" pour les selects simples anciens).
export const FREQUENCIES = PERIOD_OPTIONS;

function applyShift(d, frequency, steps, custom) {
  switch (frequency) {
    case "jour": d.setDate(d.getDate() + steps); break;
    case "semaine": d.setDate(d.getDate() + steps * 7); break;
    case "trimestre": d.setMonth(d.getMonth() + steps * 3); break;
```

```
    case "semestre": d.setMonth(d.getMonth() + steps * 6); break;
    case "annee": d.setFullYear(d.getFullYear() + steps); break;
    case "personnalise": {
      const n = Math.max(1, Number(custom?.count) || 0);
      const unit = custom?.unit || "jour";
      if (unit === "jour") d.setDate(d.getDate() + steps * n);
      else if (unit === "semaine") d.setDate(d.getDate() + steps * n * 7);
      else if (unit === "mois") d.setMonth(d.getMonth() + steps * n);
      else if (unit === "annee") d.setFullYear(d.getFullYear() + steps * n);
      break;
    }
    case "mois":
    default: d.setMonth(d.getMonth() + steps); break;
  }
}

export function shiftDateBy(dateStr, frequency, steps, custom) {
  const d = new Date(dateStr);
  applyShift(d, frequency, steps, custom);
  return d.toISOString().slice(0, 10);
}

export function shiftISOBy(iso, frequency, steps, custom) {
  const d = new Date(iso);
  applyShift(d, frequency, steps, custom);
  return d.toISOString();
}

// Compatibilité avec l'ancien usage mensuel.
export function shiftMonthDate(dateStr, months) {
  return shiftDateBy(dateStr, "mois", months);
}
export function shiftMonthISO(iso, months) {
  return shiftISOBy(iso, "mois", months);
}

// Génère `count` occurrences en décalant les champs `fields` (date ou datetime).
// `custom` = { count, unit } utilisé quand frequency === "personnalise".
export function expandRecurrence(base, frequency, count, fields, custom) {
  if (!frequency || !count || count <= 1) return [base];
  const items = [];
  for (let i = 0; i < count; i++) {
    const item = { ...base };
    fields.forEach((f) => {
      const v = base[f];
      if (!v) return;
      item[f] = String(v).includes("T") ? shiftISOBy(v, frequency, i, custom) : shiftDateBy(v, frequency, i, custom);
    });
    items.push(item);
  }
  return items;
}

------------------------------------------------------------
FICHIER : ./src/lib/reportPeriods.js
------------------------------------------------------------
// Calcul de plages de dates pour l'export de rapports sur n'importe quelle période.

export const PERIOD_TYPES = [
  { value: "semaine", label: "Semaine" },
  { value: "mois", label: "Mois" },
  { value: "trimestre", label: "Trimestre" },
  { value: "semestre", label: "Semestre" },
  { value: "annee", label: "Année" },
  { value: "personnalisee", label: "Personnalisée" },
];

const MONTHS = ["Janvier", "Février", "Mars", "Avril", "Mai", "Juin", "Juillet", "Août", "Septembre", "Octobre", "Novembre",
"Décembre"];

export function yearOptions() {
  const y = new Date().getFullYear();
  return Array.from({ length: 7 }, (_, i) => y - 5 + i);
}

export const monthOptions = MONTHS.map((m, i) => ({ value: i, label: m }));
export const quarterOptions = [1, 2, 3, 4].map((q) => ({ value: q, label: `T${q}` }));
export const halfOptions = [1, 2].map((h) => ({ value: h, label: `S${h}` }));
export const weekOptions = Array.from({ length: 53 }, (_, i) => i + 1);

export function computeRange(periodType, p) {
  const now = new Date();
  const y = Number(p?.year) || now.getFullYear();
  let start, end, label;

  if (periodType === "annee") {
    start = new Date(y, 0, 1);
    end = new Date(y, 11, 31, 23, 59, 59, 999);
    label = `Année ${y}`;
  } else if (periodType === "mois") {
    const m = p?.month != null ? Number(p.month) : now.getMonth();
    start = new Date(y, m, 1);
```

```
    end = new Date(y, m + 1, 0, 23, 59, 59, 999);
    label = `${MONTHS[m]} ${y}`;
  } else if (periodType === "trimestre") {
    const q = Number(p?.quarter) || 1;
    const m0 = (q - 1) * 3;
    start = new Date(y, m0, 1);
    end = new Date(y, m0 + 3, 0, 23, 59, 59, 999);
    label = `T${q} ${y}`;
  } else if (periodType === "semestre") {
    const h = Number(p?.half) || 1;
    const m0 = h === 1 ? 0 : 6;
    start = new Date(y, m0, 1);
    end = new Date(y, m0 + 6, 0, 23, 59, 59, 999);
    label = `S${h} ${y}`;
  } else if (periodType === "semaine") {
    const w = Number(p?.week) || 1;
    // Lundi de la semaine ISO w pour l'année y (le 4 janvier est toujours en semaine 1)
    const jan4 = new Date(y, 0, 4);
    const day1 = new Date(jan4);
    day1.setDate(jan4.getDate() - ((jan4.getDay() + 6) % 7) + (w - 1) * 7);
    start = new Date(day1);
    end = new Date(day1);
    end.setDate(day1.getDate() + 6);
    end.setHours(23, 59, 59, 999);
    label = `Semaine ${w} ${y}`;
  } else {
    // personnalisée
    start = p?.from ? new Date(p.from + "T00:00:00") : new Date(0);
    end = p?.to ? new Date(p.to + "T23:59:59") : now;
        l a b e l   =   ` $ { p ? . f r o m   | |   " — " }  !’   $ { p ? . t o   | |   " — " } ` ;
  }

  return { start, end, label };
}

------------------------------------------------------------
FICHIER : ./src/lib/thresholds.js
------------------------------------------------------------
// thresholds: array of Threshold records { domain, medium_value, critical_value }
// direction "high": value above threshold is worse (invoices, overdue tasks)
// direction "low": value below threshold is worse (budget remaining, stock value)
export function getLevel(domain, value, thresholds, direction = "high") {
  const th = thresholds?.find((x) => x.domain === domain);
  if (!th) return "none";
  const v = Number(value) || 0;
  const med = Number(th.medium_value) || 0;
  const crit = Number(th.critical_value) || 0;
  if (direction === "low") {
    if (v <= crit) return "critique";
    if (v <= med) return "moyen";
    return "normal";
  }
  if (v >= crit) return "critique";
  if (v >= med) return "moyen";
  return "normal";
}

export const LEVEL_STYLE = {
  normal: "bg-emerald-100 text-emerald-700",
  moyen: "bg-amber-100 text-amber-700",
  critique: "bg-red-100 text-red-700",
  none: "bg-secondary text-muted-foreground",
};

export const LEVEL_LABEL = {
  normal: "Normal",
  moyen: "Moyen",
  critique: "Critique",
  none: "—",
};

------------------------------------------------------------
FICHIER : ./src/lib/utils.js
------------------------------------------------------------
import { clsx } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs) {
  return twMerge(clsx(inputs))
}


export const isIframe = window.self !== window.top;


------------------------------------------------------------
FICHIER : ./src/main.jsx
------------------------------------------------------------
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from '@/App.jsx'
import '@/index.css'
```

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <App />
)

------------------------------------------------------------
FICHIER : ./src/pages/Admin.jsx
------------------------------------------------------------
import React, { useEffect, useMemo, useState } from "react";
import { base44 } from "@/api/base44Client";
import { useAuth } from "@/lib/AuthContext";
import { Users, Search, ShieldAlert, Wallet } from "lucide-react";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import KpiCard from "@/components/dashboard/KpiCard";
import SubscribersByCompanyChart from "@/components/admin/SubscribersByCompanyChart";
import SubscriberDetail from "@/components/admin/SubscriberDetail";
import AdminSuggestions from "@/components/admin/AdminSuggestions";
import AdminFaq from "@/components/admin/AdminFaq";
import AdminResubscription from "@/components/admin/AdminResubscription";
import AdminReports from "@/components/admin/AdminReports";
import { formatCurrency } from "@/lib/format";

// Restreint l'accès à toi et ton collaborateur.
// Ajoutez vos emails ici pour activer le filtre ; tant que la liste est vide,
// tous les utilisateurs ayant le rôle "admin" y ont accès.
// Exemple : ["moi@example.com", "associe@example.com"]
const ADMIN_EMAILS = [];

const PAGE_SIZE = 10;

export default function Admin() {
  const { user } = useAuth();
  const [users, setUsers] = useState([]);
  const [subs, setSubs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [query, setQuery] = useState("");
  const [sort, setSort] = useState("newest");
  const [onlySubs, setOnlySubs] = useState(true);
  const [page, setPage] = useState(0);
  const [selected, setSelected] = useState(null);

  const allowed =
    user?.role === "admin" &&
    (ADMIN_EMAILS.length === 0 || ADMIN_EMAILS.includes(user.email));

  useEffect(() => {
    if (!allowed) {
      setLoading(false);
      return;
    }
    (async () => {
      try {
        const [u, s] = await Promise.all([
          base44.entities.User.list("-created_date", 500),
          base44.entities.Subscription.list("-paid_at", 500),
        ]);
        setUsers(u || []);
        setSubs(s || []);
      } catch (e) {
        setUsers([]);
        setSubs([]);
      } finally {
        setLoading(false);
      }
    })();
  }, [allowed]);

  const enriched = useMemo(() => {
    return users.map((u) => {
      const mySubs = subs.filter((s) => s.created_by_id === u.id);
      const accountType =
        u.data?.account_type || u.account_type || mySubs[0]?.account_type || null;
      const totalPaid = mySubs.reduce((sum, s) => sum + (Number(s.amount) || 0), 0);
      const companies = mySubs.length
        ? Math.max(...mySubs.map((s) => Number(s.companies_count) || 1))
        : 0;
      return { ...u, accountType, totalPaid, companies, subsCount: mySubs.length };
    });
  }, [users, subs]);

  const subscribers = enriched.filter(
    (u) => u.accountType === "particulier" || u.accountType === "entreprise"
  );
  const jpm = subscribers.filter((u) => u.accountType === "particulier");
  const jbm = subscribers.filter((u) => u.accountType === "entreprise");
  const totalRevenue = subs.reduce((sum, s) => sum + (Number(s.amount) || 0), 0);
  const caJpm = subs.filter((s) => s.account_type === "particulier").reduce((sum, s) => sum + (Number(s.amount) || 0), 0);
  const caJbm = subs.filter((s) => s.account_type === "entreprise").reduce((sum, s) => sum + (Number(s.amount) || 0), 0);

  const byCompany = useMemo(() => {
    const map = {};
```

```
  jbm.forEach((u) => {
    const c = u.companies || 1;
    map[c] = (map[c] || 0) + 1;
  });
  return Object.entries(map)
    .map(([companies, count]) => ({ companies: `${companies} entr.`, count }))
    .sort((a, b) => parseInt(a.companies) - parseInt(b.companies));
}, [jbm]);

const filtered = useMemo(() => {
  let list = onlySubs ? subscribers : enriched;
  const q = query.trim().toLowerCase();
  if (q) {
    list = list.filter(
      (u) =>
        (u.email || "").toLowerCase().includes(q) ||
        (u.full_name || "").toLowerCase().includes(q)
    );
  }
  const sorted = [...list];
  switch (sort) {
    case "oldest":
      sorted.sort((a, b) => new Date(a.created_date) - new Date(b.created_date));
      break;
    case "type":
      sorted.sort((a, b) => (a.accountType || "").localeCompare(b.accountType || ""));
      break;
    case "companies":
      sorted.sort((a, b) => (b.companies || 0) - (a.companies || 0));
      break;
    case "paid":
      sorted.sort((a, b) => (b.totalPaid || 0) - (a.totalPaid || 0));
      break;
    default:
      sorted.sort((a, b) => new Date(b.created_date) - new Date(a.created_date));
  }
  return sorted;
}, [enriched, subscribers, onlySubs, query, sort]);

const pageCount = Math.max(1, Math.ceil(filtered.length / PAGE_SIZE));
const safePage = Math.min(page, pageCount - 1);
const pageItems = filtered.slice(safePage * PAGE_SIZE, safePage * PAGE_SIZE + PAGE_SIZE);

if (!allowed) {
  return (
    <div className="flex flex-col items-center justify-center h-64 text-center">
      <ShieldAlert className="w-10 h-10 text-destructive mb-3" />
      <h1 className="font-display text-xl">Accès restreint</h1>
      <p className="text-sm text-muted-foreground mt-1">
        Cette section est réservée aux administrateurs autorisés.
      </p>
    </div>
  );
}

if (loading) {
  return (
    <div className="flex items-center justify-center h-64">
      <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
    </div>
  );
}

return (
  <div className="space-y-6">
    <div>
      <h1 className="font-display text-3xl">Tableau de bord administrateur</h1>
      <p className="text-muted-foreground mt-1">Vue d'ensemble des abonnés et des revenus.</p>
    </div>

    <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
      <KpiCard icon={Users} label="Abonnés" value={subscribers.length} tone="default" />
      <KpiCard icon={Wallet} label="CA JPM" value={formatCurrency(caJpm, "XAF")} tone="default" />
      <KpiCard icon={Wallet} label="CA JBM" value={formatCurrency(caJbm, "XAF")} tone="accent" />
      <KpiCard icon={Wallet} label="Revenus encaissés" value={formatCurrency(totalRevenue, "XAF")} tone="default" />
    </div>

    <AdminSuggestions />

    <AdminFaq />

    <AdminReports />

    <AdminResubscription />

    {byCompany.length > 0 && <SubscribersByCompanyChart data={byCompany} />}

    <div className="bg-card border border-border rounded-2xl p-4 space-y-3">
      <div className="flex flex-col sm:flex-row gap-3 sm:items-center">
        <div className="relative flex-1">
          <Search className="w-4 h-4 absolute left-3 top-1/2 -translate-y-1/2 text-muted-foreground" />
          <Input
```

```
              value={query}
              onChange={(e) => { setQuery(e.target.value); setPage(0); }}
              placeholder="Rechercher par email ou nom…"
              className="pl-9"
            />
          </div>
          <Select value={sort} onValueChange={setSort}>
            <SelectTrigger className="sm:w-52"><SelectValue /></SelectTrigger>
            <SelectContent>
              <SelectItem value="newest">Plus récents</SelectItem>
              <SelectItem value="oldest">Plus anciens</SelectItem>
              <SelectItem value="type">Par type de compte</SelectItem>
              <SelectItem value="companies">Par nb d'entreprises</SelectItem>
              <SelectItem value="paid">Par montant payé</SelectItem>
            </SelectContent>
          </Select>
          <button
            onClick={() => { setOnlySubs((v) => !v); setPage(0); }}
            className={`text-xs px-3 py-2 rounded-lg border no-select-tap ${onlySubs ? "bg-primary text-primary-foreground
border-primary" : "bg-transparent"}`}
          >
            {onlySubs ? "Abonnés seulement" : "Tous les utilisateurs"}
          </button>
        </div>

        <div className="divide-y divide-border">
          {pageItems.length === 0 ? (
            <p className="text-sm text-muted-foreground py-6 text-center">Aucun résultat.</p>
          ) : (
            pageItems.map((u) => (
              <button
                key={u.id}
                onClick={() => setSelected(u)}
                className="w-full flex items-center justify-between gap-3 py-3 text-left hover:bg-secondary/40 rounded-lg px-2
no-select-tap"
              >
                <div className="min-w-0">
                  <p className="text-sm font-medium truncate">{u.full_name || u.email}</p>
                  <p className="text-xs text-muted-foreground truncate">{u.email}</p>
                </div>
                <div className="flex items-center gap-3 shrink-0 text-xs">
                  <span className={`px-2 py-1 rounded-full ${u.accountType === "entreprise" ? "bg-accent/15
text-accent-foreground" : u.accountType === "particulier" ? "bg-primary/10" : "bg-secondary"}`}>
                    {u.accountType === "entreprise" ? "JBM" : u.accountType === "particulier" ? "JPM" : "—"}
                  </span>
                  {u.accountType === "entreprise" && <span className="text-muted-foreground">{u.companies} entr.</span>}
                  <span className="text-muted-foreground">{formatCurrency(u.totalPaid, "XAF")}</span>
                </div>
              </button>
            ))
          )}
        </div>

        {pageCount > 1 && (
          <div className="flex items-center justify-between pt-2">
            <button
              disabled={safePage === 0}
              onClick={() => setPage((p) => p - 1)}
              className="text-sm px-3 py-2 rounded-lg border disabled:opacity-40 no-select-tap"
            >
              Précédent
            </button>
            <span className="text-xs text-muted-foreground">Page {safePage + 1} / {pageCount}</span>
            <button
              disabled={safePage >= pageCount - 1}
              onClick={() => setPage((p) => p + 1)}
              className="text-sm px-3 py-2 rounded-lg border disabled:opacity-40 no-select-tap"
            >
              Suivant
            </button>
          </div>
        )}
      </div>

      <SubscriberDetail
        user={selected}
        subscriptions={subs}
        open={!!selected}
        onOpenChange={(o) => !o && setSelected(null)}
      />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Affiliation.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Gift, Copy, Check, Loader2, Users, Wallet } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
```

```
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useToast } from "@/components/ui/use-toast";
import { formatCurrency, formatDateFr } from "@/lib/format";

const genCode = () => "JM" + Math.random().toString(36).slice(2, 8).toUpperCase();

export default function Affiliation() {
  const { toast } = useToast();
  const [affiliate, setAffiliate] = useState(null);
  const [commissions, setCommissions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [phone, setPhone] = useState("");
  const [provider, setProvider] = useState("airtel");
  const [creating, setCreating] = useState(false);
  const [copied, setCopied] = useState(false);

  useEffect(() => {
    (async () => {
      try {
        const list = await base44.entities.Affiliate.list("-created_date", 10);
        const aff = list?.[0] || null;
        setAffiliate(aff);
        if (aff) {
          const coms = await base44.entities.Commission.list("-paid_at", 200);
          setCommissions(coms || []);
        }
      } catch (e) {
        setAffiliate(null);
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  const createAffiliate = async (e) => {
    e.preventDefault();
    const digits = phone.replace(/\D/g, "");
    if (digits.length < 6) {
      toast({ title: "Numéro invalide", description: "Entrez un numéro d'au moins 6 chiffres.", variant: "destructive" });
      return;
    }
    setCreating(true);
    try {
      const created = await base44.entities.Affiliate.create({
        referral_code: genCode(),
        phone_number: "+241" + digits,
        provider,
        status: "actif",
      });
      setAffiliate(created);
      toast({ title: "Bienvenue affilié !", description: "Votre lien d'affiliation est prêt." });
    } catch (err) {
      toast({ title: "Erreur", description: err?.message || "Création impossible", variant: "destructive" });
    }
    setCreating(false);
  };

  const link = affiliate ? `${window.location.origin}/register?ref=${affiliate.referral_code}` : "";
  const totalEarned = commissions.reduce((s, c) => s + (Number(c.amount) || 0), 0);
  const referralsCount = new Set(commissions.map((c) => c.referred_user_email).filter(Boolean)).size;

  const copyLink = async () => {
    try {
      await navigator.clipboard.writeText(link);
      setCopied(true);
      setTimeout(() => setCopied(false), 2000);
    } catch {
      /* ignore */
    }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  if (!affiliate) {
    return (
      <div className="space-y-6 max-w-lg mx-auto">
        <div>
          <h1 className="font-display text-3xl">Devenir affilié</h1>
          <p className="text-muted-foreground mt-1">
            Inscrivez-vous comme affilié, partagez votre lien et gagnez 20 % du montant de chaque
                        abonnement souscrit via votre lien (particulier 5 000 !'
          </p>
        </div>
        <form onSubmit={createAffiliate} className="bg-card border border-border rounded-2xl p-5 space-y-4">
          <div className="space-y-2">
```

```
            <Label>Opérateur Mobile Money</Label>
            <Select value={provider} onValueChange={setProvider}>
              <SelectTrigger><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="airtel">Airtel Money</SelectItem>
                <SelectItem value="moov">Moov Money</SelectItem>
              </SelectContent>
            </Select>
          </div>
          <div className="space-y-2">
            <Label htmlFor="aff-phone">Numéro Mobile Money</Label>
            <div className="flex items-center gap-2">
              <span className="px-3 h-9 inline-flex items-center rounded-md border border-input bg-muted text-sm
shrink-0">+241</span>
              <Input id="aff-phone" type="tel" inputMode="numeric" value={phone} onChange={(e) => setPhone(e.target.value)}
placeholder="60367591" />
            </div>
            <p className="text-xs text-muted-foreground">Vos commissions seront versées sur ce numéro.</p>
          </div>
          <Button type="submit" disabled={creating} className="w-full">
            {creating ? <><Loader2 className="w-4 h-4 mr-2 animate-spin" /> Traitement…</> : "S'inscrire comme affilié"}
          </Button>
        </form>
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div>
        <h1 className="font-display text-3xl">Mon espace affiliation</h1>
        <p className="text-muted-foreground mt-1">Partagez votre lien et touchez 20 % de chaque abonnement parrainé.</p>
      </div>

      <div className="bg-card border border-border rounded-2xl p-5 space-y-3">
        <div className="flex items-center gap-2">
          <Gift className="w-5 h-5 text-accent" />
          <h2 className="font-display text-lg">Votre lien d'affiliation</h2>
        </div>
        <div className="flex items-center gap-2">
          <Input readOnly value={link} className="font-mono text-xs" />
          <Button onClick={copyLink} variant="outline" className="shrink-0">
            {copied ? <><Check className="w-4 h-4 mr-1" /> Copié</> : <><Copy className="w-4 h-4 mr-1" /> Copier</>}
          </Button>
        </div>
        <p className="text-xs text-muted-foreground">Code : <span className="font-mono">{affiliate.referral_code}</span></p>
      </div>

      <div className="grid grid-cols-2 gap-4">
        <div className="bg-card border border-border rounded-2xl p-5">
          <div className="flex items-center gap-2 mb-2"><Users className="w-4 h-4 text-muted-foreground" /><p className="text-sm
text-muted-foreground">Filleuls</p></div>
          <p className="font-display text-2xl">{referralsCount}</p>
        </div>
        <div className="bg-card border border-border rounded-2xl p-5">
          <div className="flex items-center gap-2 mb-2"><Wallet className="w-4 h-4 text-muted-foreground" /><p
className="text-sm text-muted-foreground">Gains cumulés</p></div>
          <p className="font-display text-2xl">{formatCurrency(totalEarned, "XAF")}</p>
        </div>
      </div>

      <div className="bg-card border border-border rounded-2xl p-5">
        <h2 className="font-display text-lg mb-3">Commissions</h2>
        {commissions.length === 0 ? (
          <p className="text-sm text-muted-foreground">Aucune commission pour l'instant. Partagez votre lien !</p>
        ) : (
          <div className="space-y-2">
            {commissions.map((c) => (
              <div key={c.id} className="flex items-center justify-between border border-border rounded-lg p-3 text-sm">
                <div className="min-w-0">
                  <p className="font-medium truncate">{c.referred_user_email || "Filleul"}</p>
                  <p className="text-xs text-muted-foreground">{c.account_type === "entreprise" ? "Entreprise" : "Particulier"}
· {c.paid_at ? formatDateFr(c.paid_at, true) : "—"}</p>
                </div>
                <p className="font-medium text-accent-foreground shrink-0">{formatCurrency(c.amount, "XAF")}</p>
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Agents.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Wallet, ListChecks, Sparkles, Building2, Boxes, Users, Bell, FileDown } from "lucide-react";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import AgentChat from "@/components/agents/AgentChat";
```

```
export default function Agents() {
  const { t } = useI18n();
  const { space } = useSpace();

  const JPM_AGENTS = [
    { name: "finances", icon: Wallet, titleKey: "agents.finances.title", descKey: "agents.finances.desc" },
    { name: "taches", icon: ListChecks, titleKey: "agents.tasks.title", descKey: "agents.tasks.desc" },
    { name: "briefing", icon: Sparkles, titleKey: "agents.briefing.title", descKey: "agents.briefing.desc" },
    { name: "notifications", icon: Bell, titleKey: "agents.notifications.title", descKey: "agents.notifications.desc" },
    { name: "rapports", icon: FileDown, titleKey: "agents.rapports.title", descKey: "agents.rapports.desc" },
  ];

  const JBM_AGENTS = [
    { name: "jbm", icon: Building2, titleKey: "agents.jbm.title", descKey: "agents.jbm.desc" },
    { name: "inventaire", icon: Boxes, titleKey: "agents.inventaire.title", descKey: "agents.inventaire.desc" },
    { name: "rh", icon: Users, titleKey: "agents.rh.title", descKey: "agents.rh.desc" },
    { name: "notifications", icon: Bell, titleKey: "agents.notifications.title", descKey: "agents.notifications.desc" },
    { name: "rapports", icon: FileDown, titleKey: "agents.rapports.title", descKey: "agents.rapports.desc" },
  ];

  const AGENTS = space === "jbm" ? JBM_AGENTS : JPM_AGENTS;
  const [active, setActive] = useState(AGENTS[0].name);
  const current = AGENTS.find((a) => a.name === active) || AGENTS[0];
  const title = current.title || (current.titleKey ? t(current.titleKey) : active);
  const desc = current.desc || (current.descKey ? t(current.descKey) : "");

  return (
    <div className="space-y-6">
      <div>
        <h1 className="font-display text-2xl sm:text-3xl">{t("agents.title")}</h1>
        <p className="text-muted-foreground mt-1">{t("agents.subtitle")}</p>
      </div>

      <div className="flex gap-2 overflow-x-auto pb-1 -mx-1 px-1">
        {AGENTS.map((a) => {
          const Icon = a.icon;
          const isActive = active === a.name;
          const label = a.title || (a.titleKey ? t(a.titleKey) : a.name);
          return (
            <button
              key={a.name}
              type="button"
              onClick={() => setActive(a.name)}
              className={`no-select-tap flex items-center gap-2 px-4 py-2 rounded-xl border text-sm whitespace-nowrap
transition-colors ${
                isActive
                  ? "bg-primary text-primary-foreground border-primary"
                  : "bg-card border-border text-foreground hover:bg-secondary"
              }`}
            >
              <Icon className="w-4 h-4 shrink-0" />
              {label}
            </button>
          );
        })}
      </div>

      {current && <p className="text-sm text-muted-foreground -mt-2">{desc}</p>}

      <div className="bg-card border border-border rounded-2xl p-4">
        <AgentChat key={active} agentName={active} />
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/BusinessDashboard.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { ShoppingCart, Boxes, Users, FileText, AlertTriangle, Bot, Settings, Building2, CalendarClock, ScrollText, Truck, Link
as LinkIcon, Calculator, BarChart3, Lightbulb, Tag, UsersRound, Wallet, CalendarRange, CalendarDays } from "lucide-react";
import KpiCard from "@/components/dashboard/KpiCard";
import DashboardWindow from "@/components/dashboard/DashboardWindow";
import TasksWindow from "@/components/dashboard/TasksWindow";
import EventsWindow from "@/components/dashboard/EventsWindow";
import QuickLinks from "@/components/dashboard/QuickLinks";
import { formatCurrency } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import TipCard from "@/components/dashboard/TipCard";
import LifeTipCard from "@/components/dashboard/LifeTipCard";
import SpaceNav from "@/components/dashboard/SpaceNav";
import { Link } from "react-router-dom";

export default function BusinessDashboard() {
  const { t } = useI18n();
  const { activeCompanyId, companies } = useSpace();
  const [products, setProducts] = useState([]);
  const [sales, setSales] = useState([]);
```

```
  const [invoices, setInvoices] = useState([]);
  const [employees, setEmployees] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    (async () => {
      const [p, s, i, e, prefs] = await Promise.all([
        base44.entities.Product.list("-created_date", 500),
        base44.entities.Sale.list("-date", 500),
        base44.entities.Invoice.list("-issue_date", 500),
        base44.entities.Employee.list("-created_date", 500),
        base44.entities.Preferences.list(),
      ]);
      setProducts(p); setSales(s); setInvoices(i); setEmployees(e);
      setPreferences(prefs?.[0] || null);
      setLoading(false);
    })();
  }, []);

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  const cid = activeCompanyId;
  const company = companies.find((c) => c.id === cid);
  const currency = preferences?.currency || "XAF";

  const myProducts = cid ? products.filter((x) => x.company_id === cid) : products;
  const mySales = cid ? sales.filter((x) => x.company_id === cid) : sales;
  const myInvoices = cid ? invoices.filter((x) => x.company_id === cid) : invoices;
  const myEmployees = cid ? employees.filter((x) => x.company_id === cid) : employees;

  const salesTotal = mySales
    .filter((s) => s.status !== "annulee")
    .reduce((sum, s) => sum + (Number(s.quantity) || 0) * (Number(s.unit_price) || 0), 0);
  const stockValue = myProducts.reduce((sum, p) => sum + (Number(p.quantity) || 0) * (Number(p.unit_cost) || 0), 0);
  const payroll = myEmployees.filter((e) => e.status === "actif").reduce((sum, e) => sum + (Number(e.salary) || 0), 0);
  const unpaid = myInvoices.filter((i) => i.status !== "payee").reduce((sum, i) => sum + (Number(i.amount) || 0), 0);
  const lowStock = myProducts.filter((p) => (Number(p.reorder_level) || 0) > 0 && (Number(p.quantity) || 0) <=
(Number(p.reorder_level) || 0));

  return (
    <div className="space-y-8">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{company?.name || t("jbm.dashboard")}</h1>
          <p className="text-muted-foreground mt-1">{t("jbm.dashboardSubtitle")}</p>
        </div>
        <SpaceNav />
      </div>
      <TipCard title="Astuces entreprise">
        {myProducts.length === 0
          ? "Commencez par enregistrer vos produits et stocks, puis saisissez vos ventes et factures pour piloter votre
entreprise."
          : lowStock.length
          ? `Vous avez ${lowStock.length} produit(s) en stock bas. Passez commande pour éviter les ruptures.`
          : unpaid > 0
          ? `Vous avez ${formatCurrency(unpaid, currency)} de factures impayées. Relancez vos clients pour sécuriser votre
trésorerie.`
          : "Suivez vos ventes et votre trésorerie chaque semaine pour anticiper les besoins de votre entreprise."}
      </TipCard>
      <LifeTipCard space="jbm" />
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        <KpiCard icon={ShoppingCart} label={t("jbm.kpi.sales")} value={formatCurrency(salesTotal, currency)} tone="default" />
        <KpiCard icon={Boxes} label={t("jbm.kpi.stock")} value={formatCurrency(stockValue, currency)} tone="accent" />
        <KpiCard icon={Users} label={t("jbm.kpi.payroll")} value={formatCurrency(payroll, currency)} tone="default" />
        <KpiCard icon={FileText} label={t("jbm.kpi.unpaid")} value={formatCurrency(unpaid, currency)} tone={unpaid > 0 ? "warn"
: "default"} />
      </div>

      <div>
        <h2 className="font-display text-lg mb-3">Accès rapide</h2>
        <QuickLinks
          items={[
            { to: "/jbm/stock", label: t("jbm.nav.stock"), icon: Boxes, desc: "Inventaire" },
            { to: "/jbm/ventes", label: t("jbm.nav.sales"), icon: ShoppingCart, desc: "Ventes" },
            { to: "/jbm/personnel", label: t("jbm.nav.personnel"), icon: Users, desc: "Employés & paie" },
            { to: "/jbm/entreprises", label: t("jbm.nav.companies"), icon: Building2, desc: "Mes entreprises" },
            { to: "/evenements", label: t("nav.events"), icon: CalendarClock, desc: "Événements & RDV" },
            { to: "/jbm/clients", label: t("nav.clients"), icon: Users, desc: "Clients" },
            { to: "/jbm/devis", label: t("nav.quotes"), icon: ScrollText, desc: "Devis" },
            { to: "/jbm/prestataires", label: t("nav.suppliers"), icon: Truck, desc: "Prestataires" },
            { to: "/jbm/partenariats", label: t("nav.partnerships"), icon: LinkIcon, desc: "Partenariats" },
            { to: "/calculateurs", label: t("nav.calculators"), icon: Calculator, desc: "Calculatrice" },
            { to: "/reporting", label: "Reporting", icon: BarChart3, desc: "Rapports" },
            { to: "/collaborateurs", label: "Collaborateurs", icon: UsersRound, desc: "Échangez et travaillez ensemble" },
            { to: "/suggestions", label: t("nav.suggestions"), icon: Lightbulb, desc: "Suggestions" },
```

```
            { to: "/assistants", label: t("nav.agents"), icon: Bot, desc: "Assistant IA Rolly" },
            { to: "/parametres", label: t("nav.settings"), icon: Settings, desc: "Préférences" },
          ]}
        />
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        <div className="bg-card border border-border rounded-2xl p-5">
          <div className="flex items-center gap-2 mb-2">
            <Link to="/jbm/stock" className="font-display text-lg hover:underline flex items-center gap-2">
              <Boxes className="w-4 h-4" /> {t("jbm.nav.stock")}
            </Link>
            {lowStock.length > 0 && (
              <span className="text-xs px-2 py-0.5 rounded-full bg-destructive/10 text-destructive flex items-center gap-1">
                <AlertTriangle className="w-3 h-3" /> {lowStock.length}
              </span>
            )}
          </div>
          <p className="text-sm text-muted-foreground">{t("jbm.dashboard.stockHint", { n: myProducts.length })}</p>
        </div>
        <div className="bg-card border border-border rounded-2xl p-5">
          <Link to="/jbm/devis" className="font-display text-lg hover:underline flex items-center gap-2">
            <ScrollText className="w-4 h-4" /> {t("nav.quotes")}
          </Link>
          <p className="text-sm text-muted-foreground mt-2">Créez et suivez vos devis clients.</p>
        </div>
      </div>

      <div>
        <h2 className="font-display text-lg mb-3">Pilotage quotidien</h2>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
          <TasksWindow />
          <EventsWindow />
          <DashboardWindow to="/finances" icon={Wallet} title={t("nav.finances")} hint="Dépenses, épargne, emprunts et gains."
/>
          <DashboardWindow to="/planning" icon={CalendarRange} title={t("nav.planning")} hint="Agenda combiné tâches &
événements." />
          <DashboardWindow to="/calendrier" icon={CalendarDays} title={t("nav.calendar")} hint="Vue mensuelle de toutes vos
activités." />
        </div>
      </div>

    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Calculators.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Calculator as CalcIcon, Sigma, FunctionSquare, Wrench } from "lucide-react";
import { base44 } from "@/api/base44Client";
import { Tabs, TabsList, TabsTrigger, TabsContent } from "@/components/ui/tabs";
import { useSpace } from "@/lib/SpaceContext";
import { loadMetrics } from "@/lib/calcSources";
import { computeGrandeurs } from "@/lib/mathEval";
import ScientificCalculator from "@/components/calculator/ScientificCalculator";
import GrandeursPanel from "@/components/calculator/GrandeursPanel";
import FonctionsPanel from "@/components/calculator/FonctionsPanel";
import QuickTools from "@/components/calculator/QuickTools";

export default function Calculators() {
  const { activeCompanyId } = useSpace();
  const [items, setItems] = useState([]);
  const [metrics, setMetrics] = useState({});
  const [loading, setLoading] = useState(true);

  const scope = computeGrandeurs(items.filter((i) => i.kind === "grandeur"), metrics);

  const reload = async () => {
    const list = await base44.entities.CalcDefinition.list("-created_date", 500);
    setItems(list || []);
  };

  useEffect(() => {
    (async () => {
      await reload();
      try { const m = await loadMetrics(activeCompanyId); setMetrics(m); } catch { /* ignore */ }
      setLoading(false);
    })();
  }, [activeCompanyId]);

  if (loading)
    return <div className="flex items-center justify-center h-64"><div className="w-6 h-6 border-2 border-border
border-t-primary rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-6">
      <div>
        <h1 className="font-display text-3xl flex items-center gap-2"><CalcIcon className="w-7 h-7" /> Calculateur</h1>
        <p className="text-muted-foreground mt-1">Calculatrice scientifique, grandeurs et fonctions personnalisées — réutilisez
vos données Finances, Stock, Ventes et Factures.</p>
```

```
      </div>

      <Tabs defaultValue="calc" className="w-full">
        <TabsList className="w-full justify-start overflow-x-auto no-scrollbar">
          <TabsTrigger value="calc" className="gap-1.5"><CalcIcon className="w-4 h-4" /> Calculatrice</TabsTrigger>
          <TabsTrigger value="grandeurs" className="gap-1.5"><Sigma className="w-4 h-4" /> Grandeurs</TabsTrigger>
          <TabsTrigger value="fonctions" className="gap-1.5"><FunctionSquare className="w-4 h-4" /> Fonctions</TabsTrigger>
          <TabsTrigger value="outils" className="gap-1.5"><Wrench className="w-4 h-4" /> Outils rapides</TabsTrigger>
        </TabsList>
        <TabsContent value="calc" className="mt-4"><ScientificCalculator scope={scope} /></TabsContent>
        <TabsContent value="grandeurs" className="mt-4"><GrandeursPanel items={items} metrics={metrics} reload={reload}
/></TabsContent>
        <TabsContent value="fonctions" className="mt-4"><FonctionsPanel items={items} metrics={metrics} reload={reload}
/></TabsContent>
        <TabsContent value="outils" className="mt-4"><QuickTools /></TabsContent>
      </Tabs>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Calendar.jsx
------------------------------------------------------------
import React, { useEffect, useState, useMemo } from "react";
import { base44 } from "@/api/base44Client";
import { ChevronLeft, ChevronRight, Plus, CalendarClock, CheckSquare } from "lucide-react";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { Link } from "react-router-dom";
import {
  format, startOfMonth, endOfMonth, eachDayOfInterval, getDay,
  isSameDay, addMonths, subMonths, parseISO,
} from "date-fns";
import { fr } from "date-fns/locale";
import { getHolidays } from "@/lib/holidays";
import EventForm from "@/components/events/EventForm";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";

const WEEKDAYS = ["Lun", "Mar", "Mer", "Jeu", "Ven", "Sam", "Dim"];
const TASK_DOT = { haute: "bg-destructive", normale: "bg-primary", basse: "bg-muted-foreground" };
const EVENT_COLOR = {
  reunion: "bg-primary",
  rendez_vous: "bg-accent",
  formation: "bg-chart-3",
  ceremonie: "bg-sidebar-primary",
  autre: "bg-muted-foreground/50",
};

export default function CalendarPage() {
  const { t } = useI18n();
  const { space, activeCompanyId } = useSpace();
  const { toast } = useToast();
  const { register } = useRefresh();
  const [tasks, setTasks] = useState([]);
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [cursor, setCursor] = useState(new Date());
  const [formOpen, setFormOpen] = useState(false);
  const [fType, setFType] = useState("all");
  const [fCat, setFCat] = useState("all");
  const [fSpace, setFSpace] = useState("all");

  const load = async () => {
    try {
      const [taskList, eventList] = await Promise.all([
        base44.entities.Task.list("-start", 500),
        base44.entities.Event.list("start", 500),
      ]);
      setTasks(taskList || []);
      const scopedEvents = space === "jpm"
        ? (eventList || [])
        : (eventList || []).filter((e) => !activeCompanyId || e.company_id === activeCompanyId);
      setEvents(scopedEvents);
    } catch {
      setTasks([]);
      setEvents([]);
    }
    setLoading(false);
  };

  useEffect(() => { register(load); });
  useEffect(() => { setLoading(true); load(); }, [space, activeCompanyId]);

  const handleSaveEvent = async (data) => {
    try {
      const payload = space === "jpm" ? data : { ...data, company_id: activeCompanyId };
      await base44.entities.Event.create(payload);
      toast({ title: "Événement créé" });
      setFormOpen(false);
      load();
```

```
    } catch {
      toast({ title: "Erreur", description: "Création impossible.", variant: "destructive" });
    }
  };

  const monthStart = startOfMonth(cursor);
  const monthEnd = endOfMonth(cursor);
  const days = eachDayOfInterval({ start: monthStart, end: monthEnd });
  const firstDow = (getDay(monthStart) + 6) % 7;
  const cells = [...Array(firstDow).fill(null), ...days];

  const itemsByDay = useMemo(() => {
    const showTasks = fType !== "event";
    const showEvents = fType !== "task";
    const visibleEvents = events.filter((ev) => {
      if (fCat !== "all" && ev.category !== fCat) return false;
      if (fSpace === "perso" && ev.company_id) return false;
      if (fSpace === "pro" && !ev.company_id) return false;
      return true;
    });
    const map = {};
    if (showTasks) {
      tasks.forEach((task) => {
        const d = task.start ? parseISO(task.start) : null;
        if (!d) return;
        const key = format(d, "yyyy-MM-dd");
        (map[key] = map[key] || []).push({ kind: "task", item: task });
      });
    }
    if (showEvents) {
      visibleEvents.forEach((ev) => {
        const d = ev.start ? parseISO(ev.start) : null;
        if (!d) return;
        const key = format(d, "yyyy-MM-dd");
        (map[key] = map[key] || []).push({ kind: "event", item: ev });
      });
    }
    // Tri : événements d'abord, puis par heure de début.
    Object.values(map).forEach((arr) =>
      arr.sort((a, b) => {
        if (a.kind !== b.kind) return a.kind === "event" ? -1 : 1;
        return new Date(a.item.start) - new Date(b.item.start);
      })
    );
    return map;
  }, [tasks, events, fType, fCat, fSpace]);

  const holidays = useMemo(() => getHolidays(cursor.getFullYear()), [cursor]);

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-3xl">{t("calendar.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("calendar.subtitle")}</p>
        </div>
        <div className="flex items-center gap-2">
          <button
            onClick={() => setFormOpen(true)}
            className="inline-flex items-center gap-2 text-sm bg-primary text-primary-foreground px-3 py-2 rounded-lg
no-select-tap min-touch"
          >
            <CalendarClock className="w-4 h-4" /> Événement
          </button>
          <Link
            to="/taches"
            className="inline-flex items-center gap-2 text-sm bg-secondary text-secondary-foreground px-3 py-2 rounded-lg
no-select-tap min-touch"
          >
            <CheckSquare className="w-4 h-4" /> Tâche
          </Link>
        </div>
      </div>

      <div className="flex items-center gap-4 text-xs text-muted-foreground flex-wrap">
        <span className="inline-flex items-center gap-1.5"><span className="w-2 h-2 rounded-full bg-primary" /> Tâche
(priorité)</span>
        <span className="inline-flex items-center gap-1.5"><span className="w-2 h-2 rounded-sm bg-accent" /> Événement
(catégorie)</span>
      </div>

      <div className="flex flex-wrap items-center gap-2">
        <Select value={fType} onValueChange={setFType}>
          <SelectTrigger className="h-9 w-auto text-sm"><SelectValue /></SelectTrigger>
          <SelectContent>
            <SelectItem value="all">Tout type</SelectItem>
            <SelectItem value="task">Tâches</SelectItem>
            <SelectItem value="event">Événements</SelectItem>
          </SelectContent>
        </Select>
        <Select value={fCat} onValueChange={setFCat}>
          <SelectTrigger className="h-9 w-auto text-sm"><SelectValue /></SelectTrigger>
```

```
          <SelectContent>
            <SelectItem value="all">Toutes catégories</SelectItem>
            <SelectItem value="reunion">Réunion</SelectItem>
            <SelectItem value="rendez_vous">Rendez-vous</SelectItem>
            <SelectItem value="formation">Formation</SelectItem>
            <SelectItem value="ceremonie">Cérémonie</SelectItem>
            <SelectItem value="autre">Autre</SelectItem>
          </SelectContent>
        </Select>
        <Select value={fSpace} onValueChange={setFSpace}>
          <SelectTrigger className="h-9 w-auto text-sm"><SelectValue /></SelectTrigger>
          <SelectContent>
            <SelectItem value="all">Tous espaces</SelectItem>
            <SelectItem value="perso">Personnel</SelectItem>
            <SelectItem value="pro">Professionnel</SelectItem>
          </SelectContent>
        </Select>
      </div>

      <div className="flex items-center justify-between bg-card border border-border rounded-2xl p-4">
        <button
          onClick={() => setCursor(subMonths(cursor, 1))}
          className="no-select-tap min-touch p-2 rounded-lg hover:bg-secondary"
          aria-label={t("calendar.prev")}
        >
          <ChevronLeft className="w-5 h-5" />
        </button>
        <p className="font-display text-lg capitalize">{format(cursor, "MMMM yyyy", { locale: fr })}</p>
        <div className="flex items-center gap-1">
          <button
            onClick={() => setCursor(new Date())}
            className="text-xs px-2.5 py-1.5 rounded-lg bg-secondary hover:bg-secondary/70 no-select-tap"
          >
            {t("calendar.today")}
          </button>
          <button
            onClick={() => setCursor(addMonths(cursor, 1))}
            className="no-select-tap min-touch p-2 rounded-lg hover:bg-secondary"
            aria-label={t("calendar.next")}
          >
            <ChevronRight className="w-5 h-5" />
          </button>
        </div>
      </div>

      {loading ? (
        <div className="flex items-center justify-center h-40">
          <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
        </div>
      ) : (
        <div className="bg-card border border-border rounded-2xl p-3">
          <div className="grid grid-cols-7 gap-1 mb-1">
            {WEEKDAYS.map((d) => (
              <div key={d} className="text-center text-xs font-medium text-muted-foreground py-1">{d}</div>
            ))}
          </div>
          <div className="grid grid-cols-7 gap-1">
            {cells.map((day, i) => {
              if (!day) return <div key={i} className="min-h-[80px] rounded-lg" />;
              const key = format(day, "yyyy-MM-dd");
              const dayItems = itemsByDay[key] || [];
              const isToday = isSameDay(day, new Date());
              const holiday = holidays[key];
              return (
                <div
                  key={i}
                  className={`min-h-[80px] rounded-lg border p-1.5 ${isToday ? "border-primary bg-primary/5" : holiday ?
"border-accent/60 bg-accent/5" : "border-border"}`}
                >
                  <p className="text-xs text-muted-foreground mb-1">{format(day, "d")}</p>
                  {holiday && <p className="text-[9px] leading-tight text-accent-foreground bg-accent/20 rounded px-1 mb-1
truncate" title={holiday}>{holiday}</p>}
                  <div className="space-y-1">
                    {dayItems.slice(0, 3).map((entry) => (
                      <div key={entry.item.id} className="flex items-center gap-1 text-[11px] leading-tight">
                        <span className={`w-1.5 h-1.5 shrink-0 ${entry.kind === "event" ? "rounded-sm " +
(EVENT_COLOR[entry.item.category] || EVENT_COLOR.autre) : "rounded-full " + (TASK_DOT[entry.item.priority] ||
TASK_DOT.normale)}`} />
                        <span className="truncate">{entry.item.title}</span>
                      </div>
                    ))}
                    {dayItems.length > 3 && (
                      <p className="text-[10px] text-muted-foreground">+{dayItems.length - 3}</p>
                    )}
                  </div>
                </div>
              );
            })}
          </div>
        </div>
      )}
```

```
      <EventForm open={formOpen} onOpenChange={setFormOpen} event={null} onSubmit={handleSaveEvent} />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Categories.jsx
------------------------------------------------------------
import React, { useEffect, useMemo, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Tag, Plus, Trash2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useSpace } from "@/lib/SpaceContext";

const DOMAINS = [
  { key: "depense", label: "Dépenses", scope: "jpm" },
  { key: "produit", label: "Produits", scope: "jbm" },
  { key: "vente", label: "Ventes", scope: "jbm" },
  { key: "tache", label: "Tâches", scope: "jpm" },
  { key: "evenement", label: "Événements", scope: "jpm" },
];

export default function Categories() {
  const { space, activeCompanyId } = useSpace();
  const [all, setAll] = useState([]);
  const [loading, setLoading] = useState(true);
  const [newName, setNewName] = useState("");
  const [domain, setDomain] = useState("depense");
  const [scopeFilter, setScopeFilter] = useState("all");
  const [error, setError] = useState("");

  const load = async () => {
    setLoading(true);
    try {
      const list = await base44.entities.Category.list("-created_date", 500);
      setAll(list || []);
    } catch {
      setAll([]);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { load(); }, []);

  const filtered = useMemo(() => {
    return all.filter((c) => {
      if (scopeFilter !== "all" && c.scope !== scopeFilter) return false;
      if (c.domain !== domain) return false;
      if (c.scope === "jbm" && activeCompanyId && c.company_id && c.company_id !== activeCompanyId) return false;
      return true;
    });
  }, [all, domain, scopeFilter, activeCompanyId]);

  const handleCreate = async (e) => {
    e.preventDefault();
    const name = newName.trim();
    if (!name) { setError("Saisissez un nom."); return; }
    const def = DOMAINS.find((d) => d.key === domain);
    const scope = def ? def.scope : (space === "jbm" ? "jbm" : "jpm");
    try {
      const created = await base44.entities.Category.create({
        name, scope, domain, company_id: scope === "jbm" ? (activeCompanyId || undefined) : undefined,
      });
      setAll((prev) => [created, ...prev]);
      setNewName("");
      setError("");
    } catch (err) {
      setError(err?.message || "Erreur à la création.");
    }
  };

  const handleDelete = async (id) => {
    try {
      await base44.entities.Category.delete(id);
      setAll((prev) => prev.filter((c) => c.id !== id));
    } catch {
      /* ignore */
    }
  };

  return (
    <div className="space-y-6">
      <div>
        <h1 className="font-display text-2xl sm:text-3xl flex items-center gap-2"><Tag className="w-6 h-6 text-accent" />
Catégories</h1>
        <p className="text-muted-foreground mt-1">Créez et organisez vos catégories pour dépenses, produits, ventes, tâches et
événements.</p>
      </div>
```

```
      <form onSubmit={handleCreate} className="bg-card border border-border rounded-2xl p-4 flex flex-col sm:flex-row gap-3
sm:items-end">
        <div className="space-y-1.5 flex-1">
          <label className="text-xs font-medium">Domaine</label>
          <Select value={domain} onValueChange={setDomain}>
            <SelectTrigger><SelectValue /></SelectTrigger>
            <SelectContent>
              {DOMAINS.map((d) => <SelectItem key={d.key} value={d.key}>{d.label}</SelectItem>)}
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-1.5 flex-1">
          <label className="text-xs font-medium">Espace</label>
          <Select value={scopeFilter} onValueChange={setScopeFilter}>
            <SelectTrigger><SelectValue /></SelectTrigger>
            <SelectContent>
              <SelectItem value="all">Tous</SelectItem>
              <SelectItem value="jpm">Personnel (JPM)</SelectItem>
              <SelectItem value="jbm">Entreprise (JBM)</SelectItem>
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-1.5 flex-[2]">
          <label className="text-xs font-medium">Nouvelle catégorie</label>
          <Input value={newName} onChange={(e) => setNewName(e.target.value)} placeholder="Nom de la catégorie" />
        </div>
        <Button type="submit" className="flex items-center gap-1"><Plus className="w-4 h-4" /> Ajouter</Button>
        {error && <p className="text-sm text-destructive sm:col-span-3">{error}</p>}
      </form>

      {loading ? (
        <div className="flex items-center justify-center h-32">
          <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
        </div>
      ) : (
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-3">
          {filtered.length === 0 && (
            <p className="text-sm text-muted-foreground col-span-full">Aucune catégorie. Créez-en une ci-dessus.</p>
          )}
          {filtered.map((c) => (
            <div key={c.id} className="bg-card border border-border rounded-2xl p-3 flex items-center justify-between gap-2">
              <div className="min-w-0">
                <p className="text-sm font-medium truncate">{c.name}</p>
                <p className="text-xs text-muted-foreground capitalize">
                  {c.scope === "jbm" ? "Entreprise" : "Personnel"} · {DOMAINS.find((d) => d.key === c.domain)?.label ||
c.domain}
                </p>
              </div>
              <Button size="icon" variant="ghost" onClick={() => handleDelete(c.id)} aria-label="Supprimer">
                <Trash2 className="w-4 h-4 text-destructive" />
              </Button>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Clients.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Users, Pencil, Trash2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
AlertDialogHeader, AlertDialogTitle } from "@/components/ui/alert-dialog";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useSpace } from "@/lib/SpaceContext";

const empty = { name: "", type: "particulier", phone: "", email: "", address: "", notes: "" };

export default function Clients() {
  const { activeCompanyId } = useSpace();
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);
  const [del, setDel] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const list = await base44.entities.Client.list("-created_date", 500);
```

```
    setItems(list || []);
    setLoading(false);
  };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const mine = activeCompanyId ? items.filter((c) => c.company_id === activeCompanyId) : items;

  const openNew = () => { setEditing(null); setForm(empty); setOpen(true); };
  const openEdit = (c) => { setEditing(c); setForm({ name: c.name || "", type: c.type || "particulier", phone: c.phone || "",
email: c.email || "", address: c.address || "", notes: c.notes || "" }); setOpen(true); };

  const save = async (e) => {
    e.preventDefault();
    if (!form.name.trim()) return;
    const payload = { ...form, company_id: activeCompanyId };
    try {
      if (editing) { await base44.entities.Client.update(editing.id, payload); toast({ title: "Client mis à jour" }); }
      else { await base44.entities.Client.create(payload); toast({ title: "Client ajouté" }); }
      setOpen(false); load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (c) => {
    try { await base44.entities.Client.delete(c.id); toast({ title: "Client supprimé" }); load(); }
    catch { toast({ title: "Erreur", variant: "destructive" }); }
    setDel(null);
  };

  if (loading) return <div className="flex items-center justify-center h-64"><div className="w-6 h-6 border-2 border-border
border-t-primary rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">Clients</h1>
          <p className="text-muted-foreground mt-1">Gérez votre portefeuille de clients.</p>
        </div>
        <Button onClick={openNew}><Plus className="w-4 h-4 mr-2" /> Ajouter</Button>
      </div>

      {mine.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground"><Users className="w-8 h-8 mx-auto mb-3 opacity-50" /><p
className="text-sm">Aucun client enregistré.</p></div>
      ) : (
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
          {mine.map((c) => (
            <div key={c.id} className="bg-card border border-border rounded-xl p-4">
              <div className="flex items-start justify-between gap-2">
                <div className="min-w-0">
                  <p className="text-sm font-medium truncate">{c.name}</p>
                  <span className="text-xs px-2 py-0.5 rounded-full bg-secondary">{c.type === "entreprise" ? "Entreprise" :
"Particulier"}</span>
                </div>
                <div className="flex items-center gap-1 shrink-0">
                  <button onClick={() => openEdit(c)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                  <button onClick={() => setDel(c)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive"><Trash2 className="w-4 h-4" /></button>
                </div>
              </div>
              <div className="mt-2 text-xs text-muted-foreground space-y-0.5">
                {c.phone && <p>{c.phone}</p>}
                {c.email && <p className="truncate">{c.email}</p>}
                {c.address && <p className="truncate">{c.address}</p>}
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader><DialogTitle className="font-display">{editing ? "Modifier le client" : "Nouveau
client"}</DialogTitle></DialogHeader>
          <form onSubmit={save} className="space-y-3">
            <div className="space-y-2"><Label>Nom</Label><Input value={form.name} onChange={(e) => setForm({ ...form, name:
e.target.value })} /></div>
            <div className="space-y-2"><Label>Type</Label>
              <Select value={form.type} onValueChange={(v) => setForm({ ...form, type: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent><SelectItem value="particulier">Particulier</SelectItem><SelectItem
value="entreprise">Entreprise</SelectItem></SelectContent>
              </Select>
            </div>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Téléphone</Label><Input value={form.phone} onChange={(e) => setForm({ ...form,
phone: e.target.value })} /></div>
              <div className="space-y-2"><Label>Email</Label><Input type="email" value={form.email} onChange={(e) => setForm({
...form, email: e.target.value })} /></div>
            </div>
```

```
            <div className="space-y-2"><Label>Adresse</Label><Input value={form.address} onChange={(e) => setForm({ ...form,
address: e.target.value })} /></div>
            <div className="space-y-2"><Label>Notes</Label><Textarea value={form.notes} onChange={(e) => setForm({ ...form,
notes: e.target.value })} rows={2} /></div>
            <DialogFooter><Button type="submit" className="w-full">Enregistrer</Button></DialogFooter>
          </form>
        </DialogContent>
      </Dialog>

      <AlertDialog open={!!del} onOpenChange={(o) => !o && setDel(null)}>
        <AlertDialogContent>
          <AlertDialogHeader><AlertDialogTitle>Supprimer ce client ?</AlertDialogTitle><AlertDialogDescription>Le client «
{del?.name} » sera supprimé définitivement.</AlertDialogDescription></AlertDialogHeader>
          <AlertDialogFooter><AlertDialogCancel>Annuler</AlertDialogCancel><AlertDialogAction className="bg-destructive
text-destructive-foreground hover:bg-destructive/90" onClick={(e) => { e.preventDefault(); remove(del);
}}>Supprimer</AlertDialogAction></AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/CodeSource.jsx
------------------------------------------------------------
import React, { useState } from 'react';
import { base44 } from '@/api/base44Client';
import { Button } from '@/components/ui/button';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { Download, FileCode, FolderTree, Loader2, ShieldCheck } from 'lucide-react';

const STRUCTURE = [
  { d: 'src/pages', n: 'Pages de l\'application (Accueil, Finances, Tâches, Ventes, Admin…)' },
  { d: 'src/components', n: 'Composants réutilisables (UI, dashboard, formulaires…)' },
  { d: 'src/lib', n: 'Utilitaires, contextes, logique métier partagée' },
  { d: 'base44/entities', n: 'Schémas de données (entités de la base)' },
  { d: 'base44/functions', n: 'Fonctions backend (paiement, IA, alertes…)' },
  { d: 'base44/workflows', n: 'Workflows automatisés (rappels, alertes email…)' },
  { d: 'base44/agents', n: 'Agents IA (briefing, finances, RH, inventaire…)' },
  { d: 'base44/emails', n: 'Modèles d\'emails transactionnels' },
  { d: 'index.html · package.json · tailwind.config.js', n: 'Configuration du projet' },
];

export default function CodeSource() {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const handleDownload = async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await base44.functions.invoke('downloadSource', {});
      const link = res?.signed_url || res?.data?.signed_url;
      if (!link) throw new Error(res?.error || res?.data?.error || 'Lien indisponible');
      window.location.href = link;
    } catch (e) {
      const body = e && e.response && e.response.data;
      setError((body && body.error) || e?.message || 'Échec du téléchargement');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-background px-4 py-6 pb-24 max-w-2xl mx-auto">
      <div className="flex items-center gap-3 mb-6">
        <div className="w-11 h-11 rounded-xl bg-primary/10 flex items-center justify-center">
          <FileCode className="w-6 h-6 text-primary" />
        </div>
        <div>
          <h1 className="font-heading text-2xl font-semibold">Code source</h1>
          <p className="text-sm text-muted-foreground">Archive complète du projet JH Manager</p>
        </div>
      </div>

      <Card className="mb-5">
        <CardHeader className="pb-3">
          <CardTitle className="flex items-center gap-2 text-base">
            <FolderTree className="w-4 h-4 text-accent" /> Contenu de l'archive
          </CardTitle>
        </CardHeader>
        <CardContent className="space-y-2.5 pt-0">
          {STRUCTURE.map((s) => (
            <div key={s.d} className="flex flex-col gap-0.5">
              <span className="font-mono text-sm text-primary">{s.d}</span>
              <span className="text-xs text-muted-foreground">{s.n}</span>
            </div>
          ))}
        </CardContent>
      </Card>

      <Card className="mb-5 border-accent/30 bg-accent/5">
```

```
        <CardContent className="flex items-start gap-3 pt-4">
          <ShieldCheck className="w-5 h-5 text-accent shrink-0 mt-0.5" />
          <p className="text-sm text-muted-foreground">
            L'archive est stockée <strong className="text-foreground">privémentement</strong>. Le lien de
            téléchargement est signé à la demande, valable 1 heure, et accessible uniquement aux administrateurs.
          </p>
        </CardContent>
      </Card>

      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">{error}</div>
      )}

      <Button onClick={handleDownload} disabled={loading} className="w-full h-12 text-base">
        {loading ? <Loader2 className="w-5 h-5 animate-spin" /> : <Download className="w-5 h-5" />}
        {loading ? 'Génération du lien…' : 'Télécharger le code source (.zip)'}
      </Button>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Collaborateurs.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { useAuth } from "@/lib/AuthContext";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { Users, MessageSquare, Inbox, Plus, Search, Check, X, Link2Off, Loader2, Crown, Flag } from "lucide-react";
import UserSearch from "@/components/collab/UserSearch";
import CompanySearch from "@/components/collab/CompanySearch";
import ChatView from "@/components/collab/ChatView";
import GroupAdmin from "@/components/collab/GroupAdmin";
import SharedPanel from "@/components/collab/SharedPanel";
import ReportDialog from "@/components/collab/ReportDialog";
import { useToast } from "@/components/ui/use-toast";

export default function Collaborateurs() {
  const { user } = useAuth();
  const [profile, setProfile] = useState(null);
  const [nameMap, setNameMap] = useState({});
  const [tab, setTab] = useState("collab");
  const [collaborators, setCollaborators] = useState([]);
  const [requests, setRequests] = useState([]);       // reçues
  const [collaborations, setCollaborations] = useState([]);
  const [groups, setGroups] = useState([]);
  const [groupInvites, setGroupInvites] = useState([]); // reçues
  const [selected, setSelected] = useState(null);      // conversation courante
  const [showCreate, setShowCreate] = useState(false);
  const [newName, setNewName] = useState("");
  const [q, setQ] = useState("");
  const [pickResults, setPickResults] = useState([]);
  const [picked, setPicked] = useState([]);
  const [busy, setBusy] = useState(false);
  const [reportTarget, setReportTarget] = useState(null);
  const { toast } = useToast();

  const loadAll = async () => {
    if (!user) return;
    try {
      const [cols, reqs, colls, grps, ginv] = await Promise.all([
        base44.entities.Collaborator.list("-created_date", 500),
        base44.entities.CollaborationRequest.list("-created_date", 200),
        base44.entities.Collaboration.list("-created_date", 200),
        base44.entities.Group.list("-created_date", 200),
        base44.entities.GroupInvite.list("-created_date", 200),
      ]);
      setCollaborators(cols || []);
      const map = {};
      (cols || []).forEach((c) => { map[c.created_by_id] = c.display_name; });
      setNameMap(map);
      setRequests((reqs || []).filter((r) => r.to_user_id === user.id && r.status === "pending"));
      setCollaborations(colls || []);
      setGroups(grps || []);
      setGroupInvites((ginv || []).filter((i) => i.to_user_id === user.id && i.status === "pending"));
      const mine = (cols || []).find((c) => c.created_by_id === user.id);
      setProfile(mine || null);
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    }
  };

  // Crée automatiquement le profil collaborateur au premier accès
  const ensureProfile = async () => {
    if (!user) return;
    try {
      const existing = await base44.entities.Collaborator.filter({ created_by_id: user.id });
      if (existing && existing.length) { setProfile(existing[0]); return; }
      const created = await base44.entities.Collaborator.create({ display_name: user.full_name || user.email || "Utilisateur"
});
```

```
      setProfile(created);
    } catch { /* ignore */ }
  };

  useEffect(() => { ensureProfile(); loadAll(); /* eslint-disable-next-line */ }, [user?.id]);

  const reload = () => { loadAll(); };

  const pendingInviteCount = requests.length + groupInvites.length;

  const acceptRequest = async (r) => {
    setBusy(true);
    try {
      await base44.entities.Collaboration.create({ members: [r.created_by_id, user.id], partner_name: nameMap[r.created_by_id]
|| "" });
      await base44.entities.CollaborationRequest.update(r.id, { status: "accepted" });
      reload();
    } catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  const rejectRequest = async (r) => {
    setBusy(true);
    try { await base44.entities.CollaborationRequest.update(r.id, { status: "rejected" }); reload(); }
    catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  const breakCollab = async (c) => {
    if (!confirm("Rompre cette collaboration ? L'autre membre sera retiré des groupes dont vous êtes créateur (et
inversement).")) return;
    setBusy(true);
    try { await base44.functions.invoke('breakCollaboration', { collaboration_id: c.id }); setSelected(null); reload(); }
    catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  const acceptGroupInvite = async (inv) => {
    setBusy(true);
    try { await base44.functions.invoke('manageGroup', { action: "accept_invite", invite_id: inv.id }); reload(); }
    catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  const rejectGroupInvite = async (inv) => {
    setBusy(true);
    try { await base44.entities.GroupInvite.update(inv.id, { status: "rejected" }); reload(); }
    catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  const searchPick = async () => {
    const term = q.trim();
    if (term.length < 2) { setPickResults([]); return; }
    const list = await base44.entities.Collaborator.list("-created_date", 200);
    const t = term.toLowerCase();
    setPickResults((list || []).filter((c) => (c.display_name || "").toLowerCase().includes(t) && c.created_by_id !== user.id));
  };

  const createGroup = async () => {
    if (!newName.trim()) { toast({ title: "Donnez un nom au groupe" }); return; }
    setBusy(true);
    try {
      const g = await base44.entities.Group.create({ name: newName.trim(), members: [user.id], admins: [] });
      for (const p of picked) {
        await base44.entities.GroupInvite.create({ group_id: g.id, group_name: g.name, to_user_id: p.created_by_id, to_name:
p.display_name, status: "pending" });
      }
      setShowCreate(false); setNewName(""); setPicked([]); setPickResults([]); setQ("");
      setTab("groupes"); reload();
      toast({ title: "Groupe créé", description: `${picked.length} invitation(s) envoyée(s)` });
    } catch (e) { toast({ title: "Erreur", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  if (!user) return <div className="p-6 flex justify-center"><Loader2 className="w-5 h-5 animate-spin" /></div>;

  return (
    <div className="space-y-5">
      <div>
        <h1 className="font-display text-2xl sm:text-3xl flex items-center gap-2"><Users className="w-6 h-6 text-accent" />
Collaborateurs</h1>
        <p className="text-muted-foreground mt-1">Échangez en 1-à-1, créez des groupes et travaillez ensemble.</p>
      </div>

      <div className="flex gap-2 border-b border-border">
        <TabBtn active={tab === "collab"} onClick={() => setTab("collab")} icon={<MessageSquare className="w-4 h-4" />}
label="Collaborations" />
        <TabBtn active={tab === "groupes"} onClick={() => setTab("groupes")} icon={<Users className="w-4 h-4" />}
label="Groupes" />
        <TabBtn active={tab === "invites"} onClick={() => setTab("invites")} icon={<Inbox className="w-4 h-4" />}
label="Invitations" badge={pendingInviteCount} />
```

```
      </div>

      {tab === "collab" && (
        <div className="space-y-4">
          <div className="bg-card border border-border rounded-2xl p-3">
            <p className="text-xs font-medium text-muted-foreground mb-2">Trouver un collaborateur</p>
            <UserSearch currentUserId={user.id} excludeIds={collaborations.flatMap((c) => c.members || [])} onSent={reload} />
          </div>
          <div className="bg-card border border-border rounded-2xl p-3">
            <p className="text-xs font-medium text-muted-foreground mb-2">Trouver une entreprise (par type et/ou lieu)</p>
            <CompanySearch currentUserId={user.id} excludeIds={collaborations.flatMap((c) => c.members || [])} onSent={reload}
/>
          </div>

          {collaborations.length === 0 ? (
            <p className="text-sm text-muted-foreground text-center py-6">Aucune collaboration active.</p>
          ) : (
            <div className="space-y-2">
              {collaborations.map((c) => {
                const other = (c.members || []).find((m) => m !== user.id);
                return (
                  <div key={c.id} className="bg-card border border-border rounded-2xl p-3">
                    <div className="flex items-center justify-between">
                      <button onClick={() => setSelected({ type: "direct", id: c.id, participants: c.members, group: null,
label: nameMap[other] || other })} className="flex items-center gap-2 min-w-0">
                        <div className="w-9 h-9 rounded-full bg-primary/15 text-primary flex items-center justify-center text-sm
font-medium">{(nameMap[other] || "?").charAt(0).toUpperCase()}</div>
                        <span className="text-sm font-medium truncate">{nameMap[other] || other}</span>
                      </button>
                      <div className="flex items-center gap-1">
                        <Button size="sm" variant="ghost" onClick={() => breakCollab(c)} className="flex items-center gap-1
text-destructive"><Link2Off className="w-4 h-4" /> Rompre</Button>
                        <Button size="sm" variant="ghost" onClick={() => setReportTarget({ type: "user", id: other, name:
nameMap[other] || other })} title="Signaler"><Flag className="w-4 h-4" /></Button>
                      </div>
                    </div>
                  </div>
                );
              })}
            </div>
          )}

          {selected && selected.type === "direct" && (
            <div className="space-y-2">
              <div className="flex items-center justify-between">
                <p className="text-sm font-medium">Conversation avec {selected.label}</p>
                <Button size="sm" variant="ghost" onClick={() => setSelected(null)}>Fermer</Button>
              </div>
              <ChatView conversationType="direct" conversationId={selected.id} participants={selected.participants} group={null}
currentUserId={user.id} />
              <SharedPanel conversation={selected} currentUserId={user.id} nameMap={nameMap} />
            </div>
          )}
        </div>
      )}

      {tab === "groupes" && (
        <div className="space-y-4">
          <Button onClick={() => setShowCreate(true)} className="flex items-center gap-1"><Plus className="w-4 h-4" /> Créer un
groupe</Button>

          {groups.length === 0 ? (
            <p className="text-sm text-muted-foreground text-center py-6">Aucun groupe. Créez-en un.</p>
          ) : (
            <div className="space-y-2">
              {groups.map((g) => (
                <div key={g.id} className="bg-card border border-border rounded-2xl p-3">
                  <button onClick={() => setSelected({ type: "group", id: g.id, participants: g.members, group: g, label: g.name
})} className="w-full text-left">
                    <div className="flex items-center justify-between">
                      <span className="text-sm font-medium flex items-center gap-1.5">{g.name} {g.created_by_id === user.id &&
<Crown className="w-3.5 h-3.5 text-accent" />}</span>
                      <span className="text-xs text-muted-foreground">{(g.members || []).length} membre(s)</span>
                    </div>
                  </button>
                  <div className="flex justify-end mt-2">
                    <Button size="sm" variant="ghost" onClick={() => setReportTarget({ type: "group", id: g.id, name: g.name })}
className="flex items-center gap-1 text-muted-foreground"><Flag className="w-4 h-4" /> Signaler</Button>
                  </div>
                </div>
              ))}
            </div>
          )}

          {selected && selected.type === "group" && (
            <div className="space-y-3">
              <div className="flex items-center justify-between">
                <p className="text-sm font-medium">{selected.label}</p>
                <Button size="sm" variant="ghost" onClick={() => setSelected(null)}>Fermer</Button>
              </div>
              <ChatView conversationType="group" conversationId={selected.id} participants={selected.participants}
group={groups.find((g) => g.id === selected.id)} currentUserId={user.id} />
```

```
              <GroupAdmin group={groups.find((g) => g.id === selected.id)} currentUserId={user.id} nameMap={nameMap}
onChanged={reload} />
              <SharedPanel conversation={selected} currentUserId={user.id} nameMap={nameMap} />
            </div>
          )}
        </div>
      )}

      {tab === "invites" && (
        <div className="space-y-4">
          <div className="space-y-2">
            <p className="text-xs font-medium text-muted-foreground">Demandes de collaboration</p>
            {requests.length === 0 ? <p className="text-sm text-muted-foreground">Aucune demande.</p> : requests.map((r) => (
              <div key={r.id} className="bg-card border border-border rounded-2xl p-3 flex items-center justify-between">
                <span className="text-sm font-medium truncate">{nameMap[r.created_by_id] || r.created_by}</span>
                <div className="flex gap-1">
                  <Button size="sm" onClick={() => acceptRequest(r)} disabled={busy} className="flex items-center gap-1"><Check
className="w-4 h-4" /> Accepter</Button>
                  <Button size="sm" variant="outline" onClick={() => rejectRequest(r)} disabled={busy}><X className="w-4 h-4"
/></Button>
                </div>
              </div>
            ))}
          </div>
          <div className="space-y-2">
            <p className="text-xs font-medium text-muted-foreground">Invitations à des groupes</p>
            {groupInvites.length === 0 ? <p className="text-sm text-muted-foreground">Aucune invitation.</p> :
groupInvites.map((inv) => (
              <div key={inv.id} className="bg-card border border-border rounded-2xl p-3 flex items-center justify-between">
                <span className="text-sm font-medium truncate">{inv.group_name}</span>
                <div className="flex gap-1">
                  <Button size="sm" onClick={() => acceptGroupInvite(inv)} disabled={busy} className="flex items-center
gap-1"><Check className="w-4 h-4" /> Rejoindre</Button>
                  <Button size="sm" variant="outline" onClick={() => rejectGroupInvite(inv)} disabled={busy}><X className="w-4
h-4" /></Button>
                </div>
              </div>
            ))}
          </div>
        </div>
      )}

      <Dialog open={showCreate} onOpenChange={setShowCreate}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>Créer un groupe</DialogTitle>
          </DialogHeader>
          <div className="space-y-3">
            <Input value={newName} onChange={(e) => setNewName(e.target.value)} placeholder="Nom du groupe" />
            <div className="space-y-2">
              <p className="text-xs font-medium text-muted-foreground">Inviter des collaborateurs</p>
              <div className="flex items-center gap-2">
                <Input value={q} onChange={(e) => setQ(e.target.value)} onKeyDown={(e) => { if (e.key === "Enter") {
e.preventDefault(); searchPick(); } }} placeholder="Rechercher…" />
                <Button size="icon" variant="outline" onClick={searchPick}><Search className="w-4 h-4" /></Button>
              </div>
              {pickResults.map((c) => {
                const sel = picked.find((p) => p.created_by_id === c.created_by_id);
                return (
                  <div key={c.id} className="flex items-center justify-between bg-muted/40 rounded-xl p-2.5">
                    <span className="text-sm truncate">{c.display_name}</span>
                    <Button size="sm" variant={sel ? "default" : "outline"} onClick={() => setPicked((prev) => sel ?
prev.filter((p) => p.created_by_id !== c.created_by_id) : [...prev, c])}>
                      {sel ? "Retirer" : "Ajouter"}
                    </Button>
                  </div>
                );
              })}
              {picked.length > 0 && <p className="text-xs text-muted-foreground">{picked.length} sélectionné(s)</p>}
            </div>
          </div>
          <DialogFooter>
            <Button variant="outline" onClick={() => setShowCreate(false)}>Annuler</Button>
            <Button onClick={createGroup} disabled={busy}>{busy ? <Loader2 className="w-4 h-4 animate-spin" /> :
"Créer"}</Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>

      <ReportDialog open={!!reportTarget} onOpenChange={(o) => !o && setReportTarget(null)} target={reportTarget} />
    </div>
  );
}

function TabBtn({ active, onClick, icon, label, badge }) {
  return (
    <button onClick={onClick} className={`no-select-tap relative flex items-center gap-1.5 px-3 py-2 text-sm border-b-2
transition-colors ${active ? "border-primary text-primary" : "border-transparent text-muted-foreground"}`}>
      {icon} {label}
      {badge > 0 && <span className="ml-1 bg-accent text-accent-foreground text-[10px] font-medium rounded-full px-1.5
py-0.5">{badge}</span>}
    </button>
```

```
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Companies.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Building2, Check, Pencil, Trash2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { useToast } from "@/components/ui/use-toast";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { BUSINESS_TYPES } from "@/lib/businessTypes";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import { useAuth } from "@/lib/AuthContext";
import SubscribeDialog from "@/components/onboarding/SubscribeDialog";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
  AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";

const empty = { name: "", business_type: "", location: "", sector: "", address: "", phone: "", email: "", tax_id: "" };
const PRICE_PER_COMPANY = 20000;

export default function Companies() {
  const { t } = useI18n();
  const { user } = useAuth();
  const { companies, activeCompanyId, setActiveCompanyId, refreshCompanies } = useSpace();
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);
  const [delTarget, setDelTarget] = useState(null);
  const [payPlan, setPayPlan] = useState(null);
  const [allowedCount, setAllowedCount] = useState(1);
  const { toast } = useToast();
  const isAdmin = user?.role === "admin";

  useEffect(() => {
    refreshCompanies();
    (async () => {
      try {
        const subs = await base44.entities.Subscription.list("-paid_at", 50);
        const mine = (subs || []).filter((s) => s.account_type === "entreprise" && s.status === "payee");
        const cnt = mine.length ? Math.max(...mine.map((s) => Number(s.companies_count) || 1)) : 1;
        setAllowedCount(cnt);
      } catch {
        setAllowedCount(1);
      }
    })();
  }, []);

  const openNew = () => {
    if (!isAdmin && companies.length >= allowedCount) {
      setPayPlan({ type: "entreprise", label: "Ajouter une entreprise", unitPrice: PRICE_PER_COMPANY });
      return;
    }
    setEditing(null); setForm(empty); setOpen(true);
  };
  const openEdit = (c) => { setEditing(c); setForm({ name: c.name || "", business_type: c.business_type || "", location:
c.location || "", sector: c.sector || "", address: c.address || "", phone: c.phone || "", email: c.email || "", tax_id: c.tax_id
|| "" }); setOpen(true); };

  const save = async (e) => {
    e.preventDefault();
    if (!form.name.trim() || !form.business_type || !form.location.trim()) return;
    try {
      if (editing) {
        await base44.entities.Company.update(editing.id, form);
        toast({ title: t("toast.taskUpdated") });
      } else {
        await base44.entities.Company.create(form);
        toast({ title: t("toast.expenseAdded") });
      }
      refreshCompanies();
      setOpen(false);
    } catch (err) {
      toast({ title: t("toast.error"), description: t("err.save"), variant: "destructive" });
    }
  };

  const remove = async (c) => {
    try {
      await base44.entities.Company.delete(c.id);
      toast({ title: t("toast.taskDeleted") });
      refreshCompanies();
    } catch (e) {
      toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
    }
```

```
    setDelTarget(null);
  };

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{t("jbm.companies.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("jbm.companies.subtitle")}</p>
        </div>
        <Button onClick={openNew}>
          <Plus className="w-4 h-4 mr-2" /> {t("jbm.companies.new")}
        </Button>
      </div>

      {!isAdmin && (
        <div className="text-xs text-muted-foreground bg-secondary/60 border border-border rounded-lg px-3 py-2">
          Entreprises : {companies.length} / {allowedCount}
          {companies.length >= allowedCount && ` — ajouter une entreprise coûte ${PRICE_PER_COMPANY.toLocaleString("fr-FR")} XAF
(débit Mobile Money) et donne accès à toutes les fonctions JBM pour la nouvelle entreprise.`}
        </div>
      )}
      {companies.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground">
          <Building2 className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p className="text-sm">{t("jbm.companies.empty")}</p>
        </div>
      ) : (
        <div className="space-y-2">
          {companies.map((c) => (
            <div key={c.id} className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
              <button onClick={() => setActiveCompanyId(c.id)} className="flex items-center gap-3 min-w-0 text-left">
                <span className={`w-5 h-5 rounded-full flex items-center justify-center shrink-0 ${activeCompanyId === c.id ?
"bg-primary text-primary-foreground" : "border border-border"}`}>
                  {activeCompanyId === c.id && <Check className="w-3 h-3" />}
                </span>
                <div className="min-w-0">
                  <p className="text-sm font-medium truncate">{c.name}</p>
                  <p className="text-xs text-muted-foreground">{[c.business_type, c.location].filter(Boolean).join(" • ") ||
c.sector || t("common.other")}</p>
                </div>
              </button>
              <div className="flex items-center gap-1 shrink-0">
                <button onClick={() => openEdit(c)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                <button onClick={() => setDelTarget(c)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-destructive/10 text-destructive"><Trash2 className="w-4 h-4" /></button>
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader><DialogTitle className="font-display">{editing ? t("jbm.companies.edit") :
t("jbm.companies.new")}</DialogTitle></DialogHeader>
          <form onSubmit={save} className="space-y-3">
            <div className="space-y-2"><Label>{t("jbm.companies.name")}</Label><Input value={form.name} onChange={(e) =>
setForm({ ...form, name: e.target.value })} /></div>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Type d'entreprise *</Label>
                <Select value={form.business_type} onValueChange={(v) => setForm({ ...form, business_type: v })}>
                  <SelectTrigger><SelectValue placeholder="Sélectionner…" /></SelectTrigger>
                  <SelectContent>{BUSINESS_TYPES.map((bt) => <SelectItem key={bt} value={bt}>{bt}</SelectItem>)}</SelectContent>
                </Select>
              </div>
              <div className="space-y-2"><Label>Lieu *</Label><Input value={form.location} onChange={(e) => setForm({ ...form,
location: e.target.value })} placeholder="Ville / quartier" /></div>
            </div>
            <div className="space-y-2"><Label>{t("jbm.companies.sector")}</Label><Input value={form.sector} onChange={(e) =>
setForm({ ...form, sector: e.target.value })} /></div>
            <div className="space-y-2"><Label>{t("jbm.companies.address")}</Label><Input value={form.address} onChange={(e) =>
setForm({ ...form, address: e.target.value })} /></div>
            <div className="grid grid-cols-2 gap-3">
              <div className="space-y-2"><Label>{t("jbm.companies.phone")}</Label><Input value={form.phone} onChange={(e) =>
setForm({ ...form, phone: e.target.value })} /></div>
              <div className="space-y-2"><Label>{t("jbm.companies.email")}</Label><Input type="email" value={form.email}
onChange={(e) => setForm({ ...form, email: e.target.value })} /></div>
            </div>
            <div className="space-y-2"><Label>{t("jbm.companies.taxId")}</Label><Input value={form.tax_id} onChange={(e) =>
setForm({ ...form, tax_id: e.target.value })} /></div>
            <DialogFooter><Button type="submit" className="w-full">{t("common.save")}</Button></DialogFooter>
          </form>
        </DialogContent>
      </Dialog>

      <AlertDialog open={!!delTarget} onOpenChange={(o) => !o && setDelTarget(null)}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>{t("jbm.companies.deleteTitle")}</AlertDialogTitle>
            <AlertDialogDescription>{t("jbm.companies.deleteDesc", { name: delTarget?.name })}</AlertDialogDescription>
```

```
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>{t("common.cancel")}</AlertDialogCancel>
            <AlertDialogAction className="bg-destructive text-destructive-foreground hover:bg-destructive/90" onClick={(e) => {
e.preventDefault(); remove(delTarget); }}>{t("common.delete")}</AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>

      <SubscribeDialog
        plan={payPlan}
        open={!!payPlan}
        onOpenChange={(o) => !o && setPayPlan(null)}
        defaultCount={companies.length + 1}
        alreadyCovered={allowedCount}
        onSuccess={() => { setPayPlan(null); window.location.reload(); }}
      />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Dashboard.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Wallet, TrendingDown, CheckCircle2, AlertTriangle, ListChecks, CalendarRange, CalendarDays, Bot, Settings, Users, Gift,
Bell, Gauge, CalendarClock, Calculator, BarChart3, Tag, UsersRound, HelpCircle } from "lucide-react";
import KpiCard from "@/components/dashboard/KpiCard";
import QuickLinks from "@/components/dashboard/QuickLinks";
import ExpenseChart from "@/components/dashboard/ExpenseChart";
import RevenueCard from "@/components/dashboard/RevenueCard";
import SpaceNav from "@/components/dashboard/SpaceNav";
import GlobalSearch from "@/components/search/GlobalSearch";
import TipCard from "@/components/dashboard/TipCard";
import LifeTipCard from "@/components/dashboard/LifeTipCard";
import BusinessDashboard from "@/pages/BusinessDashboard";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { useI18n } from "@/lib/i18n";
import { useAuth } from "@/lib/AuthContext";
import { useSpace } from "@/lib/SpaceContext";
import { Link } from "react-router-dom";

export default function Dashboard() {
  const { t } = useI18n();
  const { user } = useAuth();
  const { space, activeCompanyId, companies } = useSpace();
  const [tasks, setTasks] = useState([]);
  const [expenses, setExpenses] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    (async () => {
      try {
        const [taskList, expList, prefList] = await Promise.all([
          base44.entities.Task.list("-start", 200),
          base44.entities.Expense.list("-date", 200),
          base44.entities.Preferences.list(),
        ]);
        setTasks(taskList || []);
        setExpenses(expList || []);
        setPreferences(prefList?.[0] || null);
      } catch (err) {
        console.error("Dashboard load error:", err);
        setTasks([]);
        setExpenses([]);
        setPreferences(null);
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  // Alerte automatique : vérifie les seuils une fois par jour et crée les notifications
  useEffect(() => {
    const today = new Date().toISOString().slice(0, 10);
    const key = `jm_alerts_${today}`;
    if (localStorage.getItem(key)) return;
    (async () => {
      try {
        await base44.functions.invoke('checkAlerts', {});
        localStorage.setItem(key, "1");
      } catch { /* ignore */ }
    })();
  }, []);

  if (loading) {
    return (
      <div classNam="flex items-center justify-center h-64">
        <div classNam="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
```

```
    );
  }

  const currency = preferences?.currency || "XAF";
  const budget = preferences?.initial_budget || 0;
  const totalExpense = expenses.reduce((sum, e) => sum + (Number(e.quantity) || 0) * (Number(e.unit_cost) || 0), 0);
  const remaining = budget - totalExpense;

  const now = new Date();
  const startOfDay = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  const endOfDay = new Date(startOfDay.getTime() + 24 * 60 * 60 * 1000);

  const activeTasks = tasks.filter((task) => task.status !== "terminee");
  const todayTasks = activeTasks.filter((task) => new Date(task.start) < endOfDay && new Date(task.end) >= startOfDay);
  const overdueTasks = activeTasks.filter((task) => new Date(task.end) < now);
  const reminderDays = preferences?.reminder_days || 7;
  const limit = new Date(now.getTime() + reminderDays * 24 * 60 * 60 * 1000);
  const upcoming = activeTasks
    .filter((task) => new Date(task.start) > now && new Date(task.start) < limit)
    .sort((a, b) => new Date(a.start) - new Date(b.start))
    .slice(0, 5);

  const contextCompany = companies.find((c) => c.id === activeCompanyId);
  const headerTitle =
    space === "jbm" ? (contextCompany?.name || t("jbm.dashboard"))
    : t("dashboard.greeting");
  const headerSubtitle =
    space === "jbm" ? t("jbm.dashboardSubtitle")
    : t("dashboard.subtitle");

  return (
    <div className="space-y-8">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{headerTitle}</h1>
          <p className="text-muted-foreground mt-1">{headerSubtitle}</p>
        </div>
        <div className="flex items-center gap-2 flex-wrap">
          <GlobalSearch />
          <SpaceNav />
        </div>
      </div>

      {space === "jbm" ? (
        <BusinessDashboard />
      ) : (
      <>
      <TipCard title="Astuces personnelles">
        {overdueTasks.length > 0
          ? `Vous avez ${overdueTasks.length} tâche(s) en retard. Anticipez-les pour garder votre planning sous contrôle.`
          : remaining < 0
          ? "Vos dépenses dépassent votre budget. Revoyez vos postes de dépense dans les finances."
          : "Suivez chaque jour vos dépenses et tâches pour rester maître de votre budget personnel."}
      </TipCard>
      <LifeTipCard space="jpm" />
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        <KpiCard icon={Wallet} label={t("kpi.remaining")} value={formatCurrency(remaining, currency)} tone="default" />
        <KpiCard icon={TrendingDown} label={t("kpi.totalExpenses")} value={formatCurrency(totalExpense, currency)} tone="accent"
/>
        <KpiCard icon={CheckCircle2} label={t("kpi.todayTasks")} value={todayTasks.length} tone="default" />
        <KpiCard icon={AlertTriangle} label={t("kpi.overdue")} value={overdueTasks.length} tone={overdueTasks.length ? "warn" :
"default"} />
      </div>

      <ExpenseChart expenses={expenses} currency={currency} />

      {user?.role === "admin" && <RevenueCard />}

      <div>
        <h2 className="font-display text-lg mb-3">Accès rapide</h2>
        <QuickLinks
          items={[
            { to: "/finances", label: t("nav.finances"), icon: Wallet, desc: "Budget, dépenses, taxes & emprunts" },
            { to: "/taches", label: t("nav.tasks"), icon: ListChecks, desc: "Vos tâches" },
            { to: "/faq", label: "FAQ", icon: HelpCircle, desc: "Questions fréquentes & aide" },
            { to: "/planning", label: t("nav.planning"), icon: CalendarRange, desc: "Calendrier" },
            { to: "/calendrier", label: t("nav.calendar"), icon: CalendarDays, desc: "Vue mensuelle" },
            { to: "/evenements", label: t("nav.events"), icon: CalendarClock, desc: "Événements & RDV" },
            { to: "/calculateurs", label: t("nav.calculators"), icon: Calculator, desc: "Calculatrice & formules" },
            { to: "/reporting", label: "Reporting", icon: BarChart3, desc: "Rapports & graphiques" },
            { to: "/assistants", label: t("nav.agents"), icon: Bot, desc: "Assistant IA JPM" },
            { to: "/parametres", label: t("nav.settings"), icon: Settings, desc: "Préférences" },
            { to: "/notifications", label: t("nav.notifications"), icon: Bell, desc: "Rappels & alertes" },
            { to: "/collaborateurs", label: "Collaborateurs", icon: UsersRound, desc: "Échangez et travaillez ensemble" },
            { to: "/suggestions", label: t("nav.suggestions"), icon: Gift, desc: "Suggérez des améliorations" },
            { to: "/affiliation", label: "Affiliation", icon: Gift, desc: "Parrainez et gagnez" },
            ...(user?.role === "admin" ? [{ to: "/admin", label: "Abonnés", icon: Users, desc: "Tableau de bord admin" }] : []),
          ]}
        />
      </div>

      <div className="bg-card border border-border rounded-2xl p-5 sm:p-6">
```

```
        <div className="flex items-center justify-between mb-4">
          <h2 className="font-display text-lg">{t("dashboard.upcoming")}</h2>
          <Link to="/planning" className="text-sm text-primary hover:underline">{t("dashboard.viewPlanning")}</Link>
        </div>
        {upcoming.length === 0 ? (
          <p className="text-sm text-muted-foreground">{t("dashboard.noUpcoming", { days: reminderDays })}</p>
        ) : (
          <div className="space-y-3">
            {upcoming.map((task) => (
              <div key={task.id} className="flex items-center justify-between gap-3">
                <div className="min-w-0">
                  <p className="text-sm font-medium truncate">{task.title}</p>
                  <p className="text-xs text-muted-foreground">{formatDateFr(task.start, true)}</p>
                </div>
                <span className="text-xs px-2 py-1 rounded-full bg-secondary text-secondary-foreground shrink-0">
                  {t("priority." + (task.priority || "normale"))}
                </span>
              </div>
            ))}
          </div>
        )}
      </div>

      </>
      )}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Demo.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { motion } from "framer-motion";
import { Link } from "react-router-dom";
import { Sparkles, FlaskConical, ArrowRight, CheckCircle2, User, Building2, Wallet, ListChecks, Boxes, FileText, Bot } from
"lucide-react";
import Landing from "@/pages/Landing";
import DashboardPreview from "@/components/demo/DashboardPreview";

export default function Demo() {
  const [step, setStep] = useState("accueil"); // accueil | abonne

  return (
    <div className="min-h-screen bg-background">
      {/* Bannière Mode démo */}
      <div className="sticky top-0 z-50 bg-primary text-primary-foreground px-4 py-2.5 flex items-center justify-between gap-3">
        <div className="flex items-center gap-2 text-xs sm:text-sm font-medium">
          <FlaskConical className="w-4 h-4 shrink-0" />
          <span className="truncate">Mode démo — aperçu du parcours utilisateur (sans transaction réelle)</span>
        </div>
        <Link to="/" className="no-select-tap shrink-0 text-xs sm:text-sm underline underline-offset-2 hover:opacity-80">
          Quitter la démo
        </Link>
      </div>

      {/* Étape 1 : Accueil public */}
      {step === "accueil" && (
        <div className="relative">
          <Landing />
          {/* CTA flottant pour simuler l'abonnement */}
          <motion.div
            initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }}
            className="fixed bottom-4 inset-x-4 z-40 mx-auto max-w-md"
          >
            <button
              onClick={() => setStep("abonne")}
              className="no-select-tap min-touch w-full rounded-2xl bg-accent text-accent-foreground shadow-lg px-5 py-3.5
font-medium inline-flex items-center justify-center gap-2 hover:bg-accent/90 transition-colors"
            >
              Simuler un abonnement validé <ArrowRight className="w-4 h-4" />
            </button>
            <p className="text-center text-xs text-muted-foreground mt-1.5">
              Le bouton d'abonnement réel enverrait vers le paiement Mobile Money.
            </p>
          </motion.div>
        </div>
      )}

      {/* Étape 2 : Utilisateur abonné (validé par défaut) */}
      {step === "abonne" && (
        <div className="max-w-5xl mx-auto px-4 py-8 space-y-8">
          <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }}
            className="rounded-2xl border border-border bg-card p-5 flex flex-col sm:flex-row items-center gap-4
justify-between">
            <div className="flex items-center gap-3 text-center sm:text-left">
              <div className="w-11 h-11 rounded-2xl bg-primary/10 text-primary flex items-center justify-center shrink-0">
                <CheckCircle2 className="w-6 h-6" />
              </div>
              <div>
                        <p className="font-display text-lg">Paiement simulé
              <p className="text-sm text-muted-foreground">Abonnement activé automatiquement — aucune transaction
```

```
financière.</p>
              </div>
            </div>
            <button
              onClick={() => setStep("accueil")}
              className="no-select-tap min-touch rounded-xl border border-border px-4 py-2 text-sm font-medium
hover:bg-accent/10 transition-colors"
            >
                          !•  R e v e n i r  à  l ' a c c u e i l  p u b l i c
            </button>
          </motion.div>

          <DashboardPreview />

          {/* Rappel des deux espaces */}
          <div className="grid sm:grid-cols-2 gap-4">
            <SpaceRecap
              icon={User} tone="primary" badge="JH Personal Manager" title="Particulier"
              features={["Finances & budget", "Tâches & planning", "Assistant IA", "Collaboration"]}
              price="5 000 XAF / mois"
            />
            <SpaceRecap
              icon={Building2} tone="accent" badge="JH Business Manager" title="Entreprise"
              features={["Stock & ventes", "Devis", "Assistant IA", "Multi-entreprises"]}
              price="20 000 XAF / mois / entreprise"
            />
          </div>
        </div>
      )}
    </div>
  );
}

function SpaceRecap({ icon: Icon, tone, badge, title, features, price }) {
  const isAccent = tone === "accent";
  return (
    <div className={`rounded-2xl border p-5 ${isAccent ? "border-accent/30 bg-accent/5" : "border-primary/30 bg-primary/5"}`}>
      <div className="flex items-center gap-3 mb-3">
        <div className={`w-10 h-10 rounded-xl flex items-center justify-center ${isAccent ? "bg-accent text-accent-foreground" :
"bg-primary text-primary-foreground"}`}>
          <Icon className="w-5 h-5" />
        </div>
        <span className={`text-xs font-medium px-2 py-1 rounded-full ${isAccent ? "bg-accent/15 text-accent-foreground" :
"bg-primary/10 text-primary"}`}>{badge}</span>
        <span className="font-display text-lg ml-auto">{price}</span>
      </div>
      <ul className="space-y-1.5">
        {features.map((f) => (
          <li key={f} className="flex items-center gap-2 text-sm">
            <CheckCircle2 className={`w-4 h-4 shrink-0 ${isAccent ? "text-accent-foreground" : "text-primary"}`} /> {f}
          </li>
        ))}
      </ul>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Events.jsx
------------------------------------------------------------
import React, { useEffect, useMemo, useState } from "react";
import { base44 } from "@/api/base44Client";
import { CalendarClock, Plus } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useSpace } from "@/lib/SpaceContext";
import { formatDateFr } from "@/lib/format";
import EventItem from "@/components/events/EventItem";
import EventForm from "@/components/events/EventForm";
import { expandRecurrence } from "@/lib/recurrence";

function overlaps(a, b) {
  return new Date(a.start) < new Date(b.end) && new Date(b.start) < new Date(a.end);
}

const PAGE_SIZE = 100;

export default function Events() {
  const { space, activeCompanyId } = useSpace();
  const { toast } = useToast();
  const { register } = useRefresh();
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [formOpen, setFormOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [visibleCount, setVisibleCount] = useState(PAGE_SIZE);

  const load = async () => {
    try {
      const list = await base44.entities.Event.list("start", 300);
      const scoped = space === "jpm"
```

```
      ? (list || [])
      : (list || []).filter((e) => !activeCompanyId || e.company_id === activeCompanyId);
    setEvents(scoped);
  } catch {
    setEvents([]);
  } finally {
    setLoading(false);
  }
};

useEffect(() => { register(load); });
useEffect(() => { setLoading(true); setVisibleCount(PAGE_SIZE); load(); }, [space, activeCompanyId]);

const handleSave = async (data, recurrence) => {
  if (editing) {
    const prev = events;
    setEvents((cur) => cur.map((i) => (i.id === editing.id ? { ...i, ...data } : i)));
    try {
      await base44.entities.Event.update(editing.id, data);
      toast({ title: "Événement mis à jour" });
      load();
    } catch {
      setEvents(prev);
      toast({ title: "Erreur", description: "Modification impossible.", variant: "destructive" });
    }
  } else {
    const basePayload = space === "jpm" ? data : { ...data, company_id: activeCompanyId };
    const items = expandRecurrence(basePayload, recurrence?.frequency, recurrence?.count, ["start", "end"]);
    const tempIds = items.map(() => `tmp_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`);
    setEvents((cur) => [...items.map((it, i) => ({ ...it, id: tempIds[i] })), ...cur]);
    try {
      await base44.entities.Event.bulkCreate(items);
      toast({ title: recurrence && recurrence.count > 1 ? `${items.length} événements récurrents créés` : "Événement créé" });
      load();
    } catch {
      setEvents((cur) => cur.filter((e) => !tempIds.includes(e.id)));
      toast({ title: "Erreur", description: "Création impossible.", variant: "destructive" });
    }
  }
  setFormOpen(false);
  setEditing(null);
};

const handleDelete = async (ev) => {
  const prev = events;
  setEvents((cur) => cur.filter((i) => i.id !== ev.id));
  try {
    await base44.entities.Event.delete(ev.id);
    toast({ title: "Événement supprimé" });
  } catch {
    setEvents(prev);
    toast({ title: "Erreur", description: "Suppression impossible.", variant: "destructive" });
  }
};

const sorted = useMemo(() => [...events].sort((a, b) => new Date(a.start) - new Date(b.start)), [events]);
const conflictIds = useMemo(() => {
  const ids = new Set();
  for (let i = 0; i < sorted.length; i++) {
    for (let j = i + 1; j < sorted.length; j++) {
      if (overlaps(sorted[i], sorted[j])) {
        ids.add(sorted[i].id);
        ids.add(sorted[j].id);
      }
    }
  }
  return ids;
}, [sorted]);

const groups = useMemo(() => {
  const visible = sorted.slice(0, visibleCount);
  return visible.reduce((acc, ev) => {
    const key = new Date(ev.start).toDateString();
    acc[key] = acc[key] || [];
    acc[key].push(ev);
    return acc;
  }, {});
}, [sorted, visibleCount]);

if (loading) {
  return (
    <div className="flex items-center justify-center h-64">
      <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
    </div>
  );
}

return (
  <div className="space-y-6">
    <div className="flex items-start justify-between gap-3 flex-wrap">
      <div>
        <h1 className="font-display text-3xl flex items-center gap-2"><CalendarClock className="w-7 h-7" /> Événements</h1>
```

```
          <p className="text-muted-foreground mt-1">Planifiez vos événements et rendez-vous{space === "jpm" ? "" : space ===
"jbm" ? " de votre entreprise" : " de votre établissement"}.</p>
          {conflictIds.size > 0 && (
                        < p   c l a s s N a m e = " t e x t - s m   t e x t - d e s t r u c t i v e   m t - 2 " >&    { c o n f l i c
          )}
        </div>
        <Button onClick={() => { setEditing(null); setFormOpen(true); }} className="no-select-tap">
          <Plus className="w-4 h-4 mr-2" /> Nouvel événement
        </Button>
      </div>

      {sorted.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground">
          <CalendarClock className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p className="text-sm">Aucun événement planifié pour le moment.</p>
        </div>
      ) : (
        <div className="space-y-6">
          {Object.entries(groups).map(([day, dayEvents]) => (
            <div key={day}>
              <h2 className="text-sm font-medium text-muted-foreground mb-2">{formatDateFr(dayEvents[0].start)}</h2>
              <div className="space-y-2">
                {dayEvents.map((ev) => (
                  <EventItem
                    key={ev.id}
                    event={ev}
                    onEdit={(item) => { setEditing(item); setFormOpen(true); }}
                    onDelete={handleDelete}
                    conflict={conflictIds.has(ev.id)}
                  />
                ))}
              </div>
            </div>
          ))}
          {visibleCount < sorted.length && (
            <Button variant="outline" className="w-full" onClick={() => setVisibleCount((c) => c + PAGE_SIZE)}>
              Charger plus ({sorted.length - visibleCount} restants)
            </Button>
          )}
        </div>
      )}

      <EventForm open={formOpen} onOpenChange={setFormOpen} event={editing} onSubmit={handleSave} />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Export.jsx
------------------------------------------------------------
import React, { useEffect, useMemo, useState } from "react";
import { base44 } from "@/api/base44Client";
import { FileDown, Loader2, FileText, Sheet } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Label } from "@/components/ui/label";
import { useSpace } from "@/lib/SpaceContext";
import { useToast } from "@/components/ui/use-toast";
import { formatCurrency } from "@/lib/format";
import { exportToPdf, exportToXls } from "@/lib/exportReport";
import {
  PERIOD_TYPES, yearOptions, monthOptions, quarterOptions, halfOptions, weekOptions, computeRange,
} from "@/lib/reportPeriods";

const money = (v) => Number(v) || 0;

// Définition des sections exportables : entité, champ de date, colonnes, mapping ligne, aspects (filtre) optionnels.
const SECTIONS = [
  {
    value: "taches", label: "Tâches", entity: "Task", dateField: "start", companyScoped: false,
    columns: [
      { key: "title", label: "Titre" },
      { key: "priority", label: "Priorité" },
      { key: "status", label: "Statut" },
      { key: "category", label: "Catégorie" },
      { key: "start", label: "Début", type: "date" },
      { key: "end", label: "Fin", type: "date" },
    ],
    toRow: (r) => ({ title: r.title, priority: r.priority, status: r.status, category: r.category, start: r.start, end: r.end
}),
    aspects: { field: "status", options: [
      { value: "a_faire", label: "À faire" }, { value: "en_cours", label: "En cours" }, { value: "terminee", label: "Terminée"
},
    ] },
  },
  {
    value: "evenements", label: "Évènements", entity: "Event", dateField: "start", companyScoped: false,
    columns: [
      { key: "title", label: "Titre" },
      { key: "category", label: "Catégorie" },
      { key: "location", label: "Lieu" },
```

```
      { key: "priority", label: "Priorité" },
      { key: "start", label: "Début", type: "date" },
      { key: "end", label: "Fin", type: "date" },
    ],
    toRow: (r) => ({ title: r.title, category: r.category, location: r.location, priority: r.priority, start: r.start, end:
r.end }),
    aspects: { field: "category", options: [
      { value: "reunion", label: "Réunion" }, { value: "rendez_vous", label: "Rendez-vous" },
      { value: "formation", label: "Formation" }, { value: "ceremonie", label: "Cérémonie" }, { value: "autre", label: "Autre"
},
    ] },
  },
  {
    value: "planning", label: "Planning (tâches + évènements)", entity: null, dateField: "start", companyScoped: false,
    columns: [
      { key: "type", label: "Type" },
      { key: "titre", label: "Titre" },
      { key: "start", label: "Début", type: "date" },
      { key: "end", label: "Fin", type: "date" },
      { key: "statut", label: "Statut / Priorité" },
    ],
    toRow: (r) => ({ type: r._type, titre: r.title, start: r.start, end: r.end, statut: r.status }),
  },
  {
    value: "finances", label: "Finances (dépenses)", entity: "Expense", dateField: "date", companyScoped: false,
    columns: [
      { key: "name", label: "Libellé" },
      { key: "category", label: "Catégorie" },
      { key: "quantity", label: "Quantité" },
      { key: "unit_cost", label: "Coût unitaire", type: "currency" },
      { key: "date", label: "Date", type: "date" },
      { key: "total", label: "Total", type: "currency" },
    ],
    toRow: (r) => ({ name: r.name, category: r.category, quantity: r.quantity, unit_cost: r.unit_cost, date: r.date, total:
money(r.quantity) * money(r.unit_cost) }),
  },
  {
    value: "calculs", label: "Calculs (définitions)", entity: "CalcDefinition", dateField: "created_date", companyScoped: false,
    columns: [
      { key: "label", label: "Libellé" },
      { key: "name", label: "Identifiant" },
      { key: "kind", label: "Type" },
      { key: "expression", label: "Formule" },
      { key: "unit", label: "Unité" },
      { key: "value", label: "Valeur" },
      { key: "source_type", label: "Source" },
    ],
    toRow: (r) => ({ label: r.label, name: r.name, kind: r.kind, expression: r.expression, unit: r.unit, value: r.value,
source_type: r.source_type }),
  },
  {
    value: "devis", label: "Devis", entity: "Quote", dateField: "issue_date", companyScoped: true,
    columns: [
      { key: "number", label: "Numéro" },
      { key: "client_name", label: "Client" },
      { key: "amount", label: "Montant", type: "currency" },
      { key: "issue_date", label: "Émission", type: "date" },
      { key: "validity_date", label: "Validité", type: "date" },
      { key: "status", label: "Statut" },
    ],
    toRow: (r) => ({ number: r.number, client_name: r.client_name, amount: r.amount, issue_date: r.issue_date, validity_date:
r.validity_date, status: r.status }),
    aspects: { field: "status", options: [
      { value: "brouillon", label: "Brouillon" }, { value: "envoye", label: "Envoyé" },
      { value: "accepte", label: "Accepté" }, { value: "refuse", label: "Refusé" },
    ] },
  },
  {
    value: "ventes", label: "Ventes", entity: "Sale", dateField: "date", companyScoped: true,
    columns: [
      { key: "item_name", label: "Article" },
      { key: "customer_name", label: "Client" },
      { key: "quantity", label: "Quantité" },
      { key: "unit_price", label: "Prix unitaire", type: "currency" },
      { key: "date", label: "Date", type: "date" },
      { key: "status", label: "Statut" },
      { key: "total", label: "Total", type: "currency" },
    ],
    toRow: (r) => ({ item_name: r.item_name, customer_name: r.customer_name, quantity: r.quantity, unit_price: r.unit_price,
date: r.date, status: r.status, total: money(r.quantity) * money(r.unit_price) }),
    aspects: { field: "status", options: [
      { value: "en_attente", label: "En attente" }, { value: "payee", label: "Payée" }, { value: "annulee", label: "Annulée" },
    ] },
  },
  {
    value: "salaries", label: "Salariés", entity: "Employee", dateField: "hire_date", companyScoped: true,
    columns: [
      { key: "full_name", label: "Nom" },
      { key: "role", label: "Poste" },
      { key: "salary", label: "Salaire", type: "currency" },
      { key: "phone", label: "Téléphone" },
      { key: "email", label: "Email" },
```

```
      { key: "hire_date", label: "Embauche", type: "date" },
      { key: "status", label: "Statut" },
    ],
    toRow: (r) => ({ full_name: r.full_name, role: r.role, salary: r.salary, phone: r.phone, email: r.email, hire_date:
r.hire_date, status: r.status }),
    aspects: { field: "status", options: [
      { value: "actif", label: "Actif" }, { value: "inactif", label: "Inactif" },
    ] },
  },
  {
    value: "prestataires", label: "Prestataires", entity: "Supplier", dateField: "created_date", companyScoped: true,
    columns: [
      { key: "name", label: "Nom" },
      { key: "category", label: "Spécialité" },
      { key: "contact_person", label: "Contact" },
      { key: "phone", label: "Téléphone" },
      { key: "email", label: "Email" },
      { key: "address", label: "Adresse" },
    ],
    toRow: (r) => ({ name: r.name, category: r.category, contact_person: r.contact_person, phone: r.phone, email: r.email,
address: r.address }),
  },
  {
    value: "partenariats", label: "Partenariats", entity: "Partnership", dateField: "start_date", companyScoped: true,
    columns: [
      { key: "partner_name", label: "Partenaire" },
      { key: "partner_type", label: "Type" },
      { key: "contact_name", label: "Contact" },
      { key: "email", label: "Email" },
      { key: "phone", label: "Téléphone" },
      { key: "status", label: "Statut" },
      { key: "start_date", label: "Début", type: "date" },
    ],
    toRow: (r) => ({ partner_name: r.partner_name, partner_type: r.partner_type, contact_name: r.contact_name, email: r.email,
phone: r.phone, status: r.status, start_date: r.start_date }),
    aspects: { field: "status", options: [
      { value: "prospect", label: "Prospect" }, { value: "actif", label: "Actif" }, { value: "inactif", label: "Inactif" },
    ] },
  },
  {
    value: "factures", label: "Factures", entity: "Invoice", dateField: "issue_date", companyScoped: true,
    columns: [
      { key: "number", label: "Numéro" },
      { key: "customer_name", label: "Client" },
      { key: "amount", label: "Montant", type: "currency" },
      { key: "issue_date", label: "Émission", type: "date" },
      { key: "due_date", label: "Échéance", type: "date" },
      { key: "status", label: "Statut" },
    ],
    toRow: (r) => ({ number: r.number, customer_name: r.customer_name, amount: r.amount, issue_date: r.issue_date, due_date:
r.due_date, status: r.status }),
    aspects: { field: "status", options: [
      { value: "brouillon", label: "Brouillon" }, { value: "envoyee", label: "Envoyée" },
      { value: "payee", label: "Payée" }, { value: "en_retard", label: "En retard" },
    ] },
  },
];

export default function Export() {
  const { space, activeCompanyId, companies } = useSpace();
  const { toast } = useToast();
  const [section, setSection] = useState(space === "jbm" ? "ventes" : "finances");
  const [periodType, setPeriodType] = useState("mois");
  const now = new Date();
  const [period, setPeriod] = useState({
    year: now.getFullYear(), month: now.getMonth(), quarter: 1, half: 1, week: 1, from: "", to: "",
  });
  const [aspect, setAspect] = useState("tous");
  const [records, setRecords] = useState([]);
  const [loading, setLoading] = useState(false);
  const [busy, setBusy] = useState(false);
  const [cur, setCur] = useState("XAF");

  const cfg = SECTIONS.find((s) => s.value === section) || SECTIONS[0];
  const activeCompany = companies?.find((c) => c.id === activeCompanyId);

  // Devise depuis les préférences
  useEffect(() => {
    (async () => {
      try {
        const pr = await base44.entities.Preferences.list();
        setCur(pr?.[0]?.currency || "XAF");
      } catch { /* ignore */ }
    })();
  }, []);

  const load = async () => {
    setLoading(true);
    setRecords([]);
    try {
      let raw = [];
      if (section === "planning") {
```

```
        const [t, e] = await Promise.all([
          base44.entities.Task.list("-start", 2000),
          base44.entities.Event.list("-start", 2000),
        ]);
        raw = [
          ...(t || []).map((r) => ({ _type: "Tâche", company_id: r.company_id, title: r.title, start: r.start, end: r.end,
status: r.status })),
          ...(e || []).map((r) => ({ _type: "Évènement", company_id: r.company_id, title: r.title, start: r.start, end: r.end,
status: r.priority })),
        ];
      } else {
        raw = await base44.entities[cfg.entity].list("-created_date", 2000);
      }
      if (cfg.companyScoped && space === "jbm" && activeCompanyId) {
        raw = (raw || []).filter((r) => r.company_id === activeCompanyId);
      }
      setRecords(raw || []);
    } catch {
      setRecords([]);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { load(); /* eslint-disable-next-line */ }, [section, space, activeCompanyId]);

  const range = useMemo(() => computeRange(periodType, period), [periodType, period]);

  const filtered = useMemo(() => {
    const { start, end } = range;
    let rows = records.filter((r) => {
      const v = r[cfg.dateField];
      if (!v) return false;
      const d = new Date(v);
      return d >= start && d <= end;
    });
    if (aspect !== "tous" && cfg.aspects) {
      rows = rows.filter((r) => r[cfg.aspects.field] === aspect);
    }
    return rows.map((r) => cfg.toRow(r));
  }, [records, range, aspect, cfg, section]);

  const summary = useMemo(() => {
    const s = [
      { label: "Période", value: range.label },
      { label: "Nombre", value: String(filtered.length) },
    ];
    cfg.columns.filter((c) => c.type === "currency").forEach((c) => {
      const total = filtered.reduce((acc, r) => acc + (Number(r[c.key]) || 0), 0);
      s.push({ label: `${c.label} (total)`, value: formatCurrency(total, cur) });
    });
    return s;
  }, [filtered, cfg, range, cur]);

  const subtitle = space === "jbm"
    ? `Espace entreprise${activeCompany ? " · " + activeCompany.name : ""}`
    : "Espace personnel";

  const generate = (format) => {
    if (!filtered.length) {
      toast({ title: "Aucune donnée sur cette période", variant: "destructive" });
      return;
    }
    setBusy(true);
    try {
      const title = `${cfg.label} — ${range.label}`;
      if (format === "pdf") exportToPdf(title, subtitle, cfg.columns, filtered, cur, summary);
      else exportToXls(`${cfg.label} ${range.label}`, subtitle, cfg.columns, filtered, cur, summary);
      toast({ title: `Export ${format.toUpperCase()} généré` });
    } catch (e) {
      toast({ title: "Erreur lors de l'export", description: e?.message, variant: "destructive" });
    } finally {
      setBusy(false);
    }
  };

  return (
    <div className="space-y-6">
      <div>
        <h1 className="font-display text-2xl sm:text-3xl flex items-center gap-2"><FileDown className="w-6 h-6" />
Exportations</h1>
        <p className="text-muted-foreground mt-1">
          Générez un rapport Excel ou PDF sur n'importe quelle période pour les tâches, évènements, finances, planning, calculs,
devis, ventes, salariés, prestataires, partenariats et factures.
        </p>
      </div>

      <div className="bg-card border border-border rounded-2xl p-5 space-y-5">
        {/* Section */}
        <div className="space-y-1.5">
          <Label>Section</Label>
          <Select value={section} onValueChange={(v) => { setSection(v); setAspect("tous"); }}>
```

```
            <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
            <SelectContent>
              {SECTIONS.map((s) => <SelectItem key={s.value} value={s.value}>{s.label}</SelectItem>)}
            </SelectContent>
          </Select>
        </div>

        {/* Période */}
        <div className="space-y-1.5">
          <Label>Type de période</Label>
          <Select value={periodType} onValueChange={setPeriodType}>
            <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
            <SelectContent>
              {PERIOD_TYPES.map((p) => <SelectItem key={p.value} value={p.value}>{p.label}</SelectItem>)}
            </SelectContent>
          </Select>
        </div>

        <div className="grid grid-cols-2 gap-3">
          {(periodType === "annee" || periodType === "mois" || periodType === "trimestre" || periodType === "semestre" ||
periodType === "semaine") && (
            <div className="space-y-1.5">
              <Label>Année</Label>
              <Select value={String(period.year)} onValueChange={(v) => setPeriod((p) => ({ ...p, year: Number(v) }))}>
                <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
                <SelectContent>
                  {yearOptions().map((y) => <SelectItem key={y} value={String(y)}>{y}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          {periodType === "mois" && (
            <div className="space-y-1.5">
              <Label>Mois</Label>
              <Select value={String(period.month)} onValueChange={(v) => setPeriod((p) => ({ ...p, month: Number(v) }))}>
                <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
                <SelectContent>
                  {monthOptions.map((m) => <SelectItem key={m.value} value={String(m.value)}>{m.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          {periodType === "trimestre" && (
            <div className="space-y-1.5">
              <Label>Trimestre</Label>
              <Select value={String(period.quarter)} onValueChange={(v) => setPeriod((p) => ({ ...p, quarter: Number(v) }))}>
                <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
                <SelectContent>
                  {quarterOptions.map((q) => <SelectItem key={q.value} value={String(q.value)}>{q.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          {periodType === "semestre" && (
            <div className="space-y-1.5">
              <Label>Semestre</Label>
              <Select value={String(period.half)} onValueChange={(v) => setPeriod((p) => ({ ...p, half: Number(v) }))}>
                <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
                <SelectContent>
                  {halfOptions.map((h) => <SelectItem key={h.value} value={String(h.value)}>{h.label}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          {periodType === "semaine" && (
            <div className="space-y-1.5">
              <Label>Semaine (n°)</Label>
              <Select value={String(period.week)} onValueChange={(v) => setPeriod((p) => ({ ...p, week: Number(v) }))}>
                <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
                <SelectContent>
                  {weekOptions.map((w) => <SelectItem key={w} value={String(w)}>{w}</SelectItem>)}
                </SelectContent>
              </Select>
            </div>
          )}
          {periodType === "personnalisee" && (
            <>
              <div className="space-y-1.5">
                <Label>Du</Label>
                <Input type="date" value={period.from} onChange={(e) => setPeriod((p) => ({ ...p, from: e.target.value }))} />
              </div>
              <div className="space-y-1.5">
                <Label>Au</Label>
                <Input type="date" value={period.to} onChange={(e) => setPeriod((p) => ({ ...p, to: e.target.value }))} />
              </div>
            </>
          )}
        </div>

        {/* Aspect (filtre optionnel) */}
        {cfg.aspects && (
          <div className="space-y-1.5">
```

```
            <Label>Aspect (filtre)</Label>
            <Select value={aspect} onValueChange={setAspect}>
              <SelectTrigger className="w-full"><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="tous">Tous</SelectItem>
                {cfg.aspects.options.map((o) => <SelectItem key={o.value} value={o.value}>{o.label}</SelectItem>)}
              </SelectContent>
            </Select>
          </div>
        )}

        {/* Résumé */}
        <div className="grid grid-cols-2 sm:grid-cols-3 gap-3">
          {summary.map((s) => (
            <div key={s.label} className="bg-secondary/40 rounded-xl px-3 py-2">
              <p className="text-xs text-muted-foreground">{s.label}</p>
              <p className="text-sm font-medium truncate">{s.value}</p>
            </div>
          ))}
        </div>

        {/* Actions */}
        <div className="flex flex-wrap gap-2">
          <Button onClick={() => generate("pdf")} disabled={busy || loading || !filtered.length}>
            {busy ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <FileText className="w-4 h-4 mr-2" />}
            Exporter PDF
          </Button>
          <Button onClick={() => generate("xls")} disabled={busy || loading || !filtered.length} variant="outline">
            <Sheet className="w-4 h-4 mr-2" />
            Exporter Excel
          </Button>
        </div>
      </div>

      {/* Aperçu */}
      <div className="bg-card border border-border rounded-2xl p-5">
        <h2 className="font-display text-lg mb-3">Aperçu ({filtered.length})</h2>
        {loading ? (
          <div className="flex justify-center py-8"><Loader2 className="w-5 h-5 animate-spin text-muted-foreground" /></div>
        ) : filtered.length === 0 ? (
          <p className="text-sm text-muted-foreground">Aucun enregistrement sur cette période.</p>
        ) : (
          <div className="overflow-x-auto">
            <table className="w-full text-sm">
              <thead>
                <tr className="text-left text-muted-foreground">
                  {cfg.columns.map((c) => <th key={c.key} className="py-2 pr-3 whitespace-nowrap">{c.label}</th>)}
                </tr>
              </thead>
              <tbody>
                {filtered.slice(0, 8).map((r, i) => (
                  <tr key={i} className="border-t border-border">
                    {cfg.columns.map((c) => (
                      <td key={c.key} className="py-2 pr-3 whitespace-nowrap">
                        {c.type === "currency" ? formatCurrency(Number(r[c.key]) || 0, cur)
                          : c.type === "date" ? (r[c.key] ? new Date(r[c.key]).toLocaleDateString("fr-FR") : "")
                          : String(r[c.key] ?? "")}
                      </td>
                    ))}
                  </tr>
                ))}
              </tbody>
            </table>
            {filtered.length > 8 && <p className="text-xs text-muted-foreground mt-2">+ {filtered.length - 8} autres lignes dans
le fichier exporté.</p>}
          </div>
        )}
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Faq.jsx
------------------------------------------------------------
import React from "react";
import FaqSection from "@/components/landing/FaqSection";

export default function Faq() {
  return (
    <div className="space-y-6">
      <div>
        <h1 className="font-display text-2xl sm:text-3xl">FAQ</h1>
        <p className="text-muted-foreground mt-1">
          Questions fréquentes et aide. Posez une question à notre équipe si besoin.
        </p>
      </div>
      <FaqSection />
    </div>
  );
}
```

```
------------------------------------------------------------
FICHIER : ./src/pages/Finances.jsx
------------------------------------------------------------
import React, { useEffect, useMemo, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, PiggyBank, CheckSquare, Trash2, Tag, Landmark, Percent, Wallet, TrendingUp } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Tabs, TabsList, TabsTrigger, TabsContent } from "@/components/ui/tabs";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useSpace } from "@/lib/SpaceContext";
import { useI18n } from "@/lib/i18n";
import ExpenseForm from "@/components/finances/ExpenseForm";
import ExpenseItem from "@/components/finances/ExpenseItem";
import BulkActionBar from "@/components/common/BulkActionBar";
import PeriodPicker from "@/components/common/PeriodPicker";
import SavingsPlanList from "@/components/finances/SavingsPlanList";
import SavingsPlanForm from "@/components/finances/SavingsPlanForm";
import GainsPanel from "@/components/finances/GainsPanel";
import Loans from "@/pages/Loans";
import Taxes from "@/pages/Taxes";
import { expandRecurrence } from "@/lib/recurrence";
import { daysInPeriod, periodLabel, customOf } from "@/lib/periods";
import { formatCurrency } from "@/lib/format";
import ExportMenu from "@/components/ExportMenu";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";

const PAGE_SIZE = 100;

export default function Finances() {
  const { t } = useI18n();
  const [expenses, setExpenses] = useState([]);
  const [savings, setSavings] = useState([]);
  const [taxesJpm, setTaxesJpm] = useState([]);
  const [activeLoans, setActiveLoans] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);
  const [tab, setTab] = useState("depenses");
  const [formOpen, setFormOpen] = useState(false);
  const [editingExpense, setEditingExpense] = useState(null);
  const [budgetOpen, setBudgetOpen] = useState(false);
  const [budgetInput, setBudgetInput] = useState("");
  const [budgetPeriod, setBudgetPeriod] = useState("mois");
  const [budgetCustomCount, setBudgetCustomCount] = useState("");
  const [budgetCustomUnit, setBudgetCustomUnit] = useState("jour");
  const [savingsOpen, setSavingsOpen] = useState(false);
  const [visibleCount, setVisibleCount] = useState(PAGE_SIZE);
  const [fCat, setFCat] = useState("all");
  const [fPeriod, setFPeriod] = useState("all");
  const [fSort, setFSort] = useState("desc");
  const [selectMode, setSelectMode] = useState(false);
  const [selected, setSelected] = useState(new Set());
  const [bulkBusy, setBulkBusy] = useState(false);
  const [bulkCat, setBulkCat] = useState("");
  const { toast } = useToast();
  const { register } = useRefresh();
  const { space, activeCompanyId } = useSpace();

  const load = async () => {
    const [e, s, p, tx, ln] = await Promise.all([
      base44.entities.Expense.list("-date", 300),
      base44.entities.SavingsPlan.list("-created_date", 200).catch(() => []),
      base44.entities.Preferences.list(),
      base44.entities.Tax.list("-created_date", 500).catch(() => []),
      base44.entities.Loan.list("-created_date", 200).catch(() => []),
    ]);
    setExpenses(e);
    setSavings(s || []);
    setPreferences(p?.[0] || null);
    setTaxesJpm((tx || []).filter((t) =>
      space === "jpm"
        ? t.scope === "jpm"
        : (t.scope === "jbm" && (!activeCompanyId || t.company_id === activeCompanyId))
    ));
    setActiveLoans((ln || []).filter((l) => l.status === "en_cours"));
    setLoading(false);
  };

  useEffect(() => { register(load); });
  useEffect(() => { load(); }, [space, activeCompanyId]);
  // Recalcul automatique du reste budgétaire dès qu'une taxe ou un emprunt change.
  useEffect(() => {
    const trySub = (entity, cb) => (typeof entity?.subscribe === "function" ? entity.subscribe(cb) : () => {});
    const unsubTax = trySub(base44.entities.Tax, () => load());
    const unsubLoan = trySub(base44.entities.Loan, () => load());
    return () => { unsubTax?.(); unsubLoan?.(); };
  }, []);

  const currency = preferences?.currency || "XAF";
```

```
const budget = preferences?.initial_budget || 0;
const bp = preferences?.budget_period || "mois";
const bpCustom = { count: preferences?.budget_custom_count, unit: preferences?.budget_custom_unit };
const budgetDays = daysInPeriod(bp, new Date(), bpCustom);
const dailyBudget = budgetDays > 0 ? budget / budgetDays : 0;
const totalExpense = expenses.reduce((sum, e) => sum + (Number(e.quantity) || 0) * (Number(e.unit_cost) || 0), 0);

const activePlans = savings.filter((p) => p.status !== "archive");
const savingsForBudget = activePlans.reduce((acc, p) => {
  const pDays = daysInPeriod(p.period, new Date(p.start_date || Date.now()), customOf(p));
  const daily = pDays > 0 ? (Number(p.target_amount) || 0) / pDays : 0;
  return acc + daily * budgetDays;
}, 0);
const taxBurden = taxesJpm.reduce((s, t) => s + (Number(t.amount) || 0), 0);
const loanBurden = activeLoans.reduce((s, l) => s + (Number(l.monthly_payment) || 0), 0) * (budgetDays / 30.44);
const reste = budget - totalExpense - savingsForBudget - taxBurden - loanBurden;
const dailyAvailable = budgetDays > 0 ? reste / budgetDays : 0;

const categories = useMemo(
  () => [...new Set(expenses.map((e) => e.category).filter(Boolean))],
  [expenses]
);
const filtered = useMemo(() => {
  let list = expenses;
  if (fCat !== "all") list = list.filter((e) => e.category === fCat);
  if (fPeriod !== "all") {
    const now = new Date();
    list = list.filter((e) => {
      const d = new Date(e.date);
      if (fPeriod === "month") return d.getMonth() === now.getMonth() && d.getFullYear() === now.getFullYear();
      if (fPeriod === "30") return d >= new Date(now.getTime() - 30 * 86400000);
      if (fPeriod === "year") return d.getFullYear() === now.getFullYear();
      return true;
    });
  }
  return [...list].sort((a, b) =>
    fSort === "desc" ? new Date(b.date) - new Date(a.date) : new Date(a.date) - new Date(b.date)
  );
}, [expenses, fCat, fPeriod, fSort]);
const visible = filtered.slice(0, visibleCount);

const toggleSelect = (id) => setSelected((s) => {
  const n = new Set(s);
  if (n.has(id)) n.delete(id); else n.add(id);
  return n;
});
const allSelected = visible.length > 0 && visible.every((e) => selected.has(e.id));
const toggleAll = () => setSelected(allSelected ? new Set() : new Set(visible.map((e) => e.id)));
const exitSelect = () => { setSelectMode(false); setSelected(new Set()); };

const bulkDelete = async () => {
  const ids = [...selected];
  if (!ids.length) return;
  setBulkBusy(true);
  const prev = expenses;
  setExpenses((cur) => cur.filter((e) => !selected.has(e.id)));
  try {
    await base44.entities.Expense.deleteMany({ id: { $in: ids } });
    toast({ title: `${ids.length} dépense(s) supprimée(s)` });
    setSelected(new Set()); setBulkBusy(false); load();
  } catch {
    setExpenses(prev); setBulkBusy(false);
    toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
  }
};

const bulkSetCategory = async () => {
  const ids = [...selected];
  if (!ids.length || !bulkCat.trim()) return;
  setBulkBusy(true);
  const prev = expenses;
  const cat = bulkCat.trim();
  setExpenses((cur) => cur.map((e) => (selected.has(e.id) ? { ...e, category: cat } : e)));
  try {
    await base44.entities.Expense.updateMany({ id: { $in: ids } }, { $set: { category: cat } });
    toast({ title: `${ids.length} dépense(s) mise(s) à jour` });
    setSelected(new Set()); setBulkCat(""); setBulkBusy(false); load();
  } catch {
    setExpenses(prev); setBulkBusy(false);
    toast({ title: t("toast.error"), description: t("err.update"), variant: "destructive" });
  }
};

const handleSaveExpense = async (data, recurrence) => {
  if (editingExpense) {
    const prev = expenses;
    setExpenses((cur) => cur.map((e) => (e.id === editingExpense.id ? { ...e, ...data } : e)));
    try {
      await base44.entities.Expense.update(editingExpense.id, data);
      toast({ title: t("toast.expenseUpdated") });
      load();
    } catch (err) {
```

```
        setExpenses(prev);
        toast({ title: t("toast.error"), description: t("err.modify"), variant: "destructive" });
      }
    } else {
      const items = expandRecurrence(data, recurrence?.frequency, recurrence?.count, ["date"], recurrence?.custom);
      const tempIds = items.map(() => `tmp_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`);
      setExpenses((cur) => [...items.map((it, i) => ({ ...it, id: tempIds[i] })), ...cur]);
      try {
        await base44.entities.Expense.bulkCreate(items);
        toast({ title: recurrence && recurrence.count > 1 ? `${items.length} dépenses récurrentes créées` :
t("toast.expenseAdded") });
        load();
      } catch (err) {
        setExpenses((cur) => cur.filter((e) => !tempIds.includes(e.id)));
        toast({ title: t("toast.error"), description: t("err.create"), variant: "destructive" });
      }
    }
    setFormOpen(false);
    setEditingExpense(null);
  };

  const handleDelete = async (expense) => {
    const prev = expenses;
    setExpenses((cur) => cur.filter((e) => e.id !== expense.id));
    try {
      await base44.entities.Expense.delete(expense.id);
      toast({ title: t("toast.expenseDeleted") });
      load();
    } catch (err) {
      setExpenses(prev);
      toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
    }
  };

  const handleSaveBudget = async (e) => {
    e.preventDefault();
    const value = parseFloat(budgetInput);
    if (isNaN(value) || value < 0) return;
    const prevPrefs = preferences;
    const customCount = budgetPeriod === "personnalise" ? (parseInt(budgetCustomCount, 10) || 0) : null;
    const customUnit = budgetPeriod === "personnalise" ? budgetCustomUnit : null;
    const optimistic = preferences
      ? { ...preferences, initial_budget: value, budget_period: budgetPeriod, budget_custom_count: customCount,
budget_custom_unit: customUnit }
      : { initial_budget: value, budget_period: budgetPeriod, budget_custom_count: customCount, budget_custom_unit: customUnit,
currency };
    setPreferences(optimistic);
    setBudgetOpen(false);
    try {
      if (preferences) {
        await base44.entities.Preferences.update(preferences.id, { initial_budget: value, budget_period: budgetPeriod,
budget_custom_count: customCount, budget_custom_unit: customUnit });
      } else {
        const created = await base44.entities.Preferences.create({ initial_budget: value, budget_period: budgetPeriod,
budget_custom_count: customCount, budget_custom_unit: customUnit, currency });
        setPreferences(created);
      }
      toast({ title: t("toast.budgetUpdated") });
      load();
    } catch (err) {
      setPreferences(prevPrefs);
      toast({ title: t("toast.error"), description: t("err.budget"), variant: "destructive" });
    }
  };

  const createSavingsPlan = async (data) => {
    try {
      await base44.entities.SavingsPlan.create(data);
      toast({ title: "Plan d'épargne créé" });
      setSavingsOpen(false);
      load();
    } catch { toast({ title: t("toast.error"), description: t("err.create"), variant: "destructive" }); }
  };

  const deleteSavingsPlan = async (p) => {
    const prev = savings;
    setSavings((cur) => cur.filter((x) => x.id !== p.id));
    try {
      await base44.entities.SavingsPlan.delete(p.id);
      toast({ title: "Plan supprimé" });
      load();
    } catch {
      setSavings(prev);
      toast({ title: t("toast.error"), variant: "destructive" });
    }
  };

  const togglePlanReached = async (p) => {
    const next = p.status === "atteint" ? "actif" : "atteint";
    setSavings((cur) => cur.map((x) => (x.id === p.id ? { ...x, status: next } : x)));
    try {
      await base44.entities.SavingsPlan.update(p.id, { status: next });
```

```
      load();
    } catch {
      setSavings(savings);
      toast({ title: t("toast.error"), variant: "destructive" });
    }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{t("finances.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("finances.subtitle")}</p>
        </div>
      </div>

      <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
        <div className="bg-card border border-border rounded-2xl p-4">
          <p className="text-sm text-muted-foreground">{t("finances.budget")} <span className="text-xs">· par {periodLabel(bp,
bpCustom)}</span></p>
          <p className="font-display text-xl mt-1">{formatCurrency(budget, currency)}</p>
                    < p   c l a s s N a m e = " t e x t - x s   t e x t - m u t e d - f o r e g r o u n d   m t - 1 " >"H   { f o r m
j</p>
          <button onClick={() => { setBudgetInput(String(budget)); setBudgetPeriod(bp); setBudgetCustomCount(bpCustom.count ?
String(bpCustom.count) : ""); setBudgetCustomUnit(bpCustom.unit || "jour"); setBudgetOpen(true); }} className="text-sm
text-primary hover:underline mt-2">
            {t("finances.editBudget")}
          </button>
        </div>
        <div className="bg-card border border-border rounded-2xl p-4">
          <p className="text-sm text-muted-foreground">{t("finances.totalExpenses")}</p>
          <p className="font-display text-xl mt-1">{formatCurrency(totalExpense, currency)}</p>
        </div>
        <div className="bg-card border border-border rounded-2xl p-4">
          <p className="text-sm text-muted-foreground">Épargne programmée</p>
          <p className="font-display text-xl mt-1">{formatCurrency(savingsForBudget, currency)}</p>
          <p className="text-xs text-muted-foreground mt-1">{activePlans.length} plan(s) actif(s)</p>
        </div>
        <div className="bg-card border border-border rounded-2xl p-4">
          <p className="text-sm text-muted-foreground">Reste · budget journalier</p>
          <p className={`font-display text-xl mt-1 ${reste < 0 ? "text-destructive" : ""}`}>{formatCurrency(reste,
currency)}</p>
                    < p   c l a s s N a m e = " t e x t - x s   t e x t - m u t e d - f o r e g r o u n d   m t - 1 " >"H   { f o r m
          <p className="text-xs text-muted-foreground">dont taxes {formatCurrency(taxBurden, currency)} · emprunts
{formatCurrency(loanBurden, currency)}</p>
        </div>
      </div>

      <Tabs value={tab} onValueChange={setTab}>
        <TabsList className="w-full justify-start overflow-x-auto no-scrollbar">
          <TabsTrigger value="depenses"><Wallet className="w-4 h-4 mr-1.5" />Dépenses</TabsTrigger>
          <TabsTrigger value="epargne"><PiggyBank className="w-4 h-4 mr-1.5" />Épargne</TabsTrigger>
          <TabsTrigger value="emprunts"><Landmark className="w-4 h-4 mr-1.5" />Emprunts</TabsTrigger>
          <TabsTrigger value="taxes"><Percent className="w-4 h-4 mr-1.5" />Taxes</TabsTrigger>
          <TabsTrigger value="gains"><TrendingUp className="w-4 h-4 mr-1.5" />Gains & formules</TabsTrigger>
        </TabsList>

        <TabsContent value="depenses" className="space-y-4">
          <div className="flex items-center gap-2 flex-wrap justify-end">
            <ExportMenu
              title={t("report.expenses")}
              subtitle={t("report.expensesSub")}
              currency={currency}
              rows={expenses.map((e) => ({ ...e, total: (Number(e.quantity) || 0) * (Number(e.unit_cost) || 0) }))}
              columns={[
                { key: "name", label: t("finances.history") },
                { key: "category", label: t("jbm.product.category") },
                { key: "date", label: t("jbm.sale.date"), type: "date" },
                { key: "quantity", label: t("jbm.sale.quantity") },
                { key: "unit_cost", label: t("jbm.product.unitCost"), type: "currency" },
                { key: "total", label: t("finances.totalExpenses"), type: "currency" },
              ]}
              summary={[
                { label: t("finances.budget"), value: formatCurrency(budget, currency) },
                { label: t("finances.totalExpenses"), value: formatCurrency(totalExpense, currency) },
                { label: t("finances.remaining"), value: formatCurrency(reste, currency) },
              ]}
            />
            <Button variant={selectMode ? "secondary" : "outline"} onClick={() => (selectMode ? exitSelect() :
setSelectMode(true))}>
              <CheckSquare className="w-4 h-4 mr-2" /> {selectMode ? "Terminer" : "Sélectionner"}
            </Button>
            <Button onClick={() => { setEditingExpense(null); setFormOpen(true); }}>
              <Plus className="w-4 h-4 mr-2" /> {t("finances.add")}
```

```
            </Button>
          </div>

          <div className="flex flex-wrap items-center gap-2">
            <Select value={fCat} onValueChange={setFCat}>
              <SelectTrigger className="h-9 w-auto text-sm"><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="all">Toutes catégories</SelectItem>
                {categories.map((c) => (<SelectItem key={c} value={c}>{c}</SelectItem>))}
              </SelectContent>
            </Select>
            <Select value={fPeriod} onValueChange={setFPeriod}>
              <SelectTrigger className="h-9 w-auto text-sm"><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="all">Toute période</SelectItem>
                <SelectItem value="month">Ce mois</SelectItem>
                <SelectItem value="30">30 derniers jours</SelectItem>
                <SelectItem value="year">Cette année</SelectItem>
              </SelectContent>
            </Select>
            <Select value={fSort} onValueChange={setFSort}>
              <SelectTrigger className="h-9 w-auto text-sm"><SelectValue /></SelectTrigger>
              <SelectContent>
                <SelectItem value="desc">Plus récent</SelectItem>
                <SelectItem value="asc">Plus ancien</SelectItem>
              </SelectContent>
            </Select>
          </div>

          <div className="space-y-2">
            {filtered.length === 0 ? (
              <div className="text-center py-12 text-muted-foreground">
                <PiggyBank className="w-8 h-8 mx-auto mb-3 opacity-50" />
                <p className="text-sm">{expenses.length === 0 ? t("finances.empty") : "Aucune dépense ne correspond aux
filtres."}</p>
              </div>
            ) : (
              <div className="space-y-2">
                {selectMode && (
                  <BulkActionBar
                    count={selected.size}
                    total={visible.length}
                    allSelected={allSelected}
                    onToggleAll={toggleAll}
                    onClear={exitSelect}
                    busy={bulkBusy}
                  >
                    <div className="flex items-center gap-2">
                      <Select value={bulkCat} onValueChange={setBulkCat}>
                        <SelectTrigger className="h-9 w-40 text-sm"><SelectValue placeholder="Catégorie" /></SelectTrigger>
                        <SelectContent>
                          {categories.map((c) => <SelectItem key={c} value={c}>{c}</SelectItem>)}
                        </SelectContent>
                      </Select>
                      <Button size="sm" variant="outline" onClick={bulkSetCategory} disabled={!selected.size || !bulkCat ||
bulkBusy}>
                        <Tag className="w-4 h-4 mr-1" /> Appliquer
                      </Button>
                    </div>
                    <Button size="sm" variant="destructive" onClick={bulkDelete} disabled={!selected.size || bulkBusy}>
                      <Trash2 className="w-4 h-4 mr-1" /> Supprimer
                    </Button>
                  </BulkActionBar>
                )}
                {visible.map((e) => (
                  <ExpenseItem
                    key={e.id}
                    expense={e}
                    currency={currency}
                    selectable={selectMode}
                    selected={selected.has(e.id)}
                    onToggleSelect={toggleSelect}
                    onEdit={(exp) => { if (selectMode) return; setEditingExpense(exp); setFormOpen(true); }}
                    onDelete={handleDelete}
                  />
                ))}
                {visibleCount < filtered.length && (
                  <Button variant="outline" className="w-full" onClick={() => setVisibleCount((c) => c + PAGE_SIZE)}>
                    {t("finances.loadMore", { n: filtered.length - visibleCount })}
                  </Button>
                )}
              </div>
            )}
          </div>
        </TabsContent>

        <TabsContent value="epargne" className="space-y-4">
          <div className="flex items-center justify-end">
            <Button onClick={() => setSavingsOpen(true)}><Plus className="w-4 h-4 mr-2" /> Nouveau plan</Button>
          </div>
          <SavingsPlanList
            plans={activePlans}
```

```
            currency={currency}
            onDelete={deleteSavingsPlan}
            onToggleReached={togglePlanReached}
          />
        </TabsContent>

        <TabsContent value="emprunts">
          <Loans />
        </TabsContent>

        <TabsContent value="taxes">
          <Taxes scope={space} />
        </TabsContent>

        <TabsContent value="gains">
          <GainsPanel currency={currency} />
        </TabsContent>
      </Tabs>

      <ExpenseForm open={formOpen} onOpenChange={setFormOpen} expense={editingExpense} onSubmit={handleSaveExpense} />

      <SavingsPlanForm open={savingsOpen} onOpenChange={setSavingsOpen} onSubmit={createSavingsPlan} currency={currency} />

      <Dialog open={budgetOpen} onOpenChange={setBudgetOpen}>
        <DialogContent className="sm:max-w-sm">
          <DialogHeader>
            <DialogTitle className="font-display">{t("finances.setBudget")}</DialogTitle>
          </DialogHeader>
          <form onSubmit={handleSaveBudget} className="space-y-4">
            <div className="space-y-2">
              <Label htmlFor="budget">{t("finances.budgetLabel", { currency })}</Label>
              <Input id="budget" type="number" min="0" step="any" value={budgetInput} onChange={(e) =>
setBudgetInput(e.target.value)} autoFocus />
            </div>
            <PeriodPicker
              period={budgetPeriod}
              customCount={budgetCustomCount}
              customUnit={budgetCustomUnit}
              onChange={({ period, customCount, customUnit }) => { setBudgetPeriod(period); setBudgetCustomCount(customCount);
setBudgetCustomUnit(customUnit); }}
              label="Période du budget"
              id="budget-period"
            />
            {budgetPeriod !== "personnalise" && (
              <p className="text-xs text-muted-foreground">
                                B u d g e t   j o u r n a l i e r   :  "H   { f o r m a t C u r r e n c y ( ( p a r s e F l o a t ( b
count: budgetCustomCount, unit: budgetCustomUnit }) || 1), currency)}
              </p>
            )}
            <DialogFooter>
              <Button type="submit" className="w-full">{t("common.save")}</Button>
            </DialogFooter>
          </form>
        </DialogContent>
      </Dialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/ForgotPassword.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Mail, ArrowLeft, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";

export default function ForgotPassword() {
  const [email, setEmail] = useState("");
  const [loading, setLoading] = useState(false);
  const [sent, setSent] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    try {
      await base44.auth.resetPasswordRequest(email);
    } catch {
      // Always show success regardless
    } finally {
      setLoading(false);
      setSent(true);
    }
  };

  return (
    <AuthLayout
      icon={Mail}
```

```
      title="Réinitialiser le mot de passe"
      subtitle="Nous vous envoyons un lien pour le réinitialiser"
      footer={
        <Link to="/login" className="text-primary font-medium hover:underline">
          <ArrowLeft className="w-3 h-3 inline mr-1" />Retour à la connexion
        </Link>
      }
    >
      {sent ? (
        <p className="text-sm text-foreground text-center">
          Si un compte existe avec cet email, vous recevrez bientôt un lien de réinitialisation.
        </p>
      ) : (
        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="space-y-2">
            <Label htmlFor="email">Adresse email</Label>
            <div className="relative">
              <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
              <Input
                id="email"
                type="email"
                autoComplete="email"
                autoFocus
                placeholder="vous@exemple.com"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                className="pl-10 h-12"
                required
              />
            </div>
          </div>
          <Button type="submit" className="w-full h-12 font-medium" disabled={loading}>
            {loading ? (
              <>
                <Loader2 className="w-4 h-4 mr-2 animate-spin" />
                Envoi...
              </>
            ) : (
              "Envoyer le lien"
            )}
          </Button>
        </form>
      )}
    </AuthLayout>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Home.jsx
------------------------------------------------------------
import React from "react";
import { useAuth } from "@/lib/AuthContext";
import Landing from "@/pages/Landing";
import Onboarding from "@/pages/Onboarding";
import Dashboard from "@/pages/Dashboard";
import BusinessDashboard from "@/pages/BusinessDashboard";

export default function Home() {
  const { user, isLoadingAuth } = useAuth();

  if (isLoadingAuth || (user === undefined)) {
    return (
      <div className="fixed inset-0 flex items-center justify-center">
        <div className="w-8 h-8 border-4 border-slate-200 border-t-slate-800 rounded-full animate-spin" />
      </div>
    );
  }

  // Visiteur non connecté ! page d'accueil publique.
  if (!user) return <Landing />;

  const accountType = user?.data?.account_type || user?.account_type;

  // Utilisateur connecté sans abonnement (non admin) ! onboarding.
  if (!accountType && user.role !== "admin") return <Onboarding />;

  // Admin ! tableau de bord avec bascule JPM/JBM.
  if (user.role === "admin") return <Dashboard />;

  // Entreprise ! espace JBM.
  if (accountType === "entreprise") return <BusinessDashboard />;

  // Particulier ! espace JPM.
  return <Dashboard />;
}

------------------------------------------------------------
FICHIER : ./src/pages/Inventory.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Boxes } from "lucide-react";
```

```
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import { formatCurrency } from "@/lib/format";
import ExportMenu from "@/components/ExportMenu";
import ProductForm from "@/components/business/ProductForm";
import ProductItem from "@/components/business/ProductItem";

export default function Inventory() {
  const { t } = useI18n();
  const { activeCompanyId } = useSpace();
  const [products, setProducts] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);
  const [formOpen, setFormOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const [p, prefs] = await Promise.all([
      base44.entities.Product.list("-created_date", 500),
      base44.entities.Preferences.list(),
    ]);
    setProducts(p);
    setPreferences(prefs?.[0] || null);
    setLoading(false);
  };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const currency = preferences?.currency || "XAF";
  const myProducts = activeCompanyId
    ? products.filter((p) => p.company_id === activeCompanyId)
    : products;

  const save = async (data) => {
    const payload = { ...data, company_id: activeCompanyId };
    if (editing) {
      const prev = products;
      setProducts((cur) => cur.map((p) => (p.id === editing.id ? { ...p, ...payload } : p)));
      try {
        await base44.entities.Product.update(editing.id, payload);
        toast({ title: t("toast.taskUpdated") });
        load();
      } catch (e) {
        setProducts(prev);
        toast({ title: t("toast.error"), description: t("err.modify"), variant: "destructive" });
      }
    } else {
      const tmpId = `tmp_${Date.now()}`;
      setProducts((cur) => [{ ...payload, id: tmpId }, ...cur]);
      try {
        await base44.entities.Product.create(payload);
        toast({ title: t("toast.expenseAdded") });
        load();
      } catch (e) {
        setProducts((cur) => cur.filter((p) => p.id !== tmpId));
        toast({ title: t("toast.error"), description: t("err.create"), variant: "destructive" });
      }
    }
    setFormOpen(false);
    setEditing(null);
  };

  const remove = async (product) => {
    const prev = products;
    setProducts((cur) => cur.filter((p) => p.id !== product.id));
    try {
      await base44.entities.Product.delete(product.id);
      toast({ title: t("toast.taskDeleted") });
      load();
    } catch (e) {
      setProducts(prev);
      toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
    }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
```

```
          <h1 className="font-display text-2xl sm:text-3xl">{t("jbm.stock.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("jbm.stock.subtitle")}</p>
        </div>
        <div className="flex items-center gap-2">
          <ExportMenu
            title={t("report.stock")}
            subtitle={t("report.stockSub")}
            currency={currency}
            rows={myProducts}
            columns={[
              { key: "name", label: t("jbm.product.name") },
              { key: "sku", label: t("jbm.product.sku") },
              { key: "category", label: t("jbm.product.category") },
              { key: "quantity", label: t("jbm.product.quantity") },
              { key: "unit_cost", label: t("jbm.product.unitCost"), type: "currency" },
              { key: "sale_price", label: t("jbm.product.salePrice"), type: "currency" },
              { key: "reorder_level", label: t("jbm.product.reorder") },
            ]}
            summary={[{ label: t("jbm.kpi.stock"), value: formatCurrency(myProducts.reduce((s, p) => s + (Number(p.quantity) ||
0) * (Number(p.unit_cost) || 0), 0), currency) }]}
          />
          <Button onClick={() => { setEditing(null); setFormOpen(true); }}>
            <Plus className="w-4 h-4 mr-2" /> {t("jbm.add")}
          </Button>
        </div>
      </div>
      {myProducts.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground">
          <Boxes className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p className="text-sm">{t("jbm.stock.empty")}</p>
        </div>
      ) : (
        <div className="space-y-2">
          {myProducts.map((p) => (
            <ProductItem key={p.id} product={p} currency={currency} onEdit={(x) => { setEditing(x); setFormOpen(true); }}
onDelete={remove} />
          ))}
        </div>
      )}
      <ProductForm open={formOpen} onOpenChange={setFormOpen} product={editing} onSubmit={save} />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Invoices.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, FileText } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import { formatCurrency } from "@/lib/format";
import ExportMenu from "@/components/ExportMenu";
import InvoiceForm from "@/components/business/InvoiceForm";
import InvoiceItem from "@/components/business/InvoiceItem";

export default function Invoices() {
  const { t } = useI18n();
  const { activeCompanyId } = useSpace();
  const [invoices, setInvoices] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);
  const [formOpen, setFormOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const [i, prefs] = await Promise.all([
      base44.entities.Invoice.list("-issue_date", 500),
      base44.entities.Preferences.list(),
    ]);
    setInvoices(i);
    setPreferences(prefs?.[0] || null);
    setLoading(false);
  };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const currency = preferences?.currency || "XAF";
  const myInvoices = activeCompanyId ? invoices.filter((i) => i.company_id === activeCompanyId) : invoices;
  const unpaid = myInvoices.filter((i) => i.status !== "payee").reduce((sum, i) => sum + (Number(i.amount) || 0), 0);
  const nextNumber = `F-${new Date().getFullYear()}-${String(myInvoices.length + 1).padStart(4, "0")}`;

  const save = async (data) => {
    const payload = { ...data, company_id: activeCompanyId };
    if (editing) {
      const prev = invoices;
```

```
      setInvoices((cur) => cur.map((i) => (i.id === editing.id ? { ...i, ...payload } : i)));
      try {
        await base44.entities.Invoice.update(editing.id, payload);
        toast({ title: t("toast.taskUpdated") });
        load();
      } catch (e) {
        setInvoices(prev);
        toast({ title: t("toast.error"), description: t("err.modify"), variant: "destructive" });
      }
    } else {
      const tmpId = `tmp_${Date.now()}`;
      setInvoices((cur) => [{ ...payload, id: tmpId }, ...cur]);
      try {
        await base44.entities.Invoice.create(payload);
        toast({ title: t("toast.expenseAdded") });
        load();
      } catch (e) {
        setInvoices((cur) => cur.filter((x) => x.id !== tmpId));
        toast({ title: t("toast.error"), description: t("err.create"), variant: "destructive" });
      }
    }
    setFormOpen(false);
    setEditing(null);
  };

  const remove = async (inv) => {
    const prev = invoices;
    setInvoices((cur) => cur.filter((i) => i.id !== inv.id));
    try {
      await base44.entities.Invoice.delete(inv.id);
      toast({ title: t("toast.taskDeleted") });
      load();
    } catch (e) {
      setInvoices(prev);
      toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
    }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{t("jbm.invoices.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("jbm.invoices.subtitle")}</p>
        </div>
        <div className="flex items-center gap-2">
          <ExportMenu
            title={t("report.invoices")}
            subtitle={t("report.invoicesSub")}
            currency={currency}
            rows={myInvoices}
            columns={[
              { key: "number", label: t("jbm.invoice.number") },
              { key: "customer_name", label: t("jbm.invoice.customer") },
              { key: "amount", label: t("jbm.invoice.amount"), type: "currency" },
              { key: "issue_date", label: t("jbm.invoice.issueDate"), type: "date" },
              { key: "due_date", label: t("jbm.invoice.dueDate"), type: "date" },
              { key: "status", label: t("jbm.invoice.status") },
            ]}
            summary={[{ label: t("jbm.kpi.unpaid"), value: formatCurrency(unpaid, currency) }]}
          />
          <Button onClick={() => { setEditing(null); setFormOpen(true); }}>
            <Plus className="w-4 h-4 mr-2" /> {t("jbm.add")}
          </Button>
        </div>
      </div>
      <div className="bg-card border border-border rounded-2xl p-5">
        <p className="text-sm text-muted-foreground">{t("jbm.kpi.unpaid")}</p>
        <p className="font-display text-xl mt-1">{formatCurrency(unpaid, currency)}</p>
      </div>
      {myInvoices.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground">
          <FileText className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p>{t("jbm.invoices.empty")}</p>
        </div>
      ) : (
        <div className="space-y-2">
          {myInvoices.map((i) => (
            <InvoiceItem key={i.id} invoice={i} currency={currency} onEdit={(x) => { setEditing(x); setFormOpen(true); }}
onDelete={remove} />
          ))}
        </div>
      )}
      <InvoiceForm open={formOpen} onOpenChange={setFormOpen} invoice={editing} onSubmit={save} nextNumber={nextNumber} />
```

```
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Landing.jsx
------------------------------------------------------------
import React from "react";
import { motion } from "framer-motion";
import { Link, useNavigate } from "react-router-dom";
import {
  Wallet, ListChecks, CalendarRange, Bot, Building2, Boxes, ShoppingCart, FileText,
  Users, Gift, User, Check, ArrowRight, Sparkles, ShieldCheck, Handshake, Star, Zap,
  ChevronDown, Calculator, Bell, BarChart3, CalendarDays, Tag, UsersRound, HelpCircle,
  TrendingDown, CalendarClock, Gauge,
} from "lucide-react";
import { useAuth } from "@/lib/AuthContext";
import FaqSection from "@/components/landing/FaqSection";

const BRAND = "JH Manager";
const SHORT = "JH Personal Manager";
const BIZ = "JH Business Manager";

function useCta() {
  const { user } = useAuth();
  const navigate = useNavigate();
  const accountType = user?.data?.account_type || user?.account_type;
  const authed = !!user;
  const go = (path) => navigate(path);
  const scrollTo = (id) => document.getElementById(id)?.scrollIntoView({ behavior: "smooth" });
  return {
    authed,
    hasAccount: !!accountType,
    isParticulier: accountType === "particulier",
    isEntreprise: accountType === "entreprise",
    scrollTo,
    subscribe: (type) => {
      if (authed && !accountType) go("/");
      else if (!authed) go("/register?returnTo=/");
      else go("/");
    },
    affiliation: () => (authed ? go("/affiliation") : go("/register?returnTo=/affiliation")),
  };
}

function GradientBlob({ className }) {
  return (
    <div className={`pointer-events-none absolute rounded-full blur-3xl opacity-40 ${className}`} />
  );
}

export default function Landing() {
  const cta = useCta();

  return (
    <div className="min-h-screen bg-background overflow-x-hidden">
      {/* HERO */}
      <header className="relative px-4 pt-12 pb-16 sm:pt-16">
        <GradientBlob className="bg-primary/30 w-72 h-72 -top-10 -left-10" />
        <GradientBlob className="bg-accent/30 w-80 h-80 top-10 right-0" />
        <div className="relative max-w-5xl mx-auto text-center">
          <motion.div
            initial={{ opacity: 0, y: -10 }} animate={{ opacity: 1, y: 0 }}
            className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-primary/10 text-primary text-xs font-medium
mb-4"
          >
            <Sparkles className="w-3.5 h-3.5" /> {BRAND} · {SHORT} + {BIZ}
          </motion.div>
          <motion.h1
            initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.05 }}
            className="font-display text-4xl sm:text-6xl leading-tight"
          >
            Reprenez le contrôle de votre{" "}
            <span className="bg-gradient-to-r from-primary to-accent bg-clip-text text-transparent">vie personnelle</span>{" "}
            et de votre{" "}
            <span className="bg-gradient-to-r from-accent to-primary bg-clip-text text-transparent">entreprise</span>
          </motion.h1>
          <motion.p
            initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.1 }}
            className="text-muted-foreground mt-5 max-w-xl mx-auto text-base sm:text-lg"
          >
            {BRAND} centralise tout : finances, tâches, planning côté perso ({SHORT}), stock, ventes et partenariats
            côté entreprise ({BIZ}). Un seul compte, payé via Mobile Money.
          </motion.p>

          {/* Boutons d'accès rapide en haut */}
          <motion.div
            initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.13 }}
            className="mt-6 inline-flex flex-wrap items-center justify-center gap-2 sm:gap-3"
          >
            <button
              onClick={() => cta.scrollTo("abonnements")}
```

```
              className="no-select-tap min-touch inline-flex items-center gap-2 rounded-full bg-primary text-primary-foreground
px-4 py-2.5 text-sm font-medium hover:bg-primary/90 transition-colors shadow-sm"
            >
              <User className="w-4 h-4" /> Abonnement particulier <ChevronDown className="w-3.5 h-3.5" />
            </button>
            <button
              onClick={() => cta.scrollTo("abonnements")}
              className="no-select-tap min-touch inline-flex items-center gap-2 rounded-full bg-accent text-accent-foreground
px-4 py-2.5 text-sm font-medium hover:bg-accent/90 transition-colors shadow-sm"
            >
              <Building2 className="w-4 h-4" /> Abonnement entreprise <ChevronDown className="w-3.5 h-3.5" />
            </button>
            <button
              onClick={() => cta.scrollTo("affiliation")}
              className="no-select-tap min-touch inline-flex items-center gap-2 rounded-full border border-accent/40
bg-accent/10 text-accent-foreground px-4 py-2.5 text-sm font-medium hover:bg-accent/20 transition-colors shadow-sm"
            >
              <Gift className="w-4 h-4" /> Devenir affilié <ChevronDown className="w-3.5 h-3.5" />
            </button>
          </motion.div>

          <motion.div
            initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.16 }}
            className="mt-5 flex flex-col sm:flex-row gap-3 justify-center"
          >
            <Link to="#abonnements" className="no-select-tap min-touch inline-flex items-center justify-center gap-2 rounded-xl
bg-primary text-primary-foreground px-6 py-3 font-medium hover:bg-primary/90 transition-colors">
              Voir les abonnements <ArrowRight className="w-4 h-4" />
            </Link>
            <button onClick={cta.affiliation} className="no-select-tap min-touch inline-flex items-center justify-center gap-2
rounded-xl border border-accent/40 bg-accent/10 text-accent-foreground px-6 py-3 font-medium hover:bg-accent/20
transition-colors">
              <Gift className="w-4 h-4" /> Programme d'affiliation
            </button>
          </motion.div>
          {cta.authed && cta.hasAccount && (
            <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="mt-6">
              <Link to="/" className="text-sm text-primary font-medium hover:underline inline-flex items-center gap-1">
                Accéder à mon espace {cta.isEntreprise ? BIZ : SHORT} <ArrowRight className="w-3.5 h-3.5" />
              </Link>
            </motion.div>
          )}
        </div>
      </header>

      {/* PRÉSENTATION PLATEFORME */}
      <section className="px-4 py-12">
        <div className="max-w-4xl mx-auto">
          <SectionTitle eyebrow="La plateforme" title="Deux espaces, une seule application" />
          <p className="text-center text-muted-foreground mt-3 max-w-2xl mx-auto text-sm">
            {BRAND} sépare clairement votre vie personnelle ({SHORT}) de votre activité professionnelle ({BIZ}),
            tout en gardant tout au même endroit, accessible depuis votre téléphone.
          </p>
          <div className="grid sm:grid-cols-3 gap-4 mt-8">
            {[
              { icon: Zap, t: "Centralisé", d: "Finances, tâches, entreprise : un seul point d'entrée." },
              { icon: ShieldCheck, t: "Sécurisé", d: "Vos données restent privées, accessibles partout." },
              { icon: Bot, t: "Assisté par IA", d: "Des assistants intégrés pour vous guider au quotidien." },
            ].map((f, i) => (
              <motion.div key={f.t} initial={{ opacity: 0, y: 20 }} whileInView={{ opacity: 1, y: 0 }} viewport={{ once: true }}
transition={{ delay: i * 0.05 }}
                className="rounded-2xl border border-border bg-card p-5">
                <div className="w-10 h-10 rounded-xl bg-primary/10 flex items-center justify-center mb-3">
                  <f.icon className="w-5 h-5 text-primary" />
                </div>
                <h3 className="font-display text-lg">{f.t}</h3>
                <p className="text-sm text-muted-foreground mt-1">{f.d}</p>
              </motion.div>
            ))}
          </div>
        </div>
      </section>

      {/* FONCTIONNALITÉS — description claire et concise */}
      <section className="px-4 py-12">
        <div className="max-w-5xl mx-auto">
          <SectionTitle eyebrow="Fonctionnalités" title="Tout ce dont vous avez besoin" />
          <p className="text-center text-muted-foreground mt-3 max-w-2xl mx-auto text-sm">
            Chaque section a un rôle précis. Voici un aperçu de ce que vous trouverez dans {BRAND}.
          </p>
          <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-4 mt-8">
            {FEATURES.map((f, i) => (
              <motion.div key={f.t} initial={{ opacity: 0, y: 20 }} whileInView={{ opacity: 1, y: 0 }} viewport={{ once: true }}
transition={{ delay: (i % 3) * 0.05 }}
                className="rounded-2xl border border-border bg-card p-5 hover:shadow-md transition-shadow">
                <div className="w-10 h-10 rounded-xl bg-gradient-to-br from-primary/15 to-accent/15 flex items-center
justify-center mb-3">
                  <f.icon className="w-5 h-5 text-primary" />
                </div>
                <h3 className="font-display text-base">{f.t}</h3>
                <p className="text-sm text-muted-foreground mt-1">{f.d}</p>
              </motion.div>
```

```
            ))}
          </div>
        </div>
      </section>

      {/* SHORT */}
      <section className="px-4 py-12">
        <div className="max-w-4xl mx-auto rounded-3xl bg-gradient-to-br from-primary/10 to-primary/5 border border-primary/20
p-6 sm:p-10 relative overflow-hidden">
          <GradientBlob className="bg-primary/20 w-60 h-60 -bottom-10 -right-10" />
          <div className="relative">
            <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-primary text-primary-foreground text-xs
font-medium mb-4">
              <User className="w-3.5 h-3.5" /> {SHORT} — Gestion personnelle
            </div>
            <h2 className="font-display text-2xl sm:text-3xl">Gérez votre quotidien en toute sérénité</h2>
            <p className="text-muted-foreground mt-2 max-w-xl">
              Budget, dépenses, taxes, emprunts, tâches, planning et événements réunis pour garder le fil de votre vie
personnelle.
            </p>
            <div className="grid grid-cols-2 sm:grid-cols-3 gap-3 mt-6">
              {[
                { icon: Wallet, t: "Finances & budget" },
                { icon: ListChecks, t: "Tâches & to-do" },
                { icon: CalendarRange, t: "Planning & calendrier" },
                { icon: Bot, t: "Assistant IA" },
                { icon: Zap, t: "Calculatrice pro" },
                { icon: Users, t: "Collaboration" },
              ].map((f) => (
                <div key={f.t} className="flex items-center gap-2 rounded-xl bg-card/70 p-3 text-sm">
                  <f.icon className="w-4 h-4 text-primary shrink-0" /> {f.t}
                </div>
              ))}
            </div>
          </div>
        </div>
      </section>

      {/* BIZ */}
      <section className="px-4 py-12">
        <div className="max-w-4xl mx-auto rounded-3xl bg-gradient-to-br from-accent/10 to-accent/5 border border-accent/20 p-6
sm:p-10 relative overflow-hidden">
          <GradientBlob className="bg-accent/20 w-60 h-60 -bottom-10 -left-10" />
          <div className="relative">
            <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-accent text-accent-foreground text-xs
font-medium mb-4">
              <Building2 className="w-3.5 h-3.5" /> {BIZ} — Gestion d'entreprise
            </div>
            <h2 className="font-display text-2xl sm:text-3xl">Pilotez votre activité comme un pro</h2>
            <p className="text-muted-foreground mt-2 max-w-xl">
              Stock, ventes, clients, devis, taxes et partenariats : tout l'outil métier, en mobilité.
            </p>
            <div className="grid grid-cols-2 sm:grid-cols-3 gap-3 mt-6">
              {[
                { icon: Boxes, t: "Stock & inventaire" },
                { icon: ShoppingCart, t: "Ventes & clients" },
                { icon: FileText, t: "Devis" },
                { icon: Handshake, t: "Partenariats" },
                { icon: Bot, t: "Assistant IA" },
                { icon: Star, t: "Reporting" },
              ].map((f) => (
                <div key={f.t} className="flex items-center gap-2 rounded-xl bg-card/70 p-3 text-sm">
                  <f.icon className="w-4 h-4 text-accent-foreground shrink-0" /> {f.t}
                </div>
              ))}
            </div>
          </div>
        </div>
      </section>

      {/* ABONNEMENTS */}
      <section id="abonnements" className="px-4 py-14 scroll-mt-4">
        <div className="max-w-4xl mx-auto">
          <SectionTitle eyebrow="Tarifs" title="Choisissez votre espace" />
          <p className="text-center text-muted-foreground mt-3 max-w-2xl mx-auto text-sm">
            Un seul abonnement via Mobile Money (Airtel Money ou Moov Money, Gabon). Aucune carte bancaire requise.
          </p>
          <div className="grid sm:grid-cols-2 gap-5 mt-8">
            <PricingCard
              tone="primary" icon={User} badge={SHORT} title="Abonnement Particulier"
              price="5 000" unit="XAF / mois"
              features={["Finances & budget personnel", "Tâches, planning, calendrier", "Assistant IA", "Collaboration &
calculatrice"]}
              ctaLabel="Prendre un abonnement particulier"
              onCta={() => cta.subscribe("particulier")}
              authedDone={cta.authed && cta.hasAccount && cta.isParticulier}
            />
            <PricingCard
              tone="accent" icon={Building2} badge={BIZ} title="Abonnement Entreprise"
              price="20 000" unit="XAF / mois / entreprise"
              features={["Stock & inventaire", "Ventes, clients & devis", "Taxes & partenariats", "Assistant IA",
"Multi-entreprises"]}
```

```
              ctaLabel="Prendre un abonnement entreprise"
              onCta={() => cta.subscribe("entreprise")}
              authedDone={cta.authed && cta.hasAccount && cta.isEntreprise}
            />
          </div>

          <div id="affiliation" className="scroll-mt-4 mt-5" />
          <motion.div initial={{ opacity: 0, y: 20 }} whileInView={{ opacity: 1, y: 0 }} viewport={{ once: true }}
            className="mt-2 rounded-2xl border border-accent/30 bg-gradient-to-r from-accent/10 to-primary/5 p-5 flex flex-col
sm:flex-row items-center gap-4">
            <div className="w-12 h-12 rounded-2xl bg-accent text-accent-foreground flex items-center justify-center shrink-0">
              <Gift className="w-6 h-6" />
            </div>
            <div className="flex-1 text-center sm:text-left">
              <h3 className="font-display text-lg">Programme d'affiliation</h3>
              <p className="text-sm text-muted-foreground">
                Parrainez et gagnez 20 % de chaque abonnement (1 000 XAF/mois par particulier, 4 000 XAF/mois par entreprise).
Le filleul profite de 20 % de réduction pendant 6 mois.
              </p>
            </div>
            <button onClick={cta.affiliation} className="no-select-tap min-touch rounded-xl bg-accent text-accent-foreground
px-5 py-2.5 font-medium hover:bg-accent/90 transition-colors shrink-0">
              Devenir affilié
            </button>
          </motion.div>
        </div>
      </section>

      {/* FAQ */}
      <FaqSection />

      {/* FOOTER */}
      <footer className="border-t border-border px-4 py-10">
        <div className="max-w-4xl mx-auto flex flex-col sm:flex-row items-center justify-between gap-4 text-sm
text-muted-foreground">
          <div className="flex items-center gap-2">
            <span className="font-display text-xl text-foreground">{BRAND}</span>
            <span>· Gestion tout-en-un</span>
          </div>
          <div className="flex gap-4">
            <Link to="/conditions" className="hover:text-foreground">Conditions</Link>
            <Link to="/confidentialite" className="hover:text-foreground">Confidentialité</Link>
            <Link to="/login" className="hover:text-foreground">Connexion</Link>
          </div>
        </div>
      </footer>
    </div>
  );
}

const FEATURES = [
  { icon: Wallet, t: "Finances", d: "Budgets périodiques, dépenses, taxes (en % ou montant) et emprunts avec intérêts." },
  { icon: ListChecks, t: "Tâches", d: "Tâches ponctuelles ou périodiques, durée en minutes à mois, dates et heures." },
  { icon: CalendarClock, t: "Événements", d: "Activités, RDV et cérémonies avec rappels et catégories." },
  { icon: CalendarDays, t: "Calendrier", d: "Vue mensuelle des tâches et événements avec horaires." },
  { icon: CalendarRange, t: "Planning", d: "Planification claire de vos tâches et événements." },
  { icon: BarChart3, t: "Reporting", d: "Évolution des finances et activités + rapports PDF ou tableur." },
  { icon: Bot, t: "Assistants IA", d: "Conversation, upload et génération de fichiers multimédia par l'IA." },
  { icon: Calculator, t: "Calculateur", d: "Calculs, grandeurs, formules, opérations sur dates et historique." },
  { icon: UsersRound, t: "Collaborateurs", d: "Invitations, groupes administrés, chat, tâches et finances partagées." },
  { icon: Boxes, t: "Stock", d: "Articles, quantités, ajouts/retraits et ajouts périodiques automatiques." },
  { icon: ShoppingCart, t: "Ventes", d: "Import depuis le stock, services, tarification par formule, ventes périodiques." },
  { icon: Users, t: "Personnel", d: "Postes, salaires périodiques, primes, présences et discipline." },
  { icon: Handshake, t: "Partenariats", d: "Partenaires, modalités et notes traitées par l'IA." },
  { icon: Bell, t: "Notifications", d: "Rappels d'échéances, budget, messages, invitations et résumé matinal." },
  { icon: Gauge, t: "Seuils & alertes", d: "Alertes personnalisées sur budget, durée ou date." },
  { icon: Tag, t: "Catégories", d: "Rangez vos données par catégories prédéfinies dans chaque section." },
  { icon: Gift, t: "Affiliation", d: "Parrainez et gagnez 20 % sur chaque abonnement, à vie." },
];

function SectionTitle({ eyebrow, title }) {
  return (
    <div className="text-center">
      <motion.div initial={{ opacity: 0 }} whileInView={{ opacity: 1 }} viewport={{ once: true }}
        className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-accent/15 text-accent-foreground text-xs font-medium
mb-3">
        <Sparkles className="w-3.5 h-3.5" /> {eyebrow}
      </motion.div>
      <h2 className="font-display text-3xl sm:text-4xl">{title}</h2>
    </div>
  );
}

function PricingCard({ tone, icon: Icon, badge, title, price, unit, features, ctaLabel, onCta, authedDone }) {
  const isAccent = tone === "accent";
  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }} whileInView={{ opacity: 1, y: 0 }} viewport={{ once: true }}
      className={`rounded-3xl border p-6 flex flex-col ${isAccent ? "border-accent/30 bg-accent/5" : "border-primary/30
bg-primary/5"}`}
    >
      <div className="flex items-center gap-3">
```

```
        <div className={`w-12 h-12 rounded-2xl flex items-center justify-center ${isAccent ? "bg-accent text-accent-foreground"
: "bg-primary text-primary-foreground"}`}>
          <Icon className="w-6 h-6" />
        </div>
        <span className={`text-xs font-medium px-2 py-1 rounded-full ${isAccent ? "bg-accent/15 text-accent-foreground" :
"bg-primary/10 text-primary"}`}>{badge}</span>
      </div>
      <h3 className="font-display text-xl mt-4">{title}</h3>
      <div className="mt-2 flex items-baseline gap-1">
        <span className="font-display text-4xl">{price}</span>
        <span className="text-sm text-muted-foreground">{unit}</span>
      </div>
      <ul className="mt-4 space-y-2 flex-1">
        {features.map((f) => (
          <li key={f} className="flex items-center gap-2 text-sm">
            <Check className={`w-4 h-4 shrink-0 ${isAccent ? "text-accent-foreground" : "text-primary"}`} /> {f}
          </li>
        ))}
      </ul>
      {authedDone ? (
        <div className="mt-5 rounded-xl bg-primary/10 text-primary text-sm font-medium py-2.5 text-center">
                    A b o n n e m e n t   a c t i f  '
        </div>
      ) : (
        <button onClick={onCta} className={`no-select-tap mt-5 min-touch w-full rounded-xl font-medium py-3 transition-colors
${isAccent ? "bg-accent text-accent-foreground hover:bg-accent/90" : "bg-primary text-primary-foreground
hover:bg-primary/90"}`}>
          {ctaLabel}
        </button>
      )}
    </motion.div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Loans.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Landmark } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { formatCurrency } from "@/lib/format";
import LoanForm from "@/components/loans/LoanForm";
import LoanItem from "@/components/loans/LoanItem";
import { buildSchedule } from "@/lib/loanMath";
import { shiftDateBy } from "@/lib/recurrence";
import { customOf } from "@/lib/periods";

export default function Loans() {
  const [loans, setLoans] = useState([]);
  const [payments, setPayments] = useState([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [payingId, setPayingId] = useState(null);
  const [del, setDel] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const [l, p] = await Promise.all([
      base44.entities.Loan.list("-created_date", 200).catch(() => []),
      base44.entities.LoanPayment.list("-date", 500).catch(() => []),
    ]);
    setLoans(l || []);
    setPayments(p || []);
    setLoading(false);
  };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const active = loans.filter((x) => x.status !== "archive");
  const totalRemaining = active.reduce((s, l) => s + (Number(l.remaining_balance) || 0), 0);
  const totalMonthly = active.reduce((s, l) => s + (l.status === "rembourse" ? 0 : (Number(l.monthly_payment) || 0)), 0);

  const create = async (data) => {
    try {
      await base44.entities.Loan.create(data);
      toast({ title: "Emprunt créé" });
      setOpen(false);
      load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const pay = async (loan) => {
    setPayingId(loan.id);
    try {
      const schedule = buildSchedule(loan.principal, loan.interest_rate, loan.term_periods, loan.interest_type);
      const idx = loan.payments_made || 0;
      if (idx >= schedule.length) return;
      const row = schedule[idx];
```

```
      const newMade = idx + 1;
      const remaining = schedule[newMade - 1].balance;
      const today = new Date().toISOString().slice(0, 10);
      await base44.entities.LoanPayment.create({
        loan_id: loan.id,
        loan_name: loan.name,
        payment_number: newMade,
        amount: row.payment,
        interest_portion: row.interest,
        principal_portion: row.principal,
        remaining_balance: remaining,
        date: today,
      });
      const nextDate = newMade >= schedule.length ? loan.next_payment_date : shiftDateBy(loan.start_date, loan.frequency ||
"mois", newMade, customOf(loan));
      await base44.entities.Loan.update(loan.id, {
        payments_made: newMade,
        remaining_balance: remaining,
        next_payment_date: nextDate,
        status: newMade >= schedule.length ? "rembourse" : "en_cours",
      });
      toast({ title: `Échéance n°${newMade} enregistrée` });
      load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
    setPayingId(null);
  };

  const remove = async (loan) => {
    try {
      await base44.entities.LoanPayment.deleteMany({ loan_id: loan.id }).catch(() => {});
      await base44.entities.Loan.delete(loan.id);
      toast({ title: "Emprunt supprimé" });
      setDel(null);
      load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  if (loading) return <div className="flex items-center justify-center h-64"><div className="w-6 h-6 border-2 border-border
border-t-primary rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div><h1 className="font-display text-2xl sm:text-3xl">Emprunts & dettes</h1><p className="text-muted-foreground
mt-1">Suivez vos dettes avec intérêts et période, liées à vos finances.</p></div>
        <Button onClick={() => setOpen(true)}><Plus className="w-4 h-4 mr-2" /> Nouvel emprunt</Button>
      </div>

      <div className="grid grid-cols-2 gap-3">
        <div className="bg-card border border-border rounded-2xl p-4"><p className="text-sm text-muted-foreground">Dette
restante totale</p><p className="font-display text-xl mt-1">{formatCurrency(totalRemaining, "XAF")}</p></div>
        <div className="bg-card border border-border rounded-2xl p-4"><p className="text-sm text-muted-foreground">Engagement /
échéance</p><p className="font-display text-xl mt-1">{formatCurrency(totalMonthly, "XAF")}</p></div>
      </div>

      {active.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground"><Landmark className="w-8 h-8 mx-auto mb-3 opacity-50" /><p
className="text-sm">Aucun emprunt en cours.</p></div>
      ) : (
        <div className="space-y-3">
          {active.map((loan) => (
            <LoanItem key={loan.id} loan={loan} payments={payments} onPay={pay} onDelete={remove} payingId={payingId}
delId={del} setDelId={setDel} />
          ))}
        </div>
      )}

      <LoanForm open={open} onOpenChange={setOpen} onSubmit={create} loading={false} />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Login.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { LogIn, Mail, Lock, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";
import GoogleIcon from "@/components/GoogleIcon";
import { safeReturnTo } from "@/lib/authReturnTo";

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  // Post-login destination (e.g. the MCP OAuth consent page sends users here
```

```
// with returnTo so the grant flow can resume). Same-origin paths only.
const returnTo = safeReturnTo();

const handleSubmit = async (e) => {
  e.preventDefault();
  setError("");
  setLoading(true);
  try {
    await base44.auth.loginViaEmailPassword(email, password);
    window.location.href = returnTo;
  } catch (err) {
    setError(err.message || "Email ou mot de passe invalide");
  } finally {
    setLoading(false);
  }
};

const handleGoogle = () => {
  base44.auth.loginWithProvider("google", returnTo);
};

return (
  <AuthLayout
    icon={LogIn}
    title="Bon retour"
    subtitle="Connectez-vous à votre compte"
    footer={
      <>
        Pas encore de compte ?{" "}
        <Link
          to={"/register" + (returnTo !== "/" ? "?returnTo=" + encodeURIComponent(returnTo) : "")}
          className="text-primary font-medium hover:underline"
        >
          Créer un compte
        </Link>
      </>
    }
  >
    <Button
      variant="outline"
      className="w-full h-12 text-sm font-medium mb-6"
      onClick={handleGoogle}
    >
      <GoogleIcon className="w-5 h-5 mr-2" />
      Continuer avec Google
    </Button>

    <div className="relative mb-6">
      <div className="absolute inset-0 flex items-center">
        <div className="w-full border-t border-border" />
      </div>
      <div className="relative flex justify-center text-xs uppercase">
        <span className="bg-card px-3 text-muted-foreground">ou</span>
      </div>
    </div>

    {error && (
      <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
        {error}
      </div>
    )}

    <form onSubmit={handleSubmit} className="space-y-4">
      <div className="space-y-2">
        <Label htmlFor="email">Email</Label>
        <div className="relative">
          <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
          <Input
            id="email"
            type="email"
            autoComplete="email"
            autoFocus
            placeholder="vous@exemple.com"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="pl-10 h-12"
            required
          />
        </div>
      </div>
      <div className="space-y-2">
        <div className="flex items-center justify-between">
          <Label htmlFor="password">Mot de passe</Label>
          <Link to="/forgot-password" className="text-xs text-primary hover:underline">
            Mot de passe oublié ?
          </Link>
        </div>
        <div className="relative">
          <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
          <Input
            id="password"
            type="password"
```

```
              autoComplete="current-password"
              placeholder="••••••••"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <Button type="submit" className="w-full h-12 font-medium" disabled={loading}>
          {loading ? (
            <>
              <Loader2 className="w-4 h-4 mr-2 animate-spin" />
              Connexion...
            </>
          ) : (
            "Se connecter"
          )}
        </Button>
      </form>
    </AuthLayout>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/MorningBriefing.jsx
------------------------------------------------------------
import React, { useEffect, useRef, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Sunrise, Loader2, FileText, Sheet, ImageIcon, RefreshCw } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { exportToXls } from "@/lib/exportReport";
import html2canvas from "html2canvas";
import { jsPDF } from "jspdf";

const PRIO_LABEL = { basse: "Basse", normale: "Normale", haute: "Haute" };

export default function MorningBriefing() {
  const { toast } = useToast();
  const cardRef = useRef(null);
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [busy, setBusy] = useState(false);

  const load = async () => {
    setLoading(true);
    try {
      const res = await base44.functions.invoke('morningBriefing', {});
      setData(res);
    } catch (e) {
      toast({ title: "Erreur", description: e?.message, variant: "destructive" });
    } finally {
      setLoading(false);
    }
  };
  useEffect(() => { load(); }, []);

  const cur = data?.kpis?.currency || "XAF";

  const exportImage = async () => {
    if (!cardRef.current) return;
    setBusy(true);
    try {
      const canvas = await html2canvas(cardRef.current, { backgroundColor: "#ffffff", scale: 2 });
      const url = canvas.toDataURL("image/png");
      const a = document.createElement("a");
      a.href = url;
      a.download = `resume_matinal_${data?.date || ""}.png`;
      a.click();
      toast({ title: "Image générée" });
    } catch (e) { toast({ title: "Erreur export image", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  const exportPdf = async () => {
    if (!cardRef.current) return;
    setBusy(true);
    try {
      const canvas = await html2canvas(cardRef.current, { backgroundColor: "#ffffff", scale: 2 });
      const img = canvas.toDataURL("image/png");
      const pdf = new jsPDF({ orientation: "portrait", unit: "pt", format: "a4" });
      const pw = pdf.internal.pageSize.getWidth();
      const ph = pdf.internal.pageSize.getHeight();
      const imgW = pw - 60;
      const imgH = (canvas.height * imgW) / canvas.width;
      let y = 30;
      if (imgH < ph - 60) {
        pdf.addImage(img, "PNG", 30, y, imgW, imgH);
      } else {
        // paginated
```

```
        let remaining = canvas.height;
        let srcY = 0;
        const pageImgH = (ph - 60) * (canvas.width / imgW);
        while (remaining > 0) {
          const sliceCanvas = document.createElement("canvas");
          sliceCanvas.width = canvas.width;
          sliceCanvas.height = Math.min(pageImgH, remaining);
          const ctx = sliceCanvas.getContext("2d");
          ctx.drawImage(canvas, 0, srcY, canvas.width, sliceCanvas.height, 0, 0, canvas.width, sliceCanvas.height);
          pdf.addImage(sliceCanvas.toDataURL("image/png"), "PNG", 30, 30, imgW, (sliceCanvas.height * imgW) / canvas.width);
          srcY += sliceCanvas.height;
          remaining -= sliceCanvas.height;
          if (remaining > 0) pdf.addPage();
        }
      }
      pdf.save(`resume_matinal_${data?.date || ""}.pdf`);
      toast({ title: "PDF généré" });
    } catch (e) { toast({ title: "Erreur export PDF", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  const exportExcel = () => {
    if (!data) return;
    setBusy(true);
    try {
      const rows = [
        ...(data.tasksToday || []).map((t) => ({ section: "Tâches du jour", titre: t.title, detail: PRIO_LABEL[t.priority] ||
"", date: t.start ? formatDateFr(t.start) : "" })),
        ...(data.eventsToday || []).map((ev) => ({ section: "Événements du jour", titre: ev.title, detail: ev.location || "",
date: ev.start ? formatDateFr(ev.start) : "" })),
        ...(data.tasksUpcoming || []).map((t) => ({ section: "Tâches à venir", titre: t.title, detail: PRIO_LABEL[t.priority] ||
"", date: t.start ? formatDateFr(t.start) : "" })),
        ...(data.recentExpenses || []).map((e) => ({ section: "Dépenses récentes", titre: e.name, detail:
formatCurrency(e.amount, cur), date: e.date ? formatDateFr(e.date) : "" })),
      ];
      const columns = [
        { key: "section", label: "Section" },
        { key: "titre", label: "Titre" },
        { key: "detail", label: "Détail" },
        { key: "date", label: "Date" },
      ];
      const summary = [
        { label: "Date", value: data.date || "" },
        { label: "Budget", value: formatCurrency(data.kpis.budget, cur) },
        { label: "Dépenses du jour", value: formatCurrency(data.kpis.depensesJour, cur) },
        { label: "Reste", value: formatCurrency(data.kpis.reste, cur) },
      ];
      exportToXls("Résumé matinal", data.date || "", columns, rows, cur, summary);
      toast({ title: "Excel généré" });
    } catch (e) { toast({ title: "Erreur export Excel", description: e?.message, variant: "destructive" }); }
    finally { setBusy(false); }
  };

  return (
    <div className="space-y-5">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl flex items-center gap-2"><Sunrise className="w-6 h-6 text-accent" />
Résumé matinal</h1>
          <p className="text-muted-foreground mt-1">Votre synthèse du jour, générée puis exportable en image, PDF ou Excel.</p>
        </div>
        <div className="flex items-center gap-2 flex-wrap">
          <Button variant="outline" size="sm" onClick={load} disabled={loading}><RefreshCw className={`w-4 h-4 mr-1 ${loading ?
"animate-spin" : ""}`} /> Actualiser</Button>
          <Button size="sm" onClick={exportImage} disabled={busy || !data}><ImageIcon className="w-4 h-4 mr-1" /> Image</Button>
          <Button size="sm" onClick={exportPdf} disabled={busy || !data}><FileText className="w-4 h-4 mr-1" /> PDF</Button>
          <Button size="sm" variant="outline" onClick={exportExcel} disabled={busy || !data}><Sheet className="w-4 h-4 mr-1" />
Excel</Button>
        </div>
      </div>

      {loading ? (
        <div className="flex justify-center py-16"><Loader2 className="w-6 h-6 animate-spin text-muted-foreground" /></div>
      ) : data ? (
        <div ref={cardRef} className="bg-card border border-border rounded-2xl p-5 space-y-5">
          <div>
            <p className="text-sm text-muted-foreground">{data.greeting} · {formatDateFr(data.date)}</p>
            {data.message && <p className="text-sm mt-3 whitespace-pre-wrap leading-relaxed">{data.message}</p>}
          </div>

          <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
            <Kpi label="Budget" value={formatCurrency(data.kpis.budget, cur)} />
            <Kpi label="Dépenses du jour" value={formatCurrency(data.kpis.depensesJour, cur)} />
            <Kpi label="Total dépenses" value={formatCurrency(data.kpis.totalExpense, cur)} />
            <Kpi label="Reste" value={formatCurrency(data.kpis.reste, cur)} tone={data.kpis.reste < 0 ? "danger" : "ok"} />
            <Kpi label="Tâches du jour" value={String(data.kpis.tasksToday)} />
            <Kpi label="Tâches à venir" value={String(data.kpis.tasksUpcoming)} />
            <Kpi label="Événements du jour" value={String(data.kpis.eventsToday)} />
            <Kpi label="Plans d'épargne actifs" value={String(data.kpis.activePlans)} />
          </div>

          <Section title="Tâches du jour" items={(data.tasksToday || []).map((t) => `${t.title}${t.priority ? ` ·
```

```
${PRIO_LABEL[t.priority] || ""}` : ""}`)} />
          <Section title="Événements du jour" items={(data.eventsToday || []).map((ev) => `${ev.title}${ev.location ? ` ·
${ev.location}` : ""}`)} />
          <Section title="Tâches à venir" items={(data.tasksUpcoming || []).map((t) => `${t.title}${t.start ? ` ·
${formatDateFr(t.start)}` : ""}`)} />
          <Section title="Dépenses récentes" items={(data.recentExpenses || []).map((e) => `${e.name} ·
${formatCurrency(e.amount, cur)}${e.date ? ` · ${formatDateFr(e.date)}` : ""}`)} />
        </div>
      ) : null}
    </div>
  );
}

function Kpi({ label, value, tone }) {
  const cls = tone === "danger" ? "text-destructive" : tone === "ok" ? "text-primary" : "";
  return (
    <div className="bg-secondary/40 rounded-xl px-3 py-2">
      <p className="text-xs text-muted-foreground">{label}</p>
      <p className={`text-sm font-medium truncate ${cls}`}>{value}</p>
    </div>
  );
}

function Section({ title, items }) {
  if (!items || items.length === 0) return null;
  return (
    <div>
      <p className="text-sm font-medium mb-1.5">{title}</p>
      <ul className="space-y-1">
        {items.map((it, i) => <li key={i} className="text-sm text-muted-foreground flex gap-2"><span
className="text-accent">•</span><span>{it}</span></li>)}
      </ul>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/NotificationsPage.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Bell, BellOff, Check, Trash2, Plus } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Switch } from "@/components/ui/switch";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useToast } from "@/components/ui/use-toast";
import { formatDateFr } from "@/lib/format";

const TYPE_LABEL = { rappel: "Rappel", conseil: "Conseil", personnalisee: "Personnalisée", systeme: "Système" };

export default function NotificationsPage() {
  const [items, setItems] = useState([]);
  const [prefs, setPrefs] = useState(null);
  const [loading, setLoading] = useState(true);
  const [allEnabled, setAllEnabled] = useState(true);
  const [title, setTitle] = useState("");
  const [message, setMessage] = useState("");
  const [type, setType] = useState("personnalisee");
  const { toast } = useToast();

  const load = async () => {
    try {
      const [list, p] = await Promise.all([
        base44.entities.Notification.list("-created_date", 200),
        base44.entities.Preferences.list(),
      ]);
      setItems(list || []);
      const pr = p?.[0] || null;
      setPrefs(pr);
      setAllEnabled(pr?.notifications_enabled !== false);
    } catch {
      setItems([]);
    }
    setLoading(false);
  };

  useEffect(() => { load(); }, []);

  const toggleAll = async (val) => {
    setAllEnabled(val);
    try {
      if (prefs) await base44.entities.Preferences.update(prefs.id, { notifications_enabled: val });
      else { const c = await base44.entities.Preferences.create({ notifications_enabled: val }); setPrefs(c); }
      toast({ title: val ? "Notifications activées" : "Notifications désactivées" });
    } catch {
      setAllEnabled(!val);
      toast({ title: "Erreur", variant: "destructive" });
    }
  };
```

```
  const create = async (e) => {
    e.preventDefault();
    if (!title.trim()) return;
    try {
      await base44.entities.Notification.create({ title, message, type, source: "manuel", read: false, enabled: true });
      setTitle(""); setMessage("");
      toast({ title: "Notification créée" });
      load();
    } catch {
      toast({ title: "Erreur", variant: "destructive" });
    }
  };

  const markRead = async (n) => {
    try { await base44.entities.Notification.update(n.id, { read: true }); load(); } catch { /* ignore */ }
  };
  const markAllRead = async () => {
    try { await base44.entities.Notification.updateMany({ read: false }, { $set: { read: true } }); load(); } catch { /* ignore
*/ }
  };
  const toggleEnabled = async (n) => {
    try { await base44.entities.Notification.update(n.id, { enabled: !n.enabled }); load(); } catch { /* ignore */ }
  };
  const remove = async (n) => {
    try { await base44.entities.Notification.delete(n.id); load(); } catch { /* ignore */ }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">Notifications</h1>
          <p className="text-muted-foreground mt-1">Vos rappels et notifications personnalisées.</p>
        </div>
        <div className="flex items-center gap-2">
          <span className="text-sm text-muted-foreground">{allEnabled ? "Activées" : "Désactivées"}</span>
          <Switch checked={allEnabled} onCheckedChange={toggleAll} />
        </div>
      </div>

      <form onSubmit={create} className="bg-card border border-border rounded-2xl p-5 space-y-3">
        <h2 className="font-display text-lg">Créer une notification</h2>
        <div className="space-y-2"><Label>Titre</Label><Input value={title} onChange={(e) => setTitle(e.target.value)}
placeholder="Rappel facture" /></div>
        <div className="space-y-2"><Label>Message</Label><Input value={message} onChange={(e) => setMessage(e.target.value)}
placeholder="Penser à…" /></div>
        <div className="flex gap-2 flex-wrap">
          <Select value={type} onValueChange={setType}>
            <SelectTrigger className="w-40"><SelectValue /></SelectTrigger>
            <SelectContent>
              <SelectItem value="rappel">Rappel</SelectItem>
              <SelectItem value="conseil">Conseil</SelectItem>
              <SelectItem value="personnalisee">Personnalisée</SelectItem>
              <SelectItem value="systeme">Système</SelectItem>
            </SelectContent>
          </Select>
          <Button type="submit"><Plus className="w-4 h-4 mr-1" /> Ajouter</Button>
        </div>
        <p className="text-xs text-muted-foreground">Pour des notifications plus élaborées, demandez à l'assistant Notifications
(page Assistants).</p>
      </form>

      <div className="flex items-center justify-between">
        <h2 className="font-display text-lg">Historique</h2>
        <Button variant="outline" size="sm" onClick={markAllRead}><Check className="w-4 h-4 mr-1" /> Tout marquer lu</Button>
      </div>

      {items.length === 0 ? (
        <p className="text-sm text-muted-foreground text-center py-8">Aucune notification.</p>
      ) : (
        <div className="space-y-2">
          {items.map((n) => (
            <div key={n.id} className={`bg-card border rounded-xl p-4 flex items-start justify-between gap-3 ${n.read ?
"border-border" : "border-primary/40 bg-primary/5"} ${!n.enabled ? "opacity-50" : ""}`}>
              <div className="min-w-0">
                <div className="flex items-center gap-2 flex-wrap">
                  <p className="text-sm font-medium">{n.title}</p>
                  <span className="text-xs px-2 py-0.5 rounded-full bg-secondary">{TYPE_LABEL[n.type] || n.type}</span>
                  {!n.enabled && <span className="text-xs text-muted-foreground">désactivée</span>}
                </div>
                {n.message && <p className="text-sm text-muted-foreground mt-1">{n.message}</p>}
                <p className="text-xs text-muted-foreground mt-1">{n.created_date ? formatDateFr(n.created_date, true) : ""}</p>
              </div>
              <div className="flex items-center gap-1 shrink-0">
```

```
              {!n.read && <button onClick={() => markRead(n)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary" title="Marquer lu"><Check className="w-4 h-4" /></button>}
              <button onClick={() => toggleEnabled(n)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-secondary"
title="Activer/Désactiver">{n.enabled ? <Bell className="w-4 h-4" /> : <BellOff className="w-4 h-4" />}</button>
              <button onClick={() => remove(n)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive" title="Supprimer"><Trash2 className="w-4 h-4" /></button>
            </div>
          </div>
        ))}
      </div>
    )}
  </div>
 );
}

------------------------------------------------------------
FICHIER : ./src/pages/OAuthConsent.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { appParams } from "@/lib/app-params";
import { Button } from "@/components/ui/button";
import { ShieldCheck, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";

// App-side OAuth consent page for the app's MCP server. The platform redirects
// AI clients here (see base44/mcp/config.json `consent_path`) with an opaque
// `ctx` handle — the authorization request itself lives on the server. This page
// gates on the app-user session, fetches the display info for that handle, shows
// the categories of access being granted, and posts the approve/deny decision.
// Do not change the fetch calls, headers, or the `ctx` handle handling — styling
// and copy are safe to edit.
export default function OAuthConsent() {
  const ctx = new URLSearchParams(window.location.search).get("ctx");
  const [info, setInfo] = useState(null);
  const [checking, setChecking] = useState(true);
  const [submitting, setSubmitting] = useState(false);
  const [decided, setDecided] = useState("");
  const [error, setError] = useState("");
  const [reconnect, setReconnect] = useState("");

  useEffect(() => {
    (async () => {
      let redirecting = false;
      try {
        if (!ctx) {
          setError("This authorization link is invalid or has expired.");
          return;
        }
        // Resolve the handle first: a dead handle must never render
        // approve/deny, and the response carries the app's configured login
        // route for the signed-out redirect below. Send the session (cookie +
        // bearer token) so the server can list the granted tools for a
        // signed-in user — the same auth the approve/deny call sends; without
        // it the display request is anonymous and shows no tools.
        const infoHeaders = {};
        if (appParams.token) infoHeaders.Authorization = "Bearer " + appParams.token;
        const res = await fetch(
          `/api/apps/${appParams.appId}/mcp/consent-info?handle=${encodeURIComponent(ctx)}`,
          { credentials: "include", headers: infoHeaders },
        );
        if (!res.ok) {
          setError("This authorization link is invalid or has expired.");
          return;
        }
        const data = await res.json();
        // Gate on the server's auth result, NOT base44.auth.isAuthenticated():
        // the SDK check runs the bearer path, so a cookie-only session (platform
        // login/SSO, or a private app with a stale localStorage token) would read
        // as signed-out and redirect — even though /consent-info just
        // authenticated this same request via its cookie fallback. data.authenticated
        // keeps the redirect decision in agreement with what the server returned.
        if (!data.authenticated) {
          // The short handle rides back in returnTo; login_path is
          // owner-configured and validated server-side as a same-origin path.
          // Send from_url too: a custom-auth app coerced to platform auth (e.g.
          // public_without_login under workspace SSO) serves the platform login,
          // which honors from_url rather than returnTo. Rebuild the query from
          // `ctx` alone — never forward window.location.search raw: the platform
          // resume returns from_url verbatim, so crafted extras on the consent
          // link (app_base_url, access_token, …) would ride through the login
          // round-trip and app-params.js would persist them into the freshly
          // authenticated session.
          const returnTo =
            window.location.pathname + "?ctx=" + encodeURIComponent(ctx);
          const encoded = encodeURIComponent(returnTo);
          redirecting = true; // keep the spinner while the browser navigates
          window.location.href =
            (data.login_path || "/login") + "?returnTo=" + encoded + "&from_url=" + encoded;
          return;
        }
        setInfo(data);
      } catch (e) {
```

```
      setError("Could not load this authorization request. Please try again.");
    } finally {
      if (!redirecting) setChecking(false);
    }
  })();
}, [ctx]);

const respond = async (action) => {
  setSubmitting(true);
  setError("");
  try {
    const headers = { "Content-Type": "application/json" };
    // Cookie-backed sessions carry no token; sending "Bearer null" would
    // shadow the valid cookie, so add the header only when a token exists.
    if (appParams.token) headers.Authorization = "Bearer " + appParams.token;
    const res = await fetch(`/api/apps/${appParams.appId}/mcp/authorize-grant`, {
      method: "POST",
      credentials: "include",
      headers,
      body: JSON.stringify({ ctx, action }),
    });
    if (!res.ok) {
      // 401 = the session expired before the (single-use, still-unconsumed)
      // handle was spent; retrying the same controls re-sends the dead session
      // forever. Send the user back through login preserving `ctx` — the same
      // redirect the initial signed-out path uses — so they can return and
      // approve the still-valid handle.
      if (res.status === 401) {
        const returnTo = window.location.pathname + "?ctx=" + encodeURIComponent(ctx);
        const encoded = encodeURIComponent(returnTo);
        window.location.href =
          ((info && info.login_path) || "/login") + "?returnTo=" + encoded + "&from_url=" + encoded;
        return;
      }
      // These all come AFTER the single-use handle is atomically consumed
      // (409 tool set changed; 403 host/resource/app mismatch; 404 access
      // gone; 400 malformed/handle already used), so retrying can only 404.
      // Show a terminal reconnect state, not an impossible "try again".
      if ([400, 403, 404, 409].includes(res.status)) {
        let detail = "";
        try { detail = (await res.json()).detail; } catch (_) { /* keep default */ }
        setReconnect(detail || "This authorization can no longer be completed. Reconnect from your AI client to try again.");
        setSubmitting(false);
        return;
      }
      throw new Error("Could not complete authorization. Please try again.");
    }
    const data = await res.json();
    window.location.href = data.redirect_url;
    if (!/^https?:/i.test(data.redirect_url)) {
      // Custom-scheme redirect (native AI clients, e.g. cursor://): browsers
      // may block or not visibly navigate, so show a terminal state instead
      // of an eternal spinner.
      setDecided(action);
      setSubmitting(false);
    }
  } catch (e) {
    setError(e.message);
    setSubmitting(false);
  }
};

if (checking) {
  return (
    <AuthLayout icon={ShieldCheck} title="Authorize access">
      <div className="flex items-center justify-center py-6 text-muted-foreground">
        <Loader2 className="w-5 h-5 mr-2 animate-spin" aria-hidden="true" />
        Loading…
      </div>
    </AuthLayout>
  );
}

const client = (info && info.client_name) || "An AI client";
const appName = (info && info.app_name) || "this app";

if (decided) {
  return (
    <AuthLayout
      icon={ShieldCheck}
      title={decided === "approve" ? "Access granted" : "Access denied"}
      subtitle={`You can return to ${client} and close this window.`}
    />
  );
}

// Terminal: the authorization request is no longer valid (tool set changed +
// handle consumed). Retrying can't succeed, so show reconnect guidance with
// no approve/deny controls.
if (reconnect) {
  return (
    <AuthLayout icon={ShieldCheck} title="Reconnect required">
```

```
        <div className="p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
          {reconnect}
        </div>
      </AuthLayout>
    );
  }

  // No consent details means nothing trustworthy to approve: a failed
  // consent-info load (expired handle, rate limit, transient error) renders
  // the error alone, never the approve/deny controls.
  if (error && !info) {
    return (
      <AuthLayout icon={ShieldCheck} title="Authorize access">
        <div className="p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
          {error}
        </div>
      </AuthLayout>
    );
  }

  const tools = Array.isArray(info.tools) ? info.tools : [];

  return (
    <AuthLayout
      icon={ShieldCheck}
      title="Authorize access"
      subtitle={`${client} wants to access ${appName} on your behalf`}
    >
      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
          {error}
        </div>
      )}

      <p className="text-sm font-medium text-foreground mb-2">
        {tools.length ? `It will be able to use these tools in ${appName}:` : "No tools requested"}
      </p>
      {tools.length > 0 && (
        <ul className="space-y-2 text-sm mb-6">
          {tools.map((tool) => (
            <li key={tool.name} className="flex flex-col">
              <span className="text-foreground font-medium">
                {tool.title || tool.name}
              </span>
              {tool.description && (
                <span className="text-muted-foreground">{tool.description}</span>
              )}
            </li>
          ))}
        </ul>
      )}

      <div className="flex gap-3">
        <Button
          variant="outline"
          className="flex-1 h-12 font-medium"
          disabled={submitting}
          onClick={() => respond("deny")}
        >
          Deny
        </Button>
        <Button
          className="flex-1 h-12 font-medium"
          disabled={submitting}
          onClick={() => respond("approve")}
        >
          {submitting ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null}
          Approve
        </Button>
      </div>
    </AuthLayout>
  );
}


------------------------------------------------------------
FICHIER : ./src/pages/Onboarding.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { User, Building2, Check, ShieldCheck, Gift } from "lucide-react";
import { Link } from "react-router-dom";
import SubscribeDialog from "@/components/onboarding/SubscribeDialog";

const PLANS = [
  {
    type: "particulier",
    amount: 5000,
    label: "Compte Particulier",
    icon: User,
    tagline: "JH Personal Manager — Gestion personnelle",
    features: ["Finances & budget personnel", "Tâches et planning", "Assistant IA", "Import de fichiers pour l'analyse"],
    styles: {
```

```
      card: "border-primary/40 bg-primary/5",
      badge: "bg-primary text-primary-foreground",
      check: "text-primary",
      btn: "bg-primary text-primary-foreground hover:bg-primary/90",
    },
  },
  {
    type: "entreprise",
    amount: 20000,
    label: "Compte Entreprise",
    icon: Building2,
    tagline: "JH Business Manager — Gestion d'entreprise",
    features: ["Stock & inventaire", "Ventes, clients & devis", "Taxes & prestataires", "Partenariats", "Assistant IA"],
    styles: {
      card: "border-accent/40 bg-accent/5",
      badge: "bg-accent text-accent-foreground",
      check: "text-accent",
      btn: "bg-accent text-accent-foreground hover:bg-accent/90",
    },
  },
];

export default function Onboarding() {
  const [plan, setPlan] = useState(null);

  return (
    <div className="min-h-screen bg-background flex flex-col">
      <header className="px-6 pt-10 pb-6 text-center">
        <p className="font-display text-3xl">JH Manager</p>
        <h1 className="font-display text-2xl mt-2">Choisissez votre type de compte</h1>
        <p className="text-muted-foreground mt-1 text-sm max-w-md mx-auto">
          Un compte est soit Particulier (JH Personal Manager) ou Entreprise (JH Business Manager). L'abonnement est unique et
          débité directement sur votre Mobile Money.
        </p>
      </header>

      <div className="flex-1 px-4 sm:px-6 pb-10 max-w-3xl mx-auto w-full space-y-4">
        {PLANS.map((p) => (
          <div key={p.type} className={`rounded-3xl border p-6 ${p.styles.card}`}>
            <div className="flex items-center gap-3">
              <div className={`w-12 h-12 rounded-2xl flex items-center justify-center shrink-0 ${p.styles.badge}`}>
                <p.icon className="w-6 h-6" />
              </div>
              <div className="min-w-0">
                <h2 className="font-display text-xl">{p.label}</h2>
                <p className="text-xs text-muted-foreground">{p.tagline}</p>
              </div>
              <div className="ml-auto text-right shrink-0">
                <p className="font-display text-2xl">
                  {p.type !== "particulier" ? "dès " : ""}{p.amount.toLocaleString("fr-FR")}{" "}
                  <span className="text-sm font-body">XAF</span>
                </p>
                <p className="text-xs text-muted-foreground">
                  {p.type === "entreprise" ? "/ entreprise" : "par abonnement"}
                </p>
              </div>
            </div>

            <ul className="mt-4 space-y-1.5">
              {p.features.map((f) => (
                <li key={f} className="flex items-center gap-2 text-sm">
                  <Check className={`w-4 h-4 shrink-0 ${p.styles.check}`} /> {f}
                </li>
              ))}
            </ul>

            <button onClick={() => setPlan(p)} className={`no-select-tap mt-5 w-full min-touch rounded-xl font-medium
transition-colors ${p.styles.btn}`}>
              S'abonner — {p.label}
            </button>
          </div>
        ))}

        <Link
          to="/affiliation"
          className="no-select-tap w-full max-w-md mx-auto flex items-center justify-center gap-2 rounded-xl border
border-accent/40 bg-accent/5 text-accent-foreground px-4 py-3 font-medium hover:bg-accent/10 transition-colors"
        >
          <Gift className="w-4 h-4" /> Devenir affilié — parrainez et gagnez 20 %
        </Link>

        <p className="text-xs text-muted-foreground text-center flex items-center justify-center gap-1">
          <ShieldCheck className="w-3.5 h-3.5" /> Paiement sécurisé via Airtel Money / Moov Money (Gabon).
        </p>
      </div>

      <SubscribeDialog
        plan={plan}
        open={!!plan}
        onOpenChange={(o) => !o && setPlan(null)}
        onSuccess={() => { window.location.href = "/"; }}
      />
```

```
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Partnerships.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Link, Pencil, Trash2, Sparkles, Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
AlertDialogHeader, AlertDialogTitle } from "@/components/ui/alert-dialog";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useSpace } from "@/lib/SpaceContext";
import { formatDateFr } from "@/lib/format";

const PT = { individu: "Individu", personne_morale: "Personne morale" };
const ST = { prospect: "Prospect", actif: "Actif", inactif: "Inactif" };
const empty = { partner_name: "", partner_type: "individu", contact_name: "", email: "", phone: "", modalities: "", status:
"prospect", start_date: "" };

export default function Partnerships() {
  const { activeCompanyId } = useSpace();
  const scope = "entreprise";
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);
  const [del, setDel] = useState(null);
  const [processingId, setProcessingId] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => { const list = await base44.entities.Partnership.list("-created_date", 500); setItems(list || []);
setLoading(false); };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const mine = items.filter((p) => (activeCompanyId ? p.company_id === activeCompanyId : true) && p.scope === scope);

  const openNew = () => { setEditing(null); setForm(empty); setOpen(true); };
  const openEdit = (p) => { setEditing(p); setForm({ partner_name: p.partner_name || "", partner_type: p.partner_type ||
"individu", contact_name: p.contact_name || "", email: p.email || "", phone: p.phone || "", modalities: p.modalities || "",
status: p.status || "prospect", start_date: p.start_date || "" }); setOpen(true); };

  const save = async (e) => {
    e.preventDefault();
    if (!form.partner_name.trim()) return;
    const payload = { ...form, scope, company_id: activeCompanyId };
    try {
      if (editing) { await base44.entities.Partnership.update(editing.id, payload); toast({ title: "Partenariat mis à jour" });
}
      else { await base44.entities.Partnership.create(payload); toast({ title: "Partenariat créé" }); }
      setOpen(false); load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (p) => { try { await base44.entities.Partnership.delete(p.id); toast({ title: "Partenariat supprimé" });
load(); } catch { toast({ title: "Erreur", variant: "destructive" }); } setDel(null); };

  const processAI = async (p) => {
    const text = [p.modalities, p.contact_name && `Contact: ${p.contact_name}`].filter(Boolean).join("\n");
    if (!text.trim()) { toast({ title: "Aucune note à traiter", variant: "destructive" }); return; }
    setProcessingId(p.id);
    try {
      const res = await base44.functions.invoke('processNotes', { note: text, scope: "jbm", company_id: activeCompanyId });
      const c = res?.created || {};
      toast({ title: "Note traitée par l'IA", description: `Ajouté : ${c.expenses || 0} dépense(s), ${c.tasks || 0} tâche(s),
${c.events || 0} événement(s).` });
    } catch (e) { toast({ title: "Échec du traitement IA", description: e?.message || "Erreur", variant: "destructive" }); }
    setProcessingId(null);
  };

  if (loading) return <div className="flex items-center justify-center h-64"><div className="w-6 h-6 border-2 border-border
border-t-primary rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div><h1 className="font-display text-2xl sm:text-3xl">Partenariats</h1><p className="text-muted-foreground mt-1">Gérez
vos partenariats (individus ou personnes morales) et leurs modalités.</p></div>
        <Button onClick={openNew}><Plus className="w-4 h-4 mr-2" /> Ajouter</Button>
      </div>
```

```
      {mine.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground"><Link className="w-8 h-8 mx-auto mb-3 opacity-50" /><p
className="text-sm">Aucun partenariat enregistré.</p></div>
      ) : (
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
          {mine.map((p) => (
            <div key={p.id} className="bg-card border border-border rounded-xl p-4">
              <div className="flex items-start justify-between gap-2">
                <div className="min-w-0">
                  <p className="text-sm font-medium truncate">{p.partner_name}</p>
                  <div className="flex items-center gap-2 flex-wrap mt-0.5">
                    <span className="text-xs px-2 py-0.5 rounded-full bg-secondary">{PT[p.partner_type] ||
p.partner_type}</span>
                    <span className={`text-xs px-2 py-0.5 rounded-full ${p.status === "actif" ? "bg-primary/10 text-primary" :
"bg-muted text-muted-foreground"}`}>{ST[p.status] || p.status}</span>
                  </div>
                </div>
                <div className="flex items-center gap-1 shrink-0">
                  <button onClick={() => processAI(p)} disabled={processingId === p.id} title="Traiter la note par l'IA"
className="no-select-tap min-touch p-2 rounded-lg hover:bg-accent/10 text-accent-foreground disabled:opacity-50">
                    {processingId === p.id ? <Loader2 className="w-4 h-4 animate-spin" /> : <Sparkles className="w-4 h-4" />}
                  </button>
                  <button onClick={() => openEdit(p)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                  <button onClick={() => setDel(p)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive"><Trash2 className="w-4 h-4" /></button>
                </div>
              </div>
              <div className="mt-2 text-xs text-muted-foreground space-y-0.5">
                {p.contact_name && <p>{p.contact_name}</p>}
                {(p.phone || p.email) && <p>{[p.phone, p.email].filter(Boolean).join(" · ")}</p>}
                {p.start_date && <p>Début : {formatDateFr(p.start_date)}</p>}
                {p.modalities && <p className="mt-1 text-foreground/80 line-clamp-3">{p.modalities}</p>}
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader><DialogTitle className="font-display">{editing ? "Modifier le partenariat" : "Nouveau
partenariat"}</DialogTitle></DialogHeader>
          <form onSubmit={save} className="space-y-3">
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Nom du partenaire</Label><Input value={form.partner_name} onChange={(e) =>
setForm({ ...form, partner_name: e.target.value })} /></div>
              <div className="space-y-2"><Label>Type</Label>
                <Select value={form.partner_type} onValueChange={(v) => setForm({ ...form, partner_type: v })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent><SelectItem value="individu">Individu</SelectItem><SelectItem value="personne_morale">Personne
morale</SelectItem></SelectContent>
                </Select>
              </div>
            </div>
            <div className="space-y-2"><Label>Personne de contact</Label><Input value={form.contact_name} onChange={(e) =>
setForm({ ...form, contact_name: e.target.value })} /></div>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Téléphone</Label><Input value={form.phone} onChange={(e) => setForm({ ...form,
phone: e.target.value })} /></div>
              <div className="space-y-2"><Label>Email</Label><Input type="email" value={form.email} onChange={(e) => setForm({
...form, email: e.target.value })} /></div>
            </div>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Statut</Label>
                <Select value={form.status} onValueChange={(v) => setForm({ ...form, status: v })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent>{Object.entries(ST).map(([v, l]) => <SelectItem key={v}
value={v}>{l}</SelectItem>)}</SelectContent>
                </Select>
              </div>
              <div className="space-y-2"><Label>Date de début</Label><Input type="date" value={form.start_date} onChange={(e) =>
setForm({ ...form, start_date: e.target.value })} /></div>
            </div>
            <div className="space-y-2"><Label>Modalités du partenariat</Label><Textarea value={form.modalities} onChange={(e) =>
setForm({ ...form, modalities: e.target.value })} rows={4} placeholder="Décrivez les engagements, durée, contreparties,
conditions…" /></div>
            <DialogFooter><Button type="submit" className="w-full">Enregistrer</Button></DialogFooter>
          </form>
        </DialogContent>
      </Dialog>

      <AlertDialog open={!!del} onOpenChange={(o) => !o && setDel(null)}>
        <AlertDialogContent>
          <AlertDialogHeader><AlertDialogTitle>Supprimer ce partenariat ?</AlertDialogTitle><AlertDialogDescription>«
{del?.partner_name} » sera supprimé définitivement.</AlertDialogDescription></AlertDialogHeader>
          <AlertDialogFooter><AlertDialogCancel>Annuler</AlertDialogCancel><AlertDialogAction className="bg-destructive
text-destructive-foreground hover:bg-destructive/90" onClick={(e) => { e.preventDefault(); remove(del);
}}>Supprimer</AlertDialogAction></AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
```

```
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Personnel.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Users } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import { formatCurrency } from "@/lib/format";
import ExportMenu from "@/components/ExportMenu";
import EmployeeForm from "@/components/business/EmployeeForm";
import EmployeeItem from "@/components/business/EmployeeItem";
import AttendancePanel from "@/components/business/AttendancePanel";
import { Tabs, TabsList, TabsTrigger, TabsContent } from "@/components/ui/tabs";

// Conversion d'un montant périodique en équivalent mensuel.
const MONTH_FACTOR = { jour: 30, semaine: 4.345, mois: 1, trimestre: 1 / 3, semestre: 1 / 6, annee: 1 / 12 };
const toMonthly = (amount, periodicity) => (Number(amount) || 0) * (MONTH_FACTOR[periodicity] || 1);

export default function Personnel() {
  const { t } = useI18n();
  const { activeCompanyId } = useSpace();
  const [employees, setEmployees] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);
  const [formOpen, setFormOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [tab, setTab] = useState("employes");
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const [e, prefs] = await Promise.all([
      base44.entities.Employee.list("-created_date", 500),
      base44.entities.Preferences.list(),
    ]);
    setEmployees(e);
    setPreferences(prefs?.[0] || null);
    setLoading(false);
  };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const currency = preferences?.currency || "XAF";
  const myEmployees = activeCompanyId ? employees.filter((e) => e.company_id === activeCompanyId) : employees;
  const payroll = myEmployees
    .filter((e) => e.status === "actif")
    .reduce((sum, e) => sum + toMonthly(e.salary, e.salary_periodicity) * (Number(e.headcount) || 1), 0);
  const monthlyBonus = myEmployees
    .filter((e) => e.status === "actif" && e.bonus_type === "periodique")
    .reduce((sum, e) => sum + toMonthly(e.bonus, e.bonus_periodicity), 0);

  const save = async (data) => {
    const payload = { ...data, company_id: activeCompanyId };
    if (editing) {
      const prev = employees;
      setEmployees((cur) => cur.map((e) => (e.id === editing.id ? { ...e, ...payload } : e)));
      try {
        await base44.entities.Employee.update(editing.id, payload);
        toast({ title: t("toast.taskUpdated") });
        load();
      } catch (e) {
        setEmployees(prev);
        toast({ title: t("toast.error"), description: t("err.modify"), variant: "destructive" });
      }
    } else {
      const tmpId = `tmp_${Date.now()}`;
      setEmployees((cur) => [{ ...payload, id: tmpId }, ...cur]);
      try {
        await base44.entities.Employee.create(payload);
        toast({ title: t("toast.expenseAdded") });
        load();
      } catch (e) {
        setEmployees((cur) => cur.filter((x) => x.id !== tmpId));
        toast({ title: t("toast.error"), description: t("err.create"), variant: "destructive" });
      }
    }
    setFormOpen(false);
    setEditing(null);
  };

  const remove = async (emp) => {
    const prev = employees;
    setEmployees((cur) => cur.filter((e) => e.id !== emp.id));
    try {
      await base44.entities.Employee.delete(emp.id);
```

```
      toast({ title: t("toast.taskDeleted") });
      load();
    } catch (e) {
      setEmployees(prev);
      toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
    }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{t("jbm.personnel.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("jbm.personnel.subtitle")}</p>
        </div>
        {tab === "employes" && (
          <div className="flex items-center gap-2">
            <ExportMenu
              title={t("report.personnel")}
              subtitle={t("report.personnelSub")}
              currency={currency}
              rows={myEmployees}
              columns={[
                { key: "full_name", label: t("jbm.employee.name") },
                { key: "role", label: t("jbm.employee.role") },
                { key: "headcount", label: "Effectif" },
                { key: "salary", label: t("jbm.employee.salary"), type: "currency" },
                { key: "salary_periodicity", label: "Périodicité" },
                { key: "bonus", label: "Prime", type: "currency" },
                { key: "hire_date", label: t("jbm.employee.hireDate"), type: "date" },
                { key: "status", label: t("jbm.employee.status") },
              ]}
              summary={[
                { label: "Masse salariale (mois)", value: formatCurrency(payroll, currency) },
                { label: "Primes périodiques (mois)", value: formatCurrency(monthlyBonus, currency) },
              ]}
            />
            <Button onClick={() => { setEditing(null); setFormOpen(true); }}>
              <Plus className="w-4 h-4 mr-2" /> {t("jbm.add")}
            </Button>
          </div>
        )}
      </div>

      {tab === "employes" && (
        <div className="bg-card border border-border rounded-2xl p-5">
          <p className="text-sm text-muted-foreground">Masse salariale (équivalent mensuel)</p>
          <p className="font-display text-xl mt-1">{formatCurrency(payroll + monthlyBonus, currency)}</p>
          {monthlyBonus > 0 && (
            <p className="text-xs text-muted-foreground mt-1">Dont primes périodiques : {formatCurrency(monthlyBonus,
currency)}</p>
          )}
        </div>
      )}

      <Tabs value={tab} onValueChange={setTab}>
        <TabsList>
          <TabsTrigger value="employes">Employés & paie</TabsTrigger>
          <TabsTrigger value="presence">Absence & discipline</TabsTrigger>
        </TabsList>
        <TabsContent value="employes" className="space-y-2 mt-4">
          {myEmployees.length === 0 ? (
            <div className="text-center py-12 text-muted-foreground">
              <Users className="w-8 h-8 mx-auto mb-3 opacity-50" />
              <p className="text-sm">{t("jbm.personnel.empty")}</p>
            </div>
          ) : (
            myEmployees.map((e) => (
              <EmployeeItem key={e.id} employee={e} currency={currency} onEdit={(x) => { setEditing(x); setFormOpen(true); }}
onDelete={remove} />
            ))
          )}
        </TabsContent>
        <TabsContent value="presence" className="mt-4">
          <AttendancePanel companyId={activeCompanyId} employees={myEmployees} />
        </TabsContent>
      </Tabs>

      <EmployeeForm open={formOpen} onOpenChange={setFormOpen} employee={editing} onSubmit={save} />
    </div>
  );
}

------------------------------------------------------------
```

FICHIER : ./src/pages/Planning.jsx

```
------------------------------------------------------------
import React, { useEffect, useMemo, useState } from "react";
import { base44 } from "@/api/base44Client";
import { CalendarRange, CalendarClock, Plus } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import { formatDateFr } from "@/lib/format";
import TaskItem from "@/components/tasks/TaskItem";
import TaskForm from "@/components/tasks/TaskForm";
import EventItem from "@/components/events/EventItem";
import EventForm from "@/components/events/EventForm";

function overlaps(a, b) {
  return new Date(a.start) < new Date(b.end) && new Date(b.start) < new Date(a.end);
}

const PAGE_SIZE = 100;

export default function Planning() {
  const { t } = useI18n();
  const { space, activeCompanyId } = useSpace();
  const [tasks, setTasks] = useState([]);
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [taskFormOpen, setTaskFormOpen] = useState(false);
  const [eventFormOpen, setEventFormOpen] = useState(false);
  const [editingTask, setEditingTask] = useState(null);
  const [editingEvent, setEditingEvent] = useState(null);
  const [visibleCount, setVisibleCount] = useState(PAGE_SIZE);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    try {
      const [taskList, eventList] = await Promise.all([
        base44.entities.Task.list("start", 300),
        base44.entities.Event.list("start", 300),
      ]);
      setTasks((taskList || []).filter((task) => task.status !== "terminee"));
      const scopedEvents = space === "jpm"
        ? (eventList || [])
        : (eventList || []).filter((e) => !activeCompanyId || e.company_id === activeCompanyId);
      setEvents(scopedEvents);
    } catch { /* ignore */ }
    setLoading(false);
  };

  useEffect(() => { register(load); });
  useEffect(() => { setLoading(true); setVisibleCount(PAGE_SIZE); load(); }, [space, activeCompanyId]);

  const handleToggleDone = async (task) => {
    const prev = tasks;
    setTasks((cur) => cur.filter((item) => item.id !== task.id));
    try {
      await base44.entities.Task.update(task.id, { status: "terminee" });
      toast({ title: t("toast.taskFinished") });
      load();
    } catch (err) {
      setTasks(prev);
      toast({ title: t("toast.error"), description: t("err.update"), variant: "destructive" });
    }
  };

  const handleTaskSave = async (data) => {
    if (editingTask) {
      const prev = tasks;
      setTasks((cur) => cur.map((item) => (item.id === editingTask.id ? { ...item, ...data } : item)));
      try {
        await base44.entities.Task.update(editingTask.id, data);
        toast({ title: t("toast.taskUpdated") });
        load();
      } catch (err) {
        setTasks(prev);
        toast({ title: t("toast.error"), description: t("err.modify"), variant: "destructive" });
      }
    }
    setTaskFormOpen(false);
    setEditingTask(null);
  };

  const handleTaskDelete = async (task) => {
    const prev = tasks;
    setTasks((cur) => cur.filter((item) => item.id !== task.id));
    try {
      await base44.entities.Task.delete(task.id);
      toast({ title: t("toast.taskDeleted") });
      load();
    } catch (err) {
```

```
    setTasks(prev);
    toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
  }
};

const handleEventSave = async (data) => {
  if (editingEvent) {
    const prev = events;
    setEvents((cur) => cur.map((i) => (i.id === editingEvent.id ? { ...i, ...data } : i)));
    try {
      await base44.entities.Event.update(editingEvent.id, data);
      toast({ title: "Événement mis à jour" });
      load();
    } catch {
      setEvents(prev);
      toast({ title: "Erreur", description: "Modification impossible.", variant: "destructive" });
    }
  } else {
    const payload = space === "jpm" ? data : { ...data, company_id: activeCompanyId };
    const tempId = `tmp_${Date.now()}`;
    setEvents((cur) => [{ ...payload, id: tempId }, ...cur]);
    try {
      await base44.entities.Event.create(payload);
      toast({ title: "Événement créé" });
      load();
    } catch {
      setEvents((cur) => cur.filter((i) => i.id !== tempId));
      toast({ title: "Erreur", description: "Création impossible.", variant: "destructive" });
    }
  }
  setEventFormOpen(false);
  setEditingEvent(null);
};

const handleEventDelete = async (ev) => {
  const prev = events;
  setEvents((cur) => cur.filter((i) => i.id !== ev.id));
  try {
    await base44.entities.Event.delete(ev.id);
    toast({ title: "Événement supprimé" });
    load();
  } catch {
    setEvents(prev);
    toast({ title: "Erreur", description: "Suppression impossible.", variant: "destructive" });
  }
};

const combined = useMemo(() => {
  const items = [
    ...tasks.map((tk) => ({ kind: "task", item: tk })),
    ...events.map((ev) => ({ kind: "event", item: ev })),
  ];
  return items.sort((a, b) => new Date(a.item.start) - new Date(b.item.start));
}, [tasks, events]);

const conflictIds = useMemo(() => {
  const ids = new Set();
  for (let i = 0; i < combined.length; i++) {
    for (let j = i + 1; j < combined.length; j++) {
      if (overlaps(combined[i].item, combined[j].item)) {
        ids.add(combined[i].item.id);
        ids.add(combined[j].item.id);
      }
    }
  }
  return ids;
}, [combined]);

const groups = useMemo(() => {
  const visible = combined.slice(0, visibleCount);
  return visible.reduce((acc, entry) => {
    const key = new Date(entry.item.start).toDateString();
    acc[key] = acc[key] || [];
    acc[key].push(entry);
    return acc;
  }, {});
}, [combined, visibleCount]);

if (loading) {
  return (
    <div className="flex items-center justify-center h-64">
      <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
    </div>
  );
}

return (
  <div className="space-y-6">
    <div className="flex items-start justify-between gap-3 flex-wrap">
      <div>
        <h1 className="font-display text-3xl">{t("planning.title")}</h1>
        <p className="text-muted-foreground mt-1">{t("planning.subtitle")}</p>
```

```
          {conflictIds.size > 0 && (
            <p className="text-sm text-destructive mt-2">{t("planning.conflictWarn", { n: conflictIds.size })}</p>
          )}
        </div>
        <div className="flex items-center gap-2">
          <Button variant="outline" onClick={() => { setEditingEvent(null); setEventFormOpen(true); }}>
            <CalendarClock className="w-4 h-4 mr-2" /> Événement
          </Button>
          <Button onClick={() => { setEditingTask(null); setTaskFormOpen(true); }}>
            <Plus className="w-4 h-4 mr-2" /> {t("tasks.new")}
          </Button>
        </div>
      </div>

      {combined.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground">
          <CalendarRange className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p>{t("planning.empty")}</p>
        </div>
      ) : (
        <div className="space-y-6">
          {Object.entries(groups).map(([day, dayItems]) => (
            <div key={day}>
              <h2 className="text-sm font-medium text-muted-foreground mb-2">{formatDateFr(dayItems[0].item.start)}</h2>
              <div className="space-y-2">
                {dayItems.map((entry) => entry.kind === "task" ? (
                  <TaskItem
                    key={entry.item.id}
                    task={entry.item}
                    onEdit={(item) => { setEditingTask(item); setTaskFormOpen(true); }}
                    onDelete={handleTaskDelete}
                    onToggleDone={handleToggleDone}
                    conflict={conflictIds.has(entry.item.id)}
                  />
                ) : (
                  <EventItem
                    key={entry.item.id}
                    event={entry.item}
                    onEdit={(item) => { setEditingEvent(item); setEventFormOpen(true); }}
                    onDelete={handleEventDelete}
                    conflict={conflictIds.has(entry.item.id)}
                  />
                ))}
              </div>
            </div>
          ))}
          {visibleCount < combined.length && (
            <Button variant="outline" className="w-full" onClick={() => setVisibleCount((c) => c + PAGE_SIZE)}>
              {t("planning.loadMore", { n: combined.length - visibleCount })}
            </Button>
          )}
        </div>
      )}

      <TaskForm open={taskFormOpen} onOpenChange={setTaskFormOpen} task={editingTask} onSubmit={handleTaskSave} />
      <EventForm open={eventFormOpen} onOpenChange={setEventFormOpen} event={editingEvent} onSubmit={handleEventSave} />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Privacy.jsx
------------------------------------------------------------
import React from "react";
import { Link } from "react-router-dom";
import { useI18n } from "@/lib/i18n";

export default function Privacy() {
  const { t } = useI18n();
  return (
    <div className="min-h-screen bg-background font-body px-5 py-10 max-w-2xl mx-auto">
      <h1 className="font-display text-3xl mb-4">{t("legal.privacyTitle")}</h1>
      <div className="prose prose-sm text-muted-foreground space-y-3">
        <p>{t("legal.privacyIntro")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.privacyCollectTitle")}</h2>
        <p>{t("legal.privacyCollect")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.privacyUseTitle")}</h2>
        <p>{t("legal.privacyUse")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.privacyStorageTitle")}</h2>
        <p>{t("legal.privacyStorage")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.privacyRightsTitle")}</h2>
        <p>{t("legal.privacyRights")}</p>
      </div>
      <Link to="/login" className="text-sm text-primary hover:underline mt-6 inline-block">{t("legal.back")}</Link>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Quotes.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
```

```
import { base44 } from "@/api/base44Client";
import { Plus, ScrollText, Pencil, Trash2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
AlertDialogHeader, AlertDialogTitle } from "@/components/ui/alert-dialog";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useSpace } from "@/lib/SpaceContext";
import { formatCurrency, formatDateFr } from "@/lib/format";

const STATUS = { brouillon: "Brouillon", envoye: "Envoyé", accepte: "Accepté", refuse: "Refusé" };
const empty = { number: "", client_name: "", amount: "0", issue_date: new Date().toISOString().slice(0, 10), validity_date: "",
status: "brouillon", notes: "" };

export default function Quotes() {
  const { activeCompanyId } = useSpace();
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);
  const [del, setDel] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const list = await base44.entities.Quote.list("-issue_date", 500);
    setItems(list || []);
    setLoading(false);
  };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const mine = activeCompanyId ? items.filter((q) => q.company_id === activeCompanyId) : items;
  const nextNumber = `D-${new Date().getFullYear()}-${String(mine.length + 1).padStart(4, "0")}`;

  const openNew = () => { setEditing(null); setForm({ ...empty, number: nextNumber }); setOpen(true); };
  const openEdit = (q) => { setEditing(q); setForm({ number: q.number || "", client_name: q.client_name || "", amount:
String(q.amount ?? 0), issue_date: q.issue_date || "", validity_date: q.validity_date || "", status: q.status || "brouillon",
notes: q.notes || "" }); setOpen(true); };

  const save = async (e) => {
    e.preventDefault();
    if (!form.number.trim() || !form.client_name.trim() || !form.issue_date) return;
    const payload = { ...form, amount: parseFloat(form.amount) || 0, company_id: activeCompanyId };
    try {
      if (editing) { await base44.entities.Quote.update(editing.id, payload); toast({ title: "Devis mis à jour" }); }
      else { await base44.entities.Quote.create(payload); toast({ title: "Devis créé" }); }
      setOpen(false); load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (q) => {
    try { await base44.entities.Quote.delete(q.id); toast({ title: "Devis supprimé" }); load(); }
    catch { toast({ title: "Erreur", variant: "destructive" }); }
    setDel(null);
  };

  if (loading) return <div className="flex items-center justify-center h-64"><div className="w-6 h-6 border-2 border-border
border-t-primary rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div><h1 className="font-display text-2xl sm:text-3xl">Devis</h1><p className="text-muted-foreground mt-1">Créez et
suivez vos devis.</p></div>
        <Button onClick={openNew}><Plus className="w-4 h-4 mr-2" /> Ajouter</Button>
      </div>

      {mine.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground"><ScrollText className="w-8 h-8 mx-auto mb-3 opacity-50" /><p
className="text-sm">Aucun devis créé.</p></div>
      ) : (
        <div className="space-y-2">
          {mine.map((q) => (
            <div key={q.id} className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
              <div className="min-w-0">
                <p className="text-sm font-medium truncate">{q.number} · {q.client_name}</p>
                <p className="text-xs text-muted-foreground">{formatDateFr(q.issue_date)}{q.validity_date ? ` · valable jusqu'au
${formatDateFr(q.validity_date)}` : ""}</p>
              </div>
              <div className="flex items-center gap-2 shrink-0">
                <span className="text-xs px-2 py-1 rounded-full bg-secondary">{STATUS[q.status] || q.status}</span>
                <span className="text-sm font-medium">{formatCurrency(q.amount, "XAF")}</span>
                <button onClick={() => openEdit(q)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                <button onClick={() => setDel(q)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
```

```
text-destructive"><Trash2 className="w-4 h-4" /></button>
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader><DialogTitle className="font-display">{editing ? "Modifier le devis" : "Nouveau
devis"}</DialogTitle></DialogHeader>
          <form onSubmit={save} className="space-y-3">
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Numéro</Label><Input value={form.number} onChange={(e) => setForm({ ...form,
number: e.target.value })} /></div>
              <div className="space-y-2"><Label>Client</Label><Input value={form.client_name} onChange={(e) => setForm({
...form, client_name: e.target.value })} /></div>
            </div>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Montant (XAF)</Label><Input type="number" min="0" value={form.amount}
onChange={(e) => setForm({ ...form, amount: e.target.value })} /></div>
              <div className="space-y-2"><Label>Statut</Label>
                <Select value={form.status} onValueChange={(v) => setForm({ ...form, status: v })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent>{Object.entries(STATUS).map(([v, l]) => <SelectItem key={v}
value={v}>{l}</SelectItem>)}</SelectContent>
                </Select>
              </div>
            </div>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Date d'émission</Label><Input type="date" value={form.issue_date} onChange={(e)
=> setForm({ ...form, issue_date: e.target.value })} /></div>
              <div className="space-y-2"><Label>Valide jusqu'au</Label><Input type="date" value={form.validity_date}
onChange={(e) => setForm({ ...form, validity_date: e.target.value })} /></div>
            </div>
            <div className="space-y-2"><Label>Description / modalités</Label><Textarea value={form.notes} onChange={(e) =>
setForm({ ...form, notes: e.target.value })} rows={3} /></div>
            <DialogFooter><Button type="submit" className="w-full">Enregistrer</Button></DialogFooter>
          </form>
        </DialogContent>
      </Dialog>

      <AlertDialog open={!!del} onOpenChange={(o) => !o && setDel(null)}>
        <AlertDialogContent>
          <AlertDialogHeader><AlertDialogTitle>Supprimer ce devis ?</AlertDialogTitle><AlertDialogDescription>Le devis «
{del?.number} » sera supprimé définitivement.</AlertDialogDescription></AlertDialogHeader>
          <AlertDialogFooter><AlertDialogCancel>Annuler</AlertDialogCancel><AlertDialogAction className="bg-destructive
text-destructive-foreground hover:bg-destructive/90" onClick={(e) => { e.preventDefault(); remove(del);
}}>Supprimer</AlertDialogAction></AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Register.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { UserPlus, Mail, Lock, Loader2 } from "lucide-react";
import { InputOTP, InputOTPGroup, InputOTPSlot } from "@/components/ui/input-otp";
import AuthLayout from "@/components/AuthLayout";
import GoogleIcon from "@/components/GoogleIcon";
import { toast } from "@/components/ui/use-toast";
import { safeReturnTo } from "@/lib/authReturnTo";

export default function Register() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [showOtp, setShowOtp] = useState(false);
  const [otpCode, setOtpCode] = useState("");

  useEffect(() => {
    const params = new URLSearchParams(window.location.search);
    const ref = params.get("ref");
    if (ref) {
      try { localStorage.setItem("jpm_ref", ref); } catch { /* ignore */ }
    }
  }, []);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    if (password !== confirmPassword) {
```

```
    setError("Les mots de passe ne correspondent pas");
    return;
  }
  setLoading(true);
  try {
    await base44.auth.register({ email, password });
    setShowOtp(true);
  } catch (err) {
    setError(err.message || "L'inscription a échoué");
  } finally {
    setLoading(false);
  }
};

const handleVerify = async () => {
  setError("");
  setLoading(true);
  try {
    const result = await base44.auth.verifyOtp({ email, otpCode });
    if (result?.access_token) {
      base44.auth.setToken(result.access_token);
    }
    window.location.href = safeReturnTo();
  } catch (err) {
    setError(err.message || "Code de vérification invalide");
  } finally {
    setLoading(false);
  }
};

const handleResend = async () => {
  setError("");
  try {
    await base44.auth.resendOtp(email);
    toast({
      title: "Code envoyé",
      description: "Vérifiez votre email pour le nouveau code.",
    });
  } catch (err) {
    setError(err.message || "Impossible de renvoyer le code");
  }
};

const handleGoogle = () => {
  base44.auth.loginWithProvider("google", safeReturnTo());
};

if (showOtp) {
  return (
    <AuthLayout
      icon={Mail}
      title="Vérifiez votre email"
      subtitle={`Nous avons envoyé un code à ${email}`}
    >
      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
          {error}
        </div>
      )}
      <div className="flex justify-center mb-6">
        <InputOTP
          maxLength={6}
          value={otpCode}
          onChange={setOtpCode}
          autoFocus
          autoComplete="one-time-code"
        >
          <InputOTPGroup>
            <InputOTPSlot index={0} />
            <InputOTPSlot index={1} />
            <InputOTPSlot index={2} />
            <InputOTPSlot index={3} />
            <InputOTPSlot index={4} />
            <InputOTPSlot index={5} />
          </InputOTPGroup>
        </InputOTP>
      </div>
      <Button
        className="w-full h-12 font-medium"
        onClick={handleVerify}
        disabled={loading || otpCode.length < 6}
      >
        {loading ? (
          <>
            <Loader2 className="w-4 h-4 mr-2 animate-spin" />
            Vérification...
          </>
        ) : (
          "Vérifier"
        )}
      </Button>
      <p className="text-center text-sm text-muted-foreground mt-4">
```

```
        Vous n'avez pas reçu le code ?{" "}
        <button onClick={handleResend} className="text-primary font-medium hover:underline">
          Renvoyer
        </button>
      </p>
    </AuthLayout>
  );
}

return (
  <AuthLayout
    icon={UserPlus}
    title="Créez votre compte"
    subtitle="Inscrivez-vous pour commencer"
    footer={
      <>
        Déjà un compte ?{" "}
        <Link
          to={"/login" + (safeReturnTo() !== "/" ? "?returnTo=" + encodeURIComponent(safeReturnTo()) : "")}
          className="text-primary font-medium hover:underline"
        >
          Se connecter
        </Link>
      </>
    }
  >
    <Button
      variant="outline"
      className="w-full h-12 text-sm font-medium mb-6"
      onClick={handleGoogle}
    >
      <GoogleIcon className="w-5 h-5 mr-2" />
      Continuer avec Google
    </Button>

    <div className="relative mb-6">
      <div className="absolute inset-0 flex items-center">
        <div className="w-full border-t border-border" />
      </div>
      <div className="relative flex justify-center text-xs uppercase">
        <span className="bg-card px-3 text-muted-foreground">ou</span>
      </div>
    </div>

    {error && (
      <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
        {error}
      </div>
    )}

    <form onSubmit={handleSubmit} className="space-y-4">
      <div className="space-y-2">
        <Label htmlFor="email">Email</Label>
        <div className="relative">
          <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
          <Input
            id="email"
            type="email"
            autoComplete="email"
            autoFocus
            placeholder="vous@exemple.com"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="pl-10 h-12"
            required
          />
        </div>
      </div>
      <div className="space-y-2">
        <Label htmlFor="password">Mot de passe</Label>
        <div className="relative">
          <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
          <Input
            id="password"
            type="password"
            autoComplete="new-password"
            placeholder="••••••••"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            className="pl-10 h-12"
            required
          />
        </div>
      </div>
      <div className="space-y-2">
        <Label htmlFor="confirm">Confirmer le mot de passe</Label>
        <div className="relative">
          <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
          <Input
            id="confirm"
            type="password"
            autoComplete="new-password"
```

```
              placeholder="••••••••"
              value={confirmPassword}
              onChange={(e) => setConfirmPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <Button type="submit" className="w-full h-12 font-medium" disabled={loading}>
          {loading ? (
            <>
              <Loader2 className="w-4 h-4 mr-2 animate-spin" />
              Création du compte...
            </>
          ) : (
            "Créer un compte"
          )}
        </Button>
      </form>
    </AuthLayout>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Reporting.jsx
------------------------------------------------------------
import React, { useEffect, useState, useMemo } from "react";
import { base44 } from "@/api/base44Client";
import { BarChart3, Wallet, TrendingDown, TrendingUp, FileText, Download, Loader2, PieChart as PieIcon, LineChart, CalendarClock
} from "lucide-react";
import { Button } from "@/components/ui/button";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import ExportMenu from "@/components/ExportMenu";
import KpiCard from "@/components/dashboard/KpiCard";
import { useSpace } from "@/lib/SpaceContext";
import { useToast } from "@/components/ui/use-toast";
import { formatCurrency, formatDateFr } from "@/lib/format";
import {
  ResponsiveContainer, ComposedChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
  Legend, PieChart, Pie, Cell,
} from "recharts";

const REPORT_DOMAINS = [
  { value: "finances", label: "Dépenses" },
  { value: "ventes", label: "Ventes" },
  { value: "stock", label: "Stock" },
  { value: "taches", label: "Tâches" },
];

const PALETTE = [
  "hsl(var(--chart-1))", "hsl(var(--chart-2))", "hsl(var(--chart-3))",
  "hsl(var(--chart-4))", "hsl(var(--chart-5))", "hsl(var(--accent))",
];

const MONTH_LABELS = ["janv.", "févr.", "mars", "avr.", "mai", "juin", "juil.", "août", "sept.", "oct.", "nov.", "déc."];

function shortAmount(v) {
  const n = Number(v) || 0;
  const abs = Math.abs(n);
  if (abs >= 1_000_000) return `${(n / 1_000_000).toFixed(1).replace(".0", "")}M`;
  if (abs >= 1_000) return `${(n / 1_000).toFixed(0)}k`;
  return `${Math.round(n)}`;
}

function ChartTooltip({ active, payload, label, currency }) {
  if (!active || !payload?.length) return null;
  return (
    <div className="bg-popover border border-border rounded-lg shadow-md px-3 py-2 text-xs space-y-1">
      {label != null && <p className="font-medium text-foreground">{label}</p>}
      {payload.map((p, i) => (
        <p key={i} className="flex items-center gap-1.5" style={{ color: p.color || p.payload?.fill }}>
          <span className="inline-block w-2 h-2 rounded-full" style={{ background: p.color || p.payload?.fill }} />
          {p.name} : {formatCurrency(Number(p.value) || 0, currency)}
        </p>
      ))}
    </div>
  );
}

export default function Reporting() {
  const { space, activeCompanyId } = useSpace();
  const { toast } = useToast();
  const [expenses, setExpenses] = useState([]);
  const [sales, setSales] = useState([]);
  const [invoices, setInvoices] = useState([]);
  const [tasks, setTasks] = useState([]);
  const [events, setEvents] = useState([]);
  const [prefs, setPrefs] = useState(null);
  const [loading, setLoading] = useState(true);
  const [domain, setDomain] = useState(space === "jpm" ? "finances" : "ventes");
  const [genLoading, setGenLoading] = useState(false);
  const [genLink, setGenLink] = useState(null);
```

```
  useEffect(() => {
    (async () => {
      try {
        const [e, s, inv, pr, tk, ev] = await Promise.all([
          base44.entities.Expense.list("-date", 1000),
          base44.entities.Sale.list("-date", 1000),
          base44.entities.Invoice.list("-issue_date", 1000),
          base44.entities.Preferences.list(),
          base44.entities.Task.list("-start", 500),
          base44.entities.Event.list("start", 500),
        ]);
        setExpenses(e || []); setSales(s || []); setInvoices(inv || []);
        setPrefs(pr?.[0] || null); setTasks(tk || []); setEvents(ev || []);
      } catch { /* ignore */ }
      setLoading(false);
    })();
  }, []);

  const scope = (list) => activeCompanyId ? (list || []).filter((x) => x.company_id === activeCompanyId) : (list || []);
  const cur = prefs?.currency || "XAF";

  const myExp = space === "jpm" ? (expenses || []) : scope(expenses);
  const mySales = space === "jpm" ? [] : scope(sales);
  const myInv = space === "jpm" ? [] : scope(invoices);

  const myEvents = space === "jpm" ? (events || []) : scope(events);
  const now = new Date();
  const upcomingEvents = myEvents
    .filter((e) => new Date(e.start) >= now)
    .sort((a, b) => new Date(a.start) - new Date(b.start))
    .slice(0, 10);
  const upcomingTasks = (tasks || [])
    .filter((t) => t.status !== "terminee" && t.start && new Date(t.start) >= now)
    .sort((a, b) => new Date(a.start) - new Date(b.start))
    .slice(0, 10);

  const activityColumns = [
    { key: "label", label: "Libellé" },
    { key: "kind", label: "Type" },
    { key: "start", label: "Début", type: "date" },
    { key: "end", label: "Fin", type: "date" },
    { key: "meta", label: "Catégorie/Priorité" },
  ];
  const activityRows = [
    ...upcomingEvents.map((e) => ({ label: e.title, kind: "Événement", start: e.start, end: e.end, meta: e.category || "—" })),
    ...upcomingTasks.map((t) => ({ label: t.title, kind: "Tâche", start: t.start, end: t.end, meta: t.priority || "—" })),
  ];

  const totalExp = myExp.reduce((s, e) => s + (Number(e.quantity) || 0) * (Number(e.unit_cost) || 0), 0);
  const totalRev = mySales.reduce((s, x) => s + (Number(x.quantity) || 0) * (Number(x.unit_price) || 0), 0);
  const unpaid = myInv.filter((i) => i.status !== "payee").reduce((s, i) => s + (Number(i.amount) || 0), 0);
  const result = totalRev - totalExp;
  const budget = prefs?.initial_budget || 0;
  const remaining = budget - totalExp;

  const rows = [
    ...mySales.map((x) => ({ label: x.item_name || "Vente", type: "Revenu", tier: x.customer_name || "", date: x.date, amount:
(Number(x.quantity) || 0) * (Number(x.unit_price) || 0) })),
    ...myExp.map((e) => ({ label: e.name || "Dépense", type: "Dépense", tier: e.category || "", date: e.date, amount:
-((Number(e.quantity) || 0) * (Number(e.unit_cost) || 0)) })),
  ].sort((a, b) => (b.date || "").localeCompare(a.date || ""));

  const columns = [
    { key: "label", label: "Libellé" },
    { key: "type", label: "Type" },
    { key: "tier", label: space === "jpm" ? "Catégorie" : "Client / Catégorie" },
    { key: "date", label: "Date", type: "date" },
    { key: "amount", label: "Montant", type: "currency" },
  ];

  const summary = space === "jpm"
    ? [
        { label: "Budget", value: formatCurrency(budget, cur) },
        { label: "Total dépenses", value: formatCurrency(totalExp, cur) },
        { label: "Reste", value: formatCurrency(remaining, cur) },
      ]
    : [
        { label: "Revenus", value: formatCurrency(totalRev, cur) },
        { label: "Dépenses", value: formatCurrency(totalExp, cur) },
        { label: "Résultat", value: formatCurrency(result, cur) },
        { label: "Factures impayées", value: formatCurrency(unpaid, cur) },
      ];

  // Répartition par catégorie de dépenses
  const byCategory = {};
  myExp.forEach((e) => {
    const cat = e.category || "Autre";
    byCategory[cat] = (byCategory[cat] || 0) + (Number(e.quantity) || 0) * (Number(e.unit_cost) || 0);
  });
  const pieData = Object.entries(byCategory)
    .sort((a, b) => b[1] - a[1])
```

```
    .slice(0, 6)
    .map(([name, value]) => ({ name, value }));

  // Évolution mensuelle (12 derniers mois)
  const monthly = useMemo(() => {
    const now = new Date();
    const map = {};
    for (let i = 11; i >= 0; i--) {
      const dt = new Date(now.getFullYear(), now.getMonth() - i, 1);
      const k = `${dt.getFullYear()}-${String(dt.getMonth() + 1).padStart(2, "0")}`;
      map[k] = { key: k, label: MONTH_LABELS[dt.getMonth()], revenus: 0, depenses: 0 };
    }
    mySales.forEach((s) => {
      const k = (s.date || "").slice(0, 7);
      if (map[k]) map[k].revenus += (Number(s.quantity) || 0) * (Number(s.unit_price) || 0);
    });
    myExp.forEach((e) => {
      const k = (e.date || "").slice(0, 7);
      if (map[k]) map[k].depenses += (Number(e.quantity) || 0) * (Number(e.unit_cost) || 0);
    });
    return Object.values(map);
  }, [mySales, myExp]);

  const hasMonthly = monthly.some((m) => m.revenus > 0 || m.depenses > 0);

  const generate = async () => {
    setGenLoading(true); setGenLink(null);
    try {
      const res = await base44.functions.invoke('generateReport', { domain });
      if (res?.file_url) { setGenLink(res); toast({ title: "Rapport généré" }); }
      else toast({ title: "Aucune donnée disponible", variant: "destructive" });
    } catch { toast({ title: "Erreur lors de la génération", variant: "destructive" }); }
    setGenLoading(false);
  };

  if (loading) {
    return (
      <div classNam="flex items-center justify-center h-64">
        <div classNam="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div classNam="space-y-6">
      <div classNam="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 classNam="font-display text-2xl sm:text-3xl flex items-center gap-2"><BarChart3 classNam="w-6 h-6" /> Reporting
financier</h1>
          <p classNam="text-muted-foreground mt-1">Synthèse financière, graphiques, export PDF/Excel et rapport serveur.</p>
        </div>
        <ExportMenu title="Reporting financier" subtitle={space === "jpm" ? "Espace personnel" : `Espace ${space === "jbm" ?
"entreprise" : "école"}`} currency={cur} rows={rows} columns={columns} summary={summary} />
      </div>

      <div classNam="grid grid-cols-2 md:grid-cols-4 gap-4">
        {space === "jpm" ? (
          <>
            <KpiCard icon={Wallet} label="Budget" value={formatCurrency(budget, cur)} tone="default" />
            <KpiCard icon={TrendingDown} label="Dépenses" value={formatCurrency(totalExp, cur)} tone="accent" />
            <KpiCard icon={TrendingUp} label="Reste" value={formatCurrency(remaining, cur)} tone={remaining < 0 ? "warn" :
"default"} />
            <KpiCard icon={FileText} label="Mouvements" value={rows.length} tone="default" />
          </>
        ) : (
          <>
            <KpiCard icon={TrendingUp} label="Revenus" value={formatCurrency(totalRev, cur)} tone="default" />
            <KpiCard icon={TrendingDown} label="Dépenses" value={formatCurrency(totalExp, cur)} tone="accent" />
            <KpiCard icon={Wallet} label="Résultat" value={formatCurrency(result, cur)} tone={result < 0 ? "warn" : "default"}
/>
            <KpiCard icon={FileText} label="Factures impayées" value={formatCurrency(unpaid, cur)} tone={unpaid > 0 ? "warn" :
"default"} />
          </>
        )}
      </div>

      <div classNam="bg-card border border-border rounded-2xl p-5">
        <h2 classNam="font-display text-lg flex items-center gap-2 mb-4"><LineChart classNam="w-4 h-4 text-accent" />
Évolution sur 12 mois</h2>
        {hasMonthly ? (
          <ResponsiveContainer width="100%" height={300}>
            <ComposedChart data={monthly} margin={{ top: 8, right: 8, left: 0, bottom: 0 }}>
              <CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" vertical={false} />
              <XAxis dataKey="label" tick={{ fontSize: 12, fill: "hsl(var(--muted-foreground))" }} tickLine={false} axisLine={{
stroke: "hsl(var(--border))" }} />
              <YAxis tick={{ fontSize: 12, fill: "hsl(var(--muted-foreground))" }} tickFormatter={shortAmount} tickLine={false}
axisLine={false} width={44} />
              <Tooltip content={<ChartTooltip currency={cur} />} />
              <Legend wrapperStyle={{ fontSize: 12 }} />
              {space !== "jpm" && <Bar dataKey="revenus" name="Revenus" fill="hsl(var(--chart-1))" radius={[4, 4, 0, 0]}
maxBarSize={28} />}
              <Bar dataKey="depenses" name="Dépenses" fill="hsl(var(--chart-4))" radius={[4, 4, 0, 0]} maxBarSize={28} />
```

```
            </ComposedChart>
          </ResponsiveContainer>
        ) : (
          <p className="text-sm text-muted-foreground">Aucune donnée à afficher sur la période.</p>
        )}
      </div>

      <div className="bg-card border border-border rounded-2xl p-5">
        <h2 className="font-display text-lg flex items-center gap-2 mb-4"><PieIcon className="w-4 h-4 text-accent" />
Répartition par catégorie</h2>
        {pieData.length === 0 ? (
          <p className="text-sm text-muted-foreground">Aucune dépense à afficher.</p>
        ) : (
          <ResponsiveContainer width="100%" height={300}>
            <PieChart>
              <Pie data={pieData} dataKey="value" nameKey="name" cx="50%" cy="50%" outerRadius={100} innerRadius={45}
paddingAngle={2}>
                {pieData.map((_, i) => (
                  <Cell key={i} fill={PALETTE[i % PALETTE.length]} stroke="hsl(var(--card))" strokeWidth={2} />
                ))}
              </Pie>
              <Tooltip content={<ChartTooltip currency={cur} />} />
              <Legend wrapperStyle={{ fontSize: 12 }} />
            </PieChart>
          </ResponsiveContainer>
        )}
      </div>

      <div className="bg-card border border-border rounded-2xl p-5 space-y-4">
        <div className="flex items-center justify-between gap-3 flex-wrap">
          <h2 className="font-display text-lg flex items-center gap-2"><CalendarClock className="w-4 h-4 text-accent" />
Activités à venir</h2>
          <ExportMenu
            title="Activités à venir"
            subtitle={space === "jpm" ? "Espace personnel" : "Espace entreprise"}
            currency={cur}
            rows={activityRows}
            columns={activityColumns}
            summary={[
              { label: "Événements", value: String(upcomingEvents.length) },
              { label: "Tâches", value: String(upcomingTasks.length) },
            ]}
          />
        </div>
        {activityRows.length === 0 ? (
          <p className="text-sm text-muted-foreground">Aucune activité à venir.</p>
        ) : (
          <div className="space-y-2">
            {activityRows.map((r, i) => (
              <div key={i} className="flex items-center gap-3 text-sm">
                <span className={`inline-flex items-center px-2 py-0.5 rounded-full text-xs ${r.kind === "Événement" ?
"bg-accent/20 text-accent-foreground" : "bg-primary/15 text-primary"}`}>{r.kind}</span>
                <span className="flex-1 truncate">{r.label}</span>
                <span className="text-xs text-muted-foreground whitespace-nowrap">{formatDateFr(r.start)}</span>
              </div>
            ))}
          </div>
        )}
      </div>

      <div className="bg-card border border-border rounded-2xl p-5 space-y-4">
        <h2 className="font-display text-lg flex items-center gap-2"><Download className="w-4 h-4 text-accent" /> Rapport
serveur (CSV)</h2>
        <p className="text-sm text-muted-foreground">Générez un fichier CSV détaillé à partir de vos données (déposé sur un lien
de téléchargement).</p>
        <div className="flex items-center gap-2 flex-wrap">
          <Select value={domain} onValueChange={setDomain}>
            <SelectTrigger className="w-44"><SelectValue /></SelectTrigger>
            <SelectContent>{REPORT_DOMAINS.map((d) => <SelectItem key={d.value}
value={d.value}>{d.label}</SelectItem>)}</SelectContent>
          </Select>
          <Button onClick={generate} disabled={genLoading}>
            {genLoading ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Download className="w-4 h-4 mr-2" />}
            Générer
          </Button>
        </div>
        {genLink && (
          <a href={genLink.file_url} target="_blank" rel="noreferrer" className="inline-flex items-center gap-2 text-sm
text-primary hover:underline break-all">
            <FileText className="w-4 h-4 shrink-0" /> {genLink.filename || "Télécharger le rapport"}
          </a>
        )}
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/ResetPassword.jsx
------------------------------------------------------------
import React, { useState } from "react";
import { Link, useSearchParams } from "react-router-dom";
```

```
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Lock, Loader2, AlertTriangle } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";

export default function ResetPassword() {
  const [searchParams] = useSearchParams();
  const resetToken = searchParams.get("token");

  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    if (newPassword !== confirmPassword) {
      setError("Les mots de passe ne correspondent pas");
      return;
    }
    setLoading(true);
    try {
      await base44.auth.resetPassword({ resetToken, newPassword });
      window.location.href = "/login";
    } catch (err) {
      setError(err.message || "La réinitialisation a échoué");
    } finally {
      setLoading(false);
    }
  };

  if (!resetToken) {
    return (
      <AuthLayout
        icon={AlertTriangle}
        title="Lien invalide"
        subtitle="Ce lien de réinitialisation est manquant ou invalide"
        footer={
          <Link to="/forgot-password" className="text-primary font-medium hover:underline">
            Demander un nouveau lien
          </Link>
        }
      >
        <p className="text-sm text-foreground text-center">
          Le lien utilisé semble incomplet. Merci de demander un nouvel email de réinitialisation.
        </p>
      </AuthLayout>
    );
  }

  return (
    <AuthLayout
      icon={Lock}
      title="Nouveau mot de passe"
      subtitle="Saisissez votre nouveau mot de passe ci-dessous"
    >
      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
          {error}
        </div>
      )}
      <form onSubmit={handleSubmit} className="space-y-4">
        <div className="space-y-2">
          <Label htmlFor="password">Nouveau mot de passe</Label>
          <div className="relative">
            <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
            <Input
              id="password"
              type="password"
              autoComplete="new-password"
              autoFocus
              placeholder="••••••••"
              value={newPassword}
              onChange={(e) => setNewPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <div className="space-y-2">
          <Label htmlFor="confirm">Confirmer le mot de passe</Label>
          <div className="relative">
            <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" aria-hidden="true" />
            <Input
              id="confirm"
              type="password"
              autoComplete="new-password"
              placeholder="••••••••"
              value={confirmPassword}
```

```
              onChange={(e) => setConfirmPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <Button type="submit" className="w-full h-12 font-medium" disabled={loading}>
          {loading ? (
            <>
              <Loader2 className="w-4 h-4 mr-2 animate-spin" />
              Réinitialisation...
            </>
          ) : (
            "Réinitialiser le mot de passe"
          )}
        </Button>
      </form>
    </AuthLayout>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Sales.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, ShoppingCart } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useI18n } from "@/lib/i18n";
import { useSpace } from "@/lib/SpaceContext";
import { formatCurrency } from "@/lib/format";
import ExportMenu from "@/components/ExportMenu";
import SaleForm from "@/components/business/SaleForm";
import SaleItem from "@/components/business/SaleItem";
import { expandRecurrence } from "@/lib/recurrence";

export default function Sales() {
  const { t } = useI18n();
  const { activeCompanyId } = useSpace();
  const [sales, setSales] = useState([]);
  const [preferences, setPreferences] = useState(null);
  const [loading, setLoading] = useState(true);
  const [formOpen, setFormOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const [s, prefs] = await Promise.all([
      base44.entities.Sale.list("-date", 500),
      base44.entities.Preferences.list(),
    ]);
    setSales(s);
    setPreferences(prefs?.[0] || null);
    setLoading(false);
  };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const currency = preferences?.currency || "XAF";
  const mySales = activeCompanyId ? sales.filter((s) => s.company_id === activeCompanyId) : sales;
  const total = mySales.filter((s) => s.status !== "annulee").reduce((sum, s) => sum + (Number(s.quantity) || 0) *
(Number(s.unit_price) || 0), 0);

  const save = async (data, recurrence) => {
    const payload = { ...data, company_id: activeCompanyId };
    if (editing) {
      const prev = sales;
      setSales((cur) => cur.map((s) => (s.id === editing.id ? { ...s, ...payload } : s)));
      try {
        await base44.entities.Sale.update(editing.id, payload);
        toast({ title: t("toast.taskUpdated") });
        load();
      } catch (e) {
        setSales(prev);
        toast({ title: t("toast.error"), description: t("err.modify"), variant: "destructive" });
      }
    } else {
      const items = expandRecurrence(payload, recurrence?.frequency, recurrence?.count, ["date"]);
      const tempIds = items.map(() => `tmp_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`);
      setSales((cur) => [...items.map((it, i) => ({ ...it, id: tempIds[i] })), ...cur]);
      try {
        await base44.entities.Sale.bulkCreate(items);
        toast({ title: recurrence && recurrence.count > 1 ? `${items.length} ventes récurrentes créées` :
t("toast.expenseAdded") });
        load();
      } catch (e) {
        setSales((cur) => cur.filter((s) => !tempIds.includes(s.id)));
        toast({ title: t("toast.error"), description: t("err.create"), variant: "destructive" });
      }
```

```
    }
    setFormOpen(false);
    setEditing(null);
  };

  const remove = async (sale) => {
    const prev = sales;
    setSales((cur) => cur.filter((s) => s.id !== sale.id));
    try {
      await base44.entities.Sale.delete(sale.id);
      toast({ title: t("toast.taskDeleted") });
      load();
    } catch (e) {
      setSales(prev);
      toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
    }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{t("jbm.sales.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("jbm.sales.subtitle")}</p>
        </div>
        <div className="flex items-center gap-2">
          <ExportMenu
            title={t("report.sales")}
            subtitle={t("report.salesSub")}
            currency={currency}
            rows={mySales}
            columns={[
              { key: "item_name", label: t("jbm.sale.item") },
              { key: "customer_name", label: t("jbm.sale.customer") },
              { key: "quantity", label: t("jbm.sale.quantity") },
              { key: "unit_price", label: t("jbm.sale.unitPrice"), type: "currency" },
              { key: "date", label: t("jbm.sale.date"), type: "date" },
              { key: "status", label: t("jbm.sale.status") },
            ]}
            summary={[{ label: t("jbm.kpi.sales"), value: formatCurrency(total, currency) }]}
          />
          <Button onClick={() => { setEditing(null); setFormOpen(true); }}>
            <Plus className="w-4 h-4 mr-2" /> {t("jbm.add")}
          </Button>
        </div>
      </div>
      <div className="bg-card border border-border rounded-2xl p-5">
        <p className="text-sm text-muted-foreground">{t("jbm.kpi.sales")}</p>
        <p className="font-display text-xl mt-1">{formatCurrency(total, currency)}</p>
      </div>
      {mySales.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground">
          <ShoppingCart className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p className="text-sm">{t("jbm.sales.empty")}</p>
        </div>
      ) : (
        <div className="space-y-2">
          {mySales.map((s) => (
            <SaleItem key={s.id} sale={s} currency={currency} onEdit={(x) => { setEditing(x); setFormOpen(true); }}
onDelete={remove} />
          ))}
        </div>
      )}
      <SaleForm open={formOpen} onOpenChange={setFormOpen} sale={editing} onSubmit={save} />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Settings.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useToast } from "@/components/ui/use-toast";
import { useTheme } from "@/lib/ThemeProvider";
import { useI18n } from "@/lib/i18n";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
  AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from "@/components/ui/alert-dialog";
```

```
import LegalLinks from "@/components/settings/LegalLinks";
import PaymentCard from "@/components/settings/PaymentCard";

export default function Settings() {
  const { t, setLang } = useI18n();
  const [preferences, setPreferences] = useState(null);
  const [currency, setCurrency] = useState("XAF");
  const [language, setLanguage] = useState("fr");
  const [initialBudget, setInitialBudget] = useState("0");
  const [reminderDays, setReminderDays] = useState("7");
  const [theme, setThemeState] = useState("system");
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [deleteOpen, setDeleteOpen] = useState(false);
  const [deleting, setDeleting] = useState(false);
  const [cancelOpen, setCancelOpen] = useState(false);
  const [canceling, setCanceling] = useState(false);
  const { toast } = useToast();
  const { setTheme } = useTheme();

  useEffect(() => {
    (async () => {
      const list = await base44.entities.Preferences.list();
      const p = list?.[0] || null;
      setPreferences(p);
      setCurrency(p?.currency || "XAF");
      setLanguage(p?.language || "fr");
      setInitialBudget(String(p?.initial_budget ?? 0));
      setReminderDays(String(p?.reminder_days ?? 7));
      setThemeState(p?.theme || "system");
      if (p?.language) setLang(p.language);
      setLoading(false);
    })();
  }, []);

  const handleSave = async (e) => {
    e.preventDefault();
    setSaving(true);
    const data = {
      currency,
      initial_budget: parseFloat(initialBudget) || 0,
      reminder_days: parseInt(reminderDays, 10) || 7,
      theme,
      language,
    };
    setTheme(theme);
    setLang(language);
    try {
      if (preferences) {
        await base44.entities.Preferences.update(preferences.id, data);
      } else {
        const created = await base44.entities.Preferences.create(data);
        setPreferences(created);
      }
      toast({ title: t("toast.prefsSaved") });
    } catch (err) {
      toast({ title: t("toast.error"), description: t("err.save"), variant: "destructive" });
    }
    setSaving(false);
  };

  const handleDeleteAccount = async () => {
    setDeleting(true);
    try {
      const res = await base44.functions.invoke('deleteAccount', {});
      if (res?.dataDeleted) {
        toast({ title: t("toast.accountDeleted"), description: t("toast.accountDeletedDesc") });
        await base44.auth.logout();
        window.location.href = "/login";
      } else {
        toast({ title: t("toast.partial"), description: res?.warning || t("toast.partialDesc"), variant: "destructive" });
      }
    } catch (err) {
      toast({ title: t("toast.error"), description: err?.message || t("err.deleteAccount"), variant: "destructive" });
    }
    setDeleting(false);
    setDeleteOpen(false);
  };

  const handleCancelSubscription = async () => {
    setCanceling(true);
    try {
      const res = await base44.functions.invoke('cancelSubscription', {});
      if (res?.success) {
        try { await base44.auth.updateMe({ account_type: "" }); } catch { /* non-fatal */ }
        toast({ title: "Abonnement résilié", description: "Votre abonnement a été résilié." });
        window.location.href = "/";
      } else {
        toast({ title: "Erreur", description: res?.error || "Résiliation impossible", variant: "destructive" });
      }
    } catch (err) {
      toast({ title: "Erreur", description: err?.message || "Résiliation impossible", variant: "destructive" });
```

```
    }
    setCanceling(false);
    setCancelOpen(false);
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="max-w-lg space-y-6">
      <div>
        <h1 className="font-display text-3xl">{t("settings.title")}</h1>
        <p className="text-muted-foreground mt-1">{t("settings.subtitle")}</p>
      </div>

      <form onSubmit={handleSave} className="bg-card border border-border rounded-2xl p-6 space-y-5">
        <div className="space-y-2">
          <Label>{t("settings.language")}</Label>
          <Select value={language} onValueChange={setLanguage}>
            <SelectTrigger><SelectValue /></SelectTrigger>
            <SelectContent>
              <SelectItem value="fr">{t("lang.fr")}</SelectItem>
              <SelectItem value="en">{t("lang.en")}</SelectItem>
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-2">
          <Label>{t("settings.currency")}</Label>
          <Select value={currency} onValueChange={setCurrency}>
            <SelectTrigger><SelectValue /></SelectTrigger>
            <SelectContent>
              <SelectItem value="XAF">{t("currency.XAF")}</SelectItem>
              <SelectItem value="EUR">{t("currency.EUR")}</SelectItem>
              <SelectItem value="USD">{t("currency.USD")}</SelectItem>
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-2">
          <Label>{t("settings.theme")}</Label>
          <Select value={theme} onValueChange={setThemeState}>
            <SelectTrigger><SelectValue /></SelectTrigger>
            <SelectContent>
              <SelectItem value="system">{t("theme.system")}</SelectItem>
              <SelectItem value="light">{t("theme.light")}</SelectItem>
              <SelectItem value="dark">{t("theme.dark")}</SelectItem>
            </SelectContent>
          </Select>
        </div>
        <div className="space-y-2">
          <Label htmlFor="budget">{t("settings.budget")}</Label>
          <Input id="budget" type="number" min="0" step="any" value={initialBudget} onChange={(e) =>
setInitialBudget(e.target.value)} />
        </div>
        <div className="space-y-2">
          <Label htmlFor="reminder">{t("settings.reminder")}</Label>
          <Input id="reminder" type="number" min="1" step="1" value={reminderDays} onChange={(e) =>
setReminderDays(e.target.value)} />
        </div>
        <Button type="submit" className="w-full" disabled={saving}>
          {saving ? t("settings.saving") : t("settings.save")}
        </Button>
      </form>

      <PaymentCard />

      <div className="bg-card border border-border rounded-2xl p-6 space-y-3">
        <div>
          <h2 className="font-display text-lg">Abonnement</h2>
          <p className="text-sm text-muted-foreground mt-1">Résiliez votre abonnement à tout moment. Vous pourrez vous réabonner
ultérieurement.</p>
        </div>
        <Button variant="outline" className="text-destructive border-destructive/40 hover:bg-destructive/10" onClick={() =>
setCancelOpen(true)}>Résilier mon abonnement</Button>
      </div>

      <LegalLinks />

      <div className="bg-card border border-destructive/30 rounded-2xl p-6 space-y-3">
        <div>
          <h2 className="font-display text-lg text-destructive">{t("settings.dangerZone")}</h2>
          <p className="text-sm text-muted-foreground mt-1">{t("settings.dangerDesc")}</p>
        </div>
        <Button variant="destructive" onClick={() => setDeleteOpen(true)}>{t("settings.deleteAccount")}</Button>
      </div>

      <AlertDialog open={deleteOpen} onOpenChange={setDeleteOpen}>
        <AlertDialogContent>
```

```
          <AlertDialogHeader>
            <AlertDialogTitle>{t("settings.deleteTitle")}</AlertDialogTitle>
            <AlertDialogDescription>{t("settings.deleteDesc")}</AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel disabled={deleting}>{t("common.cancel")}</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
              onClick={(e) => { e.preventDefault(); handleDeleteAccount(); }}
              disabled={deleting}
            >
              {deleting ? t("settings.deleting") : t("settings.confirmDelete")}
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>

      <AlertDialog open={cancelOpen} onOpenChange={setCancelOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Résilier votre abonnement ?</AlertDialogTitle>
            <AlertDialogDescription>Votre abonnement sera désactivé. Vous reviendrez à l'étape de choix d'abonnement et pourrez
vous réabonner à tout moment.</AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel disabled={canceling}>Annuler</AlertDialogCancel>
            <AlertDialogAction className="bg-destructive text-destructive-foreground hover:bg-destructive/90" onClick={(e) => {
e.preventDefault(); handleCancelSubscription(); }} disabled={canceling}>{canceling ? "Résiliation..." :
"Résilier"}</AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/SuggestionsPage.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Lightbulb, Loader2, Sparkles } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useToast } from "@/components/ui/use-toast";
import { formatDateFr } from "@/lib/format";

const CAT = { amelioration: "Amélioration", correction: "Correction", autre: "Autre" };

export default function SuggestionsPage() {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [content, setContent] = useState("");
  const [category, setCategory] = useState("amelioration");
  const [submitting, setSubmitting] = useState(false);
  const { toast } = useToast();

  const load = async () => {
    const list = await base44.entities.Suggestion.list("-created_date", 200);
    setItems(list || []);
    setLoading(false);
  };
  useEffect(() => { load(); }, []);

  const submit = async (e) => {
    e.preventDefault();
    if (!content.trim()) return;
    setSubmitting(true);
    try {
      await base44.entities.Suggestion.create({ content: content.trim(), category, status: "nouveau" });
      toast({ title: "Merci ! Votre suggestion a été envoyée." });
      setContent("");
      load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
    setSubmitting(false);
  };

  return (
    <div className="space-y-6 max-w-2xl">
      <div>
        <h1 className="font-display text-2xl sm:text-3xl">Suggestions</h1>
        <p className="text-muted-foreground mt-1">Proposez des améliorations ou signalez des corrections. Vos idées nous aident
à améliorer JM.</p>
      </div>

      <form onSubmit={submit} className="bg-card border border-border rounded-2xl p-5 space-y-3">
        <div className="space-y-2"><Label>Catégorie</Label>
          <Select value={category} onValueChange={setCategory}>
            <SelectTrigger><SelectValue /></SelectTrigger>
```

```
            <SelectContent>{Object.entries(CAT).map(([v, l]) => <SelectItem key={v} value={v}>{l}</SelectItem>)}</SelectContent>
          </Select>
        </div>
        <div className="space-y-2"><Label>Votre suggestion</Label><Textarea value={content} onChange={(e) =>
setContent(e.target.value)} rows={4} placeholder="Décrivez votre idée d'amélioration ou le problème rencontré…" /></div>
        <Button type="submit" disabled={submitting} className="w-full">
          {submitting ? <><Loader2 className="w-4 h-4 mr-2 animate-spin" /> Envoi…</> : <><Lightbulb className="w-4 h-4 mr-2" />
Envoyer</>}
        </Button>
      </form>

      <div>
        <h2 className="font-display text-lg mb-3">Vos suggestions</h2>
        {loading ? (
          <div className="flex items-center justify-center h-32"><div className="w-6 h-6 border-2 border-border border-t-primary
rounded-full animate-spin" /></div>
        ) : items.length === 0 ? (
          <p className="text-sm text-muted-foreground text-center py-8">Aucune suggestion pour l'instant.</p>
        ) : (
          <div className="space-y-2">
            {items.map((s) => (
              <div key={s.id} className="bg-card border border-border rounded-xl p-4">
                <div className="flex items-center gap-2 flex-wrap mb-1">
                  <span className="text-xs px-2 py-0.5 rounded-full bg-secondary">{CAT[s.category] || s.category}</span>
                  <span className="text-xs text-muted-foreground">{formatDateFr(s.created_date, true)}</span>
                </div>
                <p className="text-sm">{s.content}</p>
              </div>
            ))}
          </div>
        )}
      </div>

      <div className="bg-accent/5 border border-accent/30 rounded-2xl p-4 flex items-start gap-2">
        <Sparkles className="w-4 h-4 text-accent shrink-0 mt-0.5" />
        <p className="text-xs text-muted-foreground">Vos suggestions sont analysées par notre IA : les idées similaires sont
regroupées pour ne garder que des propositions uniques sur le tableau de bord administrateur.</p>
      </div>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Suppliers.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Truck, Pencil, Trash2, Sparkles, Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
AlertDialogHeader, AlertDialogTitle } from "@/components/ui/alert-dialog";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useSpace } from "@/lib/SpaceContext";

const empty = { name: "", category: "", contact_person: "", phone: "", email: "", address: "", notes: "" };

export default function Suppliers() {
  const { activeCompanyId } = useSpace();
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);
  const [del, setDel] = useState(null);
  const [processingId, setProcessingId] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => { const list = await base44.entities.Supplier.list("-created_date", 500); setItems(list || []);
setLoading(false); };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const mine = activeCompanyId ? items.filter((s) => s.company_id === activeCompanyId) : items;

  const openNew = () => { setEditing(null); setForm(empty); setOpen(true); };
  const openEdit = (s) => { setEditing(s); setForm({ name: s.name || "", category: s.category || "", contact_person:
s.contact_person || "", phone: s.phone || "", email: s.email || "", address: s.address || "", notes: s.notes || "" });
setOpen(true); };

  const save = async (e) => {
    e.preventDefault();
    if (!form.name.trim()) return;
    const payload = { ...form, company_id: activeCompanyId };
    try {
      if (editing) { await base44.entities.Supplier.update(editing.id, payload); toast({ title: "Prestataire mis à jour" }); }
      else { await base44.entities.Supplier.create(payload); toast({ title: "Prestataire ajouté" }); }
```

```
      setOpen(false); load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (s) => { try { await base44.entities.Supplier.delete(s.id); toast({ title: "Prestataire supprimé" });
load(); } catch { toast({ title: "Erreur", variant: "destructive" }); } setDel(null); };

  const processAI = async (s) => {
    const text = [s.notes, s.category && `Catégorie: ${s.category}`, s.contact_person && `Contact:
${s.contact_person}`].filter(Boolean).join("\n");
    if (!text.trim()) { toast({ title: "Aucune note à traiter", variant: "destructive" }); return; }
    setProcessingId(s.id);
    try {
      const res = await base44.functions.invoke('processNotes', { note: text, scope: "jbm", company_id: activeCompanyId });
      const c = res?.created || {};
      toast({ title: "Note traitée par l'IA", description: `Ajouté : ${c.expenses || 0} dépense(s), ${c.tasks || 0} tâche(s),
${c.events || 0} événement(s).` });
    } catch (e) { toast({ title: "Échec du traitement IA", description: e?.message || "Erreur", variant: "destructive" }); }
    setProcessingId(null);
  };

  if (loading) return <div className="flex items-center justify-center h-64"><div className="w-6 h-6 border-2 border-border
border-t-primary rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div><h1 className="font-display text-2xl sm:text-3xl">Prestataires</h1><p className="text-muted-foreground mt-1">Vos
fournisseurs et prestataires de service.</p></div>
        <Button onClick={openNew}><Plus className="w-4 h-4 mr-2" /> Ajouter</Button>
      </div>

      {mine.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground"><Truck className="w-8 h-8 mx-auto mb-3 opacity-50" /><p
className="text-sm">Aucun prestataire enregistré.</p></div>
      ) : (
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
          {mine.map((s) => (
            <div key={s.id} className="bg-card border border-border rounded-xl p-4">
              <div className="flex items-start justify-between gap-2">
                <div className="min-w-0">
                  <p className="text-sm font-medium truncate">{s.name}</p>
                  {s.category && <p className="text-xs text-muted-foreground truncate">{s.category}</p>}
                </div>
                <div className="flex items-center gap-1 shrink-0">
                  <button onClick={() => processAI(s)} disabled={processingId === s.id} title="Traiter la note par l'IA"
className="no-select-tap min-touch p-2 rounded-lg hover:bg-accent/10 text-accent-foreground disabled:opacity-50">
                    {processingId === s.id ? <Loader2 className="w-4 h-4 animate-spin" /> : <Sparkles className="w-4 h-4" />}
                  </button>
                  <button onClick={() => openEdit(s)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                  <button onClick={() => setDel(s)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive"><Trash2 className="w-4 h-4" /></button>
                </div>
              </div>
              <div className="mt-2 text-xs text-muted-foreground space-y-0.5">
                {s.contact_person && <p>{s.contact_person}</p>}
                {s.phone && <p>{s.phone}</p>}
                {s.email && <p className="truncate">{s.email}</p>}
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader><DialogTitle className="font-display">{editing ? "Modifier le prestataire" : "Nouveau
prestataire"}</DialogTitle></DialogHeader>
          <form onSubmit={save} className="space-y-3">
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Nom</Label><Input value={form.name} onChange={(e) => setForm({ ...form, name:
e.target.value })} /></div>
              <div className="space-y-2"><Label>Catégorie</Label><Input value={form.category} onChange={(e) => setForm({
...form, category: e.target.value })} /></div>
            </div>
            <div className="space-y-2"><Label>Personne de contact</Label><Input value={form.contact_person} onChange={(e) =>
setForm({ ...form, contact_person: e.target.value })} /></div>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Téléphone</Label><Input value={form.phone} onChange={(e) => setForm({ ...form,
phone: e.target.value })} /></div>
              <div className="space-y-2"><Label>Email</Label><Input type="email" value={form.email} onChange={(e) => setForm({
...form, email: e.target.value })} /></div>
            </div>
            <div className="space-y-2"><Label>Adresse</Label><Input value={form.address} onChange={(e) => setForm({ ...form,
address: e.target.value })} /></div>
            <div className="space-y-2"><Label>Notes</Label><Textarea value={form.notes} onChange={(e) => setForm({ ...form,
notes: e.target.value })} rows={2} /></div>
            <DialogFooter><Button type="submit" className="w-full">Enregistrer</Button></DialogFooter>
          </form>
        </DialogContent>
      </Dialog>
```

```
      <AlertDialog open={!!del} onOpenChange={(o) => !o && setDel(null)}>
        <AlertDialogContent>
          <AlertDialogHeader><AlertDialogTitle>Supprimer ce prestataire ?</AlertDialogTitle><AlertDialogDescription>«
{del?.name} » sera supprimé définitivement.</AlertDialogDescription></AlertDialogHeader>
          <AlertDialogFooter><AlertDialogCancel>Annuler</AlertDialogCancel><AlertDialogAction className="bg-destructive
text-destructive-foreground hover:bg-destructive/90" onClick={(e) => { e.preventDefault(); remove(del);
}}>Supprimer</AlertDialogAction></AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Tasks.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, ListChecks, CheckSquare, CheckCircle2, Circle, Trash2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Tabs, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useI18n } from "@/lib/i18n";
import TaskForm from "@/components/tasks/TaskForm";
import TaskItem from "@/components/tasks/TaskItem";
import ExportMenu from "@/components/ExportMenu";
import BulkActionBar from "@/components/common/BulkActionBar";
import { expandRecurrence } from "@/lib/recurrence";

const TASK_COLUMNS = [
  { key: "title", label: "Titre" },
  { key: "status", label: "Statut" },
  { key: "priority", label: "Priorité" },
  { key: "start", label: "Début", type: "date" },
  { key: "end", label: "Fin", type: "date" },
  { key: "duration_minutes", label: "Durée (min)" },
];

const FILTERS = [
  { value: "toutes", key: "tasks.filter.all" },
  { value: "a_faire", key: "tasks.filter.todo" },
  { value: "en_cours", key: "tasks.filter.progress" },
  { value: "terminee", key: "tasks.filter.done" },
];

const PAGE_SIZE = 100;

export default function Tasks() {
  const { t } = useI18n();
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [filter, setFilter] = useState("toutes");
  const [formOpen, setFormOpen] = useState(false);
  const [editingTask, setEditingTask] = useState(null);
  const [visibleCount, setVisibleCount] = useState(PAGE_SIZE);
  const [selectMode, setSelectMode] = useState(false);
  const [selected, setSelected] = useState(new Set());
  const [bulkBusy, setBulkBusy] = useState(false);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => {
    const list = await base44.entities.Task.list("-start", 300);
    setTasks(list);
    setLoading(false);
  };

  useEffect(() => { register(load); });
  useEffect(() => { load(); }, []);

  const handleSave = async (data, recurrence) => {
    if (editingTask) {
      const prev = tasks;
      setTasks((cur) => cur.map((task) => (task.id === editingTask.id ? { ...task, ...data } : task)));
      try {
        await base44.entities.Task.update(editingTask.id, data);
        toast({ title: t("toast.taskUpdated") });
        load();
      } catch (err) {
        setTasks(prev);
        toast({ title: t("toast.error"), description: t("err.update"), variant: "destructive" });
      }
    } else {
      const items = expandRecurrence(data, recurrence?.frequency, recurrence?.count, ["start", "end"]);
      const tempIds = items.map(() => `tmp_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`);
      setTasks((cur) => [...items.map((it, i) => ({ ...it, id: tempIds[i] })), ...cur]);
      try {
        await base44.entities.Task.bulkCreate(items);
        toast({ title: recurrence && recurrence.count > 1 ? `${items.length} tâches récurrentes créées` : t("toast.taskCreated")
});
```

```
      load();
    } catch (err) {
      setTasks((cur) => cur.filter((item) => !tempIds.includes(item.id)));
      toast({ title: t("toast.error"), description: t("err.create"), variant: "destructive" });
    }
  }
  setFormOpen(false);
  setEditingTask(null);
};

const handleDelete = async (task) => {
  const prev = tasks;
  setTasks((cur) => cur.filter((item) => item.id !== task.id));
  try {
    await base44.entities.Task.delete(task.id);
    toast({ title: t("toast.taskDeleted") });
    load();
  } catch (err) {
    setTasks(prev);
    toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
  }
};

const handleToggleDone = async (task) => {
  const newStatus = task.status === "terminee" ? "a_faire" : "terminee";
  const prev = tasks;
  setTasks((cur) => cur.map((item) => (item.id === task.id ? { ...item, status: newStatus } : item)));
  try {
    await base44.entities.Task.update(task.id, { status: newStatus });
    load();
  } catch (err) {
    setTasks(prev);
    toast({ title: t("toast.error"), description: t("err.update"), variant: "destructive" });
  }
};

const toggleSelect = (id) => setSelected((s) => {
  const n = new Set(s);
  if (n.has(id)) n.delete(id); else n.add(id);
  return n;
});
const exitSelect = () => { setSelectMode(false); setSelected(new Set()); };

const bulkDelete = async () => {
  const ids = [...selected];
  if (!ids.length) return;
  setBulkBusy(true);
  const prev = tasks;
  setTasks((cur) => cur.filter((t) => !selected.has(t.id)));
  try {
    await base44.entities.Task.deleteMany({ id: { $in: ids } });
    toast({ title: `${ids.length} tâche(s) supprimée(s)` });
    setSelected(new Set()); setBulkBusy(false); load();
  } catch {
    setTasks(prev); setBulkBusy(false);
    toast({ title: t("toast.error"), description: t("err.delete"), variant: "destructive" });
  }
};

const bulkSetStatus = async (status) => {
  const ids = [...selected];
  if (!ids.length) return;
  setBulkBusy(true);
  const prev = tasks;
  setTasks((cur) => cur.map((t) => (selected.has(t.id) ? { ...t, status } : t)));
  try {
    await base44.entities.Task.updateMany({ id: { $in: ids } }, { $set: { status } });
    toast({ title: `${ids.length} tâche(s) mise(s) à jour` });
    setSelected(new Set()); setBulkBusy(false); load();
  } catch {
    setTasks(prev); setBulkBusy(false);
    toast({ title: t("toast.error"), description: t("err.update"), variant: "destructive" });
  }
};

const filtered = filter === "toutes" ? tasks : tasks.filter((task) => task.status === filter);
const sorted = [...filtered].sort((a, b) => new Date(a.start) - new Date(b.start));
const visible = sorted.slice(0, visibleCount);
const allSelected = visible.length > 0 && visible.every((t) => selected.has(t.id));
const toggleAll = () => setSelected(allSelected ? new Set() : new Set(visible.map((t) => t.id)));

if (loading) {
  return (
    <div className="flex items-center justify-center h-64">
      <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
    </div>
  );
}

return (
  <div className="space-y-6">
    <div className="flex items-center justify-between flex-wrap gap-3">
```

```
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">{t("tasks.title")}</h1>
          <p className="text-muted-foreground mt-1">{t("tasks.subtitle")}</p>
        </div>
        <div className="flex items-center gap-2">
          <ExportMenu title="Tâches" subtitle="Liste des tâches" columns={TASK_COLUMNS} rows={sorted} currency="XAF" />
          <Button variant={selectMode ? "secondary" : "outline"} onClick={() => (selectMode ? exitSelect() :
setSelectMode(true))}>
            <CheckSquare className="w-4 h-4 mr-2" /> {selectMode ? "Terminer" : "Sélectionner"}
          </Button>
          <Button onClick={() => { setEditingTask(null); setFormOpen(true); }}>
            <Plus className="w-4 h-4 mr-2" /> {t("tasks.new")}
          </Button>
        </div>
      </div>

      <Tabs value={filter} onValueChange={setFilter}>
        <TabsList>
          {FILTERS.map((f) => (
            <TabsTrigger key={f.value} value={f.value}>{t(f.key)}</TabsTrigger>
          ))}
        </TabsList>
      </Tabs>

      {sorted.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground">
          <ListChecks className="w-8 h-8 mx-auto mb-3 opacity-50" />
          <p className="text-sm">{t("tasks.empty")}</p>
        </div>
      ) : (
        <div className="space-y-2">
          {selectMode && (
            <BulkActionBar
              count={selected.size}
              total={visible.length}
              allSelected={allSelected}
              onToggleAll={toggleAll}
              onClear={exitSelect}
              busy={bulkBusy}
            >
              <Button size="sm" variant="outline" onClick={() => bulkSetStatus("terminee")} disabled={!selected.size ||
bulkBusy}>
                <CheckCircle2 className="w-4 h-4 mr-1" /> Terminer
              </Button>
              <Button size="sm" variant="outline" onClick={() => bulkSetStatus("a_faire")} disabled={!selected.size ||
bulkBusy}>
                <Circle className="w-4 h-4 mr-1" /> À faire
              </Button>
              <Button size="sm" variant="destructive" onClick={bulkDelete} disabled={!selected.size || bulkBusy}>
                <Trash2 className="w-4 h-4 mr-1" /> Supprimer
              </Button>
            </BulkActionBar>
          )}
          {visible.map((task) => (
            <TaskItem
              key={task.id}
              task={task}
              selectable={selectMode}
              selected={selected.has(task.id)}
              onToggleSelect={toggleSelect}
              onEdit={(item) => { if (selectMode) return; setEditingTask(item); setFormOpen(true); }}
              onDelete={handleDelete}
              onToggleDone={selectMode ? () => toggleSelect(task.id) : handleToggleDone}
            />
          ))}
          {visibleCount < sorted.length && (
            <Button variant="outline" className="w-full" onClick={() => setVisibleCount((c) => c + PAGE_SIZE)}>
              {t("tasks.loadMore", { n: sorted.length - visibleCount })}
            </Button>
          )}
        </div>
      )}

      <TaskForm open={formOpen} onOpenChange={setFormOpen} task={editingTask} onSubmit={handleSave} />
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Taxes.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Plus, Percent, Pencil, Trash2, RotateCw } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFooter,
AlertDialogHeader, AlertDialogTitle } from "@/components/ui/alert-dialog";
```

```
import { useToast } from "@/components/ui/use-toast";
import { useRefresh } from "@/lib/RefreshContext";
import { useSpace } from "@/lib/SpaceContext";
import { formatCurrency, formatDateFr } from "@/lib/format";
import { expandRecurrence } from "@/lib/recurrence";
import PeriodPicker from "@/components/common/PeriodPicker";
import { periodLabel, customOf } from "@/lib/periods";

const TYPE = { tva: "TVA", charge: "Charge", impot: "Impôt", autre: "Autre" };
const MODE = { montant: "Montant fixe", pourcentage: "Pourcentage" };
const empty = { name: "", value_mode: "montant", rate: "0", type: "charge", base_amount: "0", amount: "0", recurring: false,
frequency: "mois", customCount: "", customUnit: "jour", start_date: new Date().toISOString().slice(0, 10), count: "1", period:
"", description: "" };

export default function Taxes({ scope = "jpm" }) {
  const { activeCompanyId } = useSpace();
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState(null);
  const [form, setForm] = useState(empty);
  const [del, setDel] = useState(null);
  const { toast } = useToast();
  const { register } = useRefresh();

  const load = async () => { const list = await base44.entities.Tax.list("-created_date", 500); setItems(list || []);
setLoading(false); };
  useEffect(() => { register(load); }, []);
  useEffect(() => { load(); }, []);

  const mine = scope === "jbm"
    ? items.filter((t) => (t.scope === "jbm" || !t.scope) && t.company_id === activeCompanyId)
    : items.filter((t) => t.scope === "jpm");
  const total = mine.reduce((s, t) => s + (Number(t.amount) || 0), 0);
  const isJbm = scope === "jbm";

  const openNew = () => { setEditing(null); setForm(empty); setOpen(true); };
  const openEdit = (t) => {
    setEditing(t);
    setForm({
      name: t.name || "", value_mode: t.value_mode || "montant", rate: String(t.rate ?? 0), type: t.type || "charge",
      base_amount: String(t.base_amount ?? 0), amount: String(t.amount ?? 0),
      recurring: !!t.recurring, frequency: t.frequency || "mois", customCount: t.custom_count ? String(t.custom_count) : "",
customUnit: t.custom_unit || "jour", start_date: t.start_date || new Date().toISOString().slice(0, 10),
      count: "1", period: t.period || "", description: t.description || "",
    });
    setOpen(true);
  };

  // Montant calculé automatiquement en mode pourcentage.
  const computedAmount = form.value_mode === "pourcentage"
    ? ((parseFloat(form.base_amount) || 0) * (parseFloat(form.rate) || 0)) / 100
    : parseFloat(form.amount) || 0;

  const save = async (e) => {
    e.preventDefault();
    if (!form.name.trim()) return;
    const amount = Math.round(computedAmount);
    const base = {
      name: form.name.trim(),
      scope,
      value_mode: form.value_mode,
      type: form.type,
      rate: parseFloat(form.rate) || 0,
      base_amount: parseFloat(form.base_amount) || 0,
      amount,
      recurring: form.recurring,
      frequency: form.recurring ? form.frequency : null,
      custom_count: form.recurring && form.frequency === "personnalise" ? (parseInt(form.customCount, 10) || 0) : null,
      custom_unit: form.recurring && form.frequency === "personnalise" ? form.customUnit : null,
      start_date: form.start_date,
      period: form.period.trim(),
      description: form.description.trim(),
      company_id: isJbm ? activeCompanyId : null,
    };
    try {
      if (editing) {
        await base44.entities.Tax.update(editing.id, base);
        toast({ title: "Taxe mise à jour" });
      } else {
        const count = form.recurring && parseInt(form.count, 10) > 1 ? parseInt(form.count, 10) : 1;
        const custom = form.frequency === "personnalise" ? { count: parseInt(form.customCount, 10) || 0, unit: form.customUnit }
: null;
        const records = count > 1 ? expandRecurrence({ ...base }, form.frequency, count, ["start_date"], custom) : [base];
        await base44.entities.Tax.bulkCreate(records);
        toast({ title: count > 1 ? `${count} échéances ajoutées` : "Taxe ajoutée" });
      }
      setOpen(false); load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  const remove = async (t) => { try { await base44.entities.Tax.delete(t.id); toast({ title: "Taxe supprimée" }); load(); }
```

```
catch { toast({ title: "Erreur", variant: "destructive" }); } setDel(null); };

  if (loading) return <div className="flex items-center justify-center h-64"><div className="w-6 h-6 border-2 border-border
border-t-primary rounded-full animate-spin" /></div>;

  return (
    <div className="space-y-6">
      <div className="flex items-start justify-between gap-3 flex-wrap">
        <div>
          <h1 className="font-display text-2xl sm:text-3xl">Taxes & charges</h1>
          <p className="text-muted-foreground mt-1">{isJbm ? "Taxes et charges de l'entreprise." : "Vos taxes et charges
personnelles, ponctuelles ou récurrentes."}</p>
        </div>
        <Button onClick={openNew}><Plus className="w-4 h-4 mr-2" /> Ajouter</Button>
      </div>

      <div className="bg-card border border-border rounded-2xl p-5">
        <p className="text-sm text-muted-foreground">Total des taxes/charges</p>
        <p className="font-display text-xl mt-1">{formatCurrency(total, "XAF")}</p>
      </div>

      {mine.length === 0 ? (
        <div className="text-center py-12 text-muted-foreground"><Percent className="w-8 h-8 mx-auto mb-3 opacity-50" /><p
className="text-sm">Aucune taxe enregistrée.</p></div>
      ) : (
        <div className="space-y-2">
          {mine.map((t) => (
            <div key={t.id} className="bg-card border border-border rounded-xl p-4 flex items-center justify-between gap-3">
              <div className="min-w-0">
                <p className="text-sm font-medium truncate">{t.name}</p>
                <p className="text-xs text-muted-foreground">
                  {TYPE[t.type] || t.type}
                  {t.value_mode === "pourcentage" ? ` · ${t.rate}% de ${formatCurrency(t.base_amount, "XAF")}` : ""}
                  {t.recurring && t.frequency ? ` · ${periodLabel(t.frequency, customOf(t))} · dès
${formatDateFr(t.start_date)}` : (t.period ? ` · ${t.period}` : "")}
                </p>
              </div>
              <div className="flex items-center gap-2 shrink-0">
                <span className="text-sm font-medium">{formatCurrency(t.amount, "XAF")}</span>
                <button onClick={() => openEdit(t)} className="no-select-tap min-touch p-2 rounded-lg
hover:bg-secondary"><Pencil className="w-4 h-4" /></button>
                <button onClick={() => setDel(t)} className="no-select-tap min-touch p-2 rounded-lg hover:bg-destructive/10
text-destructive"><Trash2 className="w-4 h-4" /></button>
              </div>
            </div>
          ))}
        </div>
      )}

      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent className="sm:max-w-md">
          <DialogHeader><DialogTitle className="font-display">{editing ? "Modifier la taxe" : "Nouvelle taxe /
charge"}</DialogTitle></DialogHeader>
          <form onSubmit={save} className="space-y-3">
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div className="space-y-2"><Label>Nom</Label><Input value={form.name} onChange={(e) => setForm({ ...form, name:
e.target.value })} required /></div>
              <div className="space-y-2"><Label>Type</Label>
                <Select value={form.type} onValueChange={(v) => setForm({ ...form, type: v })}>
                  <SelectTrigger><SelectValue /></SelectTrigger>
                  <SelectContent>{Object.entries(TYPE).map(([v, l]) => <SelectItem key={v}
value={v}>{l}</SelectItem>)}</SelectContent>
                </Select>
              </div>
            </div>

            <div className="space-y-2"><Label>Mode de calcul</Label>
              <Select value={form.value_mode} onValueChange={(v) => setForm({ ...form, value_mode: v })}>
                <SelectTrigger><SelectValue /></SelectTrigger>
                <SelectContent>{Object.entries(MODE).map(([v, l]) => <SelectItem key={v}
value={v}>{l}</SelectItem>)}</SelectContent>
              </Select>
            </div>

            {form.value_mode === "pourcentage" ? (
              <div className="grid grid-cols-2 gap-3">
                <div className="space-y-2"><Label>Taux (%)</Label><Input type="number" min="0" step="0.01" value={form.rate}
onChange={(e) => setForm({ ...form, rate: e.target.value })} /></div>
                <div className="space-y-2"><Label>Base (assiette)</Label><Input type="number" min="0" value={form.base_amount}
onChange={(e) => setForm({ ...form, base_amount: e.target.value })} /></div>
                <div className="col-span-2 rounded-lg bg-secondary p-2 text-sm">Montant calculé : <span
className="font-medium">{formatCurrency(computedAmount, "XAF")}</span></div>
              </div>
            ) : (
              <div className="space-y-2"><Label>Montant (valeur précise)</Label><Input type="number" min="0" value={form.amount}
onChange={(e) => setForm({ ...form, amount: e.target.value })} /></div>
            )}

            {!editing && (
              <div className="rounded-xl border border-border p-3 space-y-3">
                <label className="flex items-center gap-2 text-sm">
                  <input type="checkbox" checked={form.recurring} onChange={(e) => setForm({ ...form, recurring:
```

```
e.target.checked })} className="w-4 h-4" />
                  <RotateCw className="w-4 h-4" /> Taxe récurrente
                </label>
                {form.recurring && (
                  <div className="space-y-3">
                    <PeriodPicker
                      period={form.frequency}
                      customCount={form.customCount}
                      customUnit={form.customUnit}
                      onChange={({ period, customCount, customUnit }) => setForm({ ...form, frequency: period, customCount,
customUnit })}
                      label="Périodicité"
                    />
                    <div className="grid grid-cols-2 gap-3">
                                        < d i v   c l a s s N a m e = " s p a c e - y - 2 " > < L a b e l > 1 ³ I   é c h é a n c
onChange={(e) => setForm({ ...form, start_date: e.target.value })} /></div>
                      <div className="space-y-2"><Label>Nb d'échéances</Label><Input type="number" min="1" value={form.count}
onChange={(e) => setForm({ ...form, count: e.target.value })} /></div>
                    </div>
                  </div>
                )}
              </div>
            )}

            <div className="space-y-2"><Label>Période (libre)</Label><Input value={form.period} onChange={(e) => setForm({
...form, period: e.target.value })} placeholder="2026 / T1 2026" /></div>
            <div className="space-y-2"><Label>Description</Label><Textarea value={form.description} onChange={(e) => setForm({
...form, description: e.target.value })} rows={2} /></div>
            <DialogFooter><Button type="submit" className="w-full">Enregistrer</Button></DialogFooter>
          </form>
        </DialogContent>
      </Dialog>

      <AlertDialog open={!!del} onOpenChange={(o) => !o && setDel(null)}>
        <AlertDialogContent>
          <AlertDialogHeader><AlertDialogTitle>Supprimer cette taxe ?</AlertDialogTitle><AlertDialogDescription>« {del?.name} »
sera supprimé définitivement.</AlertDialogDescription></AlertDialogHeader>
          <AlertDialogFooter><AlertDialogCancel>Annuler</AlertDialogCancel><AlertDialogAction className="bg-destructive
text-destructive-foreground hover:bg-destructive/90" onClick={(e) => { e.preventDefault(); remove(del);
}}>Supprimer</AlertDialogAction></AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Terms.jsx
------------------------------------------------------------
import React from "react";
import { Link } from "react-router-dom";
import { useI18n } from "@/lib/i18n";

export default function Terms() {
  const { t } = useI18n();
  return (
    <div className="min-h-screen bg-background font-body px-5 py-10 max-w-2xl mx-auto">
      <h1 className="font-display text-3xl mb-4">{t("legal.termsTitle")}</h1>
      <div className="prose prose-sm text-muted-foreground space-y-3">
        <p>{t("legal.termsIntro")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.termsUseTitle")}</h2>
        <p>{t("legal.termsUse")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.termsAccountsTitle")}</h2>
        <p>{t("legal.termsAccounts")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.termsLiabilityTitle")}</h2>
        <p>{t("legal.termsLiability")}</p>
        <h2 className="font-display text-base text-foreground">{t("legal.termsContactTitle")}</h2>
        <p>{t("legal.termsContact")}</p>
      </div>
      <Link to="/login" className="text-sm text-primary hover:underline mt-6 inline-block">{t("legal.back")}</Link>
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/pages/Thresholds.jsx
------------------------------------------------------------
import React, { useEffect, useState } from "react";
import { base44 } from "@/api/base44Client";
import { Gauge, Save, RefreshCw, Loader2 } from "lucide-react";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Button } from "@/components/ui/button";
import { useToast } from "@/components/ui/use-toast";
import { getLevel, LEVEL_STYLE, LEVEL_LABEL } from "@/lib/thresholds";
import { formatCurrency } from "@/lib/format";

const DOMAINS = [
  { value: "budget", label: "Budget restant (XAF)", direction: "low", measure: (d) => d.budget, money: true },
  { value: "stock", label: "Valeur du stock (XAF)", direction: "low", measure: (d) => d.stock, money: true },
  { value: "invoices", label: "Factures impayées (XAF)", direction: "high", measure: (d) => d.invoices, money: true },
  { value: "tasks", label: "Tâches en retard (nombre)", direction: "high", measure: (d) => d.tasks, money: false },
```

```
];

export default function Thresholds() {
  const { toast } = useToast();
  const [th, setTh] = useState([]);
  const [data, setData] = useState({ budget: 0, stock: 0, invoices: 0, tasks: 0 });
  const [form, setForm] = useState({});
  const [loading, setLoading] = useState(true);
  const [checking, setChecking] = useState(false);

  const load = async () => {
    try {
      const [list, prefs, exp, prod, inv, tsk] = await Promise.all([
        base44.entities.Threshold.list("-created_date", 50),
        base44.entities.Preferences.list(),
        base44.entities.Expense.list("-date", 500),
        base44.entities.Product.list("-created_date", 500),
        base44.entities.Invoice.list("-issue_date", 500),
        base44.entities.Task.list("-start", 500),
      ]);
      const map = {};
      (list || []).forEach((x) => { map[x.domain] = x; });
      setTh(Object.values(map));
      const budget = (prefs?.[0]?.initial_budget || 0) - (exp || []).reduce((s, e) => s + (Number(e.quantity) || 0) *
(Number(e.unit_cost) || 0), 0);
      const stock = (prod || []).reduce((s, p) => s + (Number(p.quantity) || 0) * (Number(p.unit_cost) || 0), 0);
      const invoices = (inv || []).filter((i) => i.status !== "payee").reduce((s, i) => s + (Number(i.amount) || 0), 0);
      const now = new Date();
      const tasks = (tsk || []).filter((x) => x.status !== "terminee" && new Date(x.end) < now).length;
      setData({ budget, stock, invoices, tasks });
      const init = {};
      DOMAINS.forEach((d) => {
        const r = map[d.value];
        init[d.value] = { medium: r?.medium_value ?? "", critical: r?.critical_value ?? "", id: r?.id || null };
      });
      setForm(init);
    } catch { /* ignore */ }
    setLoading(false);
  };
  useEffect(() => { load(); }, []);

  const saveDomain = async (d) => {
    const f = form[d.value];
    if (f.medium === "" || f.critical === "") { toast({ title: "Renseignez les deux seuils", variant: "destructive" }); return;
}
    const payload = { domain: d.value, medium_value: Number(f.medium) || 0, critical_value: Number(f.critical) || 0 };
    try {
      if (f.id) await base44.entities.Threshold.update(f.id, payload);
      else { const c = await base44.entities.Threshold.create(payload); setForm((cur) => ({ ...cur, [d.value]: {
...cur[d.value], id: c.id } })); }
      toast({ title: "Seuil enregistré" });
      load();
    } catch { toast({ title: "Erreur", variant: "destructive" }); }
  };

  if (loading) {
    return (
      <div className="flex items-center justify-center h-64">
        <div className="w-6 h-6 border-2 border-border border-t-primary rounded-full animate-spin" />
      </div>
    );
  }

  return (
    <div className="space-y-6 max-w-2xl">
      <div>
        <h1 className="font-display text-3xl flex items-center gap-2"><Gauge className="w-7 h-7" /> Seuils & alertes</h1>
        <p className="text-muted-foreground mt-1">Définissez des seuils moyen et critique par domaine. L'état actuel est calculé
automatiquement.</p>
      </div>

      <div className="bg-card border border-border rounded-2xl p-5 flex items-center justify-between gap-3 flex-wrap">
        <div>
          <h2 className="font-display text-lg">Alertes automatisées</h2>
          <p className="text-sm text-muted-foreground">Vérifie les seuils et crée des notifications (rappels) quand un seuil
moyen ou critique est atteint. La vérification se lance aussi automatiquement une fois par jour à l'ouverture du tableau de
bord.</p>
        </div>
        <Button onClick={async () => {
          setChecking(true);
          try {
      const res = await base44.functions.invoke('checkAlerts', {});
            toast({ title: res?.count ? `${res.count} alerte(s) générée(s)` : "Aucune alerte à générer" });
          } catch { toast({ title: "Erreur", variant: "destructive" }); }
          setChecking(false);
        }}>
          {checking ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <RefreshCw className="w-4 h-4 mr-2" />} Vérifier
maintenant
        </Button>
      </div>
      {DOMAINS.map((d) => {
        const f = form[d.value] || { medium: "", critical: "" };
```

```
        const cur = d.measure(data);
        const level = getLevel(d.value, cur, th, d.direction);
        const disp = d.money ? formatCurrency(cur, "XAF") : cur;
        return (
          <div key={d.value} className="bg-card border border-border rounded-2xl p-5 space-y-3">
            <div className="flex items-center justify-between gap-2">
              <h2 className="font-display text-lg">{d.label}</h2>
              <span className={`text-xs px-2 py-0.5 rounded-full ${LEVEL_STYLE[level]}`}>{LEVEL_LABEL[level]}</span>
            </div>
            <p className="text-sm text-muted-foreground">Valeur actuelle : <span className="font-medium
text-foreground">{disp}</span></p>
            <div className="grid grid-cols-2 gap-3">
              <div className="space-y-1"><Label>Seuil moyen</Label><Input type="number" value={f.medium} onChange={(e) =>
setForm((c) => ({ ...c, [d.value]: { ...c[d.value], medium: e.target.value } }))} /></div>
              <div className="space-y-1"><Label>Seuil critique</Label><Input type="number" value={f.critical} onChange={(e) =>
setForm((c) => ({ ...c, [d.value]: { ...c[d.value], critical: e.target.value } }))} /></div>
            </div>
            <Button size="sm" onClick={() => saveDomain(d)}><Save className="w-4 h-4 mr-1" /> Enregistrer</Button>
          </div>
        );
      })}
    </div>
  );
}

------------------------------------------------------------
FICHIER : ./src/utils/index.ts
------------------------------------------------------------
export function createPageUrl(pageName: string) {
    return '/' + pageName.replace(/ /g, '-');
}

------------------------------------------------------------
FICHIER : ./tailwind.config.js
------------------------------------------------------------
/** @type {import('tailwindcss').Config} */
module.exports = {
    darkMode: ["class"],
    content: ["./index.html", "./src/**/*.{ts,tsx,js,jsx}"],
  theme: {
          extend: {
                  opacity: Object.fromEntries(Array.from({ length: 101 }, (_, i) => [i, `${i / 100}`])),
                  borderRadius: {
                          lg: 'var(--radius)',
                          md: 'calc(var(--radius) - 2px)',
                          sm: 'calc(var(--radius) - 4px)'
                  },
                  colors: {
                          background: 'hsl(var(--background))',
                          foreground: 'hsl(var(--foreground))',
                          card: {
                                  DEFAULT: 'hsl(var(--card))',
                                  foreground: 'hsl(var(--card-foreground))'
                          },
                          popover: {
                                  DEFAULT: 'hsl(var(--popover))',
                                  foreground: 'hsl(var(--popover-foreground))'
                          },
                          primary: {
                                  DEFAULT: 'hsl(var(--primary))',
                                  foreground: 'hsl(var(--primary-foreground))'
                          },
                          secondary: {
                                  DEFAULT: 'hsl(var(--secondary))',
                                  foreground: 'hsl(var(--secondary-foreground))'
                          },
                          muted: {
                                  DEFAULT: 'hsl(var(--muted))',
                                  foreground: 'hsl(var(--muted-foreground))'
                          },
                          accent: {
                                  DEFAULT: 'hsl(var(--accent))',
                                  foreground: 'hsl(var(--accent-foreground))'
                          },
                          destructive: {
                                  DEFAULT: 'hsl(var(--destructive))',
                                  foreground: 'hsl(var(--destructive-foreground))'
                          },
                          border: 'hsl(var(--border))',
                          input: 'hsl(var(--input))',
                          ring: 'hsl(var(--ring))',
                          chart: {
                                  '1': 'hsl(var(--chart-1))',
                                  '2': 'hsl(var(--chart-2))',
                                  '3': 'hsl(var(--chart-3))',
                                  '4': 'hsl(var(--chart-4))',
                                  '5': 'hsl(var(--chart-5))'
                          },
                          sidebar: {
                                  DEFAULT: 'hsl(var(--sidebar-background))',
                                  foreground: 'hsl(var(--sidebar-foreground))',
                                  primary: 'hsl(var(--sidebar-primary))',
```

```
                                'primary-foreground': 'hsl(var(--sidebar-primary-foreground))',
                                accent: 'hsl(var(--sidebar-accent))',
                                'accent-foreground': 'hsl(var(--sidebar-accent-foreground))',
                                border: 'hsl(var(--sidebar-border))',
                                ring: 'hsl(var(--sidebar-ring))'
                        }
                },
                fontFamily: {
                        heading: ['var(--font-heading)'],
                        body: ['var(--font-body)'],
                        display: ['var(--font-display)'],
                        mono: ['var(--font-mono)']
                },
                keyframes: {
                        'accordion-down': {
                                from: {
                                        height: '0'
                                },
                                to: {
                                        height: 'var(--radix-accordion-content-height)'
                                }
                        },
                        'accordion-up': {
                                from: {
                                        height: 'var(--radix-accordion-content-height)'
                                },
                                to: {
                                        height: '0'
                                }
                        }
                },
                animation: {
                        'accordion-down': 'accordion-down 0.2s ease-out',
                        'accordion-up': 'accordion-up 0.2s ease-out'
                }
        }
  },
  plugins: [require("tailwindcss-animate")],
}


------------------------------------------------------------
FICHIER : ./vite.config.js
------------------------------------------------------------
import base44 from "@base44/vite-plugin"
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    base44({
      // Support for legacy code that imports the base44 SDK with @/integrations, @/entities, etc.
      // can be removed if the code has been updated to use the new SDK imports from @base44/sdk
      legacySDKImports: process.env.BASE44_LEGACY_SDK_IMPORTS === 'true',
      hmrNotifier: true,
      navigationNotifier: true,
      analyticsTracker: true,
      visualEditAgent: true
    }),
    react(),
  ]
});
```